IDA Pro. Руководство по дизассемблированию

Русский перевод книги об интерактивном дизассемблере IDA Pro: от основ обратной разработки и навигации по дизассемблированному коду до расширений, загрузочных и процессорных модулей, анализа уязвимостей и встроенного отладчика.

Больше крутых материалов в моём телеграм канале @grogrigs

Глава 1. Введение в дизассемблирование

Вы, возможно, задаётесь вопросом, чего ожидать от книги, целиком посвящённой IDA Pro. Хотя в центре внимания находится IDA, эта книга не задумана как руководство пользователя IDA Pro. Вместо этого мы будем использовать IDA как основной инструмент для обсуждения методов обратной разработки, полезных при анализе самого разного программного обеспечения - от уязвимых приложений до вредоносных программ.

Там, где это уместно, мы будем приводить подробные последовательности действий в IDA для решения конкретной задачи. Поэтому знакомство с возможностями IDA пойдёт несколько кружным путём: от основных действий при первом изучении файла до расширенного применения и настройки IDA для более сложных задач обратной разработки. Мы не пытаемся охватить все возможности IDA, но подробно рассмотрим те из них, которые чаще всего помогают справляться с практическими трудностями. Эта книга поможет сделать IDA самым мощным средством в вашем наборе инструментов.

Прежде чем переходить к особенностям IDA, полезно разобрать основы дизассемблирования и познакомиться с некоторыми другими средствами обратной разработки скомпилированного кода. Ни одно из них не обладает полным набором возможностей IDA, однако каждое охватывает отдельную часть её функций и помогает лучше понять конкретные особенности программы. Остальная часть этой главы посвящена процессу дизассемблирования.

Теория дизассемблирования

Всякий, кто изучал языки программирования, вероятно, встречался с их делением на поколения. Для тех, кто тогда задремал, кратко напомним эту классификацию.

Языки первого поколения

Это самый низкий уровень представления программы. Такие языки обычно состоят из единиц и нулей либо используют сокращённую запись, например шестнадцатеричную, и понятны главным образом специалистам, привыкшим работать с двоичными данными. На этом уровне легко запутаться, поскольку данные часто трудно отличить от инструкций: внешне они выглядят почти одинаково. Языки первого поколения также называют машинными языками, а иногда байт-кодом; программы на машинном языке часто называют двоичными файлами.

Языки второго поколения

Языки второго поколения, также называемые языками ассемблера, отделены от машинного языка фактически одной таблицей соответствий. Определённым битовым шаблонам, или кодам операций, сопоставляются короткие запоминающиеся последовательности символов - мнемоники. Иногда эти мнемоники и в самом деле помогают программистам помнить назначение соответствующих инструкций. Ассемблером называют программу, переводящую исходный текст на языке ассемблера в машинный код, пригодный для исполнения.

Языки третьего поколения

Эти языки делают ещё один шаг к выразительности естественного языка: в них появляются ключевые слова и конструкции, из которых программисты строят программы. Обычно языки третьего поколения не зависят от платформы, хотя написанные на них программы могут стать платформенно-зависимыми из-за использования возможностей конкретной операционной системы. К часто приводимым примерам относятся Фортран, Кобол, Си и Java. Для перевода программ на язык ассемблера, непосредственно в машинный код или в близкую форму наподобие байт-кода обычно применяют компиляторы.

Языки четвёртого поколения

Такие языки существуют, но к теме этой книги отношения не имеют, поэтому рассматривать их мы не будем.

Что такое дизассемблирование

В традиционной модели разработки программ компиляторы, ассемблеры и компоновщики по отдельности или совместно создают исполняемые программы. Чтобы пройти этот путь в обратном направлении, то есть выполнить обратную разработку программы, применяют средства, обращающие результаты ассемблирования и компиляции. Они вполне закономерно называются дизассемблерами и декомпиляторами и делают именно то, что следует из их названий.

Дизассемблер обращает процесс ассемблирования, поэтому на вход он получает машинный код, а на выходе создаёт текст на языке ассемблера. Декомпилятор стремится получить представление на языке высокого уровня, принимая на вход ассемблерный или даже машинный код.

Обещание «восстановить исходный код» неизменно привлекательно на конкурентном рынке программного обеспечения, поэтому создание пригодных к практическому применению декомпиляторов остаётся активной областью исследований в информатике. Ниже перечислены лишь некоторые причины сложности декомпиляции.

При компиляции часть информации теряется

На уровне машинного языка нет имён переменных и функций, а тип переменной приходится определять по способу использования данных, а не по явному объявлению. Увидев перенос 32 бит данных, аналитик должен выяснить, представляют ли эти биты целое число, 32-разрядное число с плавающей точкой или 32-разрядный указатель.

Компиляция имеет отношение «многие ко многим»

Одну исходную программу можно перевести на язык ассемблера множеством способов, а один машинный код можно множеством способов представить в виде исходного текста. Поэтому файл, который сначала скомпилировали, а затем сразу декомпилировали, нередко сильно отличается от исходного файла, поданного на вход компилятору.

Декомпиляторы сильно зависят от языка и библиотек

Обработка двоичного файла, созданного компилятором Delphi, декомпилятором, рассчитанным на получение кода на Си, может дать весьма странный результат. Точно так же обработка программы для Windows декомпилятором, ничего не знающим о программном интерфейсе Windows, вряд ли принесёт пользу.

Для точной декомпиляции требуется почти безошибочное дизассемблирование

Любые ошибки или пропуски на этапе дизассемблирования почти неизбежно перейдут в декомпилированный код.

Самый развитый из представленных на рынке декомпиляторов, Hex-Rays, рассматривается в главе 23.

Зачем нужно дизассемблирование

Средства дизассемблирования обычно применяют, чтобы понять работу программы при отсутствии исходного кода. Чаще всего дизассемблирование требуется в следующих случаях:

• анализ вредоносных программ;
• поиск уязвимостей в программах с закрытым исходным кодом;
• изучение программ с закрытым исходным кодом ради обеспечения совместимости;
• анализ созданного компилятором кода для проверки производительности и правильности работы компилятора;
• отображение инструкций программы во время отладки.

Далее каждый случай рассматривается подробнее.

Анализ вредоносных программ

Если речь не идёт о черве, написанном на сценарном языке, автор вредоносной программы вряд ли окажет вам любезность и предоставит её исходный код. Без исходников остаётся очень мало способов точно выяснить поведение программы.

Два основных подхода к анализу вредоносных программ - динамический и статический. При динамическом анализе программе позволяют выполняться в тщательно контролируемой среде, или песочнице, и с помощью различных средств наблюдения за системой записывают все заметные проявления её поведения. Статический анализ, напротив, пытается понять поведение программы без запуска, посредством изучения её кода. В случае вредоносной программы таким кодом обычно служит листинг дизассемблирования.

Анализ уязвимостей

Для простоты разделим весь процесс проверки безопасности на три этапа: обнаружение уязвимости, анализ уязвимости и разработку эксплойта. Эти этапы одинаковы независимо от наличия исходного кода, однако при работе только с двоичным файлом трудозатраты существенно возрастают.

Сначала в программе обнаруживают условие, которое потенциально можно использовать для атаки. Часто для этого применяют динамические методы, например фаззинг[1], хотя задачу можно решить и статическим анализом - обычно ценой гораздо больших усилий. После обнаружения проблемы требуется дальнейший анализ, чтобы понять, можно ли её вообще использовать и, если да, то при каких условиях.

Листинг дизассемблирования даёт сведения, необходимые для точного понимания того, как компилятор разместил переменные программы. Например, полезно знать, что объявленный программистом массив из 70 символов при размещении был округлён компилятором до 80 байт. Только листинг дизассемблирования позволяет точно установить, в каком порядке компилятор расположил переменные, объявленные глобально или внутри функций. Понимание взаимного расположения переменных в памяти часто необходимо при создании эксплойтов. В конечном счёте совместное применение дизассемблера и отладчика позволяет разработать эксплойт.

Совместимость программ

Если программа распространяется только в двоичном виде, конкурентам очень трудно создать совместимое с ней программное обеспечение или полноценную замену. Распространённый пример - драйвер оборудования, выпущенный только для одной платформы. Если изготовитель медленно добавляет поддержку других платформ или, что ещё хуже, отказывается её добавлять, для создания подходящего драйвера может потребоваться значительная работа по обратной разработке. В подобных случаях статический анализ кода остаётся почти единственным выходом, причём нередко приходится изучать не только драйвер, но и встроенное программное обеспечение устройства.

Проверка компилятора

Поскольку назначение компилятора или ассемблера состоит в создании машинного кода, для проверки соответствия результата проектным требованиям часто нужны качественные средства дизассемблирования. Аналитики также могут искать дополнительные возможности оптимизации выдаваемого компилятором кода, а с точки зрения безопасности - выяснять, не был ли сам компилятор скомпрометирован настолько, что начал внедрять потайные входы в создаваемые программы.

Представление кода при отладке

Вероятно, чаще всего дизассемблеры используют для создания листингов внутри отладчиков. К сожалению, встроенные в отладчики дизассемблеры обычно довольно примитивны. Как правило, они не способны обрабатывать весь файл сразу, а иногда отказываются дизассемблировать код, если не могут определить границы функции. Поэтому отладчик лучше применять вместе с качественным дизассемблером: это даёт больше контекста и позволяет лучше понимать происходящее во время отладки.

Как работает дизассемблирование

Теперь, когда назначение дизассемблирования вам хорошо известно, пора разобраться в самом процессе. Представьте типичную непростую задачу дизассемблера: «Возьми эти 100 Кбайт, отличи код от данных, переведи код на язык ассемблера для показа пользователю и, пожалуйста, ничего не пропусти».

К этому можно добавить сколько угодно особых пожеланий: найти функции, распознать таблицы переходов, определить локальные переменные - и тем самым ещё сильнее усложнить работу дизассемблера.

Чтобы выполнить все требования, дизассемблеру приходится выбирать различные алгоритмы при обработке переданных ему файлов. Качество полученного листинга непосредственно зависит от качества этих алгоритмов и правильности их реализации. В этом разделе мы рассмотрим два основных алгоритма, применяемых сегодня для дизассемблирования машинного кода. Одновременно мы отметим их недостатки, чтобы подготовить вас к ситуациям, в которых дизассемблер даёт сбой. Понимание ограничений программы позволит вмешаться вручную и повысить общее качество результата.

Простейший алгоритм дизассемблирования

Для начала разработаем простой алгоритм, который принимает машинный код и выдаёт текст на языке ассемблера. Это поможет понять трудности, допущения и компромиссы, лежащие в основе автоматического дизассемблирования.

Шаг 1

Первым делом нужно определить область кода, которую предстоит дизассемблировать. Это не обязательно так просто, как кажется. Инструкции обычно перемешаны с данными, и важно отличать одно от другого.

Чаще всего дизассемблируют исполняемый файл, соответствующий распространённому формату: например, формату переносимых исполняемых файлов PE, применяемому в Windows, или формату исполняемых и компонуемых файлов ELF, распространённому во многих системах семейства Unix. Обычно в таких форматах предусмотрены структуры - нередко иерархические заголовки файла, - позволяющие найти разделы с кодом и точки входа[2] в этот код.

Шаг 2

Получив начальный адрес инструкции, следует прочитать находящееся по этому адресу или смещению файла значение и найти в таблице соответствующую двоичному коду операции мнемонику языка ассемблера. В зависимости от сложности набора инструкций это может быть как простая операция, так и последовательность дополнительных действий: распознавание префиксов, изменяющих поведение инструкции, и определение требуемых операндов. В наборах с инструкциями переменной длины, таких как Intel x86, для полного разбора одной инструкции может потребоваться прочитать дополнительные байты.

Шаг 3

После чтения инструкции и разбора всех необходимых операндов её представление на языке ассемблера форматируют и добавляют в листинг. Иногда можно выбрать один из нескольких вариантов синтаксиса. Например, для ассемблера x86 преобладают синтаксисы Intel и AT&T.

> **Синтаксис ассемблера x86: AT&T и Intel.** Существуют два основных варианта записи исходного текста на языке ассемблера: AT&T и Intel. Хотя оба относятся к языкам второго поколения, они заметно различаются правилами обращения к переменным, константам и регистрам, переопределения сегмента и размера инструкции, косвенной адресации и задания смещений.
>
> Синтаксис AT&T отличается знаком `%` перед всеми именами регистров, знаком `$` перед буквальными константами, также называемыми непосредственными операндами, и порядком операндов: источник указывается слева, а приёмник - справа. Команда прибавления четырёх к регистру EAX записывается как `add $0x4,%eax`. Этот синтаксис применяют GNU Assembler и многие другие средства GNU, включая `gcc` и `gdb`.
>
> В синтаксисе Intel префиксы регистров и констант не нужны, а порядок операндов обратный: источник расположен справа, приёмник - слева. Та же команда сложения записывается как `add eax,0x4`. Синтаксис Intel используют Microsoft Assembler (MASM), Borland Turbo Assembler (TASM) и Netwide Assembler (NASM).

Шаг 4

После вывода инструкции нужно перейти к следующей и повторять описанные действия, пока не будут дизассемблированы все инструкции файла.

Существуют разные алгоритмы выбора начальной точки, следующей инструкции, способов различения кода и данных, а также условия завершения обработки. Два основных алгоритма - линейный проход и рекурсивный спуск.

Дизассемблирование линейным проходом

Алгоритм линейного прохода ищет инструкции предельно прямолинейно: там, где заканчивается одна инструкция, начинается следующая. Поэтому самое трудное решение - выбор начальной точки. Обычно предполагается, что всё содержимое разделов программы, помеченных как код, представляет собой машинные инструкции. Такие разделы, как правило, указаны в заголовках файла программы.

Дизассемблирование начинается с первого байта раздела кода и линейно движется до конца раздела, разбирая одну инструкцию за другой. Алгоритм не пытается понять поток управления программы и не учитывает нелинейные инструкции, например переходы.

Во время обработки можно хранить указатель на начало текущей инструкции. Длину каждой инструкции вычисляют при разборе и используют для определения положения следующей. Наборы с инструкциями фиксированной длины, например MIPS, дизассемблировать несколько проще, поскольку положение последующих инструкций определяется однозначно.

Главное преимущество линейного прохода - полное покрытие разделов кода программы. Один из основных недостатков - неспособность учитывать данные, смешанные с кодом. Это видно в листинге 1-1, где показана функция, обработанная дизассемблером линейного прохода. Функция содержит оператор выбора, который компилятор реализовал посредством таблицы переходов и поместил прямо внутрь функции. Инструкция `jmp` по адресу `401250`, отмеченная X, ссылается на

таблицу адресов, начинающуюся с 401257, отмеченного Y. Однако дизассемблер принимает Y за инструкцию и ошибочно создаёт соответствующую запись на языке ассемблера.

```
   40123f: 55                    push   ebp
   401240: 8b ec                 mov    ebp,esp
   401242: 33 c0                 xor    eax,eax
   401244: 8b 55 08              mov    edx,DWORD PTR [ebp+8]
   401247: 83 fa 0c              cmp    edx,0xc
   40124a: 0f 87 90 00 00 00     ja     0x4012e0
X 401250: ff 24 95 57 12 40 00  jmp    DWORD PTR [edx*4+0x401257]
Y 401257: e0 12                 loopne 0x40126b
  401259: 40                    inc    eax
  40125a: 00 8b 12 40 00 90     add    BYTE PTR [ebx-0x6fffbfee],cl
  401260: 12 40 00              adc    al,BYTE PTR [eax]
  401263: 95                    xchg   ebp,eax
  401264: 12 40 00              adc    al,BYTE PTR [eax]
  401267: 9a 12 40 00 a2 12 40  call   0x4012:0xa2004012
  40126e: 00 aa 12 40 00 b2     add    BYTE PTR [edx-0x4dffbfee],ch
  401274: 12 40 00              adc    al,BYTE PTR [eax]
  401277: ba 12 40 00 c2        mov    edx,0xc2004012
  40127c: 12 40 00              adc    al,BYTE PTR [eax]
  40127f: ca 12 40              lret   0x4012
  401282: 00 d2                 add    dl,dl
  401284: 12 40 00              adc    al,BYTE PTR [eax]
  401287: da 12                 ficom  DWORD PTR [edx]
  401289: 40                    inc    eax
  40128a: 00 8b 45 0c eb 50     add    BYTE PTR [ebx+0x50eb0c45],cl
  401290: 8b 45 10              mov    eax,DWORD PTR [ebp+16]
  401293: eb 4b                 jmp    0x4012e0
```

Листинг 1-1. Дизассемблирование линейным проходом

Если рассматривать последовательные четырёхбайтовые группы, начиная с Y, как значения с порядком байтов от младшего к старшему[3], окажется, что каждая группа представляет собой указатель на соседний адрес - цель одного из переходов: 004012e0, 0040128b, 00401290 и так далее. Следовательно, loopne по адресу Y вовсе не является инструкцией. Это результат ошибки алгоритма линейного прохода, который не сумел отличить встроенные данные от кода.

Линейный проход используют механизмы дизассемблирования в отладчиках GNU gdb и Microsoft WinDbg, а также программа objdump.

Дизассемблирование рекурсивным спуском

Рекурсивный спуск применяет другой способ поиска инструкций. В центре этого подхода находится поток управления: инструкция подлежит дизассемблированию, если на неё ссылается другая инструкция. Чтобы понять рекурсивный спуск, удобно разделить инструкции по тому, как они воздействуют на указатель инструкций процессора.

Инструкции последовательного выполнения

Такие инструкции передают управление непосредственно следующей инструкции. К ним относятся простые арифметические операции, например add; перенос данных между регистром и памятью, например mov; действия со стеком, например push и pop. Для них дизассемблирование продолжается так же, как при линейном проходе.

Инструкции условного перехода

Инструкции условного перехода, например x86 `jnz`, допускают два пути выполнения. Если условие истинно, переход совершается и указатель инструкций принимает адрес цели. Если условие ложно, выполнение продолжается линейно и следующую инструкцию можно обработать методом линейного прохода.

При статическом анализе обычно невозможно определить результат проверки условия, поэтому алгоритм рекурсивного спуска дизассемблирует оба пути. Обработку инструкции по адресу цели откладывают, добавляя её адрес в список для последующего рассмотрения.

Инструкции безусловного перехода

Безусловные переходы не соответствуют линейной модели потока и поэтому обрабатываются иначе. Как и у инструкций последовательного выполнения, у них возможна только одна следующая инструкция, но она не обязана непосредственно следовать за инструкцией перехода. Более того, как видно из листинга 1-1, после безусловного перехода вообще не обязательно должна находиться инструкция. Поэтому дизассемблировать следующие за ним байты нет оснований.

Дизассемблер рекурсивного спуска пытается определить цель безусловного перехода и добавляет адрес назначения в список ещё не исследованных адресов. К сожалению, некоторые безусловные переходы создают трудности. Если цель перехода зависит от значения, возникающего только во время выполнения, статический анализ может не позволить определить адрес назначения.

Эту проблему показывает инструкция x86 `jmp eax`. Значение в регистре EAX появляется лишь во время работы программы. При статическом анализе его нет, поэтому невозможно установить цель перехода и понять, откуда продолжать дизассемблирование.

Инструкции вызова функций

Инструкции вызова функций действуют почти так же, как безусловные переходы, включая невозможность определить цель таких команд, как `call eax`. Однако после завершения функции управление обычно возвращается к инструкции, расположенной сразу после вызова. В этом отношении вызов похож на условный переход, поскольку создаёт два пути выполнения. Адрес цели добавляют в список для отложенного дизассемблирования, а инструкцию сразу после вызова обрабатывают подобно линейному проходу.

Рекурсивный спуск может дать сбой, если возврат из вызванной функции происходит не так, как ожидается. Например, код функции способен намеренно изменить собственный адрес возврата, чтобы по завершении передать управление не туда, куда рассчитывает дизассемблер.

Простой пример приведён в следующем ошибочном листинге. Функция `foo` перед возвратом прибавляет единицу к адресу возврата.

```
foo               proc near
  FF 04 24        inc   dword ptr [esp] ; увеличить сохранённый адрес возврата
  C3              retn
foo               endp
; ---------------------------------------
bar:
  E8 F7 FF FF FF  call  foo
  05 89 45 F8 90 Xadd   eax, 90F84589h
```

В результате управление не передаётся инструкции `add`, отмеченной X и расположенной сразу после вызова `foo`. Правильное дизассемблирование выглядит так:

```
foo               proc near
  FF 04 24        inc   dword ptr [esp]
```

```
  C3              retn
foo               endp
; ------------------------------------
bar:
  E8 F7 FF FF FF  call  foo
  05              db    5 ; прежде первый байт инструкции add
  89 45 F8       Ymov   [ebp-8], eax
  90              nop
```

Этот листинг точнее показывает настоящий поток выполнения: функция `foo` возвращает управление инструкции `mov`, отмеченной Y. Важно понимать, что дизассемблер линейного прохода тоже неправильно обработает этот код, хотя и по несколько иной причине.

Инструкции возврата

В некоторых случаях у алгоритма рекурсивного спуска заканчиваются пути, по которым можно двигаться. Инструкция возврата из функции, например x86 `ret`, не сообщает, какая инструкция будет выполнена следующей. В работающей программе адрес извлекается с вершины стека, после чего выполнение продолжается по этому адресу. Дизассемблер не имеет доступа к стеку, поэтому обработка внезапно останавливается.

В этот момент дизассемблер рекурсивного спуска обращается к списку адресов, отложенных для последующего рассмотрения. Он извлекает из списка один адрес и продолжает работу с него. Именно этот рекурсивный процесс дал алгоритму название.

Одно из главных преимуществ рекурсивного спуска - более надёжное различение кода и данных. Алгоритм опирается на поток управления, поэтому гораздо реже ошибочно принимает значения данных за код. Основной недостаток - невозможность проследить косвенные пути выполнения, например переходы и вызовы, которые находят целевой адрес в таблице указателей.

Однако эвристические правила распознавания указателей на код позволяют дизассемблерам рекурсивного спуска обеспечить почти полное покрытие кода и очень хорошо отличать его от данных. В листинге 1-2 показан результат обработки тем же методом оператора выбора, который ранее был представлен в листинге 1-1.

```
0040123F   push ebp
00401240   mov  ebp, esp
00401242   xor  eax, eax
00401244   mov  edx, [ebp+arg_0]
00401247   cmp  edx, 0Ch             ; выбор из 13 вариантов
0040124A   ja   loc_4012E0           ; вариант по умолчанию
0040124A                             ; вариант 0 таблицы 00401250
00401250   jmp  ds:off_401257[edx*4] ; переход через таблицу
00401250 ; ---------------------------------------------------
00401257 off_401257:
00401257   dd offset loc_4012E0  ; ссылка на данные: sub_40123F+11r
00401257   dd offset loc_40128B  ; таблица переходов оператора выбора
00401257   dd offset loc_401290
00401257   dd offset loc_401295
00401257   dd offset loc_40129A
00401257   dd offset loc_4012A2
00401257   dd offset loc_4012AA
00401257   dd offset loc_4012B2
00401257   dd offset loc_4012BA
00401257   dd offset loc_4012C2
00401257   dd offset loc_4012CA
00401257   dd offset loc_4012D2
00401257   dd offset loc_4012DA
0040128B ; ---------------------------------------------------
0040128B
0040128B loc_40128B:             ; ссылка на код: sub_40123F+11j
0040128B                         ; ссылка на данные: sub_40123F:off_401257o
0040128B   mov  eax, [ebp+arg_4] ; вариант 1 таблицы 00401250
```

```
0040128E   jmp  short loc_4012E0 ; вариант по умолчанию
0040128E                         ; вариант 0 таблицы 00401250
```

Листинг 1-2. Дизассемблирование рекурсивным спуском

Обратите внимание: таблица адресов переходов распознана и соответствующим образом оформлена. IDA Pro - самый известный пример дизассемблера рекурсивного спуска. Понимание этого процесса поможет замечать ситуации, в которых IDA создаёт не лучший листинг, и разрабатывать способы улучшения результата.

Итоги

Обязательно ли глубоко понимать алгоритмы дизассемблирования, чтобы пользоваться дизассемблером? Нет. Полезно ли это? Да! При обратной разработке меньше всего хочется тратить время на борьбу с собственными инструментами.

Одно из многочисленных преимуществ IDA состоит в том, что, в отличие от большинства других дизассемблеров, она предоставляет широкие возможности направлять её работу и отменять принятые ею решения. В итоге готовый результат - точный листинг дизассемблирования - оказывается значительно лучше всего, что можно получить другими средствами.

В следующей главе мы рассмотрим ряд существующих инструментов, полезных во многих задачах обратной разработки. Хотя они не связаны с IDA напрямую, многие из них повлияли на IDA и сами испытали её влияние. Кроме того, они помогают понять, почему пользовательский интерфейс IDA содержит так много разных представлений информации.

Сноски

[1] Фаззинг - метод обнаружения уязвимостей, при котором для программы создают множество различных входных данных в надежде, что один из вариантов вызовет обнаруживаемый сбой, который затем удастся проанализировать и использовать.
[2] Точка входа программы - адрес инструкции, которой операционная система передаёт управление после загрузки программы в память.
[3] Процессор относят к системам с порядком байтов от старшего к младшему или от младшего к старшему в зависимости от того, какой байт многобайтового значения он сохраняет первым: наиболее значимый или наименее значимый.

Глава 2. Инструменты обратной разработки и дизассемблирования

Теперь, когда мы знакомы с основами дизассемблирования, но ещё не погрузились в особенности IDA Pro, полезно рассмотреть другие инструменты обратной разработки двоичных файлов. Многие из них появились раньше IDA и по-прежнему удобны для беглого изучения файлов и перепроверки результатов IDA.

Как мы увидим, IDA объединяет многие возможности этих инструментов в своём интерфейсе и тем самым предоставляет единую среду обратной разработки. Хотя IDA содержит встроенный отладчик, здесь отладчики не рассматриваются: им посвящены главы 24, 25 и 26.

Инструменты классификации

При первой встрече с неизвестным файлом часто нужно ответить на простой вопрос: «Что это такое?» Первое практическое правило - никогда не определять настоящее содержимое файла только по расширению его имени. Оно же второе, третье и четвёртое правило. Усвоив, что расширения имён почти ничего не гарантируют, стоит познакомиться с одной или несколькими описанными ниже программами.

file

Команда `file` - стандартная служебная программа, входящая в большинство операционных систем семейства Unix, а в Windows доступная в составе Cygwin[1] и MinGW[2]. Она пытается определить тип файла по значениям определённых полей внутри него.

Иногда `file` распознаёт распространённые строки, например `#!/bin/sh`, обозначающую сценарий командной оболочки, или `<html>`, указывающую на документ HTML. Файлы с содержимым вне набора ASCII представляют более сложную задачу. В таких случаях программа проверяет, соответствует ли структура содержимого известному формату. Часто она ищет особые значения меток, называемые магическими числами[3], которые считаются характерными для конкретных типов файлов.

Ниже приведены шестнадцатеричные представления магических чисел нескольких распространённых форматов.

Исполняемый файл PE для Windows:

```
00000000  4D 5A 90 00  03 00 00 00  04 00 00 00  FF FF 00 00  MZ..............
00000010  B8 00 00 00  00 00 00 00  40 00 00 00  00 00 00 00  ........@.......
```

Изображение JPEG:

```
00000000  FF D8 FF E0  00 10 4A 46  49 46 00 01  01 01 00 60  ......JFIF.....`
00000010  00 60 00 00  FF DB 00 43  00 0A 07 07  08 07 06 0A  .`.....C........
```

Файл класса Java:

```
00000000  CA FE BA BE  00 00 00 32  00 98 0A 00  2E 00 3E 08  .......2......>.
00000010  00 3F 09 00  40 00 41 08  00 42 0A 00  43 00 44 0A  .?..@.A..B..C.D.
```

Программа `file` распознаёт множество форматов, включая несколько видов текстовых файлов ASCII, исполняемых файлов и файлов данных. Проверки магических чисел определяются

правилами из файла `magic`. Путь к нему зависит от операционной системы; часто встречаются `/usr/share/file/magic`, `/usr/share/misc/magic` и `/etc/magic`. Подробнее такие файлы описаны в документации `file`.

> **Среда Cygwin.** Cygwin - набор служебных программ для Windows, предоставляющий командную оболочку в стиле Linux и сопутствующие средства. При установке можно выбрать множество стандартных пакетов, в том числе компиляторы `gcc` и `g++`, интерпретаторы Perl, Python и Ruby, сетевые программы `nc` и `ssh` и многое другое. После установки Cygwin многие программы для Linux можно скомпилировать и запустить в Windows.

В некоторых случаях `file` способна различать варианты одного типа файлов. Следующий пример показывает, что программа определяет не только несколько разновидностей двоичных файлов ELF, но и способ компоновки - статический или динамический, - а также наличие или отсутствие символьных данных.

```
idabook# file ch2_ex_*
ch2_ex.exe:                 исполняемый файл MS-DOS PE для Windows (консольный),
                            Intel 80386, 32-разрядный
ch2_ex_upx.exe:             исполняемый файл MS-DOS PE для Windows (консольный),
                            Intel 80386, 32-разрядный, сжат UPX
ch2_ex_freebsd:             исполняемый ELF, 32-разрядный, байты от младшего
                            к старшему, Intel 80386, версия 1 (FreeBSD),
                            для FreeBSD 5.4, динамически скомпонован,
                            в стиле FreeBSD, символы не удалены
ch2_ex_freebsd_static:      исполняемый ELF, 32-разрядный, байты от младшего
                            к старшему, Intel 80386, версия 1 (FreeBSD),
                            для FreeBSD 5.4, статически скомпонован,
                            в стиле FreeBSD, символы не удалены
ch2_ex_freebsd_static_strip: исполняемый ELF, 32-разрядный, байты от младшего
                            к старшему, Intel 80386, версия 1 (FreeBSD),
                            для FreeBSD 5.4, статически скомпонован,
                            в стиле FreeBSD, символы удалены
ch2_ex_linux:               исполняемый ELF, 32-разрядный, байты от младшего
                            к старшему, Intel 80386, версия 1 (SYSV),
                            для GNU/Linux 2.6.9, динамически скомпонован,
                            символы не удалены
ch2_ex_linux_static:        исполняемый ELF, 32-разрядный, байты от младшего
                            к старшему, Intel 80386, версия 1 (SYSV),
                            для GNU/Linux 2.6.9, статически скомпонован,
                            символы не удалены
ch2_ex_linux_static_strip:  исполняемый ELF, 32-разрядный, байты от младшего
                            к старшему, Intel 80386, версия 1 (SYSV),
                            для GNU/Linux 2.6.9, статически скомпонован,
                            символы удалены
ch2_ex_linux_stripped:      исполняемый ELF, 32-разрядный, байты от младшего
                            к старшему, Intel 80386, версия 1 (SYSV),
                            для GNU/Linux 2.6.9, динамически скомпонован,
                            символы удалены
```

> **Удаление символов из исполняемых файлов.** Удаление символов - исключение символьных данных из двоичного файла. Символы появляются в объектных файлах при компиляции. Часть из них нужна компоновщику, чтобы при создании итоговой программы или библиотеки разрешать ссылки между файлами. Другие символы содержат дополнительные сведения для отладчика.
>
> После компоновки многие символы больше не нужны. Переданные компоновщику параметры позволяют удалить их во время сборки. Символы из уже существующего двоичного файла можно удалить программой `strip`. Получившийся файл станет меньше, но его поведение не изменится.

Программа `file` и подобные инструменты не безошибочны. Файл можно определить неправильно лишь потому, что в нём случайно присутствуют признаки некоторого формата. Убедиться в этом несложно: замените в шестнадцатеричном редакторе первые четыре байта любого файла магическим числом Java `CA FE BA BE`. После этого `file` ошибочно назовёт его скомпилированным файлом класса Java. Аналогично текстовый файл, содержащий только два символа `MZ`, будет принят за исполняемый файл MS-DOS.

При любой обратной разработке полезно придерживаться общего правила: не доверять результату отдельного инструмента полностью, пока он не подтверждён несколькими другими средствами и ручным анализом.

PE Tools

PE Tools[4] - набор средств анализа исполняемых файлов и работающих процессов в Windows. На рисунке 2-1 показано главное окно программы со списком активных процессов и доступом ко всем её инструментам.

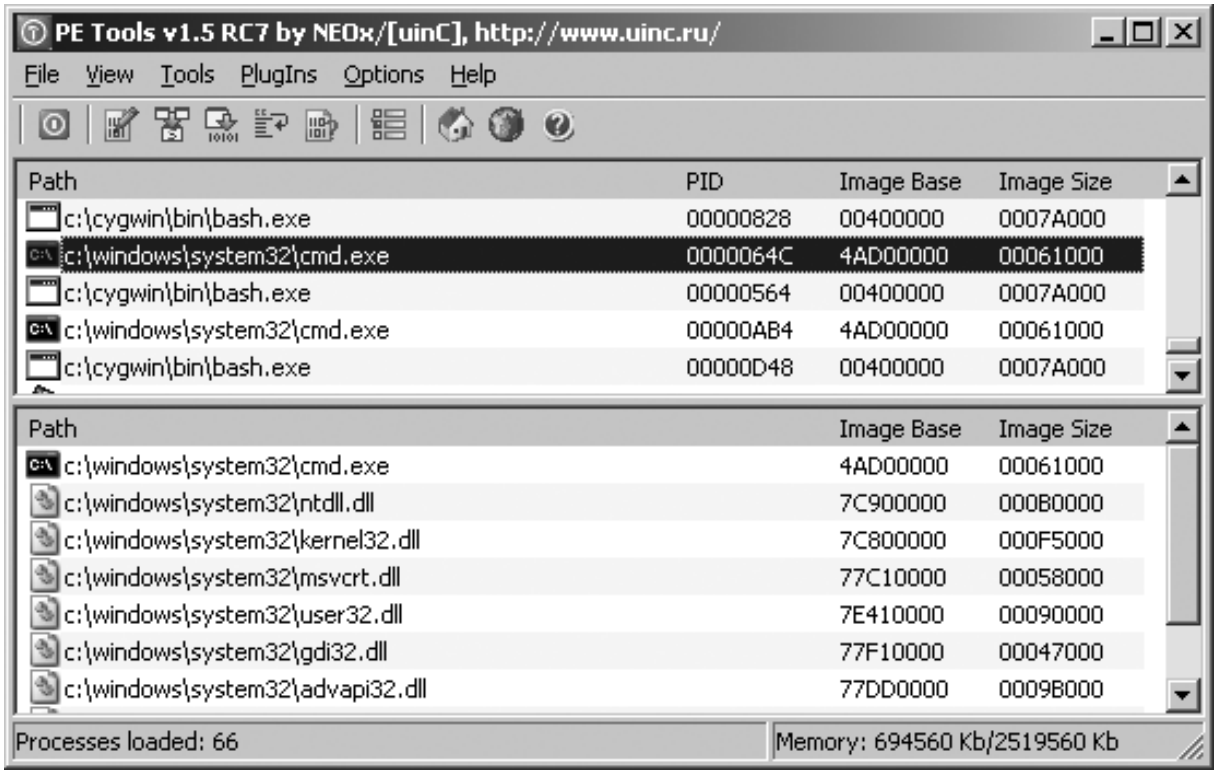


Рисунок 2-1. Программа PE Tools

Из списка процессов можно сохранить образ памяти процесса в файл или запустить PE Sniffer, чтобы определить компилятор исполняемого файла и выяснить, обрабатывался ли файл известным средством обфускации. Меню инструментов предоставляет похожие возможности для анализа файлов на диске.

Встроенный PE Editor показывает поля заголовка PE и позволяет легко изменять их значения. Исправление заголовков PE часто требуется при восстановлении допустимого файла PE из обфусцированной версии.

> **Обфускация двоичных файлов.** Обфускация - любая попытка скрыть истинный смысл чего-либо. Применительно к исполняемому файлу это попытка затруднить понимание настоящего поведения программы. Разработчики прибегают к ней по разным причинам,

среди которых часто называют защиту закрытых алгоритмов и сокрытие вредоносных намерений. Почти все вредоносные программы используют обфускацию, чтобы осложнить анализ. Авторам доступно множество готовых средств создания обфусцированных программ. Инструменты и приёмы обфускации, а также их влияние на обратную разработку подробнее рассматриваются в главе 21.

PEiD

PEiD[5] - ещё один инструмент для Windows. Его основные задачи - определение компилятора конкретного двоичного файла PE и обнаружение средств, применённых для его обфускации. На рисунке 2-2 PEiD определяет ASPack как средство обфускации одного из вариантов червя Gaobot[6].

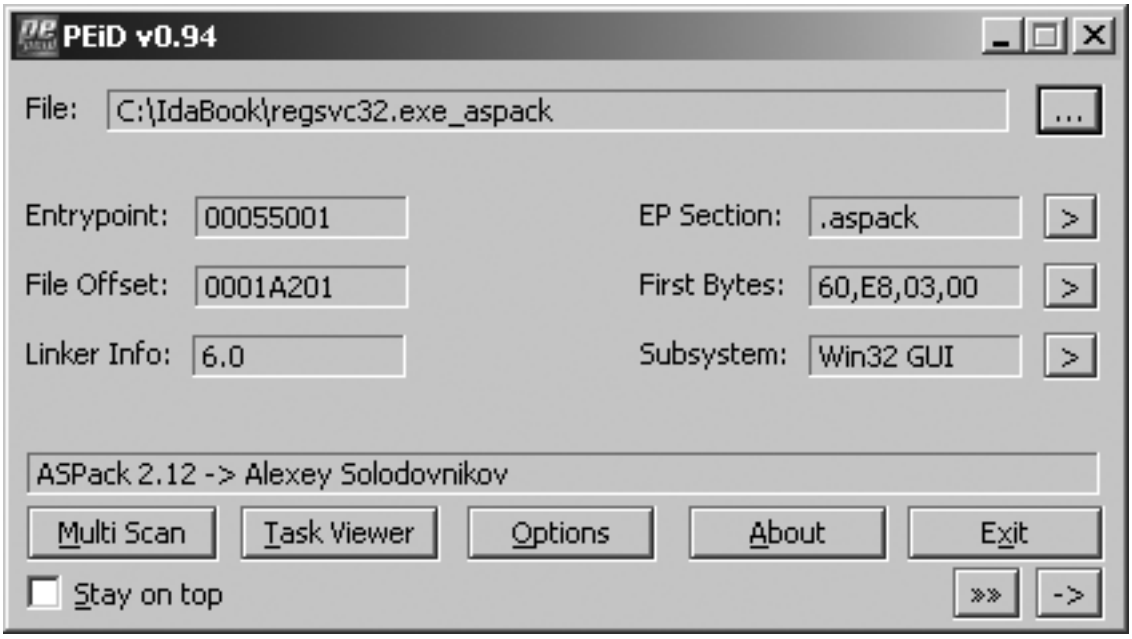


Рисунок 2-2. Программа PEiD

Многие дополнительные возможности PEiD совпадают с возможностями PE Tools: программа способна показывать сводку заголовков PE, собирать сведения о работающих процессах и выполнять простейшее дизассемблирование.

Инструменты получения сводных сведений

Поскольку наша цель - обратная разработка двоичных файлов программ, после первоначальной классификации понадобятся более развитые средства извлечения подробной информации. Описанные в этом разделе инструменты гораздо лучше знают структуру обрабатываемых файлов. В большинстве случаев каждый из них рассчитан на строго определённый формат и извлекает из входного файла конкретные сведения.

nm

При компиляции исходных файлов в объектные компилятор должен сохранить сведения о расположении глобальных, или внешних, символов. Они нужны компоновщику, чтобы разрешить ссылки при объединении объектных файлов в исполняемую программу. Если не приказать компоновщику удалить символы из итогового файла, обычно он переносит их из объектных файлов в готовую программу. Согласно справочной странице, назначение `nm` - «выводить список символов объектных файлов».

При исследовании промежуточного объектного файла с расширением `.o`, а не исполняемой программы, стандартный вывод содержит имена объявленных в файле функций и глобальных переменных. Ниже приведён пример работы `nm`.

```
idabook# gcc -c ch2_example.c
idabook# nm ch2_example.o
         U __stderrp
         U exit
         U fprintf
00000038 T get_max
00000000 t hidden
00000088 T main
00000000 D my_initialized_global
00000004 C my_unitialized_global
         U printf
         U rand
         U scanf
         U srand
         U time
00000010 T usage
idabook#
```

Для каждого символа `nm` показывает некоторые сведения. Буква обозначает тип символа:

• `U` - неопределённый символ, обычно ссылка на внешний символ;
• `T` - символ, определённый в разделе кода, обычно имя функции;
• `t` - локальный символ из раздела кода, в программе на Си обычно соответствующий статической функции;
• `D` - инициализированное значение данных;
• `C` - неинициализированное значение данных.

> **Примечание.** Прописные буквы обозначают глобальные символы, строчные - локальные. Полное описание буквенных обозначений приведено на справочной странице `nm`.

При выводе символов исполняемого файла `nm` предоставляет несколько больше сведений. Во время компоновки символам по возможности назначаются виртуальные адреса, и программа может их показать. Ниже приведён сокращённый пример.

```
idabook# gcc -o ch2_example ch2_example.c
idabook# nm ch2_example
         <. . .>
         U exit
         U fprintf
080485c0 t frame_dummy
08048644 T get_max
0804860c t hidden
08048694 T main
0804997c D my_initialized_global
08049a9c B my_unitialized_global
08049a80 b object.2
08049978 d p.0
         U printf
         U rand
         U scanf
         U srand
         U time
0804861c T usage
idabook#
```

Теперь некоторые символы, например `main`, получили виртуальные адреса; в результате компоновки появились новые символы, например `frame_dummy`; у некоторых, таких как `my_unitialized_global`, изменился тип; остальные по-прежнему не определены, поскольку ссылаются на внешние символы.

В данном случае исследуемый файл скомпонован динамически, а неопределённые символы находятся в общей библиотеке Си. Дополнительные сведения о `nm` приведены на соответствующей справочной странице.

ldd

При создании исполняемого файла необходимо разрешить адреса всех используемых библиотечных функций. Компоновщик поддерживает два способа разрешения таких вызовов: статическую и динамическую компоновку. Выбор определяется параметрами командной строки. Один исполняемый файл может сочетать оба способа[7].

При статической компоновке компоновщик объединяет объектные файлы приложения с копией необходимой библиотеки и создаёт исполняемый файл. Во время работы искать библиотечный код не требуется, поскольку он уже включён в программу.

Преимущества статической компоновки:

1. вызовы функций выполняются немного быстрее;
2. двоичные файлы проще распространять, поскольку не приходится рассчитывать на наличие нужных библиотек в системе пользователя.

Недостатки статической компоновки:

1. итоговые исполняемые файлы имеют больший размер;
2. программу труднее обновлять при изменении библиотек.

После каждого изменения библиотеки программу приходится компоновать заново. Для обратной разработки статическая компоновка тоже создаёт трудности. При анализе такого файла нет простого способа ответить на вопросы: «Какие библиотеки включены в этот файл?» и «Какие из этих функций являются библиотечными?» Особенности обратной разработки статически скомпонованного кода рассматриваются в главе 12.

При динамической компоновке копировать необходимые библиотеки не нужно. Компоновщик лишь добавляет в итоговый исполняемый файл ссылки на них - обычно на файлы `.so` или `.dll`, - благодаря чему программа получается значительно меньше.

Обновлять динамические библиотеки гораздо проще. Одна копия библиотеки используется множеством программ, поэтому её замена новой версией сразу обновляет библиотечный код для всех этих программ. Однако загрузка усложняется: вместо одного статически скомпонованного файла, уже содержащего весь нужный код, система должна найти все библиотеки и загрузить их в память. Кроме того, поставщику приходится распространять не только свою программу, но и все библиотеки, от которых она зависит. Если хотя бы одной обязательной библиотеки в системе нет, программа завершится с ошибкой.

Следующий пример показывает создание динамически и статически скомпонованных вариантов одной программы, их размеры и результат определения типа посредством `file`.

```
idabook# gcc -o ch2_example_dynamic ch2_example.c
idabook# gcc -o ch2_example_static ch2_example.c --static
idabook# ls -l ch2_example_*
-rwxr-xr-x  1 root  wheel    6017 Sep 26 11:24 ch2_example_dynamic
-rwxr-xr-x  1 root  wheel  167987 Sep 26 11:23 ch2_example_static
```

```
idabook# file ch2_example_*
ch2_example_dynamic: исполняемый ELF, 32-разрядный, байты от младшего
        к старшему, Intel 80386, версия 1 (FreeBSD), динамически
        скомпонован, символы не удалены
ch2_example_static:  исполняемый ELF, 32-разрядный, байты от младшего
        к старшему, Intel 80386, версия 1 (FreeBSD), статически
        скомпонован, символы не удалены
idabook#
```

Чтобы динамическая компоновка работала, исполняемый файл должен указывать свои библиотеки и конкретные ресурсы, требуемые от каждой из них. Поэтому, в отличие от статически скомпонованного файла, список зависимостей динамически скомпонованной программы получить очень просто.

Служебная программа `ldd`, название которой означает «список динамических зависимостей», выводит динамические библиотеки, необходимые исполняемому файлу. Ниже она определяет зависимости веб-сервера Apache.

```
idabook# ldd /usr/local/sbin/httpd
/usr/local/sbin/httpd:
        libm.so.4 => /lib/libm.so.4 (0x280c5000)
        libaprutil-1.so.2 => /usr/local/lib/libaprutil-1.so.2 (0x280db000)
        libexpat.so.6 => /usr/local/lib/libexpat.so.6 (0x280ef000)
        libiconv.so.3 => /usr/local/lib/libiconv.so.3 (0x2810d000)
        libapr-1.so.2 => /usr/local/lib/libapr-1.so.2 (0x281fa000)
        libcrypt.so.3 => /lib/libcrypt.so.3 (0x2821a000)
        libpthread.so.2 => /lib/libpthread.so.2 (0x28232000)
        libc.so.6 => /lib/libc.so.6 (0x28257000)
idabook#
```

Программа `ldd` доступна в Linux и системах семейства BSD. В OS X похожую задачу решает `otool` с параметром `-L`: `otool -L имя_файла`. В Windows зависимости можно вывести программой `dumpbin` из комплекта Visual Studio: `dumpbin /dependents имя_файла`.

objdump

Если `ldd` решает довольно узкую задачу, то `objdump` чрезвычайно универсальна. Её назначение - «показывать сведения из объектных файлов»[8]. Для этой обширной задачи предусмотрено более тридцати параметров командной строки, извлекающих разные данные. В частности, `objdump` умеет выводить перечисленные ниже сведения.

Заголовки разделов. Сводные сведения о каждом разделе файла программы.

Закрытые заголовки. Схема размещения программы в памяти и другие сведения, необходимые загрузчику времени выполнения, включая список обязательных библиотек, подобный результату `ldd`.

Отладочные сведения. Отладочные данные, встроенные в файл программы.

Символьные сведения. Содержимое таблицы символов в виде, похожем на вывод `nm`.

Листинг дизассемблирования. Программа `objdump` линейным проходом дизассемблирует разделы, помеченные как код. Для x86 она умеет создавать текст в синтаксисе AT&T или Intel; результат можно сохранить в текстовый файл. Такой файл называют статическим листингом дизассемблирования. Его можно применять для обратной разработки, но по нему трудно перемещаться и ещё труднее последовательно вносить изменения без ошибок.

`objdump` входит в комплект GNU Binutils[9] и доступна в Linux, FreeBSD и Windows через Cygwin. Для доступа к объектным файлам она использует библиотеку описания двоичных файлов `libbfd`, также входящую в Binutils, и поэтому понимает поддерживаемые этой библиотекой форматы, включая ELF и PE.

Для разбора только ELF существует программа `readelf`. Она предоставляет большинство возможностей `objdump`, но не использует `libbfd`.

otool

Проще всего описать `otool` как аналог `objdump` для OS X. Она предназначена для разбора двоичных файлов Mach-O. В следующем примере `otool` выводит зависимости Mach-O от динамических библиотек, то есть выполняет задачу, похожую на работу `ldd`.

```
idabook# file osx_example
osx_example: исполняемый Mach-O для ppc
idabook# otool -L osx_example
osx_example:
        /usr/lib/libstdc++.6.dylib (версия совместимости 7.0.0, текущая 7.4.0)
        /usr/lib/libgcc_s.1.dylib (версия совместимости 1.0.0, текущая 1.0.0)
        /usr/lib/libSystem.B.dylib (версия совместимости 1.0.0, текущая 88.1.5)
```

Кроме того, `otool` показывает заголовки и таблицы символов, а также дизассемблирует раздел кода. Полное описание возможностей приведено на справочной странице программы.

dumpbin

`dumpbin` - программа командной строки из комплекта Microsoft Visual Studio. Подобно `otool` и `objdump`, она выводит разнообразные сведения о файлах PE для Windows. Ниже `dumpbin` показывает динамические зависимости системного калькулятора аналогично `ldd`.

```
$ dumpbin /dependents calc.exe
Microsoft (R) COFF/PE Dumper, версия 8.00.50727.762
Авторские права (C) Microsoft Corporation. Все права защищены.

Данные файла calc.exe

Тип файла: ИСПОЛНЯЕМЫЙ ОБРАЗ

  Образ имеет следующие зависимости:

    SHELL32.dll
    msvcrt.dll
    ADVAPI32.dll
    KERNEL32.dll
    GDI32.dll
    USER32.dll
```

Другие параметры `dumpbin` позволяют извлекать сведения из разных разделов PE, включая символы, имена импортируемых и экспортируемых функций и дизассемблированный код. Дополнительная документация доступна в сети разработчиков Microsoft[10].

c++filt

Языкам, допускающим перегрузку функций, нужен способ различать варианты одной функции, поскольку все они имеют одинаковое имя. В следующем примере на C++ объявлено несколько вариантов функции `demo`.

```
void demo(void);
void demo(int x);
void demo(double x);
void demo(int x, double y);
void demo(double x, int y);
```

```
void demo(char* str);
```

Как правило, объектный файл не может содержать две функции с одним именем. Для поддержки перегрузки компилятор создаёт уникальные имена, включая в них сведения о последовательности типов аргументов. Получение уникальных имён для одноимённых функций называется декорированием имён[11]. Если вывести посредством `nm` символы скомпилированной версии приведённого кода, получится примерно следующее:

```
idabook# g++ -o cpp_test cpp_test.cpp
idabook# nm cpp_test | grep demo
0804843c T _Z4demoPc
08048400 T _Z4demod
08048428 T _Z4demodi
080483fa T _Z4demoi
08048414 T _Z4demoid
080483f4 T _Z4demov
```

Стандарт C++ не задаёт способ декорирования имён, поэтому разработчики компиляторов создают собственные схемы. Чтобы расшифровать показанные варианты `demo`, нужен инструмент, понимающий схему конкретного компилятора - в данном случае `g++`. Именно для этого предназначена программа `c++filt`.

Она рассматривает каждое входное слово как возможное декорированное имя и пытается определить создавший его компилятор. Если слово похоже на допустимое декорированное имя, программа выводит восстановленный вариант. Нераспознанное слово остаётся без изменений.

Передав `c++filt` результаты предыдущего вызова `nm`, можно восстановить имена функций.

```
idabook# nm cpp_test | grep demo | c++filt
0804843c T demo(char*)
08048400 T demo(double)
08048428 T demo(double, int)
080483fa T demo(int)
08048414 T demo(int, double)
080483f4 T demo()
```

Декорированные имена содержат дополнительные сведения о функциях, которые `nm` обычно не показывает. При обратной разработке они могут быть чрезвычайно полезны. В более сложных случаях из них удаётся получить имена классов или соглашения о вызовах функций.

Инструменты глубокого исследования

До сих пор мы рассматривали средства поверхностного анализа, которым достаточно минимальных знаний о внутреннем устройстве файла, а также программы, извлекающие строго определённые данные благодаря подробному знанию формата. Теперь перейдём к инструментам, которые ищут сведения определённого вида независимо от типа исследуемого файла.

strings

Иногда содержимому файла полезно задать общий вопрос, не требующий знания его структуры: «Есть ли внутри текстовые строки?» Сначала нужно определить, что считать строкой. В первом приближении это непрерывная последовательность печатных знаков. Обычно определение дополняют минимальной длиной и конкретным набором знаков.

Например, можно искать все последовательности как минимум из четырёх печатных знаков ASCII и выводить их на экран. Структура файла почти не ограничивает такой поиск: строки одинаково легко искать в двоичном файле ELF и в документе Microsoft Word.

Программа strings специально предназначена для извлечения текстовых строк и часто не учитывает формат файла. Со стандартными параметрами - последовательности не короче четырёх знаков в семибитном ASCII - она может выдать следующий результат:

```
idabook# strings ch2_example
/lib/ld-linux.so.2
__gmon_start__
libc.so.6
_IO_stdin_used
exit
srand
puts
time
printf
stderr
fwrite
scanf
__libc_start_main
GLIBC_2.0
PTRh
[^_]
usage: ch2_example [max]
A simple guessing game!
Please guess a number between 1 and %d.
Invalid input, quitting!
Congratulations, you got it in %d attempt(s)!
Sorry too low, please try again
Sorry too high, please try again
```

Здесь встречаются строки, похожие на сообщения программы, но также имена функций и библиотек. Нельзя поспешно судить по ним о поведении программы. Аналитики часто попадают в ловушку, пытаясь вывести назначение программы только из результата strings. Само присутствие строки в двоичном файле вовсе не означает, что программа когда-либо её использует.

При работе со strings следует помнить ещё несколько особенностей:

• в исполняемом файле по умолчанию просматриваются только загружаемые и инициализированные разделы; параметр -a заставляет проверить весь входной файл;
• программа не сообщает положение найденной строки; параметр -t добавляет смещение каждой строки в файле;
• во многих файлах применяются другие наборы знаков; параметр -e позволяет искать широкие знаки, например 16-разрядный Юникод.

Дизассемблеры

Как уже отмечалось, некоторые инструменты создают статические листинги дизассемблирования двоичных объектных файлов. Файлы PE, ELF и Mach-O можно обрабатывать соответственно программами dumpbin, objdump и otool. Однако ни одна из них не работает с произвольными блоками двоичных данных.

Иногда встречается файл, не соответствующий распространённому формату. Тогда нужен инструмент, способный начать дизассемблирование с указанного пользователем смещения.

Два примера таких потоковых дизассемблеров для набора инструкций x86 - ndisasm и diStorm[12]. Программа ndisasm входит в Netwide Assembler (NASM)[13]. В следующем примере она дизассемблирует фрагмент кода командной оболочки, созданный с помощью Metasploit[14].

```
idabook# ./msfpayload linux/x86/shell_findport CPORT=4444 R > fs
idabook# ls -l fs
-rw-r--r-- 1 ida ida 62 Dec 11 15:49 fs
```

```
idabook# ndisasm -u fs
00000000  31D2              xor edx,edx
00000002  52                push edx
00000003  89E5              mov ebp,esp
00000005  6A07              push byte +0x7
00000007  5B                pop ebx
00000008  6A10              push byte +0x10
0000000A  54                push esp
0000000B  55                push ebp
0000000C  52                push edx
0000000D  89E1              mov ecx,esp
0000000F  FF01              inc dword [ecx]
00000011  6A66              push byte +0x66
00000013  58                pop eax
00000014  CD80              int 0x80
00000016  66817D02115C      cmp word [ebp+0x2],0x5c11
0000001C  75F1              jnz 0xf
0000001E  5B                pop ebx
0000001F  6A02              push byte +0x2
00000021  59                pop ecx
00000022  B03F              mov al,0x3f
00000024  CD80              int 0x80
00000026  49                dec ecx
00000027  79F9              jns 0x22
00000029  52                push edx
0000002A  682F2F7368        push dword 0x68732f2f
0000002F  682F62696E        push dword 0x6e69622f
00000034  89E3              mov ebx,esp
00000036  52                push edx
00000037  53                push ebx
00000038  89E1              mov ecx,esp
0000003A  B00B              mov al,0xb
0000003C  CD80              int 0x80
```

Гибкость потокового дизассемблирования полезна во многих случаях. Например, при анализе сетевой атаки пакеты могут содержать код командной оболочки. Потоковый дизассемблер позволяет обработать соответствующую часть пакета и изучить поведение вредоносной нагрузки.

Другой пример - исследование образа постоянного запоминающего устройства, для которого не удалось найти описание разметки. Одни части образа содержат данные, другие - код. Потоковый дизассемблер можно направить только на те области, которые предположительно содержат код.

Итоги

Рассмотренные в этой главе инструменты не обязательно лучшие в своих категориях. Однако они широко доступны всем, кто занимается обратной разработкой двоичных файлов. Ещё важнее то, что именно подобные средства во многом определили направление развития IDA.

В следующих главах мы будем встречаться с такими возможностями снова. Знакомство с их происхождением заметно упростит понимание интерфейса IDA и многочисленных представлений информации, которые он предлагает.

Сноски

[1] Сайт Cygwin: <http://www.cygwin.com/>.
[2] Сайт MinGW: <http://www.mingw.org/>.
[3] Магическое число - особое значение метки, наличие которого, согласно спецификации формата, указывает на соответствие файла этому формату. Иногда выбор числа связан с забавной историей. Метка `MZ` в заголовках исполняемых файлов MS-DOS представляет собой инициалы Марка Збиковски, одного из первоначальных архитекторов MS-DOS. Хорошо известное шестнадцатеричное магическое число `0xcafebabe` файлов классов Java выбрали потому, что

эту последовательность цифр легко запомнить.

[4] Сайт PE Tools: <http://petools.org.ru/petools.shtml>.
[5] Сайт PEiD: <http://peid.info/>.
[6] Описание червя Gaobot: <http://securityresponse.symantec.com/security_response/writeup.jsp?docid=2003-112112-1102-99>.
[7] Подробнее о компоновке см. книгу Джона Левина «Компоновщики и загрузчики» (Морган Кауфман, 2000).
[8] Документация `objdump`: <http://www.sourceware.org/binutils/docs/binutils/objdump.html#objdump/>.
[9] Сайт GNU Binutils: <http://www.gnu.org/software/binutils/>.
[10] Документация `dumpbin`: <http://msdn.microsoft.com/en-us/library/c1h23y6c(VS.71).aspx>.
[11] Обзор декорирования имён: <http://en.wikipedia.org/wiki/Name_mangling>.
[12] Сайт diStorm: <http://www.ragestorm.net/distorm/>.
[13] Сайт NASM: <http://nasm.sourceforge.net/>.
[14] Сайт Metasploit: <http://www.metasploit.com/>.

Глава 3. Общие сведения об IDA Pro

Interactive Disassembler Professional, более известный как IDA Pro или просто IDA, - продукт компании Hex-Rays[1] из бельгийского Льежа. Главный создатель IDA - выдающийся программист Ильфак Гильфанов, которого обычно называют просто Ильфаком.

История IDA началась более десяти лет назад с консольной программы для MS-DOS. Это происхождение помогает понять особенности её интерфейса. В частности, для всех поддерживаемых платформ[2] поставляются варианты IDA без графического интерфейса, которые по-прежнему используют консольный стиль первоначальных версий для DOS.

В основе IDA лежит дизассемблер рекурсивного спуска, однако разработчики вложили значительные усилия в расширяющую его логику. Чтобы преодолеть один из главных недостатков рекурсивного спуска, IDA применяет множество эвристических приёмов и находит дополнительный код, который основной алгоритм мог пропустить.

Кроме самого дизассемблирования, IDA старается не только отличать представление данных от представления кода, но и определять точный тип показанных данных. Хотя код в IDA записан на языке ассемблера, одна из главных целей программы - создать картину, как можно более близкую к исходному тексту. Для этого IDA снабжает листинг сведениями о типах, а также выведенными именами переменных и функций. Такие пояснения сокращают количество необработанных шестнадцатеричных значений и увеличивают объём понятной пользователю символьной информации.

Отношение Hex-Rays к пиратству

Пользователю IDA следует знать несколько обстоятельств. IDA - главный продукт Hex-Rays, поэтому компания очень серьёзно относится к его несанкционированному распространению. В прошлом она наблюдала прямую связь между появлением пиратских версий IDA и падением продаж. Прежний издатель, DataRescue, дошёл даже до публикации имён нарушителей в своём «Зале позора»[3]. Для ограничения пиратства и соблюдения условий лицензии IDA использует несколько защитных механизмов.

Во-первых, каждая копия IDA содержит водяную метку, однозначно связывающую её с покупателем. Если такая копия появляется на сайте с нелегальными программами, Hex-Rays может установить первоначального владельца и впоследствии отказывать ему в продаже. На форуме поддержки IDA нередко обсуждают «утёкшие» копии программы.

Во-вторых, IDA проверяет, не работают ли в локальной сети другие копии с тем же лицензионным ключом. При запуске версии для Windows программа рассылает пакет UDP на порт `23945` и ожидает ответы экземпляров IDA из той же подсети. Число ответов сравнивается с количеством рабочих мест в лицензии. Если копий слишком много, IDA отказывается запускаться. При этом одна лицензия разрешает одновременно запускать несколько экземпляров IDA на одном компьютере.

Наконец, для каждого покупателя создаётся отдельный файл ключа. При запуске IDA ищет действительный файл `ida.key` и немедленно завершается, если не находит его. По этому же файлу определяется право на обновления. По сути, `ida.key` служит подтверждением покупки, поэтому его нужно беречь, чтобы не потерять возможность получать будущие версии.

Получение IDA Pro

Прежде всего, IDA - не свободная программа. Продажа IDA составляет часть дохода сотрудников Hex-Rays. Для знакомства с основными возможностями существует бесплатная версия[4] с ограниченной функциональностью, однако она отстаёт от актуальных выпусков. На момент написания книги это был урезанный вариант IDA 5.0, тогда как текущей являлась версия 6.1. Бесплатный выпуск подробнее рассматривается в приложении А.

Кроме того, Hex-Rays распространяла ограниченную демонстрационную копию[5] текущей версии. Если восторженные отзывы специалистов по обратной разработке ещё не убедили вас приобрести IDA, работа с бесплатным или демонстрационным вариантом поможет понять ценность самой программы и сопутствующей поддержки.

Варианты IDA

Начиная с версии 6.0 для Windows, Linux и OS X выпускались графический и консольный варианты IDA. Единообразный интерфейс на трёх платформах обеспечивала межплатформенная графическая библиотека Qt.

По функциональности IDA Pro предлагалась в двух редакциях: стандартной и расширенной. Они различались главным образом количеством поддерживаемых процессорных архитектур. Согласно перечню процессоров[6], стандартная редакция стоимостью около 540 долларов США поддерживала более 30 семейств, а почти вдвое более дорогая расширенная - более 50. Только расширенная редакция поддерживала, среди прочего, x64, AMD64, MIPS, PPC и SPARC.

Лицензии IDA

При покупке IDA предлагались два вида лицензий. Согласно сайту Hex-Rays[7], именная лицензия закрепляется за конкретным пользователем, который вправе применять её на любом числе своих компьютеров. Компьютерная лицензия закрепляется за определённым компьютером; на нём программой могут пользоваться разные люди, если одновременно работает только один из них.

Именная лицензия разрешает установить IDA на любое количество компьютеров, но запускать эти копии вправе только владелец лицензии. При наличии одной лицензии в каждый момент IDA может работать лишь на одном из этих компьютеров.

> **Примечание.** В отличие от многих лицензий на закрытые программы, лицензия IDA прямо разрешает пользователю выполнять обратную разработку самой IDA.

Покупка IDA

До версии 6.0 покупатель получал графическую версию для Windows и консольные варианты для Windows, Linux и OS X. Начиная с версии 6.0 требовалось выбрать операционную систему. Каждая копия IDA 6.x включала консольный и основанный на Qt графический варианты только для указанной системы. Дополнительные лицензии для других систем продавались дешевле основной.

Приобрести IDA можно было у официальных распространителей со страницы продаж или непосредственно у Hex-Rays по факсу либо электронной почте. Купленную копию доставляли на компакт-диске или предоставляли для загрузки; покупатель получал год поддержки и обновлений.

Помимо установщика IDA, диск содержал комплект разработчика IDA SDK и разные служебные программы. При загрузке через сеть пользователь обычно получал только установочный набор и должен был отдельно загружать остальные составляющие.

Известно, что Hex-Rays ограничивала продажи в некоторые страны из-за сложившейся там ситуации с пиратством. Компания также ведёт чёрный список пользователей, нарушивших лицензионные условия, и может отказаться работать как с ними, так и с их работодателями.

Обновление IDA

В меню справки IDA есть команда проверки доступных обновлений. Кроме того, по дате окончания поддержки в файле ключа программа автоматически предупреждает о приближении этого срока.

Для обновления обычно нужно отправить `ida.key` в Hex-Rays. Компания проверяет ключ и сообщает порядок получения новой версии. Если выпуск IDA слишком стар и уже не даёт обычного права на обновление, владельцу просроченного ключа стоит воспользоваться льготной ценой.

> **Предупреждение.** Если недостаточно строго хранить файл ключа, посторонний человек может запросить причитающееся вам обновление и тем самым лишить вас возможности обновить собственную копию IDA.

Перед обновлением любой версии настоятельно рекомендуется создать резервную копию установленной IDA или поставить новую версию в совершенно другой каталог. Иначе можно потерять изменённые файлы настройки. В новой версии соответствующие изменения придётся внести повторно.

Также потребуется перенести, заново скомпилировать или получить новые варианты пользовательских подключаемых модулей. Подробнее о них и порядке установки рассказывается в главе 17.

Источники поддержки IDA

У пользователя IDA неизбежно возникают вопросы. Если мы хорошо справились со своей задачей, в большинстве случаев этой книги будет достаточно. Если всё же понадобится дополнительная помощь, полезны следующие источники.

Официальная справка

В IDA встроена вызываемая из меню справочная система, однако в основном она описывает интерфейс и подсистему сценариев. Для SDK справки нет, а на вопросы вида «Как сделать то или иное?» система отвечает редко.

Страница поддержки и форумы Hex-Rays

На странице поддержки Hex-Rays[8] собраны ссылки на разные материалы об IDA, включая форумы для владельцев лицензий. Ильфак и другие основные разработчики Hex-Rays часто участвуют в обсуждениях.

Форумы служат и хорошей отправной точкой для неофициальной поддержки SDK: многие опытные пользователи готовы делиться практическими советами. На вопросы о SDK нередко отвечают: «Читайте включаемые файлы».

Обычная покупка IDA не включает официальную поддержку SDK, но Hex-Rays предлагала отдельный годовой договор стоимостью 10 000 долларов США. Отличный вводный материал по SDK - работа Стива Микаллефа «Создание подключаемых модулей IDA на C/C++»[9].

OpenRCE.org

На сайте <http://www.openrce.org/> сформировалось активное сообщество специалистов по обратной разработке. Там опубликовано множество статей о необычных способах применения IDA и работают пользовательские форумы. Как и на форумах Hex-Rays, здесь много опытных пользователей, готовых подсказать решение почти любой проблемы с IDA.

Форумы RCE

Форумы обратной разработки кода RCE на сайте <http://www.woodmann.com/> содержат бесчисленное множество сообщений об использовании IDA Pro. Тематика значительно шире одной IDA и охватывает многие инструменты и приёмы обратной разработки двоичных файлов.

IDA Palace

Хотя этот сайт не раз менял место размещения, IDA Palace[10] посвящён материалам, связанным с IDA. На нём можно найти ссылки на статьи об использовании программы, а также сценарии и подключаемые модули для расширения её возможностей.

Блог Ильфака

В блоге Ильфака[11] часто публиковались разборы применения IDA к разным задачам: от обычного дизассемблирования до отладки и анализа вредоносных программ. Другие сотрудники Hex-Rays также рассказывали о новейших и ещё разрабатываемых возможностях IDA.

Установка IDA

Когда первоначальная радость от получения нового блестящего диска IDA утихнет и вы приступите к установке, на диске обнаружатся каталоги `utilities` и `sdk`. В них находятся дополнительные служебные программы и комплект разработчика IDA соответственно. Мы подробно рассмотрим их далее.

В корневом каталоге диска расположен установочный файл. В Windows это обычная исполняемая программа установки, а в Linux и OS X - сжатый архив `.tar`.

Установка в Windows

Установить IDA в Windows очень просто. Установщик требует пароль, полученный вместе с диском или по электронной почте при сетевой покупке. После запуска появляется несколько информационных окон, и лишь одно из них требует выбора.

Как показано на рисунке 3-1, можно указать каталог установки или согласиться с предложенным. В дальнейшем выбранное расположение, независимо от способа выбора, обозначается как <КАТАЛОГ_IDA>.

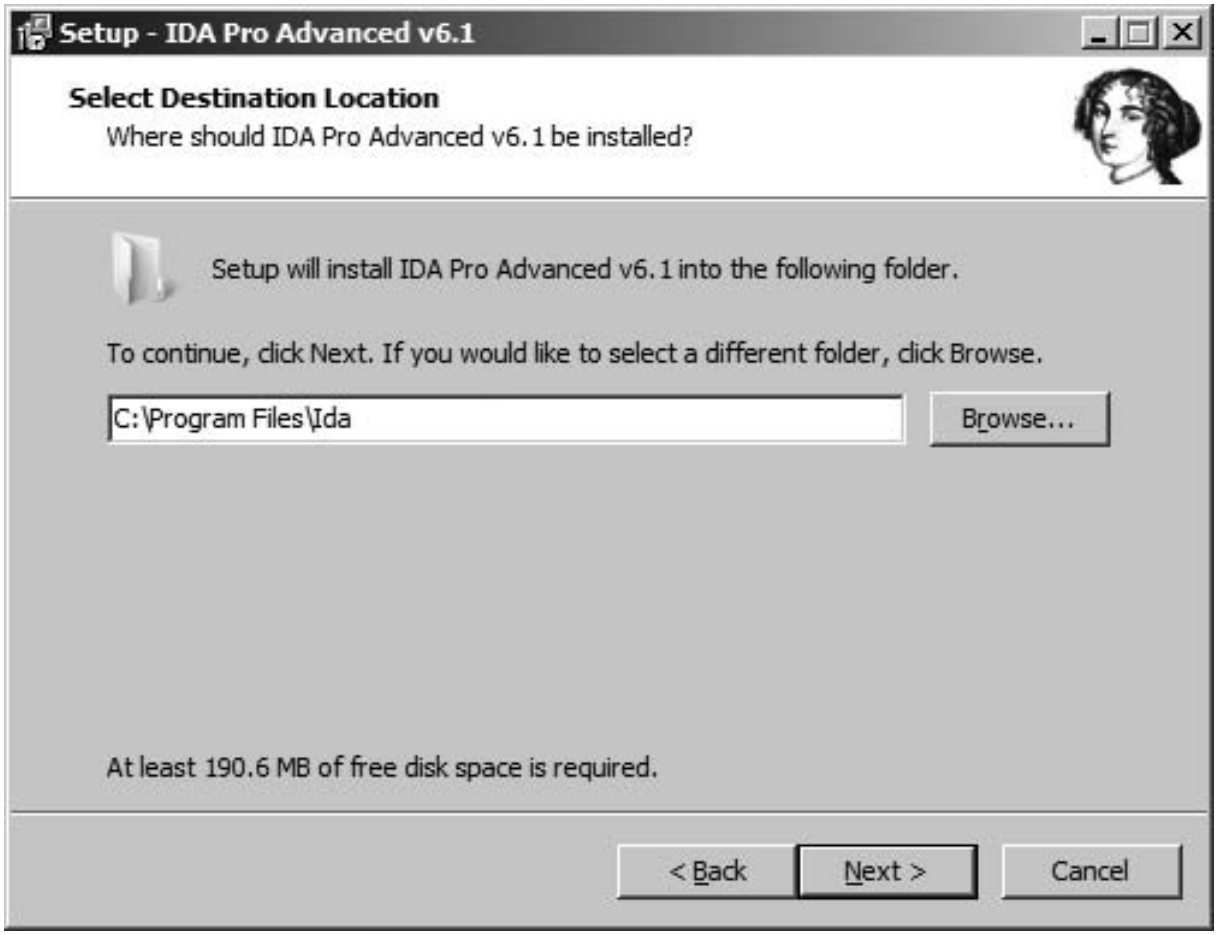


Рисунок 3-1. Выбор каталога установки

В каталоге IDA находится файл ключа `ida.key` и следующие исполняемые файлы:

• `idag.exe` - исходный графический вариант IDA для Windows; начиная с версии 6.2 его поставка прекращалась;
• `idaq.exe` - графический вариант для Windows на основе Qt, доступный с версии 6.0;
• `idaw.exe` - текстовый вариант IDA для Windows.

После перехода в IDA 6.0 на межплатформенную библиотеку Qt первоначальный графический вариант для Windows, `idag.exe`, был объявлен устаревшим, а начиная с версии 6.2 его планировали исключить из поставки.

Установка в OS X и Linux

Для установки в OS X или Linux распакуйте подходящий архив в выбранный каталог. В Linux команда может выглядеть так:

```
# tar -xvzf ida61l.tgz
```

В OS X используется похожая команда:

```
# tar -xvzf ida61m.tgz
```

В обоих случаях появится корневой каталог `ida` со всеми необходимыми файлами.

В OS X и Linux графическая программа называется `idaq`, консольная - `idal`. Консольный вариант внешне очень похож на консольную версию для Windows, показанную на рисунке 3-2.

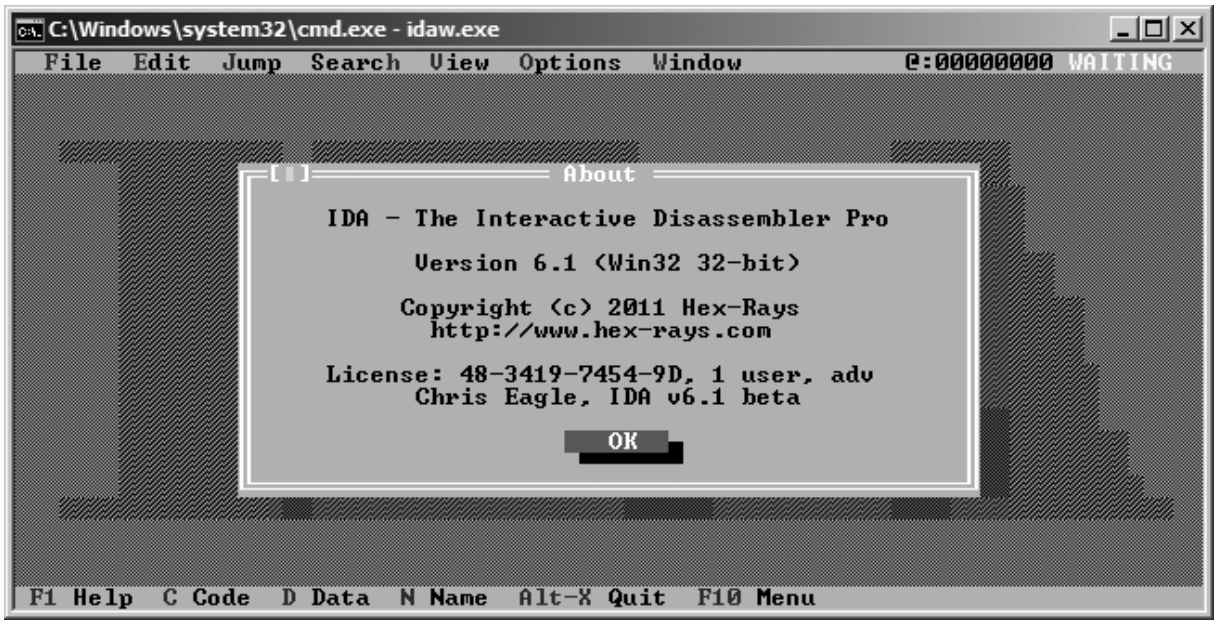


Рисунок 3-2. Консольная версия IDA Pro

Пользователям Linux может потребоваться проверить посредством `ldd`, доступны ли в системе все необходимые IDA общие библиотеки. В частности, подключаемый модуль IDAPython рассчитывал найти Python 2.6. Для выполнения требований IDA иногда приходилось обновлять Python или создавать символьные ссылки.

IDA и SELinux

В Linux с включённой SELinux при загрузке процессорного модуля IDA может сообщить, что ей «не удалось разрешить исполняемый стек для общего объекта». Проблему можно исправить отдельно для каждого модуля командой `execstack`:

```
execstack -c <КАТАЛОГ_IDA>/procs/pc.ilx
```

32-разрядная и 64-разрядная IDA

Владельцы расширенной редакции увидят по два варианта каждого исполняемого файла: например, `idag.exe` и `idag64.exe` либо `idaq` и `idaq64`. Вариант с суффиксом `64` способен дизассемблировать 64-разрядный код, но сами исполняемые файлы IDA оставались 32-разрядными.

Поэтому при запуске IDA в 64-разрядной системе всё вспомогательное программное обеспечение должно быть доступно в 32-разрядном варианте. Например, пользователю 64-разрядной Linux для сценариев IDAPython требовалась 32-разрядная версия Python. Правила совместного применения 32- и 64-разрядных программ приведены в документации операционной системы.

Структура каталога IDA

Перед началом работы необязательно сразу знать всё содержимое установленной IDA. Тем не менее сейчас удобно кратко рассмотреть её устройство. Понимание структуры каталогов станет важнее при переходе к расширенным возможностям.

В Windows и Linux перечисленные далее каталоги находятся в `<КАТАЛОГ_IDA>`, а в OS X - в `<КАТАЛОГ_IDA>/idaq.app/Contents/MacOS`.

cfg

Содержит файлы настройки: основной `ida.cfg`, файл графического интерфейса `idagui.cfg` и файл текстового интерфейса `idatui.cfg`. Полезные настройки IDA рассматриваются в главе 11.

idc

Содержит основные файлы встроенного языка сценариев IDC. Подробнее сценарии IDC рассматриваются в главе 15.

ids

Содержит символьные файлы, называемые в IDA файлами IDS. Они описывают общие библиотеки, на которые могут ссылаться загруженные в IDA программы. В них перечислены все экспортируемые библиотекой элементы, тип и количество параметров функций, тип возвращаемого значения и применяемое соглашение о вызовах.

loaders

Содержит расширения IDA, которые при загрузке распознают и разбирают известные форматы, например PE и ELF. Загрузочные модули рассматриваются в главе 18.

plugins

Содержит модули, добавляющие в IDA новые, чаще всего определённые пользователем возможности. Подключаемым модулям посвящена глава 17.

procs

Содержит процессорные модули, поддерживаемые установленной редакцией IDA. Они переводят машинный код на язык ассемблера и создают листинг, показанный в интерфейсе. Эти модули рассматриваются в главе 19.

sig

Содержит сигнатуры известного кода для сопоставления с образцами. Благодаря этому IDA распознаёт участки кода стандартных библиотек и способна заметно сократить время анализа. Сигнатуры создаются технологией быстрого определения и распознавания библиотек FLIRT, описанной в главе 12.

til

Содержит библиотеки типов, посредством которых IDA хранит схемы структур данных, характерные для разных библиотек компиляторов. Настройка библиотек типов рассматривается в главе 13.

Особенности интерфейса IDA

Происхождение IDA из MS-DOS заметно и сегодня. Независимо от выбранного интерфейса - текстового или графического - программа широко использует горячие клавиши. Само по себе это неплохо, но может привести к неожиданному результату, если пользователь думает, что вводит текст, а почти каждое нажатие запускает команду.

Такое случается и в графическом интерфейсе: вы устанавливаете указатель в нужное место, собираетесь что-либо изменить и ожидаете, что набранный текст появится под указателем. Однако IDA - не обычный текстовый редактор.

Почти все данные IDA принимает через диалоговые окна. Если вы хотите что-либо ввести, убедитесь, что перед вами открыто подходящее окно. Единственное исключение - редактирование шестнадцатеричных данных, доступное только в окне шестнадцатеричного представления.

Наконец, важно помнить: в IDA нет отмены действий. Если случайное нажатие вызвало команду горячей клавиши, не тратьте время на поиск отмены в меню - её там нет. Журнала команд, который позволил бы выяснить, что именно вы только что сделали, также не существует.

Итоги

Закончив с предварительными подробностями, можно перейти к полезной работе в IDA. В следующих главах вы научитесь выполнять первоначальный анализ файлов, понимать представления данных IDA и изменять их, чтобы глубже разобраться в поведении программы.

Часть II. Основы работы с IDA

Сноски

[1] На протяжении многих лет IDA продавала компания DataRescue. В январе 2008 года Ильфак передал продвижение и продажу IDA своей компании Hex-Rays.
[2] На момент написания книги поддерживались Windows, Linux и OS X.
[3] «Зал позора» был перенесён на сайт Hex-Rays: <http://www.hex-rays.com/idapro/hallofshame.html>.
[4] Страница бесплатной версии: <http://www.hex-rays.com/idapro/idadownfreeware.htm>.
[5] Страница демонстрационной версии: <http://www.hex-rays.com/idapro/idadowndemo.htm>.
[6] Перечень поддерживаемых процессоров: <http://www.hex-rays.com/idapro/idaproc.htm>.
[7] Условия заказа и лицензирования: <http://www.hex-rays.com/idapro/idaorder.htm>.
[8] Страница поддержки: <http://www.hex-rays.com/idapro/idasupport.htm>.
[9] Работа Стива Микаллефа: <http://www.binarypool.com/idapluginwriting/idapw.pdf>.
[10] Сайт IDA Palace: <http://old.idapalace.net/>.
[11] Блог Ильфака: <http://www.hexblog.com/>.

Глава 4. Начало работы с IDA

Пора наконец приступить к работе с IDA. Остальная часть книги посвящена возможностям программы и их применению в обратной разработке. В этой главе сначала рассматриваются варианты, предлагаемые при запуске IDA, затем объясняется, что происходит при открытии двоичного файла для анализа. В завершение мы кратко познакомимся с интерфейсом и подготовимся к последующим главам.

Для единообразия примеры здесь и далее приводятся в графическом интерфейсе Qt для Windows, если только задача не требует другого варианта IDA, например при отладке в Linux.

Запуск IDA

При каждом запуске IDA ненадолго появляется заставка со сводными сведениями о лицензии. Затем программа предлагает три способа перехода в рабочую среду, как показано на рисунке 4-1.

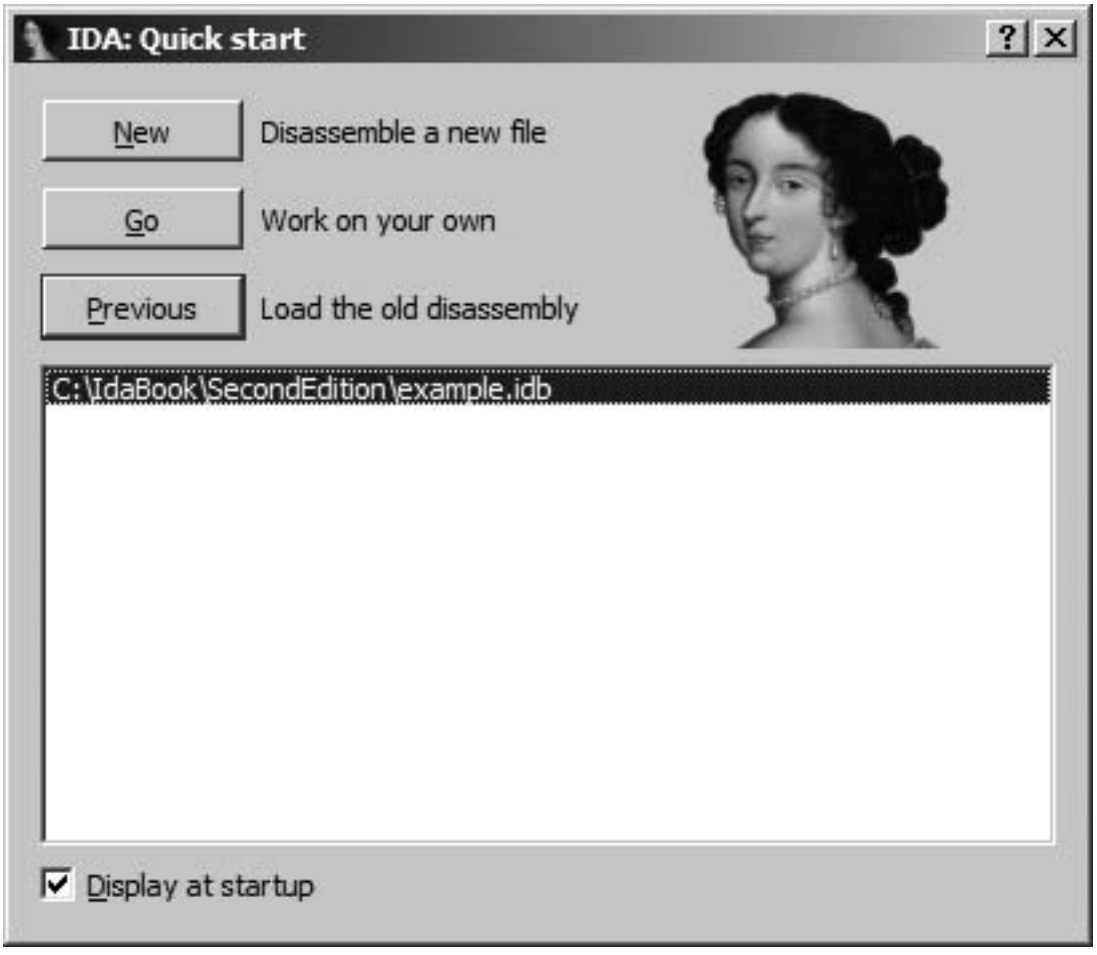


Рисунок 4-1. Запуск IDA

Если приветствие не нужно, снимите флажок его показа при запуске в нижней части окна. После этого будущие сеансы будут начинаться так, словно нажата кнопка перехода: сразу откроется пустая рабочая область IDA.

Чтобы позднее вернуть приветственное окно, удобное в том числе для открытия недавних файлов, измените параметр `DisplayWelcome` в разделе реестра IDA на `1`. Другой способ - выбрать в меню

окон команду восстановления скрытых сообщений; при этом снова появятся все ранее скрытые уведомления.

> **Примечание.** При установке в Windows IDA создаёт раздел реестра `HKEY_CURRENT_USER\Software\Hex-Rays\IDA`[1]. В нём хранятся многие параметры, изменяемые внутри программы, а не через файлы настройки. На других платформах эти значения записываются в двоичный файл `$HOME/.idapro/ida.reg`, который неудобно редактировать вручную.

Три варианта из приветственного окна по-разному переводят пользователя в рабочую область.

Создать

Открывается стандартное окно выбора анализируемого файла. После выбора появляются дополнительные окна с параметрами анализа, а затем файл загружается, анализируется и отображается.

Перейти

Загрузка прекращается, и IDA открывает пустую рабочую область. После этого двоичный файл можно перетащить в окно программы или открыть командой из меню файла.

Команда открытия показывает уже упомянутое окно выбора. По умолчанию IDA ограничивает список фильтром известных расширений. Измените или отключите фильтр, например выберите показ всех файлов, чтобы нужный файл действительно появился[2].

При таком открытии IDA пытается автоматически определить тип выбранного файла. Внимательно проверьте окно загрузки и убедитесь, что для обработки выбраны подходящие загрузочные модули.

Предыдущий

Этот вариант открывает один из недавних файлов, перечисленных непосредственно под кнопкой. В Windows список заполняется из подраздела `History` в разделе реестра IDA, а в других системах - из `ida.reg`.

Первоначально история содержит не более десяти элементов. Предел можно увеличить до ста, изменив соответствующий параметр в `idagui.cfg` или `idatui.cfg`; подробнее это описано в главе 11. Список истории - самый удобный способ продолжить работу с недавно использованной базой данных.

Загрузка файлов в IDA

При открытии нового файла появляется окно загрузки, показанное на рисунке 4-2. IDA формирует перечень возможных типов и выводит его в верхней части окна. Перечень соответствует загрузочным модулям, которые лучше всего подходят для выбранного файла.

Для его создания IDA поочерёдно запускает модули из каталога `loaders` и выясняет, какие из них распознают файл[3]. На рисунке 4-2 это удалось как загрузчику PE для Windows `pe.ldw`, так и

загрузчику EXE для MS-DOS `dos.ldw`. Такое совпадение закономерно: формат PE расширяет формат EXE для MS-DOS.

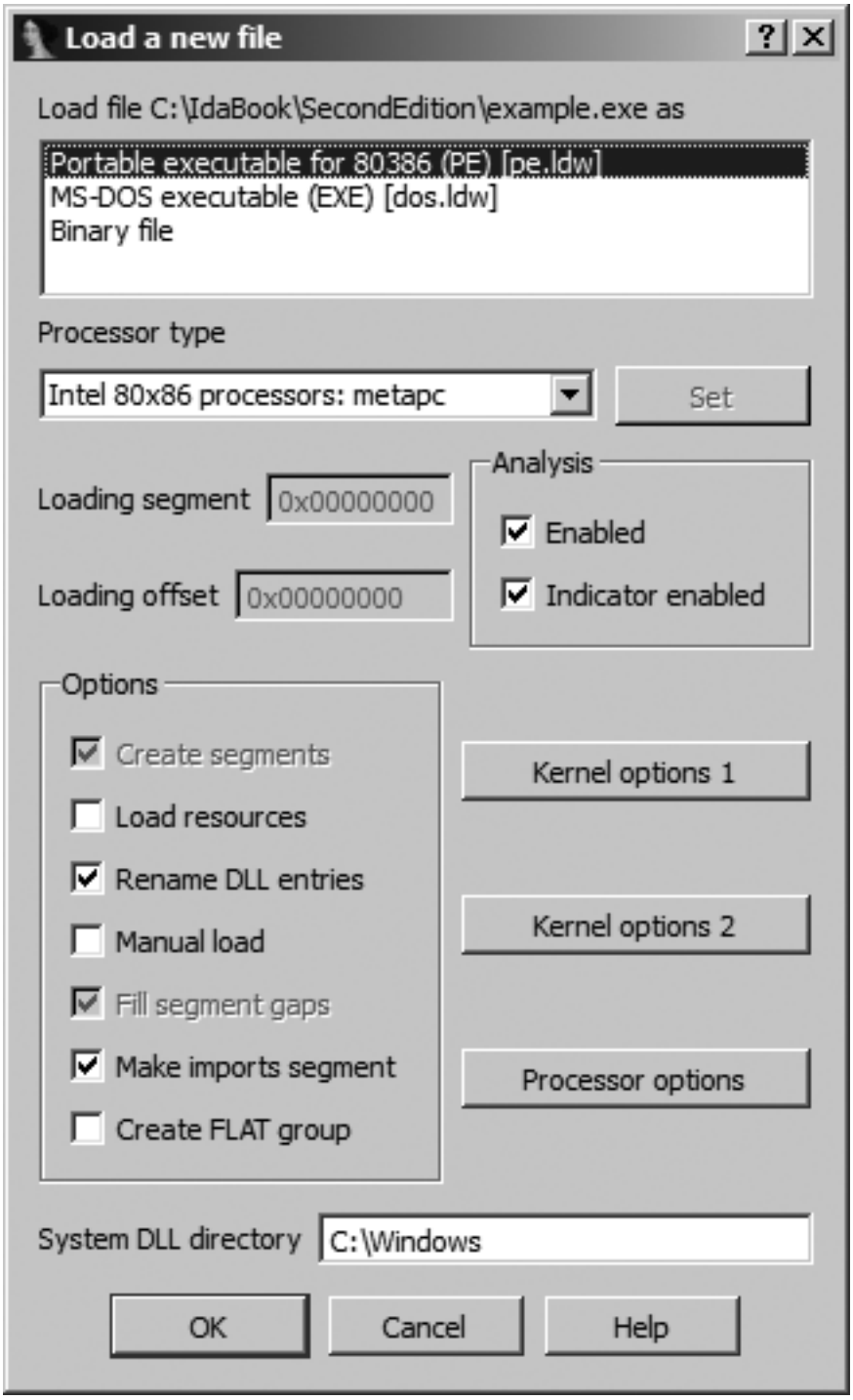


Рисунок 4-2. Окно загрузки нового файла IDA

Последний вариант, «двоичный файл», присутствует всегда. Это стандартный загрузчик IDA для неизвестных форматов и самый низкоуровневый способ открыть любой файл. Если предложено несколько загрузчиков и у вас нет сведений, противоречащих решению IDA, разумно сначала согласиться с выбранным по умолчанию.

Иногда двоичный загрузчик оказывается единственным вариантом. Это означает, что остальные модули не распознали файл. Если вы решите продолжить, выберите тип процессора в соответствии со своими знаниями о содержимом.

В раскрывающемся списке типа процессора указывается процессорный модуль из каталога `procs`, который будет применён при дизассемблировании. Обычно IDA правильно выбирает его по заголовкам исполняемого файла. Если определить процессор не удалось, перед продолжением нужно сделать это вручную.

Поля сегмента и смещения загрузки доступны только при сочетании двоичного формата с процессором семейства x86. Двоичный загрузчик не может извлечь схему расположения в памяти, поэтому введённые значения объединяются и образуют базовый адрес содержимого файла. Если вы забыли указать базовый адрес сразу, его можно позднее изменить командой перебазирования программы из меню редактирования сегментов.

Кнопка параметров ядра открывает настройки анализа, которыми IDA дополняет рекурсивный спуск. В подавляющем большинстве случаев стандартные значения дают наилучший листинг. Остальные сведения приведены в справке IDA.

Кнопка параметров процессора открывает настройки выбранного процессорного модуля. Они существуют не для каждого модуля. Справка по ним ограничена, поскольку доступные возможности сильно зависят от архитектуры и качества реализации конкретного модуля.

Оставшиеся флажки позволяют точнее управлять загрузкой. Они описаны в справке, применимы не ко всем форматам и обычно не требуют изменения. Отдельные случаи настройки рассматриваются в главе 21.

Применение двоичного загрузчика

При выборе двоичного загрузчика будьте готовы выполнить больше работы самостоятельно. Без сведений из заголовка именно пользователь должен решать задачи, которые более развитые модули обычно выполняют автоматически. Такой загрузчик может понадобиться при анализе образа постоянного запоминающего устройства или полезной нагрузки эксплойта, извлечённой из сетевого пакета либо журнала.

Если двоичный загрузчик используется вместе с процессорным модулем x86, появляется окно, показанное на рисунке 4-3. Поскольку распознаваемого заголовка нет, пользователь должен указать, считать ли код 16-разрядным или 32-разрядным. IDA умеет различать такие режимы также для ARM и MIPS.

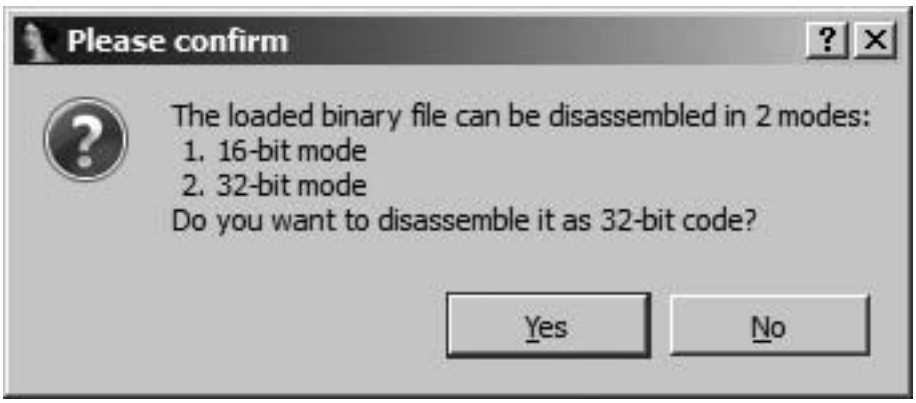


Рисунок 4-3. Выбор режима x86

В двоичном файле нет сведений о расположении в памяти - по крайней мере, в форме, которую IDA способна распознать. Для x86 базовый адрес задаётся в уже описанных полях сегмента и смещения. Для остальных процессоров IDA показывает окно организации памяти, представленное на рисунке 4-4.

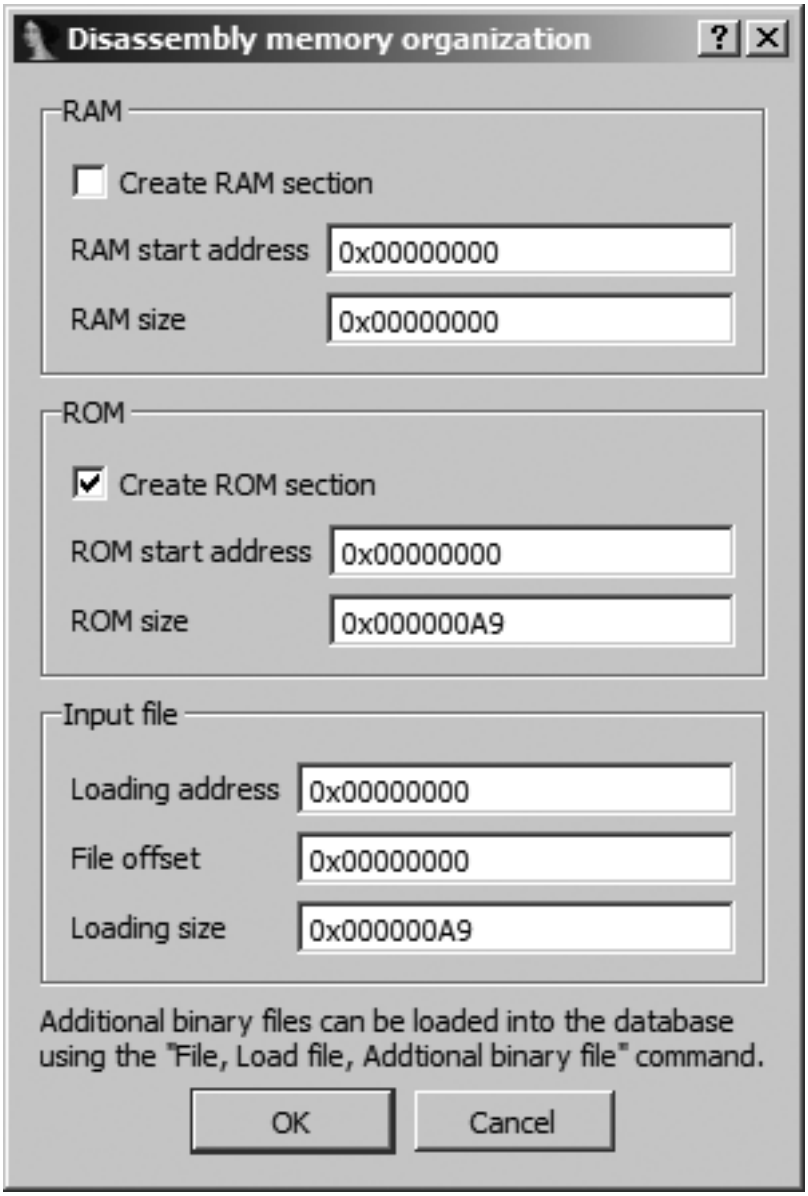


Рисунок 4-4. Окно организации памяти

В нём можно создать раздел оперативной памяти, раздел постоянной памяти или оба сразу и задать диапазон адресов каждого. Параметры входного файла определяют загружаемую часть - по умолчанию весь файл - и адрес, по которому она будет размещена.

На рисунке 4-5 показан последний этап двоичной загрузки: вежливое напоминание о предстоящей ручной работе.

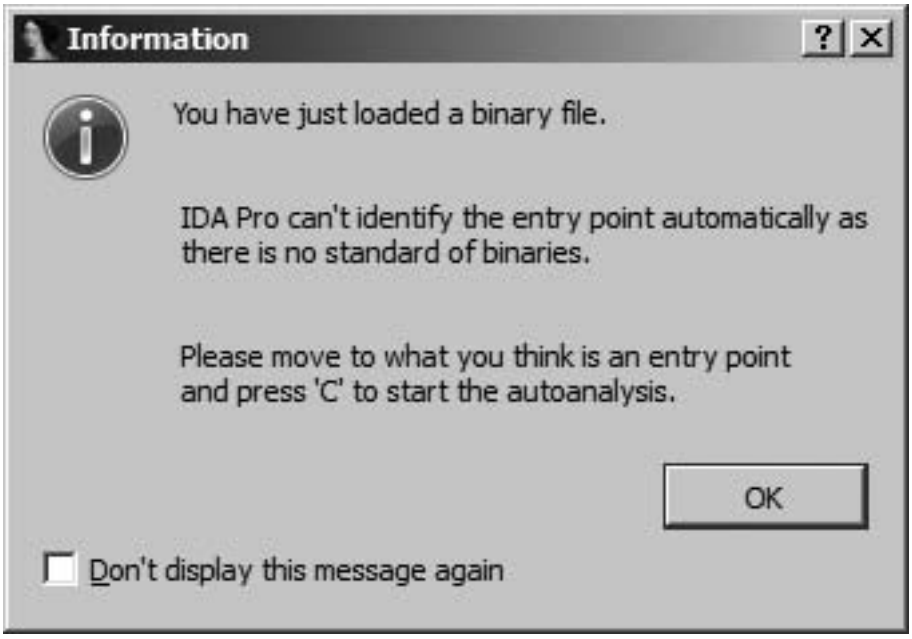


Рисунок 4-5. Загрузка двоичного файла

Сообщение подчёркивает, что без заголовка IDA не может отличить байты кода от байтов данных. Нужно указать хотя бы одну точку входа, преобразовав байт или байты по выбранному адресу в код. Для этого служит горячая клавиша `C`. Пока пользователь не обозначит хотя бы один байт как код, IDA не начинает первоначальное дизассемблирование двоичного файла.

Файлы базы данных IDA

После выбора параметров и подтверждения окна начинается настоящая загрузка. IDA помещает исполняемый файл в память, анализирует нужные области и создаёт базу данных. Её части сначала хранятся в четырёх файлах с тем же основным именем, что у исходной программы, и расширениями `.id0`, `.id1`, `.nam` и `.til`.

- `.id0` содержит базу данных на основе Б-дерева;
- `.id1` содержит флаги, описывающие каждый байт программы;
- `.nam` содержит индекс именованных мест программы, показанных в окне имён, которое рассматривается в главе 5;
- `.til` хранит локальные определения типов для конкретной базы.

Форматы этих файлов принадлежат IDA и почти не поддаются редактированию вне программы.

Для удобства при закрытии проекта четыре файла упаковываются и при желании сжимаются в один файл IDB. Говоря о базе данных IDA, обычно имеют в виду именно его. Несжатая база часто примерно в десять раз больше исходного двоичного файла.

После правильного закрытия базы файлы `.id0`, `.id1`, `.nam` и `.til` не должны оставаться в рабочем каталоге. Их наличие обычно означает, что база не была закрыта должным образом, например из-за сбоя IDA, и могла повредиться.

> **Предупреждения загрузчика.** Во время анализа загрузочному модулю иногда требуются дополнительные сведения пользователя. Например, файл PE может быть создан с отладочными данными PDB. Если IDA предполагает наличие базы программных данных, она спрашивает, следует ли искать соответствующий файл PDB в локальном хранилище символов и на сервере символов Microsoft.

> Другой пример связан с обфусцированными программами, в том числе вредоносными. Обфускация часто нарушает формальные требования к структуре файла и мешает загрузчикам, рассчитанным на правильный формат. Поэтому модуль PE проверяет таблицы импорта. Если они выглядят повреждёнными, IDA предупреждает, что файл мог быть упакован или изменён ради затруднения анализа, и советует перезагрузить его со снятым флажком создания раздела импорта. Примеры этой ошибки и способы работы с ней приведены в главе 21.

После создания базы IDA больше не нужен доступ к исходному исполняемому файлу, если только вы не собираетесь отлаживать его во встроенном отладчике. Это удобно с точки зрения безопасности: при анализе вредоносной программы специалисты могут передавать друг другу базу, не распространяя сам опасный файл. Не известно ни одного случая применения базы IDA как средства доставки вредоносной программы.

По сути, IDA - приложение для работы с базой данных. Новая база автоматически создаётся и заполняется из исполняемого файла. Все представления IDA - это разные виды одной базы, показывающие сведения в форме, удобной для специалиста по обратной разработке.

Изменения пользователя отражаются в представлениях и сохраняются в базе, но не воздействуют на исходный исполняемый файл. Сила IDA заключается в инструментах анализа и изменения данных внутри этой базы.

Создание базы данных IDA

После выбора файла и параметров IDA начинает создавать базу. Управление получает выбранный загрузочный модуль. Он читает файл с диска, разбирает известные заголовки, создаёт разделы кода и данных в соответствии с ними, определяет точки входа и возвращает управление IDA.

В этом отношении модуль IDA похож на загрузчик операционной системы. По сведениям из заголовков он определяет расположение программы в виртуальной памяти и соответствующим образом настраивает базу.

Затем механизм дизассемблирования по одному передаёт адреса выбранному процессорному модулю. Для каждого адреса тот определяет тип и длину инструкции, а также возможные места продолжения выполнения: например, является ли инструкция последовательной или выполняет переход.

Убедившись, что найдены все инструкции файла, IDA вторично проходит по их адресам и просит процессорный модуль сформировать текст каждой инструкции на языке ассемблера.

После дизассемблирования программа автоматически проводит дополнительный анализ и извлекает сведения, полезные специалисту. По завершении первоначальной обработки база может содержать перечисленные далее данные.

Определение компилятора

Знать компилятор программы часто полезно. Это помогает понять соглашения о вызовах функций и определить возможные связанные библиотеки. При загрузке IDA пытается установить, каким компилятором создан входной файл.

Если компилятор распознан, IDA ищет характерные для него шаблонные последовательности кода. Найденные функции выделяются цветом, чтобы сократить объём кода, который нужно исследовать

вручную.

Определение аргументов и локальных переменных функций

В каждой распознанной функции, то есть по адресам, являющимся целями инструкций `call`, IDA подробно анализирует поведение указателя стека. Благодаря этому программа распознаёт обращения к переменным в стеке и восстанавливает устройство кадра стека функции[4].

Имена таких переменных создаются автоматически с учётом того, используются ли они как локальные данные функции или как аргументы, переданные при вызове.

Сведения о типах данных

IDA знает распространённые библиотечные функции и требуемые ими параметры. На этой основе она добавляет в базу комментарии в местах передачи аргументов. Такие пояснения экономят аналитику немало времени, которое иначе пришлось бы тратить на поиск в справочниках по программным интерфейсам.

Закрытие базы данных IDA

При любом закрытии базы - вместе с IDA или перед переходом к другой базе - появляется окно сохранения, показанное на рисунке 4-6.

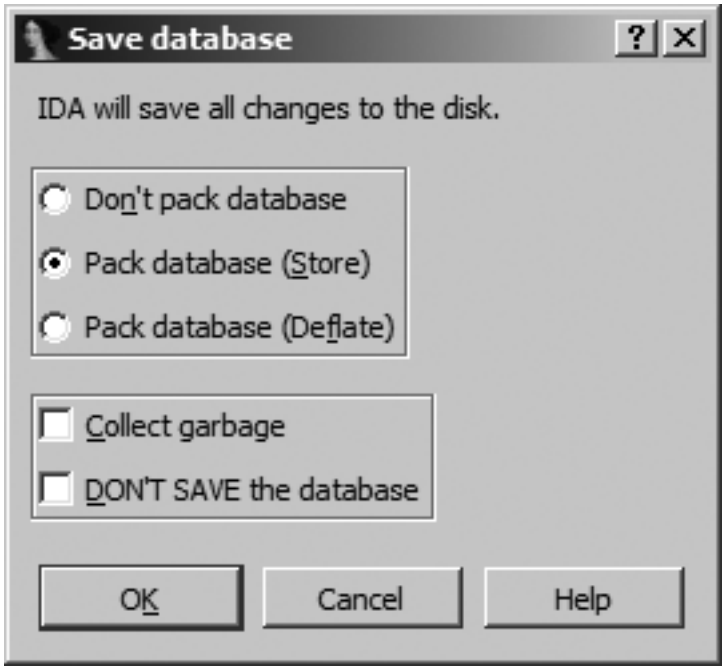


Рисунок 4-6. Окно сохранения базы данных

При первом сохранении имя базы создаётся из имени входного файла путём замены его расширения на `.idb`: например, из `example.exe` получается `example.idb`. Если расширения не было, `.idb` просто добавляется: `httpd` превращается в `httpd.idb`.

Доступны следующие варианты.

Не упаковывать базу

Изменения записываются в четыре составных файла, после чего рабочая область закрывается без создания IDB. При закрытии базы этот вариант не рекомендуется.

Упаковать без сжатия

Четыре составных файла помещаются в единый IDB. Существующий IDB перезаписывается без подтверждения. Сжатие не применяется. После создания архива составные файлы удаляются.

Упаковать со сжатием

Этот вариант совпадает с предыдущим, но составные файлы внутри IDB сжимаются.

Собрать мусор

Перед закрытием IDA удаляет из базы неиспользуемые страницы памяти. В сочетании со сжатием это создаёт IDB наименьшего размера. Обычно сбор мусора нужен только при серьёзном недостатке места на диске.

Не сохранять базу

Может показаться странным намеренно отказываться от сохранения работы. Однако это единственный способ отбросить все изменения, сделанные после последнего сохранения. IDA удаляет четыре составных файла, не трогая существующий IDB. Ближе всего к отмене или возврату в IDA именно этот вариант.

Повторное открытие базы

Обычно вернуться к существующей базе очень просто: достаточно выбрать её любым способом открытия файла в IDA[5]. Во второй и последующие разы база открывается гораздо быстрее, поскольку повторный анализ не требуется. Кроме того, рабочая область восстанавливается в том состоянии, в котором была закрыта.

Но иногда IDA аварийно завершается - из-за собственной ошибки или ошибки новейшего подключаемого модуля. Открытая база при этом может остаться повреждённой. После перезапуска и попытки открыть её IDA показывает одно из окон на рисунках 4-7 и 4-8.

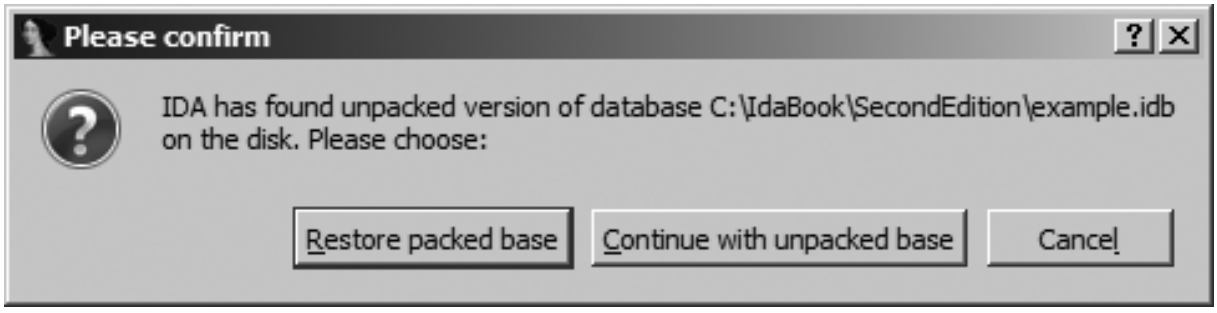


Рисунок 4-7. Окно восстановления базы данных

При сбое IDA не успевает закрыть активную базу, поэтому промежуточные файлы не удаляются. Если с этой базой уже работали раньше, одновременно могут существовать IDB и потенциально повреждённые промежуточные файлы. IDB содержит последнее заведомо исправное состояние, а промежуточные файлы - изменения после последнего сохранения.

В такой ситуации IDA предлагает вернуться к сохранённой версии или продолжить работу с открытым, возможно повреждённым вариантом. Продолжение с неупакованной базой вовсе не гарантирует восстановления работы. Скорее всего, данные находятся в несогласованном состоянии, и программа покажет окно исправления.

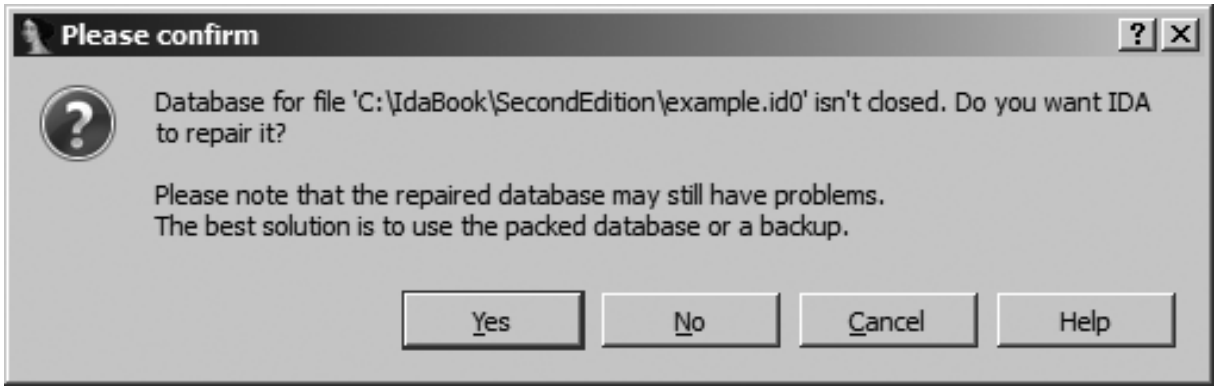


Рисунок 4-8. Окно исправления базы данных

Здесь сама IDA советует восстановиться из упакованных данных, поэтому выбор исправленной базы остаётся на ваш страх и риск. Если активную базу ещё ни разу не сохраняли и после сбоя остались только промежуточные файлы, окно исправления появится при следующей попытке открыть исходную программу.

Знакомство с рабочей областью IDA

Вам предстоит провести немало времени перед рабочей областью IDA, поэтому стоит познакомиться с её частями. На рисунке 4-9 показано стандартное расположение элементов. Поведение интерфейса во время анализа файла описано в следующем разделе.

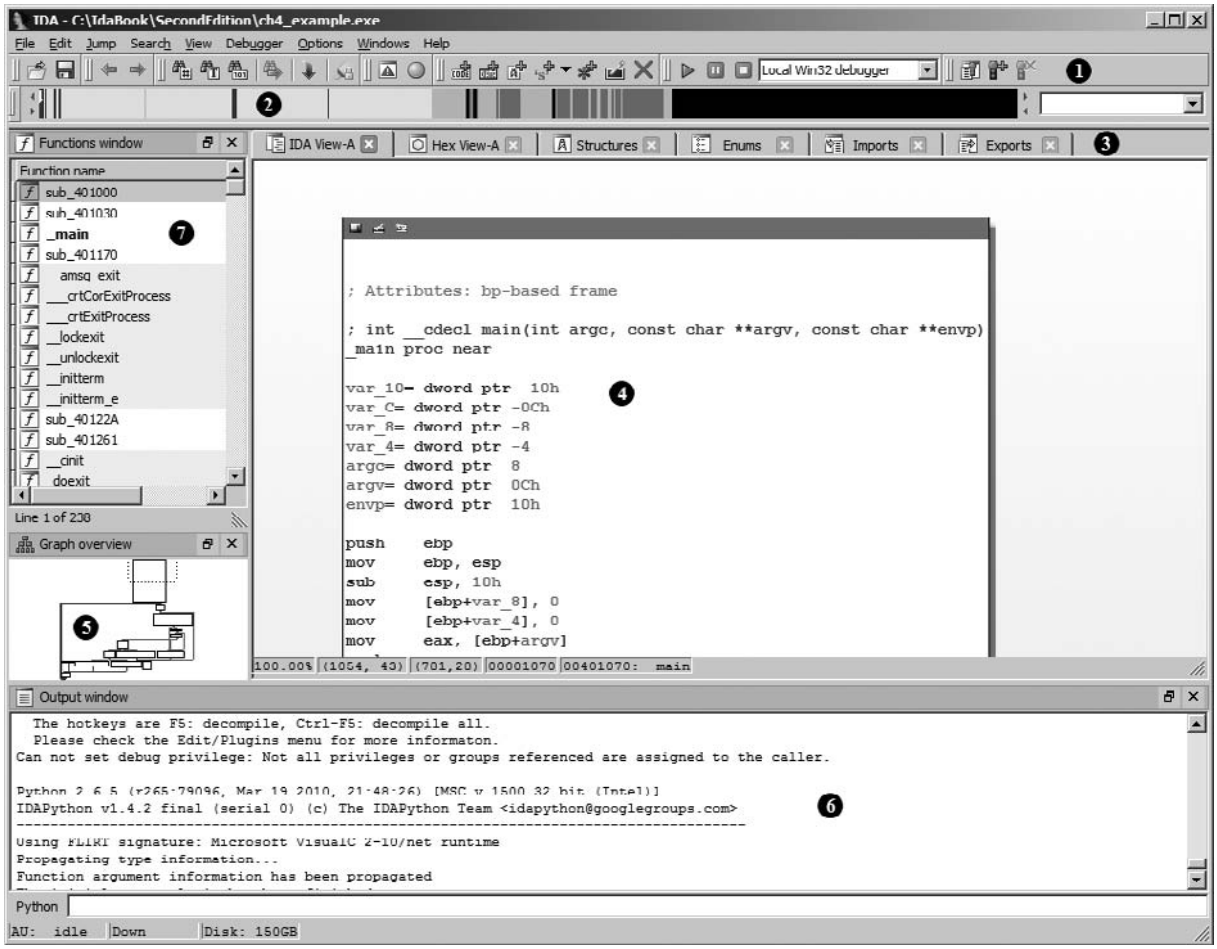


Рисунок 4-9. Рабочая область IDA

1. Панели инструментов

Область 1 содержит кнопки наиболее часто применяемых операций. Панели добавляются и удаляются через меню просмотра, а перетаскивание позволяет расположить их удобным образом.

На рисунке показан основной режим с одним рядом кнопок. Расширенный режим, включаемый в меню панелей инструментов, содержит три полных ряда.

2. Полоса навигации

Горизонтальная цветная полоса 2 - обзорный навигатор IDA, также называемый полосой навигации. Она линейно представляет адресное пространство загруженного файла; по умолчанию показан весь диапазон двоичного файла.

Щелчок правой кнопкой позволяет изменить масштаб. Цвета соответствуют видам содержимого, например коду и данным. Небольшой указатель текущего положения, по умолчанию жёлтый, отмечает адрес, который сейчас показан в окне дизассемблирования.

При наведении на любую часть полосы всплывает описание соответствующего места в файле. Щелчок переносит представление дизассемблирования к выбранному адресу. Цвета настраиваются через общие параметры.

Полосу можно перетащить из основной рабочей области и получить отдельный обзорный навигатор, показанный на рисунке 4-10. Там также видны указатель текущего положения - направленная вниз стрелка слева от отметки 1 - и легенда цветов функциональных групп.

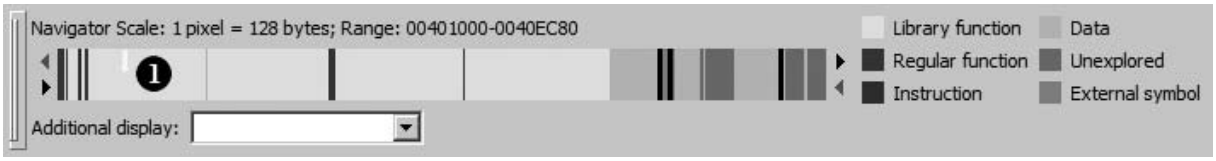


Рисунок 4-10. Обзорный навигатор

3. Вкладки представлений

Для каждого открытого представления данных создаётся вкладка 3. В представлениях отображаются извлечённые из двоичного файла сведения, то есть разные виды базы. Большая часть анализа выполняется именно через них.

На рисунке 4-9 открыты представление IDA, список функций и обзор графа. Дополнительные окна доступны через подменю открытия представлений. Там же можно восстановить случайно или намеренно закрытое окно.

4. Представление дизассемблирования

Основное представление данных отмечено цифрой 4. Оно поддерживает два вида: граф, выбранный по умолчанию, и листинг.

В режиме графа IDA показывает блок-схему одной функции. Вместе с обзором графа она помогает понять поток выполнения по наглядной структуре функции. Когда активно представление IDA, пробел переключает граф и листинг.

Чтобы сделать листинг стандартным, откройте вкладку графа в общих параметрах и снимите флажок использования графа по умолчанию, как показано на рисунке 4-11.

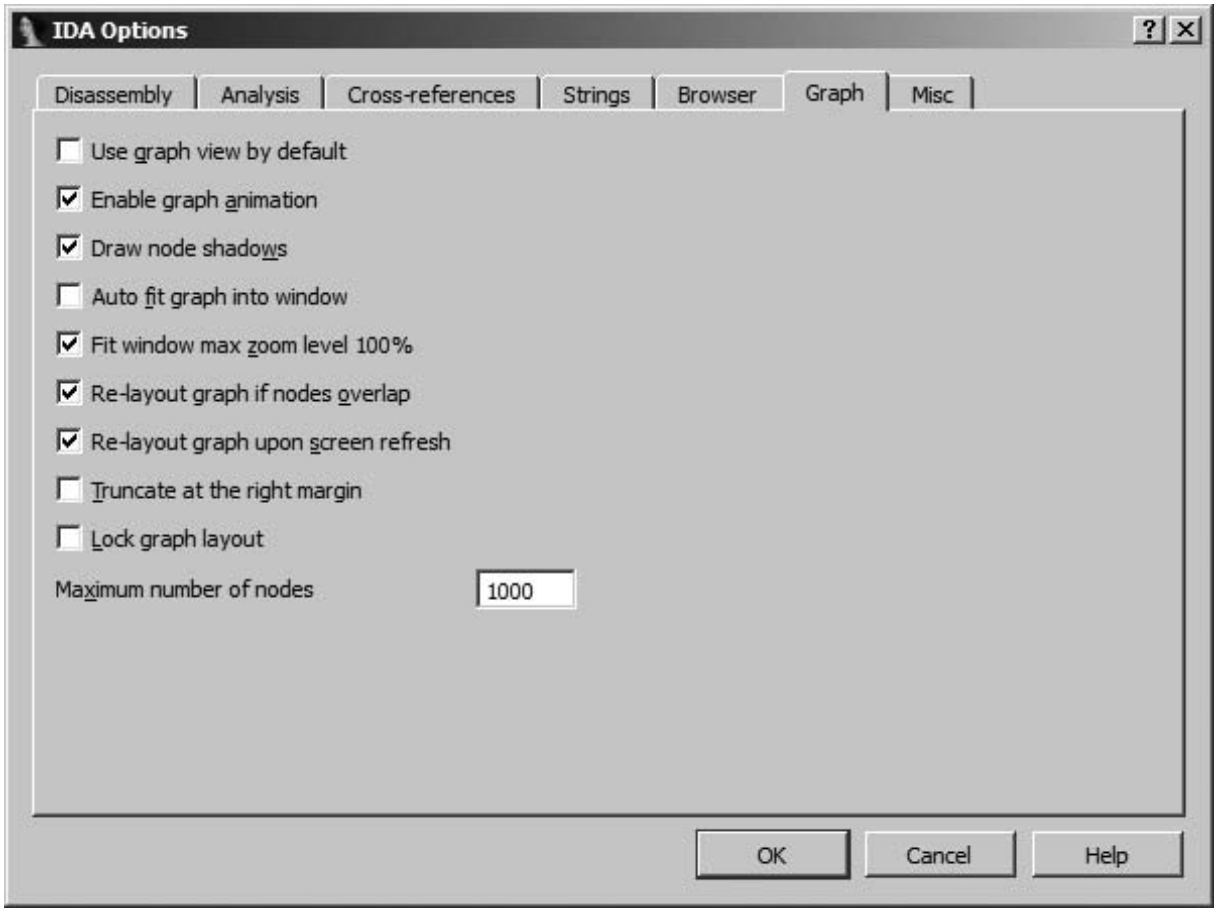


Рисунок 4-11. Параметры графа IDA

5. Обзор графа

Граф функции редко целиком помещается на экране. Уменьшенный обзор 5, доступный только в режиме графа, показывает его общую структуру. Пунктирный прямоугольник соответствует видимой области. Щелчок в обзоре перемещает основное представление в выбранное место.

6. Окно вывода

В окне 6 появляются информационные сообщения IDA: ход анализа файла и ошибки запрошенных пользователем операций. По назначению оно примерно соответствует консоли.

7. Окно функций

Окно 7 завершает набор стандартных элементов. Оно подробнее рассматривается в главе 5.

Поведение интерфейса во время первоначального анализа

При первоначальном автоматическом анализе нового файла рабочая область непрерывно меняется. Наблюдая за ней, можно понять ход обработки. В частности:

• в окне вывода появляются сообщения о ходе работы;
• формируются начальное положение и содержимое окна дизассемблирования;
• заполняется список функций, который затем периодически обновляется;
• полоса навигации меняется по мере распознавания кода и данных, выделения функций и определения библиотечного кода посредством сопоставления с образцами;
• указатель текущего положения движется по полосе и показывает анализируемые области.

Ниже приведён характерный вывод первоначального анализа новой программы. Сообщения образуют рассказ о процессе и позволяют проследить последовательность действий IDA.

```
Loading file 'C:\IdaBook\ch4_example.exe' into database...
Detected file format: Portable executable for 80386 (PE)
  0. Creating a new segment  (00401000-0040C000) ... ... OK
  1. Creating a new segment  (0040C000-0040E000) ... ... OK
  2. Creating a new segment  (0040E000-00411000) ... ... OK
Reading imports directory...
  3. Creating a new segment  (0040C120-0040E000) ... ... OK
Plan  FLIRT signature: Microsoft VisualC 2-10/net runtime
autoload.cfg: vc32rtf.sig autoloads mssdk.til
Assuming __cdecl calling convention by default
main() function at 401070, named "_main"
Marking typical code sequences...
Flushing buffers, please wait...ok
File 'C:\IdaBook\ch4_example.exe' is successfully loaded into the database.
Compiling file 'C:\Program Files\IdaPro\idc\ida.idc'...
Executing function 'main'...
Compiling file 'C:\Program Files\IdaPro\idc\onload.idc'...
Executing function 'OnLoad'...
IDA is analysing the input file...
X You may start to explore the input file right now.
------------------------------------------------------------------------------
Python 2.6.5 (r265:79096, Mar 19 2010, 21:48:26) [MSC v.1500 32 bit (Intel)]
IDAPython v1.4.2 final (serial 0) (c) The IDAPython Team
<idapython@googlegroups.com>
------------------------------------------------------------------------------
Using FLIRT signature: Microsoft VisualC 2-10/net runtime
Propagating type information...
Function argument information has been propagated
Y The initial autoanalysis has been finished.
```

Особенно полезны две строки. Сообщение X говорит, что анализ продвинулся достаточно далеко и уже можно перемещаться по представлениям. Но перемещение не означает изменение: редактировать базу следует лишь после завершения анализа. Иначе механизм автоматической обработки позднее способен изменить вашу правку или из-за неё выполнить свою работу неправильно.

Сообщение Y означает, что автоматические изменения представлений закончены. Теперь базу можно безопасно редактировать.

Советы по организации рабочей области

IDA предоставляет огромный объём сведений, поэтому интерфейс легко перегрузить. Несколько советов помогут использовать его эффективнее:

• чем больше места на экране отведено IDA, тем удобнее работа; это хороший повод приобрести очень большой монитор или даже два;
• не забывайте о подменю открытия представлений: через него восстанавливаются случайно закрытые окна;
• команда сброса рабочей области быстро возвращает исходную раскладку;

• командой сохранения рабочей области можно записать особенно удобное расположение окон, а командой загрузки - быстро к нему вернуться;
• шрифт изменяется только в окне дизассемблирования, как в графе, так и в листинге; для этого служит команда выбора шрифта в меню параметров.

Сообщение об ошибках

Как и в любой программе, в IDA иногда встречаются ошибки. Если вам кажется, что вы нашли одну из них, можно рассчитывать на чрезвычайно отзывчивую поддержку Hex-Rays. Не удивляйтесь, если уже в течение суток на запрос ответит сам Ильфак.

Сообщить об ошибке можно письмом на `support@hex-rays.com` или через соответствующий раздел форума Hex-Rays. В любом случае сначала убедитесь, что проблема воспроизводится, и будьте готовы предоставить компании связанную с ней базу данных.

Помните, что поддержка SDK предоставляется Hex-Rays за отдельную плату. Если ошибка относится к установленному вами подключаемому модулю, нужно обратиться к его автору. При разработке собственного модуля остаётся воспользоваться форумами пользователей IDA и надеяться на помощь коллег.

Итоги

Знание рабочей области заметно улучшает работу с IDA. Обратная разработка двоичного кода и без борьбы с инструментами достаточно сложна. Параметры первоначальной загрузки и последующий автоматический анализ закладывают основу всей дальнейшей работы.

Иногда, особенно для простого файла, автоматического анализа оказывается достаточно. Но если вам интересно, в чём заключается интерактивность IDA, пора глубже изучить многочисленные представления данных. В следующих главах будут рассмотрены основные окна, случаи их применения и способы использования для уточнения и обновления базы.

Сноски

[1] В старых версиях IDA применялся раздел `HKEY_CURRENT_USER\Software\Datarescue\IDA`.
[2] В системах, отличных от Windows, исполняемые файлы нередко вообще не имеют расширения.
[3] Загрузочные модули IDA подробнее рассматриваются в главе 18.
[4] Кадры стека подробнее рассматриваются в главе 6.
[5] Если только файл не называется `rocket_science.idb`.

Глава 5. Представления данных IDA

К этому моменту вы уже должны уверенно загружать двоичные файлы в IDA и позволять программе творить свою магию, пока сами наслаждаетесь любимым напитком. После завершения первоначального анализа пора брать управление в свои руки.

Лучший способ познакомиться с представлениями IDA - просто просмотреть вкладки, которые программа заполняет сведениями о двоичном файле. По мере освоения интерфейса сеансы обратной разработки будут становиться быстрее и результативнее.

Прежде чем переходить к основным окнам, полезно усвоить несколько общих правил.

В IDA нет отмены

Если случайное нажатие неожиданно изменило базу, восстанавливать прежнее состояние представлений придётся самостоятельно.

Почти у каждого действия есть пункт меню, горячая клавиша и кнопка

Панель инструментов IDA широко настраивается, как и соответствие горячих клавиш командам меню.

Щелчок правой кнопкой открывает полезное контекстное меню

Такое меню не содержит абсолютно всех допустимых в выбранном месте действий, но хорошо напоминает о наиболее распространённых операциях.

Теперь рассмотрим главные представления данных IDA.

Основные представления IDA

В стандартной настройке IDA 6.1 во время загрузки и первоначального анализа нового файла создаёт семь окон. Доступ к ним предоставляют вкладки непосредственно под полосой навигации, ранее показанной на рисунке 4-9.

Сразу видны представление IDA, окно функций и окно вывода. Любое окно из этой главы можно открыть через подменю представлений в меню просмотра, даже если оно не появляется по умолчанию. Об этом стоит помнить: случайно закрыть окно довольно легко.

Клавиша `Esc` - одна из самых полезных в IDA. Когда активно окно дизассемблирования, она действует подобно кнопке возврата в веб-обозревателе и очень удобна для навигации, подробно описанной в главе 6. Но в любом другом активном окне `Esc` закрывает это окно. Иногда именно это и требуется, а иногда вам сразу захочется вернуть закрытую вкладку.

Окно дизассемблирования

Окно дизассемблирования, также называемое представлением IDA, служит главным средством анализа и изменения двоичного файла. Поэтому особенно важно хорошо понимать показанные в нём сведения.

Доступны два вида: стандартный граф и текстовый листинг. Большинство пользователей предпочитает один из них в зависимости от того, как им удобнее воспринимать поток выполнения программы.

Чтобы сделать текстовый листинг стандартным, откройте общие параметры и на вкладке графа отключите его использование по умолчанию. Когда окно дизассемблирования активно, пробел в любой момент переключает граф и листинг.

Граф IDA

На рисунке 5-1 показана очень простая функция в виде графа. Такое представление напоминает блок-схему: функция разделена на базовые блоки[1], благодаря чему поток управления между ними виден наглядно.

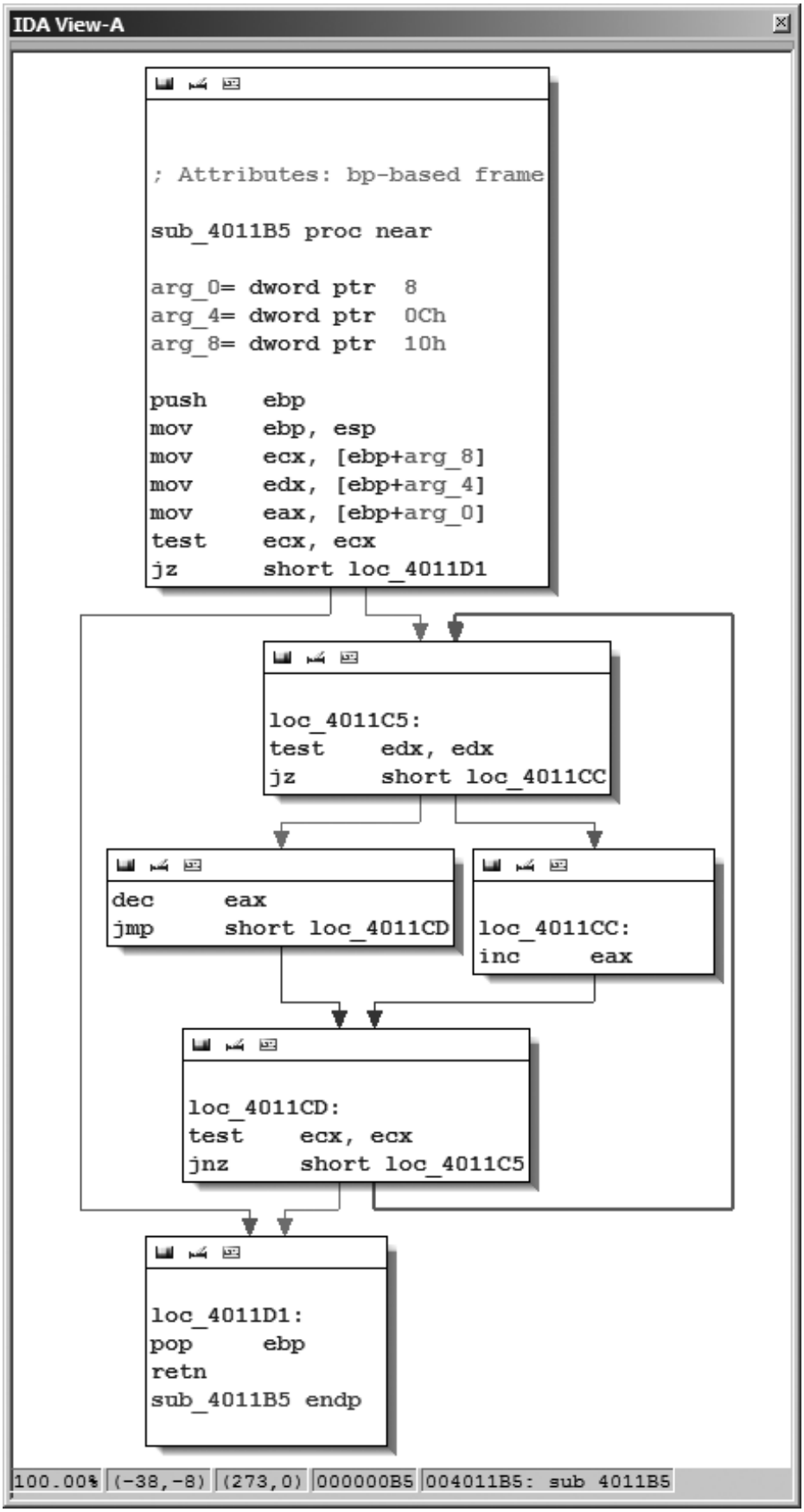


Рисунок 5-1. Граф IDA

Цветные стрелки обозначают разные виды потока[2] между блоками. Из блока, завершающегося условным переходом, выходят два пути:

• зелёная по умолчанию стрелка «да» означает, что переход выполнен;
• красная по умолчанию стрелка «нет» означает, что переход не выполнен.

Если возможен только один последующий блок, к нему ведёт обычная стрелка, по умолчанию синяя.

В режиме графа IDA одновременно показывает одну функцию. Масштаб изменяется сочетанием `Ctrl` с колёсиком мыши. С клавиатуры для увеличения используется `Ctrl` и `+`, а для уменьшения - `Ctrl` и `-`; знаки берутся с цифровой части клавиатуры.

Граф большой или сложной функции может стать перегруженным и неудобным для перемещения. Тогда помогает окно обзора графа на рисунке 5-2. В нём всегда видна полная структура блоков, а пунктирная рамка обозначает область, показанную в основном окне. Перетаскивая рамку, можно быстро перейти к нужному месту.

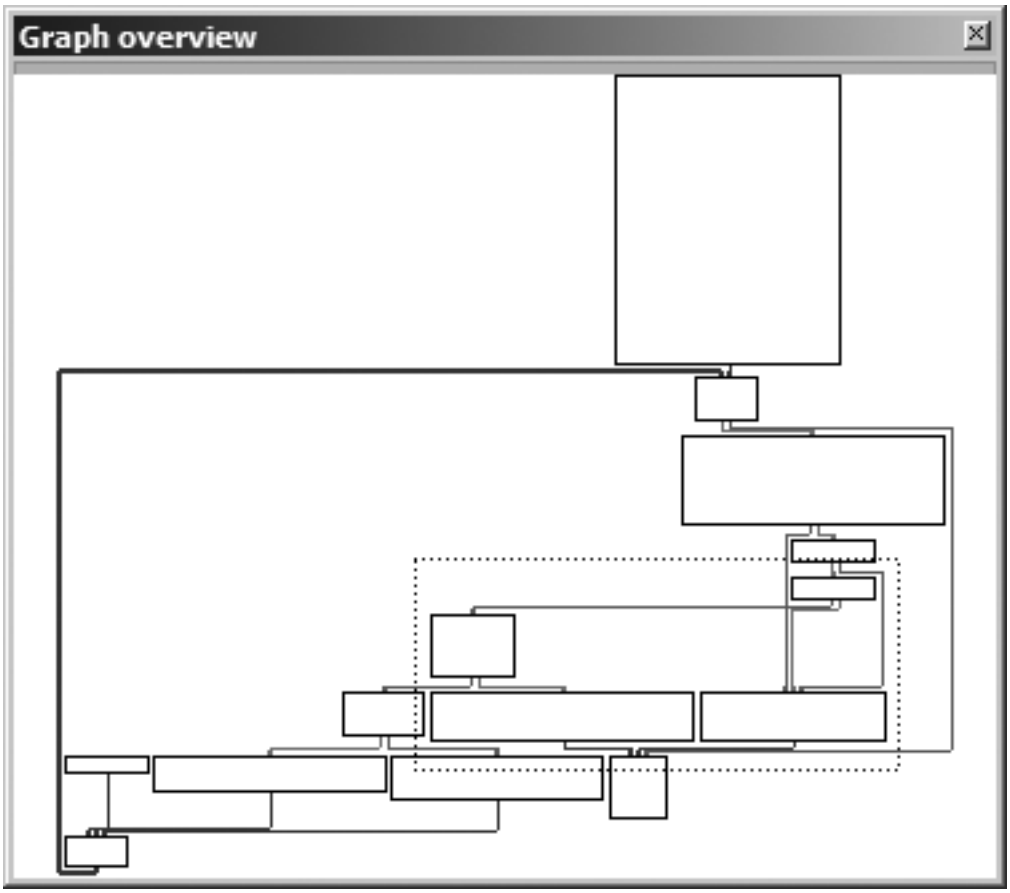


Рисунок 5-2. Окно обзора графа

Перемещение

Кроме перетаскивания рамки в обзоре граф можно перемещать, перетаскивая его фон.

> **Неужели здесь чего-то не хватает?** В графе может показаться, что для каждой строки доступно меньше сведений. IDA намеренно скрывает традиционные элементы, например виртуальные адреса, чтобы базовые блоки занимали меньше места. Дополнительные части строк включаются на вкладке дизассемблирования в общих параметрах. Например, после включения префиксов строк к каждой инструкции добавляется виртуальный адрес, и граф с рисунка 5-1 принимает вид, показанный на рисунке 5-3.

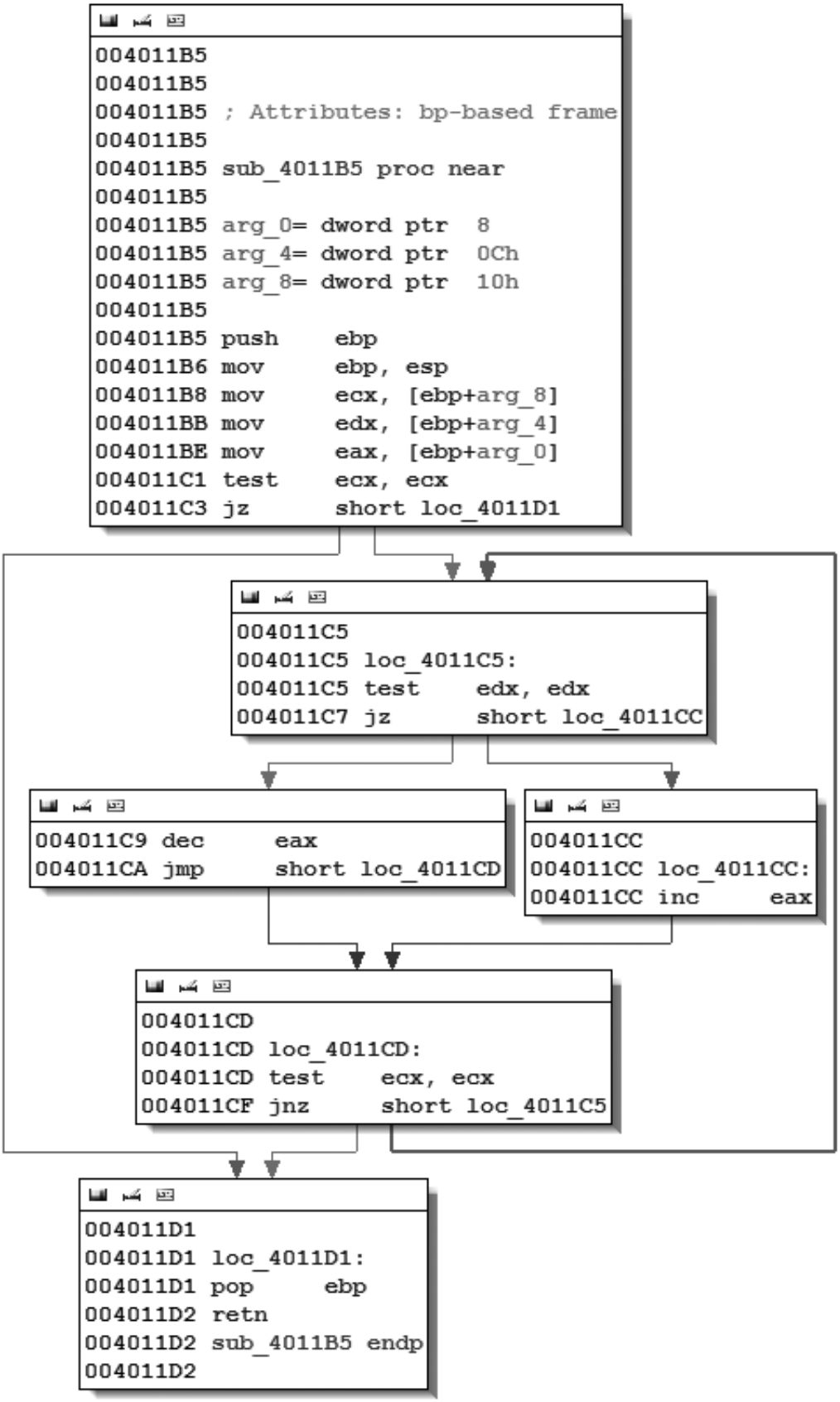


Рисунок 5-3. Граф с включёнными префиксами строк

Перестановка блоков

Отдельный блок можно перетащить за строку заголовка. Учтите, что IDA лишь минимально меняет маршруты связанных с ним рёбер. Вершины рёбер можно перемещать вручную. Чтобы добавить новую вершину, удерживайте `Shift` и дважды щёлкните нужное место ребра.

Вернуться к стандартной раскладке можно через контекстную команду размещения графа.

Группировка и сворачивание блоков

Один или несколько блоков можно объединить и свернуть, уменьшив перегруженность изображения. Особенно удобно так отмечать уже исследованные области. Для сворачивания откройте контекстное меню заголовка блока и выберите группировку узлов.

Дополнительные окна дизассемблирования

Чтобы одновременно видеть графы двух функций, откройте ещё одно окно дизассемблирования через меню представлений. Первое окно называется `IDA View-A`, следующие - `IDA View-B`, `IDA View-C` и так далее.

Окна независимы: можно одновременно держать граф в одном, листинг в другом или три разных графа в трёх окнах.

Этими примерами возможности управления не исчерпываются. Дополнительная работа с графами рассматривается в главе 9 и во встроенной справке IDA.

Текстовый листинг IDA

Текстовое окно - традиционный способ просмотра и изменения созданного IDA листинга. В отличие от графа одной функции оно показывает весь дизассемблированный текст программы и является единственным способом просматривать области данных двоичного файла. Все сведения из графа в той или иной форме доступны и здесь.

На рисунке 5-4 показан листинг той же функции, что на рисунках 5-1 и 5-3. Инструкции расположены линейно, а виртуальные адреса видны по умолчанию. Обычно адрес имеет вид `[ИМЯ_РАЗДЕЛА]:[ВИРТУАЛЬНЫЙ_АДРЕС]`, например `.text:004011C1`.

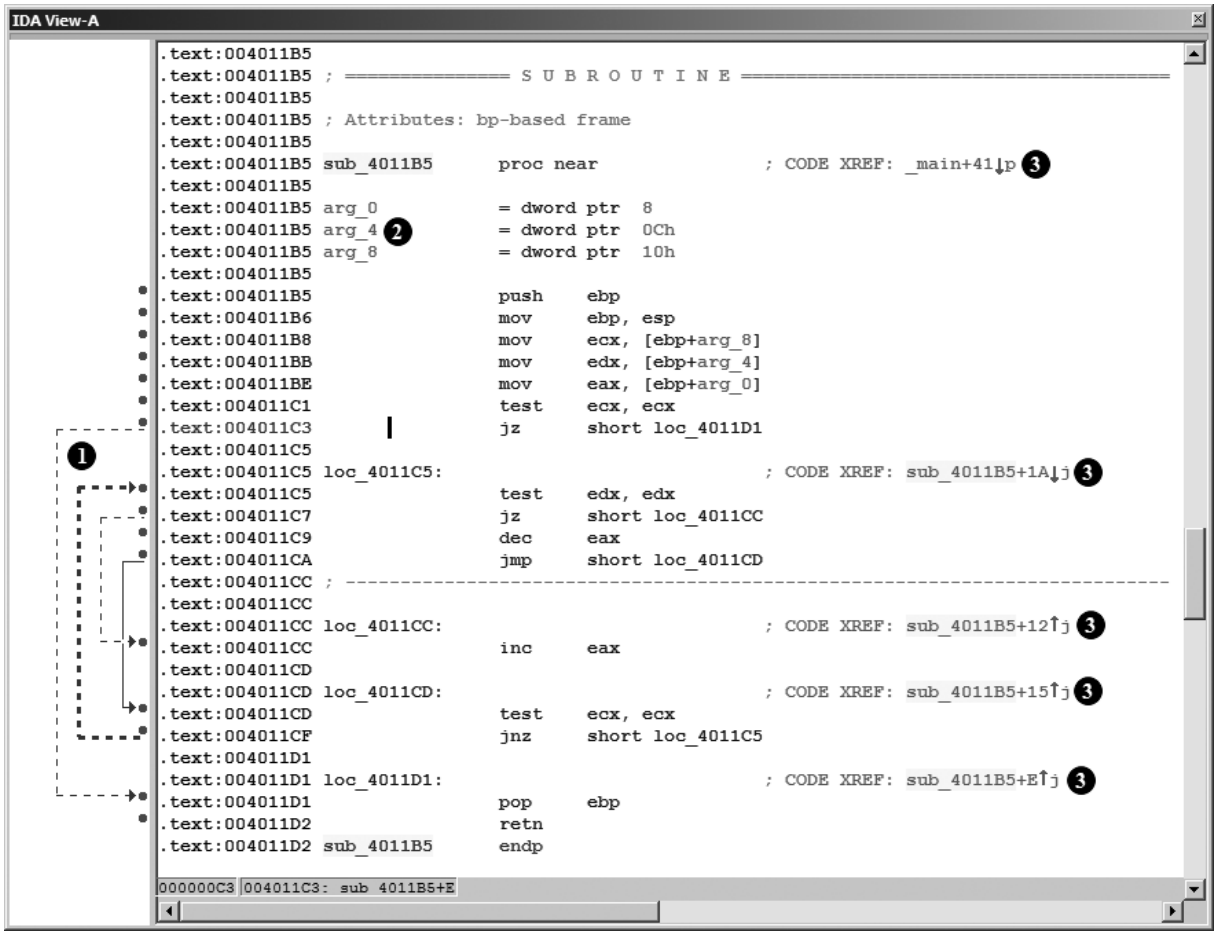


Рисунок 5-4. Текстовое представление IDA

Левая часть, отмеченная 1, называется областью стрелок и показывает нелинейный поток внутри функции. Сплошные стрелки соответствуют безусловным переходам, пунктирные - условным. Если управление передаётся к более раннему адресу, линия рисуется утолщённой. Такой обратный поток часто указывает на цикл. На рисунке стрелка цикла идёт от `004011CF` к `004011C5`.

Объявления в области 2, также присутствующие в графе, отражают наиболее вероятное, по мнению IDA, устройство кадра стека функции[3]. Оно вычисляется посредством подробного анализа указателя стека и применяемого функцией указателя кадра. Представление стека рассматривается в главе 6.

Комментарии в области 3, начинающиеся с точки с запятой, являются перекрёстными ссылками. В данном случае это ссылки на код, сообщающие, что другая инструкция передаёт управление в место с таким комментарием. Перекрёстным ссылкам посвящена глава 9.

Далее в книге для большинства примеров будет применяться текстовый листинг. К графу мы обратимся лишь там, где он заметно повышает наглядность. Способы очистки и комментирования текстового представления описаны в главе 7.

Окно функций

В этом окне перечислены все функции, распознанные IDA в базе. Запись может выглядеть так:

```
malloc              .text               00BDC260 00000180 R . . . B . .
```

Она сообщает, что функция `malloc` находится в разделе `.text` по виртуальному адресу `00BDC260`, имеет длину 384 байта, то есть `0x180`, возвращает управление вызывающей стороне - флаг `R` - и обращается к локальным переменным через регистр EBP - флаг `B`.

Все флаги описаны во встроенной справке. Их также можно увидеть как изменяемые флажки в свойствах функции, открыв контекстное меню. Двойной щелчок по записи переносит окно дизассемблирования к выбранной функции.

Окно вывода

Окно в нижней части рабочей области завершает стандартный набор, видимый при открытии нового файла. Оно служит консолью IDA и показывает выполняемые программой действия.

При первоначальном открытии здесь появляются этапы анализа и операции создания базы. Во время дальнейшей работы выводится состояние запрошенных пользователем действий. Содержимое можно скопировать в системный буфер или полностью очистить через контекстное меню. Кроме того, именно сюда чаще всего направляют результат пользовательские сценарии и подключаемые модули.

Дополнительные представления IDA

Помимо окон дизассемблирования, функций и вывода, IDA открывает другие вкладки под полосой навигации. Они предоставляют специализированные виды базы. Их полезность зависит от особенностей исследуемого файла и опыта пользователя. Некоторые настолько специализированы, что подробно рассматриваются только в последующих главах.

Шестнадцатеричное представление

Название этого окна несколько условно: оно поддерживает разные форматы и одновременно служит шестнадцатеричным редактором. По умолчанию содержимое программы выводится по 16 байт в строке с соответствующими знаками ASCII сбоку.

Можно открыть несколько таких окон: `Hex View-A`, `Hex View-B`, `Hex View-C` и так далее. Первое по умолчанию синхронизировано с первым окном дизассемблирования. При прокрутке одного второе переходит к тому же виртуальному адресу. Выбранной инструкции соответствуют выделенные байты.

На рисунке 5-5 указатель листинга находится по адресу `0040108C` на инструкции `call`, поэтому в шестнадцатеричном окне выделены составляющие её пять байтов.

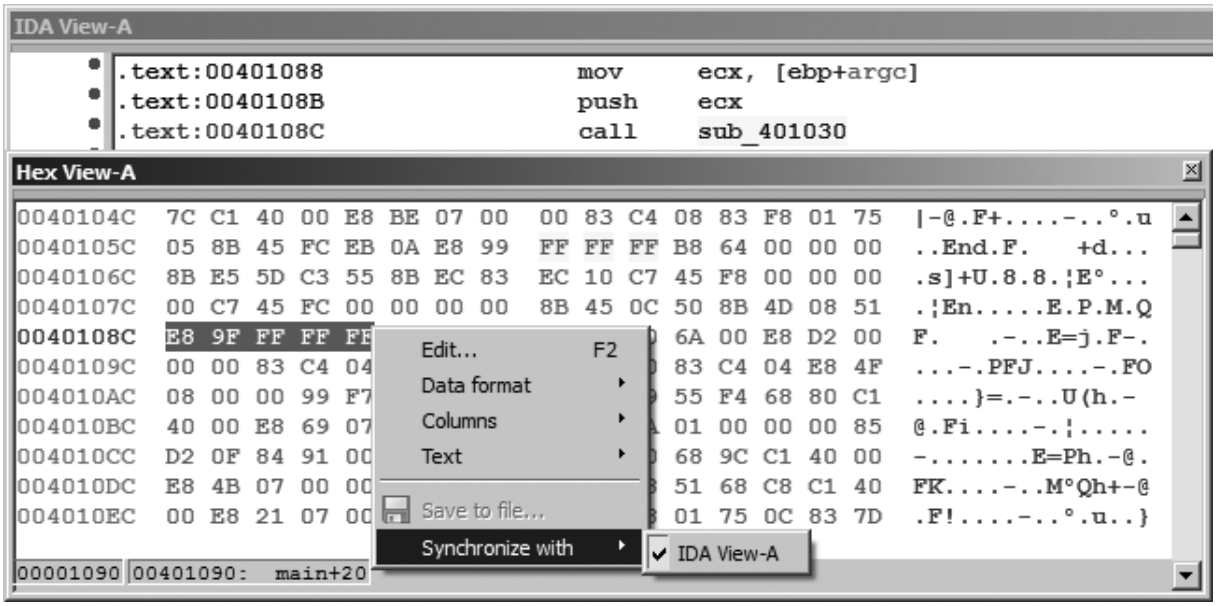


Рисунок 5-5. Синхронизированные шестнадцатеричное представление и листинг

На рисунке также открыто контекстное меню. В нём выбирается окно дизассемблирования, с которым нужно синхронизировать байты, или отключается связь для независимой прокрутки.

Команда редактирования превращает окно в шестнадцатеричный редактор. Перед возвращением к просмотру изменения нужно либо подтвердить, либо отменить.

Меню формата данных позволяет выбрать шестнадцатеричные значения размером 1, 2, 4 или 8 байт, знаковые и беззнаковые десятичные целые, а также разные форматы чисел с плавающей точкой. Можно изменить число столбцов и включить либо отключить текстовую часть.

Иногда шестнадцатеричное окно заполнено вопросительными знаками. Так IDA сообщает, что не знает значений в данном диапазоне виртуальных адресов. Например, раздел `bss`[4] обычно не занимает места в файле, но при загрузке расширяется для статических данных программы.

Окно экспорта

Здесь перечислены точки входа файла: начальная точка выполнения из заголовка, а также функции и переменные, экспортируемые для других файлов. Экспортированные функции часто встречаются в общих библиотеках, например DLL для Windows.

Для каждой записи показываются имя, виртуальный адрес и, если применимо, порядковый номер[5]. В исполняемом файле всегда есть хотя бы начальная точка, которой IDA присваивает имя `start`. Пример записи:

```
LoadLibraryA                    7C801D77 578
```

Двойной щелчок переносит листинг к связанному адресу. Аналогичные сведения предоставляют программы командной строки `objdump -T`, `readelf -s` и `dumpbin /EXPORTS`.

Окно импорта

Это парное окно к экспорту. Оно перечисляет все функции, импортируемые исследуемым двоичным файлом. Импорт имеет смысл только при использовании общих библиотек; статически

скомпонованная программа не имеет внешних зависимостей и, следовательно, импортов.

Для каждого элемента - функции или данных - указаны имя и содержащая его библиотека. Код импортированной функции находится в общей библиотеке, поэтому приведённый адрес относится к соответствующей записи таблицы импорта[6]. Например:

```
0040E108  GetModuleHandleA        KERNEL32
```

Двойной щелчок по приведённой записи перенесёт листинг к адресу `0040E108`. В шестнадцатеричном представлении там будут видны четыре вопросительных знака. IDA выполняет статический анализ и не может знать, какой адрес окажется в этой области при запуске программы.

Окно импорта предоставляет сведения, аналогичные результатам `objdump -T`, `readelf -s` и `dumpbin /IMPORTS`.

Важно помнить: здесь показаны только символы, которые двоичный файл поручает обработать динамическому загрузчику автоматически. Символы, загружаемые самой программой посредством `dlopen` и `dlsym` либо `LoadLibrary` и `GetProcAddress`, в списке не появятся.

Окно структур

В нём показано устройство сложных типов данных, например структур и объединений Си, которые IDA считает используемыми в двоичном файле. Во время анализа программа обращается к обширной библиотеке сигнатур типов функций и пытается сопоставить типы параметров с памятью программы.

На рисунке 5-6 IDA предполагает использование структуры `sockaddr`[7].

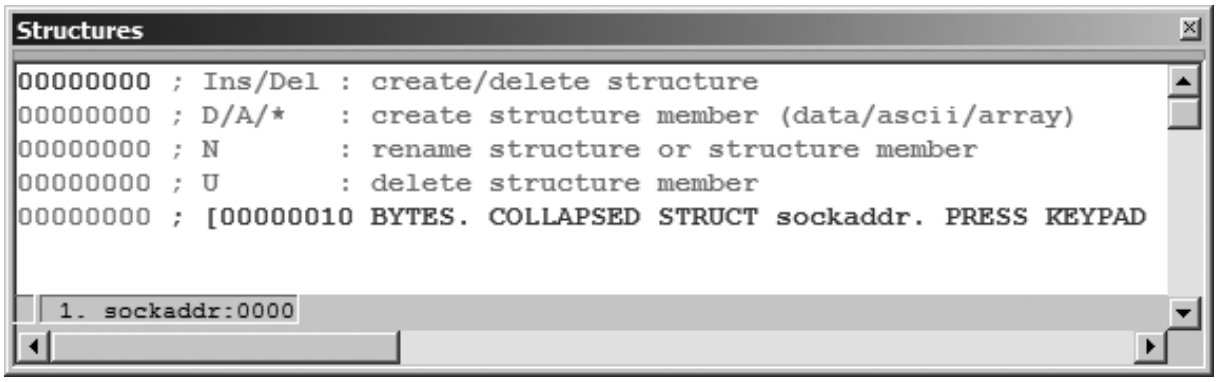


Рисунок 5-6. Окно структур

К такому выводу можно прийти разными путями. Например, IDA могла заметить вызов библиотечной функции Си `connect`[8], устанавливающей сетевое соединение. Двойной щелчок по имени раскрывает структуру и показывает её подробное устройство, включая имена и размеры полей.

У окна структур два основных назначения:

1. служить готовым справочником по устройству стандартных структур;
2. позволять создавать собственные структуры и применять их как шаблоны разметки памяти после обнаружения пользовательского типа в программе.

Определение и применение структур подробнее рассматривается в главе 8.

Окно перечислений

Это окно похоже на окно структур. Когда IDA обнаруживает стандартный перечислимый тип Си, он появляется в списке. Замена целочисленных констант элементами перечисления делает листинг значительно понятнее. Здесь же можно определять собственные перечислимые типы и применять их к дизассемблированному файлу.

Представления третьего уровня

Последние рассматриваемые окна IDA по умолчанию не открывает. Все они доступны через подменю представлений, но содержат сведения, которые обычно не нужны немедленно, поэтому сначала не загромождают рабочую область.

Окно строк

Это встроенный аналог программы `strings` с дополнительными возможностями. В IDA 5.1 и более ранних выпусках окно входило в стандартную рабочую область, но начиная с версии 5.2 его нужно открывать через меню.

Окно выводит извлечённые из двоичного файла строки и адрес каждой из них. Двойной щелчок переносит листинг к выбранной строке. Вместе с перекрёстными ссылками из главы 9 это позволяет быстро найти интересное сообщение и проследить все места программы, которые к нему обращаются.

Например, строка `SOFTWARE\Microsoft\Windows\CurrentVersion\Run` вызывает естественный вопрос: зачем приложение обращается именно к этому разделу реестра Windows? Как будет показано в следующей главе, перейти к использующей строку инструкции можно всего четырьмя щелчками.

IDA не хранит извлечённые строки постоянно. При каждом открытии окна база полностью сканируется заново в соответствии с текущими настройками. Они доступны через контекстную команду настройки.

В окне на рисунке 5-7 выбираются искомые виды строк. По умолчанию IDA ищет завершённые нулём строки Си длиной не менее пяти знаков из семибитного набора ASCII.

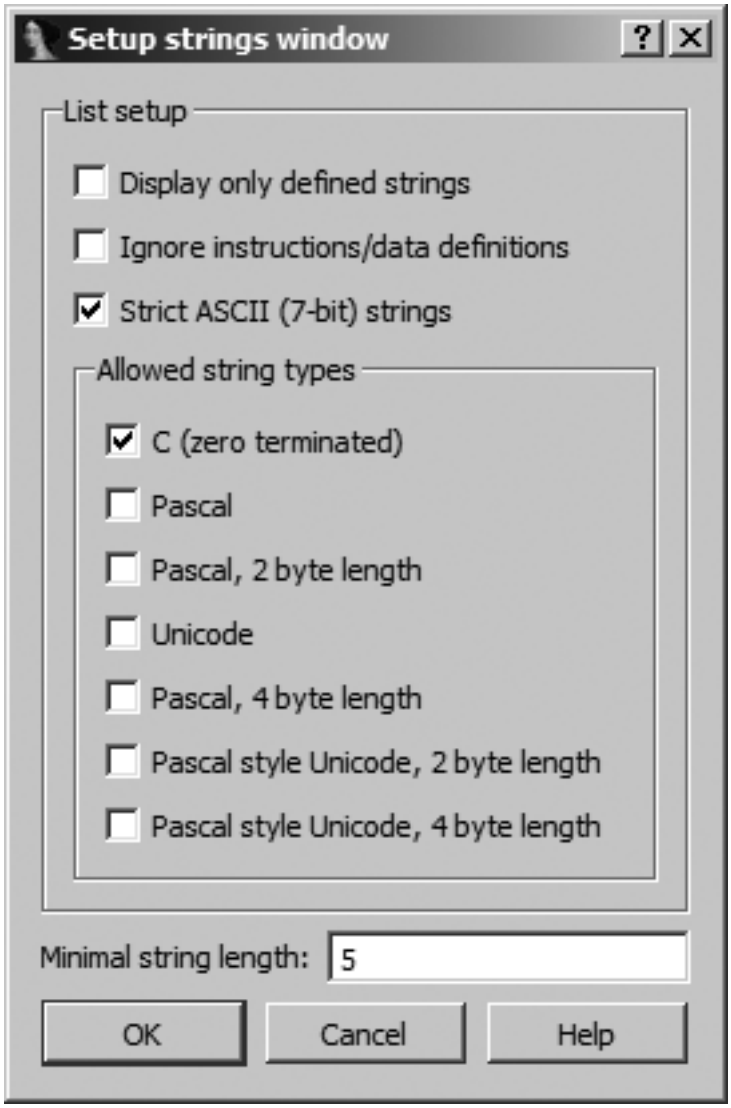


Рисунок 5-7. Окно настройки строк

Если ожидаются строки другого вида, настройку нужно изменить. В программах для Windows часто применяется Юникод, а двоичные файлы Borland Delphi используют строки в стиле Паскаля с двухбайтовым полем длины. После каждого подтверждения настроек IDA заново сканирует базу.

Два параметра заслуживают особого внимания.

Показывать только определённые строки

Список ограничивается именованными строковыми элементами данных, которые IDA создала автоматически или пользователь определил вручную. Остальные параметры отключаются, а дополнительный поиск строк не выполняется.

Игнорировать определения инструкций и данных

IDA ищет строки поверх уже существующих инструкций и определений данных. Благодаря этому можно обнаружить:

1. строки, встроенные в раздел кода и ошибочно преобразованные в инструкции;

2. строки внутри данных, оформленных как другой тип, например массив байтов или целых чисел.

Одновременно появляется много мусорных строк - случайных последовательностей из пяти и более знаков ASCII, которые необязательно имеют смысл. Результат похож на запуск `strings` с параметром `-a`.

Рисунок 5-8 показывает, что при неверной настройке IDA находит не все строки. Здесь параметр игнорирования инструкций и определений данных отключён.

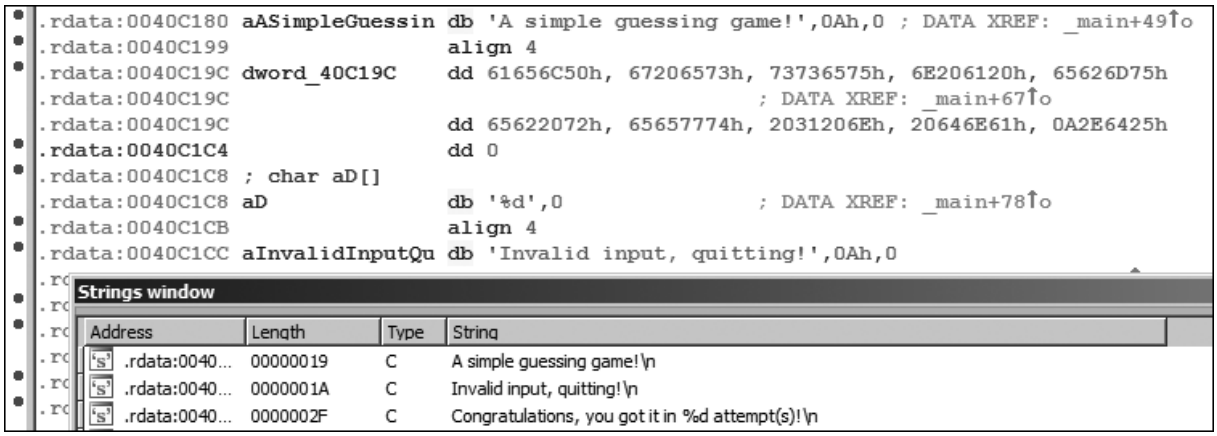


Рисунок 5-8. Пример необнаруженных строковых данных

В результате строка по адресу `.rdata:0040C19C` - «Пожалуйста, угадайте число от 1 до %d» - остаётся незамеченной. Вывод прост: ищите все ожидаемые виды строк во всех местах, где они могут встретиться.

Окно имён

Окно на рисунке 5-9 содержит сводный список всех глобальных имён двоичного файла. Имя - символьное описание виртуального адреса программы. При первоначальной загрузке IDA получает имена из таблицы символов и посредством анализа сигнатур.

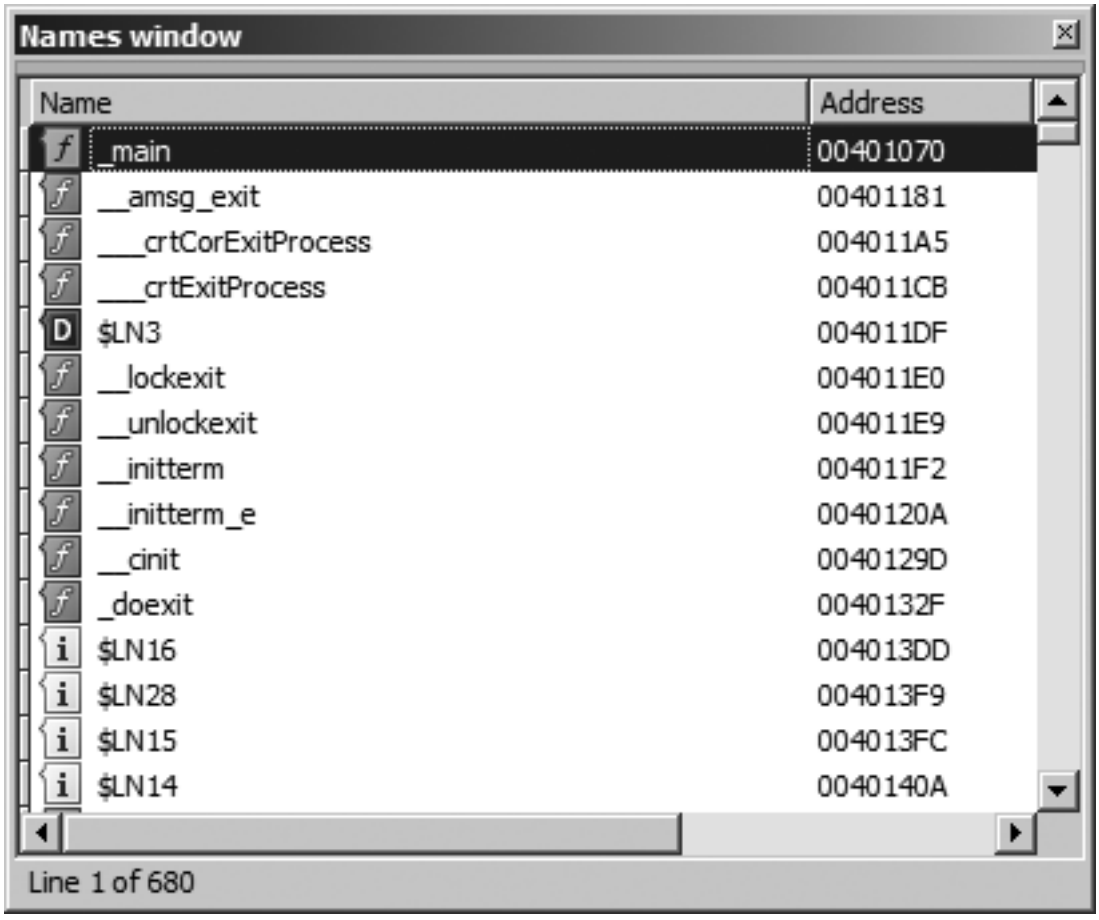


Рисунок 5-9. Окно имён

Список сортируется по алфавиту или по виртуальному адресу в прямом либо обратном порядке. Он удобен для быстрого перехода к известным местам программы. Двойной щелчок немедленно переносит листинг к выбранному имени.

Имена обозначаются цветом и буквой:

• `F` - обычная функция, которую IDA не считает библиотечной;
• `L` - библиотечная функция, распознанная сопоставлением с сигнатурой; если подходящей сигнатуры нет, она будет считаться обычной;
• `I` - импортированное имя, чаще всего функция общей библиотеки; в отличие от библиотечной функции, её тела нет в текущем листинге;
• `C` - именованный код, который IDA не относит ни к одной функции; такое возможно, если имя найдено в таблице символов, но вызов соответствующего адреса не обнаружен;
• `D` - данные, обычно глобальная переменная;
• `A` - строковые данные, то есть используемая область со знаками одного из известных IDA строковых типов, например завершённая нулём строка Си в ASCII.

В листинге встречается множество именованных мест, отсутствующих в этом окне. Во время дизассемблирования IDA создаёт имена для всех адресов, на которые есть прямая ссылка как на код - цель перехода или вызова - либо как на данные - чтение, запись или получение адреса.

Если место присутствует в таблице символов, используется имя из неё. В противном случае IDA создаёт стандартное имя, сочетая виртуальный адрес с префиксом типа. Адрес обеспечивает уникальность, поскольку два места не могут иметь один виртуальный адрес. Такие автоматически созданные имена в окне имён не показываются.

Распространённые префиксы:

• sub_xxxxxx - подпрограмма по адресу xxxxxx;
• loc_xxxxxx - инструкция по адресу xxxxxx;
• byte_xxxxxx - 8-разрядные данные;
• word_xxxxxx - 16-разрядные данные;
• dword_xxxxxx - 32-разрядные данные;
• unk_xxxxxx - данные неизвестного размера.

Далее в книге будут рассмотрены дополнительные алгоритмы выбора имён для данных программы.

Окно сегментов

Здесь показан сводный перечень сегментов двоичного файла. То, что IDA называет сегментами, при описании структуры файла обычно называют разделами. Не следует путать их с сегментами памяти процессоров, использующих сегментную архитектуру.

Для каждого сегмента приводятся имя, начальный и конечный адреса и флаги разрешений. Диапазон адресов соответствует виртуальной памяти, в которую раздел будет отображён при выполнении. Пример для программы Windows:

```
Имя   Начало   Конец    R W X D L Выравн. База Тип    Класс AD es   ss   ds   fs       gs
UPX0   00401000 00407000 R W X . L para   0001 public CODE 32 0000 0000 0001 FFFFFFFF FFFFFFFF
UPX1   00407000 00408000 R W X . L para   0002 public CODE 32 0000 0000 0001 FFFFFFFF FFFFFFFF
UPX2   00408000 0040803C R W . . L para   0003 public DATA 32 0000 0000 0001 FFFFFFFF FFFFFFFF
.idata 0040803C 00408050 R W . . L para   0003 public XTRN 32 0000 0000 0001 FFFFFFFF FFFFFFFF
UPX2   00408050 00409000 R W . . L para   0003 public DATA 32 0000 0000 0001 FFFFFFFF FFFFFFFF
```

Нестандартные имена и два одновременно исполняемых и записываемых сегмента сразу вызывают подозрение: возможен самоизменяющийся код, который рассматривается в главе 21.

Знание размера сегмента ещё не означает знания всего его содержимого. По разным причинам сегмент часто занимает на диске меньше места, чем в памяти. IDA показывает значения той части, которую удалось заполнить из файла, а для остатка выводит вопросительные знаки.

Двойной щелчок переносит листинг к началу сегмента. Контекстное меню позволяет добавлять и удалять сегменты и изменять их свойства. Это особенно полезно для файлов нестандартного формата, структуру которых загрузчик IDA не распознал.

Похожие сведения в командной строке предоставляют `objdump -h`, `readelf -S` и `dumpbin /HEADERS`.

Окно сигнатур

IDA располагает обширной библиотекой сигнатур для распознавания известных блоков кода. С их помощью определяются стандартные начальные последовательности компиляторов, а функции классифицируются как известные библиотечные или добавленные статической компоновкой.

Распознанные библиотечные функции можно не исследовать и сосредоточиться на неизвестном коде, который, вероятно, гораздо интереснее внутреннего устройства `printf`.

Окно перечисляет сигнатуры, уже сопоставленные с открытым файлом. Пример для PE в Windows:

```
Файл      Состояние   Функций  Название библиотеки
vc32rtf   Применена   501      Среда выполнения Microsoft VisualC 2-8/net
```

IDA применила `vc32rtf` из `<КАТАЛОГ_IDA>/sigs` и распознала 501 библиотечную функцию - на 501 функцию меньше для обратной разработки.

Дополнительную сигнатуру приходится применять как минимум в двух случаях. Во-первых, IDA могла не распознать компилятор и потому не выбрать подходящий набор; тогда предварительный анализ подсказывает, какую сигнатуру следует попробовать принудительно. Во-вторых, можно создать собственную сигнатуру для библиотеки, не входящей в поставку, например статического варианта OpenSSL из FreeBSD 8.0.

DataRescue предоставляла комплект создания пользовательских сигнатур для механизма IDA. Этот процесс рассматривается в главе 12. Чтобы применить новую сигнатуру, нажмите `Insert` или откройте контекстное меню и выберите её из списка известных установленной IDA.

Окно библиотек типов

По замыслу оно похоже на окно сигнатур. Библиотеки типов объединяют накопленные IDA сведения о стандартных типах и объявлениях функций, извлечённые из заголовочных файлов распространённых компиляторов.

Обрабатывая заголовки, IDA узнаёт типы параметров библиотечных функций и может соответствующим образом пояснять листинг. Оттуда же берутся размеры и устройство сложных структур. Сведения собираются в файлах TIL из `<КАТАЛОГ_IDA>/til` и применяются при анализе.

Как и для сигнатур, сначала IDA должна определить используемые программой библиотеки, чтобы выбрать подходящие TIL. Дополнительную библиотеку типов можно загрузить клавишей `Insert` или через контекстное меню. Подробнее они рассматриваются в главе 13.

Окно вызовов функций

Функция может вызывать другие функции и сама вызываться ими. Граф отношений вызывающих и вызываемых функций называется графом или деревом вызовов; его построение описано в главе 9.

Иногда полный граф не нужен, а достаточно непосредственных соседей выбранной функции. Будем считать Y соседом X, если Y непосредственно вызывает X или X непосредственно вызывает Y.

При открытии окна IDA определяет соседей функции под указателем и выводит список, подобный рисунку 5-10.

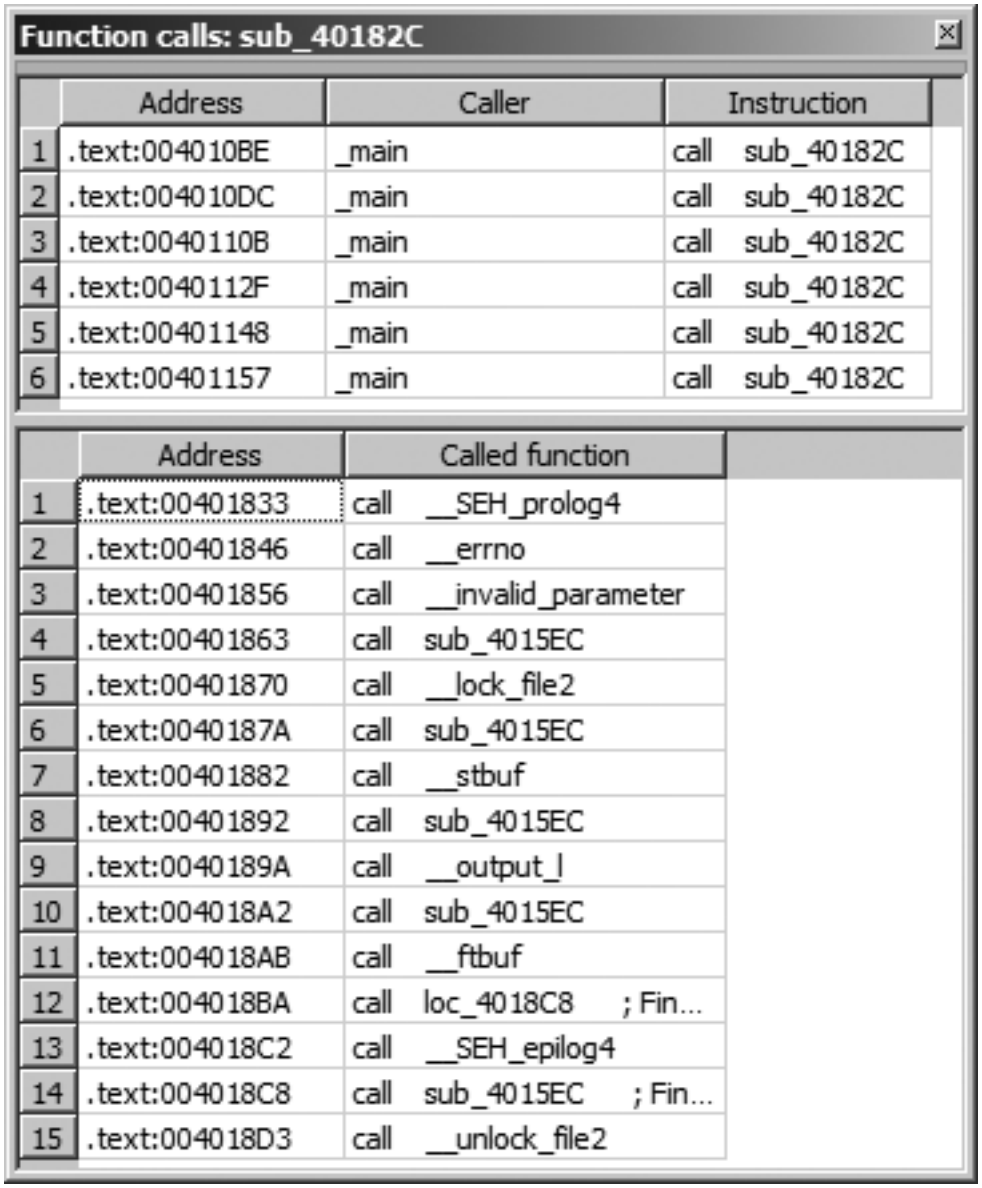


Рисунок 5-10. Окно вызовов функций

Здесь `sub_40182C` вызывается из шести мест `_main`, а сама `_main` выполняет ещё 15 вызовов. Двойной щелчок переносит листинг к выбранной вызывающей или вызываемой функции. Основой этого окна служат перекрёстные ссылки, подробно рассмотренные в главе 9.

Окно проблем

Здесь IDA сообщает о трудностях дизассемблирования и принятых решениях. Иногда пользователь способен исправить листинг и помочь программе, иногда - нет. Проблемы встречаются даже в простейших файлах, и часто разумно их игнорировать. Для исправления многих из них нужно понимать двоичный файл лучше IDA, что удаётся далеко не всегда.

Пример списка:

```
Адрес           Тип        Инструкция
.text:0040104C  BOUNDS     call    eax
.text:004010B0  BOUNDS     call    eax
.text:00401108  BOUNDS     call    eax
.text:00401350  BOUNDS     call    dword ptr [eax]
```

```
.text:004012A0   DECISION   push    ebp
.text:004012D0   DECISION   push    ebp
.text:00401560   DECISION   jmp     ds:__set_app_type
.text:004015F8   DECISION   dd 0FFFFFFFFh
.text:004015FC   DECISION   dd 0
```

Для каждой проблемы указаны адрес, тип и находящаяся там инструкция. `BOUNDS` возникает, когда цель вызова или перехода нельзя определить - например, IDA не знает значения EAX - либо адрес назначения кажется выходящим за виртуальный диапазон программы.

`DECISION` чаще всего вообще не является настоящей ошибкой. Он отмечает адрес, где IDA решила трактовать байты как инструкции, хотя рекурсивный спуск из главы 1 никогда не ссылался на это место.

Полный перечень типов и советы приведены в разделе списка проблем встроенной справки.

Итоги

Поначалу множество представлений IDA может ошеломить. Проще сначала освоить основные окна и лишь затем переходить к дополнительным. Не следует считать обязательным применение всего, что предлагает программа: не каждое окно полезно в любой задаче.

Помимо описанных представлений, по мере освоения IDA вам встретится множество диалоговых окон. Важнейшие из них будут вводиться далее по мере необходимости.

Кроме стандартного графа дизассемблирования, графы в этой главе не рассматривались. В меню IDA они выделены в отдельную категорию, отличную от представлений. Причины объясняются в главе 9, целиком посвящённой графам.

Теперь интерфейс IDA уже должен казаться привычнее. В следующей главе мы начнём изучать многочисленные способы изменения листинга, которые помогают лучше понять поведение программы и упрощают повседневную работу.

Сноски

[1] Базовый блок - наибольшая последовательность инструкций, которая выполняется от начала до конца без ветвления. У него одна точка входа - первая инструкция - и одна точка выхода - последняя. Первая инструкция часто служит целью перехода, а последняя сама является инструкцией перехода.
[2] Термином «поток» IDA обозначает возможное продолжение выполнения после инструкции. Обычный поток соответствует последовательному выполнению, поток перехода - возможной передаче управления в несмежное место, а поток вызова - обращению к подпрограмме.
[3] Кадр стека, или запись активации, - блок памяти в стеке времени выполнения, содержащий переданные функции параметры и объявленные в ней локальные переменные. Он создаётся при входе в функцию и освобождается при выходе. Подробнее кадры рассматриваются в главе 6.
[4] Компилятор создаёт раздел `bss` для неинициализированных статических переменных. Поскольку начальных значений нет, выделять им место в образе файла незачем; в заголовке сохраняется лишь размер раздела. При запуске загрузчик выделяет требуемую память и заполняет её нулями.
[5] Порядковый номер экспорта позволяет обращаться к функции общей библиотеки по числу, а не по имени. Это ускоряет поиск адреса и позволяет скрыть имена функций. Такие номера применяются в DLL для Windows.
[6] В таблице импорта загрузчик сохраняет адреса импортированных функций после загрузки обязательных библиотек. Одна запись содержит адрес одной функции.
[7] `sockaddr` - тип стандартной библиотеки Си, часто представляющий конечную точку сетевого соединения. Такая переменная может хранить адрес IP и номер порта при установлении соединения TCP с удалённым компьютером.
[8] Объявление функции: `int connect(int sockfd, const struct sockaddr *serv_addr, socklen_t addrlen);`.

Глава 6. Навигация по дизассемблированному коду

В этой и следующей главах рассматривается главное, что делает IDA Pro интерактивной: удобство навигации и изменения данных. Здесь основное внимание уделяется перемещению по листингу логичным способом.

До сих пор мы видели, что на базовом уровне IDA объединяет возможности многих распространённых инструментов обратной разработки в едином представлении дизассемблирования. Умение перемещаться по нему необходимо для уверенной работы с программой.

Статический текстовый листинг сам по себе позволяет только прокручивать строки вверх и вниз. Даже в лучшем текстовом редакторе по такому документу трудно перемещаться: обычно доступен лишь встроенный поиск, подобный `grep`. База данных, лежащая в основе IDA, обеспечивает гораздо более развитую навигацию.

Основы навигации в IDA

На первых порах одни только средства перемещения IDA уже могут оказаться очень полезными. Помимо обычного поиска, знакомого по текстовым редакторам, программа создаёт и показывает полный набор перекрёстных ссылок, действующих подобно гиперссылкам на веб-странице. В большинстве случаев для перехода к интересующему месту достаточно двойного щелчка.

Навигация двойным щелчком

При дизассемблировании каждому месту программы назначается виртуальный адрес. Значит, перейти можно куда угодно, если известен нужный адрес. Но хранить в памяти целый каталог чисел затруднительно.

Именно поэтому ещё первые программисты стали присваивать важным адресам символьные имена. Это похоже на замену числовых кодов операций мнемониками: программы становится легче читать, писать и запоминать.

Как уже отмечалось, во время анализа IDA получает имена из таблицы символов или создаёт их по способу обращения к адресу. Любое имя в окне дизассемблирования одновременно служит целью навигации, похожей на гиперссылку. Отличий два:

1. имя никак особо не выделяется как ссылка;
2. для перехода требуется двойной, а не одиночный щелчок.

Этот механизм уже встречался в окнах функций, импорта и экспорта: двойной щелчок по имени переносил листинг к соответствующему месту. В следующем фрагменте цели перехода отмечены X. Двойной щелчок по любой из них переместит представление к выбранному адресу.

```
.text:0040132B loc_40132B:             ; ссылка на код: Ysub_4012E4+B^j
.text:0040132B       cmp     edx, 0CDh
.text:00401331       jg      short Xloc_40134E
.text:00401333       jz      Xloc_4013BF
.text:00401339       sub     edx, 0Ah
.text:0040133C       jz      short Xloc_4013A7
.text:0040133E       sub     edx, 0C1h
.text:00401344       jz      short Xloc_4013AF
.text:00401346       dec     edx
```

```
.text:00401347       jz      short Xloc_4013B7
.text:00401349       jmp     Xloc_4013DD ; вариант по умолчанию
.text:00401349                           ; вариант 0 таблицы переходов 00401300
.text:0040134E ; ---------------------------------------------------------
.text:0040134E
.text:0040134E loc_40134E:             ; ссылка на код: Ysub_4012E4+4D^j
```

IDA особо обрабатывает ещё два вида элементов. Первый - перекрёстные ссылки, отмеченные Y. Обычно они состоят из имени и шестнадцатеричного смещения. Ссылка справа от `loc_40134E` указывает на место на 4D в шестнадцатеричной системе, то есть на 77 байт, дальше начала `sub_4012E4`. Двойной щелчок по её тексту переносит к ссылающейся инструкции, в данном случае по адресу `00401331`. Подробнее перекрёстные ссылки рассматриваются в главе 9.

Второй вид - шестнадцатеричные значения. Если показанное число является допустимым виртуальным адресом внутри двоичного файла, двойной щелчок переносит к нему. В следующем фрагменте значения Z подходят для перехода, а отмеченные `[` не подходят.

```
.data:00409013       db   [4
.data:00409014       dd Z4037B0h
.data:00409018       db   [0
.data:00409019       db  [0Ah
.data:0040901A       dd Z404590h
.data:0040901E       db   [0
.data:0040901F       db  [0Ah
.data:00409020       dd Z404DA8h
```

Наконец, двойной щелчок работает и в окне вывода. Если сообщение начинается с описанной выше цели, оно переносит к указанному месту.

```
Propagating type information...
Function argument information has been propagated
The initial autoanalysis has been finished.
\ 40134e is an interesting location
] Testing: 40134e
\ loc_4013B7
] Testing: loc_4013B7
```

В этом примере два сообщения, отмеченные `\`, позволяют перейти к адресам в их начале. Остальные строки, включая отмеченные `]`, ничего не делают.

Переход к адресу

Иногда точный адрес известен, но в текущем листинге нет удобного имени для двойного щелчка. Самый примитивный вариант - прокручивать окно, пока нужное место не появится. Это приемлемо только для известного виртуального адреса, поскольку листинг упорядочен именно по адресам.

Если известно лишь имя, например функция `foobar`, прокрутка превращается в поиск иголки в стоге сена. Можно отсортировать список функций по алфавиту, найти имя и дважды щёлкнуть его. Можно воспользоваться поиском IDA, но ради уже известного места это обычно избыточно.

Проще всего открыть окно перехода к адресу, показанное на рисунке 6-1.



Рисунок 6-1. Окно перехода к адресу

Оно вызывается командой перехода к адресу или клавишей `G`, когда активно окно дизассемблирования. Запомнить клавишу помогает английское слово «перейти», с которого она начинается.

Укажите имя или шестнадцатеричное значение и подтвердите ввод - листинг немедленно переместится. Введённые значения сохраняются и позднее доступны в раскрывающемся списке, что упрощает возврат к прежним целям.

История навигации

Если сравнивать IDA с веб-обозревателем, имена и адреса подобны гиперссылкам. Есть и другая общая возможность: движение назад и вперёд по порядку посещения мест.

При каждом переходе текущее положение добавляется в историю. Команда перехода к предыдущей позиции возвращает к последней записи и действует как кнопка «назад». Ей соответствует клавиша `Esc` - одна из самых полезных в IDA.

Но будьте осторожны: если активно не окно дизассемблирования, `Esc` закрывает текущее окно. Случайно закрытое представление всегда можно вернуть через меню просмотра.

Переход назад особенно удобен после нескольких последовательных вызовов функций, когда нужно вернуться к исходной точке анализа.

Команда перехода к следующей позиции движется по истории вперёд, подобно кнопке веб-обозревателя. Ей соответствует сочетание `Ctrl+Enter`, хотя оно используется реже, чем `Esc`.

Две кнопки панели инструментов на рисунке 6-2 выполняют знакомые действия назад и вперёд. У каждой есть раскрывающийся список, позволяющий сразу выбрать любое место истории, не проходя весь путь последовательно.

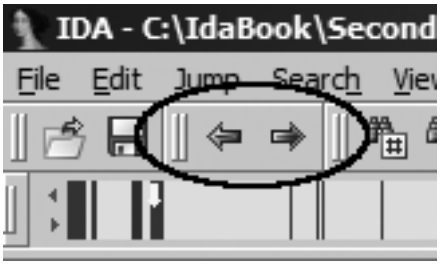


Рисунок 6-2. Кнопки перехода назад и вперёд

Кадры стека

IDA Pro работает на низком уровне, поэтому многие её возможности требуют знакомства с устройством скомпилированных языков, машинного кода и памяти программы. Время от времени книга будет объяснять соответствующую теорию, чтобы представления IDA оставались понятными.

Один из таких терминов - кадр стека. Это блок памяти в стеке времени выполнения, выделенный конкретному вызову функции. Программисты объединяют выполняемые операторы в функции, также называемые процедурами, подпрограммами или методами. Иногда этого требует язык, но обычно такое устройство просто считается хорошей практикой.

Пока функция не выполняется, ей почти не нужна память. После вызова память требуется по нескольким причинам. Во-первых, вызывающая сторона передаёт параметры, или аргументы, которые нужно где-то разместить. Во-вторых, самой функции нужно временное пространство для локальных переменных, доступных только до её завершения.

Компилятор использует кадры стека, также называемые записями активации, чтобы выделение и освобождение памяти для параметров и локальных переменных было незаметно программисту. Перед передачей управления он вставляет код размещения аргументов, а при входе - код выделения памяти для локальных данных.

Из-за способа построения кадра там же обычно сохраняется адрес возврата. Каждый рекурсивный вызов получает отдельный кадр, поэтому его данные аккуратно отделены от данных предыдущего вызова.

При вызове функции выполняются следующие действия:

1. Вызывающая сторона размещает параметры в местах, заданных соглашением о вызовах. Если они помещаются в стек, указатель стека изменяется.
2. Управление передаётся функции, обычно инструкцией x86 `CALL` или MIPS `JAL`. Адрес возврата сохраняется в стеке либо регистре процессора.
3. При необходимости функция настраивает указатель кадра[1] и сохраняет регистры, значения которых вызывающая сторона ожидает получить неизменными.
4. Для локальных переменных выделяется память, часто посредством изменения указателя стека.
5. Функция выполняет работу, обращается к параметрам и при необходимости создаёт результат. Возвращаемое значение обычно помещается в заранее определённый регистр или набор регистров.
6. После завершения освобождается память локальных переменных, как правило обращением действий шага 4.
7. Сохранённые на шаге 3 регистры восстанавливаются, включая указатель кадра вызывающей стороны.
8. Управление возвращается вызывающей стороне, например инструкцией x86 `RET` или MIPS `JR`. В зависимости от соглашения здесь же из стека могут удаляться параметры.
9. После возврата вызывающей стороне иногда приходится самостоятельно удалить параметры и вернуть указателю стека значение до шага 1.

Шаги 3 и 4 настолько часто выполняются при входе, что вместе называются прологом функции. Шаги 6-8 в конце образуют эпилог. Всё, кроме шага 5 - тела функции, - является накладными расходами вызова.

Соглашения о вызовах

Теперь можно подробнее рассмотреть устройство кадров. Далее используются архитектура x86 и поведение распространённых компиляторов Microsoft Visual C/C++ и GNU `gcc`/`g++`.

Важнейший этап - размещение параметров вызывающей функцией. Она обязана сохранить их именно там, где ожидает вызываемая функция, иначе неизбежны серьёзные ошибки. Способ передачи аргументов задаётся соглашением о вызовах.

Соглашение определяет, следует ли помещать параметры в конкретные регистры, стек или одновременно туда и туда. Не менее важно, кто удаляет их из стека после завершения: вызывающая или вызываемая сторона. Соблюдение объявленного правила необходимо для целостности указателя стека.

Соглашение Си cdecl

Стандартное соглашение большинства компиляторов Си для x86 называется соглашением Си. Модификатор `_cdecl` позволяет программе на C/C++ принудительно выбрать его, если стандарт был переопределён. Далее будем называть соглашение `cdecl`.

Оно предписывает вызывающей стороне помещать аргументы в стек справа налево и самой удалять их после возврата. Благодаря порядку справа налево самый левый, то есть первый, параметр всегда оказывается на вершине стека при вызове. Его легко найти независимо от общего количества. Поэтому `cdecl` хорошо подходит функциям с переменным числом аргументов, например `printf`.

Обязанность вызывающей стороны очищать стек означает, что сразу после возврата часто встречается инструкция корректировки указателя. Именно вызывающая сторона знает, сколько аргументов фактически передала, тогда как функция с переменным числом параметров заранее этого не знает.

Рассмотрим функцию:

```
void demo_cdecl(int w, int x, int y, int z);
```

По умолчанию она ожидает четыре аргумента справа налево, а очищать стек должен вызывающий код. Компилятор может создать такой вызов:

```
; demo_cdecl(1, 2, 3, 4);
X push   4           ; параметр z
  push   3           ; параметр y
  push   2           ; параметр x
  push   1           ; параметр w
  call   demo_cdecl  ; вызов функции
Y add    esp, 16     ; восстановить прежнее значение ESP
```

Четыре инструкции `push`, начиная с X, сдвигают ESP на 16 байт: четыре 32-разрядных целых. После возврата инструкция Y обращает это изменение. Если функцию вызвать 50 раз, за каждым вызовом будет следовать похожая корректировка.

Следующий вариант тоже соблюдает `cdecl`, но не требует явной очистки после каждого вызова:

```
; demo_cdecl(1, 2, 3, 4);
mov   [esp+12], 4   ; параметр z в четвёртую позицию стека
mov   [esp+8], 3    ; параметр y в третью позицию стека
mov   [esp+4], 2    ; параметр x во вторую позицию стека
mov   [esp], 1      ; параметр w на вершину стека
call  demo_cdecl    ; вызов функции
```

Здесь компилятор заранее выделил пространство для аргументов в прологе. Их запись не меняет указатель стека, поэтому после вызова корректировка не нужна. Такой приём используют компиляторы GNU `gcc` и `g++`.

Оба способа приводят к тому, что при вызове указатель стека направлен на самый левый аргумент.

Соглашение stdcall

Слово «стандартное» здесь несколько вводит в заблуждение: так Microsoft назвала собственное соглашение, обозначаемое модификатором `_stdcall`:

```
void _stdcall demo_stdcall(int w, int x, int y);
```

Чтобы избежать путаницы, далее используется название stdcall.

Как и cdecl, это соглашение помещает параметры в стек справа налево. Отличие в том, что после завершения их удаляет вызываемая функция. Для этого она должна точно знать количество аргументов, поэтому stdcall подходит только функциям с фиксированным числом параметров. Функция с переменным количеством, например printf, применять его не может.

demo_stdcall ожидает три целых, занимающих в 32-разрядной системе 12 байт. Особая форма инструкции x86 RET одновременно извлекает адрес возврата и прибавляет 12 к указателю стека:

```
ret 12     ; вернуться и удалить из стека 12 байт
```

Главное преимущество stdcall - отсутствие отдельного кода очистки после каждого вызова, благодаря чему программа становится немного меньше и быстрее. Microsoft по соглашению применяет его ко всем экспортированным из DLL функциям с фиксированным числом аргументов. Об этом важно помнить при восстановлении объявлений и создании двоично совместимых замен библиотечных компонентов.

Соглашение fastcall для x86

Разновидность stdcall, соглашение fastcall, передаёт до двух параметров в регистрах процессора, а не в стеке. Модификатор понимают Microsoft Visual C/C++ и GNU gcc/g++ начиная с версии 3.4.

Первый и второй параметры помещаются соответственно в ECX и EDX. Остальные идут в стек справа налево, как в stdcall. При возврате функция также сама удаляет стековые аргументы. Объявление выглядит так:

```
void fastcall demo_fastcall(int w, int x, int y, int z);
```

Вызов может быть скомпилирован следующим образом:

```
; demo_fastcall(1, 2, 3, 4);
push   4              ; параметр z во вторую позицию стека
push   3              ; параметр y на вершину стека
mov    edx, 2         ; параметр x в EDX
mov    ecx, 1         ; параметр w в ECX
call   demo_fastcall  ; вызов функции
```

После возврата корректировка не требуется: demo_fastcall сама удалит y и z. Поскольку два аргумента переданы в регистрах, очищаются только 8 байт, хотя всего параметров четыре.

Соглашения о вызовах C++

Нестатические методы классов C++ отличаются от обычных функций необходимостью предоставить указатель this на объект, через который вызван метод. Адрес объекта передаёт вызывающая сторона как неявный параметр.

Стандарт C++ не определяет способ его передачи, поэтому компиляторы действуют по-разному. Microsoft Visual C++ предлагает соглашение thiscall: this помещается в ECX, а нестатический метод очищает стек подобно stdcall.

GNU g++ считает this неявным первым параметром и во всём остальном следует cdecl. Поэтому перед вызовом this оказывается на вершине стека, а после возврата вызывающая сторона удаляет аргументы; хотя бы один аргумент существует всегда. Другие особенности

скомпилированного C++ рассматриваются в главе 8.

Другие соглашения о вызовах

Для полного описания всех существующих соглашений потребовалась бы отдельная книга. Они зависят от языка, компилятора и процессора, поэтому при встрече с редким компилятором иногда нужно проводить дополнительное исследование. Особого упоминания заслуживают оптимизированный код, ручной ассемблер и системные вызовы.

Экспортируемая для других программистов функция должна следовать известному соглашению, чтобы к ней можно было легко обратиться. Для внутренней функции правило достаточно знать только самой программе. Поэтому оптимизирующий компилятор иногда выбирает особый способ передачи аргументов ради скорости. Так бывает, например, с параметром `/GL` в Microsoft Visual C++ и ключевым словом `regparm` в GNU `gcc`/`g++`.

Программист на языке ассемблера полностью управляет передачей параметров. Если его функции не предназначены для стороннего вызова, он волен применять любой способ. Поэтому ручной ассемблер требует особой осторожности. Он часто встречается в обфускации и коде командной оболочки.

Системный вызов - особый вызов службы операционной системы. Обычно он переводит процесс из пользовательского режима в режим ядра. Способ зависит от системы и процессора. Например, в Linux на x86 применяются инструкции `int 0x80` и `sysenter`, тогда как другие системы x86 могут использовать только `sysenter` либо другой номер прерывания.

Во многих системах x86, кроме Linux, параметры системного вызова кладутся в стек, а номер службы - в EAX. В Linux параметры передаются в определённых регистрах, а при их нехватке иногда в памяти.

Размещение локальных переменных

В отличие от параметров, для расположения локальных переменных общего соглашения нет. Компилятор вычисляет необходимый объём и решает, можно ли держать значения в регистрах или нужно использовать стек.

Точный выбор не важен ни вызывающей стороне, ни функциям, которые будут вызваны далее. Более того, обычно устройство локальных данных невозможно определить лишь по исходному тексту.

Примеры кадров стека

Рассмотрим функцию для 32-разрядной x86:

```
void bar(int j, int k);

void demo_stackframe(int a, int b, int c) {
    int x;
    char buffer[64];
    int y;
    int z;
    bar(z, y);
```

```
}
```

Локальным переменным требуется как минимум 76 байт: три целых по 4 байта и массив из 64 байт. Функция может использовать `stdcall` или `cdecl`, и устройство кадра при этом не изменится.

На рисунке 6-3 показан вариант без отдельного указателя кадра, где эту роль играет ESP. При входе достаточно однострочного пролога:

```
sub   esp, 76     ; выделить память всем локальным переменным
```

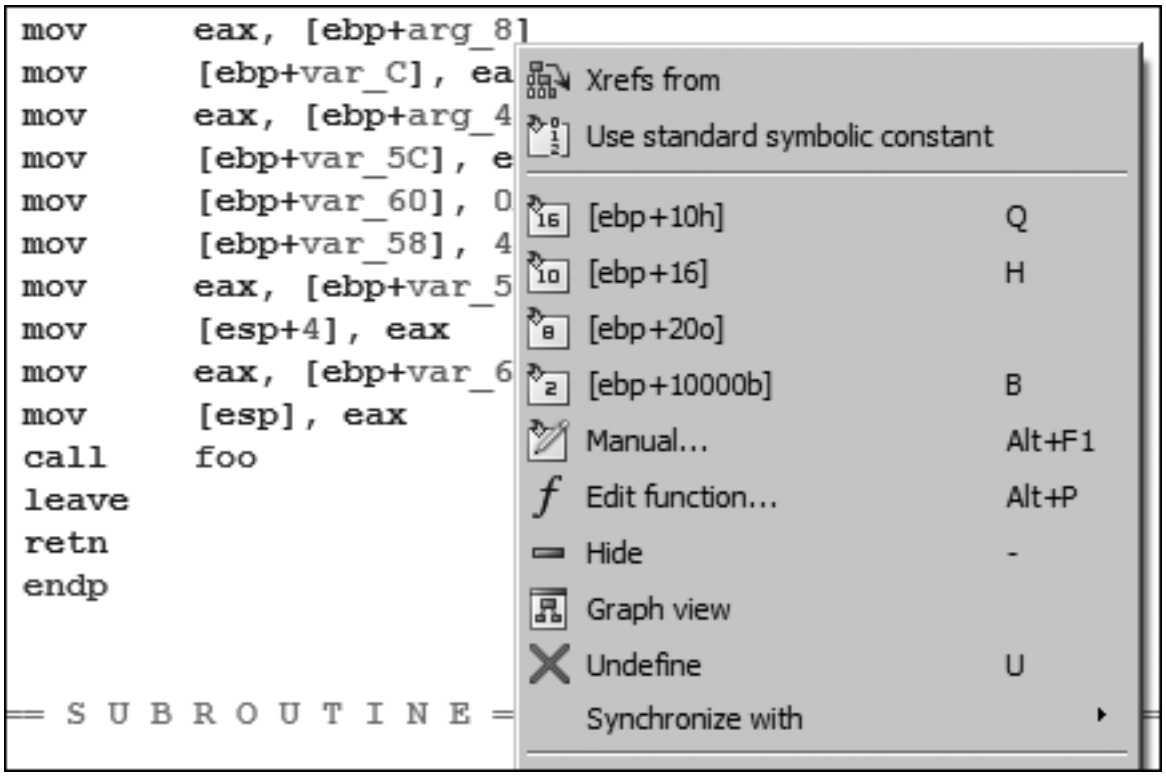


Рисунок 6-3. Кадр стека на основе ESP

Столбец смещений показывает адрес вида «база плюс смещение» для каждой переменной и параметра.

Использование ESP требует от компилятора больше работы: указатель часто меняется, и смещения нужно каждый раз пересчитывать. Вызов `bar` может выглядеть так:

```
X push   dword [esp+4]  ; передать y
Y push   dword [esp+4]  ; передать z
  call   bar
  add    esp, 8         ; при cdecl стек очищает вызывающая сторона
```

Инструкция X правильно получает y по схеме рисунка. На первый взгляд Y обращается к y повторно, но X уже изменила ESP, и все смещения временно сдвинулись. Теперь z действительно находится по `[esp+4]`.

При анализе кадра на основе указателя стека нужно отслеживать каждое изменение ESP и соответственно корректировать будущие смещения. Преимущество состоит в том, что все остальные регистры остаются свободны.

Перед возвратом локальные данные нужно убрать, чтобы ESP указывал на сохранённый адрес. Эпилог:

```
add   esp, 76     ; перейти к сохранённому адресу возврата
ret               ; вернуться к вызывающей стороне
```

Работу со смещениями можно упростить ценой отдельного регистра и кода его настройки. В x86 указателем кадра обычно служит расширенный базовый регистр EBP. Большинство компиляторов делает это по умолчанию, но позволяет отказаться. Например, GNU `gcc/g++` поддерживает параметр `-fomit-frame-pointer`.

Для кадра `demo_stackframe` с отдельным указателем нужен новый пролог:

```
Z push  ebp       ; сохранить EBP вызывающей стороны
[ mov   ebp, esp  ; направить EBP на сохранённое значение
\ sub   esp, 76   ; выделить память локальным переменным
```

Инструкция Z сохраняет текущий EBP. Согласно двоичному интерфейсу приложений System V для 32-разрядных процессоров Intel[2], функция вправе изменять EAX, ECX и EDX, но обязана сохранять остальные регистры вызывающей стороны. Поэтому EBP нужно восстановить перед возвратом.

Другие регистры, например ESI и EDI, компилятор может сохранить одновременно с EBP или после выделения локальных данных. Стандартного места для них в кадре нет.

Инструкция `mov` направляет EBP на текущую вершину стека, затем `sub` выделяет память. Полученная схема показана на рисунке 6-4.

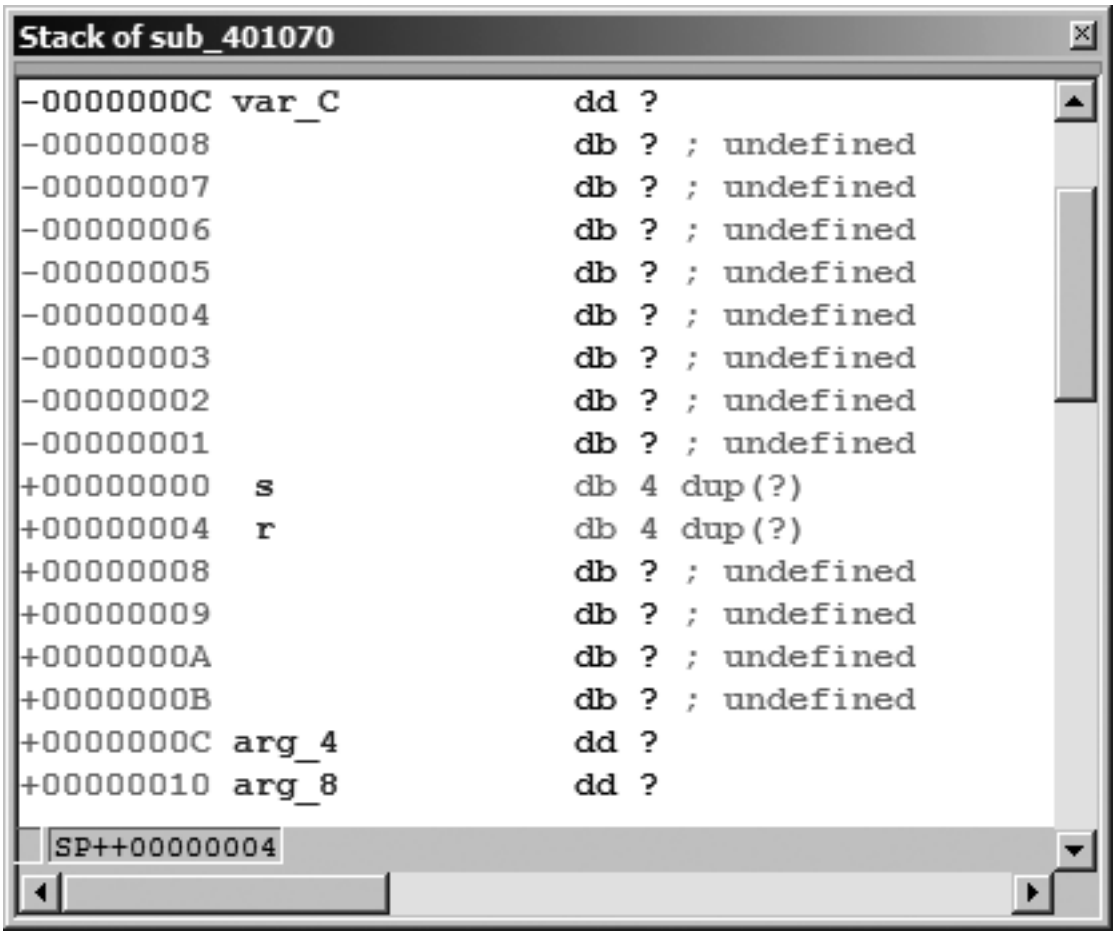


Рисунок 6-4. Кадр стека на основе EBP

Все смещения теперь вычисляются относительно EBP. Чаще всего положительные значения обращаются к параметрам, отрицательные - к локальным переменным. Изменение ESP больше не влияет на адреса внутри кадра.

Вызов `bar` принимает вид:

```
] push  dword [ebp-72]  ; передать y
  push  dword [ebp-76]  ; передать z
```

```
  call  bar
  add   esp, 8          ; при cdecl стек очищает вызывающая сторона
```

Изменение ESP первой инструкцией никак не влияет на последующий доступ к z.

Эпилог должен восстановить указатель кадра вызывающей стороны. Сначала удаляются локальные данные, затем извлекается прежний EBP:

```
mov   esp, ebp    ; удалить локальные переменные
pop   ebp         ; восстановить EBP вызывающей стороны
ret               ; извлечь адрес и вернуться
```

Операция настолько распространена, что в x86 есть сокращающая её инструкция `leave`:

```
leave             ; скопировать EBP в ESP, затем извлечь EBP
ret               ; извлечь адрес и вернуться
```

На других архитектурах имена регистров и инструкций различаются, но общий процесс построения кадра тот же. Полезно изучить характерные прологи и эпилоги своей архитектуры, чтобы быстро переходить к более интересному телу функции.

Представления стека в IDA

Кадр стека существует только во время работы программы. Но это не значит, что при статическом анализе его следует игнорировать: весь код настройки кадров присутствует в двоичном файле, и его исследование позволяет подробно восстановить устройство.

Один из самых сложных видов анализа IDA как раз определяет кадры всех дизассемблированных функций. Программа отслеживает поведение указателя стека, каждую операцию `push` и `pop`, а также сложение и вычитание констант.

Сначала вычисляется точный размер области локальных переменных. Затем IDA определяет наличие отдельного указателя кадра, например по последовательности `push ebp` и `mov ebp, esp`, и распознаёт обращения к данным кадра.

Увидев в `demo_stackframe` инструкцию

```
mov   eax, [ebp+8]
```

IDA понимает, что в EAX загружается первый аргумент a, согласно рисунку 6-4. Анализ позволяет отличить обращения к аргументам ниже сохранённого адреса возврата от локальных данных выше него. Также программа устанавливает, какие места действительно используются. Кадр на рисунке имеет размер 96 байт, но ссылаться, вероятно, будут лишь на семь переменных: четыре локальные и три параметра.

Понимание функции часто сводится к пониманию обрабатываемых типов. Первую подсказку даёт структура кадра. IDA предлагает краткое и подробное представления.

Рассмотрим скомпилированную `gcc` версию:

```
void demo_stackframe(int a, int b, int c) {
    int x = c;
    char buffer[64];
    int y = b;
    int z = 10;
    buffer[0] = 'A';
    bar(z, y);
}
```

Локальные x и y получают значения c и b, z инициализируется числом 10, а первый знак 64-байтового массива `buffer` - буквой A. Соответствующий листинг рассматривается далее.

```
.text:00401090 ; ======== П О Д П Р О Г Р А М М А ========================
.text:00401090
.text:00401090 ; Свойства: Хкадр на основе EBP
.text:00401090
.text:00401090 demo_stackframe proc near ; ссылка на код: sub_4010C1+41p
.text:00401090
[ .text:00401090 var_60 = dword ptr -60h
  .text:00401090 var_5C = dword ptr -5Ch
  .text:00401090 var_58 = byte ptr -58h
  .text:00401090 var_C  = dword ptr -0Ch
  .text:00401090 arg_4  = dword ptr  0Ch
  .text:00401090 arg_8  = dword ptr  10h
  .text:00401090
  .text:00401090         push    ebp
  .text:00401091         mov     ebp, esp
  .text:00401093         sub     esp, Y78h
  .text:00401096         mov     eax, [ebp+\arg_8]
  .text:00401099        ]mov     [ebp+var_C], eax
  .text:0040109C        ^mov     eax, [ebp+arg_4]
  .text:0040109F        ^mov     [ebp+var_5C], eax
  .text:004010A2        _mov     [ebp+var_60], 0Ah
  .text:004010A9        `mov     [ebp+var_58], 41h
  .text:004010AD         mov     eax, [ebp+var_5C]
  .text:004010B0        Zmov     [esp+4], eax
  .text:004010B4         mov     eax, [ebp+var_60]
  .text:004010B7        Zmov     [esp], eax
  .text:004010BA         call    bar
  .text:004010BF         leave
  .text:004010C0         retn
  .text:004010C0 demo_stackframe endp
```

В этом листинге много характерных элементов записи IDA.

В точке X программа сообщает, что по анализу пролога функция использует EBP как указатель кадра. В Y видно, что `gcc` выделил 120 байт локального пространства: `78h` равно 120. Сюда входят 8 байт для двух параметров `bar` в Z, но объём всё равно значительно больше рассчитанных нами 76 байт. Компиляторы иногда добавляют заполнение ради требуемого выравнивания.

Начиная с `[`, IDA показывает краткое представление стека: все переменные, на которые есть прямые ссылки, их размеры и смещения относительно указателя кадра.

Имена зависят от положения относительно сохранённого адреса возврата. Локальные данные находятся выше него, параметры - ниже. Префикс `var_` дополняется шестнадцатеричным расстоянием в байтах выше сохранённого указателя кадра. Например, `var_C` - четырёхбайтовая переменная на 12 байт выше EBP, то есть `[ebp-0Ch]`.

Имена параметров используют `arg_` и смещение от первого параметра. Для четырёхбайтовых значений получаются `arg_0`, `arg_4`, `arg_8`, `arg_C` и так далее. Здесь `arg_0` отсутствует, потому что параметр a не используется: IDA не нашла обращения к `[ebp+8]`. В кратком представлении вообще остаются без имён многие места, на которые код прямо не ссылается.

> **Примечание.** IDA автоматически создаёт имена только для тех стековых переменных, к которым функция непосредственно обращается.

В листинге нет числовых обращений вроде `[ebp-12]`: IDA заменила постоянные смещения символьными именами. Это соответствует её стремлению поднять представление на более высокий уровень. Позднее имя любой стековой переменной можно изменить на более понятное.

Краткое представление служит картой между именами и смещениями. Например, `[ebp+arg_8]` можно было бы показать как `[ebp+10h]` или `[ebp+16]`. Числовой формат выбирается через контекстное меню `arg_8` в точке `\`, показанное на рисунке 6-5.

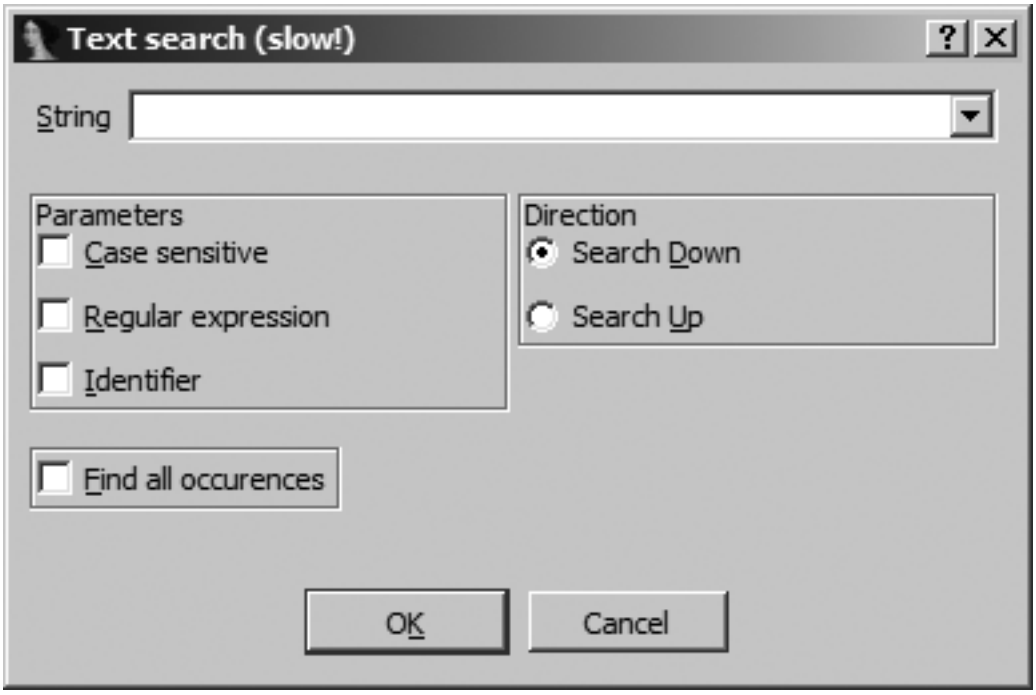


Рисунок 6-5. Выбор другого формата отображения

Поскольку исходный текст нам известен, созданные IDA имена можно сопоставить первоначальным:

1. Параметры a, b и c соответствуют `arg_0`, `arg_4` и `arg_8`; первый отсутствует, потому что не используется.
2. Локальная x инициализируется из c, значит `var_C` соответствует x: в точке `]` она получает `arg_8`.
3. Локальная y инициализируется из b, значит `var_5C` соответствует y: в `^` она получает `arg_4`.
4. `var_60` соответствует z, поскольку в `_` ей присваивается 10.
5. Массив `buffer` начинается с `var_58`, поскольку в точке с обратным апострофом его первый байт получает ASCII `0x41`, то есть A.
6. Два аргумента `bar` в Z записываются в стек, а не помещаются туда инструкциями `push`. Так действуют современные версии `gcc`, начиная с 3.4. IDA распознаёт правило и не создаёт ссылки на локальные переменные для двух элементов на вершине кадра.

Кроме краткого вида, существует подробное представление, учитывающее каждый выделенный байт. Оно открывается двойным щелчком по имени переменной. Для `var_C` появится окно на рисунке 6-6; `Esc` закрывает его.

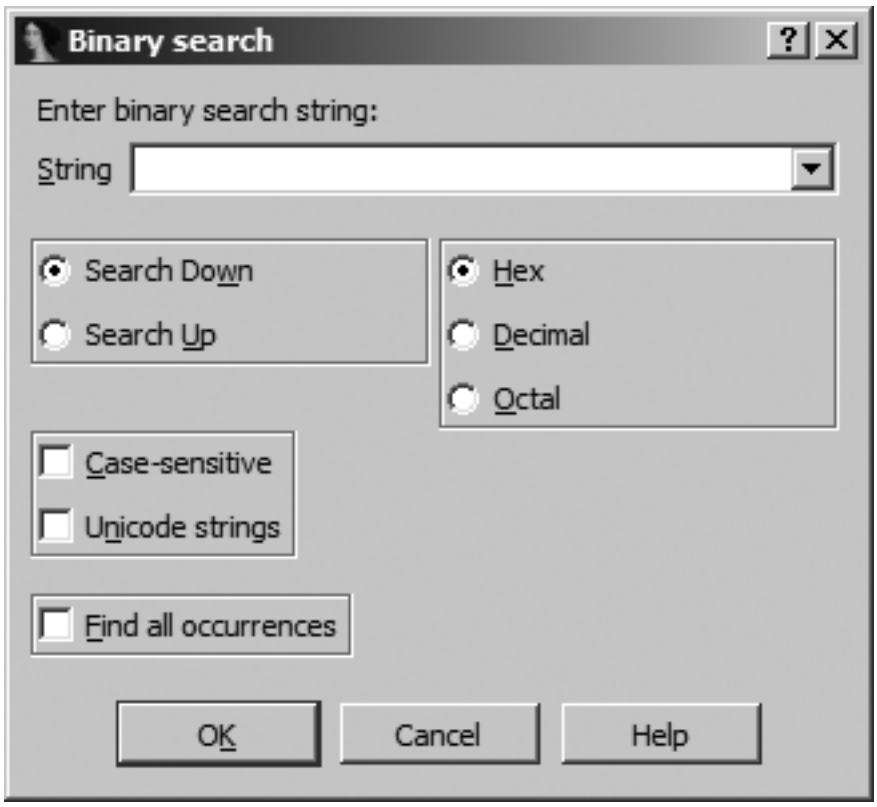


Рисунок 6-6. Представление кадра стека IDA

Подробный вид занимает гораздо больше места. На рисунке показано только 32 байта - небольшая часть всего кадра. Байты без прямых ссылок не получают имён. Например, параметр a, соответствующий `arg_0`, не использовался, поэтому смещения от `+00000008` до `+0000000B` оставлены без определения.

Напротив, `arg_4` прямо читается в EAX в точке `^`. По переносу 32 бит IDA выводит размер 4 байта. Обозначение `db` выделяет 1 байт, `dw` - 2 байта, или слово, `dd` - 4 байта, или двойное слово.

Два особых псевдоимени на рисунке начинаются с пробела: `s` обозначает сохранённые регистры, здесь только EBP, а `r` - сохранённый адрес возврата. Они включены, чтобы учесть каждый байт.

Подробный вид раскрывает работу компилятора. Между сохранённым EBP и x (`var_C`) вставлены 8 дополнительных байт со смещениями от `-00000001` до `-00000008`. Расчёт по смещениям краткого вида показывает, что для 64-байтового `buffer` выделено 76 байт.

Если не изучать исходники `gcc`, остаётся предположить, что дополнительное место обеспечивает выравнивание. Обычно оно не влияет на поведение: программа запросила 64 байта и не должна использовать больше. Но разработчику эксплойта при переполнении важно знать, что интересные последствия начнутся лишь после передачи как минимум 76 байт - фактического размера области с точки зрения компилятора.

В главе 8 мы вернёмся к кадрам стека при работе с массивами, структурами и другими сложными типами.

Поиск в базе данных

IDA упрощает переход к уже известным объектам, а специализированные окна сводят имена, строки, импорт и другие категории. Для общего поиска служит длинное меню соответствующих команд. Большинство перемещает к следующему элементу определённого вида: например, к

следующей инструкции.

Полезно изучить и меню перехода. Многие его команды сначала показывают список целей. Переход к функции, например, открывает полный список функций для быстрого выбора.

Особого внимания заслуживают два универсальных вида: текстовый и двоичный поиск.

Текстовый поиск

Текстовый поиск ищет подстроку в листинге. Он запускается сочетанием `Alt+T` и открывает окно на рисунке 6-7. Параметры определяют подробности; поддерживаются регулярные выражения в стиле POSIX.

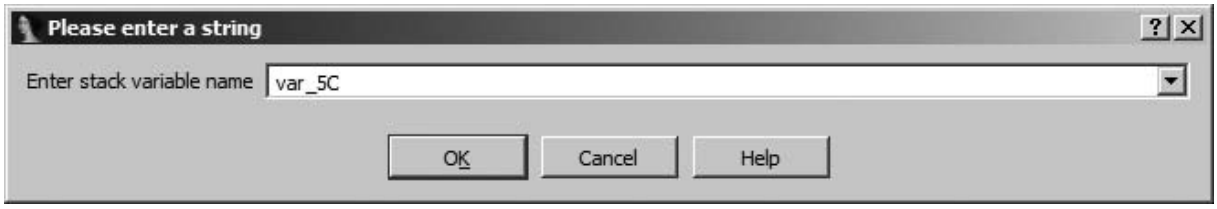


Рисунок 6-7. Окно текстового поиска

Режим поиска идентификатора назван не вполне точно: на деле он требует совпадения целого слова. Совпасть может любое слово строки ассемблера, в том числе мнемоника или константа. Поиск идентификатора `401116` не найдёт имя `loc_401116`, потому что число является только его частью.

Параметр поиска всех совпадений открывает результаты в отдельном окне, откуда легко перейти к любому месту. Следующее совпадение прежнего запроса ищется сочетанием `Ctrl+T`.

Двоичный поиск

Для известной последовательности байтов текстовый поиск не подходит. Двоичный поиск проверяет содержимое шестнадцатеричного представления: собственно байты или часть ASCII в зависимости от записи запроса. Он запускается сочетанием `Alt+B`; окно показано на рисунке 6-8.

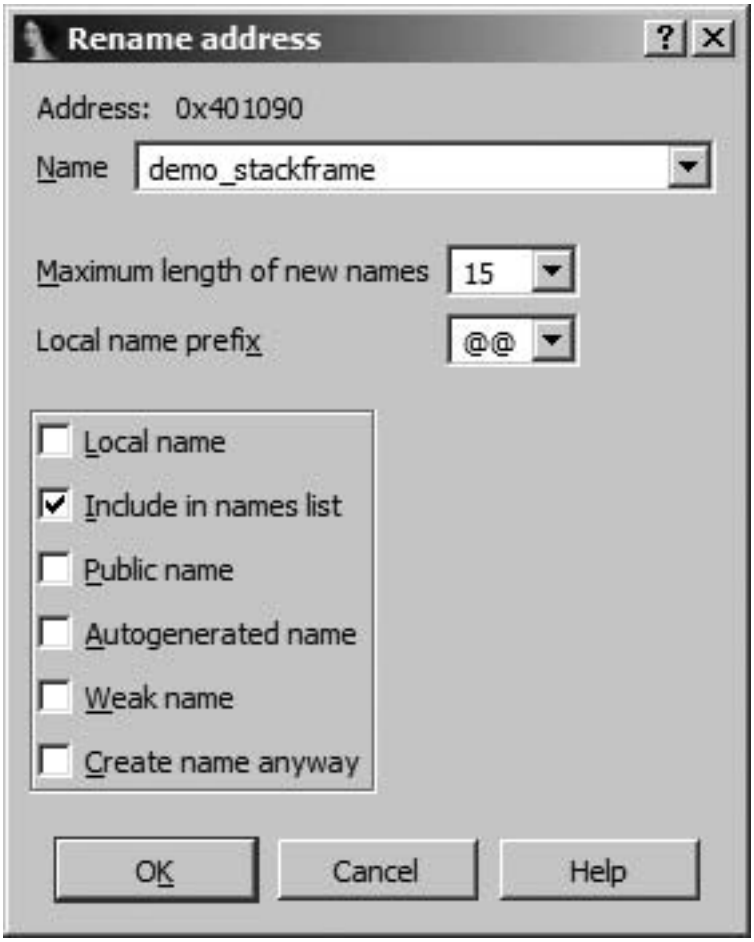


Рисунок 6-8. Окно двоичного поиска

Шестнадцатеричная последовательность задаётся двухзначными числами через пробел, например `CA FE BA BE`. Регистр букв здесь не важен сам по себе: `ca fe ba be` означает те же байты.

Для поиска встроенного текста строка заключается в кавычки. Параметр Юникода выбирает соответствующее представление строки.

Флажок учёта регистра способен вызвать путаницу. Для текста всё очевидно: без него запрос `hello` найдёт `HELLO`. Но при шестнадцатеричном поиске без учёта регистра запрос `E9 41 C3` неожиданно совпадёт с `E9 61 C3`, поскольку `0x41` - знак A, а `0x61` - знак a.

> **Примечание.** При поиске точной последовательности шестнадцатеричных байтов обязательно включайте учёт регистра. Особенно это важно для кодов операций, а не текста ASCII.

Следующее двоичное совпадение ищется сочетанием `Ctrl+B`. Необязательно находиться в шестнадцатеричном окне: запрос можно задать и из листинга, после чего успешный результат перенесёт его к адресу с подходящими байтами.

Итоги

Цель главы состояла в том, чтобы дать минимальные навыки уверенного перемещения по листингу. Подавляющее большинство взаимодействий с IDA использует уже рассмотренные операции.

Освоив навигацию, логично перейти к изменению базы под конкретную задачу. В следующей главе мы начнём вносить простейшие правки, добавляя сведения, которые получили благодаря пониманию содержимого и поведения двоичного файла.

Сноски

[1] Указатель кадра - регистр, указывающий внутрь кадра стека. Переменные обычно адресуются по смещению относительно него.
[2] Спецификация интерфейса: <http://www.sco.com/developers/devspecs/abi386-4.pdf>.

Глава 7. Изменение дизассемблированного кода

После навигации важнейшие возможности IDA связаны с изменением листинга под нужды пользователя. Благодаря базе данных правка автоматически распространяется на все представления, сохраняя их согласованность.

IDA позволяет легко добавлять сведения и перестраивать отображение. Там, где это уместно, программа сама выполняет глобальную замену, без труда преобразует инструкции в данные и обратно и предлагает возможности, которых нет у обычных дизассемблеров.

> **Примечание.** В IDA нет отмены. Помните об этом при изменении базы. Единственная близкая замена - часто сохраняться и при необходимости возвращаться к недавней версии.

Имена и переименование

До сих пор нам встречались две категории имён: связанные с виртуальными адресами и относящиеся к переменным кадра стека. В большинстве случаев IDA создаёт их автоматически по уже рассмотренным правилам и называет фиктивными.

К сожалению, стандартное имя редко раскрывает назначение адреса или переменной и мало помогает понять поведение программы. Поэтому одно из первых и самых частых изменений при анализе - замена стандартных имён осмысленными.

IDA позволяет изменить любое имя и сама обновляет все его употребления в листинге. Обычно достаточно выделить имя и нажать `N`, чтобы открыть окно переименования. Та же команда доступна в контекстном меню. Для стековых переменных и адресов процесс несколько различается.

Параметры и локальные переменные

Стековые имена устроены проще всего: они не связаны с конкретным виртуальным адресом и не появляются в окне имён. Подобно переменным языков программирования, они имеют область видимости функции, которой принадлежит кадр.

Каждая функция программы может содержать собственную `arg_0`, но внутри одной функции двух переменных с таким именем быть не может. Окно переименования показано на рисунке 7-1.

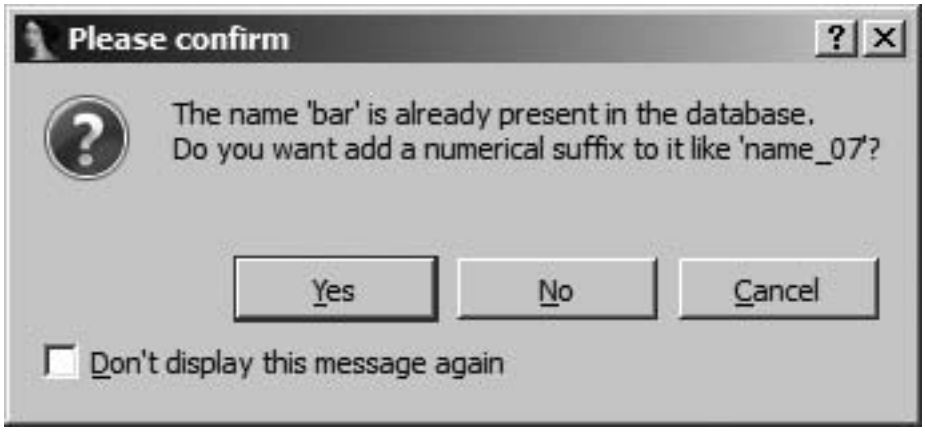


Рисунок 7-1. Переименование стековой переменной

После ввода нового имени IDA заменяет старое во всём контексте текущей функции. Если в `demo_stackframe` переименовать `var_5C` в `y`, получится следующий листинг; изменения отмечены X.

```
.text:00401090 ; ======== П О Д П Р О Г Р А М М А ========================
.text:00401090
.text:00401090 ; Свойства: кадр на основе BP
.text:00401090
.text:00401090 demo_stackframe proc near ; ссылка на код: sub_4010C1+41p
.text:00401090
.text:00401090 var_60 = dword ptr -60h
.text:00401090 Xy     = dword ptr -5Ch
.text:00401090 var_58 = byte ptr -58h
.text:00401090 var_C  = dword ptr -0Ch
.text:00401090 arg_4  = dword ptr  0Ch
.text:00401090 arg_8  = dword ptr  10h
.text:00401090
.text:00401090         push    ebp
.text:00401091         mov     ebp, esp
.text:00401093         sub     esp, 112
.text:00401096         mov     eax, [ebp+arg_8]
.text:00401099         mov     [ebp+var_C], eax
.text:0040109C         mov     eax, [ebp+arg_4]
.text:0040109F         mov     [ebp+y], eax
.text:004010A2         mov     [ebp+var_60], 0Ah
.text:004010A9         mov     [ebp+var_58], 41h
.text:004010AD         mov     eax, [ebp+Xy]
.text:004010B0         mov     [esp+4], eax
.text:004010B4         mov     eax, [ebp+var_60]
.text:004010B7         mov     [esp], eax
.text:004010BA         call    bar
.text:004010BF         leave
.text:004010C0         retn
.text:004010C0 demo_stackframe endp
```

Чтобы вернуть стандартное имя, откройте то же окно и оставьте поле пустым - IDA создаст прежний вариант заново.

Именованные адреса

Переименование адреса или присвоение имени безымянному месту начинается той же клавишей `N`, но окно выглядит иначе. На рисунке 7-2 показаны точный именуемый адрес и доступные свойства.

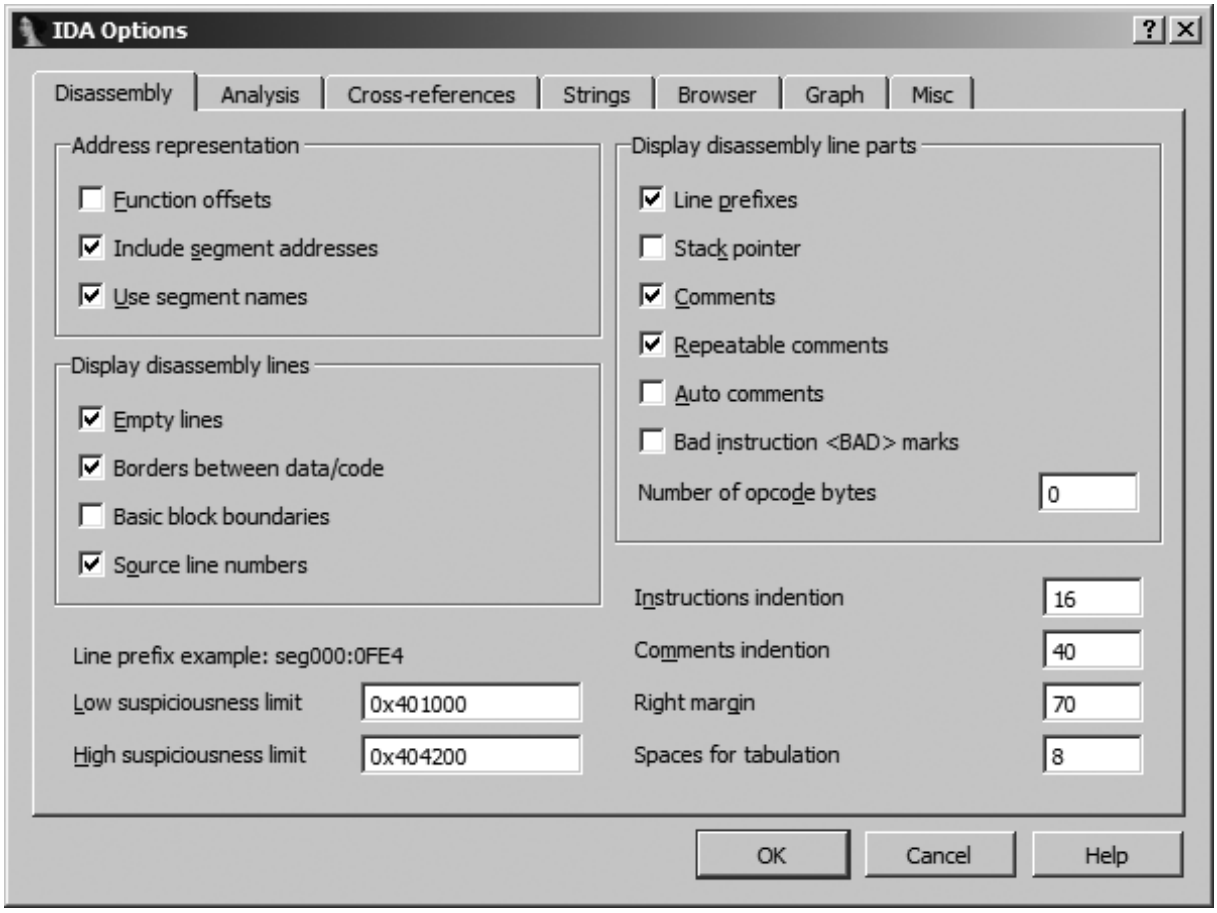


Рисунок 7-2. Переименование адреса

Максимальная длина берётся из `<КАТАЛОГ_IDA>/cfg/ida.cfg`. Более длинное имя использовать можно: IDA лишь предупредит и предложит увеличить предел. Новый предел действует только в текущей базе; новые базы снова получают значение из файла настройки.

Именованному адресу можно назначить следующие свойства.

Локальное имя

Область видимости ограничивается текущей функцией, поэтому уникальность проверяется только внутри неё. В разных функциях допустимы одинаковые локальные имена, в одной - нет.

Адрес за пределами функции нельзя сделать локальным. Это относится к именам самих функций и глобальным переменным. Чаще всего локальные имена дают целям переходов внутри функции, например ветвям управляющей конструкции.

Включить в список имён

Имя добавляется в соответствующее окно, откуда к нему легче вернуться. Автоматические фиктивные имена по умолчанию туда не попадают.

Общедоступное имя

Обычно это символ, экспортируемый двоичным файлом, например общей библиотекой. IDA обнаруживает такие имена в заголовках при загрузке, но свойство можно назначить вручную. В основном это лишь добавляет пометку `public` в листинг и окно имён.

Автоматически созданное имя

У этого свойства нет заметного влияния. Его выбор не заставляет IDA создать имя автоматически.

Слабое имя

Слабый символ - особая разновидность общедоступного, применяемая лишь при отсутствии обычного символа с таким именем. Свойство важно для ассемблера, но почти ничего не меняет в листинге IDA.

Создать имя в любом случае

Два адреса одной функции не могут иметь одно имя; то же относится к двум глобальным адресам. Поведение параметра зависит от области.

Если глобальному объекту, например функции или переменной, присвоить уже занятое имя, IDA в любом случае покажет окно конфликта на рисунке 7-3 и предложит добавить уникальный числовой суффикс.

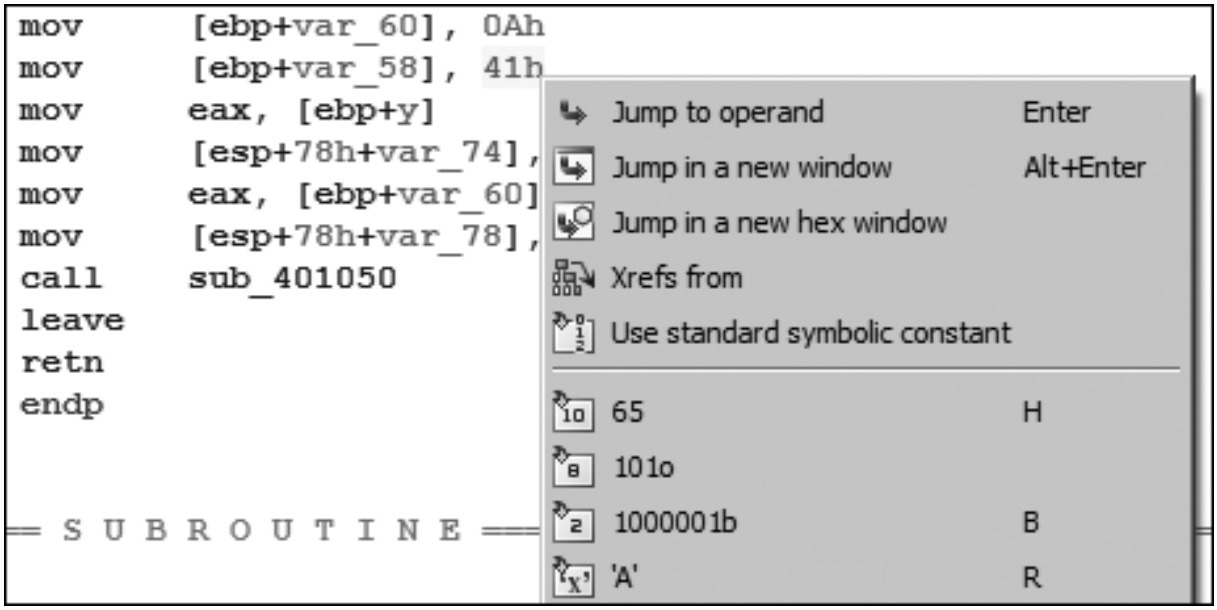


Рисунок 7-3. Окно конфликта имён

Для локального имени повторение по умолчанию просто отклоняется. Чтобы заставить IDA добавить суффикс, нужно выбрать создание в любом случае. Самый простой выход из любого конфликта - выбрать действительно свободное имя.

Имена регистров

Третий, часто упускаемый вид - имя регистра. В пределах функции регистр можно переименовать. Это полезно, когда компилятор держит переменную не в стеке, а, например, в EDX, и хочется обозначить её назначение понятнее.

Выделите регистр и нажмите `N` либо выберите переименование в контекстном меню. Фактически создаётся псевдоним, действующий до конца текущей функции. В её начале IDA даже показывает запись вида `псевдоним = регистр`. Все употребления регистра заменяются этим именем. В коде, не принадлежащем функции, переименование регистра невозможно.

Комментарии в IDA

В базу можно встраивать заметки о ходе анализа. Особенно удобно описывать последовательность ассемблерных инструкций на более высоком уровне - например, оператором Си кратко передать поведение функции. При повторном изучении комментарий освежит память быстрее, чем новый разбор инструкций.

IDA поддерживает несколько видов комментариев для разных целей. Они связываются с любой строкой через меню редактирования, горячие клавиши или контекстное меню. Рассмотрим листинг функции `bar`.

```
.text:00401050 ; ======== П О Д П Р О Г Р А М М А ========================
.text:00401050
.text:00401050 ]; void bar(int j, int k);
.text:00401050 ; Свойства: кадр на основе BP
.text:00401050
.text:00401050 ^bar proc near       ; ссылка на код: demo_stackframe+2Ap
.text:00401050
.text:00401050 arg_0 = dword ptr  8
.text:00401050 arg_4 = dword ptr  0Ch
.text:00401050
.text:00401050   push  ebp
.text:00401051   mov   ebp, esp
.text:00401053   sub   esp, 8
.text:00401056 \Следующие три строки проверяют j < k
.text:00401056   mov   eax, [ebp+arg_0]
.text:00401059   cmp   eax, [ebp+arg_4]
.text:0040105C   jge   short loc_40106C Z; повторяемый комментарий отражается у ссылки
.text:0040105E   mov   [esp], offset aTheSecondParam [; «Второй параметр больше»
.text:00401065   call  printf
.text:0040106A   jmp   short locret_40108E X; перейти к концу функции
.text:0040106C ; ---------------------------------------------------------
.text:0040106C
.text:0040106C loc_40106C:            ; ссылка на код: bar+Cj
.text:0040106C   mov   eax, [ebp+arg_0] Y; повторяемый комментарий отражается у ссылки
.text:0040106F   cmp   eax, [ebp+arg_4]
.text:00401072   jle   short loc_401082
.text:00401074   mov   [esp], offset aTheFirstParame [; «Первый параметр больше»
.text:0040107B   call  printf
.text:00401080   jmp   short locret_40108E
.text:00401082 ; ---------------------------------------------------------
.text:00401082
.text:00401082 loc_401082:            ; ссылка на код: bar+22j
.text:00401082   mov   [esp], offset aTheParametersA [; «Параметры равны»
.text:00401089   call  printf
.text:0040108E
.text:0040108E locret_40108E:         ; ссылка на код: bar+1Aj
.text:0040108E                         ; bar+30j
.text:0040108E   leave
.text:0040108F   retn
.text:0040108F bar endp
```

Большинство комментариев начинается с точки с запятой, как во многих ассемблерах. Это соответствует знаку решётки в сценарных языках или `//` в C++.

Обычные комментарии

Простейший комментарий помещается в конец существующей строки, как в X. Щёлкните правую часть листинга и нажмите двоеточие `:`, чтобы открыть окно ввода. Если ввести несколько строк, они будут выровнены справа от листинга.

Для редактирования или удаления нужно снова открыть окно и изменить либо стереть весь текст. По умолчанию обычные комментарии синие.

IDA сама широко ими пользуется. Во время анализа она поясняет параметры, передаваемые функциям, если знает их имена или типы. Обычно сведения поступают из библиотек типов, рассматриваемых в главах 8 и 13, но их можно задать вручную.

Повторяемые комментарии

Такой комментарий вводится один раз, но автоматически появляется во многих местах. В предыдущем листинге он находится в Y. По умолчанию он тоже синий, поэтому отличается поведением, а не видом.

Если один адрес ссылается на другой с повторяемым комментарием, этот текст отражается у исходной ссылки. Отражённый вариант по умолчанию серый. Горячая клавиша - точка с запятой `;`, из-за чего обычный и повторяемый комментарии легко перепутать.

Текст в Z совпадает с Y, потому что инструкция `jge short loc_40106C` из Z ссылается на адрес Y `0040106C`.

Обычный комментарий в месте отражения имеет преимущество и скрывает повторяемый. После его удаления отражённый текст снова появится.

Для строк существует виртуальная разновидность повторяемого комментария. Когда IDA создаёт строковую переменную, у всех ссылок автоматически показывается её содержимое. Пользователь не может редактировать этот виртуальный текст. В листинге три таких случая отмечены `[`.

Предшествующие и последующие строки

Это полноформатные комментарии непосредственно перед или после выбранной строки. Только они не начинаются с точки с запятой. Предшествующий текст отмечен `\`. Отличить два вида можно по адресу строки относительно соседней инструкции.

Комментарии функций

Они группируются в верхней части листинга функции. В точке `]` таким образом записано объявление. Сначала выделите имя функции в `^`, затем добавьте обычный или повторяемый комментарий.

Повторяемый комментарий функции отражается во всех местах её вызова. IDA автоматически создаёт пояснение в виде объявления при назначении типа функции, описанном в главе 8.

Основные преобразования кода

Часто созданный IDA листинг вполне устраивает пользователя, но чем дальше файл от обычной программы распространённого компилятора, тем больше управления анализом приходится брать на себя. Это особенно верно для обфусцированного кода и неизвестных IDA форматов.

Поддерживаются следующие преобразования:

• данные в код;
• код в данные;
• последовательность инструкций в функцию;
• изменение начала или конца существующей функции;
• изменение формата операндов инструкций.

Необходимость зависит от сложности файла, знакомости IDA с компилятором и личных предпочтений. Чем больше проблем возникло при автоматическом анализе, тем больше ручных исправлений потребуется.

Параметры отображения кода

Самое простое преобразование - настройка объёма сведений в каждой строке. IDA рассматривает её как набор частей. Метка, мнемоника и операнды присутствуют всегда, а дополнительные элементы выбираются на вкладке дизассемблирования в общих параметрах, показанной на рисунке 7-4.

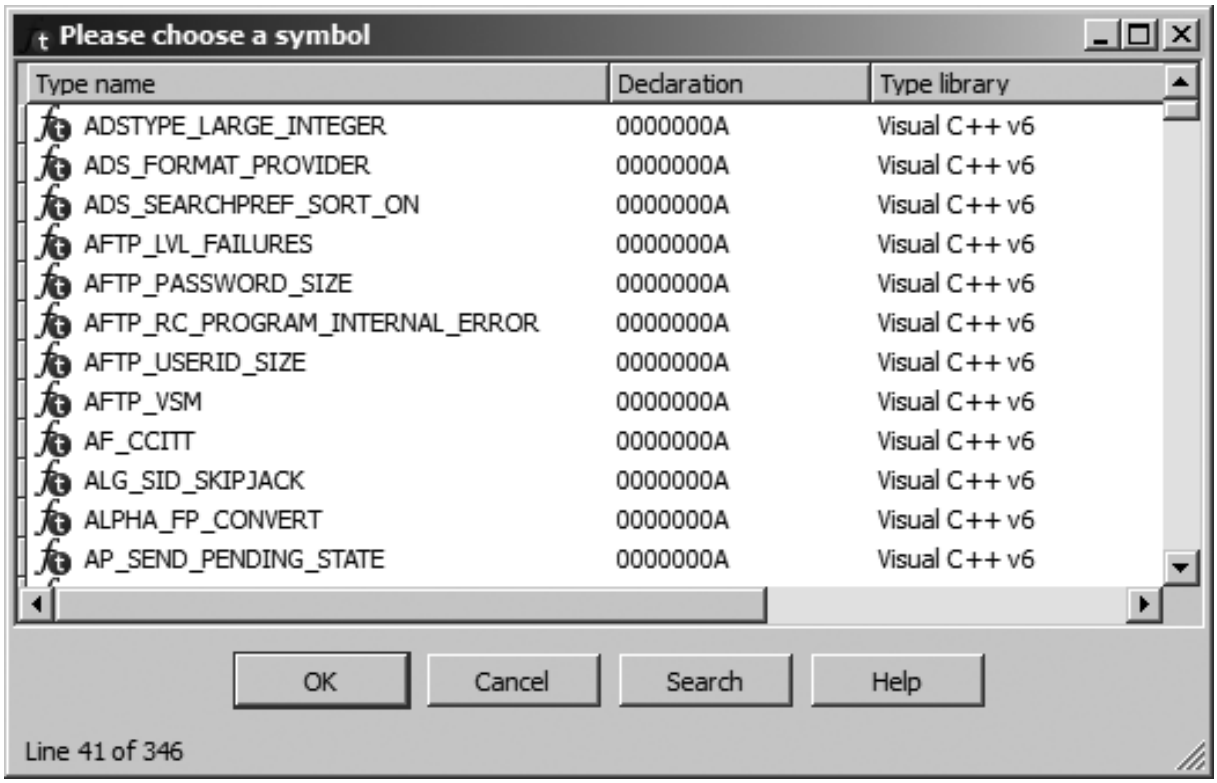


Рисунок 7-4. Параметры отображения строк дизассемблирования

В верхней правой части окна доступны следующие элементы строки. Для текстового листинга по умолчанию включены префиксы, обычные и повторяемые комментарии.

Префиксы строк

Префикс - часть `раздел:адрес`. Если отключить параметр, она исчезнет, как по умолчанию в графе. В приведённом ниже листинге префиксы скрыты.

Указатель стека

IDA подробно отслеживает изменения указателя стека каждой функции, что необходимо для восстановления кадра. При включении параметра в столбце X показывается относительное изменение на протяжении функции.

Это помогает заметить несоответствие соглашения о вызовах, например нераспознанный `stdcall`, или необычные манипуляции со стеком. После первой инструкции в примере смещение равно четырём байтам, после третьей - `0x7C`, а перед завершением снова становится нулевым.

Если при инструкции возврата значение не равно нулю, IDA помечает ошибку и выделяет строку красным. Причиной может быть намеренное противодействие автоматическому анализу либо незнакомый программе пролог или эпилог.

Обычные и повторяемые комментарии

Отключение соответствующего параметра скрывает данный вид заметок и уменьшает перегруженность листинга.

Автоматические комментарии

IDA способна пояснять некоторые инструкции, напоминая их действие. Для очевидных команд вроде x86 `mov` комментарии не добавляются. Примеры отмечены Y. Пользовательская заметка имеет преимущество; чтобы увидеть автоматическую, нужно удалить обычный и повторяемый комментарии строки.

Метки неправильных инструкций <BAD>

IDA может отмечать допустимые для процессора инструкции, которые понимают не все ассемблеры, в частности недокументированные. Тогда байты оформляются как данные, а инструкция приводится в комментарии после `<BAD>`. Цель - создать листинг, пригодный для большинства ассемблеров. Подробнее это описано во встроенной справке.

Число байтов кода операции

Многие дизассемблеры показывают рядом с инструкцией породившие её машинные байты. В IDA их можно видеть через синхронизированное шестнадцатеричное окно либо непосредственно вперемешку с ассемблером, задав количество байтов в строке.

Для архитектуры с постоянной длиной инструкции это просто. В x86 размер меняется от одного до более чем двенадцати байтов. Независимо от настоящей длины IDA резервирует указанное пространство и сдвигает остальную строку вправо. В примере выбрано пять байтов, столбец отмечен Z. Знак + в [сообщает, что инструкция целиком не поместилась.

```
X   Z
000 55                  push    ebp
004 89 E5               mov     ebp, esp
004 83 EC 78            sub     esp, 78h      Y; вычитание целых
07C 8B 45 10            mov     eax, [ebp+arg_8]
07C 89 45 F4            mov     [ebp+var_C], eax
07C 8B 45 0C            mov     eax, [ebp+arg_4]
07C 89 45 A4            mov     [ebp+var_5C], eax
07C C7 45 A0 0A [00+    mov     [ebp+var_60], 0Ah
07C C6 45 A8 41         mov     [ebp+var_58], 41h
07C 8B 45 A4            mov     eax, [ebp+var_5C]
07C 89 44 24 04         mov     [esp+4], eax
07C 8B 45 A0            mov     eax, [ebp+var_60]
07C 89 04 24            mov     [esp], eax
07C E8 91 FF FF FF      call    bar           Y; вызов процедуры
07C C9                  leave                 Y; выход из процедуры высокого уровня
000 C3                  retn                  Y; ближний возврат из процедуры
```

Отступы и поля настраиваются в нижней правой части окна. Все изменения действуют только для текущей базы. Глобальные значения хранятся в `<КАТАЛОГ_IDA>/cfg/ida.cfg`.

Форматирование операндов инструкций

При дизассемблировании IDA решает, как показывать операнды, особенно целочисленные константы. Они могут быть относительными смещениями переходов и вызовов, абсолютными адресами глобальных переменных, арифметическими значениями или определёнными программистом константами.

Ради читаемости IDA по возможности заменяет числа символами. Иногда решение следует из контекста инструкции, например вызова, иногда - из вида данных, например глобальной переменной или смещения кадра. Если назначение неясно, обычно выбирается шестнадцатеричная запись.

Если вы не относитесь к немногочисленным людям, которые мыслят шестнадцатеричными числами даже во сне, параметры форматирования окажутся полезными. Контекстное меню константы показано на рисунке 7-5.

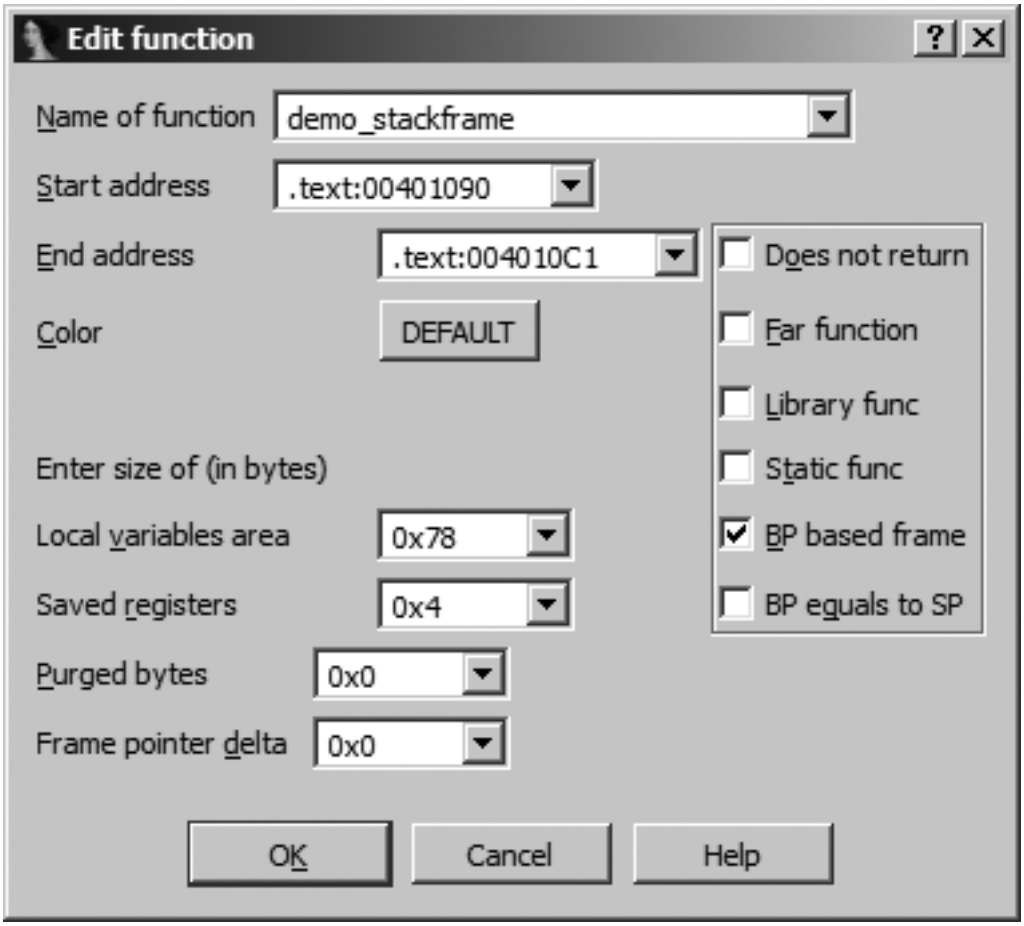


Рисунок 7-5. Параметры форматирования констант

Значение 41h можно представить десятичным, восьмеричным или двоичным. Поскольку оно входит в печатный диапазон ASCII, доступен и символьный вид. Меню сразу показывает точный текст замены.

В исходном коде программисты часто используют именованные константы из директив `#define` или перечислений. После компиляции уже нельзя понять, было ли значение символьным или буквальным числом. IDA хранит большой каталог констант распространённых библиотек, включая стандартную библиотеку Си и программный интерфейс Windows.

Каталог открывается контекстной командой стандартной символьной константы. Для `0Ah` появится окно на рисунке 7-6 со всеми известными IDA именами этого значения.

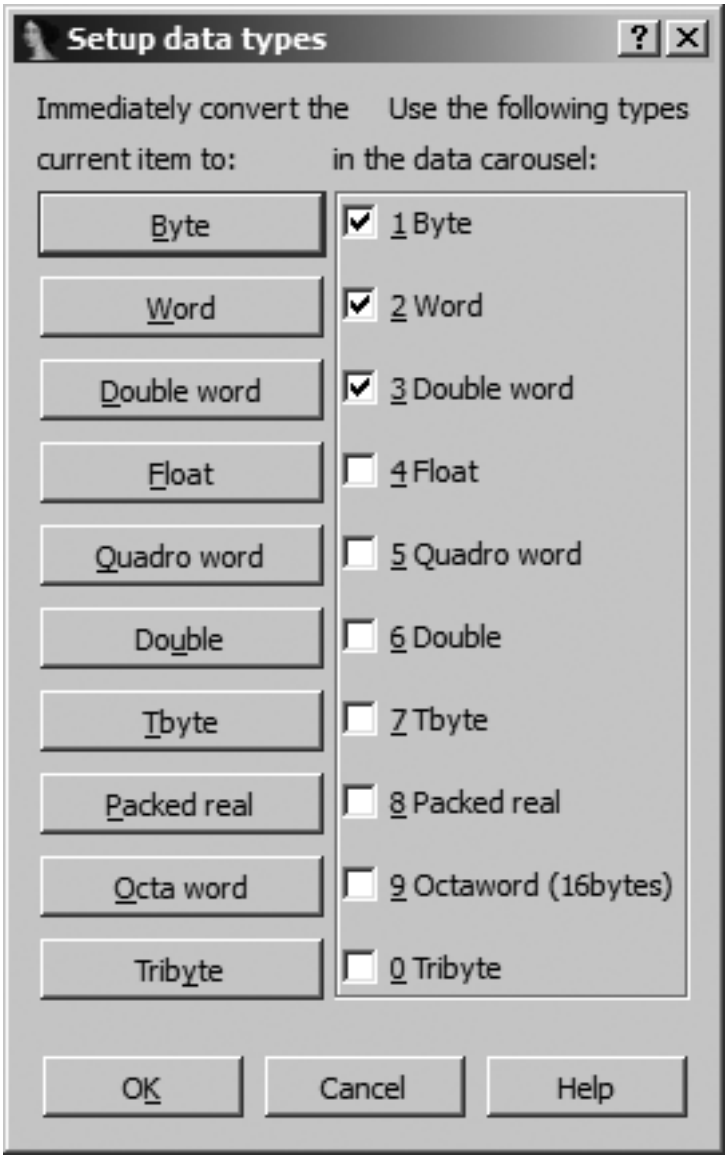


Рисунок 7-6. Окно выбора символа

Если число применяется при создании сетевого соединения X.25, можно выбрать `AF_CCITT` и получить строку:

```
.text:004010A2    mov     [ebp+var_60], AF_CCITT
```

Список помогает быстро найти вероятное известное имя и не просматривать вручную обширную документацию программного интерфейса.

Изменение функций

После автоматического анализа функции приходится исправлять по разным причинам. IDA может не найти функцию, если не обнаружила ведущего к ней вызова. Иногда неверно определяется конец, особенно если компилятор разбил функцию на несколько диапазонов адресов или ради экономии объединил общие завершающие последовательности двух функций.

Создание функций

Новую функцию можно создать из инструкций, ещё не принадлежащих другой функции, или из сырых байтов данных, которые IDA никак не определила.

Установите указатель на первый байт или инструкцию и выберите создание функции в меню редактирования. При необходимости IDA сначала преобразует данные в код, затем просматривает его вперёд, определяет структуру и ищет возврат.

Если подходящий конец найден, программа создаёт имя, анализирует кадр стека и оформляет код как функцию. При невозможности найти конец или встрече с недопустимой инструкцией операция завершается неудачно.

Удаление функций

Существующую функцию можно удалить соответствующей командой меню. Это требуется, если автоматический анализ ошибочно принял данные или часть другой конструкции за самостоятельную функцию.

Фрагменты функций

Фрагменты часто встречаются в коде Microsoft Visual C++. Компилятор перемещает редко выполняемые блоки, чтобы часто используемые помещались на страницы памяти, которые с меньшей вероятностью будут выгружены.

При таком разделении IDA следует по переходам и пытается найти все части. Обычно найденные фрагменты перечисляются в заголовке функции:

```
.text:004037AE ChunkedFunc proc near
.text:004037AE
.text:004037AE var_420    = dword ptr -420h
.text:004037AE var_41C    = dword ptr -41Ch
.text:004037AE var_4      = dword ptr -4
.text:004037AE hinstDLL   = dword ptr  8
.text:004037AE fdwReason  = dword ptr  0Ch
.text:004037AE lpReserved = dword ptr  10h
.text:004037AE
.text:004037AE ; ФРАГМЕНТ ФУНКЦИИ ПО АДРЕСУ X.text:004040D7, РАЗМЕР 00000011 БАЙТ
.text:004037AE ; ФРАГМЕНТ ФУНКЦИИ ПО АДРЕСУ .text:004129ED, РАЗМЕР 0000000A БАЙТ
.text:004037AE ; ФРАГМЕНТ ФУНКЦИИ ПО АДРЕСУ .text:00413DBC, РАЗМЕР 00000019 БАЙТ
.text:004037AE
.text:004037AE    push    ebp
.text:004037AF    mov     ebp, esp
```

Двойной щелчок по адресу в X переносит к фрагменту. В листинге его границы обозначены комментариями и ссылкой на родительскую функцию:

```
.text:004040D7 ; НАЧАЛО ФРАГМЕНТА ФУНКЦИИ ChunkedFunc
.text:004040D7
.text:004040D7 loc_0040C0D7: ; ссылка на код: ChunkedFunc+72j
.text:004040D7    dec     eax
.text:004040D8    jnz     loc_403836
.text:004040DE    call    sub_4040ED
.text:004040E3    jmp     loc_403836
.text:004040E3 ; КОНЕЦ ФРАГМЕНТА ФУНКЦИИ ChunkedFunc
```

Иногда IDA пропускает часть либо, наоборот, принимает самостоятельную функцию за фрагмент. Новый фрагмент создаётся выделением диапазона, ещё не принадлежащего другой функции, и командой добавления хвоста. Затем выбирается родительская функция.

Примечание. В листинге используется термин «фрагмент функции», а в меню IDA та же сущность называется «хвостом функции».

Для удаления установите указатель внутри фрагмента и выберите удаление хвоста, после чего подтвердите действие.

Если фрагменты приносят больше проблем, чем пользы, отключите создание хвостов в параметрах ядра при первоначальной загрузке. Тогда функции будут содержать переходы за собственные границы. В текстовом виде IDA выделяет их красными линиями и стрелками, а в графе целевые блоки не показываются.

Свойства функций

IDA связывает с каждой распознанной функцией ряд свойств, многие из которых изменяются в окне на рисунке 7-7.

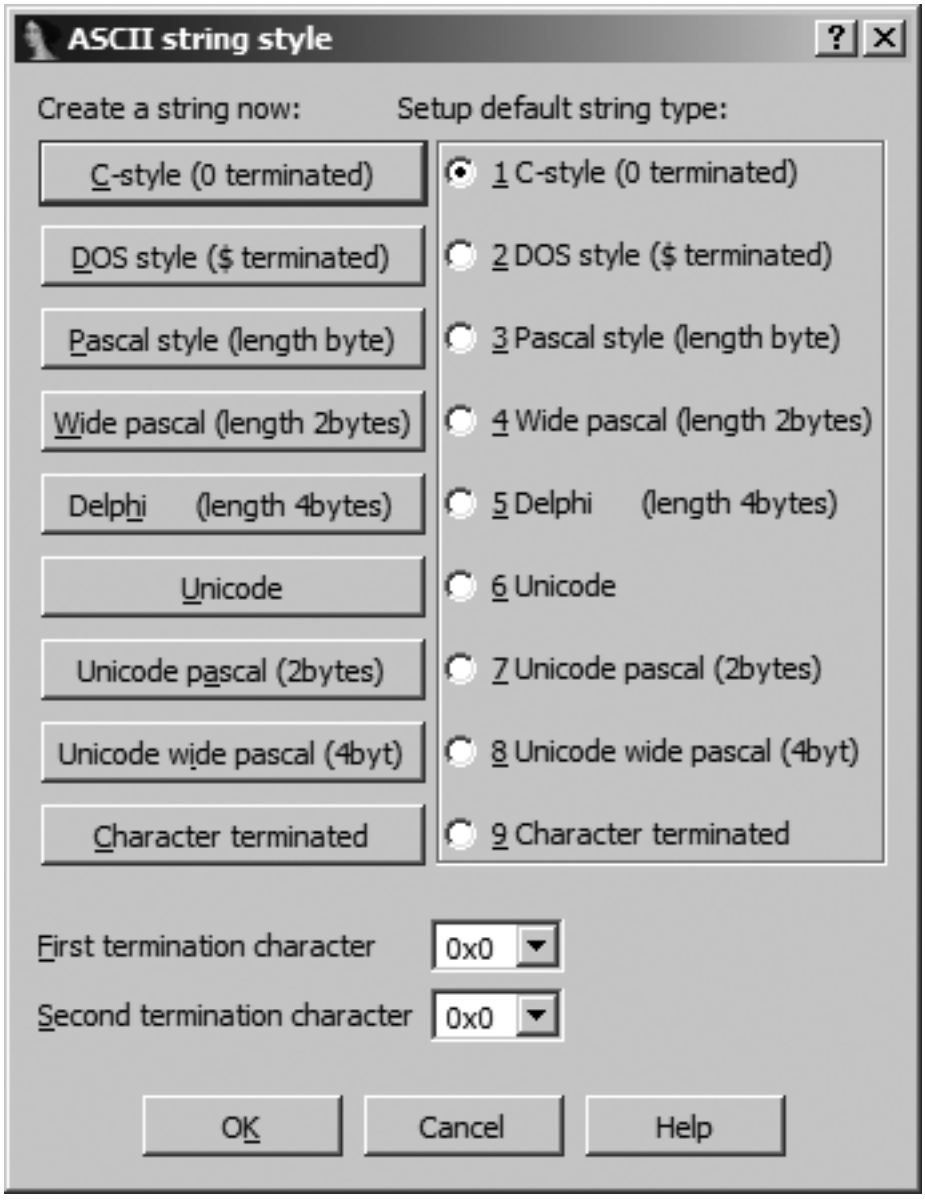


Рисунок 7-7. Окно редактирования функции

Имя функции

Дополнительный способ переименования.

Начальный адрес

Адрес первой инструкции. Обычно IDA определяет его при анализе или создании функции.

Конечный адрес

Адрес сразу после последней инструкции, чаще всего после возврата. Он не входит в функцию. Если автоматический анализ ошибся, значение можно исправить вручную.

Область локальных переменных

Количество байтов стека, выделенных локальным данным, как на рисунке 6-4. Обычно вычисляется по поведению указателя стека.

Сохранённые регистры

Число байтов для регистров вызывающей стороны. IDA считает эту область расположенной над сохранённым адресом возврата и под локальными данными. Если компилятор сохраняет регистры поверх локальных переменных, их место включается в область локальных данных, а не в отдельную область регистров.

Удаляемые байты

Число байтов параметров, удаляемых функцией при возврате. Для `cdecl` всегда равно нулю, для `stdcall` соответствует стековым аргументам. В x86 IDA определяет значение по инструкции `RET N`.

Смещение указателя кадра

Иногда компилятор направляет указатель в середину локальной области, а не на сохранённое прежнее значение. Расстояние между этими точками называется смещением указателя кадра и обычно вычисляется автоматически.

Такой приём ускоряет обращения: больше переменных остаётся в пределах однобайтового знакового смещения от `-128` до `+127`.

Дополнительные флажки характеризуют функцию и обычно отражают результат автоматического анализа.

Не возвращает управление

После вызова такой функции IDA не предполагает продолжение со следующей инструкции.

Дальняя функция

Применяется в сегментных архитектурах, где вызывающая сторона должна указать сегмент и смещение. Необходимость обычно определяется моделью памяти, например большой вместо плоской в x86.

Библиотечная функция

Помечает код как библиотечный: вспомогательную процедуру компилятора или часть статически скомпонованной библиотеки. Он показывается отдельным цветом.

Статическая функция

Только добавляет модификатор `static` в перечень свойств.

Кадр на основе BP

Означает наличие отдельного указателя кадра, обычно распознаваемого по прологу. Если анализ не справился, свойство можно включить вручную, одновременно скорректировав размеры сохранённых регистров и локальной области.

При включённом свойстве обращения через указатель кадра используют символьные имена стековых переменных. Иначе предполагается адресация относительно указателя стека.

BP равен SP

Некоторые функции при входе направляют указатель кадра на вершину стека вместе с указателем стека. Тогда следует включить этот флажок. Это равносильно смещению указателя кадра, равному размеру локальной области.

Корректировки указателя стека

IDA старается отслеживать изменение указателя на каждой инструкции. От точности зависит устройство кадра. Если влияние инструкции неизвестно, иногда нужна ручная корректировка.

Простейший случай - вызов функции `stdcall` из общей библиотеки, о которой IDA ничего не знает. Программа не понимает, что вызываемая функция сама очистила стек, и для остатка текущей функции показывает неверное смещение:

```
  .text:004010EB 01C      push    eax
  .text:004010F3 020      push    2
  .text:004010FB 024      push    1
  Y .text:00401102 028    call    some_imported_func
  .text:00401107 X028     mov     ebx, eax
```

Поскольку some_imported_func удаляет три параметра, правильное значение в X должно быть 01C. Можно выделить адрес Y, вызвать изменение указателя стека сочетанием Alt+K и указать 12 байт.

Если функция вызывается много раз, исправлять каждый вызов утомительно и легко что-нибудь пропустить. Лучше сообщить IDA поведение самой some_imported_func.

Переход к импортированной функции приводит к записи таблицы импорта:

```
.idata:00418078 ; Тип сегмента: внешние символы
.idata:00418078 ; _idata
.idata:00418078    extrn some_imported_func:dword ; ссылка на данные: sub_401034r
```

Даже для импорта IDA разрешает изменить число удаляемых байтов. Укажите, сколько функция снимает со стека при возврате, и значение автоматически распространится на все места вызова, мгновенно исправив расчёты.

Для улучшения автоматического анализа IDA пытается разрешать расхождения указателя стека решением системы линейных уравнений. Поэтому пользователь может даже не заметить, что программа изначально не знала подробностей функции. Приём описан в записи блога Ильфака «Симплекс-метод в IDA Pro»: <http://hexblog.com/2006/06/>.

Преобразование данных в код и обратно

При автоматическом анализе байты иногда классифицируются неверно: данные принимаются за инструкции или код оформляется как значения. Причин много. Некоторые компиляторы встраивают данные в раздел кода, на некоторые инструкции нет прямых ссылок, а обфусцированные программы намеренно размывают границу.

Первый способ исправления - удалить текущее определение. Выделите объект и нажмите U либо выберите отмену определения в меню. Байты снова станут необработанными значениями. Диапазон можно предварительно выделить перетаскиванием.

Например, исходная функция:

```
.text:004013E0 sub_4013E0 proc near
.text:004013E0    push    ebp
.text:004013E1    mov     ebp, esp
.text:004013E3    pop     ebp
.text:004013E4    retn
.text:004013E4 sub_4013E0 endp
```

После отмены определения превращается в байты:

```
.text:004013E0 unk_4013E0 db  55h ; U
.text:004013E1             db  89h
.text:004013E2             db 0E5h
.text:004013E3             db  5Dh ; ]
.text:004013E4             db 0C3h
```

Чтобы дизассемблировать неопределённые байты, установите указатель на первый и нажмите C либо выберите преобразование в код. IDA идёт вперёд до уже определённого объекта или недопустимой инструкции. Для большого диапазона сначала выделите его.

Обратное преобразование немного сложнее: в контекстном меню его нет. Используйте команду данных или клавишу D. Для большого фрагмента проще сначала отменить определения всех инструкций, а затем выбрать нужный формат данных.

Основные преобразования данных

Правильное оформление данных не менее важно для понимания программы, чем точный код. IDA выбирает формат по сведениям из разных источников:

1. Тип и размер выводятся из использования регистров. Загрузка 32-разрядного регистра из памяти указывает на четырёхбайтовое значение, хотя отличить целое от указателя не всегда можно.
2. Объявление функции задаёт типы параметров. IDA хранит большую библиотеку объявлений и пытается связать переданный аргумент с адресом памяти. Например, если единственный параметр имеет тип указателя на `CRITICAL_SECTION` из программного интерфейса Windows и адрес удалось определить, соответствующая область помечается объектом этого типа.
3. Последовательность байтов сама подсказывает вероятный тип. Длинный набор печатных знаков ASCII разумно считать массивом знаков, то есть строкой.

Задание размера данных

Простейшая правка - изменение размера. Часто встречаются обозначения `db`, `dw` и `dd`, соответствующие 1, 2 и 4 байтам. Размер и тип настраиваются в окне на рисунке 7-8.

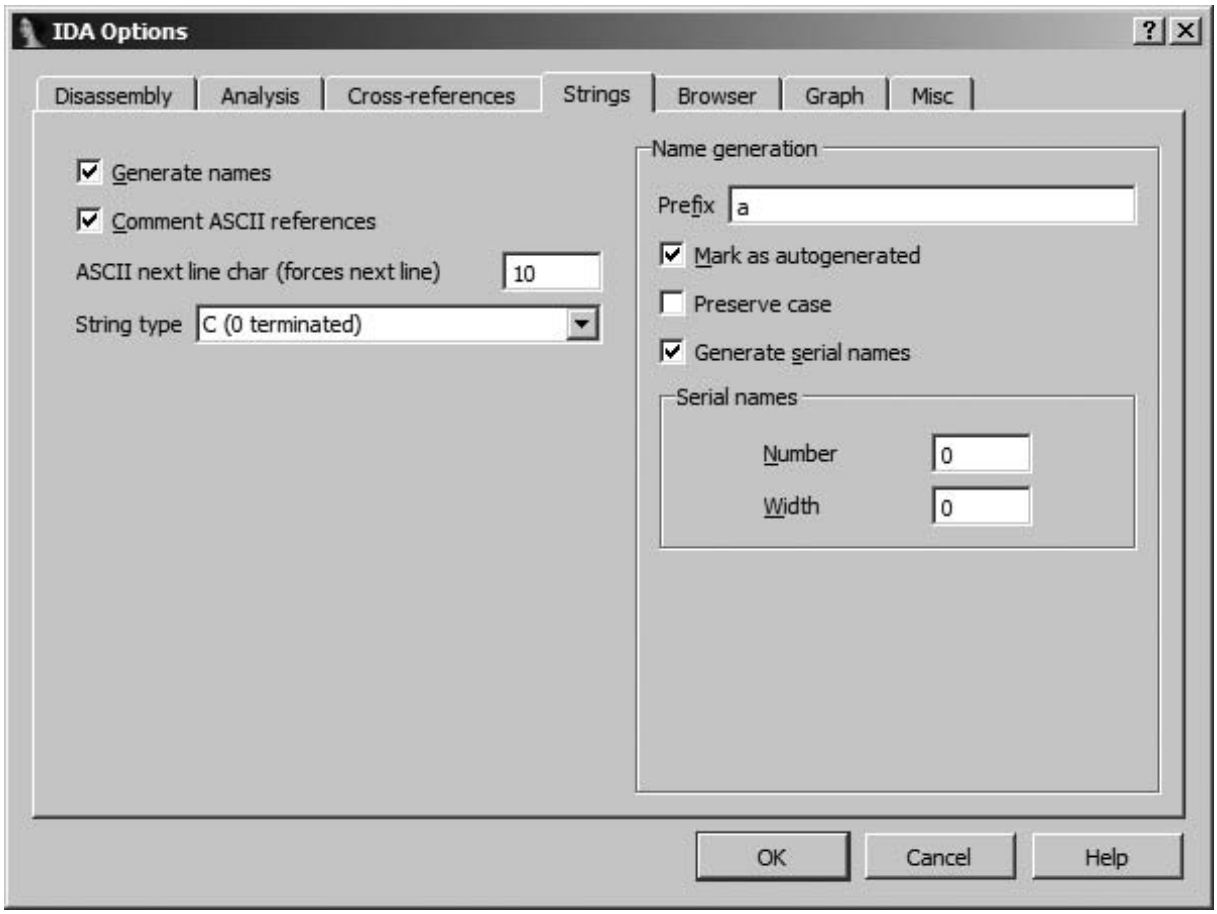


Рисунок 7-8. Окно настройки типов данных

Кнопки слева немедленно меняют выбранный объект. Флажки справа настраивают «карусель данных» - циклический список доступных типов. Каждой кнопке соответствует флажок. Сама настройка не меняет листинг, но включённые типы появляются в контекстном меню данных.

Клавиша `D` переводит выбранный адрес к следующему типу карусели. При списке `db`, `dw`, `dd` байт становится словом, слово - двойным словом, а двойное слово снова байтом. Для кода или неопределённых байтов выбирается первый тип.

Размер может увеличиться, уменьшиться или остаться прежним. При уменьшении лишние байты становятся неопределёнными. При увеличении IDA предупреждает, если следующие байты уже заняты, и спрашивает о прямом преобразовании в данные. Согласие означает отмену достаточного числа последующих определений. Например, переход от db к dd поглощает ещё три байта.

Тип и размер меняются в любом месте с данными, включая стек. Откройте подробное представление кадра двойным щелчком по переменной и отредактируйте её как обычный объект.

Работа со строками

IDA понимает множество строковых форматов и по умолчанию ищет завершённые нулём строки Си. Чтобы принудительно преобразовать данные, выберите вид в меню строк. Если байты ему соответствуют, они объединяются в одну переменную. Клавиша A применяет стандартный строковый формат.

Первое окно настройки показано на рисунке 7-9. Несмотря на название ASCII, оно поддерживает гораздо больше вариантов.

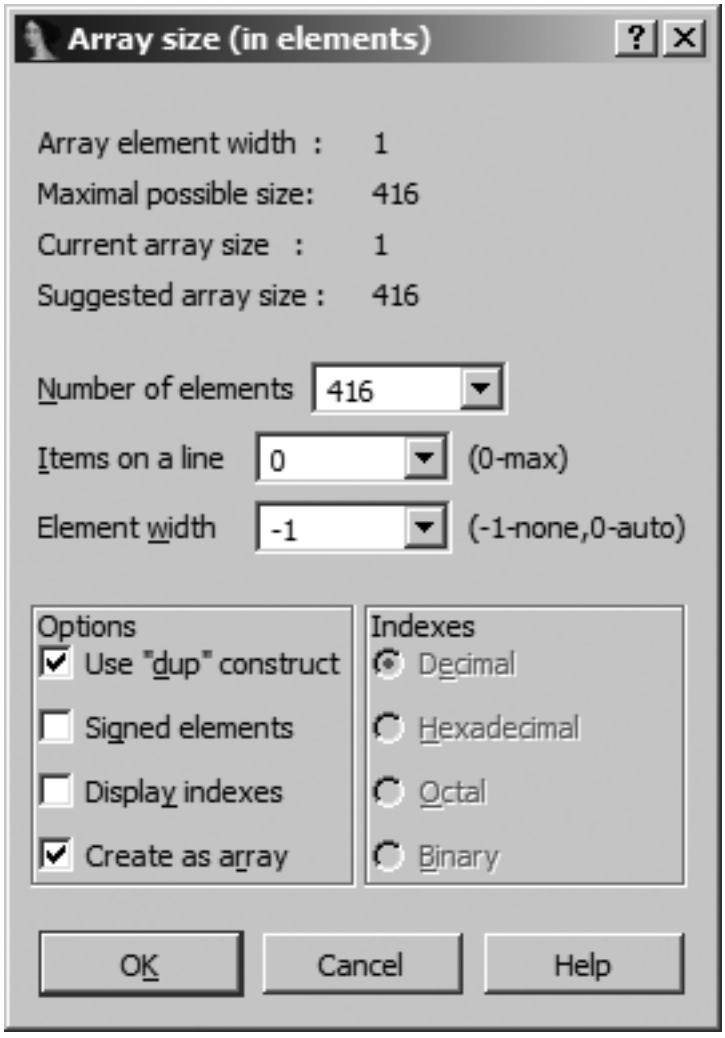


Рисунок 7-9. Настройка строковых данных

Кнопки слева создают строку выбранного вида в текущем месте, если данные подходят. Для строк с завершающим знаком внизу задаются до двух таких знаков. Переключатели справа выбирают стандартный вид для клавиши A.

Второе окно находится на вкладке строк общих параметров и показано на рисунке 7-10. Там также выбирается стандартный тип, но большинство настроек управляет именами и отображением независимо от формата. Область создания имён видна только при включённом параметре.

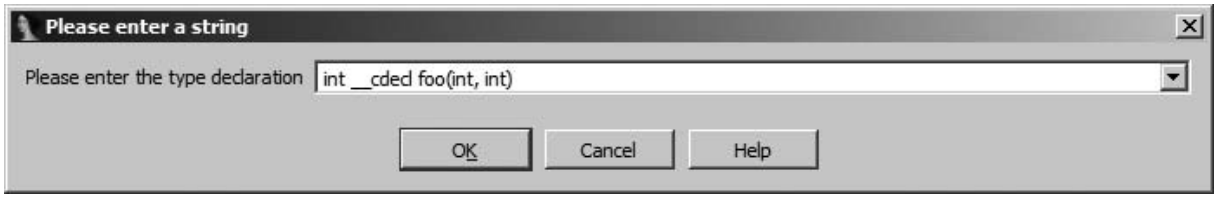


Рисунок 7-10. Параметры строк IDA

Если автоматическое создание имён отключено, строковые переменные получают фиктивные имена с префиксом `asc_`.

При включении имена создаются по следующим правилам. Без последовательных имён стандартный префикс соединяется со знаками строки до предельной длины. Например:

```
.rdata:00402069 aThisIsACharact db 'This is a Character array',0
```

Для имени используются заглавные начала слов, а недопустимые знаки, например пробелы, отбрасываются. Пометка автоматически созданного имени задаёт ему отдельный цвет, по умолчанию тёмно-синий вместо синего пользовательских имён. Сохранение регистра запрещает преобразование букв.

Последовательные имена получают числовые суффиксы, начиная с указанного номера и с заданным числом разрядов. При настройке с рисунка первыми будут `a000`, `a001` и `a002`.

Создание массивов

В листинге из языка высокого уровня мало подсказок о размере массива. Если каждый элемент занимает отдельную строку, объявление получается огромным. В следующем фрагменте данные идут после `unk_402060`. Только первый элемент имеет прямые ссылки, что часто указывает на начало массива: остальные обычно адресуются вычислением индекса от базы.

```
.rdata:00402060 unk_402060 db 0 ; ссылки на данные: sub_401350+8o
.rdata:00402060                 ; sub_401350+18o
.rdata:00402061             db 0
.rdata:00402062             db 0
.rdata:00402063             db 0
.rdata:00402064             db 0
.rdata:00402065             db 0
.rdata:00402066             db 0
.rdata:00402067             db 0
.rdata:00402068             db 0
.rdata:00402069             db 0
.rdata:0040206A             db 0
```

IDA объединяет последовательные определения в массив. Выберите первый элемент, здесь `unk_402060`, и откройте команду массива. В контекстном меню она доступна только для уже определённых данных. Тип массива наследуется от первого элемента; здесь это байты.

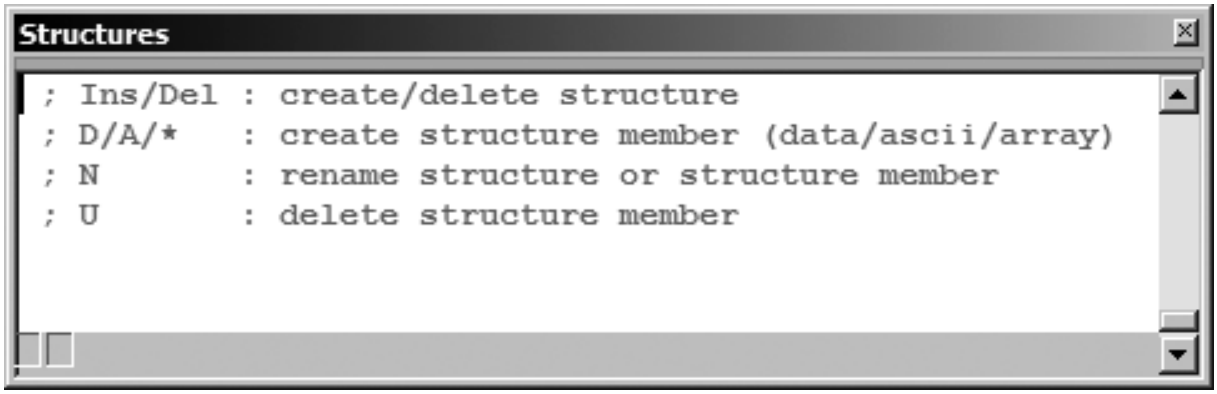


Рисунок 7-11. Окно создания массива

> **Примечание.** Перед созданием массива задайте правильный размер элементов, изменив тип первого объекта.

Поля окна имеют следующее назначение.

Ширина элемента массива

Размер одного элемента, здесь 1 байт, определяемый выбранным объектом.

Максимально возможный размер

Автоматически рассчитанное число элементов, которые помещаются до следующего определённого объекта. Больший размер потребует отменить последующие определения и поглотить их.

Число элементов

Точный размер массива. Общий объём равен числу элементов, умноженному на ширину.

Элементов в строке

Количество значений на одной строке листинга, позволяющее сократить занимаемое место.

Ширина элемента

Только форматирование столбца при выводе нескольких значений в строке.

Использовать конструкцию dup

Одинаковые значения объединяются одним объектом с количеством повторений.

Знаковые элементы

Выбирает знаковое или беззнаковое представление.

Показывать индексы

Добавляет индексы массива как обычные комментарии. Это удобно для поиска значения в большом массиве. Дополнительно можно выбрать формат индекса.

Создать как массив

Обычно параметр включён. Отключение создаёт заданное число последовательных элементов без объединения в массив.

С настройками рисунка получается компактное объявление массива байтов `byte_402060`, состоящего из 416, то есть `1A0h`, нулей:

```
.rdata:00402060 byte_402060 db 1A0h dup(0) ; ссылка на данные: sub_401350+8o
.rdata:00402060                            ; sub_401350+18o
```

Благодаря `dup` 416 строк сокращаются до одной. Создание массивов внутри кадров стека рассматривается в следующей главе.

Итоги

Эта и предыдущая главы охватывают самые частые действия пользователя IDA. Изменяя базу, вы объединяете собственные выводы со знаниями, полученными программой при анализе, и создаёте гораздо более полезный результат.

Как и в исходном коде, осмысленные имена, точные типы и подробные комментарии помогают не только самому аналитику вспомнить проделанную работу, но и всем, кому она достанется позднее.

В следующей главе мы перейдём к сложным структурам данных, включая структуры Си, и рассмотрим некоторые низкоуровневые особенности скомпилированного C++.

Глава 8. Типы и структуры данных

Самый очевидный путь к пониманию двоичной программы - составить перечень вызываемых библиотечных функций. Программа на Си, обращающаяся к `connect`, создаёт сетевое соединение. Программа Windows, вызывающая `RegOpenKey`, работает с реестром. Но чтобы понять, как и зачем совершаются эти вызовы, нужен дальнейший анализ.

Для этого прежде всего следует выяснить передаваемые параметры. В случае `connect` важно не только наличие вызова, но и точный сетевой адрес назначения. Понимание данных - ключ к восстановлению сигнатуры функции: количества, типов и порядка параметров. Поэтому необходимо знать, как типы и структуры представлены на уровне языка ассемблера.

В этой главе рассматривается передача сведений о типах в IDA, размещение структур в памяти и доступ к их полям.

Распространение типов от функций

Простейший способ установить тип переменной - заметить, что она передаётся функции с известным объявлением. Во время анализа IDA старается пояснить типы на основе использования переменной. По возможности она берёт из объявления даже формальное имя параметра вместо фиктивного.

Это видно в вызове `connect`:

```
.text:004010F3    push    10h             ; namelen
.text:004010F5    lea     ecx, X[ebp+name]
.text:004010F8    push    ecx             ; name
.text:004010F9    mov     edx, X[ebp+s]
.text:004010FF    push    edx             ; s
.text:00401100    call    connect
```

Каждая `push` снабжена именем передаваемого параметра из известного IDA объявления. Две локальные переменные X также названы в соответствии с параметрами. Обычно такие имена гораздо полезнее стандартных.

Распространение не ограничено библиотечными функциями из встроенных библиотек типов. IDA способна переносить имена и типы параметров любой функции базы, если пользователь явно задал её объявление.

При первоначальном анализе всем аргументам назначаются фиктивные имена и общий тип `int`, если распространение не подсказало другое. Тип функции изменяется командой назначения типа, контекстным меню имени или клавишей Y.

Для следующей функции открывается окно на рисунке 8-1:

```
.text:00401050 ; ======== П О Д П Р О Г Р А М М А ========================
.text:00401050
.text:00401050 ; Свойства: кадр на основе BP
.text:00401050
.text:00401050 foo proc near ; ссылка на код: demo_stackframe+2Ap
.text:00401050
.text:00401050 arg_0 = dword ptr  8
.text:00401050 arg_4 = dword ptr  0Ch
.text:00401050
.text:00401050    push    ebp
.text:00401051    mov     ebp, esp
```

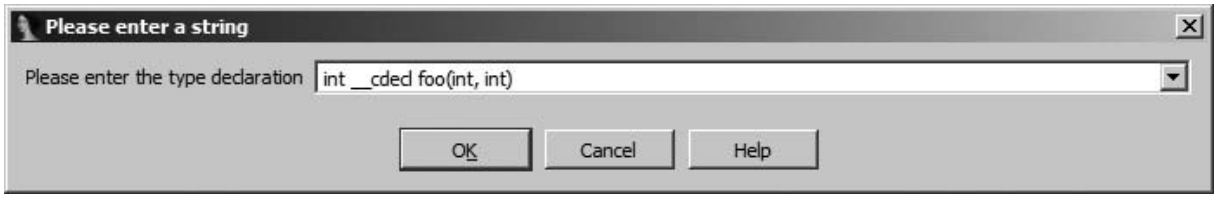


Рисунок 8-1. Задание типа функции

IDA предполагает возвращаемый тип `int`, по виду `ret` правильно определяет соглашение `cdecl`, использует изменённое имя функции и считает все параметры целыми. Пока аргументы не переименованы, отображаются только их типы.

Если ввести объявление

```
int __cdecl foo(float f, char *ptr);
```

IDA добавит комментарий с прототипом X и изменит имена аргументов Y:

```
.text:00401050 ; ======== П О Д П Р О Г Р А М М А ========================
.text:00401050
.text:00401050 ; Свойства: кадр на основе BP
.text:00401050
.text:00401050 X; int __cdecl foo(float f, char *ptr)
.text:00401050 foo proc near ; ссылка на код: demo_stackframe+2Ap
.text:00401050
.text:00401050 Yf   = dword ptr  8
.text:00401050 Yptr = dword ptr  0Ch
.text:00401050
.text:00401050       push    ebp
.text:00401051       mov     ebp, esp
```

Затем сведения распространяются на все вызовы. Имена f и ptr появляются как комментарии P и переименовывают бывшие фиктивные переменные Q:

```
.text:004010AD    mov     eax, [ebp+Qptr]
.text:004010B0    mov     [esp+4], eax  P; ptr
.text:004010B4    mov     eax, [ebp+Qf]
.text:004010B7    mov     [esp], eax    P; f
.text:004010BA    call    foo
```

Для импортированной библиотечной функции IDA часто уже знает объявление. Чтобы увидеть его, задержите указатель мыши над именем[1]. Если порядок параметров неизвестен, по крайней мере в окне импорта должна быть указана библиотека. Тогда следует обратиться к справочным страницам или документации программного интерфейса, например MSDN[2]. Если ничего не помогает, помните: поиск в сети - ваш друг.

Далее мы будем распознавать структуры, восстанавливать их устройство и улучшать листинг IDA. Поскольку классы C++ являются сложным развитием структур Си, глава завершается обратной разработкой скомпилированного C++.

Распознавание структур данных

Примитивный тип обычно соответствует размеру регистра или операнда процессора. Составные типы - массивы и структуры - требуют более сложных последовательностей для доступа к отдельным элементам. Прежде чем применять возможности IDA, нужно понять, как выглядит такой код.

Доступ к элементам массива

Массив - простейшая составная структура памяти: непрерывный блок последовательных элементов одного типа. Его размер равен числу элементов, умноженному на размер каждого.

Для объявления

```
int array_demo[100];
```

минимальное количество байтов вычисляется так:

```
int bytes = 100 * sizeof(int);
```

Элемент выбирается постоянным или переменным индексом:

```
X array_demo[20] = 15;  // постоянный индекс
for (int i = 0; i < 100; i++) {
Y    array_demo[i] = i; // переменный индекс
}
```

Пусть `sizeof(int)` равно 4. Тогда X обращается к целому на смещении 80 байт, а Y последовательно к смещениям 0, 4, 8 и так далее. Первое значение `20 * 4` компилятор вычисляет заранее. Второе обычно приходится получать во время выполнения, умножая текущий i на 4.

Точный способ зависит не только от вида индекса, но и от места размещения массива.

Глобальные массивы

Для массива в глобальной области, например разделе `.data` или `.bss`, базовый адрес известен при компиляции. Поэтому адрес элемента с постоянным индексом можно вычислить заранее.

Рассмотрим программу:

```
int global_array[3];

int main() {
    int idx = 2;
    global_array[0] = 10;
    global_array[1] = 20;
    global_array[2] = 30;
    global_array[idx] = 40;
}
```

Листинг:

```
.text:00401000 _main proc near
.text:00401000
.text:00401000 idx = dword ptr -4
.text:00401000
.text:00401000    push    ebp
.text:00401001    mov     ebp, esp
.text:00401003    push    ecx
.text:00401004    mov     [ebp+idx], 2
.text:0040100B Xmov     dword_40B720, 10
.text:00401015 Ymov     dword_40B724, 20
.text:0040101F Zmov     dword_40B728, 30
.text:00401029    mov     eax, [ebp+idx]
.text:0040102C [mov     dword_40B720[eax*4], 40
.text:00401037    xor     eax, eax
.text:00401039    mov     esp, ebp
.text:0040103B    pop     ebp
.text:0040103C    retn
.text:0040103C _main endp
```

Хотя глобальная переменная одна, строки X, Y и Z выглядят как три разные. Только вычисление `eax * 4` в [намекает, что `dword_40B720` - массив.

По фиктивным именам видно, что он занимает 12 байт с адреса `0040B720`. Постоянные индексы 0, 1 и 2 компилятор превратил в адреса `0040B720`, `0040B724` и `0040B728`. Если оформить `dword_40B720` как массив из трёх элементов, IDA подчеркнёт смещения:

```
.text:0040100B    mov     dword_40B720, 10
.text:00401015    mov     dword_40B720+4, 20
.text:0040101F    mov     dword_40B720+8, 30
```

Следует запомнить два вывода. Во-первых, постоянный индекс глобального массива выглядит как обращение к отдельной глобальной переменной и почти не выдаёт существование массива. Во-вторых, переменный индекс раскрывает базовый адрес, к которому добавляется вычисленное смещение. Множитель индекса, здесь 4, сообщает размер элемента, хотя не его тип.

Массивы в стеке

Адрес стекового массива при компиляции неизвестен, но на практике компилятор обращается с ним почти так же, как с глобальным.

```
int main() {
    int stack_array[3];
    int idx = 2;
    stack_array[0] = 10;
    stack_array[1] = 20;
    stack_array[2] = 30;
    stack_array[idx] = 40;
}
```

Листинг:

```
.text:00401000 _main proc near
.text:00401000
.text:00401000 var_10 = dword ptr -10h
.text:00401000 var_C  = dword ptr -0Ch
.text:00401000 var_8  = dword ptr -8
.text:00401000 idx    = dword ptr -4
.text:00401000
.text:00401000    push    ebp
.text:00401001    mov     ebp, esp
.text:00401003    sub     esp, 10h
.text:00401006    mov     [ebp+idx], 2
.text:0040100D Xmov     [ebp+var_10], 10
.text:00401014 Ymov     [ebp+var_C], 20
.text:0040101B Zmov     [ebp+var_8], 30
.text:00401022    mov     eax, [ebp+idx]
.text:00401025 [mov     [ebp+eax*4+var_10], 40
.text:0040102D    xor     eax, eax
.text:0040102F    mov     esp, ebp
.text:00401031    pop     ebp
.text:00401032    retn
.text:00401032 _main endp
```

Снова вместо массива видны три переменные. По присваиваемым константам X, Y и Z можно понять, что это элементы, первый из которых находится в `var_10`, имеющей самый низкий адрес.

Для `stack_array[1]` нужно смещение 4 байта от `var_10`. Массив расположен по `ebp - 0x10`, поэтому элемент находится по `ebp - 0x10 + 4`, то есть `ebp - 0x0C`. IDA закономерно показывает это как локальную `var_C`.

Таким образом, постоянный индекс снова скрывает массив. Только строка [указывает, что `var_10` - первый элемент, а множитель 4 раскрывает размер каждого значения.

У стекового варианта есть дополнительная подсказка: по положению `idx` видно, что массив содержит не более трёх элементов, иначе он перекроет эту переменную. Разработчику эксплойта это помогает определить, сколько данных помещается до повреждения следующего объекта.

Массивы в куче

Массив в куче создаётся динамической функцией вроде `malloc` в Си или `new` в C++. Все обращения строятся от адреса, возвращённого распределителем памяти.

```
int main() {
    int *heap_array = (int*)malloc(3 * sizeof(int));
    int idx = 2;
    heap_array[0] = 10;
    heap_array[1] = 20;
    heap_array[2] = 30;
    heap_array[idx] = 40;
}
```

Соответствующий листинг:

```
.text:00401000 _main proc near
.text:00401000
.text:00401000 heap_array = dword ptr -8
.text:00401000 idx        = dword ptr -4
.text:00401000
.text:00401000    push    ebp
.text:00401001    mov     ebp, esp
.text:00401003    sub     esp, 8
.text:00401006 \push    0Ch       ; size_t
.text:00401008    call    _malloc
.text:0040100D    add     esp, 4
.text:00401010    mov     [ebp+heap_array], eax
.text:00401013    mov     [ebp+idx], 2
.text:0040101A    mov     eax, [ebp+heap_array]
.text:0040101D Xmov     dword ptr [eax], 10
.text:00401023    mov     ecx, [ebp+heap_array]
.text:00401026 Ymov     dword ptr [ecx+4], 20
.text:0040102D    mov     edx, [ebp+heap_array]
.text:00401030 Zmov     dword ptr [edx+8], 30
.text:00401037    mov     eax, [ebp+idx]
.text:0040103A    mov     ecx, [ebp+heap_array]
.text:0040103D [mov     dword ptr [ecx+eax*4], 40
.text:00401044    xor     eax, eax
.text:00401046    mov     esp, ebp
.text:00401048    pop     ebp
.text:00401049    retn
.text:00401049 _main endp
```

Начальный адрес из EAX сохраняется в `heap_array`. Перед каждым обращением его приходится читать, а затем добавлять смещение. В X, Y и Z применяются соответственно 0, 4 и 8 байт. В [переменный индекс по-прежнему умножается на размер элемента.

У кучевого массива есть удобная особенность: если известны общий размер и размер элемента, легко вычислить количество. Параметр `0x0C`, переданный `malloc` в \, означает 12 выделенных байт. Деление на 4 даёт три элемента.

Надёжнее всего массив узнаётся по переменному индексу, который масштабируется размером элемента перед сложением с базовым адресом. Постоянные индексы почти не выдают массив и выглядят очень похоже на доступ к полям структуры.

Доступ к полям структуры

Структуры в стиле Си - разнородные наборы, объединяющие элементы разных типов. В отличие от массива, поля выбираются по имени, но компилятор превращает имена в числовые смещения. Поэтому в листинге доступ к полю почти неотличим от массива с постоянным индексом.

При обработке определения компилятор подсчитывает занятые байты и определяет смещение каждого поля. В дальнейших примерах используется:

```
struct ch8_struct {   // размер  мин. смещение  обычное смещение
    int field1;       //   4          0                 0
    short field2;     //   2          4                 4
    char field3;      //   1          6                 6
    int field4;       //   4          7                 8
    double field5;    //   8         11                16
};                    // минимум 19 байт, обычно 24 байта
```

Минимальный размер равен сумме полей, но нельзя считать, что компилятор обязательно использует минимум. По умолчанию поля выравниваются для эффективного чтения и записи: четырёхбайтовые целые по смещениям, кратным 4, восьмибайтовые вещественные по смещениям, кратным 8.

Для выравнивания вставляются заполняющие байты, поэтому фактический размер может превышать сумму. В примере стандартные смещения дают 24 байта.

Структуру можно уплотнить параметрами компилятора. Microsoft Visual C/C++ и GNU `gcc`/`g++` понимают директиву `pack`, а GNU также атрибут `packed` для отдельной структуры. Выравнивание в 1 байт даёт минимальные смещения и 19 байт.

Некоторые процессоры быстрее работают с естественным выравниванием, а другие вообще вызывают исключение при нарушении требуемой границы.

Доступ к полю, как и к элементу массива, складывает базовый адрес структуры со смещением. Но индекс массива можно вычислить во время выполнения благодаря одинаковому размеру элементов, тогда как смещение поля структуры должно быть известно заранее.

Структуры в глобальной памяти

Как и у глобальных массивов, адрес глобальной структуры известен во время компиляции. Поэтому компилятор заранее вычисляет адрес каждого поля, и во время выполнения никакие арифметические действия не нужны. Рассмотрим программу:

```
struct ch8_struct global_struct;

int main() {
    global_struct.field1 = 10;
    global_struct.field2 = 20;
    global_struct.field3 = 30;
    global_struct.field4 = 40;
    global_struct.field5 = 50.0;
}
```

При стандартном выравнивании дизассемблированный код выглядит примерно так:

```
.text:00401000 _main           proc near
.text:00401000                 push    ebp
.text:00401001                 mov     ebp, esp
.text:00401003                 mov     dword_40EA60, 10
.text:0040100D                 mov     word_40EA64, 20
.text:00401016                 mov     byte_40EA66, 30
.text:0040101D                 mov     dword_40EA68, 40
.text:00401027                 fld     ds:dbl_40B128
```

```
.text:0040102D                fstp    dbl_40EA70
.text:00401033                xor     eax, eax
.text:00401035                pop     ebp
.text:00401036                retn
.text:00401036 _main          endp
```

Здесь вообще нет арифметики для обращения к полям. Без исходного кода нельзя уверенно утверждать, что программа использует структуру: все смещения уже вычислены компилятором, поэтому листинг выглядит как работа с пятью отдельными глобальными переменными. Та же неоднозначность возникала у глобального массива с постоянными индексами.

Структуры в стеке

Структуру, размещённую в стеке, также трудно распознать только по схеме кадра. Если объявить предыдущую структуру внутри `main`, получится следующий листинг:

```
.text:00401000 _main          proc near
.text:00401000
.text:00401000 var_18         = dword ptr -18h
.text:00401000 var_14         = word ptr -14h
.text:00401000 var_12         = byte ptr -12h
.text:00401000 var_10         = dword ptr -10h
.text:00401000 var_8          = qword ptr -8
.text:00401000
.text:00401000                push    ebp
.text:00401001                mov     ebp, esp
.text:00401003                sub     esp, 18h
.text:00401006                mov     [ebp+var_18], 10
.text:0040100D                mov     [ebp+var_14], 20
.text:00401013                mov     [ebp+var_12], 30
.text:00401017                mov     [ebp+var_10], 40
.text:0040101E                fld     ds:dbl_40B128
.text:00401024                fstp    [ebp+var_8]
.text:00401027                xor     eax, eax
.text:00401029                mov     esp, ebp
.text:0040102B                pop     ebp
.text:0040102C                retn
.text:0040102C _main          endp
```

Относительные смещения полей внутри кадра известны во время компиляции, поэтому арифметики снова нет. Листинг создаёт обманчивое впечатление, будто используются пять отдельных переменных. На самом деле `var_18` должна обозначать начало 24-байтовой структуры, а остальные имена относятся к её полям.

Структуры в куче

Структуры в куче гораздо лучше раскрывают свой размер и расположение полей. Их адрес заранее неизвестен, поэтому при каждом обращении компилятор вынужден формировать нужное смещение относительно указателя. Для глобальной структуры он располагает постоянным начальным адресом, для стековой - постоянным отношением к указателю кадра, а для структуры в куче ему известен только указатель на начало выделенного блока.

Если изменить пример так, чтобы `main` выделяла структуру в куче, получится:

```
.text:00401000 _main          proc near
.text:00401000
.text:00401000 heap_struct    = dword ptr -4
.text:00401000
.text:00401000                push    ebp
.text:00401001                mov     ebp, esp
.text:00401003                push    ecx
```

```
.text:00401004             ]   push    24              ; size_t
.text:00401006                 call    _malloc
.text:0040100B                 add     esp, 4
.text:0040100E                 mov     [ebp+heap_struct], eax
.text:00401011                 mov     eax, [ebp+heap_struct]
.text:00401014             X   mov     dword ptr [eax], 10
.text:0040101A                 mov     ecx, [ebp+heap_struct]
.text:0040101D             Y   mov     word ptr [ecx+4], 20
.text:00401023                 mov     edx, [ebp+heap_struct]
.text:00401026             Z   mov     byte ptr [edx+6], 30
.text:0040102A                 mov     eax, [ebp+heap_struct]
.text:0040102D             [   mov     dword ptr [eax+8], 40
.text:00401034                 mov     ecx, [ebp+heap_struct]
.text:00401037                 fld     ds:dbl_40B128
.text:0040103D             \\   fstp    qword ptr [ecx+10h]
.text:00401040                 xor     eax, eax
.text:00401042                 mov     esp, ebp
.text:00401044                 pop     ebp
.text:00401045                 retn
.text:00401045 _main           endp
```

В отличие от двух предыдущих случаев, здесь можно установить точный размер и схему структуры. По аргументу `malloc` в строке `]` видно, что она занимает 24 байта. Поля расположены так:

• четырёхбайтовое поле по смещению 0, строка X;
• двухбайтовое поле по смещению 4, строка Y;
• однобайтовое поле по смещению 6, строка Z;
• четырёхбайтовое поле по смещению 8, строка `[`;
• восьмибайтовое поле по смещению 16, или `10h`, строка `\\`.

По инструкциям сопроцессора можно дополнительно заключить, что последнее поле имеет тип `double`. Если ту же программу собрать с однобайтовым выравниванием полей, листинг изменится следующим образом:

```
.text:00401000 _main           proc near
.text:00401000
.text:00401000 heap_struct     = dword ptr -4
.text:00401000
.text:00401000                 push    ebp
.text:00401001                 mov     ebp, esp
.text:00401003                 push    ecx
.text:00401004                 push    19              ; size_t
.text:00401006                 call    _malloc
.text:0040100B                 add     esp, 4
.text:0040100E                 mov     [ebp+heap_struct], eax
.text:00401011                 mov     eax, [ebp+heap_struct]
.text:00401014                 mov     dword ptr [eax], 10
.text:0040101A                 mov     ecx, [ebp+heap_struct]
.text:0040101D                 mov     word ptr [ecx+4], 20
.text:00401023                 mov     edx, [ebp+heap_struct]
.text:00401026                 mov     byte ptr [edx+6], 30
.text:0040102A                 mov     eax, [ebp+heap_struct]
.text:0040102D                 mov     dword ptr [eax+7], 40
.text:00401034                 mov     ecx, [ebp+heap_struct]
.text:00401037                 fld     ds:dbl_40B128
.text:0040103D                 fstp    qword ptr [ecx+0Bh]
.text:00401040                 xor     eax, eax
.text:00401042                 mov     esp, ebp
.text:00401044                 pop     ebp
.text:00401045                 retn
.text:00401045 _main           endp
```

Изменились только общий размер, теперь 19 байт, и смещения полей, из которых исчезло заполнение.

Независимо от выравнивания, поиск создаваемых и обрабатываемых в куче структур обычно быстрее всего позволяет восстановить их схему. Однако одна функция редко обращается сразу ко

всем полям. Приходится прослеживать указатель между функциями и отмечать смещения при каждом разыменовании, постепенно собирая полное определение.

Массивы структур

Составные типы можно вкладывать друг в друга: создавать массивы структур, структуры внутри структур и поля-массивы. Рассмотрим массив из пяти элементов `ch8_struct`:

```
int main() {
    int idx = 1;
    struct ch8_struct *heap_struct;
    heap_struct = (struct ch8_struct*)malloc(sizeof(struct ch8_struct) * 5);
X   heap_struct[idx].field1 = 10;
}
```

Для обращения к `field1` в строке X индекс умножается на размер элемента массива, то есть на размер структуры, после чего прибавляется смещение поля. Соответствующий листинг:

```
.text:00401000 _main           proc near
.text:00401000
.text:00401000 idx             = dword ptr -8
.text:00401000 heap_struct     = dword ptr -4
.text:00401000
.text:00401000                 push    ebp
.text:00401001                 mov     ebp, esp
.text:00401003                 sub     esp, 8
.text:00401006                 mov     [ebp+idx], 1
.text:0040100D             Y   push    120             ; size_t
.text:0040100F                 call    _malloc
.text:00401014                 add     esp, 4
.text:00401017                 mov     [ebp+heap_struct], eax
.text:0040101A                 mov     eax, [ebp+idx]
.text:0040101D             Z   imul    eax, 24
.text:00401020                 mov     ecx, [ebp+heap_struct]
.text:00401023             [   mov     dword ptr [ecx+eax], 10
.text:0040102A                 xor     eax, eax
.text:0040102C                 mov     esp, ebp
.text:0040102E                 pop     ebp
.text:0040102F                 retn
.text:0040102F _main           endp
```

В строке Y из кучи запрашивается 120 байт. В строке Z индекс умножается на 24, а в строке [результат прибавляется к начальному адресу массива. Дополнительного смещения поля нет. Следовательно, размер элемента равен 24 байтам, число элементов равно `120 / 24 = 5`, а по смещению 0 находится четырёхбайтовое поле. Короткий листинг ничего не сообщает о распределении оставшихся 20 байт между другими полями.

Создание структур в IDA

Средства объединения элементов массива сворачивают длинные последовательности объявлений данных в одну строку. Средства работы со структурами решают похожую задачу для кода: вместо `[edx+10h]` можно получить более понятное выражение `[edx+ch8_struct.field5]`.

Обнаружив работу программы со структурой, нужно решить, стоит ли вводить имена её полей или достаточно числовых смещений. Иногда IDA распознаёт структуру из стандартной библиотеки Си или интерфейса программирования Windows, уже знает её точную схему и сама заменяет числа символическими именами. Если тип нестандартный либо его применение не удалось распознать, определение приходится создавать вручную.

Создание новой структуры или объединения

Структуры создаются в окне `Structures`. Определение нельзя применить в листинге, пока оно не появится в этом окне. Типы, известные IDA и распознанные в программе, добавляются туда автоматически.

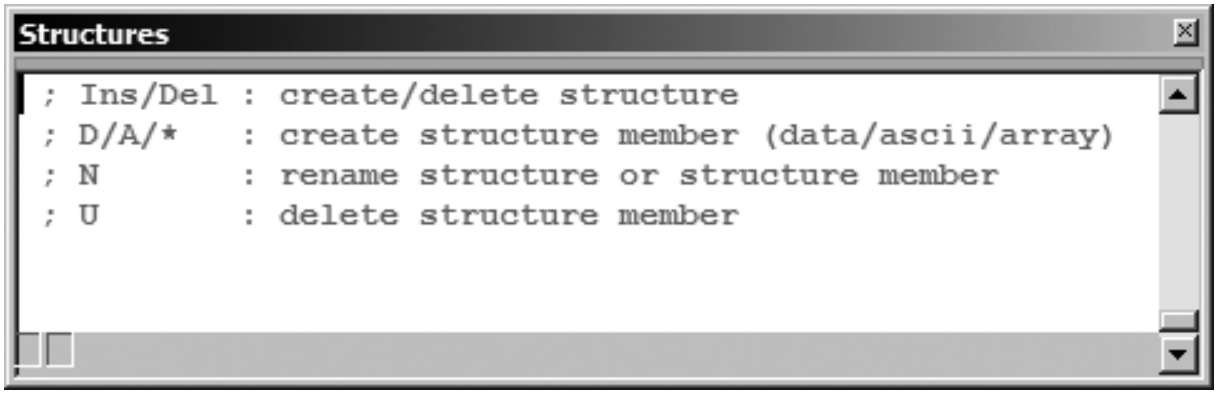


Рисунок 8-2. Окно структур

Автоматический анализ может пропустить структуру по двум причинам. Во-первых, IDA может знать её схему, но не располагать достаточными доказательствами применения типа в программе. Во-вторых, структура может быть нестандартной и совершенно неизвестной IDA. В обоих случаях работа начинается в окне структур.

Первые четыре строки окна напоминают о доступных действиях. Для добавления определения нажмите `Insert`: откроется диалог `Create Structure/Union`.

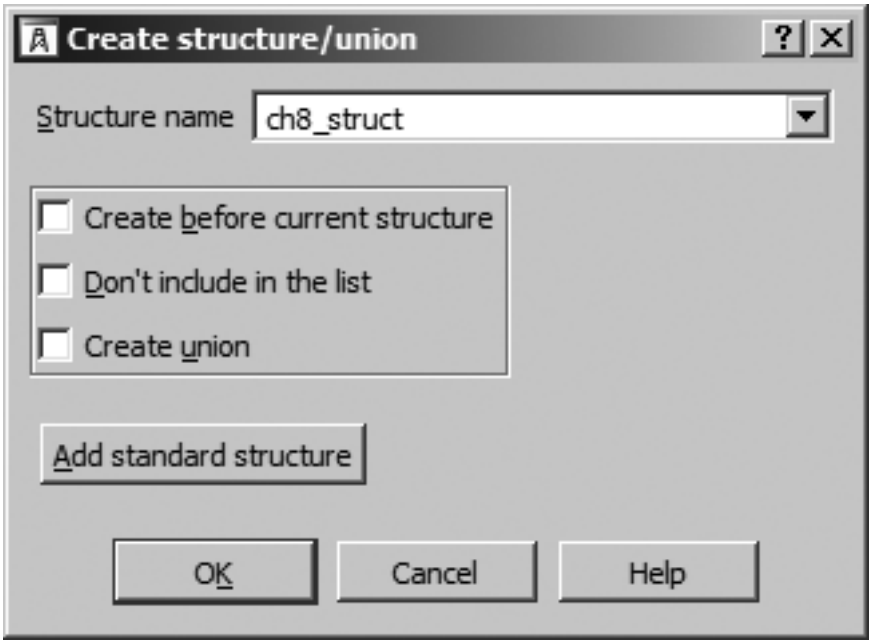


Рисунок 8-3. Диалог создания структуры или объединения

Укажите имя в поле `Structure name`. Первые два флажка определяют положение нового типа в окне и необходимость показывать его в списке. Флажок `Create union` переключает создание структуры на объединение в стиле Си.[3] У структуры размер складывается из размеров полей, а у объединения равен размеру крупнейшего поля. Кнопка `Add standard structure` открывает список известных IDA структурных типов; к нему мы вернёмся далее.

После ввода имени и нажатия OK в окне появляется пустое определение:

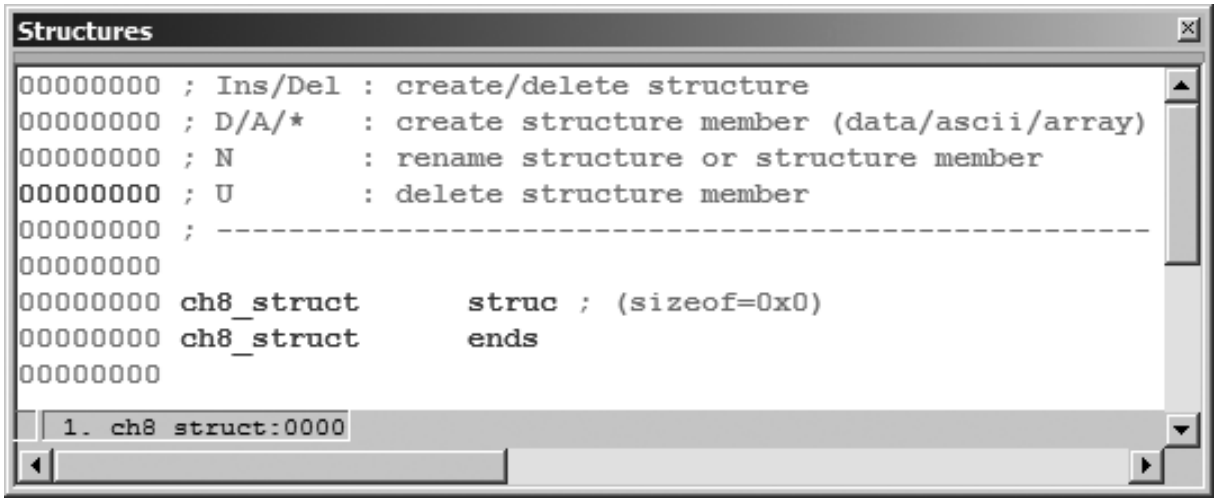


Рисунок 8-4. Пустое определение структуры

Теперь его нужно дополнить полями.

Редактирование полей

Для создания полей служат команды `D`, `A` и клавиша `*` цифровой клавиатуры. Сначала полезна только `D`, причём её действие зависит от положения курсора. Рекомендуется такой порядок:

1. Установите курсор на последнюю строку определения, содержащую `ends`, и нажмите `D`. В конец будет добавлено поле типа, первым выбранного в карусели данных. Оно получит имя `field_N`, где N - числовое смещение от начала структуры, например `field_0`.

2. Чтобы изменить размер, поставьте курсор на имя нового поля и нажимайте `D`, перебирая типы карусели. Размер, отсутствующий в карусели, можно включить командой `Options > Setup Data Types`. Для поля-массива откройте контекстное меню, выберите `Array` и задайте параметры массива.

3. Чтобы переименовать поле, поставьте курсор на его имя и нажмите `N` либо выберите в контекстном меню `Rename`, затем введите новое имя.

При ручном построении структуры учитывайте следующее:

• Слева в окне показывается восьмизначное шестнадцатеричное смещение поля.
• После добавления или удаления поля, а также изменения его размера IDA обновляет размер структуры в первой строке определения.
• К полям можно добавлять комментарии так же, как к строкам дизассемблированного кода.
• Несмотря на подсказку вверху окна, клавиша `U` удаляет поле только тогда, когда оно последнее. В остальных случаях поле лишь становится неопределённым: имя исчезает, но занимаемые байты остаются.
• За выравнивание отвечает пользователь. IDA не различает уплотнённые и неуплотнённые структуры. Необходимые байты заполнения нужно добавлять фиктивными полями подходящего размера; после создания следующих полей их при желании можно сделать неопределёнными.
• Чтобы удалить байты из середины структуры, сначала сделайте связанное поле неопределённым, затем выберите `Edit > Shrink Struct Type`.
• Чтобы вставить байты в середину, выберите поле, которое должно идти после них, затем выполните `Edit > Expand Struct Type` и задайте число вставляемых байтов.

• Если известен только размер структуры, создайте два поля. Первое сделайте массивом из размер - 1 байтов, второе - однобайтовым. После создания второго поля сделайте первое неопределённым. Общий размер сохранится, а по мере изучения схемы освободившиеся байты можно будет превращать в настоящие поля.

Повторяя добавление поля, выбор его размера, вставку заполнения и переименование, можно вручную получить неуплотнённую `ch8_struct`:

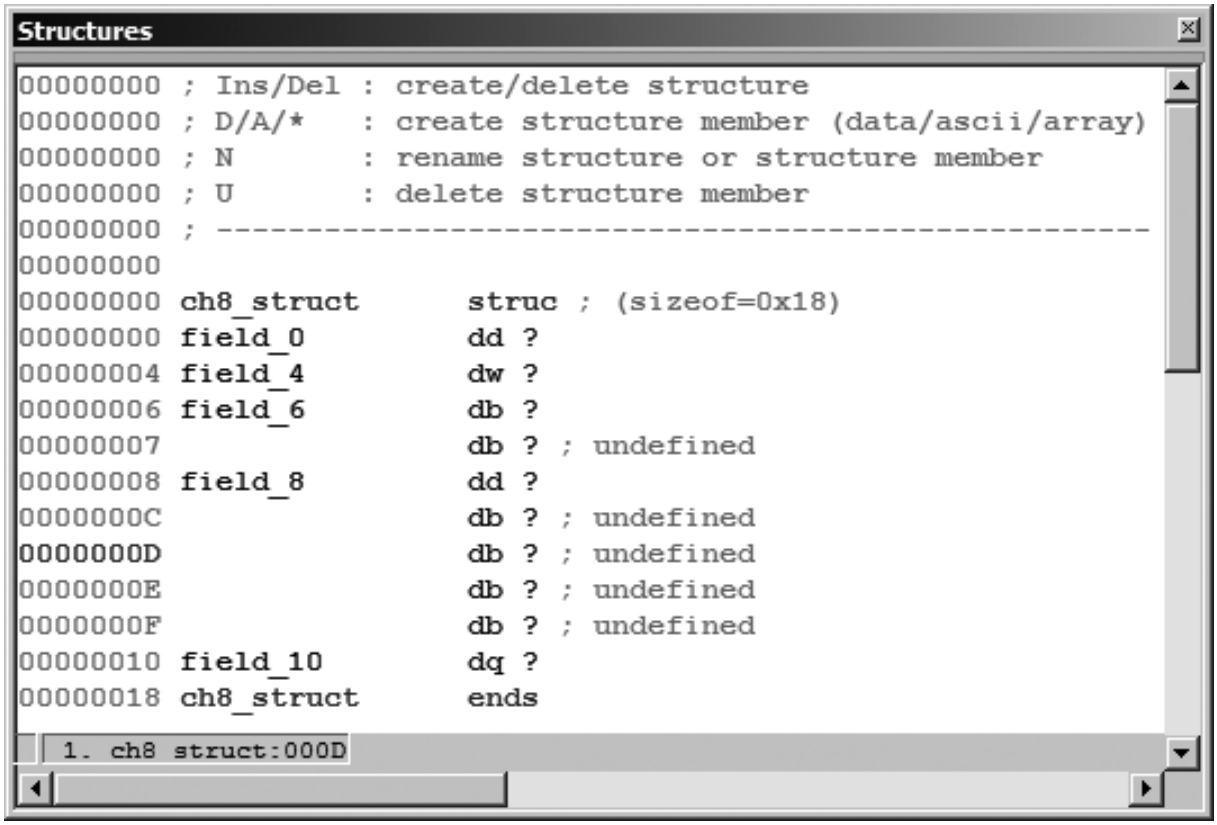


Рисунок 8-5. Определение `ch8_struct`*, созданное вручную*

В примере добавлены байты заполнения для правильного выравнивания, а поля получили имена из предыдущих листингов. Их смещения и общий размер структуры, 24 байта, совпадают с уже установленными значениями.

Готовое определение можно свернуть в одну строку. Выберите любое поле и нажмите - на цифровой клавиатуре:

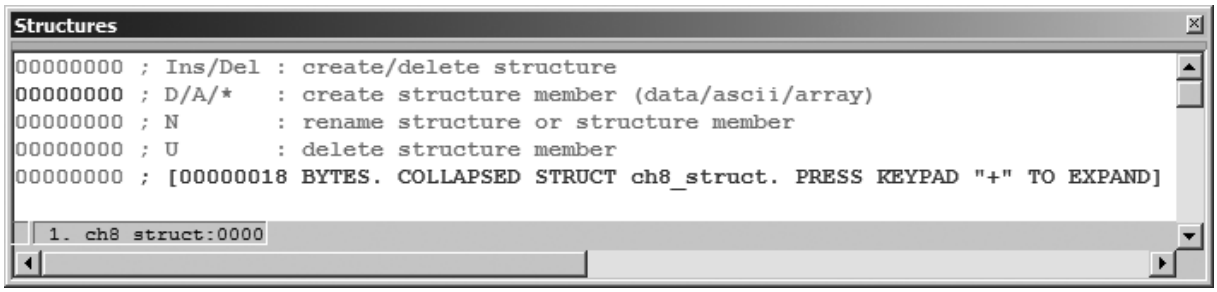


Рисунок 8-6. Свёрнутое определение структуры

Большинство встроенных определений IDA изначально показываются в таком виде, поскольку обычно не требуют редактирования. Для разворачивания нажмите + на цифровой клавиатуре или дважды щёлкните имя структуры.

Кадры стека как особые структуры

Подробное представление кадра стека похоже на определение структуры не случайно: внутри IDA обрабатывает их одинаково. Оба объекта представляют непрерывный блок байтов, разделённый на именованные поля с числовыми смещениями. Небольшое различие состоит в системе отсчёта. В кадре стека применяются положительные и отрицательные смещения относительно указателя кадра или адреса возврата, а в обычной структуре - только положительные смещения от её начала.

Применение шаблонов структур

Определения структур применяются в листинге двумя способами. Во-первых, ими форматируют обращения к памяти: числовое `[ebx+8]` превращается в символическое `[ebx+ch8_struct.field4]`. Иерархическая запись сразу сообщает тип структуры и имя выбранного поля. Этот способ особенно полезен при работе через указатель. Во-вторых, структурные типы можно назначать стековым и глобальным переменным.

Чтобы понять применение шаблона к операнду, удобно представить определение структуры как набор именованных констант смещений. Начало определения `ch8_struct` тогда выглядело бы так:

```
enum {
    ch8_struct.field1 = 0,
    ch8_struct.field2 = 4,
    ch8_struct.field3 = 6,
    ch8_struct.field4 = 8,
    ch8_struct.field5 = 16
};
```

Имея такой набор, IDA позволяет заменить любую постоянную в операнде эквивалентным символическим выражением. Например, обращение `[ecx+10h]` может означать доступ к `field5` структуры `ch8_struct`.

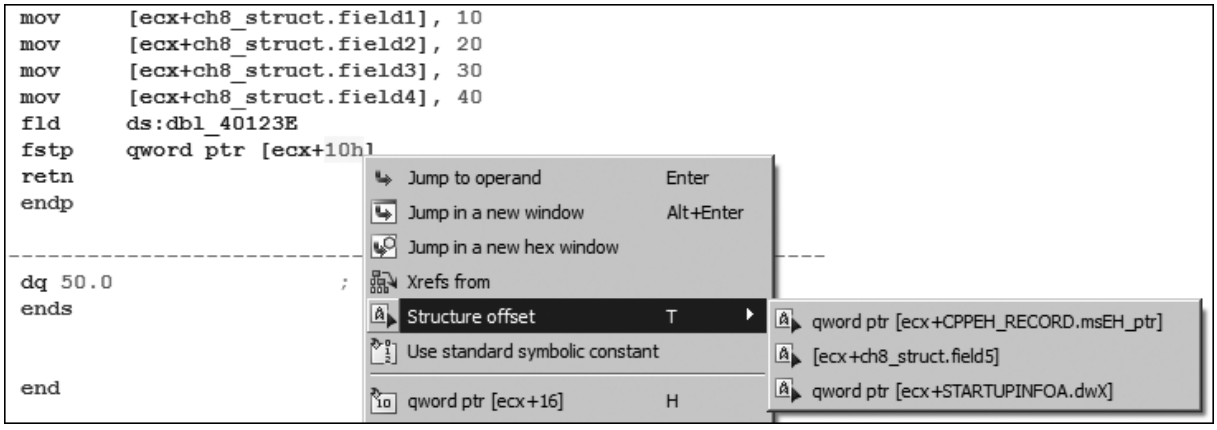


Рисунок 8-7. Применение смещения структуры

Пункт `Structure offset` в контекстном меню значения `10h` предлагает три способа представить операнд. Варианты берутся из структур, у которых имеется поле со смещением 16.

Вместо форматирования отдельных обращений можно назначить структурный тип целой стековой или глобальной переменной. Для стековой переменной дважды щёлкните её имя, чтобы открыть подробное представление кадра, затем выполните `Edit > Struct Var` или нажмите `Alt+Q` и выберите известную структуру.

При выборе IDA объединяет нужное число байтов стека в одну переменную соответствующего типа и преобразует все связанные обращения. Вернёмся к примеру со стековой структурой:

```
.text:00401006                mov     [ebp+var_18], 10
```

```
.text:0040100D                mov     [ebp+var_14], 20
.text:00401013                mov     [ebp+var_12], 30
.text:00401017                mov     [ebp+var_10], 40
.text:0040101E                fld     ds:dbl_40B128
.text:00401024                fstp    [ebp+var_8]
```

Мы установили, что `var_18` является первым полем 24-байтовой структуры. До преобразования кадр выглядит так:

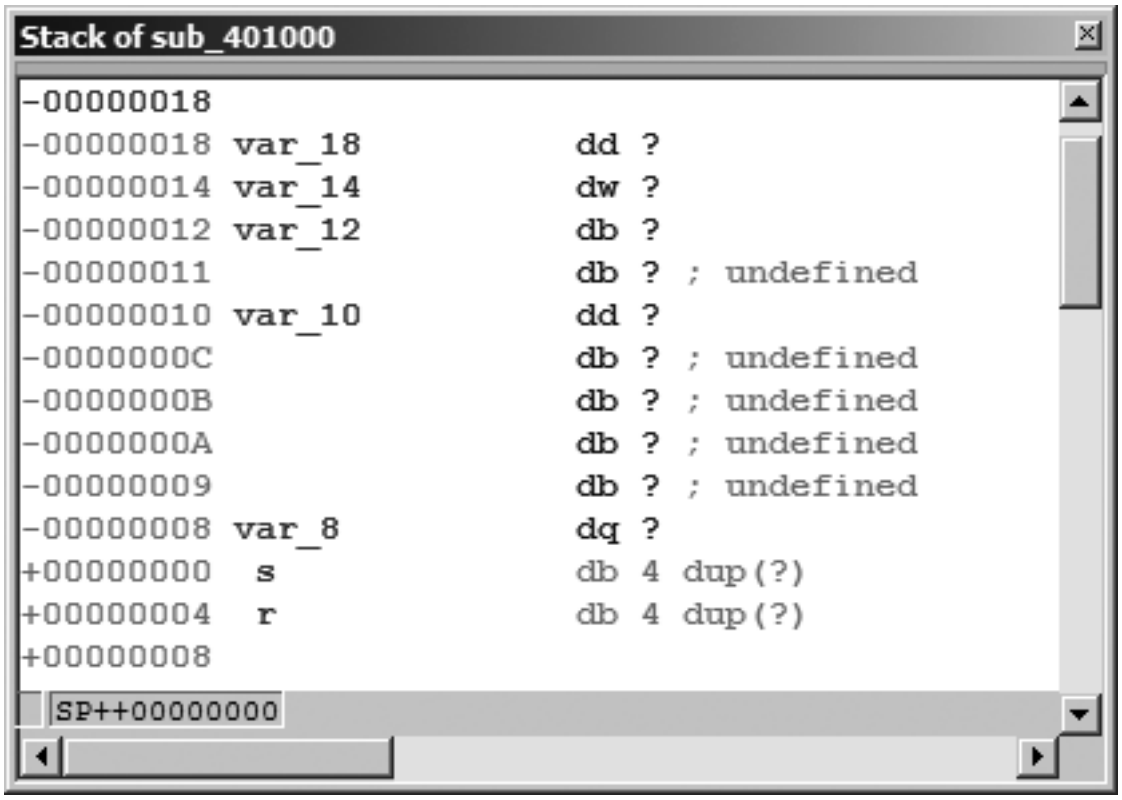


Рисунок 8-9. Структура в стеке до форматирования

Если выбрать `var_18` и назначить ей тип `ch8_struct`, IDA свернёт начинающиеся с неё 24 байта в одну переменную:

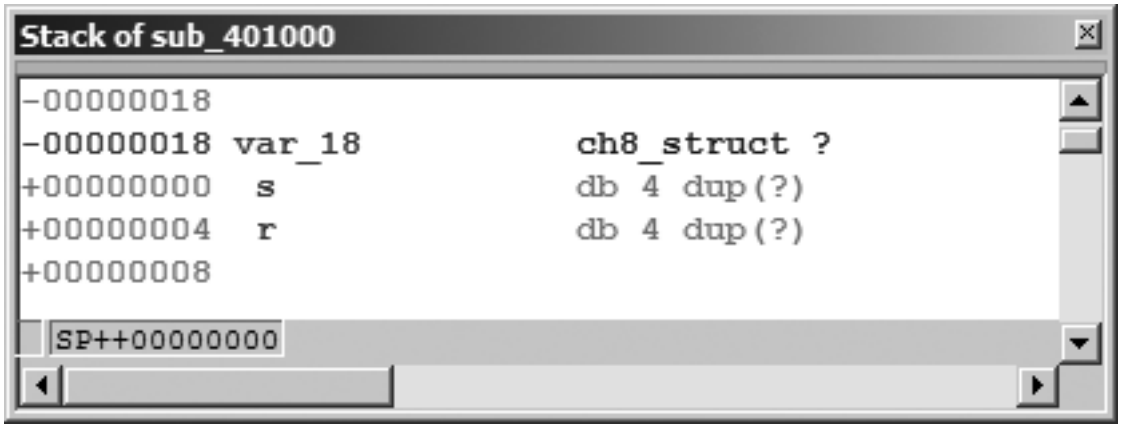


Рисунок 8-10. Структура в стеке после форматирования

IDA предупредит, что при преобразовании некоторые прежние переменные будут уничтожены. Это ожидаемо: они ошибочно представляли отдельные поля как самостоятельные объекты.

Теперь любое обращение внутрь 24-байтового блока считается обращением к полю. IDA старается сопоставить смещение с определением и автоматически преобразует листинг:

```
.text:00401006                mov     [ebp+var_18.field1], 10
```

```
.text:0040100D                mov     [ebp+var_18.field2], 20
.text:00401013                mov     [ebp+var_18.field3], 30
.text:00401017                mov     [ebp+var_18.field4], 40
.text:0040101E                fld     ds:dbl_40B128
.text:00401024                fstp    [ebp+var_18.field5]
```

Имена полей заметно улучшают читаемость и точнее отражают работу исходной программы с данными.

Глобальная переменная форматируется почти так же: выберите переменную или адрес начала структуры, затем выполните `Edit > Struct Var` или нажмите `Alt+Q`. Для неопределённых глобальных данных, но не для стека, можно также выбрать структуру в контекстном меню.

Импорт новых структур

Ручное создание и редактирование быстро утомляет. IDA умеет разбирать отдельные объявления данных на Си, но не на C++, и целые заголовочные файлы Си, автоматически создавая внутренние определения структур. Если доступны исходные тексты или хотя бы заголовки исследуемой программы, это экономит много времени.

Разбор объявлений структур Си

Окно локальных типов открывается командой `View > Open Subviews > Local Types`. Оно содержит типы, разобранные в текущей базе. В новой базе список пуст. Чтобы добавить объявление, нажмите `Insert` или выберите одноимённый пункт контекстного меню.

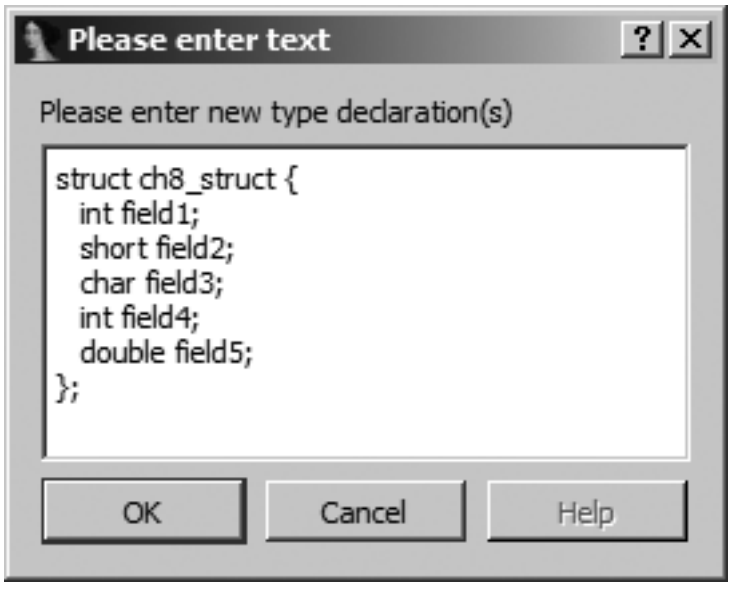


Рисунок 8-11. Диалог ввода локального типа

Ошибки разбора выводятся в окне сообщений IDA. Успешно разобранный тип вместе с объявлением появляется в списке локальных типов:

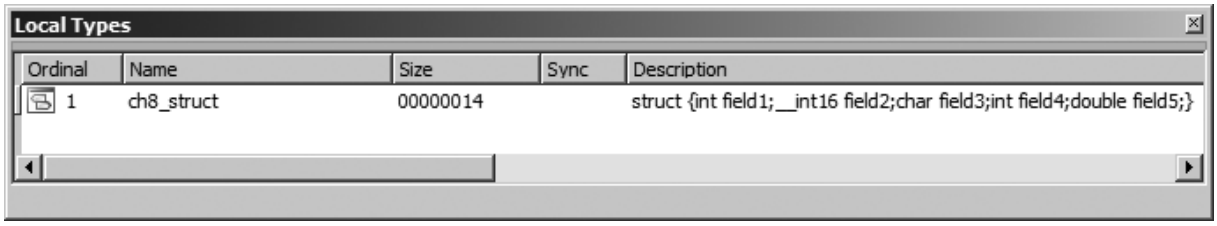


Рисунок 8-12. Окно локальных типов

По умолчанию синтаксический анализатор IDA выравнивает поля по четырёхбайтовой границе. Для другого выравнивания добавьте директиву `#pragma pack`: IDA распознает её и применит указанное значение.

Локальный тип не сразу появляется в окне структур. Проще всего щёлкнуть его правой кнопкой мыши и выбрать `Synchronize to idb`. Другой способ состоит в импорте из списка стандартных структур, куда добавляется каждый новый локальный тип.

Разбор заголовочных файлов Си

Выберите `File > Load File > Parse C Header File` и укажите заголовок. При успехе IDA сообщает `Compilation successful`; ошибки перечисляются в окне сообщений.

Все успешно разобранные структуры добавляются в списки локальных и стандартных типов текущей базы. Если имя нового типа совпадает с существующим, прежнее определение заменяется новым. В окне структур импортированные типы не появляются, пока пользователь явно не синхронизирует или не добавит их.

При разборе заголовков важно помнить:

• Встроенный анализатор может применять иное стандартное выравнивание, чем компилятор исследуемой программы. По умолчанию это 4 байта, но директива `#pragma pack` учитывается.
• Поддерживается директива препроцессора `#include`. Вложенные файлы ищутся рядом с разбираемым заголовком и в каталогах, перечисленных в поле `Include directories` диалога `Options > Compiler`.
• Анализатор понимает только стандартные типы Си, но также обрабатывает `#define` и `typedef`. Поэтому, например, `uint32_t` будет разобран правильно, если ранее встретилось подходящее объявление `typedef`.
• Даже без исходного кода часто проще набросать схему структуры на Си в текстовом редакторе и разобрать полученный заголовок либо вставить объявление как локальный тип, чем пользоваться громоздкими средствами ручного редактирования.
• Новые определения доступны только в текущей базе. Для других баз их придётся создать снова; далее описано, как упростить перенос с помощью библиотек типов TIL.

Чтобы увеличить вероятность успешного разбора, упрощайте объявления: применяйте стандартные типы Си и по возможности избегайте вложенных заголовков. Важнее всего получить правильную схему памяти. Она зависит прежде всего от размера каждого поля и выравнивания, а не от безупречно точного имени типа. Если ради успешного разбора нужно заменить `uint32_t` на равный ему по размеру `int`, это разумная уступка.

Использование стандартных структур

IDA знает множество структур библиотек и программных интерфейсов. При создании базы она пытается определить компилятор и платформу, затем загружает шаблоны из соответствующих заголовков. Когда в листинге обнаруживается работа с известным типом, его определение добавляется в окно структур. Таким образом, это окно содержит не все известные IDA структуры, а только относящееся к текущей программе подмножество.

Дополнительный стандартный тип можно добавить вручную. В окне структур нажмите `Insert`, затем в диалоге создания выберите `Add standard structure`. Откроется общий список для обнаруженного компилятора и формата файла. В него также входят структуры, добавленные после разбора заголовков.

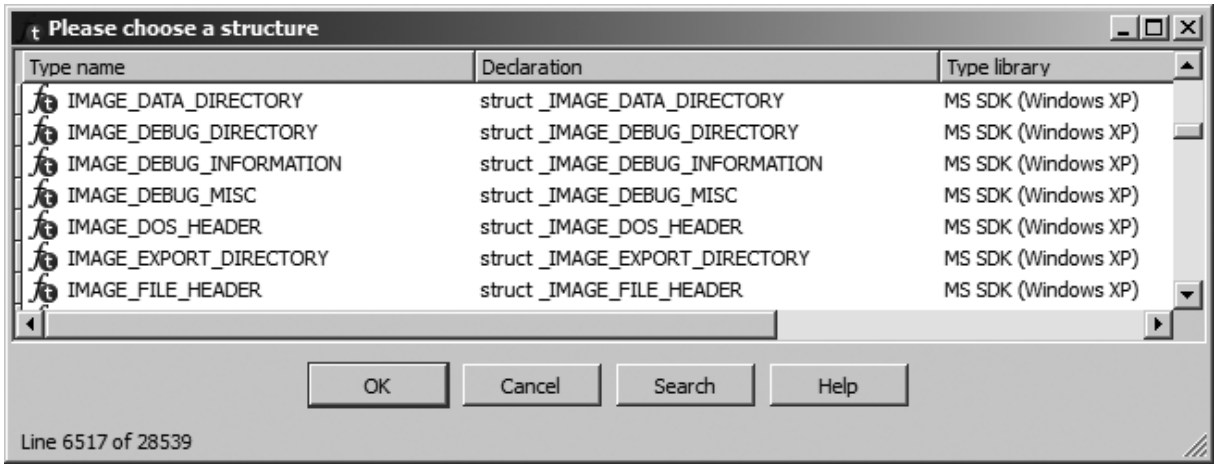


Рисунок 8-13. Выбор стандартной структуры

Поиск находит частичное совпадение текста. Поддерживается и быстрый переход по префиксу: начните вводить первые символы имени, они появятся в строке состояния, а список перейдёт к первому совпадению. Вместе с выбранной структурой в окно добавляются все вложенные в неё типы.

Пример: заголовки PE

Рассмотрим заголовки исполняемого файла Windows в формате PE. По умолчанию они не загружаются в базу, но при первоначальном создании можно включить `Manual load`. Это гарантирует лишь наличие байтов заголовков, но обычно не придаёт им структуры: программа редко обращается к собственным заголовкам, поэтому автоматическому анализатору незачем применять к ним шаблоны.

Файл PE начинается с заголовка `IMAGE_DOS_HEADER`. Одно из его полей указывает на `IMAGE_NT_HEADERS`, описывающий размещение образа в памяти. Сразу после ручной загрузки начало файла может выглядеть так:

```
HEADER:00400000 __ImageBase     db  4Dh ; M
HEADER:00400001                 db  5Ah ; Z
HEADER:00400002                 db  90h
HEADER:00400003                 db    0
HEADER:00400004                 db    3
HEADER:00400005                 db    0
HEADER:00400006                 db    0
HEADER:00400007                 db    0
HEADER:00400008                 db    4
HEADER:00400009                 db    0
HEADER:0040000A                 db    0
HEADER:0040000B                 db    0
HEADER:0040000C                 db 0FFh
HEADER:0040000D                 db 0FFh
```

```
HEADER:0040000E                 db    0
HEADER:0040000F                 db    0
```

Без справочника по PE смысл отдельных байтов почти неясен. Сначала добавьте стандартные определения `IMAGE_DOS_HEADER` и, заодно, `IMAGE_NT_HEADERS`. Затем выберите `__ImageBase` и командой `Edit > Struct Var` или сочетанием `Alt+Q` назначьте тип `IMAGE_DOS_HEADER`. Первые 64 байта свернутся в одну запись:

```
HEADER:00400000 __ImageBase IMAGE_DOS_HEADER <5A4Dh, 90h, 3, 0, 4, 0, 0FFFFh, 0, 0B8h, \
HEADER:00400000                               0, 0, 0, 40h, 0, 0, 0, 0, 0, 80h>
HEADER:00400040 db 0Eh
```

Тип теперь виден, но значения всё ещё трудно истолковать. Нажмите + на цифровой клавиатуре, чтобы развернуть объект и увидеть имена полей:

```
HEADER:00400000 __ImageBase     dw 5A4Dh                ; e_magic
HEADER:00400000                 dw 90h                  ; e_cblp
HEADER:00400000                 dw 3                    ; e_cp
HEADER:00400000                 dw 0                    ; e_crlc
HEADER:00400000                 dw 4                    ; e_cparhdr
HEADER:00400000                 dw 0                    ; e_minalloc
HEADER:00400000                 dw 0FFFFh               ; e_maxalloc
HEADER:00400000                 dw 0                    ; e_ss
HEADER:00400000                 dw 0B8h                 ; e_sp
HEADER:00400000                 dw 0                    ; e_csum
HEADER:00400000                 dw 0                    ; e_ip
HEADER:00400000                 dw 0                    ; e_cs
HEADER:00400000                 dw 40h                  ; e_lfarlc
HEADER:00400000                 dw 0                    ; e_ovno
HEADER:00400000                 dw 4 dup(0)             ; e_res
HEADER:00400000                 dw 0                    ; e_oemid
HEADER:00400000                 dw 0                    ; e_oeminfo
HEADER:00400000                 dw 0Ah dup(0)           ; e_res2
HEADER:00400000             X   dd 80h                  ; e_lfanew
HEADER:00400040                 db 0Eh
```

Названия полей здесь не слишком выразительны, поэтому справочник всё равно пригодится. Он сообщает, что `e_lfanew` в строке X содержит файловое смещение `IMAGE_NT_HEADERS`. Повторив те же действия по адресу `00400080`, получим начало второй структуры:

```
HEADER:00400080                 dd 4550h                ; Signature
HEADER:00400080                 dw 14Ch                 ; FileHeader.Machine
HEADER:00400080             X   dw 5                    ; FileHeader.NumberOfSections
HEADER:00400080                 dd 4789ADF1h            ; FileHeader.TimeDateStamp
HEADER:00400080                 dd 1400h                 ; FileHeader.PointerToSymbolTable
HEADER:00400080                 dd 14Eh                 ; FileHeader.NumberOfSymbols
HEADER:00400080                 dw 0E0h                 ; FileHeader.SizeOfOptionalHeader
HEADER:00400080                 dw 307h                 ; FileHeader.Characteristics
HEADER:00400080                 dw 10Bh                 ; OptionalHeader.Magic
HEADER:00400080                 db 2                    ; OptionalHeader.MajorLinkerVersion
HEADER:00400080                 db 38h                  ; OptionalHeader.MinorLinkerVersion
HEADER:00400080                 dd 800h                 ; OptionalHeader.SizeOfCode
HEADER:00400080                 dd 800h                 ; OptionalHeader.SizeOfInitializedData
HEADER:00400080                 dd 200h                 ; OptionalHeader.SizeOfUninitializedData
HEADER:00400080                 dd 1000h                ; OptionalHeader.AddressOfEntryPoint
HEADER:00400080                 dd 1000h                ; OptionalHeader.BaseOfCode
HEADER:00400080                 dd 2000h                ; OptionalHeader.BaseOfData
HEADER:00400080             Y   dd 400000h              ; OptionalHeader.ImageBase
```

Эти имена уже полезнее: в файле пять разделов, строка X, а предпочтительный виртуальный адрес загрузки равен `00400000`, строка Y. Для обратного сворачивания нажмите - на цифровой клавиатуре.

Файлы TIL

IDA хранит сведения о типах данных и объявлениях функций в файлах TIL. В каталоге `<IDADIR>/til` поставляются библиотеки для распространённых компиляторов и программных интерфейсов. Окно `Types`, открываемое через `View > Open Subview > Type Libraries`, показывает загруженные библиотеки и позволяет подключать новые. Обычно подходящие файлы выбираются автоматически по свойствам двоичного файла, и пользователю не приходится работать с ними напрямую.

Загрузка новых библиотек типов

Иногда IDA не распознаёт компилятор, например из-за обфускации. Тогда откройте окно типов, нажмите `Insert` и выберите нужный файл `.til`. Все содержащиеся в нём структуры будут добавлены в список стандартных, а к функциям с совпадающими объявлениями применятся новые сведения. Иными словами, получив дополнительное знание о функции, IDA сразу использует его в базе.

Перенос библиотек типов

Созданные вручную или разобранные из заголовков структуры сохраняются в отдельной библиотеке типов текущей базы. Её основное имя совпадает с именем базы: например, базе `some_file.idb` соответствует `some_file.til`. Обычно этот файл виден только пока база открыта. Файл `.idb` представляет собой архив компонентов базы, похожий на архив TAR; при открытии рабочие файлы, включая `.til`, извлекаются.

Перенести пользовательские типы можно двумя способами.[4] Первый, неофициальный, состоит в копировании `.til` из открытой базы в каталог библиотек IDA, после чего его можно загрузить в другой базе через окно типов.

Более официальный способ - создать сценарий IDC, который заново построит структуры. Используйте `File > Produce File > Dump Typeinfo to IDC File`. Однако сценарий включает только типы, присутствующие в окне структур, и может пропустить часть объявлений, разобранных из заголовков. Копирование `.til` сохраняет их все.

Hex-Rays также предоставляет отдельную программу `tilib` для создания библиотек вне IDA. Зарегистрированные пользователи получают её на странице загрузки IDA в архиве ZIP; установка сводится к распаковке в `<IDADIR>`. Программа умеет показывать содержимое существующей библиотеки и создавать новую из заголовков Си, но не C++. Например, следующая команда выводит содержимое библиотеки типов Visual Studio 6:

```
C:\Program Files\IdaPro>tilib -l til\pc\vc6win.til
```

Для создания библиотеки указываются разбираемый заголовок и выходной файл. Параметры командной строки позволяют добавить каталоги заголовков или уже разобранные библиотеки, необходимые для разрешения зависимостей. Следующая команда создаёт `ch8.til` с объявлением `ch8_struct`:

```
C:\Program Files\IdaPro>tilib -c -hch8_struct.h ch8.til
```

Чтобы IDA увидела результат, его нужно переместить в `<IDADIR>/til`. Дополнительные возможности описаны в файле `README` и в справке, которую `tilib` выводит без аргументов. До версии 6.1 программа поставлялась только для Windows, но созданные ею библиотеки совместимы со всеми вариантами IDA.

Основы обратной разработки программ на C++

Классы C++ являются объектно-ориентированным расширением структур Си, поэтому обсуждение структур естественно завершить обзором скомпилированного C++. Язык слишком сложен для подробного изложения в этой книге; далее рассматриваются лишь основные признаки и некоторые различия между Microsoft Visual C++ и GNU g++.

Твёрдое знание самого языка C++ значительно облегчает анализ. Наследование и полиморфизм непросто освоить даже на уровне исходного кода; попытка начать сразу с языка ассемблера почти неизбежно приведёт к затруднениям.

Указатель this

Указатель `this` доступен во всех нестатических функциях-членах C++. При вызове он указывает на объект, через который вызвана функция:

```
// object1, object2 и *p_obj имеют один тип.
object1.member_func();
object2.member_func();
p_obj->member_func();
```

В трёх вызовах `member_func` указатель `this` принимает соответственно значения `&object1`, `&object2` и `p_obj`. Удобно считать его скрытым первым параметром каждой нестатической функции-члена. Microsoft Visual C++ применяет соглашение `thiscall` и передаёт `this` в регистре ECX. Компилятор GNU g++ обращается с ним как с обычным первым параметром функции.

При анализе программы, собранной Visual C++, загрузка адреса в ECX непосредственно перед вызовом обычно указывает сразу на два обстоятельства: вызывается функция-член, а исходный файл, вероятно, был собран Visual C++. Если один адрес передаётся нескольким функциям, можно предположить, что они относятся к одной иерархии классов.

Если функция использует ECX до его инициализации, значение, вероятно, подготовил вызывающий код. Это ещё один признак функции-члена, хотя таким же образом может проявляться соглашение `fastcall`. Если функция-член передаёт свой `this` другим функциям, они, вероятно, принадлежат тому же классу.

В коде GNU g++ такие вызовы выделяются слабее. Тем не менее функцию, чей первый параметр заведомо не является указателем, можно исключить из числа нестатических функций-членов.

Виртуальные функции и таблицы виртуальных функций

Виртуальные функции обеспечивают полиморфизм C++. Для каждого класса с виртуальными функциями, включая производные классы, компилятор создаёт таблицу указателей на эти функции, обычно называемую виртуальной таблицей. В каждый объект такого класса добавляется скрытое поле с указателем на соответствующую таблицу; как правило, это первое поле объекта.

При создании объекта указатель настраивается на таблицу его фактического класса. Вызов виртуальной функции разрешается поиском нужной записи в этой таблице. Именно так выбирается реализация во время выполнения.

Рассмотрим два класса:

```
class BaseClass {
public:
    BaseClass();
    virtual void vfunc1() = 0;
```

```
    virtual void vfunc2();
    virtual void vfunc3();
    virtual void vfunc4();
private:
    int x;
    int y;
};

class SubClass : public BaseClass {
public:
    SubClass();
    virtual void vfunc1();
    virtual void vfunc3();
    virtual void vfunc5();
private:
    int z;
};
```

`SubClass` наследует `BaseClass`. У базового класса четыре виртуальные функции, у производного - пять: четыре унаследованных и новая `vfunc5`. Объявление `vfunc1 с = 0` делает её чисто виртуальной. В базовом классе реализации нет; пока производный класс не предоставит её, создавать объекты этого типа нельзя. `SubClass` такую реализацию содержит и поэтому является конкретным классом.

На первый взгляд у `BaseClass` два поля данных, а у `SubClass` три. Но к ним добавляется указатель на виртуальную таблицу. Поэтому объект `BaseClass` фактически содержит три поля, а `SubClass` - четыре. В обоих случаях скрытый указатель идёт первым; производный класс наследует его от базового.

Упрощённую схему объекта и таблиц можно представить так:

```
объект SubClass в куче              данные только для чтения
+----------------------+            +--------------------------+
| указатель на таблицу |----------->| &SubClass::vfunc1        |
| x                    |            | &BaseClass::vfunc2       |
| y                    |            | &SubClass::vfunc3        |
| z                    |            | &BaseClass::vfunc4       |
+----------------------+            | &SubClass::vfunc5        |
                                    +--------------------------+

таблица BaseClass
+--------------------------+
| &purecall                |
| &BaseClass::vfunc2       |
| &BaseClass::vfunc3       |
| &BaseClass::vfunc4       |
+--------------------------+
```

Рисунок 8-14. Упрощённое расположение виртуальных таблиц

В таблице `SubClass` сохранились указатели на `BaseClass::vfunc2` и `BaseClass::vfunc4`, поскольку производный класс их не переопределяет. Для чистой виртуальной `BaseClass::vfunc1` адреса реализации нет. Компилятор обычно помещает в её ячейку адрес обработчика `purecall`, который теоретически не должен вызываться, а при ошибочном вызове обычно завершает программу.

При описании класса в IDA скрытый указатель необходимо включать первым полем и учитывать в полном размере объекта. Особенно хорошо это видно по аргументу оператора `new`: запрашиваемый размер включает объявленные поля класса и его предков, а также указатель на виртуальную таблицу.[5]

Пусть объект `SubClass` создаётся в куче, его адрес сохраняется в указателе на `BaseClass`, а затем передаётся функции, которая вызывает `vfunc3`:

```
void call_vfunc(BaseClass *b) {
    b->vfunc3();
```

```
}

int main() {
    BaseClass *bc = new SubClass();
    call_vfunc(bc);
}
```

Поскольку `vfunc3` виртуальная, должен быть выбран вариант `SubClass::vfunc3`, соответствующий фактическому объекту. Это разрешается следующим кодом:

```
.text:004010A0 call_vfunc      proc near
.text:004010A0
.text:004010A0 b               = dword ptr 8
.text:004010A0
.text:004010A0                 push    ebp
.text:004010A1                 mov     ebp, esp
.text:004010A3                 mov     eax, [ebp+b]
.text:004010A6             X   mov     edx, [eax]
.text:004010A8                 mov     ecx, [ebp+b]
.text:004010AB             Y   mov     eax, [edx+8]
.text:004010AE             Z   call    eax
.text:004010B0                 pop     ebp
.text:004010B1                 retn
.text:004010B1 call_vfunc      endp
```

В строке X из первого поля объекта читается адрес таблицы `SubClass` и сохраняется в EDX. В строке Y из неё выбирается третий указатель, то есть адрес `SubClass::vfunc3`. Строка Z выполняет косвенный вызов.

Индексирование виртуальной таблицы ничем не отличается от обращения к структуре. Её можно описать в IDA отдельным типом:

```
00000000 SubClass_vtable struc ; (sizeof=0x14)
00000000 vfunc1          dd ?
00000004 vfunc2          dd ?
00000008 vfunc3          dd ?
0000000C vfunc4          dd ?
00000010 vfunc5          dd ?
00000014 SubClass_vtable ends
```

После применения типа ссылка становится гораздо понятнее:

```
.text:004010AB                 mov     eax, [edx+SubClass_vtable.vfunc3]
```

Жизненный цикл объекта

Понимание создания и уничтожения объектов помогает восстанавливать иерархии, вложенные объекты, конструкторы и деструкторы.[6]

Конструкторы глобальных и статических объектов вызываются при запуске, до входа в `main`. Конструктор стекового объекта вызывается, когда объект входит в область видимости. Часто это происходит сразу при входе в функцию, но для объявления внутри блока - только при фактическом выполнении этого блока.

Создание объекта в куче состоит из двух шагов: `new` выделяет память, затем конструктор инициализирует объект. Существенное различие состоит в том, что Microsoft Visual C++ перед вызовом конструктора проверяет результат `new` на нулевое значение, а GNU g++ действует иначе.

Конструктор выполняет действия в таком порядке:

1. Если есть базовый класс, вызывается его конструктор.
2. При наличии виртуальных функций указатель таблицы настраивается на таблицу текущего класса. Он может перезаписать значение, установленное базовым конструктором, что и требуется.

3. Вызываются конструкторы полей, которые сами являются объектами.
4. Выполняется тело конструктора, написанное программистом.

В исходном C++ конструктор не имеет возвращаемого типа. Однако код Microsoft Visual C++ фактически возвращает `this` в EAX. Это лишь внутренняя особенность реализации, недоступная программисту на C++.

Деструкторы вызываются приблизительно в обратных обстоятельствах. Глобальные и статические объекты уничтожаются после завершения `main`; стековые - при выходе из области видимости; кучевые - оператором `delete` непосредственно перед освобождением памяти.

Порядок действий деструктора в основном обратен порядку конструктора:

1. При наличии виртуальных функций указатель объекта возвращается к таблице текущего класса. Это нужно, если при создании производный класс заменил его своим значением.
2. Выполняется тело деструктора, написанное программистом.
3. Вызываются деструкторы полей-объектов.
4. Последним вызывается деструктор базового класса.

Цепочка вызовов конструкторов и деструкторов базовых классов раскрывает иерархию наследования. Кроме того, прямые ссылки на виртуальную таблицу класса обычно встречаются только в его конструкторах и деструкторах. Найдя таблицу, можно воспользоваться перекрёстными ссылками IDA и быстро обнаружить эти функции.

Декорирование имён

Декорирование имён позволяет компилятору различать перегруженные варианты функций.[7] К исходному имени добавляются символы, кодирующие возвращаемый тип, класс-владелец, типы и порядок параметров.

Это особенность реализации компилятора, а не часть стандарта C++, поэтому производители применяют несовместимые схемы. IDA понимает соглашения Microsoft Visual C++, GNU g++ и ещё нескольких компиляторов. По умолчанию она показывает восстановленное имя в комментарии рядом с декорированным.



Рисунок 8-15. Параметры отображения восстановленных имён

Диалог открывается командой `Options > Demangled Names`. Основные варианты позволяют показывать восстановленные имена в комментариях, заменять ими сами имена или полностью отключить восстановление.

При выводе в комментарии листинг выглядит так:

```
.text:00401050 ; protected: __thiscall SubClass::SubClass(void)
X .text:00401050 ??0SubClass@@IAE@XZ proc near
  ...
.text:004010DC Y call ??0SubClass@@IAE@XZ ; SubClass::SubClass(void)
```

При замене самого имени:

```
X .text:00401050 protected: __thiscall SubClass::SubClass(void) proc near
  ...
.text:004010DC Y call SubClass::SubClass(void)
```

Флажок `Assume GCC v3.x names` различает схемы GNU g++ 2.9.x и версий 3.x или новее. Обычно IDA определяет их автоматически. Кнопки `Setup short names` и `Setup long names` открывают подробные параметры формата, описанные во встроенной справке.

Декорированное имя несёт большую часть сигнатуры функции. Поэтому восстановление сразу раскрывает типы параметров и результата, тогда как для функции без такого имени приходится долго исследовать поток входных и выходных данных.

Идентификация типов во время выполнения

C++ предоставляет операторы `typeid` и `dynamic_cast`, которые определяют и проверяют тип объекта во время выполнения. Для этого компилятор встраивает в двоичный файл сведения, позволяющие установить фактический тип полиморфного объекта независимо от типа указателя.

Как и декорирование имён, механизм RTTI зависит от компилятора и не имеет единого внутреннего представления.

Ниже рассматривается лишь поиск RTTI и получение имени класса в Microsoft Visual C++ и GNU g++. Подробное восстановление наследования, особенно множественного, требует знания конкретной реализации компилятора.

Рассмотрим программу с полиморфизмом:

```
class abstract_class {
public:
    virtual int vfunc() = 0;
};

class concrete_class : public abstract_class {
public:
    concrete_class();
    int vfunc();
};

void print_type(abstract_class *p) {
    cout << typeid(*p).name() << endl;
}

int main() {
    abstract_class *sc = new concrete_class();
    print_type(sc);
}
```

`print_type` должна вывести фактический тип объекта по указателю `p`, то есть `concrete_class`. Для этого компиляторы размещают сведения о классе рядом с его виртуальной таблицей. Непосредственно перед таблицей находится указатель на структуру, по которой определяется имя владельца.

В GNU g++ он указывает на `type_info`, содержащую указатель на имя класса. В Microsoft Visual C++ он указывает на `RTTICompleteObjectLocator`, откуда можно перейти к `TypeDescriptor`; в последнем находится символьный массив с именем полиморфного класса.

RTTI требуется только программам, использующим `typeid` или `dynamic_cast`. Компиляторы позволяют отключать его генерацию, поэтому отсутствие этих сведений в двоичном файле вполне нормально.

Восстановление наследования

Некоторые реализации RTTI позволяют восстановить иерархию полностью, но требуют знания внутренних форматов компилятора. Если RTTI отсутствует, остаются другие признаки.

Самый простой способ - проследить цепочку конструкторов базовых классов при создании объекта. Главная помеха здесь - встраиваемые конструкторы: их машинный код подставляется на место вызова, поэтому отдельной инструкции `call` нет.[8]

Другой способ - сравнить виртуальные таблицы. Полезны следующие ориентиры:

• Если две таблицы содержат одинаковое число записей, соответствующие классы могут быть связаны наследованием.
• Если таблица класса X длиннее таблицы класса Y, X может быть производным от Y.
• Если таблицы X и Y содержат общие записи, возможны три отношения: X наследует Y, Y наследует X либо оба наследуют общий класс Z.
• Если таблицы X и Y имеют общие записи, но в X присутствует `purecall`, которого нет в соответствующей позиции Y, то Y является производным от X.

Этот перечень не исчерпывающий, но для `BaseClass` и `SubClass` применимы последние три признака. Последний из них особенно убедительно показывает, что `SubClass` наследует `BaseClass`.

Материалы по обратной разработке C++

Для дальнейшего изучения полезны следующие работы:

• статья Игоря Скочинского «Reversing Microsoft Visual C++ Part II: Classes, Methods and RTTI»: <http://www.openrce.org/articles/full_view/23>;
• работа Пола Винсента Сабанала и Марка Винсента Ясона «Reversing C++»: <http://www.blackhat.com/presentations/bh-dc-07/Sabanal_Yason/Paper/bh-dc-07-Sabanal_Yason-WP.pdf>.

Многие подробности относятся именно к Microsoft Visual C++, но основные приёмы применимы и к другим компиляторам C++.

Итоги

Сложные типы данных встречаются почти во всех нетривиальных программах. Для обратной разработки необходимо понимать, как код обращается к составным структурам, и уметь замечать признаки их внутреннего расположения.

IDA предоставляет для этого множество средств. Их уверенное применение помогает быстрее установить, какие данные обрабатываются, и сосредоточиться на том, как и зачем программа их использует.

В следующей главе завершается обзор основных возможностей IDA: рассматриваются перекрёстные ссылки и графы. После этого мы перейдём к более сложным средствам, отличающим IDA от других инструментов обратной разработки.

Сноски

[1] Наведение на любое имя показывает всплывающее окно с десятью строками листинга по целевому адресу. Для библиотечной функции туда часто входит её объявление.
[2] Библиотека MSDN: <http://msdn.microsoft.com/library/>.
[3] Объединение, как и структура, содержит именованные поля различных типов. Однако все поля объединения перекрываются в памяти, поэтому его размер равен размеру самого крупного поля.
[4] Обсуждение способов переноса: <http://www.hex-rays.com/forum/viewtopic.php?f=6&t=986>. Ссылка доступна только зарегистрированным пользователям.
[5] Оператор `new` выделяет память в C++ примерно так же, как `malloc` в Си, но является частью самого языка, тогда как `malloc` - функция стандартной библиотеки.
[6] Конструктор автоматически инициализирует создаваемый объект. Необязательный деструктор вызывается, когда объект покидает область видимости или уничтожается иным способом.
[7] Перегрузка позволяет нескольким функциям иметь одно имя, если каждый вариант отличается количеством параметров либо их последовательностью и типами, то есть имеет уникальную сигнатуру.
[8] Функция, объявленная `inline`, обрабатывается подобно макросу: её код подставляется вместо явного вызова. Исчезновение инструкции `call` скрывает сам факт вызова функции.

Глава 9. Перекрёстные ссылки и графы

При обратной разработке двоичных файлов часто возникают вопросы: «Откуда вызывается эта функция?» и «Какие функции обращаются к этим данным?» Чтобы ответить, необходимо собрать ссылки на различные элементы программы и от них.

Предположим, найдена функция со стековым буфером, переполнение которого позволяет воздействовать на выполнение программы. Если функция скрыта глубоко в сложном приложении, сначала нужно понять, каким путём до неё добраться. Сама уязвимость бесполезна, пока функцию нельзя заставить выполниться. Поэтому следует выяснить, кто её вызывает и какие данные передаются по цепочке. Рассуждение продолжается вверх по возможным цепочкам вызовов, пока не найдётся путь, на который можно повлиять.

Другой пример - двоичный файл с множеством строк, среди которых обнаружена подозрительная фраза вроде «Выполняется атака типа отказ в обслуживании!». Само наличие текста ещё не доказывает, что программа действительно проводит такую атаку. Нужно найти код, который обращается к строке, и проверить его назначение.

IDA отвечает на подобные вопросы с помощью развитой системы перекрёстных ссылок. Она показывает связи между кодом и данными несколькими способами, в том числе в виде графов. В этой главе рассматриваются виды ссылок, средства доступа к ним и правила толкования.

Перекрёстные ссылки

В интерфейсе IDA перекрёстные ссылки часто сокращённо называются `xref`. Далее это сокращение сохраняется только в названиях команд и диалогов.

Существуют две основные категории: ссылки на код и ссылки на данные. Каждая ссылка направлена от одного адреса к другому; оба адреса могут относиться как к коду, так и к данным. В терминах теории графов адреса являются узлами ориентированного графа, а ссылки - его рёбрами.

Простейшую схему можно представить так:

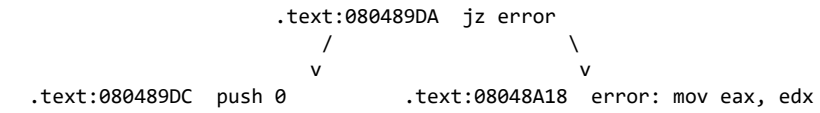


Рисунок 9-1. Основные части графа

Узлы также называют вершинами. Стрелки на ориентированных рёбрах задают допустимое направление перехода. В примере из верхнего узла можно попасть в любой нижний, но вернуться по этим рёбрам нельзя.

Ссылки на код особенно важны для построения графов потока управления и вызовов функций.

В листинге сведения обычно помещаются в комментарий заголовка функции:

```
.text:00401000 ; Attributes: bp-based frame
.text:00401000
.text:00401000 sub_401000      proc near               ; CODE XREF: _main+2A↓p
.text:00401000
```

Рисунок 9-2. Обычная перекрёстная ссылка

Текст `CODE XREF` означает ссылку на код, а `DATA XREF` - ссылку на данные. Далее указан исходный адрес, например `_main+2A`. Такая форма информативнее абсолютного `.text:0040154A`: она сообщает, что ссылка исходит из функции `_main`, на расстоянии `0x2A`, или 42 байта, от её начала.

Стрелка вверх или вниз после адреса показывает относительное направление. Стрелка вниз означает, что источник находится по большему адресу и для перехода к нему нужно прокрутить листинг вниз; стрелка вверх указывает на меньший адрес. Однобуквенный суффикс в конце комментария обозначает разновидность ссылки.

Ссылки на код

Ссылка на код указывает, что инструкция передаёт или может передать управление другой инструкции. В IDA способ передачи управления называется потоком. Различаются три основных вида:

• обычный последовательный поток;
• переход;
• вызов.

Переходы и вызовы дополнительно делятся на ближние и дальние. Дальние адреса встречаются только в программах с сегментированной адресацией.

В следующих примерах используется дизассемблированная версия программы:

```
int read_it;            // переменная, которую main читает
int write_it;           // переменная, которую main изменяет трижды
int ref_it;             // переменная, чей адрес получает main

void callflow() {}      // функция, дважды вызываемая из main

int main() {
    int *p = &ref_it;   // ссылка на адрес
    *p = read_it;       // ссылка чтения
    write_it = *p;      // ссылка записи
    callflow();         // ссылка вызова
    if (read_it == 3) { // ссылка перехода
        write_it = 2;   // ссылка записи
    }
    else {              // ссылка перехода
        write_it = 1;   // ссылка записи
    }
    callflow();         // ссылка вызова
}
```

Этот пример создаёт все основные виды перекрёстных ссылок IDA.

Обычный поток

Обычный поток представляет последовательное выполнение от одной инструкции к следующей. Он подразумевается для всех инструкций без ветвления, например `add`, и не имеет особого обозначения. Если инструкция A образует обычный поток к B, то B непосредственно следует за A в листинге.

В следующем коде все инструкции, кроме отмеченных X и Y, имеют обычный поток к своему непосредственному преемнику:

```
.text:00401010 _main           proc near
.text:00401010
.text:00401010 p               = dword ptr -4
```

```
.text:00401010
.text:00401010                 push    ebp
.text:00401011                 mov     ebp, esp
.text:00401013                 push    ecx
.text:00401014             `   mov     [ebp+p], offset ref_it
.text:0040101B                 mov     eax, [ebp+p]
.text:0040101E             ^   mov     ecx, read_it
.text:00401024                 mov     [eax], ecx
.text:00401026                 mov     edx, [ebp+p]
.text:00401029                 mov     eax, [edx]
.text:0040102B             _   mov     write_it, eax
.text:00401030             Z   call    callflow
.text:00401035             ^   cmp     read_it, 3
.text:0040103C                 jnz     short loc_40104A
.text:0040103E             _   mov     write_it, 2
.text:00401048             X   jmp     short loc_401054
.text:0040104A ; ---------------------------------------------------------
.text:0040104A
.text:0040104A loc_40104A:         ; CODE XREF: _main+2C j
.text:0040104A             _   mov     write_it, 1
.text:00401054
.text:00401054 loc_401054:         ; CODE XREF: _main+38 j
.text:00401054             Z   call    callflow
.text:00401059                 xor     eax, eax
.text:0040105B                 mov     esp, ebp
.text:0040105D                 pop     ebp
.text:0040105E             Y   retn
.text:0040105E _main           endp
```

Листинг 9-1. Источники и цели перекрёстных ссылок

Поток вызова

Инструкциям вызова функций, таким как `call` в строках Z, назначается поток вызова к целевой функции. Обычно у них есть и обычный поток, поскольку после возврата выполнение продолжается со следующей инструкции. Если анализ IDA установил, что функция не возвращает управление, обычного потока после вызова не будет.

Ссылки вызова отображаются у целевой функции:

```
.text:00401000 callflow proc near       ; CODE XREF: _main+20 p
.text:00401000                          ; _main:loc_401054 p
.text:00401000          push    ebp
.text:00401001          mov     ebp, esp
.text:00401003          pop     ebp
.text:00401004          retn
.text:00401004 callflow endp
```

Два комментария сообщают о двух вызовах `callflow`. Адрес источника показывается как смещение внутри вызывающей функции, если самой инструкции не присвоено имя; иначе используется её имя. Суффикс `p` обозначает вызов процедуры.

Поток перехода

Поток перехода назначается каждой условной и безусловной инструкции ветвления. Условная ветвь также имеет обычный поток для случая, когда переход не выполнен. У безусловной ветви обычного потока нет.

Разрыв разделительной линии перед `loc_40104A` в листинге 9-1 показывает отсутствие последовательного потока между соседними строками. Ссылки переходов выводятся у адреса назначения и содержат исходный адрес. Их отличает суффикс `j`.

Ссылки на данные

Ссылки на данные показывают, как программа обращается к памяти. Они могут быть связаны с любым байтом базы, имеющим виртуальный адрес, но не со стековыми переменными. Чаще всего встречаются три вида:

• чтение содержимого;
• запись содержимого;
• получение адреса.

Глобальные переменные предыдущего примера дают все три варианта:

```
.data:0040B720 read_it   dd ?        ; DATA XREF: _main+E r
.data:0040B720                        ; _main+25 r
.data:0040B724 write_it  dd ?        ; DATA XREF: _main+1B w
.data:0040B724                        ; _main+2E w ...
.data:0040B728 ref_it    db ?        ; DATA XREF: _main+4 o
.data:0040B729           db ?
.data:0040B72A           db ?
.data:0040B72B           db ?
```

Ссылки чтения

Ссылка чтения означает обращение к содержимому памяти. Её источником может быть только инструкция, а целью - любая программная область. В листинге 9-1 переменная `read_it` читается в строках ^; суффикс `r` помечает соответствующие комментарии.

Первое чтение переносит 32 бита в ECX, поэтому IDA оформляет `read_it` как двойное слово `dd`. В целом она использует характер обращений и передачу параметров функциям, чтобы определить размер и тип переменных.

Ссылки записи

Переменная `write_it` изменяется в строках _. Комментарии с суффиксом `w` перечисляют инструкции записи. Размер снова выводится по 32-битному регистру EAX.

Многоточие после списка означает, что ссылок больше установленного предела отображения, который по умолчанию равен двум. Значение меняется параметром `Number of displayed xrefs` на вкладке `Cross-references` диалога `Options > General`.

Источник ссылки записи всегда является инструкцией, а цель может находиться где угодно. Запись в байт инструкции обычно означает самомодифицирующийся код. Такой приём считается нежелательным, но часто встречается в распаковщиках и процедурах снятия обфускации вредоносных программ.

Ссылки на адрес

Третий вид показывает использование адреса, а не содержимого. В строке ` ` листинга 9-1 берётся адрес ref_it, поэтому комментарий у переменной имеет суффикс o`.

Такие ссылки обычно возникают при работе с указателями в коде и данных. Например, доступ к массиву складывает смещение с его начальным адресом, поэтому первый элемент глобального массива часто имеет ссылку на адрес. Строки, являющиеся массивами символов, обычно обнаруживаются по тому же признаку.

В отличие от чтения и записи, ссылка на адрес может исходить как от инструкции, так и из области данных. Характерный пример - таблица указателей, включая виртуальную таблицу. Каждая её ячейка ссылается на адрес функции:

```
.rdata:00408148 off_408148 dd offset SubClass::vfunc1(void) ; DATA XREF: SubClass::SubClass(void)+12 o
.rdata:0040814C            dd offset BaseClass::vfunc2(void)
.rdata:00408150            dd offset SubClass::vfunc3(void)
.rdata:00408154            dd offset BaseClass::vfunc4(void)
.rdata:00408158            dd offset SubClass::vfunc5(void)
```

Адрес таблицы используется в конструкторе `SubClass::SubClass(void)`. В заголовке `SubClass::vfunc3(void)` видна обратная ссылка на соответствующую ячейку:

```
.text:00401080 public: virtual void __thiscall SubClass::vfunc3(void) proc near
.text:00401080                 ; DATA XREF: .rdata:00408150 o
```

Здесь проявляется важный признак C++: виртуальная функция не вызывается напрямую и не должна быть целью обычной ссылки вызова. Вместо этого на неё указывает хотя бы одна ячейка виртуальной таблицы, поэтому имеется хотя бы одна ссылка на адрес. Поскольку производный класс не обязан переопределять каждую функцию, одна реализация может входить в несколько таблиц. Прослеживание таких ссылок назад помогает находить виртуальные таблицы в области данных.

Списки перекрёстных ссылок

Число комментариев у одного адреса ограничено настройкой, по умолчанию двумя. Пока ссылок не больше, работать с ними просто: при наведении IDA показывает исходную область во всплывающей подсказке, а двойной щелчок по адресу переносит листинг к источнику.

Полный список можно открыть двумя способами. Первый: установите курсор на адрес-цель и выберите `View > Open Subviews > Cross-References`.

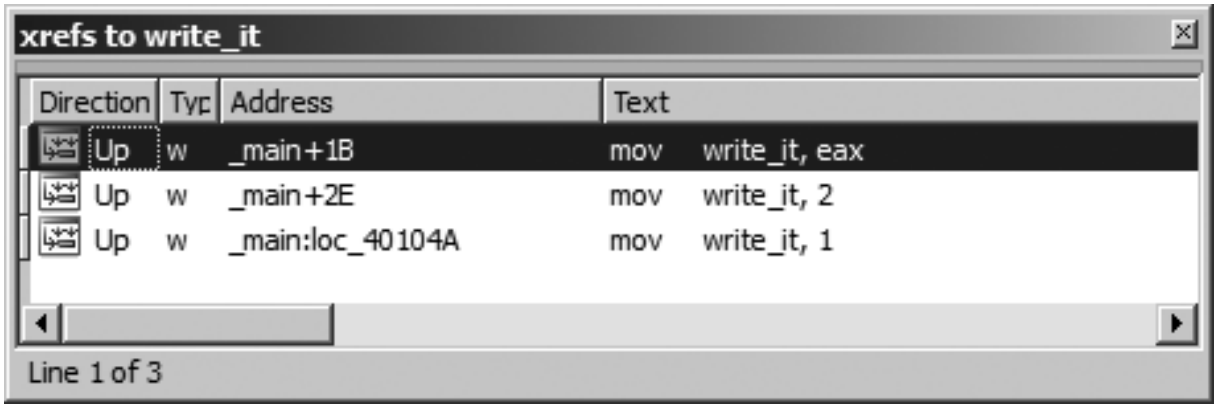


Рисунок 9-3. Окно перекрёстных ссылок

Столбцы показывают направление к источнику, вид ссылки, исходный адрес и дизассемблированный текст с комментариями. Двойной щелчок по строке переносит листинг к соответствующему адресу. Это немодальное окно остаётся открытым как отдельная вкладка.

Второй способ: выделите интересующее имя и выберите `Jump > Jump to xref` либо нажмите `Ctrl+X`.

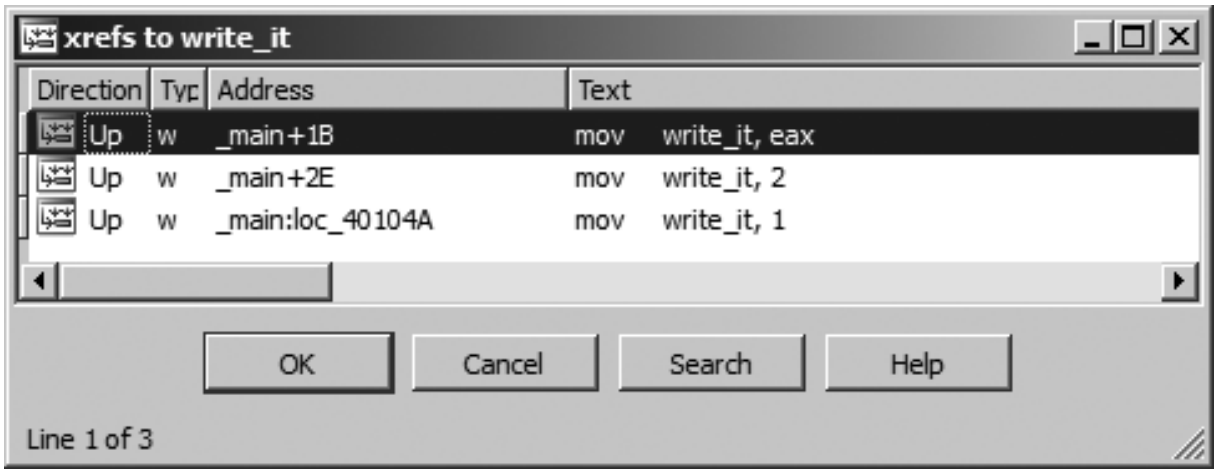


Рисунок 9-4. Диалог перехода по перекрёстной ссылке

Внешне диалог почти совпадает с отдельным окном, но является модальным: прежде чем продолжить обычную работу с IDA, его нужно закрыть. Основная задача диалога - выбрать источник и перейти к нему. Двойной щелчок закрывает диалог и устанавливает листинг на выбранный адрес.

Есть и второе различие. Диалог можно вызвать сочетанием клавиш или через контекстное меню на любом вхождении символа, то есть даже у источника ссылки. Отдельное окно открывается только тогда, когда курсор стоит на адресе-цели.

Списки позволяют быстро найти все места вызова опасной функции. Например, `strcpy` копирует строку вместе с завершающим нулём, не проверяя вместимость приёмного массива. Найдите любое обращение к ней, нажмите `Ctrl+X` и последовательно просмотрите все ссылки вызова. Если искать первое обращение неудобно, можно даже добавить комментарий со словом `strcpy` и открыть список через него: IDA обрабатывает имя символа в комментарии почти как операнд инструкции.

Вызовы функций

Специализированный список только для вызовов открывается командой `View > Open Subviews > Function Calls`.

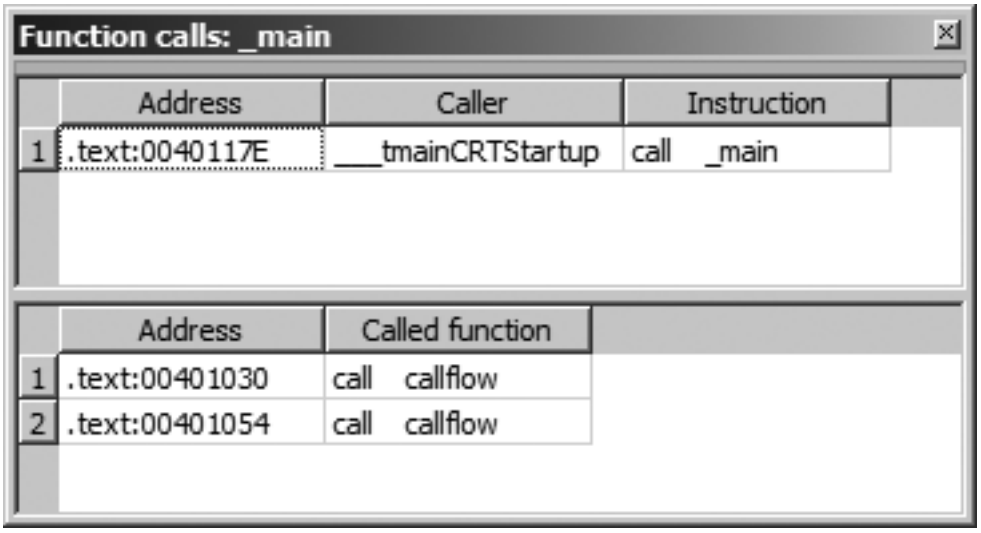


Рисунок 9-5. Окно вызовов функций

В верхней половине перечислены места, вызывающие текущую функцию, выбранную положением курсора. В нижней - все вызовы, выполняемые ею самой. Любая строка переносит листинг к соответствующему адресу.

Ограничившись связями между функциями, можно рассматривать не отдельные адреса, а более общие отношения между крупными частями программы. На этом основаны графы IDA.

Построение графов в IDA

Поскольку перекрёстная ссылка связывает два адреса, она естественным образом становится ребром графа. В зависимости от задачи узел может обозначать отдельную инструкцию, группу инструкций, называемую базовым блоком, или целую функцию.

IDA располагает двумя отдельными средствами:

• внешними графами, которые открываются в дополнительной программе;
• встроенным интерактивным представлением.

Внешние средства построения графов

Внешний режим передаёт созданное IDA описание сторонней программе просмотра. В выпусках для Windows до 6.1 поставлялась `wingraph32`. В IDA 6.0 для других систем по умолчанию применялась `dotty`. Начиная с 6.1 все варианты IDA поставляются с межплатформенной программой `qwingraph`, основанной на Qt и происходящей от `wingraph32`. Для Linux параметры `dotty` сохранились в файле настройки, но по умолчанию отключены.

Программа просмотра задаётся переменной `GRAPH_VISUALIZER` в `<IDADIR>/cfg/ida.cfg`. При запросе внешнего графа IDA сохраняет описание во временный файл и запускает указанную программу.

Поддерживаются два языка описания: GDL и DOT из проекта Graphviz. Выбор задаёт переменная `GRAPH_FORMAT` в том же файле настройки; допустимы значения `GDL` и `DOT`. Формат должен поддерживаться выбранной программой просмотра.[1]

В подменю `View > Graphs` доступны пять внешних графов:

• схема потока функции;
• граф вызовов всего двоичного файла;
• ссылки на выбранный символ;
• ссылки от выбранного символа;
• настраиваемый граф перекрёстных ссылок.

Для схемы потока и графа вызовов IDA умеет сохранять описание GDL, но не DOT, через подменю `File > Produce file`. Другие описания иногда можно сохранить средствами самой программы просмотра.

Главное ограничение внешних графов - отсутствие связи с рабочим листингом IDA. Доступны лишь возможности программы просмотра, обычно масштабирование и прокрутка.

Базовые блоки

Базовый блок - последовательность из одной или нескольких инструкций с единственным входом в начало и единственным выходом из конца. Каждая инструкция, кроме последней, передаёт управление ровно одному следующему элементу того же блока. Каждая инструкция, кроме первой, получает управление ровно от одного предшественника внутри блока.

При выделении блоков вызов функции обычно не считают выходом за его пределы, если только известно, что функция не возвращает управление.

Важное свойство: после выполнения первой инструкции все остальные инструкции блока гарантированно выполнятся до его конца. Это полезно для динамического наблюдения. Необязательно ставить точку останова на каждую инструкцию или выполнять программу пошагово. Достаточно остановиться в начале каждого блока и при срабатывании отметить весь блок как выполненный. Именно так действует компонент Process Stalker среды PaiMei Педрама Амини.[2]

Внешние схемы потока

Установите курсор внутри функции и выберите `View > Graphs > Flow Chart` либо нажмите `F12`. Результат ближе всего к встроенному графическому представлению IDA. Это не традиционная блок-схема из вводного курса программирования, а граф потока управления: инструкции объединены в базовые блоки, а рёбра показывают переходы между ними.

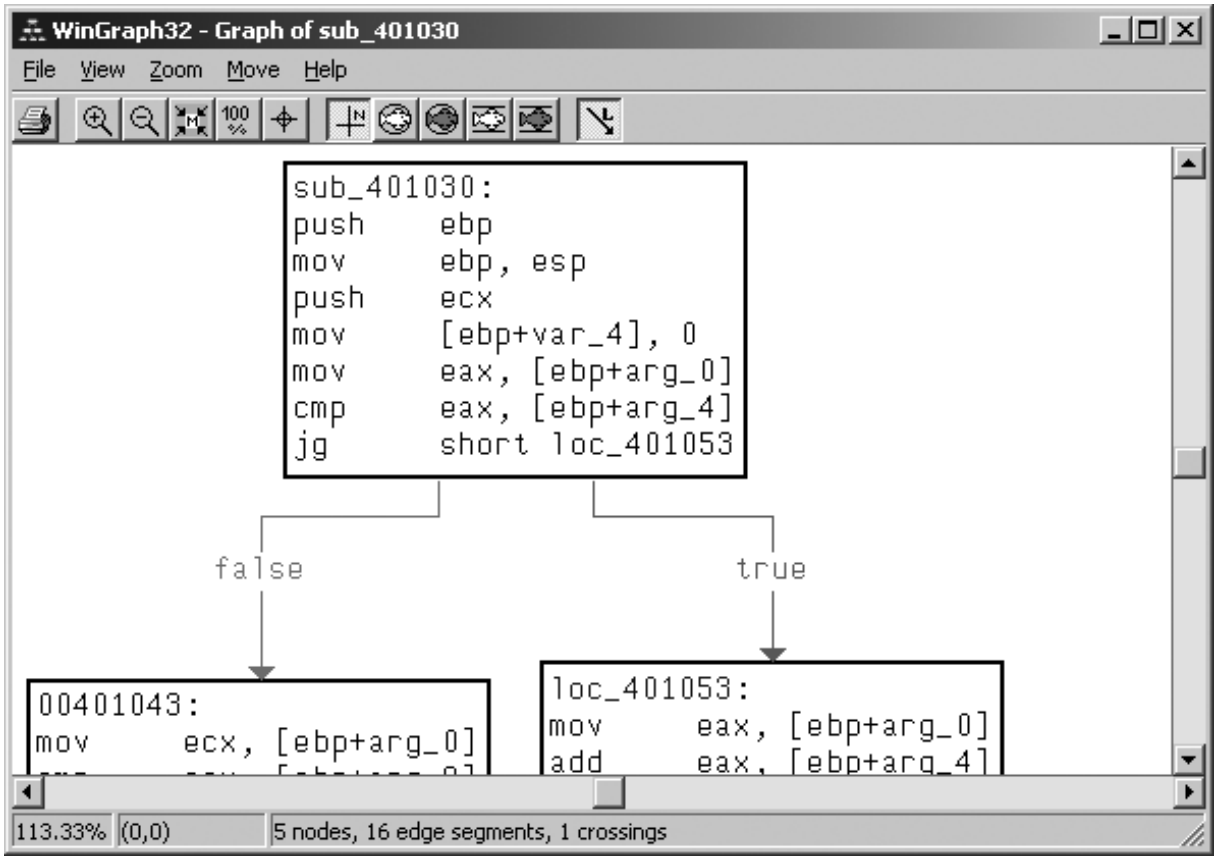


Рисунок 9-6. Внешний граф потока управления

Внешнее представление сообщает мало адресных сведений, поэтому его трудно сопоставлять с текстовым листингом. Граф строится от точки входа функции по обычным потокам и переходам каждой инструкции.

Внешние графы вызовов

Граф вызовов быстро показывает иерархию функций программы. Каждая функция становится узлом, а ссылка вызова - ребром. Построение от одной функции можно представить как рекурсивный спуск по всем вызываемым ею функциям.

Обычно разумно прекращать спуск на библиотечной функции: её поведение проще узнать из документации, чем восстанавливать по машинному коду. В динамически скомпонованном файле кода библиотеки вообще нет. Статическая компоновка создаёт противоположную проблему: включённый библиотечный код делает граф огромным.

Рассмотрим простую программу:

```
#include <stdio.h>

void depth_2_1() {
    printf("внутри depth_2_1\n");
}

void depth_2_2() {
    fprintf(stderr, "внутри depth_2_2\n");
}

void depth_1() {
    depth_2_1();
    depth_2_2();
    printf("внутри depth_1\n");
}

int main() {
    depth_1();
}
```

После динамической сборки GNU `gcc` выберите `View > Graphs > Function Calls`:

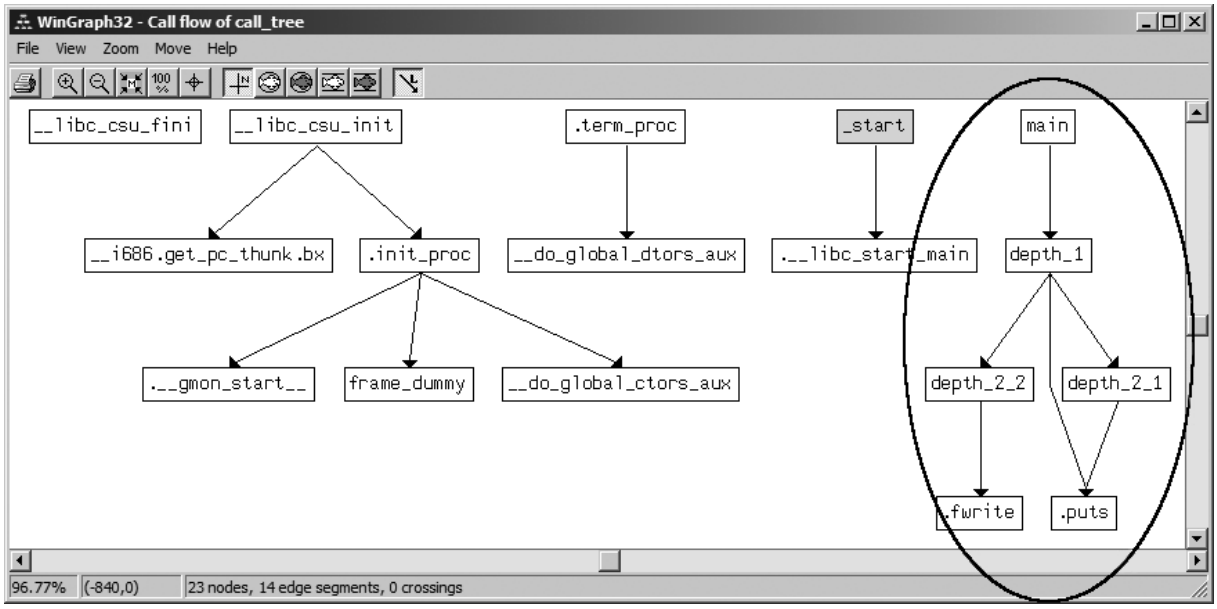


Рисунок 9-7. Внешний граф вызовов функций

Компилятор заменил `printf` и `fprintf` вызовами `puts` и `fwrite`, более эффективными для постоянных строк. Разные виды узлов IDA окрашивает разными цветами, которые нельзя настроить.

Почему граф почти вдвое сложнее исходной программы? Компилятор добавил обвязку инициализации и завершения библиотек, а также подготовку параметров перед передачей

управления `main`.

У статически скомпонованного варианта результат почти непригоден:

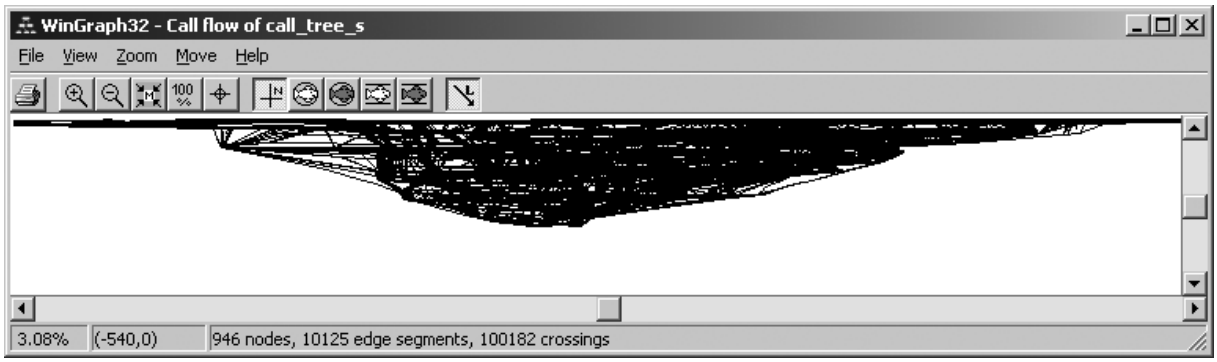


Рисунок 9-8. Граф вызовов статически скомпонованного двоичного файла

Внешний просмотрщик первоначально масштабирует граф целиком. В показанном случае имеется 946 узлов, 10 125 отрезков рёбер и 100 182 пересечения. Масштабирование не уменьшает логическую сложность, а найти `main` можно лишь чтением подписей. Когда они становятся различимы, на экране помещаются только десятки узлов.

Внешние графы перекрёстных ссылок

Для глобального символа, функции или переменной доступны два направления:

- `View > Graphs > Xrefs To` - ссылки на символ;
- `View > Graphs > Xrefs From` - ссылки от символа.

Граф ссылок на символ рекурсивно поднимается по всем входящим связям, пока не достигнет объектов, на которые больше никто не ссылается. Он отвечает на вопрос: «Какая последовательность вызовов приводит к этой функции?»

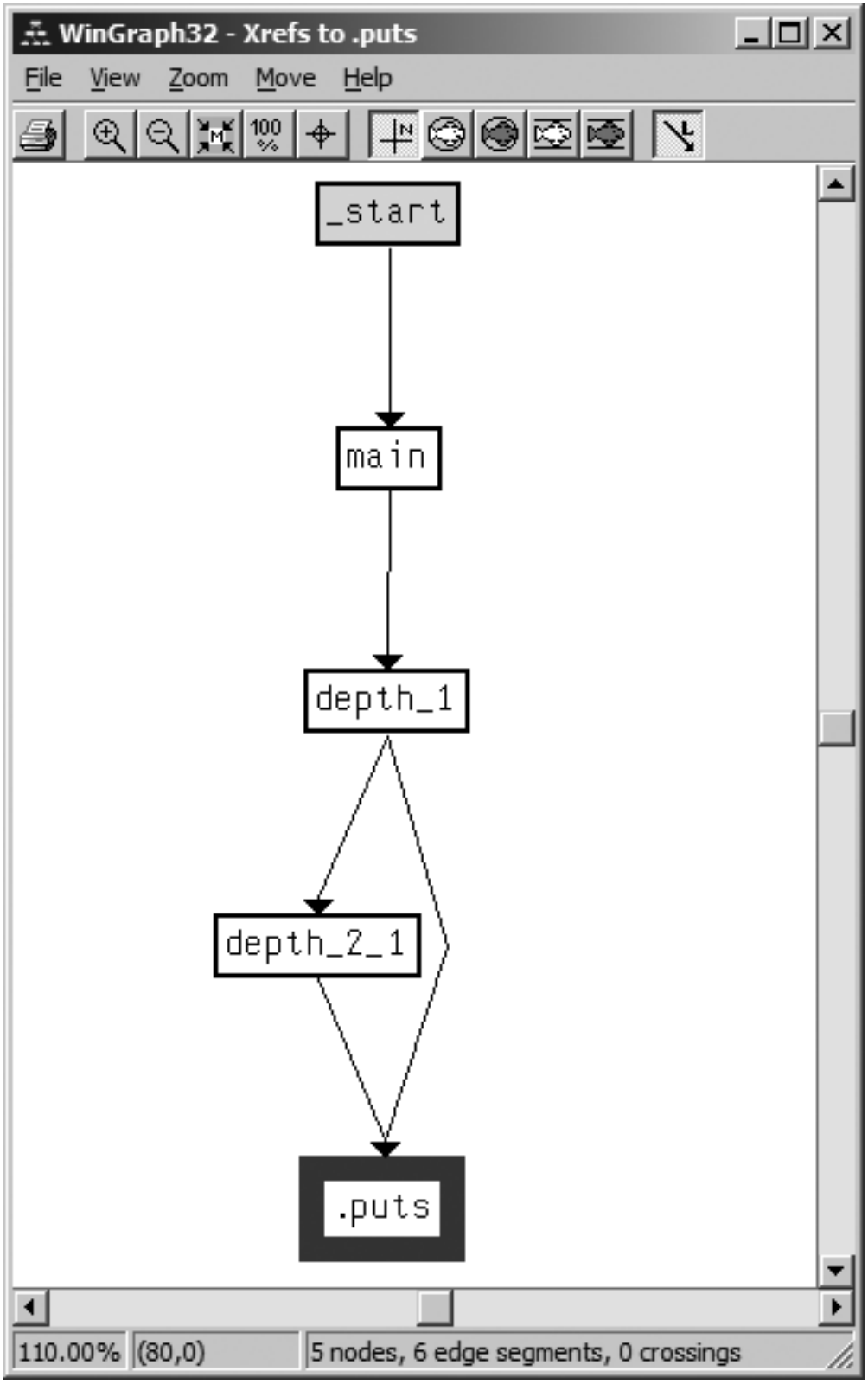


Рисунок 9-9. Граф ссылок на символ

Так же можно показать все обращения к глобальной переменной и цепочки функций, ведущие к этим местам. Только графы перекрёстных ссылок способны включать ссылки на данные.

Граф ссылок от символа рекурсивно следует по исходящим связям. Для функции учитываются только вызовы, поэтому обращения к глобальным данным не показываются. Для инициализированного глобального указателя прослеживается ссылка на хранимый адрес. В случае функции результат фактически является графом вызовов с выбранной функцией в корне:

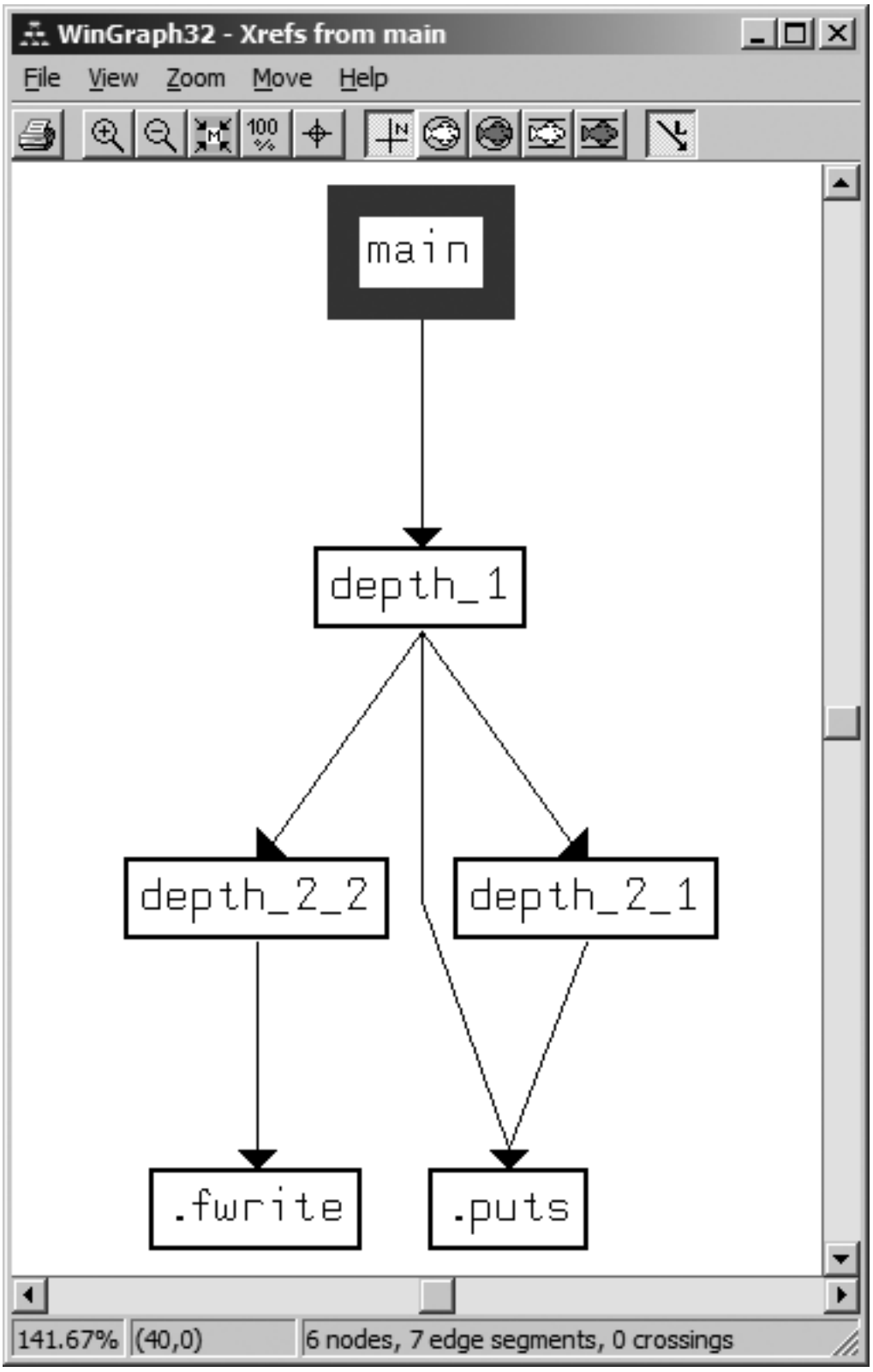


Рисунок 9-10. Граф ссылок от функции

У сложной функции здесь возникает та же проблема перегруженности.

Настраиваемые графы перекрёстных ссылок

Настраиваемые графы, называемые в IDA `User xref charts`, дают наибольшую свободу. В одном результате можно объединить входящие и исходящие ссылки, задать предельную глубину рекурсии и включить либо исключить отдельные категории символов.

Команда `View > Graphs > User Xrefs Chart` открывает диалог настройки:

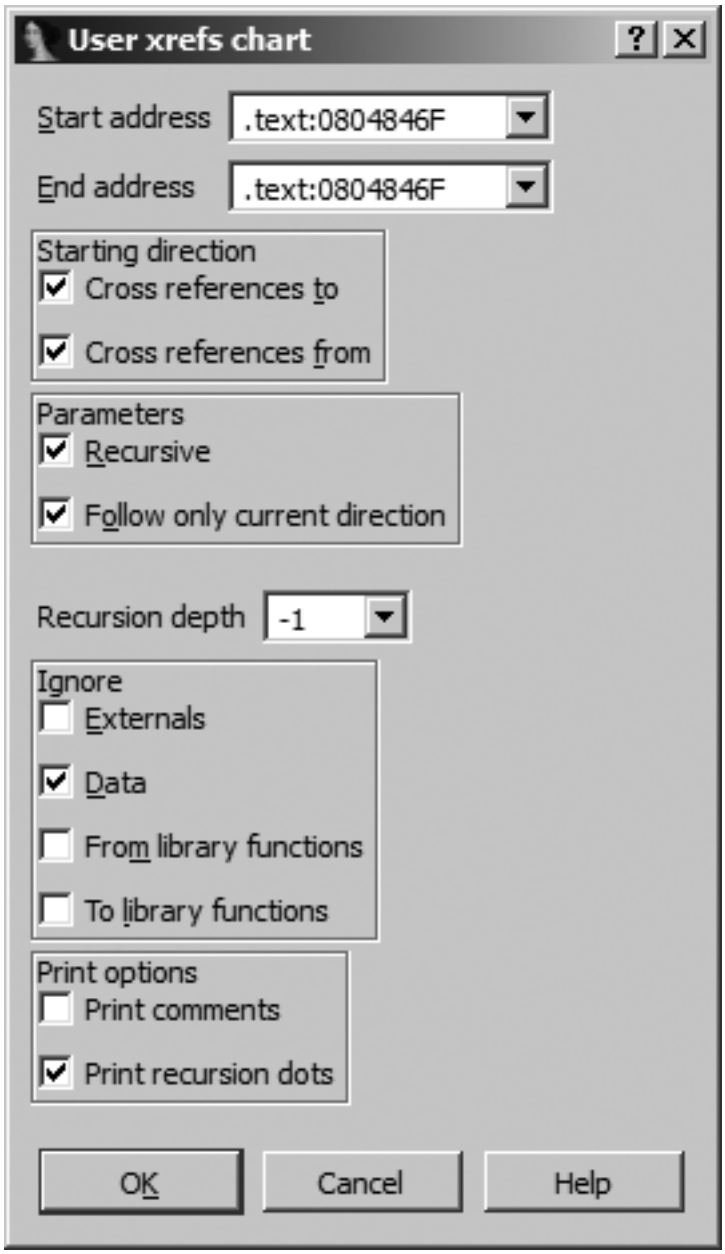


Рисунок 9-11. Диалог пользовательского графа перекрёстных ссылок

Каждый глобальный символ в заданном диапазоне адресов становится узлом. Чаще всего исследуется один символ, поэтому начальный и конечный адреса совпадают. Если они различаются, граф включает все нелокальные символы диапазона. Крайний вариант от первого до последнего адреса базы фактически превращается в граф вызовов всего двоичного файла.

Параметры имеют следующий смысл.

Начальное направление. Можно искать ссылки от выбранного символа, на него либо в обе стороны. При остальных стандартных настройках только входящее направление даёт обычный граф `Xrefs To`, а только исходящее - `Xrefs From`.

Параметры обхода. `Recursive` включает рекурсивный спуск по исходящим или подъём по входящим ссылкам. `Follow only current direction` запрещает менять направление на последующих узлах. Например, если из A достижим B, то при спуске из B добавятся только узлы, достижимые от B; новые объекты, ссылающиеся на B, не попадут в граф. Если снять этот флажок и выбрать оба начальных направления, каждый новый узел будет исследоваться в обе стороны.

Глубина рекурсии. Ограничивает число уровней обхода и тем самым размер графа. Значение `-1` разрешает максимально глубокий обход и создаёт самый крупный результат.

Исключения. Позволяют не добавлять определённые категории узлов. Особенно сильно граф статически скомпонованной программы упрощается при исключении ссылок из библиотечных функций. Для этого IDA должна правильно распознать как можно больше библиотечного кода; эта тема рассматривается в главе 12.

Параметры печати. `Print comments` добавляет комментарии функции в её узел. `Print recursion dots` показывает узел с многоточием, когда обход мог бы продолжиться за установленным пределом.

Ниже показан граф `depth_1`, созданный со стандартными параметрами и глубиной 1:

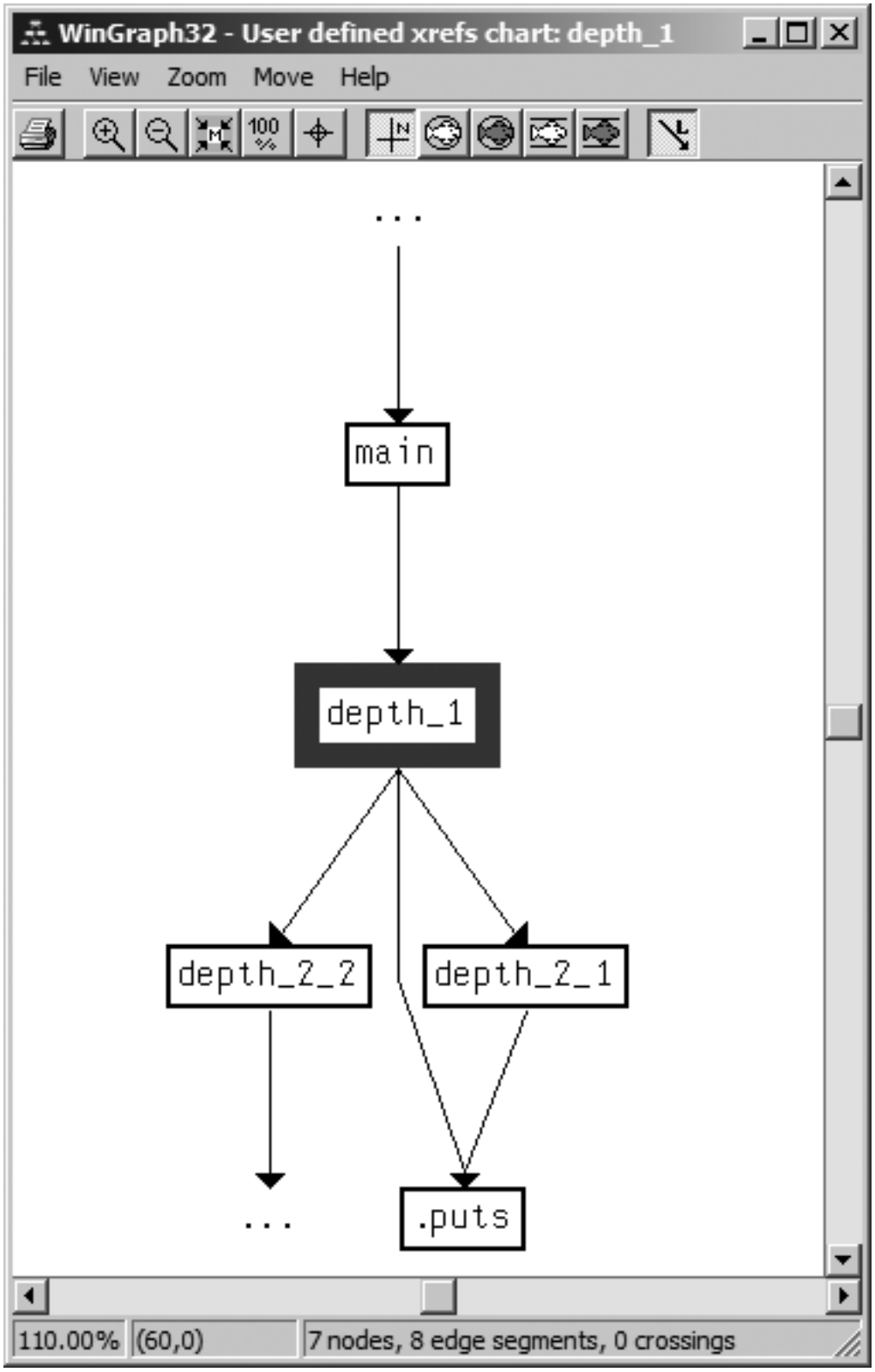


Рисунок 9-12. Пользовательский граф перекрёстных ссылок для `depth_1`

Это самое мощное внешнее средство IDA. Внешние схемы потока во многом заменены встроенным представлением, а остальные виды внешних графов являются заранее настроенными частными случаями пользовательского графа.

Встроенное графическое представление IDA

Начиная с версии 5.0 IDA содержит интерактивное графическое представление, тесно связанное с дизассемблером. Оно служит альтернативой обычному текстовому листингу и показывает функцию как граф потока управления.

В один момент видна только одна функция. Инструкции, не принадлежащие ни одной функции, в таком режиме открыть нельзя. Для одновременного просмотра нескольких функций или внесистемного кода нужно вернуться к текстовому представлению.

Между режимами переключаются пробелом либо пунктами `Text View` и `Graph View` контекстного меню. Граф удобно перемещать перетаскиванием фона. Для большой схемы лучше пользоваться обзорным окном: пунктирный прямоугольник показывает текущую видимую область, а его перетаскивание сразу перемещает основной вид без многократного отпускания мыши.

Большинство действий одинаково в обоих режимах. Двойной щелчок и журнал переходов продолжают работать. Переход к адресу вне функции, например к глобальной переменной, автоматически включает текстовый вид; возврат в функцию восстанавливает граф.

Краткое описание стека находится в корневом базовом блоке. Двойной щелчок по стековой переменной открывает подробный кадр. Все команды форматирования операндов из текстового режима доступны и в графическом.

Управление узлами

Главное отличие интерфейса связано с отдельными узлами:

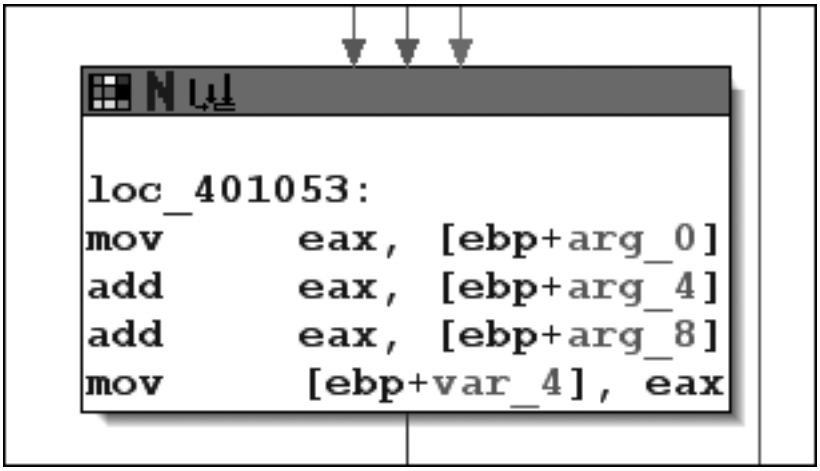


Рисунок 9-13. Обычный развёрнутый узел графа

Три кнопки в заголовке слева направо позволяют:

1. изменить цвет фона;
2. назначить или изменить имя узла;
3. открыть список входящих перекрёстных ссылок.

Цветом удобно отмечать уже исследованные блоки или выделять особенно интересный код. Он применяется и к соответствующим инструкциям в текстовом режиме. Для сброса выберите в контекстном меню заголовка `Set node color to default`.

Средняя кнопка назначает имя адресу первой инструкции базового блока. Блоки часто уже имеют автоматически созданные имена, потому что являются целями переходов, но имя бывает не всегда. Например:

```
.text:00401041 X   jg      short loc_401053
.text:00401043 Y   mov     ecx, [ebp+arg_0]
```

У инструкции X два возможных продолжения: `loc_401053` и следующая строка Y. Поэтому X завершает один базовый блок, а Y начинает новый, хотя на Y нет явного перехода и автоматического имени.

Правая кнопка открывает ссылки, ведущие в узел. Поскольку в графическом режиме комментарии ссылок по умолчанию скрыты, это самый простой способ перейти к источнику. В такой список дополнительно включается обычный поток с обозначением ^, потому что линейный предшественник узла на схеме не всегда очевиден.

Чтобы показывать обычные комментарии ссылок прямо в графе, на вкладке `Cross-References` диалога `Options > General` установите `Number of displayed xrefs` больше нуля.

Группировка узлов

Узлы можно сворачивать по одному или группами, уменьшая визуальную перегруженность. Выделите заголовки нужных узлов с нажатой `Ctrl`, затем откройте контекстное меню любого из них и выберите `Group nodes`. IDA предложит текст свёрнутого узла, по умолчанию первую инструкцию группы.

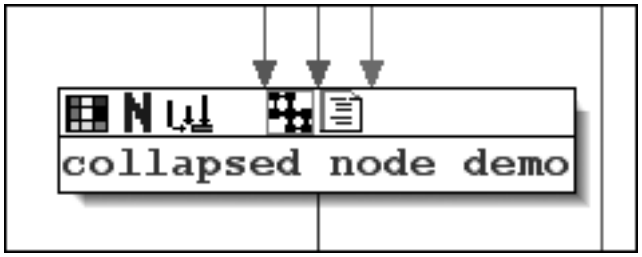


Рисунок 9-14. Свёрнутый узел графа

В заголовке появляются две новые кнопки: развернуть группу и изменить её подпись. Разворачивание возвращает исходный вид узлов, но не отменяет их принадлежность группе. У развёрнутой группы вместо этих кнопок показывается `Collapse Group`; также её можно снова скрыть командой `Hide Group`.

Чтобы полностью удалить объединение, выберите `Ungroup Nodes` в меню заголовка свёрнутого узла или любого узла развёрнутой группы. Если группа была свёрнута, она при этом развернётся.

Итоги

Графы заметно помогают анализировать двоичные файлы. После привычного текстового листинга к ним может потребоваться привыкнуть, но вся прежняя информация остаётся доступной, только представляется иначе. Например, перекрёстные ссылки становятся рёбрами между базовыми блоками.

Важно выбрать подходящий вид. Чтобы понять, как достигается функция, полезен граф вызовов или входящих ссылок. Чтобы установить путь к конкретной инструкции, нужен граф потока управления.

Часть прежних неудобств была вызвана ограничениями `wingraph32` и внешних графов. Встроенный интерактивный режим решил многие из них. Однако IDA прежде всего дизассемблер, а

не специализированная система анализа графов. Для задач, сосредоточенных именно на графах, существуют отдельные инструменты, например BinNavi компании Zynamics.[3]

Сноски

[1] Исходные тексты `wingraph32` и `qwingraph`, а также справочные материалы GDL и DOT упоминались на сайтах Hex-Rays, AbsInt и Graphviz. Адрес проекта Graphviz: <http://www.graphviz.org/>.
[2] Страница среды PaiMei: <http://pedram.redhive.com/code/paimei/>.
[3] Страница BinNavi: <http://www.zynamics.com/binnavi.html>. В марте 2011 года компания Zynamics была приобретена Google.

Глава 10. Многоликая IDA

Много лет главной версией IDA считался графический выпуск для Windows. Начиная с IDA 6.0 полноценный графический интерфейс появился также у пользователей Linux и OS X. Однако по-прежнему существует несколько способов работать с IDA.

Первоначально IDA была консольной программой для MS-DOS, и текстовая версия до сих пор доступна на всех платформах. Вместе со встроенной удалённой отладкой это делает IDA мощным межплатформенным средством анализа.

Кроме интерактивного режима, все выпуски поддерживают пакетную обработку множества файлов. Для эффективной автоматизации нужно понимать возможности и ограничения каждого варианта и выбирать подходящий. В этой главе рассматриваются консольная IDA и её пакетный режим.

Консольная IDA

В основе всех консольных выпусков лежит библиотека ввода-вывода TVision, первоначально созданная Borland и перенесённая на Windows, Linux, Mac OS X и другие системы. Hex-Rays предоставляет исходный код своего варианта платным пользователям IDA на странице загрузки.

Общая библиотека обеспечивает сходный интерфейс на всех платформах. Различия всё же существуют: поддержка мыши и изменения размера окна, а также передача сочетаний клавиш приложению зависят от терминала.

Общие возможности консольного режима

Текстовая IDA работает внутри терминала или командной оболочки. Ограничения мыши и размера зависят от системы и терминальной программы.

В верхней строке находится меню и состояние, в нижней - панель часто используемых действий, похожая на текстовую панель инструментов. Команды вызываются сочетаниями клавиш, а при поддержке мыши - щелчками. Почти все возможности графического выпуска присутствуют и здесь, причём большинство сочетаний сохраняется.

Между двумя панелями располагаются окна IDA. При размере около 80 на 25 символов и без графики места мало, поэтому по умолчанию обычно открыты только дизассемблированный код и сообщения.

Вместо вкладок TVision использует перекрывающиеся текстовые окна. Клавиша `F6` циклически переключает их. Каждому присваивается номер, показанный в левом верхнем углу.

При наличии мыши окно перемещается за верхнюю рамку, а меняет размер перетаскиванием правого нижнего угла. Без мыши выберите `Window > Resize/Move` или нажмите `Ctrl+F5`: стрелки перемещают окно, а `Shift` со стрелками меняет его размер. Если терминал допускает изменение размеров, IDA подстраивается автоматически.

Без графики недоступно встроенное представление потока управления и не показываются стрелки переходов на левом поле. Остальные дополнительные окна из графической версии доступны через `View > Open Subviews`.

Отдельного шестнадцатеричного окна нет. Текущий дизассемблированный вид переключается в дамп и обратно командой `Options > Dump/Normal View` или `Ctrl+F4`. Чтобы одновременно видеть

код и дамп, откройте второе окно через `View > Open Subviews > Disassembly` и переключите его. Синхронизировать два вида нельзя.

При поддержке мыши двойной щелчок по имени переносит к соответствующему адресу. То же делает `Enter` на имени, причём и в графической версии. `Enter` на стековой переменной открывает подробное представление кадра. Без мыши меню управляется обычными сочетаниями `Alt` с выделенной буквой.

Особенности консоли Windows

Терминал `cmd.exe`, а в семействе Windows 9x - `command.exe`, не слишком гибок, но хорошо поддерживается IDA. Консольный исполняемый файл называется `idaw.exe`, графический - `idag.exe`. Для 64-разрядных файлов в расширенной редакции служат `idaw64.exe` и `idag64.exe`.

Для работы мыши отключите режим быстрого редактирования `QuickEdit`. До запуска IDA щёлкните заголовок терминала правой кнопкой, откройте свойства и снимите соответствующий флажок на вкладке параметров. Изменение во время работы IDA не учитывается.

В отличие от терминала под X Window, `cmd.exe` нельзя произвольно растянуть мышью. В консольной IDA для Windows команда `Window > Set Video Mode` предлагает шесть фиксированных размеров, вплоть до 255 на 100 символов.

Встроенного графического режима нет, но внешние графы доступны. Команды `View > Graphs` запускают настроенный просмотрщик, например `qwingraph`. Можно открыть несколько графов и продолжать работу в IDA.

Особенности консоли Linux

Консольный файл называется `idal`, а для анализа 64-разрядных программ - `idal64`. До IDA 6.0 консольные версии Linux и OS X входили в обычный комплект. При их переносе требовалось скопировать ключ `ida.key`, из-за чего IDA приходилось хотя бы однажды установить в Windows. В системах семейства Unix ключ можно было поместить в `$HOME/.idapro/ida.key`; каталог личных настроек создавался при первом запуске.

В IDA 6.x установка проще: продукт приобретается для конкретной платформы, а установщик размещает графическую и консольную версии вместе с ключом.

Основная навигация похожа на Windows, но выбор терминалов гораздо шире. В поставке есть `tvtuning.txt` с рекомендациями для разных вариантов, включая удалённые клиенты SecureCRT и PuTTY.

Главная проблема - передача сочетаний клавиш самой IDA. Например, `Alt+F` может открыть меню файла терминала, а не IDA. Можно подобрать терминал без конфликтов, перенастроить его либо изменить сочетания IDA. Последний вариант лучше одинаково применить на всех рабочих машинах, иначе раскладки будут путаться и затруднят совместную работу.

В обычной текстовой консоли Linux размер фиксирован, а мышь зависит от сервера GPM. Если GPM не используется, запускайте IDA с параметром TVision `noGPM`:

```
# TVOPT=noGPM ./idal [файл для дизассемблирования]
```

Цветовая палитра консоли ограничена. В `Options > Colors` доступны четыре готовые схемы и настройка 16 цветов; иногда их приходится менять, чтобы текст не сливался с фоном.

Под X Window можно применять `konsole`, `gnome-terminal`, `xterm` и другие программы. Кроме `xterm`, почти все имеют собственные меню и сочетания, способные конфликтовать с IDA. Авторы предпочитают KDE `konsole` за внешний вид, небольшое число конфликтов и плавную работу мыши.

Джереми Купер создал собственный перенос TVision на X11, чтобы запускать IDA в отдельном окне X, не занимая весь терминал. Собранная библиотека заменяет `libtvision.so`, которую использует `idal`. Если после замены не загружается шрифт VGA, его нужно установить и добавить путь к нему в настройки сервера X. Такое окно также можно перенаправить по SSH на другую машину, например запустить IDA в Linux, а показать на Mac.

При удалённой работе с библиотекой TVision от Hex-Rays рекомендуется эмуляция `xterm` и параметры из `tvtuning.txt`. Например, для мыши в SecureCRT требуется:

```
TVOPT=xtrack
```

Переменную можно экспортировать, чтобы не задавать при каждом запуске. Полный перечень параметров содержится в `linux.cpp` из исходных текстов TVision.

Внешние графы консольной версии Linux доступны только в оконной среде и при правильно заданной переменной `GRAPH_VISUALIZER` в `ida.cfg`. Выпуски до 6.0 создают только GDL; для них можно установить просмотрщик вроде aiSee и прописать его в `<IDADIR>/cfg/ida.cfg`.

Стандартная настройка выглядит примерно так:

```
GRAPH_VISUALIZER = "qwingraph.exe -remove -timelimit 10"
```

Параметр `remove` просит удалить временный входной файл, а `timelimit` задаёт число секунд на аккуратную раскладку. Если времени не хватает, `qwingraph` переходит к быстрому упрощённому размещению. Начиная с IDA 6.0 параметр находится в условном блоке с отдельными значениями для Windows и других систем, поэтому редактировать нужно правильную часть файла.

Для типичной установки aiSee можно указать:

```
GRAPH_VISUALIZER = "/usr/local/bin/aisee"
```

Лучше задавать полный путь. Пользователи старых выпусков могут также собрать открытый `qwingraph` из исходного кода Hex-Rays и подключить его самостоятельно.

Особенности консоли OS X

Исполняемые файлы называются так же, как в Linux: `idal` и `idal64`. Для консольного ввода-вывода также используется TVision.

Раскладка клавиатуры Mac создаёт трудности, прежде всего потому, что клавиша Option/Alt ведёт себя не так, как Alt на обычной клавиатуре персонального компьютера.

При запуске через стандартный Terminal настройте Option как Alt. Тогда будут работать сочетания IDA, например `Alt+F` для меню файла. Иначе вместо Alt придётся нажимать `Esc`, то есть `Esc`, затем `F`, но в IDA `Esc` также означает возврат или закрытие окна, поэтому этот способ неудобен.

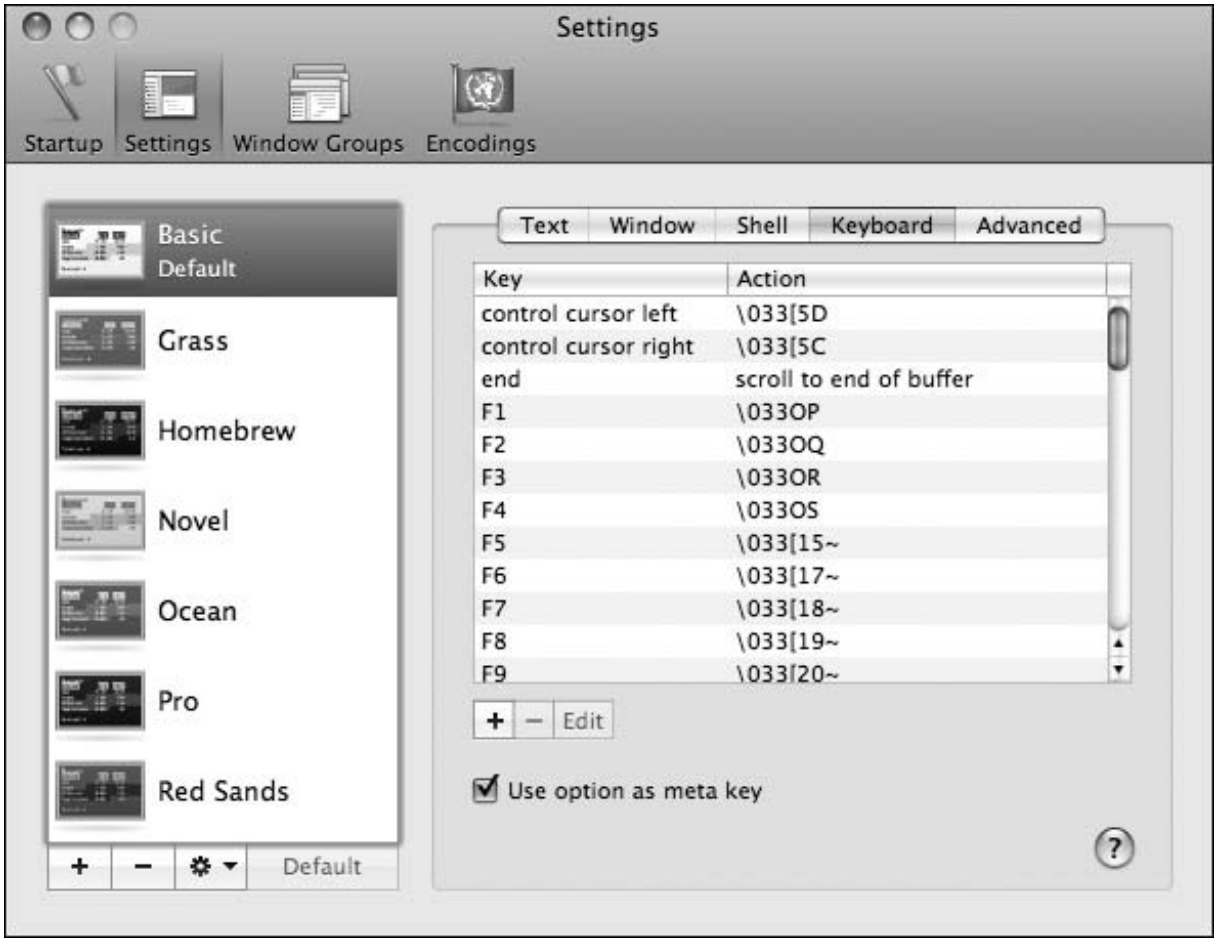


Рисунок 10-1. Настройка клавиатуры Terminal в OS X

В диалоге настроек Terminal включите `Use option key as meta key`. Альтернативой служит iTerm, где доступны и функция Alt у Option, и мышь. Существуют также перенесённые на X11 варианты терминала GNOME, но для большинства пользователей достаточно Terminal или iTerm.

Ещё один вариант - установить X11 из дополнительного пакета OS X и изменённую библиотеку Джереми Купера `libtvision.dylib`, чтобы запускать IDA как собственное приложение X11. Для удобного доступа к программам X11 можно добавить `/usr/X11R6/bin` в переменную `PATH`, отредактировав `/etc/profile`.

В такой конфигурации IDA запускается из `xterm` в отдельном окне с полноценной мышью. Однако X11 воспринимает Option/Alt как `Mode_switch` и не передаёт её IDA. Клавиши можно переназначить программой `xmodmap`. Создайте или измените файл `~/.Xmodmap`:

```
clear Mod1
keycode 66 = Alt_L
keycode 69 = Alt_R
add Mod1 = Alt_L
add Mod1 = Alt_R
```

Стандартный сценарий запуска X11 `/etc/X11/xinit/xinitrc` читает этот файл автоматически. Если используется собственный `~/.xinitrc`, добавьте в него:

```
xmodmap $HOME/.Xmodmap
```

Затем отключите переопределение раскладки системными настройками:

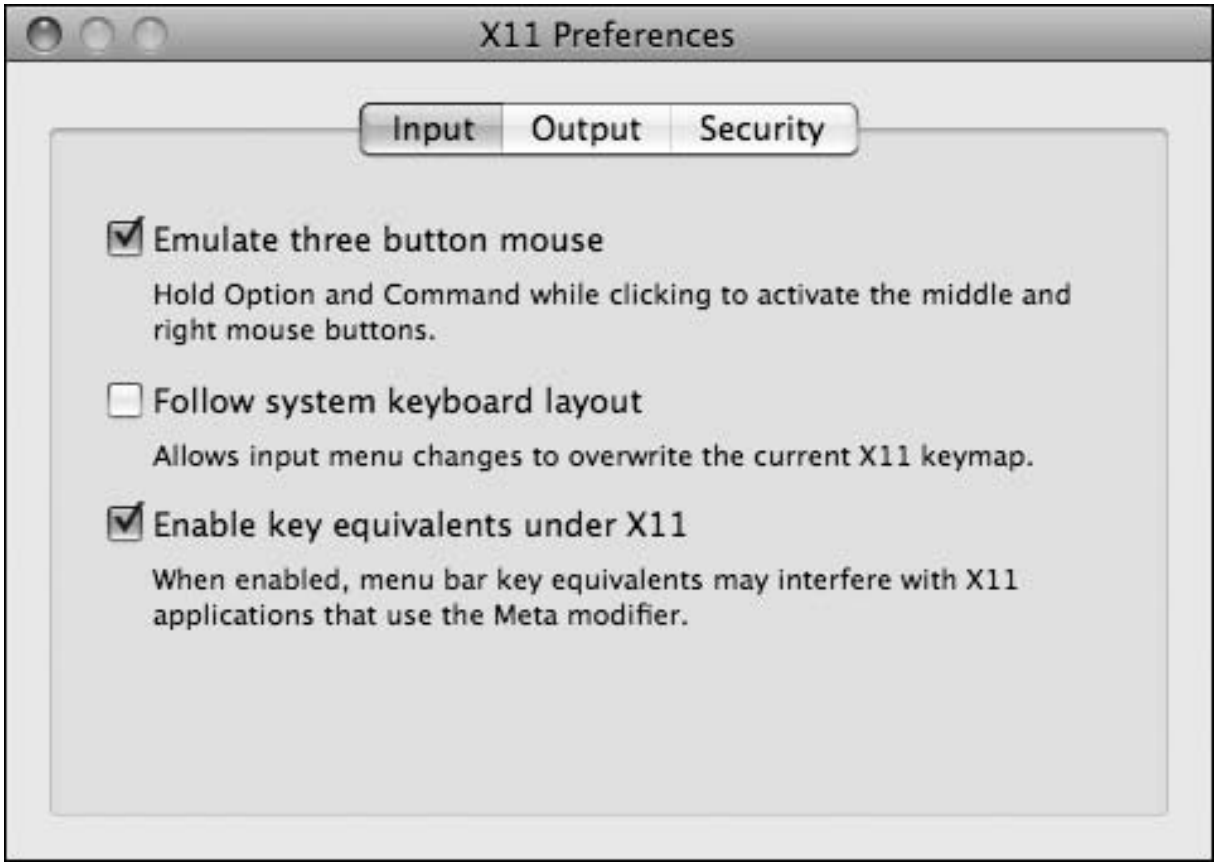


Рисунок 10-2. Параметры X11 в OS X

Снимите флажок `Follow system keyboard layout` и перезапустите X11. Проверить модификаторы можно командой `xmodmap`:

```
idabook:~ idabook$ xmodmap
xmodmap: up to 2 keys per modifier, (keycodes in parentheses):

shift       Shift_L (0x40), Shift_R (0x44)
lock        Caps_Lock (0x41)
control     Control_L (0x43), Control_R (0x46)
mod1        Alt_L (0x42), Alt_R (0x45)
mod2        Meta_L (0x3f)
mod3
mod4
mod5
```

Если у `mod1` не перечислены `Alt_L` и `Alt_R`, раскладка не применилась; снова выполните команду из `~/.xinitrc`.

Пакетный режим IDA

Все варианты IDA могут работать без участия пользователя. Обычно программа запускается, выполняет указанный сценарий IDC и завершается после него. Поведение задаётся параметрами командной строки.

Графическим версиям для пакетной работы консоль не требуется, поэтому их легко включать в сценарии автоматизации и оболочки; окна при этом не показываются. Консольные `idaw.exe` и `idaw64.exe` создают терминальное окно, которое закрывается после обработки. Вывод можно подавить перенаправлением в нулевое устройство:

```
C:\Program Files\Ida>idaw -B some_program.exe > NUL
```

Основные параметры:

• `-A` включает автономный режим и запрещает диалоги, требующие ответа. Исключением может стать лицензионное соглашение, если пользователь ещё ни разу его не принял.
• `-c` удаляет существующую базу для указанного файла и создаёт новую.
• `-S` задаёт сценарий, выполняемый при запуске. Для `myscript.idc` используется форма `-Smyscript.idc` без пробела. IDA ищет файл в `<IDADIR>/idc`. При установленном IDAPython можно указать и сценарий Python.
• `-B` включает стандартную пакетную обработку и равнозначен сочетанию `-A -c -Sanalysis.idc`. Поставляемый `analysis.idc` ждёт завершения автоматического анализа, сохраняет листинг в `.asm`, затем закрывает IDA с сохранением новой базы.

Ключевую роль играет `-S`: IDA завершится только тогда, когда это сделает указанный сценарий. Если в нём нет команды завершения, параметры лишь автоматизируют запуск. Сценарии IDC рассматриваются в главе 15.

Из-за ограничений TVision консольные версии Linux и OS X прежде требовали настоящий терминал TTY, что мешало перенаправлению вывода и фоновому запуску. Более новые варианты TVision распознают `TVHEADLESS`, позволяющую перенаправить стандартный вывод:

```
# TVHEADLESS=1 ./idal -B input_file.exe > /dev/null
```

Для полного отделения от терминала нужно также перенаправить стандартный ввод и поток ошибок. Дополнительные способы пакетного запуска, в том числе выполнение подключаемого модуля вместо единственного сценария, описаны в публикации Ильфака Гильфанова: <http://hexblog.com/2007/03/on_batch_analysis.html>.

Итоги

Графические версии IDA остаются наиболее полными, но консольные варианты и пакетная обработка дают большую свободу при создании сложных автоматизированных решений анализа.

На этом обзор основных возможностей IDA завершён. В следующих главах рассматриваются расширенные параметры настройки и дополнительные программы, улучшающие анализ двоичных файлов.

Часть III. Расширенное применение IDA

Глава 11. Настройка IDA

Поработав с IDA, пользователь обычно выбирает удобные параметры и хочет применять их по умолчанию к каждой новой базе. Одни изменения сохраняются между сеансами, другие приходится задавать заново. В этой главе рассматриваются файлы настройки, параметры меню, места хранения значений и различие между глобальными и относящимися к конкретной базе установками.

Файлы настройки

Большая часть стандартного поведения определяется файлами в `<IDADIR>/cfg`. Важное исключение - `<IDADIR>/plugins/plugins.cfg`, относящийся к подключаемым модулям и рассматриваемый в главе 17.

Многие файлы в основном каталоге предназначены для модулей процессоров и применяются только к определённым архитектурам. Главных файлов три:

• `ida.cfg` - общие параметры всех версий;
• `idagui.cfg` - параметры графического интерфейса;
• `idatui.cfg` - параметры текстового интерфейса.

Главный файл ida.cfg

В начале запуска IDA читает `ida.cfg`, чтобы назначить стандартные процессоры расширениям файлов и настроить использование памяти. После выбора архитектуры файл читается повторно для обработки остальных параметров. Эти настройки действуют независимо от интерфейса.

Среди важных общих параметров:

• `VPAGESIZE` - управление памятью;
• `CREATE_BACKUPS` - создание резервных копий;
• `GRAPH_VISUALIZER` - внешняя программа просмотра графов.

При работе с очень крупным входным файлом IDA иногда сообщает, что памяти для новой базы недостаточно. Обычно помогает увеличение `VPAGESIZE` и повторное открытие файла.

В `ida.cfg` также задаётся формат строк дизассемблированного кода и многие стандартные значения из `Options > General`:

• `OPCODE_BYTES` - число показываемых байтов машинного кода;
• `INDENTATION` - отступ инструкций;
• `SHOW_SP` - показ смещения указателя стека;
• `SHOW_XREFS` - наибольшее число ссылок у строки.

Отдельные параметры управляют видом строк в графическом режиме.

`MAX_NAMES_LENGTH` ограничивает длину имён программных адресов, но не стековых переменных. Стандартное значение 15 выбрано из-за ограничений некоторых ассемблеров. Если сгенерированный IDA листинг не планируется собирать снова, предел можно безопасно увеличить.

Множество `NameChars` определяет символы, допустимые в пользовательских именах. По умолчанию разрешены буквы, цифры и `_$?@`. Чтобы, например, применять точку, добавьте её сюда. Лучше не разрешать точку с запятой, двоеточие, запятую и пробел: они служат разделителями

частей строки листинга и создают неоднозначность.

Два параметра влияют на разбор заголовков Си:

• C_HEADER_PATH перечисляет каталоги поиска зависимостей #include. Стандартно там указан распространённый каталог Microsoft Visual Studio; для другого компилятора или нестандартного размещения путь нужно изменить.
• C_PREDEFINED_MACROS задаёт макросы препроцессора, которые IDA учитывает даже без встретившихся объявлений. Это частично помогает при отсутствии некоторых подключаемых заголовков.

Во второй половине ida.cfg находятся параметры модулей процессоров. Единственной документацией нередко служат комментарии в самом файле. Эти значения обычно становятся стандартными установками раздела Processor options в начальном диалоге загрузки.

В конце обработки IDA ищет <IDADIR>/cfg/idauser.cfg. В поставке его нет, но пользователь может создать файл самостоятельно. Он расширяет ida.cfg, а повторённые параметры переопределяют основные. Это безопаснее прямого редактирования и упрощает обновление: достаточно скопировать прежний idauser.cfg в новую установку.

Файл графического интерфейса idagui.cfg

Файл <IDADIR>/cfg/idagui.cfg примерно делится на три части:

• стандартное поведение графического интерфейса;
• сочетания клавиш;
• расширения файлов для диалога открытия.

Большинство параметров снабжено комментариями.

В графической версии Windows параметр HELPFILE указывает дополнительный файл справки, не заменяя основной. При заданном файле Ctrl+F1 ищет в нём тему, соответствующую слову под курсором, а при отсутствии совпадения открывает оглавление. Так можно подключить справочник по инструкциям x86, которых нет в основной справке IDA. Поддерживается только старый формат WinHelp .hlp, но не скомпилированная справка HTML .chm.

> **Примечание.** Windows Vista и более новые версии не поддерживают 32-разрядные файлы WinHelp без отдельного WinHlp32.exe.

IDA не записывает изменения базы непосредственно в исходный двоичный файл, но позволяет исправлять инструкции и данные внутри базы. Меню таких действий по умолчанию скрыто. Параметр DISPLAY_PATCH_SUBMENU показывает его как Edit > Patch Program; подробности приведены в главе 14.

Однострочное поле внизу рабочего пространства называется командной строкой IDA. DISPLAY_COMMAND_LINE управляет его видимостью. Если ввод коротких сценариев не нужен, скрытие освобождает немного места. Это не системная командная строка и команды операционной системы здесь не выполняются.

Раздел сочетаний связывает действия IDA с клавишами. Переназначение позволяет добавить быстрый вызов команды, выбрать более запоминающееся сочетание или устранить конфликт с операционной системой и терминалом.

Почти каждое действие меню или панели перечислено в этом разделе, но внутреннее имя не всегда совпадает с подписью интерфейса. Например, Jump > Jump to Problem соответствует JumpQ и сочетанию Ctrl+Q. Если комментария нет, найдите имя действия во встроенной справке:

результат обычно ведёт к описанию связанной команды меню.

Примеры назначений:

```
"Abort" = 0       // немедленно завершить IDA без сохранения
"Quit"  = "Alt-X" // завершить с сохранением
```

Нецитированное значение `0` означает отсутствие сочетания. Клавиши записываются строкой в кавычках.

Последняя часть `idagui.cfg` связывает описания типов файлов с расширениями и формирует список `Files of type` диалога открытия. Параметр `FILE_EXTENSIONS` содержит все ассоциации. Например:

```
CLASS_JAVA, "Файлы классов Java", "*.cla*;*.cls"
```

Три разделённых запятыми компонента - внутреннее имя, описание и шаблон. Допустимы подстановочные знаки и несколько шаблонов через точку с запятой.

Другой вид записи объединяет уже существующие категории:

```
EXE, "Исполняемые файлы", EXE_*
```

Здесь последний элемент не заключён в кавычки. Собственную ассоциацию можно определить так:

```
IDA_BOOK, "Файлы книги по IDA", "*.book"
```

Имя должно быть свободным. Одного добавления в `FILE_EXTENSIONS` недостаточно: чтобы категория появилась в диалоге, `IDA_BOOK` нужно также включить в `DEFAULT_FILE_FILTER`.

Последняя строка `idagui.cfg` подключает `<IDADIR>/cfg/idauserg.cfg`. Как и основной пользовательский файл, его следует создать самостоятельно и поместить туда переопределения вместо прямого изменения поставляемого файла.

Файл консольного интерфейса idatui.cfg

Для консольной версии аналогом служит `<IDADIR>/cfg/idatui.cfg`. Структура и синтаксис сочетаний почти те же, поэтому важны лишь отличия.

Параметров `DISPLAY_PATCH_SUBMENU` и `DISPLAY_COMMAND_LINE` здесь нет. Консольный диалог открытия намного проще, поэтому отсутствует и настройка ассоциаций файлов.

Зато имеются параметры только текстового режима. Например, `NOVICE` включает режим новичка, скрывающий часть сложных возможностей и почти все дополнительные окна.

Поскольку консольные пользователи особенно зависят от клавиатуры, IDA поддерживает макросы сочетаний. Примеры находятся в `idatui.cfg`, но свои определения лучше хранить в `<IDADIR>/cfg/idausert.cfg`, консольном аналоге `idauserg.cfg`:

```
MACRO "Alt-H"        // переход к метке start
{
    "G"
    's' 't' 'a' 'r' 't'
    "Enter"
}
```

После ключевого слова `MACRO` указывается вызывающее сочетание, а между фигурными скобками - последовательность имён клавиш или отдельных символов. Пример по `Alt+H` открывает диалог перехода клавишей `G`, посимвольно вводит `start` и нажимает `Enter`. Нельзя заменить отдельные символы строкой `"start"`: IDA воспримет её как имя клавиши и сообщит об ошибке.

Примечание. Макросы и режим новичка отсутствуют в графической версии IDA.

Если при разборе любого файла настройки обнаружена ошибка, IDA немедленно завершается с сообщением о её характере. Запустить программу снова можно только после исправления.

Дополнительные параметры IDA

Множество настроек доступно только через интерфейс. Форматирование отдельных строк рассматривалось в главе 7; остальные пункты находятся в меню `Options`.

Большинство изменений действует лишь в открытой базе и сохраняется в её файле при закрытии. Исключения, например `Options > Colors` и `Options > Font`, являются глобальными и применяются во всех следующих сеансах.

В Windows глобальные значения хранятся в реестре по ключу `HKEY_CURRENT_USER\Software\Hex-Rays\IDA`. В других системах используется файл собственного формата `$HOME/.idapro/ida.reg`.

Там же запоминаются диалоги, в которых пользователь отметил «больше не показывать». В Windows соответствующие записи находятся в `HKEY_CURRENT_USER\Software\Hex-Rays\IDA\Hidden Messages`. Чтобы вернуть скрытое сообщение, удалите его значение из этого раздела.

Цвета IDA

Почти любой элемент можно перекрасить через `Options > Colors`:

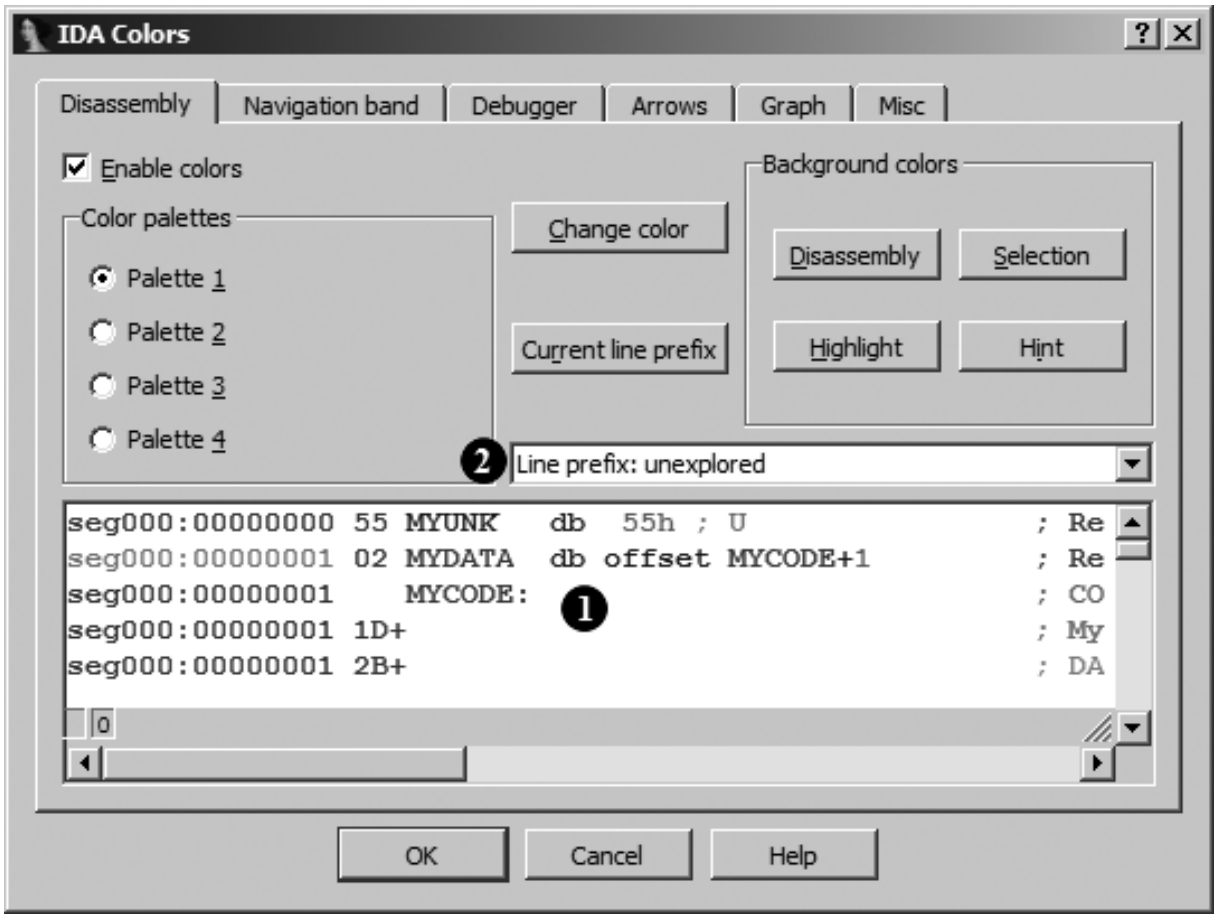


Рисунок 11-1. Диалог выбора цветов

Вкладка `Disassembly` управляет частями строк листинга. В области примера показаны все виды текста; при выборе элемента его тип появляется в отдельном поле, после чего кнопка `Change Color` назначает цвет.

Другие вкладки задают цвета полосы навигации, отладчика, стрелок переходов и графического представления. `Graph` отвечает за узлы, заголовки и рёбра; текст внутри узлов по-прежнему настраивается на `Disassembly`. Вкладка `Misc` управляет окном сообщений.

Панели инструментов

Графическая IDA содержит более двух десятков панелей. Обычно они закреплены под строкой меню. В `View > Toolbars` доступны две готовые схемы:

- `Basic mode` включает семь панелей;
- `Advanced mode` включает все.

Каждую панель можно отделить, перетащить и закрепить в другом месте либо полностью скрыть:

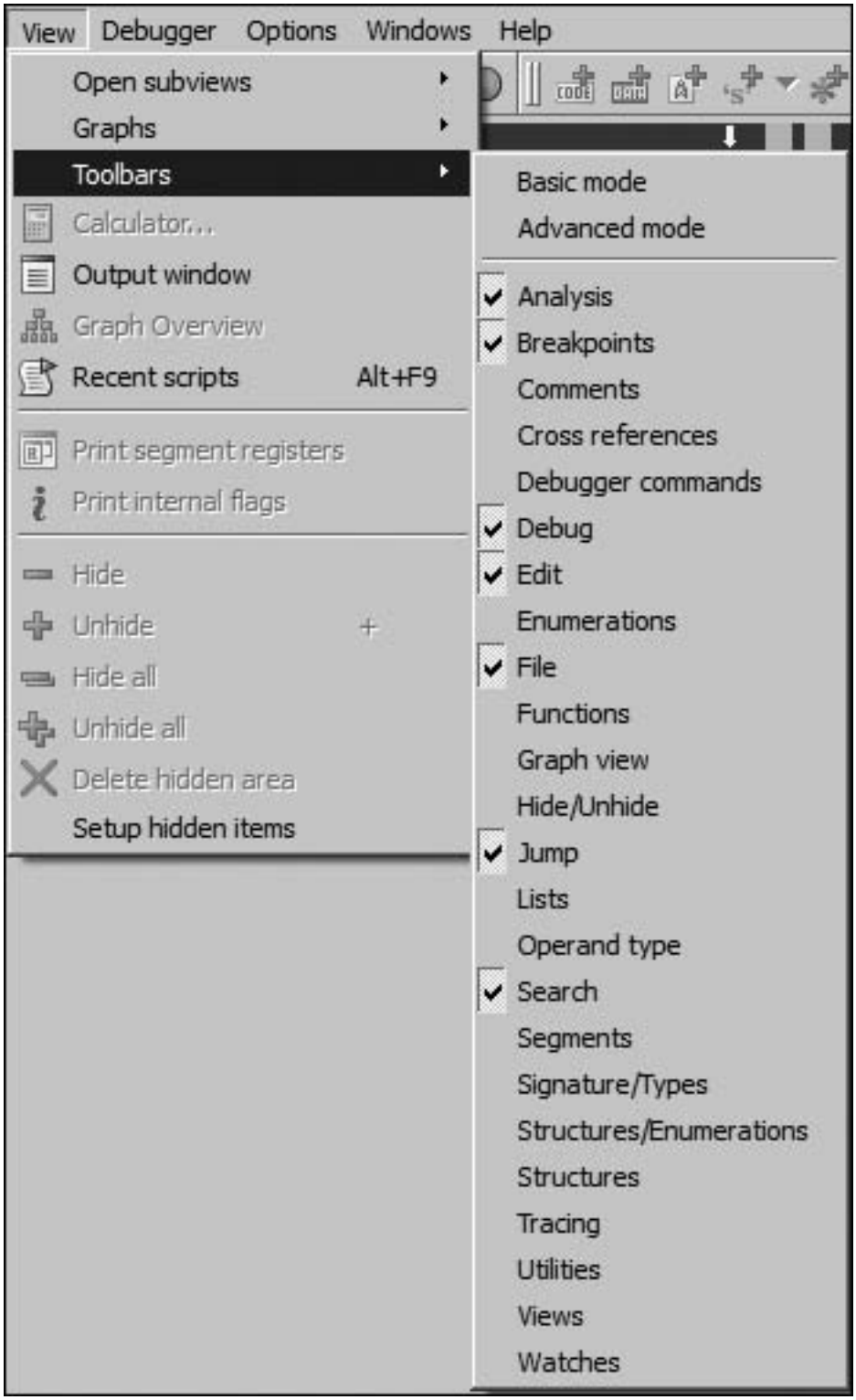


Рисунок 11-2. Меню настройки панелей инструментов

То же меню открывается правым щелчком по области закрепления. Отключение `Main toolbar` скрывает все панели и освобождает наибольшую площадь для листинга.

Расположение сохраняется в текущей базе. Поэтому при переходе к другой базе восстанавливается её собственная схема, а при открытии нового двоичного файла применяется текущая стандартная.

Чтобы сделать удачную раскладку стандартной, выберите `Windows > Save Desktop`:

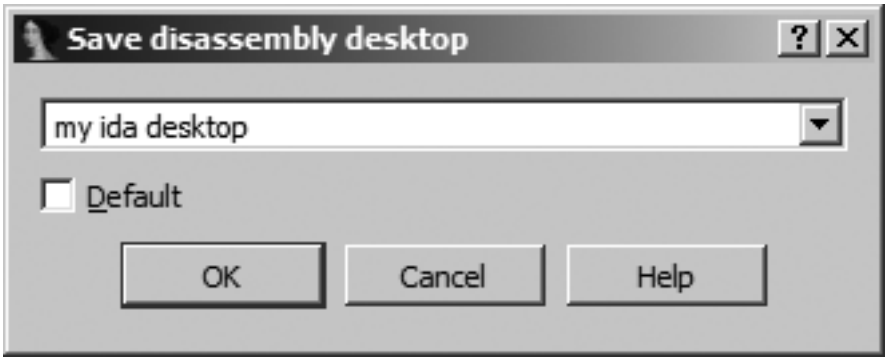


Рисунок 11-3. Диалог сохранения рабочего стола дизассемблера

Каждой схеме присваивается имя. Флажок `Default` делает текущую раскладку стандартной для новых баз и целью команды `Windows > Reset desktop`. Сохранённую схему можно восстановить через `Windows > Load Desktop`.

Несколько рабочих столов особенно удобны при смене числа, размера и разрешения мониторов, например у ноутбука в разных стыковочных станциях или при подключении проектора.

Итоги

Сначала стандартное поведение и расположение окон IDA обычно вполне устраивают. По мере освоения появляются личные предпочтения. Перечислить все параметры в одной главе невозможно, но теперь известны основные места их хранения и наиболее полезные точки настройки. Остальные возможности лучше изучать постепенно по мере практической необходимости.

Глава 12. Распознавание библиотек по сигнатурам FLIRT

Пора перейти от наиболее очевидных возможностей IDA к действиям после сообщения «Начальный автоматический анализ завершён». В этой главе рассматривается распознавание стандартных последовательностей: кода статически скомпонованных библиотек, процедур запуска и вспомогательных функций, добавленных компилятором.

При обратной разработке не следует тратить время на библиотечные функции, поведение которых проще узнать из справочной страницы, исходного кода или документации. Статическая компоновка стирает границу между приложением и библиотекой: всё объединяется в единый исполняемый файл. К счастью, IDA умеет отмечать известный код и оставлять исследователю уникальную часть программы.

Технология быстрого распознавания библиотек

FLIRT, то есть технология быстрого обнаружения и распознавания библиотек, объединяет методы IDA для выявления библиотечного кода. В её основе лежит сопоставление образцов: IDA быстро проверяет, совпадает ли дизассемблированная функция с одной из известных сигнатур.

Поставляемые файлы находятся в `<IDADIR>/sig`. Большинство относится к библиотекам распространённых компиляторов Windows, хотя имеются и варианты для других систем.

Формат сигнатур собственный: основная масса данных сжата и помещена под заголовок IDA. Имя файла не всегда ясно сообщает исходную библиотеку. Иногда внутри хранится комментарий, который можно увидеть среди первых строк извлечённого текста:

```
# strings sigfile | head -n 3
```

В самой IDA комментарии доступны двумя путями:

• `View > Open Subviews > Signatures` показывает сигнатуры, уже применённые к базе;
• `File > Load File > FLIRT Signature File` открывает список всех доступных файлов при ручной загрузке.

Применение сигнатур FLIRT

При первом открытии двоичного файла IDA применяет специальные стартовые сигнатуры к точке входа. Код запуска разных компиляторов достаточно своеобразен, поэтому совпадение помогает определить, каким компилятором создан файл.

main и _start

Точка входа - адрес первой выполняемой инструкции. Она не обязана совпадать с `main`. Способ передачи параметров командной строки задаётся форматом файла, а ожидания `main` - языком и средой выполнения. Между загрузчиком и `main` должен выполниться начальный код, который согласует эти требования. Именно его IDA считает точкой входа и обычно называет `_start`.

Тот же код выполняет подготовку среды. В C++ он вызывает конструкторы глобальных объектов до `main`, а завершающая обвязка после `main` вызывает их деструкторы перед прекращением процесса.

Если компилятор распознан, IDA загружает сигнатуры соответствующих библиотек и применяет их к остальному файлу. В поставке преобладают закрытые компиляторы, например Microsoft Visual C++ и Borland Delphi, потому что вместе с ними распространяется ограниченное число готовых двоичных библиотек.

У открытых компиляторов вроде GNU `gcc` вариантов гораздо больше. Каждая система и даже каждый выпуск может содержать собственную сборку стандартной библиотеки. Изменения исходного кода, версии компилятора и параметров оптимизации заметно меняют машинный код. Собрать, например, все варианты `libc.a` из всех выпусков Linux практически невозможно. Поэтому готовых сигнатур для открытых библиотек мало, но Hex-Rays предоставляет средства создания собственных.

Ручная загрузка требуется в нескольких случаях. IDA может определить компилятор, но не иметь нужной библиотеки. Тогда придётся обойтись без распознавания либо найти точную статическую библиотеку и создать сигнатуры. Иногда не распознаётся сам компилятор, особенно после обфускации процедур запуска. Сначала такой код нужно достаточно очистить, иначе совпадений ожидать нельзя.

Для ручного выбора выполните `File > Load File > FLIRT Signature File`:

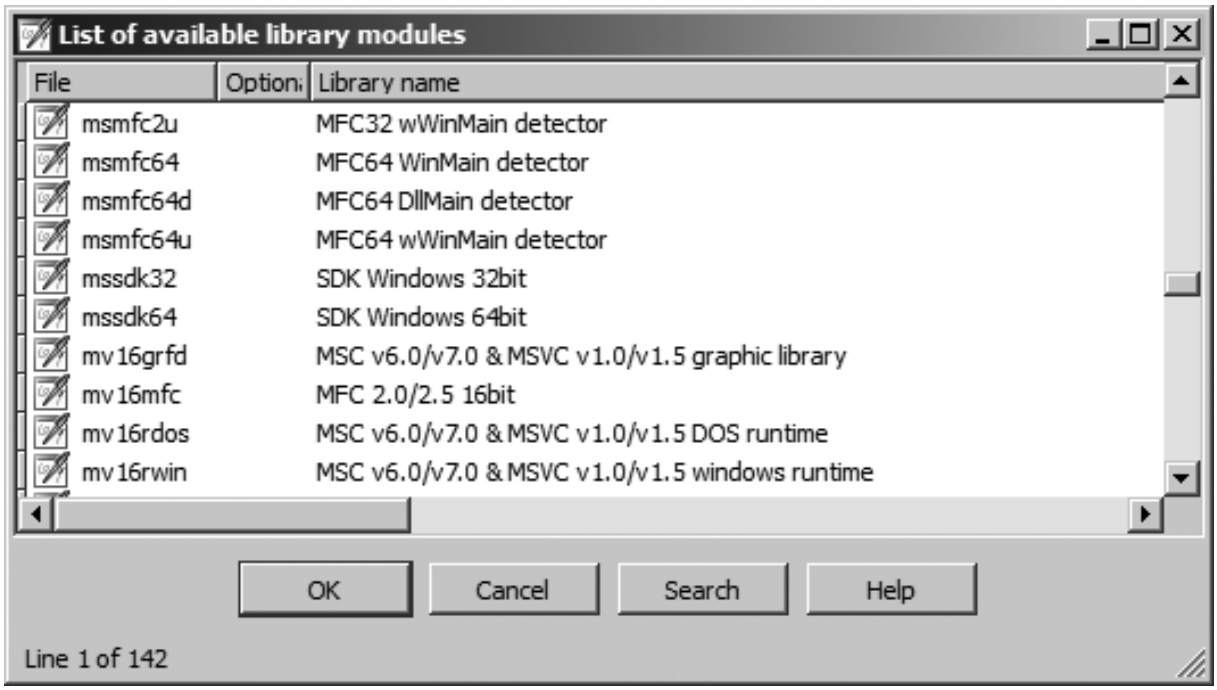


Рисунок 12-1. Выбор сигнатуры FLIRT

Столбец `File` содержит файлы `.sig` из `<IDADIR>/sig`. Другой каталог выбрать нельзя, поэтому собственные сигнатуры также копируются сюда. В `Library name` выводится встроенный комментарий; его точность зависит от автора файла.

После выбора IDA сравнивает сигнатуры со всеми функциями базы. За один раз загружается только один набор, поэтому для нескольких библиотек действие повторяется. Совпавшая функция помечается библиотечной и автоматически переименовывается.

> **Предупреждение.** Автоматически переименовываются только функции со стандартными фиктивными именами IDA. Если пользователь уже дал функции имя, позднее совпадение

сигнатуры его не заменит. Поэтому сигнатуры лучше применять в начале анализа.

В статически скомпонованном файле без удаления символов имена помогают ориентироваться, но сами по себе не доказывают IDA, что функция библиотечная. После удаления символов остаются сотни безликих имён. В обоих случаях для надёжной классификации нужны сигнатуры.

Без них полоса навигации выглядит так:

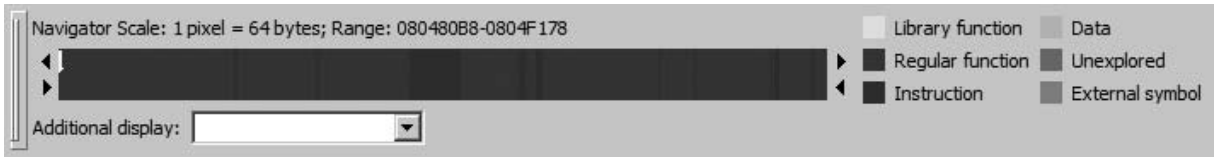


Рисунок 12-2. Статически скомпонованный файл без сигнатур

Ни одна функция не помечена библиотечной, поэтому исследователь рискует анализировать лишний код. После подходящего набора вид меняется:

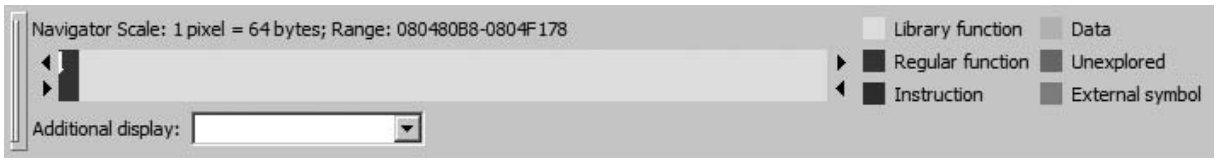


Рисунок 12-3. Статически скомпонованный файл с применёнными сигнатурами

Полоса навигации наглядно показывает успешность набора. При большом числе совпадений обширные области получают цвет библиотечного кода и правильные имена. В примере уникальный код приложения, вероятно, сосредоточен у левого края.

Нужно помнить два обстоятельства:

1. Сигнатуры полезны и в файле с сохранёнными символами, поскольку классифицируют функции, а не только переименовывают их.
2. Один статический файл может включать несколько библиотек, следовательно, потребуется несколько наборов.

Ниже показан двоичный файл, скомпонованный со стандартной библиотекой Си и криптографической OpenSSL, после первого набора:

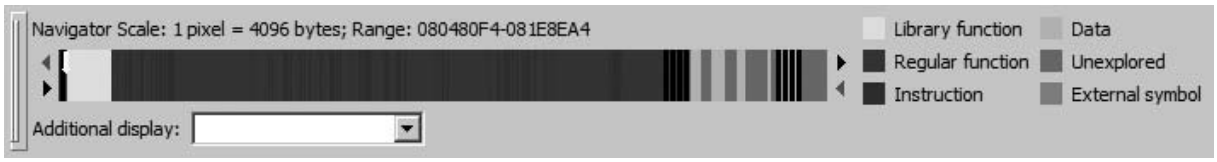


Рисунок 12-4. Статический файл после применения первого набора сигнатур

Светлая узкая полоса слева соответствует распознанной OpenSSL. Библиотеки при статической компоновке часто дописываются после кода приложения. Поэтому справа, вероятно, находятся другие библиотеки, а само приложение занимает ещё более узкую область слева.

После сигнатур `libc`, `libcrypto`, `libkrb5`, `libresolv` и других получается:

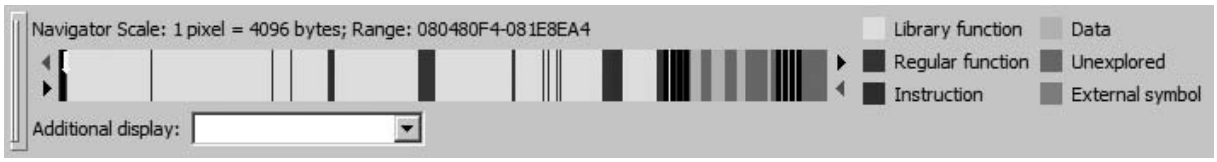


Рисунок 12-5. Статический файл после применения нескольких наборов

Одни наборы выбраны по строкам внутри файла, другие - по связи с уже найденными библиотеками. В середине остаётся тёмная область и ещё одна у левого края. Дальнейший анализ показывает, что средняя принадлежит нераспознанной библиотеке, а левая - приложению.

Создание сигнатур FLIRT

Поставлять сигнатуры для каждой существующей статической библиотеки невозможно. Поэтому Hex-Rays распространяет набор FLAIR, предназначенный для получения библиотек и создания описаний для распознавания. Он доступен владельцам IDA на носителе или странице загрузки.

FLAIR распространяется архивом ZIP. Лучше распаковывать его в отдельный каталог `flair`, поскольку внутри нет общего верхнего каталога. Следует брать самый новый выпуск, номер которого не превышает версию IDA.

В комплект входят справочные тексты:

• `readme.txt` - общий обзор процесса;
• `plb.txt` - работа с анализатором статических библиотек `plb`;
• `pat.txt` - формат файлов образцов;
• `sigmake.txt` - создание `.sig` из образцов программой `sigmake`.

Каталог `bin` содержит исполняемые программы, а `startup` - готовые образцы процедур запуска разных компиляторов и форматов PE, ELF и других. До версии 6.1 средства FLAIR выпускались только для Windows, но полученные сигнатуры работают во всех вариантах IDA.

Общая последовательность

На первый взгляд процесс состоит из четырёх простых шагов:

1. Получить точную статическую библиотеку.
2. Создать из неё файл образцов одним из анализаторов FLAIR.
3. Обработать образцы программой `sigmake` и получить сигнатуру.
4. Скопировать новый файл в `<IDADIR>/sig`.

На практике прост только последний шаг.

Поиск и получение статической библиотеки

Сначала нужно понять, какая именно библиотека использована. Если символы не удалены, имена функций позволяют найти вероятного поставщика по документации и сети.

В очищенном файле помогают характерные строки. Например, следующая почти однозначно выдаёт библиотеку и выпуск:

```
OpenSSL 1.0.0b-fips 16 Nov 2010
```

Уведомления об авторских правах и тексты ошибок тоже бывают достаточно уникальными. При использовании консольной `strings` задавайте `-a`, чтобы просмотреть весь файл.

Для открытой библиотеки исходный код обычно доступен, но непосредственно создать из него сигнатуры нельзя. Можно собрать собственную копию, однако отличия среды, версии компилятора и параметров часто сделают совпадения неточными.

Лучше определить точную систему, выпуск и дистрибутив, где создан исследуемый файл, а затем взять библиотеки с идентично настроенной машины. Первичную подсказку даёт программа `file`:

```
$ file sample_file_1
sample_file_1: ELF 32-bit LSB executable, Intel 80386, version 1 (FreeBSD),
statically linked, for FreeBSD 8.0 (800107), stripped
```

Здесь логично начать с `libc.a` системы FreeBSD 8.0. Следующий результат менее определён:

```
$ file sample_file_2
sample_file_2: ELF 32-bit LSB executable, Intel 80386, version 1 (GNU/Linux),
statically linked, for GNU/Linux 2.6.32, stripped
```

Здесь известно лишь семейство Linux, что при множестве дистрибутивов мало помогает. Поиск строк может сузить круг:

```
GCC: (GNU) 4.5.1 20100924 (Red Hat 4.5.1-4)
```

Теперь стоит искать среди Red Hat и производных систем, поставлявшихся с `gcc` 4.5.1. Такие метки GCC часто переживают удаление символов.

Программа `file` не всегда даёт исчерпывающий ответ:

```
$ file sample_file_3
sample_file_3: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),
dynamically linked (uses shared libs), stripped
```

Этот файл взят из Solaris 10 для x86, но вывод слишком общий. Здесь снова могут помочь строки.

Создание файлов образцов

После получения библиотек для каждой создаётся текстовый файл `.pat`. Нужно выбрать анализатор FLAIR, соответствующий формату:

- `plb.exe` или `plb` - библиотеки OMF, часто у Borland;
- `pcf.exe` или `pcf` - библиотеки COFF, часто у Microsoft;
- `pelf.exe` или `pelf` - библиотеки ELF во многих системах Unix;
- `ppsx.exe` или `ppsx` - библиотеки Sony PlayStation PSX;
- `ptmobj.exe` или `ptmobj` - библиотеки TriMedia;
- `pomf166.exe` или `pomf166` - объектные файлы Keil OMF 166.

Укажите анализатор, исходную библиотеку и выходной файл. Для `libc.a` из FreeBSD 8.0:

```
$ ./pelf libc.a libc_FreeBSD80.pat
libc.a: skipped 1, total 1089
```

Сообщение означает, что одна функция пропущена, а 1089 образцов созданы. У каждого анализатора собственные параметры, которые выводятся при запуске без аргументов. Подробности `plb` приведены в `plb.txt`; многие его ключи поддерживают и другие программы. Часто достаточно двух имён файлов.

Файл образцов содержит по одной строке на функцию:

```
57568B7C240C8B742410FC8B4C2414C1E902F3A775108B4C241483E103F3A675 1E A55D 003E :0000 _memcmp
0FBC442404740340C39031C0C3..................................... 00 0000 000D :0000 _ffs
57538B7C240C8B4C2410FC31C083F90F7E1B89FAF7DA83E20389D389D1F3 12 9E31 0032 :0000 _bzero
```

Формат подробно описан в `pat.txt`. В начале записывается до 32 первых байтов функции. Байты, способные измениться из-за перемещения, заменяются двумя точками. Если функция короче 32 байтов, точками строка дополняется до 64 символов.

Далее идут дополнительные признаки: 16-разрядная контрольная сумма части функции, её длина и список символов, на которые она ссылается. Длинные функции с множеством ссылок образуют

очень крупные записи; в приведённом файле отдельные строки превышали 20 000 символов.

Существуют сторонние средства, создающие образцы из готовой базы IDA. Например, подключаемый модуль IDB_2_PAT Дж. К. Робертса формирует их для одной или нескольких функций. Это полезно, когда похожий код встретится в других базах, а исходной библиотеки нет.

Создание файлов сигнатур

Из .pat нужно получить пригодный для IDA файл .sig. Его закрытый двоичный формат сжимает сведения и ускоряет сопоставление. Разделение этапов позволяет применять сторонние генераторы образцов.

В простейшем случае используется sigmake:

```
$ ./sigmake libssl.pat libssl.sig
```

Если ошибок нет, результат копируется в <IDADIR>/sig. Однако с первого раза это удаётся редко.

> **Примечание.** В sigmake.txt рекомендуется соглашение имён MS-DOS 8.3. Это не строгое требование, но в диалоге выбора у длинного основного имени будут видны только первые восемь символов.

Главная трудность - коллизии, когда две функции имеют одинаковый образец. Тогда невозможно однозначно выбрать имя. sigmake требует, чтобы каждая сигнатура соответствовала не более чем одному имени; при неоднозначности она не создаёт .sig, а формирует файл исключений .exc.

Обычный первый запуск выглядит так:

```
$ ./sigmake libc_FreeBSD80.pat libc_FreeBSD80.sig
libc_FreeBSD80.sig: modules/leaves: 1088/1024, COLLISIONS: 10
See the documentation to learn how to resolve collisions.
```

Рядом появляется libc_FreeBSD80.exc. При каждом запуске sigmake ищет файл исключений для указанных образцов. Если его нет и встречаются коллизии, создаётся текст с конфликтующими функциями.

В начале нового .exc находятся инструкции:

```
;--------- (удалите эти строки, чтобы sigmake прочитала файл)
; добавьте '+' в начало строки, чтобы выбрать модуль
; добавьте '-', если не уверены в выборе
; ничего не добавляйте, чтобы исключить все модули
```

Все четыре строки с точкой с запятой нужно удалить, иначе последующий разбор завершится ошибкой. Далее перечислены группы совпадений:

```
_index   00 0000 538B4424088A4C240C908A1838D974074084DB75F531C05BC3..............
_strchr  00 0000 538B4424088A4C240C908A1838D974074084DB75F531C05BC3..............
_rindex  00 0000 538B5424088A4C240C31C0908A1A38D9750289D04284DB75F35BC3..........
_strrchr 00 0000 538B5424088A4C240C31C0908A1A38D9750289D04284DB75F35BC3..........
_flsl    01 EF04 5531D289E58B450885C0741183F801B201740AD1E883C20183F80175F65D89D0
_fls     01 EF04 5531D289E58B450885C0741183F801B201740AD1E883C20183F80175F65D89D0
```

Здесь index неотличима от strchr, rindex от strrchr, а flsl от fls. Это закономерно: функции в парах выполняют одинаковые или почти одинаковые действия.

Для каждой группы разрешено выбрать не более одной функции:

- + перед именем означает применять его при совпадении;
- - означает добавлять имя только как комментарий;
- отсутствие знака исключает имя.

Один допустимый вариант:

```
+_index   00 0000 538B4424088A4C240C908A1838D974074084DB75F531C05BC3..............
_strchr  00 0000 538B4424088A4C240C908A1838D974074084DB75F531C05BC3..............
_rindex  00 0000 538B5424088A4C240C31C0908A1A38D9750289D04284DB75F35BC3..........
_strrchr 00 0000 538B5424088A4C240C31C0908A1A38D9750289D04284DB75F35BC3..........
_flsl    01 EF04 5531D289E58B450885C0741183F801B201740AD1E883C20183F80175F65D89D0
-_fls     01 EF04 5531D289E58B450885C0741183F801B201740AD1E883C20183F80175F65D89D0
```

Первому образцу назначается имя `index`, второй полностью исключается, а при третьем в базу добавляется комментарий о `fls`.

Правила разрешения коллизий:

1. Для минимального решения достаточно удалить четыре начальные строки комментариев.
2. Никогда не ставьте + или - более чем перед одной функцией группы.
3. Если группа содержит единственную функцию, оставьте её без знака.
4. После повторной неудачи `sigmake` дописывает в существующий `.exc` новые сведения и комментарии. Перед следующим запуском лишнее нужно удалить, а прежнее решение исправить.

Сохраните файл и повторите исходную команду:

```
$ ./sigmake libc_FreeBSD80.pat libc_FreeBSD80.sig
```

Отсутствие ошибок и появление `.sig` означают успех. После копирования в `<IDADIR>/sig` набор доступен через `File > Load File > FLIRT Signature File`.

У генераторов образцов и `sigmake` много дополнительных параметров, описанных в `plb.txt` и `sigmake.txt`. Особенно полезен `-n`, встраивающий понятное описание для диалога выбора:

```
$ ./sigmake -n"Стандартная библиотека Си FreeBSD 8.0" libc_FreeBSD80.pat libc_FreeBSD80.sig
```

Имя можно также задать директивой в `.exc`, но файл исключений нужен не всегда, поэтому параметр командной строки удобнее.

Стартовые сигнатуры

Стартовые сигнатуры применяются при первой загрузке, чтобы определить компилятор. После успешного распознавания IDA автоматически подключает связанные библиотечные наборы.

Поскольку компилятор ещё неизвестен, стартовые образцы группируются по формату входного файла. Для Windows PE применяются сигнатуры процедур запуска именно PE.

`sigmake` обрабатывает образцы входных процедур разных компиляторов и объединяет их в общий файл для одного формата. Каталог `startup` комплекта FLAIR содержит применяемые IDA образцы и сценарий `startup.bat`, который строит из них сигнатуры.

Для PE там находятся файлы `pe_*.pat`: например, `pe_vc.pat` для Visual Studio и `pe_gcc.pat` для Cygwin и `gcc`. Дополнительные образцы PE нужно добавить в существующий файл либо создать новый с префиксом `pe_`, чтобы сценарий обнаружил его.

Формат стартовой строки немного отличается от библиотечного: она может связывать совпадение с дополнительными наборами, которые следует применить. Кроме примеров в `startup`, этот формат в справочных текстах FLAIR не описан.

Итоги

Автоматическое распознавание библиотечного кода резко сокращает время анализа статически скомпонованных файлов. FLIRT выполняет сопоставление, а FLAIR позволяет расширять систему собственными сигнатурами, созданными из имеющихся статических библиотек. Для исследователя, регулярно работающего со статической компоновкой, владение этим процессом обязательно.

Глава 13. Расширение знаний IDA

Качественный дизассемблированный текст - не просто мнемоники и операнды, полученные из байтов. Его нужно дополнять сведениями программных интерфейсов: объявлениями функций, стандартными типами и комментариями. В главе 8 рассматривались структуры данных. Теперь речь пойдёт о программах `idsutils` и `loadint`, расширяющих знания IDA. Они доступны владельцам продукта на носителе и сайте Hex-Rays.

Дополнение сведений о функциях

IDA получает информацию о функциях из библиотек типов `.til` и файлов IDS `.ids`. При начальном анализе они повышают точность и читаемость: добавляют типы и имена параметров, соглашения вызова и комментарии библиотечных функций.

Библиотеки `.til` хранят не только сложные структуры, но и объявления функций. Эти сведения используются несколькими способами.

Во-первых, при динамической компоновке IDA не знает соглашения вызова внешней функции. Совпавшее объявление сообщает, кто очищает стек и как корректировать указатель. Это особенно важно для `stdcall`, где очистку выполняет вызываемая функция.

Во-вторых, IDA подписывает значения, помещаемые в стек перед вызовом. Полнота комментариев зависит от объявления. Обе записи ниже допустимы, но вторая содержит имена формальных параметров:

```
LSTATUS _stdcall RegOpenKey(HKEY, LPCTSTR, PHKEY);
LSTATUS _stdcall RegOpenKey(HKEY hKey, LPCTSTR lpSubKey, PHKEY phkResult);
```

В стандартных библиотеках типов описано множество распространённых программных интерфейсов, в том числе значительная часть Windows. Вызов `RegOpenKey` может выглядеть так:

```
.text:00401006        lea     eax, [ebp+hKey]
.text:00401009        push    eax             ; phkResult
.text:0040100A        push    offset SubKey   ; "Software\Hex-Rays\IDA"
.text:0040100F        push    80000001h        ; hKey
.text:00401014        call    ds:RegOpenKeyA
.text:0040101A        mov     [ebp+var_8], eax
```

IDA добавила справа имена параметров. Если они присутствуют в объявлении, она также пытается переименовать соответствующие переменные: здесь появились локальная `hKey` и глобальная `SubKey`. Без формальных имён комментарии содержали бы только типы.

Для `lpSubKey` имя параметра не выводится, потому что операнд указывает на глобальную строку и место занято её повторяемым комментарием. IDA также знает, что `RegOpenKey` использует `stdcall`, и правильно учитывает восстановление стека.

Само объявление показывается у таблицы импорта:

```
.idata:0040A000 ; LSTATUS __stdcall RegOpenKeyA(HKEY hKey, LPCSTR lpSubKey, PHKEY phkResult)
.idata:0040A000                 extrn RegOpenKeyA:dword ; CODE XREF: _main+14 p
.idata:0040A000                                         ; DATA XREF: _main+14 r
```

Комментарий получен из `.til` с описанием программного интерфейса Windows.

Собственные объявления полезны, когда программа динамически или статически связана с неизвестной IDA библиотекой, например графической либо криптографической OpenSSL. Тогда IDA сможет автоматически размечать вызовы.

При наличии заголовков их можно разобрать через `File > Load File > Parse C Header File`, добавив объявления в локальную `.til`. Программа `tilib` создаёт отдельную глобальную библиотеку, которую затем копируют в `<IDADIR>/til`.

Но исходные тексты доступны не всегда. Для получения хотя бы части сведений непосредственно из библиотеки служит комплект IDS из трёх программ, создающих `.ids`.

Ручное указание очищаемых байтов

Неизвестная импортированная функция `stdcall` нарушает анализ указателя стека. IDA должна знать не только само соглашение, но и точное число байтов, снимаемых перед возвратом.

При отсутствии `.til` и `.ids` IDA пытается вывести это математически симплекс-методом, но пользователь может задать значение вручную. Перейдите к записи импорта и выберите `Edit > Functions > Edit Function` либо нажмите `Alt+P`:

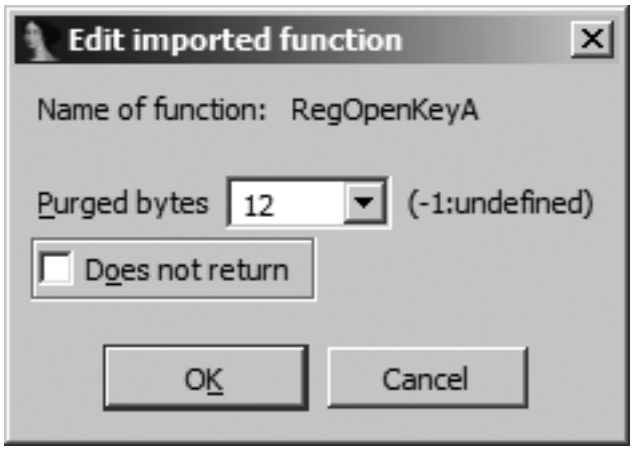


Рисунок 13-1. Редактирование импортированной функции

Диалог сокращён, поскольку тела импортированной функции в базе нет, а значит, нельзя исследовать её кадр и получить прямое доказательство соглашения. Значение `-1` в `Purged bytes` означает отсутствие сведений. Введите правильное число, и IDA учтёт его во всех местах вызова. Если поведение уже известно из типов, поле заполнено заранее; результат симплекс-анализа сюда не записывается.

Файлы IDS

Файл `.ids` описывает содержимое совместно используемой библиотеки, перечисляя экспортируемые функции. Для каждой могут быть указаны:

• имя;
• порядковый номер;
• применение `stdcall`;
• число очищаемых байтов стека;
• необязательный комментарий для мест обращения.

Фактически `.ids` является сжатым текстовым `.idt`.

При загрузке исполняемого файла IDA определяет его зависимости и ищет соответствующие описания в иерархии `<IDADIR>/ids`. Эти файлы не обязаны содержать полные сигнатуры параметров, поэтому сами по себе не всегда обеспечивают анализ аргументов. Но правильные

сведения о соглашении и очистке стека позволяют точно отслеживать SP.

Если библиотека экспортирует декорированные имена, IDA иногда восстанавливает сигнатуру из них. В целом `.til` полезнее для разметки вызовов, но для её создания нужны заголовки.

Создание IDS

Комплект `idsutils` включает два анализатора:

• `dll2idt` извлекает сведения из DLL Windows;
• `ar2idt` обрабатывает библиотеки формата `ar`.

На выходе создаётся `.idt` с одной строкой на экспорт, связывающей порядковый номер с именем. Основной синтаксис:

1. Строка экспорта начинается с положительного порядкового номера.
2. После пробела идёт `Name=имя`, например `Name=RegOpenKeyA`.
3. Особый номер 0 задаёт имя самой библиотеки:

```
0 Name=advapi32.dll
```

4. Необязательная директива `Pascal` указывает `stdcall` и число очищаемых байтов:

```
483 Name=RegOpenKeyA Pascal=12
```

5. `Comment` задаёт комментарий для мест обращения:

```
483 Name=RegOpenKeyA Pascal=12 Comment=Открыть раздел реестра
```

Дополнительные директивы описаны в `readme.txt` набора.

Сначала получите библиотеку и обработайте подходящей программой. Для `ssleay32.dll` из OpenSSL:

```
$ ./dll2idt.exe ssleay32.dll
Convert DLL to IDT file. Copyright 1997 by Yury Haron. Version 1.5
File: ssleay32.dll ... ok
```

Создаётся `SSLEAY32.idt`; регистр имени берётся из самой DLL. Начало файла:

```
ALIGNMENT 4
;DECLARATION
;
0 Name=SSLEAY32.dll
;
121 Name=BIO_f_ssl
173 Name=BIO_new_buffer_ssl_connect
122 Name=BIO_new_ssl
174 Name=BIO_new_ssl_connect
124 Name=BIO_ssl_copy_session_id
```

Анализатор не может определить `stdcall` и число очищаемых байтов. Директивы `Pascal` и `Comment` при необходимости добавляются вручную до сжатия.

Затем примените `zipids` и скопируйте результат:

```
$ ./zipids.exe SSLEAY32.idt
File: SSLEAY32.idt ... {219 entries [0/0/0]} packed
$ cp SSLEAY32.ids ../Ida/ids
```

Теперь IDA автоматически загружает описание для программы, связанной с `ssleay32.dll`. Без копирования файл можно подключить вручную через `File > Load File > IDS File`.

Связывание IDS с другими библиотеками

Файлы `.ids` можно связать с определёнными `.sig` и `.til`. Для этого используется текстовый `<IDADIR>/ida/idsnames`.

Он решает три задачи.

Сопоставление имени совместно используемой библиотеки с IDS. Это нужно, когда имя не укладывается в формат MS-DOS 8.3:

```
libc.so.6     libc.ids          +
```

Автоматическая загрузка TIL вместе с IDS. Следующая строка подключает `openssl.til` при загрузке `SSLEAY32.ids`:

```
SSLEAY32.ids  SSLEAY32.ids      +   openssl.til
```

Автоматическая загрузка IDS вместе с FLIRT. Здесь применение `libssl.sig` подключает `SSLEAY32.ids`:

```
libssl.sig    SSLEAY32.ids      +
```

Точный синтаксис приведён в самом `idsnames`. В главе 15 будет показан сценарный способ создания более содержательных `.idt` с использованием анализа функций IDA.

Дополнение стандартных комментариев программой loadint

Автоматические комментарии IDA кратко объясняют инструкции:

```
.text:08048654    lea     ecx, [esp+arg_0] ; загрузить эффективный адрес
.text:08048658    and     esp, 0FFFFFFF0h  ; логическое И
```

Их источник - `<IDADIR>/ida.int`, где записи сгруппированы сначала по процессору, затем по инструкции. При включённых автоматических комментариях IDA ищет подходящий текст и выводит справа.

Комплект `loadint` позволяет изменять и добавлять записи. В него входят документация и многочисленные файлы `.cmt` для процессорных модулей. Например, x86 соответствует `pc.cmt`.

Чтобы изменить комментарии:

1. Найдите файл нужного процессора.
2. Отредактируйте текст.
3. Соберите новую базу `ida.int`.
4. Скопируйте её в главный каталог IDA и перезапустите программу.

Команда сборки:

```
$ ./loadint comment.cmt ida.int
Comment base loader. Version 2.04. Copyright (c) 1991-2011 Hex-Rays

17566 cases, 17033 strings, total length: 580575
```

Можно не только менять текст, но и включать отключённые записи. Например, в `pc.cmt` обычная инструкция `mov` по умолчанию закомментирована, чтобы не перегружать листинг:

```
NN_ltr:                 "Загрузить регистр задачи"
//NN_mov:               "Переместить данные"
NN_movsp:               "Переместить в специальный регистр или из него"
```

Удалите `//` у средней строки и пересоберите базу, чтобы комментарий показывался для `mov`.

Программе требуется `ida.hlp`. Если файл не найден, появляется ошибка:

```
$ ./loadint comment.cmt ida.int
Comment base loader. Version 2.04. Copyright (c) 1991-2011 Hex-Rays
Can't initialize help system.
File name: 'ida.hlp', Reason: can't find file (take it from IDA distribution).
```

Скопируйте `ida.hlp` в каталог `loadint` либо укажите каталог IDA параметром `-n`:

```
$ ./loadint -n <IDADIR> comment.cmt ida.int
```

`comment.cmt` является главным входным файлом и связывает типы процессоров с файлами комментариев. Каждый процессорный `.cmt` сопоставляет конкретные инструкции с текстом. Система опирается на перечисления в стиле Си: типы процессоров определены в `comment.cmt`, а все возможные инструкции - в `allins.hpp`.

Для совершенно новой архитектуры работа сложнее и связана с созданием процессорного модуля:

1. В `allins.hpp` создайте набор констант, по одной для каждой инструкции.
2. Создайте файл комментариев, сопоставляющий константы с текстом.
3. Определите константу типа процессора и свяжите её с файлом в `comment.cmt`.
4. Запустите `loadint`, чтобы построить новую базу.

Итоги

`idsutils` и `loadint` становятся особенно ценными за пределами типовых сценариев. Небольшие затраты на один `.ids` или `.til` могут сэкономить много часов во всех будущих проектах с той же библиотекой. IDA не может поставляться с описанием каждой существующей библиотеки, поэтому эти средства позволяют самостоятельно заполнять пробелы.

Глава 14. Исправление двоичных файлов и другие ограничения IDA

Новички часто спрашивают: «Как исправлять двоичные файлы с помощью IDA?» Короткий ответ: никак. IDA предназначена для понимания поведения программы и получения наилучшего дизассемблированного текста, а не для удобного изменения исследуемого файла.

Но в меню есть `Edit > Patch Program` и `File > Produce File > Create EXE File`, поэтому ответ требует пояснения. В этой главе рассматриваются реальные возможности IDA и способы использовать её хотя бы для подготовки исправлений.

Меню Patch Program

В графической версии это скрытая возможность, включаемая в `idagui.cfg`; в консольной она доступна по умолчанию.

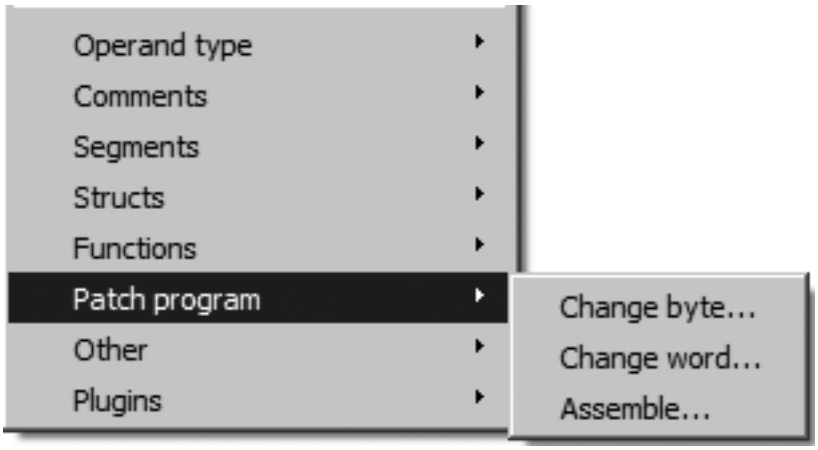


Рисунок 14-1. Подменю исправления программы

Несмотря на названия, три команды изменяют только базу IDA. После создания базы программа больше не обращается к исходному файлу. Точнее было бы назвать меню «Исправить базу».

Тем не менее оно позволяет увидеть результат предполагаемых изменений. Позднее эти сведения можно экспортировать и применить к оригиналу другим средством.

Изменение отдельных байтов базы

Команда `Edit > Patch Program > Change Byte` редактирует один или несколько байтов:

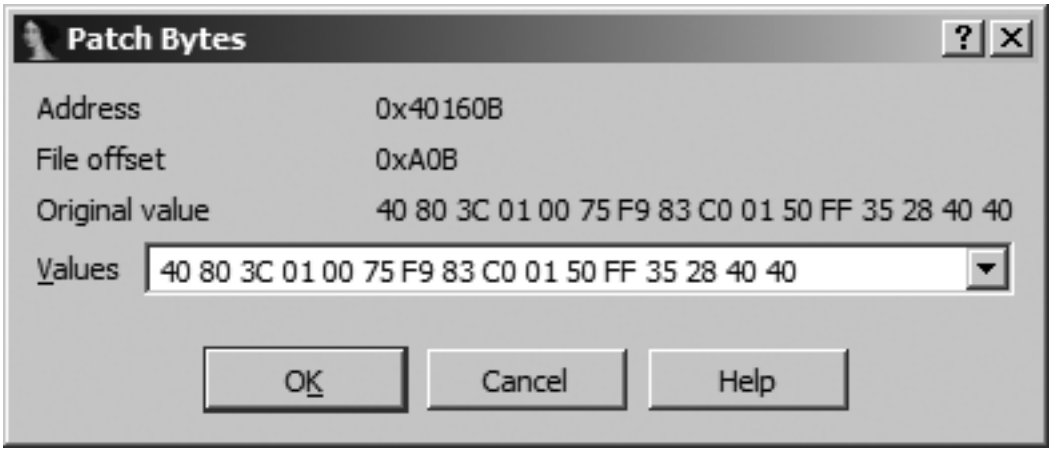


Рисунок 14-2. Диалог исправления байтов

Показываются 16 значений начиная с курсора. Чтобы изменить более дальний байт, закройте диалог, переставьте курсор и откройте снова.

В диалоге указаны виртуальный адрес и файловое смещение. Последнее показывает шестнадцатеричную позицию байта в исходном файле и пригодится при внешнем исправлении. Поле `Original value` всегда хранит первоначальное значение независимо от числа изменений. Автоматического отката нет, хотя его можно реализовать сценарием.

Начиная с IDA 5.5 удобнее пользоваться встроенным редактированием в шестнадцатеричном окне.

Изменение слова

`Patch Word` меняет одно двухбайтовое слово:



Рисунок 14-3. Диалог исправления слова

Значение показывается в естественном порядке байтов процессора: у x86 младший байт идёт первым, у MIPS часто старший. Это нужно учитывать при вводе. Виртуальный адрес, файловое смещение и исходное значение работают как в предыдущем диалоге. На практике шестнадцатеричный редактор снова удобнее.

Диалог ассемблирования

Самая интересная команда - `Edit > Patch Program > Assemble`. Она доступна только у процессорных модулей со встроенным ассемблером. Модуль x86 его поддерживает, MIPS - нет; в последнем случае IDA сообщает, что ассемблирование не поддерживается.

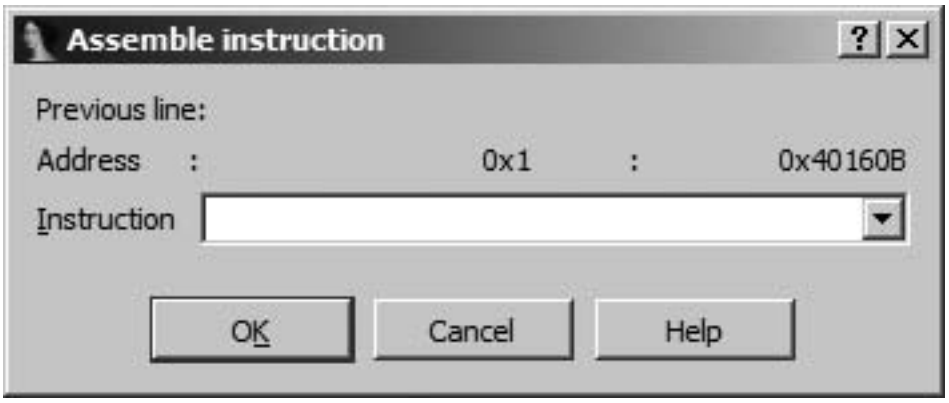


Рисунок 14-4. Диалог ассемблирования инструкции

Введите одну инструкцию. Синтаксис x86 совпадает с листингом IDA. После `OK` или `Enter` инструкция собирается, а байты записываются в базу с адреса из поля `Address`. Допустимы существующие символические имена:

```
mov [ebp+var_4], eax
call sub_401896
```

IDA разрешает ссылки самостоятельно. Диалог остаётся открытым на адресе сразу после новой инструкции и показывает предыдущий ввод.

Нужно внимательно следить за длиной. Если новая инструкция короче старой, остаются лишние байты; их можно заполнить nop. Если длиннее, она перезаписывает начало следующих инструкций. Встроенный ассемблер похож на редактор с постоянно включённой заменой: вставить место без перезаписи нельзя.

Поэтому возможности годятся для небольших исправлений, помещающихся в существующее пространство. Для крупного дополнения нужно найти выделенную, но неиспользуемую область, обычно заполнение между разделами.

В PE разделы на диске часто начинаются по границе 512 байтов. Если размер не кратен 512, компилятор дополняет его нулями:

```
.text:0040963E ; [00000006 BYTES: COLLAPSED FUNCTION RtlUnwind. PRESS KEYPAD "+" TO EXPAND]
.text:00409644                 align 200h
.text:00409644 _text           ends
.text:00409644
.idata:0040A000 ; Раздел 2. (виртуальный адрес 0000A000)
```

`align 200h` показывает заполнение с `.text:00409644` до следующей границы, `.text:00409800`. Здесь доступно 444 байта: `0x409800 - 0x409644 = 0x1BC`. Можно перенаправить функцию в эту область, выполнить вставленный код и вернуться.

Следующий раздел памяти начинается только по адресу `.idata:0040A000`, поскольку виртуальные разделы выравниваются по 4 Кбайт. Диапазон `00409800-0040A000` теоретически даёт ещё 2048 байтов, но в файле им ничего не соответствует. Для его использования пришлось бы вставить блок на диске, увеличить размер `.text` в заголовке PE и сдвинуть файловые позиции `.idata` и всех последующих разделов. Это возможно только при хорошем знании формата PE и аккуратном пересчёте заголовков.

Выходные файлы IDA и подготовка исправлений

Меню `File > Produce File` предлагает MAP, ASM, INC, LST, EXE, DIF и HTML. Их возможности различаются.

Файлы MAP

`.map` описывает общую схему двоичного файла: разделы и символы внутри них. При создании выбираются имя и категории экспортируемых символов.

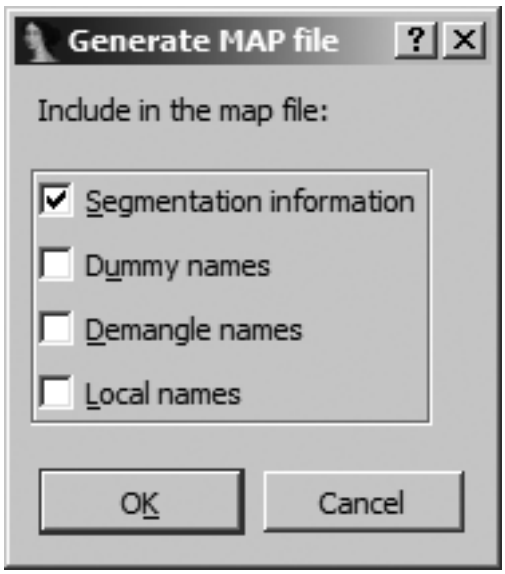


Рисунок 14-5. Параметры создания файла MAP

Адреса логические, в форме номера сегмента и смещения:

```
  Start         Length     Name                   Class
  0001:00000000 000008644H .text                  CODE
  0002:00000000 000001DD6H .rdata                 DATA
  0003:00000000 000002B84H .data                  DATA

  Address         Publics by Value

  0001:00000000   _main
  0001:00000069   _fprintf
```

`_fprintf` находится по смещению `69h` первого сегмента `.text`. Файлы IDA совместимы с Turbo Debugger и в основном используются для восстановления имён при отладке файла с удалёнными символами.

Файлы ASM

IDA может выгрузить текущую базу в `.asm`, стараясь включить достаточно сведений, в том числе структуры, для повторной сборки. Успех зависит от того, понимает ли выбранный ассемблер синтаксис IDA.

Целевой синтаксис задаётся `Target assembler` на вкладке `Analysis` диалога `Options > General`. По умолчанию выгружается вся база. Чтобы ограничить диапазон, выделите его мышью или сочетаниями `Shift` со стрелками. В консольной версии поставьте начало командой `Anchor`, `Alt+L`, затем расширьте выделение стрелками.

Файлы INC

Файл включения содержит определения структур и перечислений, то есть выгрузку окон соответствующих типов в форме, пригодной для ассемблера.

Файлы LST

`.lst` - обычная текстовая копия окна дизассемблированного кода. Диапазон можно ограничить выделением так же, как для ASM.

Файлы EXE

Наиболее многообещающая команда на деле сильнее всего ограничена. Для большинства форматов она не работает и сообщает, что такой выходной файл не поддерживается.

Восстановить исполняемый файл из базы сложно. В ней представлены главным образом загруженные разделы оригинала, но IDA обрабатывает не всё и при импорте теряет часть сведений. Например, раздел ресурсов `.rsrc` у PE по умолчанию не загружается, поэтому восстановить его из базы невозможно.

Другие сведения преобразуются и недоступны в исходном виде: таблицы символов, импорта и экспорта пришлось бы тщательно собирать заново. Поэтому полноценная генерация EXE обычно недостижима.

Одна из попыток решить задачу для PE - комплект сценариев `pe_scripts` Атли Мара Гудмундссона. `pe_write.idc` выгружает рабочий образ из базы. Последовательность такова:

1. Загрузите PE в IDA и снимите флажок `Make imports section` в диалоге загрузчика.
2. Выполните `pe_sections.idc`, чтобы отобразить в базе все разделы оригинала.
3. Внесите нужные изменения.
4. Запустите `pe_write.idc` и сохраните новую копию PE.

Сценарии IDC рассматриваются в главе 15.

Файлы DIF

DIF - текстовый перечень всех байтов, изменённых в базе. Это наиболее полезный выходной формат для переноса правок в оригинал:

```
This difference file is created by The Interactive Disassembler

dif_example.exe
000002F8: 83 FF
000002F9: EC 75
000002FA: 04 EC
000002FB: FF 68
```

После заголовка и имени исходного файла каждая строка содержит файловое смещение, первоначальное значение и текущее значение в базе. В примере изменены четыре байта с `0x2F8` по `0x2FB`.

Написать программу, разбирающую такой файл и применяющую изменения, несложно. Пример `ida_patcher.c` опубликован на сопроводительном сайте книги: <http://www.idabook.com/chapter14/ida_patcher.c>.

Файлы HTML

IDA создаёт цветной листинг, добавляя к содержимому LST разметку HTML. Цвета напоминают рабочее окно, но гиперссылок нет. Поэтому переходить по именам не удобнее, чем в обычном тексте; логичным улучшением были бы ссылки на все именованные адреса.

Итоги

IDA не является редактором двоичных файлов. Однако она хорошо подходит для ввода и наглядной проверки предполагаемых изменений. В сочетании с DIF, сценариями и внешними программами это позволяет создавать исправленные копии.

Далее рассматриваются способы расширения IDA. Базовое владение сценариями и понимание архитектуры подключаемых модулей необходимы каждому, кто хочет добавлять отсутствующее поведение.

Часть IV. Расширение возможностей IDA

Глава 15. Сценарии IDA

Ни одна программа не может заранее удовлетворить все потребности. Разработчикам приходится либо бесконечно добавлять функции, либо дать пользователю средство самостоятельно решать задачи. IDA выбрала второй путь и предоставляет сценарии с широким программным управлением.

Сценарий может быть одной строкой или полноценной программой, автоматизирующей работу и сложный анализ. С точки зрения автоматизации это макрос, а с точки зрения исследования - язык запросов к содержимому базы.

Поддерживаются два языка. Собственный IDC синтаксически похож на Си. Начиная с IDA 5.4 подключаемый модуль IDAPython Гергея Эрделъи интегрирует Python. В этой главе рассматриваются основы обоих вариантов и наиболее полезные функции.

Запуск сценариев

Доступны три команды:

- `File > Script File`;
- `File > IDC Command`;
- `File > Python Command`, если Python установлен.

Первая запускает отдельный файл. Выполненные сценарии добавляются в список недавних:

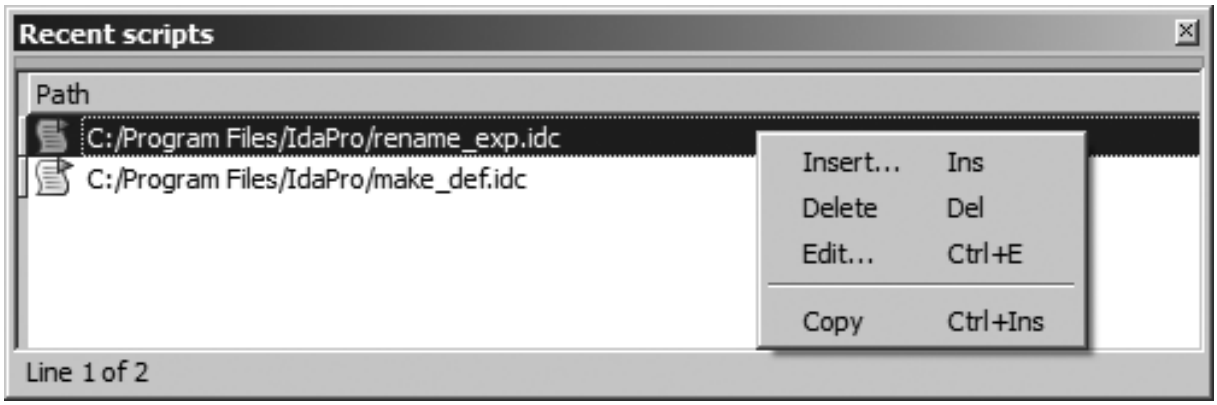


Рисунок 15-1. Окно недавних сценариев

Двойной щелчок повторно запускает файл. В контекстном меню его можно удалить из списка или открыть в редакторе, указанном на вкладке `Misc` диалога `Options > General`.

Для нескольких выражений удобнее диалог команды IDC или Python:

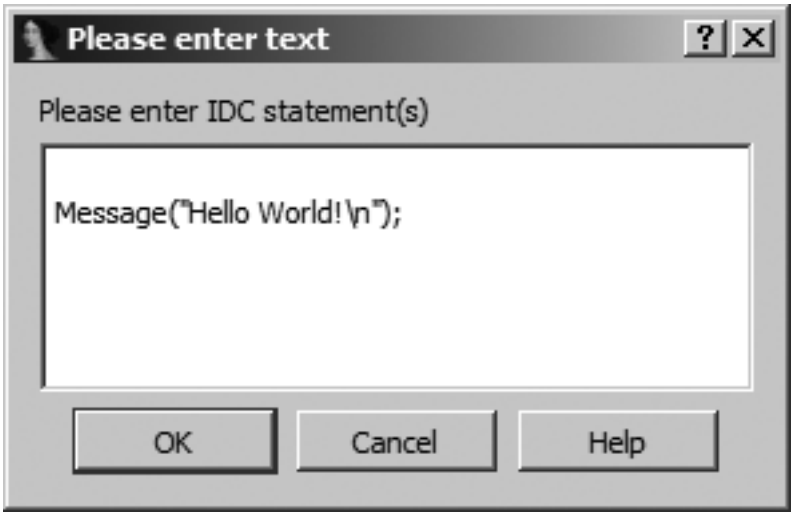


Рисунок 15-2. Диалог ввода сценария

У него есть ограничения, но создавать отдельный файл ради короткой проверки не требуется.

Третий способ - командная строка внизу графического интерфейса. Её видимость задаёт `DISPLAY_COMMAND_LINE` в `<IDADIR>/cfg/idagui.cfg`; начиная с IDA 5.4 она включена по умолчанию.

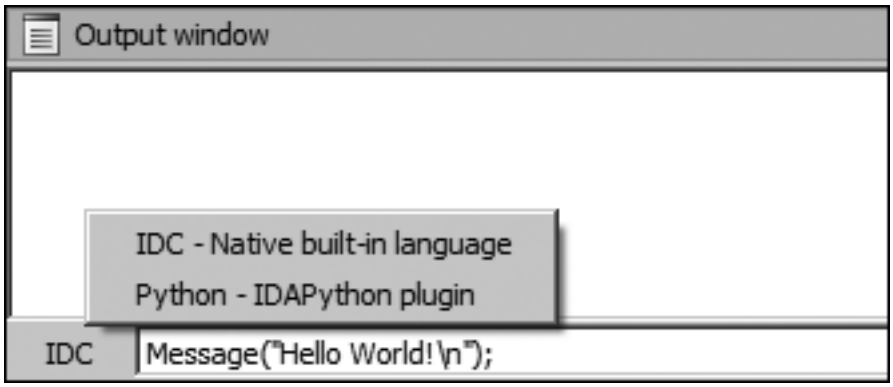


Рисунок 15-3. Командная строка IDA

Слева указан активный интерпретатор. Щелчок по метке переключает IDC и Python. Несколько выражений в одной строке разделяются точкой с запятой, а стрелка вверх открывает историю.

Язык IDC

Во встроенной справке есть разделы `IDC language` с синтаксисом и `Index of IDC functions` с полным перечнем встроенных функций.

IDC заимствует большую часть синтаксиса Си. С версии 5.6 появились объектно-ориентированные средства и исключения, приблизившие его к C++. Далее внимание уделено отличиям.

Переменные IDC

IDC имеет слабую типизацию: явный тип переменной не объявляется. Основные значения:

• целые, в документации называемые `long`;
• строки;

• числа с плавающей точкой.

Чаще всего используются первые два. Строка является самостоятельным типом: не нужно выделять память или следить за завершающим нулём. С IDA 5.6 добавлены объекты, ссылки и указатели на функции.

Переменную нужно объявить до использования. Локальное объявление начинается с `auto` и может включать начальное значение:

```
auto addr, reg, val; // несколько переменных без начальных значений
auto count = 0;      // объявление с инициализацией
```

Поддерживаются многострочные комментарии `/* ... */` и однострочные `//`. В одном выражении можно объявить несколько переменных; каждое простое выражение завершается точкой с запятой.

Классических массивов Си, указателей, структур и объединений в старом IDC нет. В 5.6 появились срезы, ссылки и классы.

Глобальные переменные объявляются словом `extern` внутри или вне функции, но без начального значения:

```
extern outsideGlobal;

static main() {
    extern insideGlobal;
    outsideGlobal = "Глобальная строка";
    insideGlobal = 1;
}
```

Память для глобальной переменной создаётся при первом упоминании в сеансе IDA. Значение сохраняется при открытии и закрытии разных баз, пока не завершена сама IDA.

Выражения IDC

Поддерживается почти вся арифметика и логика Си, включая тернарный оператор `?:`. Составные присваивания +=, *=, >>= и подобные отсутствуют. Оператор запятой появился в 5.6.

Все целые считаются знаковыми. Сравнения всегда знаковые, а `>>` выполняет арифметический сдвиг с копированием старшего бита. Логический сдвиг нужно получить маской:

```
result = (x >> 1) & 0x7fffffff; // обнулить старший бит
```

Присваивание строки копирует её значение, поэтому аналоги `strcpy` и `strdup` не нужны. Сложение строк выполняет сцепление: `"Привет, " + "мир"` даёт `"Привет, мир"` без `strcat`.

С версии 5.6 доступен оператор среза с начальным включительным и конечным исключительным индексом:

```
auto str = "Строка для среза";
auto s1, s2, s3, s4;
s1 = str[7:10]; // "для"
s2 = str[:6];   // "Строка": пропуск начала означает 0
s3 = str[11:];  // "среза": пропуск конца означает конец строки
s4 = str[5];    // "а": один элемент
```

Хотя обычного массива нет, строку со срезами можно использовать подобным образом.

Операторы IDC

Простые операторы заканчиваются точкой с запятой. Из составных конструкций Си отсутствует только `switch`.

В циклах помните об отсутствии составного присваивания:

```
auto i;
for (i = 0; i < 10; i += 2) {}    // недопустимо
for (i = 0; i < 10; i = i + 2) {} // допустимо
```

В 5.6 появились `try`, `catch` и `throw`, синтаксически похожие на исключения C++.

Блоки используют фигурные скобки. Новые переменные разрешено объявлять в начале блока, но область видимости соблюдается нестрого:

```
if (1) {
    auto x;
    x = 10;
}
else {
    auto y;
    y = 3;
}
Message("x = %d\n", x);
Message("y = %d\n", y);
```

Результат сообщает `x = 10` и `y = 0`. Даже y из невыполненного блока доступна после него. Однако локальная переменная одной функции всё равно недоступна другой.

Функции IDC

Пользовательские функции разрешены только в отдельных `.idc`, но не в диалоге команды. Объявление начинается со `static`, а параметры перечисляются без типов:

```
static my_func(x, y, z) {
    auto a, b, c; // локальные переменные должны быть первыми
    // тело функции
}
```

До IDA 5.6 все параметры передавались по значению. Затем появилась передача по ссылке. В отличие от C++, способ задаётся при вызове, а не в объявлении: перед аргументом ставится &.

```
auto q = 0, r = 1, s = 2;
my_func(q, r, s);  // все по значению
my_func(q, &r, s); // r по ссылке и может измениться у вызывающего кода
```

Объявление не указывает наличие и тип результата. `return` может вернуть значение любого типа, причём разные ветви одной функции способны возвращать строку и число. Если явного `return` нет, возвращается 0.

Ограничение диалога IDC

Диалог удобен для быстрой проверки коротких последовательностей, но внутри нельзя объявлять функции. IDA сама оборачивает введённые выражения во временную функцию. Новое объявление оказалось бы вложенным, что синтаксис IDC не допускает.

С IDA 5.6 ссылки на функции можно передавать и возвращать:

```
static getFunc() {
    return Message;
}

static useFunc(func, arg) {
    func(arg);
}
static main() {
```

```
    auto f = getFunc();
    f("Привет, мир\n");
    useFunc(f, "Напечатай меня\n");
}
```

Для функций & не требуется: они всегда передаются по ссылке.

Объекты IDC

В IDA 5.6 появились классы и объекты. Далее предполагается знакомство с объектно-ориентированным языком вроде C++ или Java.

Корневой класс называется `object`; поддерживается одиночное наследование. Спецификаторов `public` и `private` нет, все члены открыты. В объявлении класса перечисляются только методы, а поле создаётся обычным присваиванием значения члену объекта.

```
class ExampleClass {
    ExampleClass(x, y) { // конструктор
        this.a = x;
        this.b = y;
    }
    ~ExampleClass() { // деструктор
    }
    foo(x) {
        this.a = this.a + x;
    }
};

static main() {
    // ExampleClass ex; // так нельзя: это недопустимое объявление
    auto ex = ExampleClass(1, 2);
    ex.foo(10);
    ex.z = "строка"; // поле появилось у ex, но не у самого класса
}
```

Ссылочная переменная инициализируется результатом вызова конструктора, а к членам обращаются через точку. Подробный синтаксис классов приведён во встроенной справке.

Программы IDC

Для задачи длиннее нескольких выражений лучше создать отдельный `.idc`: его можно сохранять, переносить и повторно применять. Такой файл обязан содержать функцию `main` без параметров. Обычно также подключается `idc.idc` с полезными макросами:

```
#include <idc.idc>

static main() {
    // действия сценария
}
```

Поддерживаются директивы препроцессора Си:

- `#include <file>` подключает файл;
- `#define <name> [value]` создаёт макрос с необязательным значением;
- `#ifdef <name>` проверяет существование макроса;
- `#else` задаёт альтернативную ветвь;
- `#endif` завершает условный блок;
- `#undef <name>` удаляет макрос.

IDA заранее определяет, среди прочего, `_NT_`, `_LINUX_`, `_MAC_`, `_GUI_` и `_TXT_`, позволяющие проверять среду выполнения.

Обработка ошибок

Сообщения IDC об ошибках скромны. Ошибки разбора не дают программе запуститься: это нарушения синтаксиса, неизвестные переменные и неправильное число аргументов. За один запуск сообщается только о первой проблеме. Иногда место и причина точны, например `hello_world.idc,20: Missing semicolon`, а иногда появляется лишь бесполезное указание на конец файла. Пятнадцать синтаксических ошибок могут потребовать пятнадцати запусков.

Ошибки выполнения встречаются реже и немедленно прекращают сценарий. Пример - вызов неизвестной функции, почему-либо пропущенный при разборе.

Бесконечный или слишком долгий сценарий трудно остановить. Через несколько секунд IDA показывает единственный диалог отмены:



Рисунок 15-4. Диалог прекращения сценария

Полноценного отладчика IDC нет. Обычно приходится широко применять вывод сообщений. `try` и `catch`, появившиеся в 5.6, позволяют хотя бы корректно обрабатывать сбои.

Постоянное хранение данных

Обычных массивов IDC не имеет, но документация упоминает глобальные постоянные массивы. Точнее считать их именованными разреженными объектами. Они сохраняются внутри базы между запусками сценариев и сеансами IDA.

Значение хранится по индексу. В одной позиции могут одновременно находиться одно целое и одна строка, но не число с плавающей точкой. Внутренний механизм IDA называется `netnode`; функции массива предоставляют более простой интерфейс.

Основные функции:

```
long CreateArray(string name)
```

Создаёт объект и возвращает числовой дескриптор. Если имя уже занято, возвращает `-1`.

```
long GetArrayId(string name)
```

Находит существующий объект по имени или возвращает `-1`.

```
long SetArrayLong(long id, long idx, long value)
long SetArrayString(long id, long idx, string str)
```

Сохраняют целое или строку по индексу. Возвращают 1 при успехе и 0 при неверном дескрипторе.

```
string_or_long GetArrayElement(long tag, long id, long idx)
```

Читает значение. tag равен AR_LONG для целого либо AR_STR для строки.

```
long DelArrayElement(long tag, long id, long idx)
```

Удаляет целую или строковую часть позиции согласно tag.

```
void DeleteArray(long id)
long RenameArray(long id, string newname)
```

Первая удаляет объект со всем содержимым. Вторая переименовывает и возвращает 1 при успехе.

Такие массивы могут имитировать глобальные переменные и сложные типы либо сохранять результаты между запусками. Дескриптор передаётся функциям или повторно получается по имени. Если сведения нужны только одному запуску, объект следует удалить перед завершением, чтобы старые значения не повлияли на следующий.

Привязка IDC к клавишам

При каждом запуске IDA выполняет <IDADIR>/idc/ida.idc. В поставке его main пуст. Чтобы связать свою функцию с клавишей, подключите файл и вызовите AddHotkey внутри main:

```
#include <idc.idc>
#include <my_amazing_script.idc>

static main() {
    AddHotkey("z", "MyAmazingFunc");
}
```

Теперь z вызывает MyAmazingFunc. Если сочетание уже занято меню или модулем, AddHotkey молча завершается неудачей; узнать об этом можно только по отсутствию вызова.

Стандартный каталог включения - <IDADIR>/idc. Файл в другом месте подключается полным путём. При самостоятельной проверке сценарий может содержать main, но перед включением в ida.idc эту функцию нужно переименовать, иначе возникнет конфликт с основным main.

Полезные функции IDC

Встроенных функций очень много; они перечислены в справке Index of IDC functions, зачастую лишь в виде объявлений из idc.idc. Имена не всегда последовательны: чтение может называться GetXXX или иначе, изменение - SetXXX, MakeXXX либо совершенно по-другому. Поэтому приходится просматривать перечень и документацию. Те же возможности доступны через оболочки IDAPython.

Чтение и изменение данных

```
long Byte(long addr)
long Word(long addr)
long Dword(long addr)
```

Читают по виртуальному адресу байт, двухбайтовое слово и четырёхбайтовое двойное слово.

```
void PatchByte(long addr, long val)
void PatchWord(long addr, long val)
void PatchDword(long addr, long val)
```

Записывают соответствующее значение в базу.

```
bool isLoaded(long addr)
```

Возвращает 1, если по адресу есть действительные данные.

Все функции учитывают порядок байтов процессорного модуля. Запись отбрасывает старшие байты: `PatchByte(0x401010, 0x1234)` сохраняет `0x34`.

При чтении неверного адреса возвращаются соответственно `0xFF`, `0xFFFF` и `0xFFFFFFFF`. Они неотличимы от настоящих данных, поэтому сначала лучше вызвать `isLoaded`. После изменения экран иногда обновляется не сразу; прокрутите от адреса и вернитесь.

Взаимодействие с пользователем

```
void Message(string format, ...)
```

Выводит форматированный текст в окно сообщений подобно `printf`.

```
void print(...)
```

Выводит строковое представление каждого аргумента.

```
void Warning(string format, ...)
```

Показывает форматированное сообщение в диалоге.

```
string AskStr(string default, string prompt)
```

Запрашивает строку и возвращает её либо 0 при отмене.

```
string AskFile(long doSave, string mask, string prompt)
```

Открывает выбор файла. `doSave = 1` разрешает новое имя для записи, `0` выбирает существующий файл для чтения. `mask`, например `*.idc`, фильтрует список. При отмене возвращается 0.

```
long AskYN(long default, string prompt)
```

Задаёт вопрос с вариантами «да», «нет», «отмена». Стандартный выбор: 1, 0 или `-1`; возвращается выбранное число.

```
long ScreenEA()
bool Jump(long addr)
```

Первая получает адрес под курсором, вторая переводит листинг к адресу. `Message` часто заменяет отладчик, а семейство `AskXXX` содержит дополнительные диалоги для чисел и других значений.

Работа со строками

```
string form(string format, ...)    // до IDA 5.6
string sprintf(string format, ...) // IDA 5.6 и новее
```

Формируют строку подобно `sprintf` в Си.

```
long atol(string val)
long xtol(string val)
```

Преобразуют десятичную или шестнадцатеричную строку в целое; `xtol` допускает префикс `0x`.

```
string ltoa(long val, long radix)
```

Возвращает запись числа по основанию 2, 8, 10 или 16.

```
long ord(string ch)
long strlen(string str)
long strstr(string str, string substr)
string substr(string str, long start, long end)
```

Соответственно возвращают код ASCII одного символа, длину, позицию подстроки или -1, и диапазон [start, end). В новых версиях последний равнозначен str[start:end]. Без срезов для перебора символов приходится последовательно получать односимвольные подстроки.

Файловый ввод и вывод

```
long fopen(string filename, string mode)
void fclose(long handle)
```

Открывают файл в режиме, похожем на Си (r, w и так далее), и закрывают его. При ошибке открытия возвращается 0.

```
long filelength(long handle)
long fgetc(long handle)
long fputc(long val, long handle)
```

Получают длину, читают и записывают один байт. Ошибка обычно обозначается -1, успешная запись - 0.

```
long fprintf(long handle, string format, ...)
long writestr(long handle, string str)
string_or_long readstr(long handle)
```

Записывают форматированный текст или строку и читают до следующего перевода строки включительно. В конце файла readstr возвращает -1.

```
long writelong(long handle, long val, long bigendian)
long readlong(long handle, long bigendian)
long writeshort(long handle, long val, long bigendian)
long readshort(long handle, long bigendian)
```

Читают и записывают четырёх- и двухбайтовые целые. bigendian = 1 задаёт старший байт первым, 0 - младший.

```
bool loadfile(long handle, long pos, long addr, long length)
bool savefile(long handle, long pos, long addr, long length)
```

loadfile переносит length байтов из позиции pos файла в базу по адресу addr; savefile выполняет обратное действие.

Имена в базе

```
string Name(long addr)
```

Возвращает имя адреса или пустую строку. Локальные пользовательские имена не возвращаются.

```
string NameEx(long from, long addr)
```

Также возвращает локальное имя, если from находится в той же функции, что и addr.

```
bool MakeNameEx(long addr, string name, long flags)
```

Назначает имя с атрибутами из битовой маски: локальное, открытое, показываемое в окне имён и так далее.

```
long LocByName(string name)
long LocByNameEx(long funcaddr, string localname)
```

Первая ищет глобальное имя, вторая локальное внутри функции. При отсутствии возвращается BADADDR, то есть -1.

Работа с функциями

```
long GetFunctionAttr(long addr, long attrib)
```

Возвращает атрибут функции, содержащей адрес. Например, `FUNCATTR_END` даёт её конец.

```
string GetFunctionName(long addr)
```

Возвращает имя содержащей функции либо пустую строку.

```
long NextFunction(long addr)
long PrevFunction(long addr)
```

Возвращают начало следующей или предыдущей функции либо `-1`. Начало функции по имени находится через `LocByName`.

Ссылки на код

Нужно различать исходящие потоки от адреса и входящие ссылки на адрес. Функции образуют последовательности и поддерживают перебор.

```
long Rfirst(long from)
long Rnext(long from, long current)
```

Возвращают первую и следующую цель передачи управления из `from`. При завершении выдаётся `BADADDR`.

```
long XrefType()
```

Сообщает вид последней найденной ссылки: `fl_CN` и `fl_CF` для ближнего и дальнего вызова, `fl_JN` и `fl_JF` для перехода, `fl_F` для обычного потока.

```
long RfirstB(long to)
long RnextB(long to, long current)
```

Перебирают адреса, передающие управление в `to`.

Каждый поиск меняет внутреннее состояние последней ссылки. Поэтому `XrefType` нужно вызвать до следующей функции перебора.

Ссылки на данные

```
long Dfirst(long from)
long Dnext(long from, long current)
```

Перебирают цели обращений к данным из адреса.

```
long DfirstB(long to)
long DnextB(long to, long current)
```

Перебирают источники обращений к данным по адресу `to`.

`XrefType` возвращает `dr_O` для использования адреса, `dr_W` для записи и `dr_R` для чтения. Здесь действует то же требование немедленно сохранить вид ссылки.

Форматирование базы

```
void MakeUnkn(long addr, long flags)
```

Делает элемент неопределённым. Флаги управляют последующими элементами и удалением имён. MakeUnknown выполняет то же для больших диапазонов.

```
long MakeCode(long addr)
```

Преобразует байты в инструкцию и возвращает её длину либо 0.

```
bool MakeByte(long addr)
```

Создаёт байт данных; также существуют MakeWord и MakeDword.

```
bool MakeComm(long addr, string comment)
bool MakeFunction(long begin, long end)
bool MakeStr(long begin, long end)
```

Добавляют комментарий, функцию и строку текущего типа. Если конец функции или строки равен BADADDR, IDA пытается найти его автоматически.

Поиск по базе

Семейство FindXXX принимает битовую маску. Полезные флаги:

- SEARCH_DOWN - двигаться к большим адресам;
- SEARCH_NEXT - пропустить текущее совпадение;
- SEARCH_CASE - учитывать регистр при поиске текста и байтов.

```
long FindCode(long addr, long flags)
long FindData(long addr, long flags)
```

Ищут инструкцию или элемент данных.

```
long FindBinary(long addr, long flags, string binary)
```

Ищет шестнадцатеричную последовательность. Без SEARCH_CASE байты букв ASCII совпадают и с другим регистром: 41 42 может найти 61 62.

```
long FindText(long addr, long flags, long row, long column, string text)
```

Ищет текст с заданной строки и столбца листинга. Один адрес может занимать несколько строк.

SEARCH_NEXT не задаёт направление. Без него допустимо вернуть сам переданный адрес, если элемент уже совпадает.

Части строки листинга

```
string GetDisasm(long addr)
string GetMnem(long addr)
string GetOpnd(long addr, long opnum)
```

Возвращают полный текст без адреса, мнемонику и текст операнда. Операнды нумеруются с нуля слева направо.

```
long GetOpType(long addr, long opnum)
long GetOperandValue(long addr, long opnum)
```

Возвращают код типа операнда и связанное целое значение; смысл последнего зависит от типа.

```
string CommentEx(long addr, long type)
```

При type = 0 возвращает обычный комментарий, при 1 повторяемый; при отсутствии - пустую строку.

Примеры сценариев IDC

Перебор функций

Следующий сценарий печатает границы, размер локальной области и параметры каждой функции:

```
#include <idc.idc>

static main() {
    auto addr, end, args, locals, frame, firstArg, name, ret;
    addr = 0;
    for (addr = NextFunction(addr); addr != BADADDR; addr = NextFunction(addr)) {
        name = Name(addr);
        end = GetFunctionAttr(addr, FUNCATTR_END);
        locals = GetFunctionAttr(addr, FUNCATTR_FRSIZE);
        frame = GetFrame(addr);
        ret = GetMemberOffset(frame, " r");
        if (ret == -1) continue;
        firstArg = ret + 4;
        args = GetStrucSize(frame) - firstArg;
        Message("Функция: %s, начало %x, конец %x\n", name, addr, end);
        Message("   Локальная область: %d байтов\n", locals);
        Message("   Параметры: %d байтов (%d шт.)\n", args, args / 4);
    }
}
```

Листинг 15-1. Перебор функций

`GetFrame` получает кадр как структуру, `GetStrucSize` его размер, а `GetMemberOffset` смещение сохранённого адреса возврата с именем `r`. Первый параметр находится на 4 байта дальше. У импортированных функций кадра нет, поэтому отсутствие адреса возврата служит простым фильтром.

Перебор инструкций

Сценарий считает инструкции функции под курсором:

```
#include <idc.idc>

static main() {
    auto func, end, count, inst;
    func = GetFunctionAttr(ScreenEA(), FUNCATTR_START);
    if (func != -1) {
        end = GetFunctionAttr(func, FUNCATTR_END);
        count = 0;
        inst = func;
        while (inst < end) {
            count++;
            inst = FindCode(inst, SEARCH_DOWN | SEARCH_NEXT);
        }
        Warning("%s содержит %d инструкций\n", Name(func), count);
    }
    else {
        Warning("По адресу %x функция не найдена", ScreenEA());
    }
}
```

Листинг 15-2. Перебор инструкций

Сначала определяется начало и конец функции, затем `FindCode` находит последовательные инструкции. Решение предполагает непрерывный диапазон. Для функций из нескольких фрагментов следовало бы обходить потоки управления.

Перебор вызовов

Разбор мнемоники `call` зависел бы от архитектуры и потребовал отдельно извлекать цель. Ссылки процессорно-независимы и сразу дают адрес.

```
#include <idc.idc>

static main() {
    auto func, end, target, inst, name, flags, xref;
    flags = SEARCH_DOWN | SEARCH_NEXT;
    func = GetFunctionAttr(ScreenEA(), FUNCATTR_START);
    if (func != -1) {
        name = Name(func);
        end = GetFunctionAttr(func, FUNCATTR_END);
        for (inst = func; inst < end; inst = FindCode(inst, flags)) {
            for (target = Rfirst(inst); target != BADADDR; target = Rnext(inst, target)) {
                xref = XrefType();
                if (xref == fl_CN || xref == fl_CF) {
                    Message("%s вызывает %s из 0x%x\n", name, Name(target), inst);
                }
            }
        }
    }
    else {
        Warning("По адресу %x функция не найдена", ScreenEA());
    }
}
```

Листинг 15-3. Перебор вызовов функции

Для каждой инструкции рассматриваются исходящие ссылки, но остаются только ближние и дальние вызовы. Вариант снова предполагает непрерывную функцию, хотя его легко переделать в обход по потоку.

Обратная задача - найти всех вызывающих опасные функции вроде `strcpy` и `sprintf`. Они копируют без ограничения размера приёмника и при забытых проверках часто приводят к переполнению буфера.

```
#include <idc.idc>

static list_callers(bad_func) {
    auto func, addr, xref, source;
    func = LocByName(bad_func);
    if (func == BADADDR) {
        Warning("%s не найдена в базе", bad_func);
    }
    else {
        for (addr = RfirstB(func); addr != BADADDR; addr = RnextB(func, addr)) {
            xref = XrefType();
            if (xref == fl_CN || xref == fl_CF) {
                source = GetFunctionName(addr);
                Message("%s вызывается из 0x%x в %s\n", bad_func, addr, source);
            }
        }
    }
}

static main() {
    list_callers("_strcpy");
    list_callers("_sprintf");
}
```

Листинг 15-4. Перебор вызывающих функций

После поиска имени перебираются входящие ссылки, выбираются вызовы и выводится имя содержащей функции.

Для импорта ELF имя может оказаться неожиданным. Таблица PLT способна содержать `.memcpy`, показанную как `_memcpy`, поскольку точка запрещена в именах IDA. Одновременно символ `memcpy` может находиться в разделе `extern`. Для поиска вызовов нужна именно запись PLT, на которую ссылается код программы.

Перебор экспортируемых функций

Следующий сценарий создаёт `.idt` после открытия совместно используемой библиотеки:

```
#include <idc.idc>

static main() {
    auto entryPoints, i, ord, addr, name, purged, file, fd;
    file = AskFile(1, "*.idt", "Выберите выходной файл IDT");
    fd = fopen(file, "w");
    entryPoints = GetEntryPointQty();
    fprintf(fd, "ALIGNMENT 4\n");
    fprintf(fd, "0 Name=%s\n", GetInputFile());
    for (i = 0; i < entryPoints; i++) {
        ord = GetEntryOrdinal(i);
        if (ord == 0) continue;
        addr = GetEntryPoint(ord);
        if (ord == addr) continue; // у точки входа нет порядкового номера
        name = Name(addr);
        fprintf(fd, "%d Name=%s", ord, name);
        purged = GetFunctionAttr(addr, FUNCATTR_ARGSIZE);
        if (purged > 0) fprintf(fd, " Pascal=%d", purged);
        fprintf(fd, "\n");
    }
}
```

Листинг 15-5. Создание файла IDT

`GetEntryPointQty` возвращает число экспортов, `GetEntryOrdinal` порядковый номер, `GetEntryPoint` адрес экспорта по номеру, а `GetInputFile` имя загруженного файла. В отличие от `dll2idt`, IDA может добавить число очищаемых байтов, если анализ функции его установил.

Поиск и разметка параметров функций

В x86 коде `gcc` новее 3.4 часто размещает параметры инструкциями `mov`, а не `push`. Старые версии IDA распознают это хуже. При обычных помещениях в стек комментарии полны:

```
.text:08048894    push    0               ; protocol
.text:08048896    push    1               ; type
.text:08048898    push    2               ; domain
.text:0804889A    call    _socket
```

При записи относительно ESP они могут исчезнуть:

```
.text:080487AD    mov     [esp+8], 0
.text:080487B5    mov     [esp+4], 1
.text:080487BD    mov     [esp], 2
.text:080487C4    call    _socket
```

Сценарий ищет такие инструкции в функции с кадром на EBP и подписывает смещения:

```
#include <idc.idc>

static main() {
    auto addr, op, end, idx;
```

```
    auto func_flags, type, val, search;
    search = SEARCH_DOWN | SEARCH_NEXT;
    addr = GetFunctionAttr(ScreenEA(), FUNCATTR_START);
    func_flags = GetFunctionFlags(addr);
    if (func_flags & FUNC_FRAME) {
        end = GetFunctionAttr(addr, FUNCATTR_END);
        for (; addr < end && addr != BADADDR; addr = FindCode(addr, search)) {
            type = GetOpType(addr, 0);
            if (type == 3) {
                if (GetOperandValue(addr, 0) == 4) MakeComm(addr, "arg_0");
            }
            else if (type == 4) {
                idx = strstr(GetOpnd(addr, 0), "[esp");
                if (idx != -1) {
                    val = GetOperandValue(addr, 0);
                    MakeComm(addr, form("arg_%d", val));
                }
            }
        }
    }
}
```

Листинг 15-6. Автоматическое распознавание параметров

`GetFunctionFlags` проверяет свойства функции. Для каждого операнда памяти с базой ESP определяется глубина и добавляется комментарий:

```
.text:080487AD    mov     [esp+8], 0   ; arg_8
.text:080487B5    mov     [esp+4], 1   ; arg_4
.text:080487BD    mov     [esp], 2     ; arg_0
.text:080487C4    call    _socket
```

Это уже показывает назначение трёх `mov`. Расширенная версия с чтением сигнатуры может восстановить типы и имена:

```
.text:080487AD    mov     [esp+8], 0   ; int protocol
.text:080487B5    mov     [esp+4], 1   ; int type
.text:080487BD    mov     [esp], 2     ; int domain
.text:080487C4    call    _socket
```

Полный расширенный сценарий опубликован на странице примеров книги: <http://www.idabook.com/ch15_examples>.

Эмуляция поведения машинного кода

Сценарий может повторить действия самомодифицирующейся программы или декодировать данные без запуска подозрительного файла. Это полезно и при отсутствии нужной аппаратной платформы, например для MIPS в среде Windows.

Ниже приведён код x86 из задания Capture the Flag конференции DEFCON 15, декодирующий встроенный закрытый ключ:

```
.text:08049EDE                mov     [ebp+var_4], 0
.text:08049EE5 loc_8049EE5:
.text:08049EE5                cmp     [ebp+var_4], 3C1h
.text:08049EEC                ja      short locret_8049F0D
.text:08049EEE                mov     edx, [ebp+var_4]
.text:08049EF1                add     edx, 804B880h
.text:08049EF7                mov     eax, [ebp+var_4]
.text:08049EFA                add     eax, 804B880h
.text:08049EFF                mov     al, [eax]
.text:08049F01                xor     eax, 4Bh
.text:08049F04                mov     [edx], al
.text:08049F06                lea     eax, [ebp+var_4]
.text:08049F09                inc     dword ptr [eax]
.text:08049F0B                jmp     short loc_8049EE5
```

Буквальный перевод в IDC:

```
auto var_4, edx, eax, al;
var_4 = 0;
while (var_4 <= 0x3C1) {
    edx = var_4;
    edx = edx + 0x804B880;
    eax = var_4;
    eax = eax + 0x804B880;
    al = Byte(eax);
    al = al ^ 0x4B;
    PatchByte(edx, al);
    var_4++;
}
```

Листинг 15-7. Эмуляция языка ассемблера средствами IDC

Механические правила:

1. Для каждого регистра и стековой переменной объявите переменную IDC.
2. Каждую инструкцию замените выражением с тем же действием.
3. Стек моделируйте чтением и записью переменных сценария.
4. Чтение другой памяти выполняйте через `Byte`, `Word` или `Dword`.
5. Запись выполняйте через `PatchByte`, `PatchWord` или `PatchDword`.
6. Если условие завершения цикла непонятно, начните с `while (1)` и добавьте `break` в месте выхода.
7. Вызовы функций требуют отдельно воспроизвести их поведение и правдоподобный результат. Это может сделать эмуляцию непрактичной.

Необязательно сразу понимать весь алгоритм. Достаточно правильно перевести отдельные инструкции; затем работающий сценарий поможет исследовать общий смысл. После понимания пример сокращается до:

```
auto var_4, addr;
for (var_4 = 0; var_4 <= 0x3C1; var_4++) {
    addr = 0x804B880 + var_4;
    PatchByte(addr, Byte(addr) ^ 0x4B);
}
```

Если базу изменять не нужно, вместо `PatchByte` можно выводить текст через `Message` или записывать двоичный результат в файл.

IDAPython

IDAPython встраивает интерпретатор Python и предоставляет оболочки для возможностей IDC. Его преимущества - родные типы данных Python, богатая стандартная библиотека и доступ к значительной части комплекта разработки IDA, недоступной IDC.

Начиная с IDA 5.4 модуль поставляется стандартно и включается при наличии совместимого Python. Версия для Windows устанавливает подходящий Python сама, а в Linux и OS X это раньше оставалось пользователю. В описываемой IDA 6.1 для Linux ожидался Python 2.6; с Python 2.7 можно было создать символическую ссылку:

```
# ln -s /usr/lib/libpython2.7.so.1.0 /usr/lib/libpython2.6.so.1
```

В OS X при слишком старом системном Python рекомендовался установщик с официального сайта.

Использование IDAPython

Предоставляются три модуля:

• `idaapi` открывает основное программное API из комплекта разработки;
• `idc` содержит аналоги функций IDC;
• `idautils` предлагает удобные средства, часто генераторы списков функций, ссылок и других объектов базы.

`idc` и `idautils` импортируются автоматически, `idaapi` нужно подключать явно.

В IDA существует один экземпляр интерпретатора до завершения программы. Все сценарии словно выполняются в одной оболочке Python: однажды импортированный модуль остаётся доступным, а переменные и функции сохраняются до переопределения или закрытия IDA.

Знавшему IDC или комплект разработки легко освоить `idc` и `idaapi`; достаточно изучить удобства `idautils`. Модуль `idc` в основном повторяет API IDC, поэтому встроенная справка и описания выше применимы напрямую.

Примеры IDAPython

Перебор функций

```
funcs = Functions()
for f in funcs:
    name = Name(f)
    end = GetFunctionAttr(f, FUNCATTR_END)
    locals = GetFunctionAttr(f, FUNCATTR_FRSIZE)
    frame = GetFrame(f)
    if frame is None:
        continue
    ret = GetMemberOffset(frame, " r")
    if ret == -1:
        continue
    firstArg = ret + 4
    args = GetStrucSize(frame) - firstArg
    Message("Функция: %s, начало %x, конец %x\n" % (name, f, end))
    Message("   Локальная область: %d байтов\n" % locals)
    Message("   Параметры: %d байтов (%d шт.)\n" % (args, args / 4))
```

Листинг 15-8. Перебор функций на Python

Главное упрощение - генератор `Functions`, который естественно используется в цикле `for`.

Перебор инструкций

```
from idaapi import *

func = get_func(here())
if func is not None:
    fname = Name(func.startEA)
    count = 0
    for i in FuncItems(func.startEA):
        count = count + 1
    Warning("%s содержит %d инструкций\n" % (fname, count))
else:
    Warning("По адресу %x функция не найдена" % here())
```

Листинг 15-9. Перебор инструкций на Python

get_func из idaapi возвращает объект func_t, а FuncItems из idautils перебирает все инструкции, включая фрагменты. Генератор нельзя напрямую передать len, поэтому счётчик увеличивается вручную.

Перебор перекрёстных ссылок

```
from idaapi import *

func = get_func(here())
if func is not None:
    fname = Name(func.startEA)
    for i in FuncItems(func.startEA):
        for xref in XrefsFrom(i, 0):
            if xref.type == fl_CN or xref.type == fl_CF:
                Message("%s вызывает %s из 0x%x\n" % (fname, Name(xref.to), i))
else:
    Warning("По адресу %x функция не найдена" % here())
```

Листинг 15-10. Перебор вызовов на Python

Генератор XrefsFrom возвращает объекты xrefblk_t с типом и целевым адресом ссылки.

Перебор экспортов

```
file = AskFile(1, "*.idt", "Выберите выходной файл IDT")
with open(file, "w") as fd:
    fd.write("ALIGNMENT 4\n")
    fd.write("0 Name=%s\n" % GetInputFile())
    for i in range(GetEntryPointQty()):
        ord = GetEntryOrdinal(i)
        if ord == 0:
            continue
        addr = GetEntryPoint(ord)
        if ord == addr:
            continue
        fd.write("%d Name=%s" % (ord, Name(addr)))
        purged = GetFunctionAttr(addr, FUNCATTR_ARGSIZE)
        if purged > 0:
            fd.write(" Pascal=%d" % purged)
        fd.write("\n")
```

Листинг 15-11. Создание IDT на Python

Здесь нет специального генератора экспортов, поэтому применяются те же функции, что в IDC. Отличие - обычные файловые средства Python, которые предпочтительнее функций IDC.

Итоги

Сценарии являются мощным способом расширения IDA. Они подходят для небольших задач, быстрых опытов и автоматизации анализа. Готовые примеры распространялись на сайте Hex-Rays и бывшем сайте IDA Palace.

IDC ограничен простыми типами и не предоставляет полноценные библиотеки вроде стандартной библиотеки Си или программного интерфейса Windows. IDAPython снимает многие ограничения, но для максимального контроля требуются скомпилированные расширения и комплект разработки IDA. Они сложнее, зато дают возможности, рассматриваемые в следующей главе.

Глава 16. Комплект разработки IDA

В книге часто говорилось, что «IDA делает» то или иное, хотя значительная часть интеллекта находится в её модулях. Например, решения анализа принимает процессорный модуль, поэтому возможности IDA зависят от его качества. Для обычного пользователя модульная архитектура скрыта интерфейсом.

Когда IDC недостаточно по скорости или возможностям, следует перейти к комплекту разработки IDA, SDK, и создавать скомпилированные модули.

> **Примечание.** Сам IDC построен поверх SDK. Любое действие IDC в итоге вызывает функции комплекта, но обратное неверно: многие возможности SDK не имеют аналога в сценариях.

SDK открывает внутренние интерфейсы как библиотеки C++ и заголовочные файлы. Он нужен для:

• загрузчиков новых форматов;
• процессорных модулей новых архитектур;
• подключаемых модулей, более мощных, чем сценарии.

Осторожность при расширении

В C++ доступны системные интерфейсы и сторонние библиотеки, но сложные возможности могут нарушить устойчивость IDA. Программа в основном однопоточная, низкоуровневые структуры базы не защищены синхронизацией.

До IDA 5.5 нельзя было создавать дополнительные потоки, одновременно обращающиеся к базе. В 5.5 и новее потоки допустимы, но вызовы SDK следует ставить в очередь через `exec_request_t` и `execute_sync` из `kernwin.hpp`. Любая блокирующая операция делает интерфейс неотзывчивым до завершения.

Эта глава знакомит с общими средствами, одинаково полезными для подключаемых, загрузочных и процессорных модулей. Специфика рассматривается в главах 17-19.

Общие сведения об SDK

Комплект распространяется архивом на носителе IDA или сайте Hex-Rays для владельцев. Имя соответствует версии, например `idasdk61.zip` для IDA 6.1. Документация минимальна: основной `readme.txt` и отдельные README для трёх видов модулей.

До версии 4.9 интерфейсы могли меняться так, что модуль для SDK 4.8 не собирался с 4.9 без исправлений. В 4.9 Hex-Rays стабилизировала API: старый исходный код обычно собирается новым комплектом, а двоичные модули работают в более новых IDA.

Это не означает полной заморозки. Новые версии добавляют возможности и являются надмножествами прежних; модуль с новым API не обязан работать в старой IDA. Некоторые функции переименовывались или объявлялись устаревшими. Макросы позволяют запретить устаревшие вызовы и заметить их при сборке.

Установка

До SDK 5.4 архив не имел верхнего каталога и содержал имена папок, совпадающие с IDA. Поэтому его следовало распаковывать в отдельный каталог вроде `idasdk53`. Начиная с 5.4 верхний каталог уже есть, например `idasdk61`. Расположение относительно `<IDADIR>` не важно; далее корень называется `<SDKDIR>`.

Структура каталогов

`bin`. Сюда примеры сборки помещают готовые модули. Установка состоит в копировании из соответствующего подкаталога в каталог IDA. Здесь же находится программа окончательной обработки процессорных модулей.

`etc`. Исходные тексты двух служебных программ для сборки некоторых модулей; готовые версии также поставляются.

`include`. Все заголовки открытого API: доступные структуры и функции. Основной `readme.txt` кратко описывает важные файлы. Это главная документация в духе «читайте исходный код».

`ldr`. Исходники и сценарии сборки примеров загрузчиков; README лишь перечисляет содержимое.

`lib`. Библиотеки компоновки для разных компиляторов и вариантов IDA. Имена подкаталогов показывают назначение. Например, `x86_win_vc_32` в 6.1 или `vc.w32` ранее предназначен Visual Studio и 32-разрядной IDA Windows; `x64_mac_gcc_64` или прежний `gcc64.mac64` - 64-разрядной IDA OS X с GCC.

`module`. Примеры процессорных модулей и их сборка.

`plugins`. Примеры подключаемых модулей. README даёт общий обзор архитектуры.

Корень. Общие make-файлы, главный `readme.txt` и инструкции `install_xxx.txt` для компиляторов, например `install_visual.txt`.

Документации мало, поэтому знания обычно получают опытным путём и чтением комплекта. Вопросы можно задавать на форуме Hex-Rays. Из сторонних материалов полезно руководство Стива Микаллефа по созданию модулей IDA на C/C++.

Настройка среды сборки

Написать код порой легче, чем собрать модуль: разные компиляторы Windows используют несовместимые форматы библиотек, а поддержать их единым проектом трудно.

Примеры SDK изначально рассчитаны на Borland. Дополнительные `install_xxx.txt` описывают другие компиляторы. Некоторые примеры имеют проекты Visual Studio, а `install_visual.txt` объясняет настройку Visual C++ Express 2005.

Для Unix-подобных средств в Linux или MinGW сценарий `idamake.pl` преобразует make-файлы Borland в обычный формат перед сборкой; процесс описан в `install_linux.txt`.

> **Примечание.** Командные сценарии ожидают переменную среды `IDA`, указывающую на `<SDKDIR>`. Её можно задать глобально либо изменить `<SDKDIR>/allmake.mak` и `<SDKDIR>/allmake.unx`.

Авторы предпочитают для модулей Windows набор MinGW с `gcc` и `make`. Примеры следующих глав включают независимые make-файлы и проекты Visual Studio.

Программный интерфейс IDA

API определён заголовками `<SDKDIR>/include`. Единого указателя функций нет, поэтому ответ на вопрос «как сделать X» приходится искать на форумах или в самих заголовках. Здесь полезны быстрый текстовый поиск и знание подходящих слов.

Можно отталкиваться от знакомых функций IDC, но названия SDK редко совпадают, хотя действия одинаковы. В результате приходится учить два набора. Приложение Б сопоставляет функции IDC с действиями SDK 6.1.

Другой путь - знать назначение заголовков. Связанные функции и структуры объединены по областям. Например, взаимодействие с пользователем находится в `kernwin.hpp`.

Особенности заголовков

Большинство имеет расширение `.hpp`, некоторые `.h`, что легко перепутать. Главный файл `ida.hpp` следует включать во все проекты.

По умолчанию SDK запрещает опасные функции вроде `strcpy` и `sprintf`. Полный список находится в `pro.h`. Для их разрешения нужно определить макрос до `ida.hpp`:

```
#define USE_DANGEROUS_FUNCTIONS
#include <ida.hpp>
```

Иначе при вызове, например, `snprintf` сборка может жаловаться на неизвестный `dont_use_snprintf`. Вместо опасных функций предусмотрены безопасные аналоги `qstrncpy`, `qsnprintf` и другие из `pro.h`.

Также ограничены стандартные потоки и функции файлового ввода-вывода: `stdin`, `stdout`, `fopen`, `fwrite`, `fprintf`. Вместо них есть `qfopen`, `qfprintf` и другие `qXXX`. Чтобы вернуть стандартные, определите `USE_STANDARD_FILE_FUNCTIONS` до включения `fpro.h`.

Комментарии в заголовках описывают назначение, структуры и функции и вместе составляют документацию API.

Основные заголовочные файлы

`area.hpp`. Определяет `area_t`, непрерывный диапазон адресов и базовый класс других диапазонов. Обычно подключается косвенно.

`auto.hpp`. Работа с автоматическим анализатором и его очередями, выполняемыми, когда IDA свободна.

`bytes.hpp`. Чтение и запись байтов базы, их свойства, флаги операндов, обычные и повторяемые комментарии.

`dbg.hpp`. Программное управление отладчиком.

`entry.hpp`. Точки входа; у совместно используемых библиотек ими считаются экспортируемые функции и данные.

`expr.hpp`. Конструкции IDC: изменение и добавление функций IDC, выполнение выражений из модуля.

`fpro.h`. Альтернативные файловые функции `qfopen` и другие.

`frame.hpp`. Работа с кадрами стека.

`funcs.hpp`. Дизассемблированные функции и сигнатуры FLIRT.

`gdl.hpp`. Создание графов DOT и GDL.

`ida.hpp`. Главный заголовок. Содержит `idainfo` и глобальную `inf` со сведениями текущей базы и параметрами настройки.

`idp.hpp`. Основа процессорных модулей, включая глобальные `ph` с описанием процессора и `ash` с описанием ассемблера.

`kernwin.hpp`. Интерфейс с пользователем: аналоги `AskXXX`, позиционирование окна и сочетания клавиш.

`lines.hpp`. Форматированные цветные строки листинга.

`loader.hpp`. Структуры `loader_t` и `plugin_t`, загрузка файлов и запуск подключаемых модулей.

`name.hpp`. Имена обычных адресов. Имена полей структур и кадра описаны в других заголовках.

`netnode.hpp`. Класс `netnode` и низкоуровневое постоянное хранилище базы.

`pro.h`. Общие типы и макросы, включая `IDA_SDK_VERSION` с версии 5.2 для условной сборки под разные комплекты.

`search.hpp`. Различные виды поиска по базе.

`segment.hpp`. `segment_t`, производный от `area_t`, и функции разделов `.text`, `.data` и других.

`struct.hpp`. `struc_t` и работа со структурами базы.

`typeinf.hpp`. Библиотеки типов, сигнатуры функций, результаты и параметры.

`ua.hpp`. Классы `op_t`, `insn_t`, дизассемблирование инструкций и вывод частей строки.

`xref.hpp`. Добавление, удаление и перебор ссылок на код и данные.

Это примерно половина поставляемых заголовков; по мере освоения стоит изучить остальные. Открытые функции API отмечены `ida_export` и только они экспортируются библиотеками компоновки. `idaapi` означает лишь соглашение `stdcall` в Windows и не делает функцию доступной. Интересную функцию без `ida_export` вызвать из модуля нельзя.

Узлы netnode

Большая часть API моделирует понятные элементы файла. Класс `netnode` выглядит загадочнее, потому что не соответствует разделу, функции или инструкции.

Это самое низкоуровневое универсальное хранилище базы, доступное через SDK. Напрямую оно требуется редко: высокоуровневые структуры скрывают, что сохраняют сведения именно здесь. Например, `nalt.hpp` показывает, что импортированные библиотеки и функции находятся в узле `import_node`. Постоянные массивы IDC тоже построены на `netnode`.

Подробности находятся в `netnode.hpp`. Внутреннее устройство скрыто, а класс служит непрозрачной оболочкой. Его единственное поле `netnodenumber` - уникальный целочисленный идентификатор: 32 бита в 32-разрядной системе и 64 бита в 64-разрядной.

Часто номер совпадает с виртуальным адресом, естественно связывая каждый адрес с произвольными сведениями о нём, например текстом комментария.

Рекомендуется работать методами объекта `netnode`, а не свободными функциями из заголовка, хотя многие методы лишь тонко оборачивают их.

Каждый узел может иметь:

• имя до 512 символов;
• основное значение до 1024 байтов;
• 256 разреженных массивов в каждой поддерживаемой категории.

Имя читают `name` и меняют `rename`. Основное значение можно трактовать как целое через `set_long` и `long_value`, строку через `set` и `valstr` либо произвольный двоичный блок через `set` и `valobj`.

Массивы делятся на перекрывающиеся категории:

• с 32-разрядным числовым индексом, более четырёх миллиардов возможных позиций;
• с 8-разрядным индексом, до 256 позиций;
• хеш-таблицы со строковыми ключами.

Каждый элемент содержит до 1024 байтов. Данные хранятся в узлах B-дерева файла ID0, который при закрытии архивируется в IDB. В интерфейсе содержимое не показывается, поэтому это удобное постоянное хранилище результатов модулей и сценариев.

Создание узлов

Само объявление переменной C++ ещё не создаёт внутренний объект. Он появляется, когда узлу назначают имя, основное значение или элемент массива.

```
#ifdef __EA64__
typedef ulonglong nodeidx_t;
#else
typedef ulong nodeidx_t;
#endif

class netnode {
    netnode();
    netnode(nodeidx_t num);
    netnode(const char *name, size_t namlen=0, bool do_create=false);
    bool create(const char *name, size_t namlen=0);
    bool create();
};

netnode n0;
netnode n1(0x00401110);
netnode n2("$ node 2");
netnode n3("$ node 3", 0, true);
```

Листинг 16-1. Объявление узлов `netnode`

Только `n3` гарантированно создаётся. `n1` и `n2` могут уже существовать. В `n1` можно записывать сразу, потому что номер задан. Если имени `$ node 2` ещё нет, `n2` нужно явно создать:

```
if (BADNODE == (nodeidx_t)n2) {
    n2.create("$ node 2");
}
```

Преобразование в `nodeidx_t` возвращает `netnodenumber`. Перед записью номер обязан быть действительным. Он может задаваться явно, как у `n1`, либо назначаться `create` или флагом конструктора. Автоматические номера начинаются с `0xFF000000` и увеличиваются.

Безымянный `n0` можно создать так:

```
n0.create();
```

Записывать в него уже можно, но для будущего поиска придётся сохранить номер или назначить имя. Узел по адресу легко находится через адрес, остальные лучше именовать. Рекомендуемый префикс пользовательских имён - `"$ "`, чтобы не столкнуться с внутренними именами IDA.

Хранение данных

Для элемента массива нужны индекс, разрядность индекса, значение, его размер и один из 256 массивов. Разрядность задаётся выбранной функцией. Массив выбирает тег, обычно символ.

Документация выделяет:

• `altval`, стандартный тег `'A'`;
• `supval`, стандартный тег `'S'`;
• `hashval`, стандартный тег `'H'`;
• `charval`, без стандартного тега.

Тип значения описывает способ доступа, а не жёстко закреплённый массив. Любой вид можно хранить под любым тегом; программист обязан помнить формат каждой позиции.

Целые altval

```
netnode n("$ idabook", 0, true);
sval_t index = 1000;
ulong value = 0x12345678;
n.altset(index, value);
value = n.altval(index);
n.altset(index, value, (char)3);
value = n.altval(index, (char)3);
```

Листинг 16-2. Доступ к целым значениям узла

`XXXset` записывает, `XXXval` читает. Для 8-разрядных индексов используются варианты `_idx8`, которым тег нужно передавать всегда:

```
netnode n("$ idabook", 0, true);
uchar index = 80;
ulong value = 0x87654321;
n.altset_idx8(index, value, 'A');
value = n.altval_idx8(index, 'A');
n.altset_idx8(index, value, (char)3);
value = n.altval_idx8(index, (char)3);
```

Произвольные значения supval

Они содержат от 1 до 1024 байтов. При 32-разрядном индексе стандартный массив `'S'`, но допустим любой тег. Для строк предусмотрено особое поведение.

```
netnode n("$ idabook", 0, true);

char *string_data = "пример строки supval";
char binary_data[] = {0xfe, 0xdc, 0x4e, 0xc7, 0x90, 0x00, 0x13, 0x8a,
                      0x33, 0x19, 0x21, 0xe5, 0xaa, 0x3d, 0xa1, 0x95};

n.supset(1000, binary_data, sizeof(binary_data));
n.supset(1001, string_data);
n.supset(500, binary_data, sizeof(binary_data), (char)200);
```

Листинг 16-3. Запись значений `supval`

`supset` принимает индекс, указатель, длину и необязательный тег. Нулевая или пропущенная длина означает строку: вычисляется `strlen(data) + 1`, и завершающий ноль сохраняется.

При чтении нужно подготовить буфер. Можно всегда выделять 1024 байта либо сначала запросить точный размер. supval предназначена для произвольных данных и возвращает число скопированных байтов. supstr специализируется на строках и возвращает длину без завершающего нуля, хотя сам ноль копируется. При выходном указателе NULL обе функции лишь сообщают необходимый размер, включая завершитель.

```
int len = n.supval(1000, NULL, 0);
char *outbuf = new char[len];
n.supval(1000, outbuf, len);

len = n.supstr(1001, NULL, 0);
char *outstr = new char[len];
n.supval(1001, outstr, len);

char buf[1024];
len = n.supval(500, buf, sizeof(buf), (char)200);
```

Листинг 16-4. Чтение значений supval

Через supval можно получить любые данные любого массива. Например, altset внутри реализована как запись supval размером с целое:

```
bool altset(sval_t alt, nodeidx_t value, char tag=atag) {
    return supset(alt, &value, sizeof(value), tag);
}
```

Хеш- и однобайтовые значения

hashval использует строковый ключ. Перегруженные hashset записывают целое или массив байтов, а hashval, hashstr и hashval_long читают. Тег выбирает одну из 256 таблиц, стандартная - 'H'.

charval и charset работают с одним байтом. Стандартного массива нет, тег обязателен. По сути это оболочки над однобайтовым supval.

Перебор массивов

Для всех видов имеются XXX1st, XXXnxt, XXXlast и XXXprev. Пример прямого и обратного обхода 'A':

```
netnode n("$ idabook", 0, true);

for (nodeidx_t idx = n.alt1st(); idx != BADNODE; idx = n.altnxt(idx)) {
    ulong val = n.altval(idx);
    msg("Найдено altval['A'][%d] = %d\n", idx, val);
}

for (nodeidx_t idx = n.altlast(); idx != BADNODE; idx = n.altprev(idx)) {
    ulong val = n.altval(idx);
    msg("Найдено altval['A'][%d] = %d\n", idx, val);
}
```

Листинг 16-5. Перебор целых значений узла

Другие виды обходятся похоже, но хеш-функции возвращают строковые ключи, по которым затем читаются значения.

Связь с глобальными массивами IDC

CreateArray добавляет к пользовательскому имени префикс $ idc_array и создаёт netnode, чей номер становится идентификатором массива. SetArrayLong пишет в 'A', SetArrayString в 'S', а AR_LONG и AR_STR для GetArrayElement фактически выбирают эти теги.

Удаление данных

Весь узел удаляется так:

```
netnode n("$ idabook", 0, true);
n.kill();
```

У функций удаления похожие имена, поэтому легко стереть лишнее:

```
netnode n("$ idabook", 0, true);
n.altdel(100);            // один элемент стандартного массива A
n.altdel(100, (char)3);   // один элемент массива 3
n.altdel();               // весь стандартный массив A
n.altdel_all('A');        // то же самое
n.altdel_all((char)3);    // весь массив 3
```

Пропуск индекса у altdel удаляет целый массив. Подобные функции существуют для остальных видов.

Полезные типы SDK

area_t **(**area.hpp**).** Диапазон адресов с включительным startEA и исключительным endEA; базовый класс нескольких типов. Методы вычисляют размер и сравнивают области.

func_t **(**funcs.hpp**).** Производный от area_t. Хранит признаки кадра, локальных переменных и параметров. Описывает как функцию, так и её отдельные несмежные фрагменты, называемые хвостами.

segment_t **(**segment.hpp**).** Ещё один диапазон. Хранит имя, права чтения, записи и выполнения, вид сегмента и разрядность адресов 16, 32 или 64.

idc_value_t **(**expr.hpp**).** Значение IDC, способное содержать строку, целое или число с плавающей точкой. Используется при вызове IDC из скомпилированного модуля.

idainfo **(**ida.hpp**).** Характеристики открытой базы. Глобальная inf содержит имя процессорного модуля, формат файла (f_PE, f_MACHO), точку входа beginEA, границы minEA и maxEA, порядок байтов mf и параметры из ida.cfg.

struc_t **(**struct.hpp**).** Описание структуры данных или кадра стека. Флаги показывают структуру или объединение, свёрнутое или развёрнутое состояние; внутри находится массив членов.

member_t **(**struct.hpp**).** Одно поле с начальным и конечным смещением в родительской структуре.

op_t **(**ua.hpp**).** Операнд инструкции. Поле n содержит номер с нуля, type один из optype_t, остальные поля зависят от вида адресации.

insn_t **(**ua.hpp**).** Инструкция с адресом ea, типом itype, длиной size и массивом до шести Operands типа op_t. Для стандартных модулей itype берётся из allins.hpp; у стороннего значения задаёт разработчик. Оно обычно не связано напрямую с двоичным кодом операции.

Это лишь введение в наиболее часто используемые классы, а не полный справочник.

Часто используемые функции SDK

Хотя SDK написан на C++ и определяет классы, объекты базы часто изменяются обычными свободными функциями, принимающими указатель. Ниже приведены аналоги многих средств IDC, но с другими именами.

Основной доступ к базе

В `bytes.hpp` объявлены:

• `get_byte(ea_t addr)`, `get_word`, `get_long` - чтение 1, 2 и 4 байтов;
• `get_many_bytes(ea_t addr, void *buffer, ssize_t len)` - копирование диапазона в буфер;
• `patch_byte`, `patch_word`, `patch_long` - изменение одного значения;
• `patch_many_bytes(ea_t addr, const void *buffer, size_t len)` - изменение диапазона;
• `get_original_byte`, `get_original_word`, `get_original_long` - первоначальные значения до всех исправлений;
• `isLoaded(ea_t addr)` - наличие действительных данных.

«Первоначальное» означает самое первое значение. Если байт менялся дважды, доступны текущий и исходный варианты, но промежуточный после первой правки уже не получить.

Пользовательский интерфейс

Все запросы проходят через диспетчер `callui` с одной из констант `ui_notification_t` и дополнительными параметрами. Удобные оболочки из `kernwin.hpp` скрывают детали:

• `msg(char *format, ...)` печатает форматированный текст;
• `warning(char *format, ...)` показывает диалог;
• `askstr(int hist, char *default, char *format, ...)` запрашивает строку, а `HIST_xxx` задаёт историю;
• `askfile_c(int dosave, char *default, char *prompt, ...)` открывает выбор файла для записи или чтения и возвращает имя либо `NULL`;
• `askyn_c(int default, char *prompt, ...)` задаёт вопрос «да/нет/отмена»;
• `AskUsingForm_c(const char *form, ...)` строит собственный диалог по текстовому описанию формы;
• `get_screen_ea()` возвращает адрес курсора;
• `jumpto(ea_t addr)` переходит к адресу.

API позволяет создавать и более сложные одно- и многостолбцовые списки выбора через `choose` и `choose2`.

Имена

• `get_name(ea_t from, ea_t addr, char *namebuf, size_t maxsize)` копирует имя в буфер и видит локальное имя, если `from` относится к той же функции;
• `set_name(ea_t addr, char *name, int flags)` назначает имя с флагами из `name.hpp`;
• `get_name_ea(ea_t funcaddr, char *localname)` ищет локальное имя и возвращает адрес либо `BADADDR`.

Функции и кадры

Из funcs.hpp и frame.hpp:

• func_t *get_func(ea_t addr) возвращает функцию, содержащую адрес;
• size_t get_func_qty() число функций;
• func_t *getn_func(size_t n) функцию по индексу;
• func_t *get_next_func(ea_t addr) следующую функцию;
• get_func_name(ea_t addr, char *name, size_t namesize) копирует имя;
• struc_t *get_frame(ea_t addr) возвращает кадр как структуру.

Структуры

struc_t используется и для обычных типов, и для кадров. Многие функции принимают идентификатор tid_t:

• get_struc_id(char *name) находит идентификатор по имени;
• get_struc(tid_t id) возвращает указатель на структуру;
• get_struc_size(struc_t *s) размер;
• get_member(struc_t *s, asize_t offset) поле по смещению;
• get_member_by_name(struc_t *s, char *name) поле по имени;
• add_struc(uval_t index, char *name, bool is_union=false) создаёт структуру в указанной позиции или в конце при BADADDR;
• add_struc_member(...) добавляет поле по смещению либо в конец.

У add_struc_member флаги FF_XXX из bytes.hpp задают тип, typeinfo_t содержит сведения сложного типа и может быть NULL для простого, а size указывает число байтов.

Сегменты

• getseg(ea_t addr) возвращает сегмент адреса;
• get_segm_by_name(char *name) сегмент по имени;
• add_segm(ea_t para, ea_t start, ea_t end, char *name, char *sclass) создаёт диапазон [start, end) с классом CODE, DATA или другим; para задаёт базу при сегментированной адресации, иначе равен 0;
• add_segm_ex(segment_t *s, char *name, char *sclass, int flags) создаёт сегмент из заполненного объекта и флагов ADDSEG_XXX;
• get_segm_qty() и getnseg(int n) перечисляют сегменты;
• set_segm_name меняет имя;
• get_segm_name копирует отфильтрованное имя;
• get_true_segm_name копирует точное имя без замены запрещённых символов.

Простое создание и заполнение segment_t не добавляет раздел в базу; нужно вызвать add_segm. То же относится к оболочкам func_t и struc_t: устойчивые изменения выполняются специальными функциями создания, изменения или удаления.

Перекрёстные ссылки

Для кода в xref.hpp имеются:

• get_first_cref_from и get_next_cref_from - исходящие потоки;

• get_first_cref_to и get_next_cref_to - входящие потоки.

Для данных:

• get_first_dref_from и get_next_dref_from - исходящие обращения;
• get_first_dref_to и get_next_dref_to - входящие обращения.

При завершении все возвращают BADADDR. Прямого аналога XrefType нет: внутренний lastXR не экспортируется. Для точного вида ссылки применяется xrefblk_t, показанный ниже.

Приёмы перебора через API

Функции

Первый способ похож на IDC:

```
for (func_t *f = get_next_func(0); f != NULL; f = get_next_func(f->startEA)) {
    char fname[1024];
    get_func_name(f->startEA, fname, sizeof(fname));
    msg("%08x: %s\n", f->startEA, fname);
}
```

Второй использует индексы:

```
for (int idx = 0; idx < get_func_qty(); idx++) {
    char fname[1024];
    func_t *f = getn_func(idx);
    get_func_name(f->startEA, fname, sizeof(fname));
    msg("%08x: %s\n", f->startEA, fname);
}
```

Третий работает с блоком управления областями areacb_t. Глобальная funcs содержит все func_t:

```
int a = funcs.get_next_area(0);
while (a != -1) {
    char fname[1024];
    func_t *f = (func_t*)funcs.getn_area(a);
    get_func_name(f->startEA, fname, sizeof(fname));
    msg("%08x: %s\n", f->startEA, fname);
    a = funcs.get_next_area(f->startEA);
}
```

Похожая глобальная segs хранит указатели segment_t всех разделов.

Поля кадра стека

```
func_t *func = get_func(get_screen_ea());
msg("Размер локальных переменных: %d\n", func->frsize);
msg("Размер сохранённых регистров: %d\n", func->frregs);
struc_t *frame = get_frame(func);
if (frame) {
    size_t ret_addr = func->frsize + func->frregs;
    for (size_t m = 0; m < frame->memqty; m++) {
        char fname[1024];
        get_member_name(frame->members[m].id, fname, sizeof(fname));
        if (frame->members[m].soff < func->frsize) msg("Локальная переменная ");
        else if (frame->members[m].soff > ret_addr) msg("Параметр ");
        msg("%s находится по смещению %x\n", fname, frame->members[m].soff);
        if (frame->members[m].soff == ret_addr)
```

```
            msg("%s является сохранённым адресом возврата\n", fname);
    }
}
```

Листинг 16-6. Перебор полей кадра стека

`frsize` и `frregs` задают размеры локальной области и сохранённых регистров. Сразу после них находится адрес возврата. `memqty` равно числу полей и длине массива `members`; по `soff` поле классифицируется.

Ссылки с определением вида

Простой перебор входящих ссылок почти решает задачу поиска вызывающих функций, но не сообщает, является ли связь вызовом. Для этого нужен `xrefblk_t`:

```
struct xrefblk_t {
    ea_t from;
    ea_t to;
    uchar iscode;
    uchar type;
    uchar user;

    bool first_from(ea_t from, int flags);
    bool next_from(void);
    bool first_to(ea_t to, int flags);
    bool next_to(void);
};
```

Поля содержат источник, цель, признак кода, точный вид и признак пользовательской ссылки. Методы `first_*` начинают обход, `next_*` продолжают.

Флаги:

```
#define XREF_ALL  0x00 // все ссылки
#define XREF_FAR  0x01 // без обычного последовательного потока
#define XREF_DATA 0x02 // только данные
```

Отдельного флага «только код» нет. Нужно проверить `iscode` либо сравнить `type` с `fl_JN`, `fl_CN` и другими.

Исправленная функция поиска вызовов:

```
void list_callers(char *bad_func) {
    char name_buf[MAXNAMELEN];
    ea_t func = get_name_ea(BADADDR, bad_func);
    if (func == BADADDR) {
        warning("%s не найдена в базе", bad_func);
    }
    else {
        xrefblk_t xr;
        for (bool ok = xr.first_to(func, XREF_ALL); ok; ok = xr.next_to()) {
            if (xr.type != fl_CN && xr.type != fl_CF) continue;
            char *name = get_func_name(xr.from, name_buf, sizeof(name_buf));
            if (name)
                msg("%s вызывается из 0x%x в %s\n", bad_func, xr.from, name);
            else
                msg("%s вызывается из 0x%x\n", bad_func, xr.from);
        }
    }
}
```

Теперь `type` позволяет оставить только ближние и дальние вызовы. Одного `iscode` недостаточно, поскольку он истинен также для переходов и обычного потока.

Итоги

Рассмотренные структуры и функции лишь обозначают поверхность API. В каждой области имеются более специализированные средства и гораздо более тонкое управление базой, чем через IDC. Следующие главы посвящены подключаемым, загрузочным и процессорным модулям и продолжают раскрывать возможности SDK.

Глава 17. Архитектура подключаемых модулей IDA

В следующих главах рассматриваются модули, создаваемые с помощью SDK, а также появившаяся с IDA 5.7 возможность писать те же виды расширений на языках сценариев. Даже если собственный модуль не планируется, понимание архитектуры полезно: большая часть сторонних средств для IDA распространяется именно так.

Подключаемый модуль можно считать скомпилированным и более мощным аналогом сценария. Обычно он связан с клавишей или меню и доступен после открытия базы. Он бывает общим для разных форматов и архитектур либо строго специализированным. Скомпилированный модуль имеет полный доступ к API и выполняет гораздо более сложные задачи.

Написание подключаемого модуля

Все модули IDA являются совместно используемыми библиотеками своей платформы. Экспортировать функции не требуется. Вместо этого каждый вид экспортирует переменную определённого класса. Для подключаемого модуля это `plugin_t` из `loader.hpp`.

Начиная с SDK 4.9 Hex-Rays старается сохранять двоичную совместимость, поэтому старый модуль часто можно просто скопировать в новую IDA. При этом API растёт, иногда функции устаревают. Они помещаются под проверку `NO_OBSOLETE_FUNCS`. Определите этот макрос до заголовков, чтобы сборка выявляла применение устаревших символов.

Класс `plugin_t` не имеет методов:

```
class plugin_t {
public:
    int version;
    int flags;
    int (idaapi* init)(void);
    void (idaapi* term)(void);
    void (idaapi* run)(int arg);
    char *comment;
    char *help;
    char *wanted_name;
    char *wanted_hotkey;
};
```

Каждый модуль экспортирует объект с именем `PLUGIN`; `loader.hpp` оформляет экспорт, а автор объявляет и инициализирует объект.

Поля plugin_t

`version`. Версия API, обычно `IDP_INTERFACE_VERSION` из `idp.hpp`. Значение не менялось с SDK 4.9.

`flags`. Битовая комбинация `PLUGIN_XXX` из `loader.hpp`, определяющая поведение. Многим модулям достаточно 0.

`init`. Указатель на функцию инициализации без параметров, возвращающую целое. IDA вызывает её, предлагая модулю загрузиться.

`term`. Функция выгрузки без параметров и результата. Она освобождает память, закрывает дескрипторы и сохраняет состояние. При отсутствии действий допустим `NULL`.

`run`. Основное действие при запуске из меню, клавишей или сценарием. Получает одно целое и ничего не возвращает.

`comment`. Строка комментария, непосредственно IDA не используется и может быть `NULL`.

`help`. Многострочная справка, также может быть `NULL`.

`wanted_name`. Короткое имя для `Edit > Plugins`. Уникальность не обязательна, но одинаковые имена создают неоднозначность.

`wanted_hotkey`. Предпочитаемое сочетание, например `"Alt-F8"`. При конфликте оно назначается последнему запросившему модулю; пользователь может переопределить настройку.

Пример:

```
int idaapi idabook_plugin_init(void);
void idaapi idabook_plugin_term(void);
void idaapi idabook_plugin_run(int arg);

char idabook_comment[] = "Пример подключаемого модуля";
char idabook_name[] = "Idabook";
char idabook_hotkey[] = "Alt-F9";

plugin_t PLUGIN = {
    IDP_INTERFACE_VERSION,
    0,
    idabook_plugin_init,
    idabook_plugin_term,
    idabook_plugin_run,
    idabook_comment,
    NULL,
    idabook_name,
    idabook_hotkey
};
```

Указатели позволяют IDA найти функции без экспорта и без обязательных имён.

Жизненный цикл

Есть три момента возможной загрузки:

1. Сразу при запуске IDA, даже без базы, если в `PLUGIN.flags` есть `PLUGIN_FIX`.
2. После процессорного модуля и до его выгрузки, если есть `PLUGIN_PROC`.
3. При каждом открытии базы, если предыдущие флаги отсутствуют.

IDA вызывает `PLUGIN.init`, которая проверяет текущее состояние: формат файла, архитектуру и другие условия. Она возвращает:

• `PLUGIN_SKIP` - не загружать;
• `PLUGIN_OK` - сделать доступным для текущей базы и загрузить при первом запуске пользователем;
• `PLUGIN_KEEP` - сделать доступным и удерживать в памяти.

Загруженный модуль запускается пользователем через меню или клавишу, тогда вызывается `PLUGIN.run`. Другой способ - подписаться на события IDA и получать вызовы обратной связи.

При выгрузке вызывается ненулевая `PLUGIN.term`. Модуль без специальных флагов выгружается вместе с базой. `PLUGIN_UNL` требует выгружать после каждого `run`, поэтому следующий запуск снова вызывает `init`. `PLUGIN_PROC` выгружается с процессорным модулем при закрытии базы. `PLUGIN_FIX` остаётся до завершения IDA.

Инициализация

Статическая часть выполняется при компиляции через поля PLUGIN, динамическая - в init.

При запуске IDA проверяет все файлы <IDADIR>/plugins и вызывает init у PLUGIN_FIX. Они загружаются раньше других модулей и могут видеть события загрузчиков и процессоров. Обычно такая init возвращает PLUGIN_OK или PLUGIN_KEEP. Для одноразового действия можно выполнить его прямо в init и вернуть PLUGIN_SKIP.

После загрузки процессорного модуля IDA вызывает init у всех PLUGIN_PROC. Глобальная ph позволяет проверить архитектуру:

```
int idaapi mips_init() {
    if (ph.id != PLFM_MIPS) return PLUGIN_SKIP;
    return PLUGIN_OK;
}
```

Наконец, при загрузке базы вызывается init остальных модулей. Они могут проверять ELF, PE, Mach-O, компилятор и процессор.

При успешном результате init должна выполнить разовую подготовку, а term освободить полученные ресурсы. PLUGIN_KEEP предотвращает повторное выделение и подходит, если следующие запуски зависят от накопленного состояния. Альтернатива - хранить состояние в netnode, сохраняя его даже между сеансами. Если каждый запуск независим, PLUGIN_OK экономит память.

Уведомления о событиях

Для автоматической реакции модуль регистрирует функцию обратного вызова через hook_to_notification_point:

```
typedef int idaapi hook_cb_t(void *user_data, int notification_code, va_list va);

bool hook_to_notification_point(
    hook_type_t hook_type,
    hook_cb_t *callback,
    void *user_data);

int idaapi idabook_plugin_init() {
    hook_to_notification_point(HT_IDB, idabook_database_cb, NULL);
}
```

Четыре крупные категории:

• процессорные события HT_IDP;
• пользовательский интерфейс HT_UI;
• отладчик HT_DBG;
• база HT_IDB.

Внутри каждой есть коды, например idb_event::byte_patched и idb_event::cmt_changed. IDA передаёт код и дополнительные параметры, описанные в соответствующем заголовке.

```
int idabook_database_cb(void *user_data, int notification_code, va_list va) {
    ea_t addr;
    ulong original, current;
    switch (notification_code) {
        case idb_event::byte_patched:
            addr = va_arg(va, ea_t);
            current = get_byte(addr);
            original = get_original_byte(addr);
            msg("%x изменён на %x, исходное значение %x\n",
```

```
                addr, current, original);
            break;
    }
    return 0;
}
```

`va_list` содержит разное число аргументов; для `byte_patched` это один `ea_t`, извлекаемый `va_arg`.

При выгрузке подписку обязательно снять:

```
void idaapi idabook_plugin_term() {
    unhook_from_notification_point(HT_IDB, idabook_database_cb, NULL);
}
```

Иначе IDA вскоре вызовет функцию уже выгруженной библиотеки и аварийно завершится.

Выполнение модуля

Все функции вызываются из главного цикла событий. Пока модуль работает, IDA не обрабатывает интерфейс и очередь анализа. Поэтому нужно быстро вернуть управление. Иначе остаётся ждать либо принудительно завершать процесс с риском повредить открытую базу.

Для долгой операции вызовите `show_wait_box`, периодически проверяйте `wasBreak` и в конце обязательно `hide_wait_box`. Проверка одновременно даёт IDA возможность обновить интерфейс.

Не следует просто переносить работу `run` в новый поток. IDA не потокобезопасна: глобальные данные и транзакции базы не защищены, параллельное изменение может повредить базу.

Минимальный `run`:

```
void idaapi idabook_plugin_run(int arg) {
    msg("Модуль idabook запущен!\n");
}
```

Модулю доступны C++, API IDA и библиотеки платформы, например программный интерфейс Windows. Для полезного поведения нужно подобрать соответствующие функции SDK и реализовать их вызовы в `run` либо обработчиках событий.

В качестве примера на основе листинга 16-6 можно написать такую функцию:

```
void idaapi extended_plugin_run(int arg) {
    func_t *func = get_func(get_screen_ea()); // функция в позиции курсора
    msg("Размер локальных переменных: %d\n", func->frsize);
    msg("Размер сохранённых регистров: %d\n", func->frregs);
    struc_t *frame = get_frame(func);         // указатель на кадр стека
    if (frame) {
        size_t ret_addr = func->frsize + func->frregs; // смещение адреса возврата
        for (size_t m = 0; m < frame->memqty; m++) {  // перебор элементов
            char fname[1024];
            get_member_name(frame->members[m].id, fname, sizeof(fname));
            if (frame->members[m].soff < func->frsize) {
                msg("Локальная переменная ");
            }
            else if (frame->members[m].soff > ret_addr) {
                msg("Параметр ");
            }
            msg("%s находится по смещению %x в кадре\n",
                fname, frame->members[m].soff);
            if (frame->members[m].soff == ret_addr) {
                msg("%s - сохранённый адрес возврата\n", fname);
            }
        }
    }
}
```

Это ядро модуля, который при каждом запуске выводит сведения о кадре стека выбранной функции.

Сборка модулей

В Windows модуль представляет собой допустимую динамическую библиотеку с расширением `.plw` или `.p64`. В Linux это разделяемый объект `.plx` или `.plx64`, а в macOS - `.pmc` или `.pmc64`. Сборка требует точных параметров: ошибка почти в любом из них приводит к неудаче.

В SDK есть несколько примеров со своими файлами `makefile`, изначально рассчитанными на средства сборки Borland для Windows. Для другой цепочки инструментов или другой платформы это неудобно. В документах `install_xxx.txt` описан сценарий `<SDKDIR>/bin/idamake.pl`, который преобразует такие файлы в формат GNU Make, а затем запускает GNU Make и GCC.

Мы предпочитаем упрощённый файл сборки для инструментов GNU, в Windows работающих через MinGW. Листинг 17-1 легко приспособить к собственному проекту:

```
# Укажите здесь каталог SDK
IDA_SDK=../../

PLATFORM=$(shell uname | cut -f 1 -d _)

ifneq "$(PLATFORM)" "MINGW32"
IDA=$(HOME)/ida
endif

# Укажите желаемое имя собранного модуля
PROC=idabook_plugin

ifeq "$(PLATFORM)" "MINGW32"
PLATFORM_CFLAGS=-D__NT__ -D__IDP__ -DWIN32 -Os -fno-rtti
PLATFORM_LDFLAGS=-shared -s
LIBDIR=$(shell find ../../ -type d | grep -E "(lib|lib/)gcc.w32")
ifeq ($(strip $(LIBDIR)),)
LIBDIR=../../lib/x86_win_gcc_32
endif
IDALIB=$(LIBDIR)/ida.a
PLUGIN_EXT=.plw

else ifeq "$(PLATFORM)" "Linux"
PLATFORM_CFLAGS=-D__LINUX__
PLATFORM_LDFLAGS=-shared -s
IDALIB=-lida
IDADIR=-L$(IDA)
PLUGIN_EXT=.plx

else ifeq "$(PLATFORM)" "Darwin"
PLATFORM_CFLAGS=-D__MAC__
PLATFORM_LDFLAGS=-dynamiclib
IDALIB=-lida
IDADIR=-L$(IDA)/idaq.app/Contents/MacOs
PLUGIN_EXT=.pmc
endif

# Параметры компилятора для платформы
CFLAGS=-Wextra -Os $(PLATFORM_CFLAGS)

# Параметры компоновщика для платформы
LDFLAGS=$(PLATFORM_LDFLAGS)

# Дополнительные необходимые библиотеки
EXTRALIBS=
# Каталог назначения для собранных модулей
```

```
OUTDIR=$(IDA_SDK)bin/plugins/

# Перечень объектных файлов проекта
OBJS=idabook_plugin.o

BINARY=$(OUTDIR)$(PROC)$(PLUGIN_EXT)

all: $(OUTDIR) $(BINARY)

clean:
 -@rm *.o
 -@rm $(BINARY)

$(OUTDIR):
 -@mkdir -p $(OUTDIR)

CC=g++
INC=-I$(IDA_SDK)include/

%.o: %.cpp
 $(CC) -c $(CFLAGS) $(INC) $< -o $@

LD=g++

$(BINARY): $(OBJS)
 $(LD) $(LDFLAGS) -o $@ $(OBJS) $(IDADIR) $(IDALIB) $(EXTRALIBS)

# Замените idabook_plugin именем модуля и добавьте его зависимости
idabook_plugin.o: idabook_plugin.cpp
```

Листинг 17-1. Упрощённый файл сборки модуля IDA

Команда `uname` определяет платформу, после чего выбираются соответствующие параметры. Дополнительные исходные файлы добавляют как имена объектных файлов в переменную `OBJS` и в конец файла. Необходимые библиотеки перечисляют в `EXTRALIBS`.

Переменная `IDA_SDK` задаёт путь к `<SDKDIR>` - абсолютный либо относительный. В примере каталог SDK находится на два уровня выше каталога модуля, поскольку проект размещён в `<SDKDIR>/plugins/idabook_plugin`. При ином размещении значение следует изменить. Собранный модуль сохраняется в `<SDKDIR>/bin/plugins`, однако успешная сборка ещё не означает установку.

Сборка модулей средствами Microsoft Visual C++ Express описана в `install_visual.txt`. Чтобы создать проект с нуля в Visual Studio 2008, выполните следующие действия.

1. Выберите **File > New > Project**. Откроется окно, показанное на рисунке 17-1.

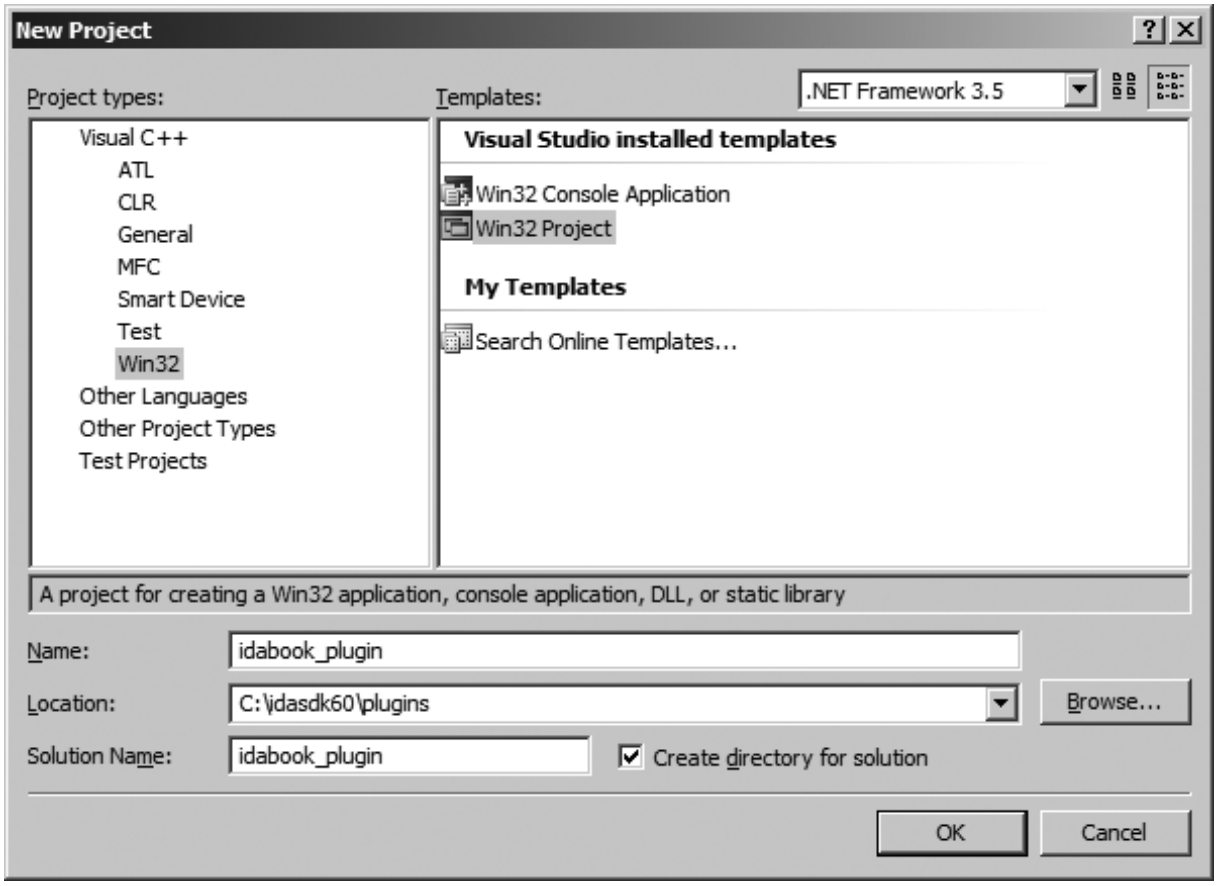


Рисунок 17-1. Создание проекта в Visual Studio

2. Укажите тип проекта **Visual C++/Win32**, выберите шаблон **Win32 Project**, задайте имя и расположение. Обычно проекты модулей удобно хранить в `<SDKDIR>/plugins`. Нажмите **OK**, а в появившемся мастере - **Next**. На странице **Application Settings** выберите тип **DLL** и дополнительный параметр **Empty project**, затем нажмите **Finish**, как показано на рисунке 17-2.

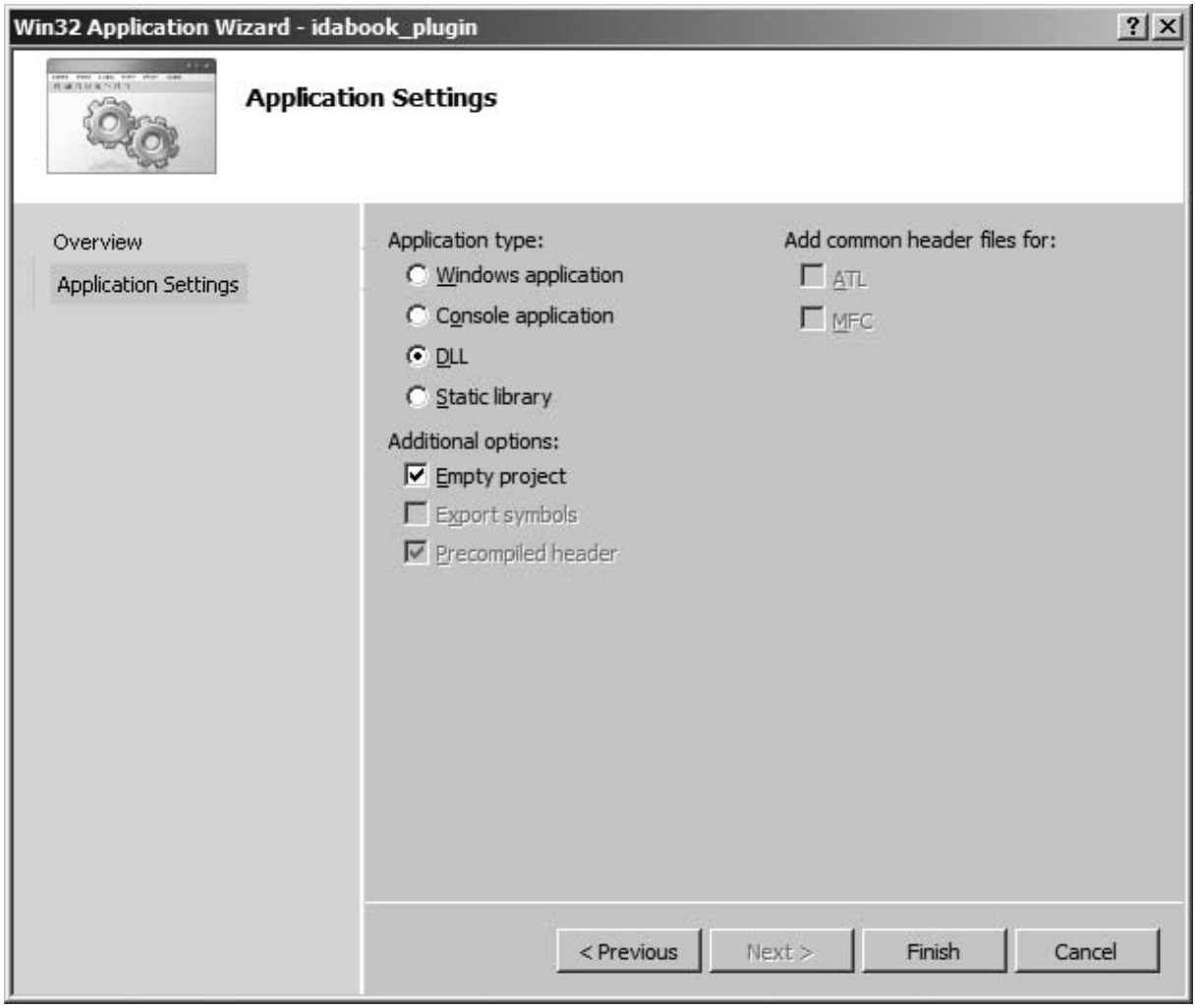


Рисунок 17-2. Мастер приложения Win32 в Visual Studio

3. После создания основы проекта настройте дополнительные параметры. В Visual Studio 2008 они открываются командой **Project > Properties**. Параметры C/C++ появляются только после добавления в проект исходного файла - нового или уже существующего.

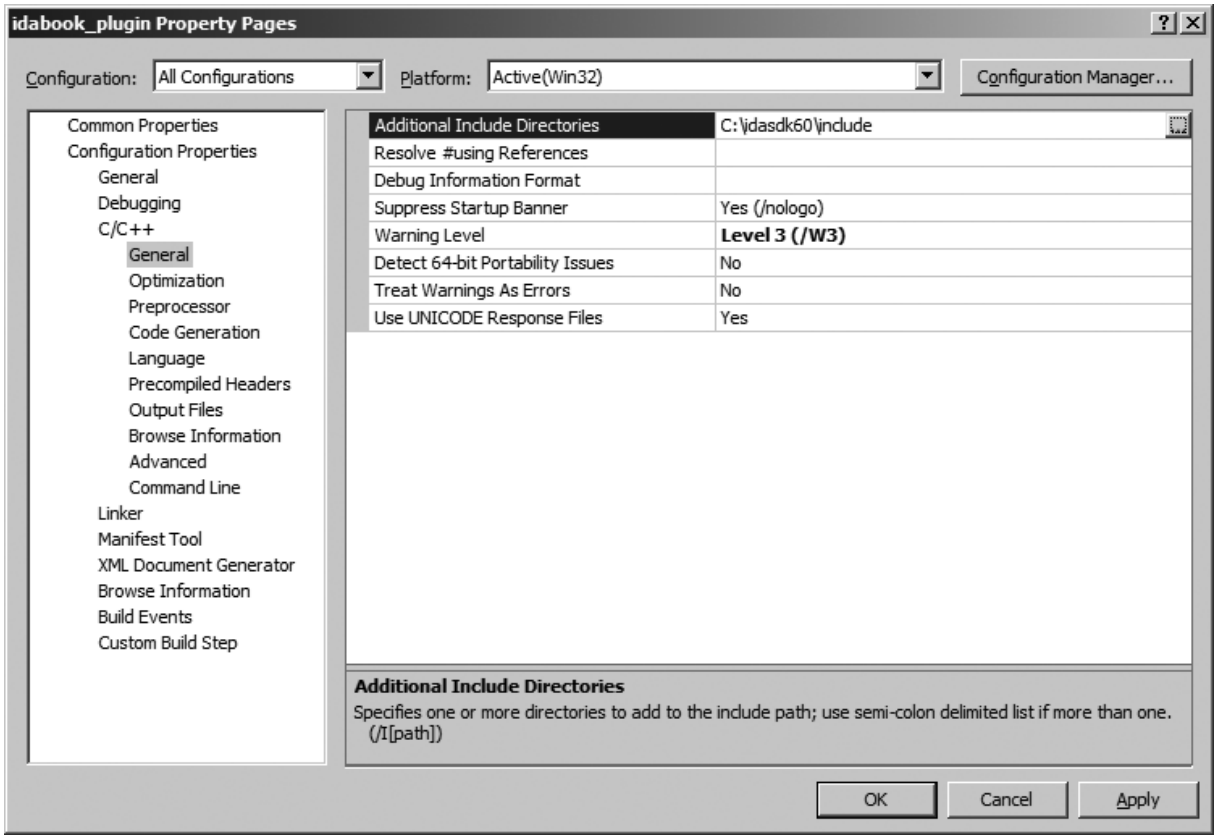


Рисунок 17-3. Свойства проекта Visual Studio

Параметры распределены по разделу **Configuration Properties** в левой части окна. При выборе категории справа отображается список её свойств. Категории образуют иерархию, а значения изменяются через окна выбора файлов, однострочные и многострочные поля либо раскрывающиеся списки.

Visual Studio позволяет задать разные параметры для вариантов **Debug** и **Release**. Если нужны оба варианта, измените свойства в каждой конфигурации. Чтобы применить их сразу ко всем, выберите **All Configurations** в списке **Configurations**.

Категория	Свойство	Значение
General	**Output Directory**	По выбору, обычно `<SDKDIR>\bin\plugins`
C/C++ > General	**Additional Include Directories**	Добавить `<SDKDIR>\include`
C/C++ > Preprocessor	**Preprocessor Definitions**	Дописать `;__NT__;__IDP__`
C/C++ > Code Generation	**Runtime Library**	**Multithreaded** для выпуска, **Multithreaded Debug** для отладки; не варианты DLL[1]
Linker > General	**Output File**	Заменить расширение на `.plw`
Linker > General	**Additional Library Directories**	Добавить `<SDKDIR>\lib\x86_win_vc_32`[2]

Категория	Свойство	Значение
Linker > Input	**Additional Dependencies**	Добавить `ida.lib` из `lib\x86_win_vc_32`
Linker > Command Line	**Additional options**	Добавить `/EXPORT:PLUGIN`

Таблица 17-1. Параметры 32-разрядного модуля в Visual Studio

Установка модулей

По сравнению со сборкой установка проста: собранный файл копируют в `<IDADIR>/plugins`.

Windows не позволяет перезаписать используемый исполняемый файл. Поэтому прежнюю версию необходимо сначала выгрузить из IDA. В зависимости от параметров загрузки это может произойти при закрытии базы, но модуль с флагом `PLUGIN_FIX` способен потребовать полного завершения IDA.

В Linux и macOS используемый исполняемый файл можно перезаписать, поэтому предварительная выгрузка не нужна. Однако новая версия будет загружена только тогда, когда IDA в следующий раз выполнит загрузку модулей.

Некоторые модули распространяются только в двоичном виде, другие - вместе с исходным кодом. Обычно нужно выбрать сборку, соответствующую версии IDA, и скопировать её в `<IDADIR>/plugins`.

Обязательно прочитайте сопроводительную документацию, если она есть: некоторым модулям для правильной работы нужны дополнительные компоненты.

Настройка модулей

IDA предоставляет ограниченные возможности настройки через файл `<IDADIR>/plugins/plugins.cfg`. В нём можно задать:

• другое название пункта меню вместо поля `wanted_name`;
• нестандартное расположение или расширение файла модуля;
• другую либо дополнительную клавишу запуска вместо `wanted_hotkey`;
• целое число, передаваемое функции `PLUGIN.run` при запуске;
• необязательный флаг `DEBUG` для модулей отладчика, рассматриваемых в главе 24.

Синтаксис строки описан в самом `plugins.cfg`. Несколько примеров:

```
; Точка с запятой начинает комментарий. Строка состоит
; из трёх обязательных и двух необязательных частей:
; имя_модуля файл_модуля клавиша [целый аргумент run] [DEBUG]
The_IdaBook_Plugin   idabook_plugin   Alt-F2  1
IdaBook_Plugin_Alt   idabook_plugin   Alt-F3  2
```

Поля `wanted_name` и `wanted_hotkey` выбирает автор, и они встраиваются в модуль при сборке. У модулей разных авторов могут совпасть имена либо сочетания клавиш.

Поле `plugin_name` задаёт надпись, добавляемую в меню **Edit > Plugins** вместо `PLUGIN.wanted_name`. Одному модулю можно назначить несколько имён, а следовательно,

несколько пунктов меню. Перед добавлением в меню знаки подчёркивания заменяются пробелами.

Поле `plugin_file` определяет файл собранного модуля. При полном пути IDA загружает файл оттуда, иначе ищет его в `<IDADIR>/plugins`. Если расширение не указано, используется стандартное для текущей платформы; указанное расширение требует точного совпадения имени.

Поле `hotkey` задаёт сочетание клавиш и заменяет `PLUGIN.wanted_hotkey`. Так можно устранить конфликт между двумя модулями или предоставить несколько способов запуска одного модуля.

Разным клавишам полезно назначать разные целые аргументы `PLUGIN.run`: модуль узнает, каким способом его вызвали, и выбирает соответствующее действие. В приведённом примере при запуске `idabook_plugin` сочетанием `Alt-F3` функция `PLUGIN.run` получает число 2.

Расширение IDC

До сих пор модули в основном изменяли базу или извлекали из неё сведения. Теперь расширим возможности языка сценариев IDC.[3] Поскольку IDC реализован поверх API IDA, этот программный интерфейс позволяет добавлять в язык новые функции.

Как объяснялось в главах 15 и 16, глобальные массивы IDC - ограниченное представление сетевых узлов. При создании массива задаётся имя, а возвращается его идентификатор. Внутри к имени добавляется префикс `$ idc_array` , а идентификатором фактически служит индекс сетевого узла.

Если индекс уже известен, его можно передать функциям массива IDC. Но доступ по имени невозможен из-за автоматически добавляемого префикса. Исправить это позволяет функция `set_idc_func_ex` из SDK.

Она объявлена в `expr.hpp` и связывает новую функцию IDC с её реализацией на C++:

```
typedef error_t (idaapi *idc_func_t)(idc_value_t *argv, idc_value_t *res);
bool set_idc_func_ex(const char *idc_name, idc_func_t idc_impl,
                     const char *args, int extfunc_flags);
```

Тип `idc_func_t` введён здесь лишь для краткости и в SDK отсутствует. Аргументы задают имя новой функции IDC, указатель на её реализацию, завершённый нулём массив типов аргументов и флаги. Флаги указывают, требуется ли открытая база и может ли функция не возвращать управление.

Завершим регистрацию в функции инициализации модуля:

```
int idaapi init(void) {
    static const char idc_str_args[] = { VT_STR2, 0 };
    set_idc_func_ex("CreateNetnode", idc_create_netnode, idc_str_args, 0);
    return PLUGIN_KEEP;
}
```

Так создаётся функция IDC `CreateNetnode`, связанная с `idc_create_netnode`. Она принимает один строковый параметр типа `VT_STR2`.

Ниже приведена сама реализация:

```
/*
 * Встроенная реализация CreateNetnode. Возвращает идентификатор
 * нового сетевого узла, пригодный для всех функций массивов IDC.
 */
static error_t idaapi idc_create_netnode(idc_value_t *argv, idc_value_t *res)
{
    res->vtype = VT_LONG;                  // результат - индекс сетевого узла
    if (argv[0].vtype == VT_STR2) {        // проверка типа аргумента
        netnode n(argv[0].c_str(), 0, true); // создание сетевого узла
        res->num = (nodeidx_t)n;           // значение результата
    }
```

```
    else {
        res->num = -1;                    // недопустимый аргумент
    }
    return eOk;
}
```

argv содержит аргументы CreateNetnode, в данном случае один, а через res возвращается результат. Тип SDK idc_value_t представляет одно значение IDC: его поля хранят текущий тип и содержимое.

Сначала функция объявляет целочисленный результат VT_LONG. Переменные IDC не имеют постоянного типа, поэтому текущий тип приходится указывать явно. Затем проверяется строковый аргумент VT_STR2, создаётся сетевой узел с заданным именем, а его индекс записывается в res->num. Строковый результат пришлось бы задавать методом res->set_string. При неверном типе аргумента возвращается недопустимый индекс -1.

Остальные функции и структура модуля выглядят так:

```
void idaapi term(void) {}   // при завершении ничего не требуется
void idaapi run(int arg) {} // запуск пользователем невозможен

plugin_t PLUGIN = {
    IDP_INTERFACE_VERSION,
    // Модуль загружается при запуске IDA, скрыт из меню Edit > Plugins
    // и изменяет базу данных.
    PLUGIN_FIX | PLUGIN_HIDE | PLUGIN_MOD,
    init, // инициализация
    term, // завершение; допускается NULL
    run,  // запуск модуля
    "",  // длинное описание
    "",  // многострочная справка
    "",  // предпочтительное короткое имя
    ""   // предпочтительная клавиша запуска
};
```

Благодаря PLUGIN_FIX модуль загружается вместе с IDA, а PLUGIN_HIDE не добавляет его в меню **Edit > Plugins**. Он остаётся в памяти для всех баз, вся работа выполняется в init, поэтому методу run делать нечего.

После установки модуль позволяет обращаться из IDC к любому именованному сетевому узлу:

```
auto n, val;
n = CreateNetnode("$ imports");       // префикс $ idc_array не добавляется
val = GetArrayElement(AR_STR, n, 0);  // получение нулевого элемента
```

Дополнительные сведения о взаимодействии SDK с IDC находятся в expr.hpp.

Возможности пользовательского интерфейса

Модулю нередко приходится запрашивать у пользователя сведения или показывать результат. Помимо функций семейства askXXX, упомянутых в главе 16, API IDA предлагает более сложные средства взаимодействия.

Модули для графической версии IDA могут напрямую обращаться и к средствам интерфейса Qt либо Windows. Это позволяет использовать почти любые графические элементы.

Однако после простых функций askXXX задача усложняется. Одна из причин в том, что SDK пытается предоставить единый программный интерфейс для довольно сложного отображения графических элементов и получения пользовательского ввода.

Окна выбора SDK

Сначала рассмотрим функции `choose` и `choose2`, объявленные вместе с управляющими константами в `kernwin.hpp`. Обе показывают список данных и предлагают выбрать один или несколько элементов. Данные могут быть практически любого типа: вызывающая сторона предоставляет функции форматирования, создающие строки окна.

`choose` показывает один столбец, а `choose2` - несколько. Далее используются простейшие варианты с параметрами по умолчанию; полный набор возможностей описан в `kernwin.hpp`.

После исключения параметров по умолчанию простейшая форма `choose` имеет вид:

```
ulong choose(void *obj,
             int width,
             ulong (idaapi *sizer)(void *obj),
             char *(idaapi *getline)(void *obj, ulong n, char *buf),
             const char *title);
```

`obj` указывает на отображаемый блок данных, а `width` задаёт ширину столбца. Функция `sizer` разбирает данные и возвращает число строк. Функция `getline` создаёт строковое представление одного элемента.

Указатель `obj` может вести к данным любого типа, если `sizer` умеет подсчитать элементы, а `getline` - найти элемент по целому индексу и представить его строкой. Параметр `title` задаёт заголовок окна. `choose` возвращает индекс выбранного элемента от 1 до *n* либо ноль, если пользователь отменил выбор. Листинг 17-2 взят из простого демонстрационного модуля.

```
#include <kernwin.hpp>

// Демонстрационные данные
int data[] = {0xdeafbeef, 0xcafebabe, 0xfeedface, 0};

// Ожидается, что obj указывает на завершённый нулём
// массив ненулевых целых чисел.
ulong idaapi idabook_sizer(void *obj) {
    int *p = (int*)obj;
    int count = 0;
    while (*p++) count++;
    return count;
}

/*
 * obj указывает на массив целых чисел.
 * n задаёт форматируемую строку (1..n), а ноль означает заголовок.
 * buf указывает на выходной буфер размером MAXSTR (1024).
 */
char * idaapi idabook_getline(void *obj, ulong n, char *buf) {
    int *p = (int*)obj;
    if (n == 0) {
        qstrncpy(buf, "Значение", strlen("Значение") + 1);
    }
    else {
        qsnprintf(buf, 32, "0x%08.8x", p[n - 1]);
    }
    return buf;
}

void idaapi run(int arg) {
    int choice = choose(data, 16, idabook_sizer, idabook_getline,
                        "Выбор Idabook");
    msg("Пользователь выбрал строку %d\n", choice);
}
```

Листинг 17-2. Пример применения функции `choose`

При запуске этого модуля появляется окно с рисунка 17-4.

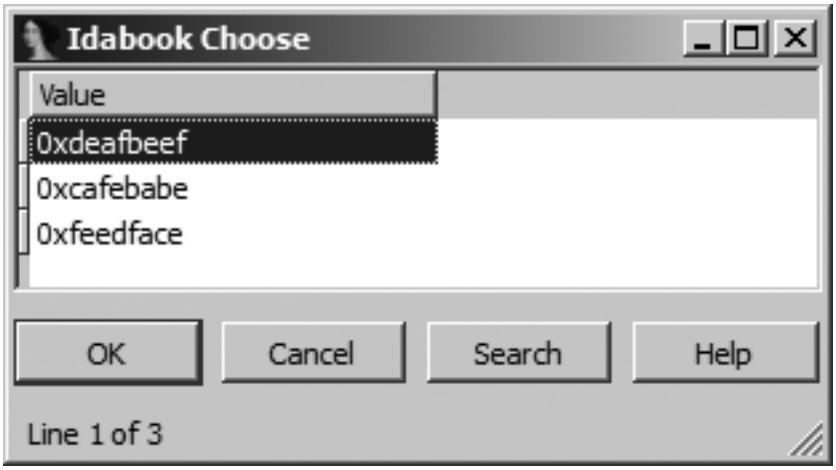


Рисунок 17-4. Пример окна выбора

Функция `choose2` создаёт многостолбцовое окно. Её простейшая форма с параметрами по умолчанию выглядит так:

```
ulong choose2(void *obj,
              int ncol,
              const int *widths,
              ulong (idaapi *sizer)(void *obj),
              void (idaapi *getline)(void *obj, ulong n, char* const *cells),
              const char *title);
```

`ncol` задаёт число столбцов, а массив `widths` - ширину каждого из них. Поскольку строка содержит несколько столбцов, `getline` должна заполнить данные для всех столбцов этой строки.

```
#include <kernwin.hpp>

// Демонстрационные данные
int data[] = {0xdeafbeef, 0xcafebabe, 0xfeedface, 0};
// Ширина каждого столбца
int widths[] = {16, 16, 16};
// Заголовки столбцов
char *headers[] = {"Десятичное", "Шестнадцатеричное", "Восьмеричное"};
// Строки формата для столбцов
char *formats[] = {"%d", "0x%x", "0%o"};

// obj указывает на завершённый нулём массив ненулевых целых чисел.
ulong idaapi idabook_sizer(void *obj) {
    int *p = (int*)obj;
    int count = 0;
    while (*p++) count++;
    return count;
}

/*
 * obj указывает на массив целых чисел.
 * n задаёт форматируемую строку (1..n), а ноль означает заголовок.
 * cells - массив указателей, по одному на столбец. Вывод каждого
 * столбца не должен превышать MAXSTR (1024) символа.
 */
void idaapi idabook_getline_2(void *obj, ulong n, char* const *cells) {
    int *p = (int*)obj;
    if (n == 0) {
        for (int i = 0; i < 3; i++) {
            qstrncpy(cells[i], headers[i], widths[i]);
        }
    }
    else {
        for (int i = 0; i < 3; i++) {
            qsnprintf(cells[i], widths[i], formats[i], p[n - 1]);
        }
    }
```

```
}

void run(int arg) {
    int choice = choose2(data, 3, widths, idabook_sizer,
                         idabook_getline_2, "Выбор Idabook 2");
    msg("Выбрана строка %d\n", choice);
}
```

Листинг 17-3. Пример применения функции `choose2`

Результат показан на рисунке 17-5.

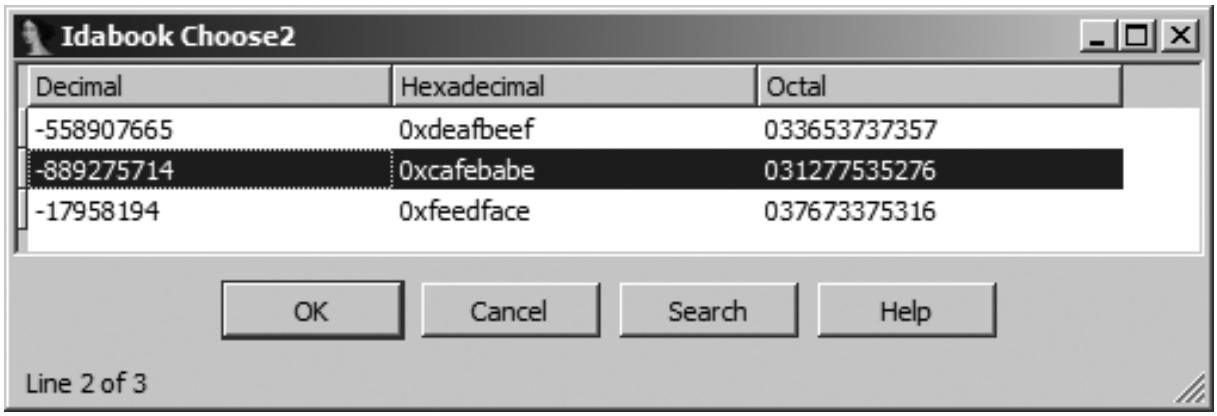


Рисунок 17-5. Пример окна `choose2`

Обе функции допускают гораздо более сложное применение. Они создают модальные[4] и немодальные окна, поддерживают выбор нескольких элементов и принимают обработчики различных событий.

Немодальное окно отображается как новая вкладка рядом с другими окнами IDA, например **Imports**. Само окно импорта IDA реализовано через `choose2`. Остальные возможности описаны в `kernwin.hpp`.

Создание настраиваемых форм средствами SDK

Для более сложных элементов SDK предоставляет функцию:

```
int AskUsingForm_c(const char *form, ...);
```

Несмотря на простой прототип, это одна из самых сложных интерфейсных функций SDK. Параметр `form` описывает расположение элементов в создаваемом окне.

Подобно `printf`, функция принимает строку формата. Но здесь она содержит как спецификаторы вывода, так и спецификаторы полей формы, которые при показе превращаются в элементы ввода. Набор спецификаторов отличается от `printf` и полностью описан в `kernwin.hpp`.

Основная форма спецификатора поля:

```
<#текст подсказки#метка:тип:ширина:ширина_на_экране:@hlp[]>
```

Его части означают следующее:

• `#текст подсказки#` - необязательная всплывающая подсказка без окружающих знаков #;
• `метка` - постоянный текст слева от поля, а для кнопки - надпись на ней;
• `тип` - один знак, определяющий разновидность поля;
• `ширина` - наибольшее число вводимых символов, а для кнопки - целый код, отличающий её от других;
• `ширина_на_экране` - видимая ширина поля;

• @hlp[] - номер раздела файла IDA.HLP; поскольку содержимое задаёт Hex-Rays, это поле обычно бесполезно, и вместо него оставляют двоеточие.

Знак типа определяет создаваемый элемент и требуемый аргумент в переменной части вызова:

Тип	Поле	Аргумент va_list
A	строка ASCII	char*, размер не меньше MAXSTR
S	сегмент	sel_t*
N	шестнадцатеричное число в записи C	uval_t*
n	знаковое шестнадцатеричное число в записи C	sval_t*
L	число в стандартной системе счисления, обычно шестнадцатеричной, в записи C	ulonglong*
l	знаковый вариант предыдущего поля	longlong*
M	шестнадцатеричное число без префикса 0x	uval_t*
D	десятичное число	sval_t*
O	восьмеричное число в записи C	sval_t*
Y	двоичное число с префиксом 0b	sval_t*
H	символьное значение в записи C	sval_t*
$	адрес	ea_t*
I	идентификатор	char*, размер не меньше MAXNAMELEN
B	кнопка	функция обратного вызова formcb_t
K	кнопка выбора цвета	bgcolor_t*
C	флажок	ushort*, битовая маска отмеченных пунктов

Тип	Поле	Аргумент `va_list`
`R`	переключатель	`ushort*`, номер выбранного пункта

Все числовые поля воспринимают введённое значение как выражение IDC, которое разбирается и вычисляется после нажатия **OK**. Каждому полю соответствует указатель, используемый и для ввода, и для вывода. При создании формы начальные значения считываются по указателям, а при возврате туда записываются значения пользователя.

Для кнопки `B` передаётся адрес функции, вызываемой при нажатии:

```
// Обработчик кнопок
typedef void (idaapi *formcb_t)(TView *fields[], int code);
```

Аргумент `code` равен коду кнопки, указанному в позиции ширины. Одна функция с оператором `switch` может обрабатывать несколько кнопок.

Для переключателей и флажков используется сокращённый синтаксис:

```
<#подсказка пункта#метка:тип>
```

Последовательные спецификаторы образуют группу, а последний завершается дополнительным знаком `>`:

```
<#подсказка пункта#метка:тип>>
```

Группа обводится рамкой. Заголовок и подсказку рамки задают в первом элементе:

```
<#подсказка пункта#заголовок#подсказка рамки#метка:тип>
```

Если подсказки не нужны, остаётся более короткая запись:

```
<##заголовок##метка:тип>
```

На рисунке 17-6 показана форма, которая будет разбираться далее.

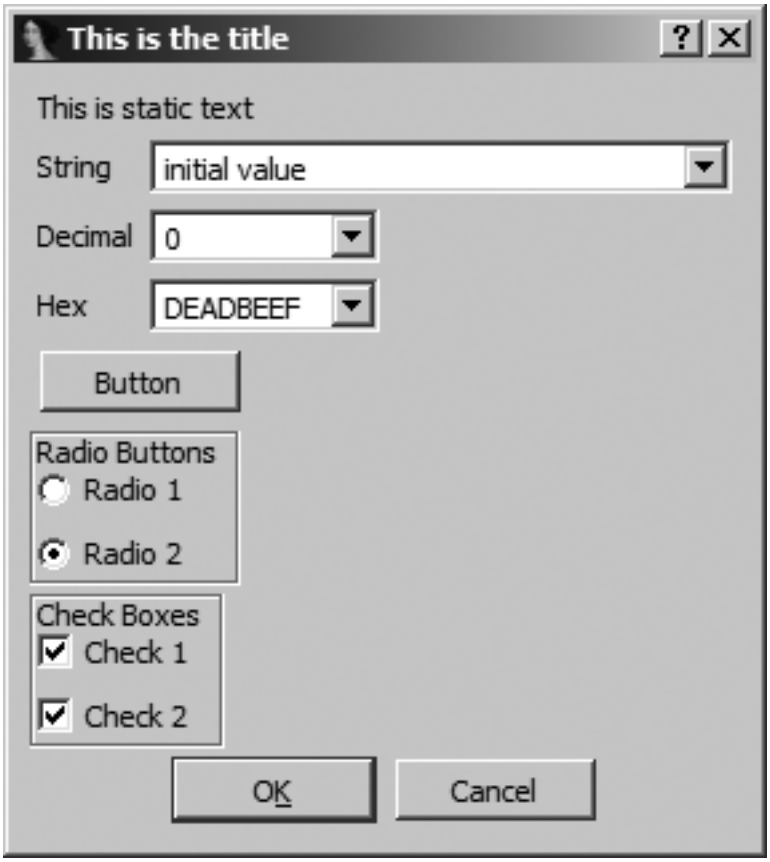


Рисунок 17-6. Пример окна `AskUsingForm_c`

Строка формата состоит из отдельных строк, каждая из которых описывает часть окна. Помимо полей она может содержать постоянный текст, выводимый буквально. Заголовок формы должен завершаться двумя переводами строки. Директивы поведения, например `STARTITEM`, задают начальные параметры: эта директива выбирает поле, которое первым получит фокус.

```
char *dialog =
    "STARTITEM 0\n"             // первое поле получает фокус
    "Заголовок формы\n\n"       // после заголовка два перевода строки
    "Постоянный текст\n"
    "<Строка:A:32:32::>\n"      // поле ASCII, требуется char[MAXSTR]
    "<Десятичное:D:10:10::>\n" // десятичное поле, sval_t*
    "<#Без начального 0x#Шестнадцатеричное:M:8:10::>\n"
    "<Кнопка:B::::>\n"          // кнопка без кода, formcb_t
    "<##Переключатели##Пункт 1:R>\n"
    "<Пункт 2:R>>\n"            // последний переключатель группы
                                  // ushort*: номер выбранного пункта
    "<##Флажки##Флажок 1:C>\n"   // флажок с заголовком рамки
    "<Флажок 2:C>>\n";           // последний флажок группы
                                  // ushort*: битовая маска
```

Размещение одного элемента в строке облегчает сопоставление спецификаторов с рисунком 17-6. Текстовые и числовые поля выглядят как раскрывающиеся списки: IDA запоминает недавно введённые значения соответствующего типа и предлагает их повторно.

Следующий код показывает форму и обрабатывает результат:

```
void idaapi button_func(TView *fields[], int code) {
    msg("Кнопка нажата!\n");
}

void idaapi run(int arg) {
    char input[MAXSTR];
    sval_t dec = 0;
    uval_t hex = 0xdeadbeef;
    ushort radio = 1;      // изначально выбран пункт 1
```

```
    ushort checkmask = 3;  // изначально отмечены оба флажка
    qstrncpy(input, "начальное значение", sizeof(input));
    if (AskUsingForm_c(dialog, input, &dec, &hex,
                       button_func, &radio, &checkmask) == 1) {
        msg("Введённая строка: %s\n", input);
        msg("Десятичное: %d, шестнадцатеричное: %x\n", dec, hex);
        msg("Выбран переключатель %d\n", radio);
        for (int n = 0; checkmask; n++) {
            if (checkmask & 1) {
                msg("Флажок %d отмечен\n", n);
            }
            checkmask >>= 1;
        }
    }
}
```

При обработке переключателей и флажков первый пункт каждой группы имеет номер ноль. `AskUsingForm_c` предоставляет значительные возможности, а остальные её параметры и режимы подробно описаны в `kernwin.hpp`.

Создание интерфейса только для Windows

Разработчики модулей не раз сталкивались с трудностями при создании интерфейса. Для модуля, рассчитанного только на графическую версию IDA под Windows (`idag.exe`), доступен весь графический API Windows.

Автор модуля mIDA компании Tenable Security предложил отдельный подход к созданию дочерних окон MDI.[5] Сложности разработки и пример решения обсуждались в форуме поддержки IDA.

Модуль ida-x86emu использует другой подход. Дескриптор главного окна IDA можно получить так:

```
HWND mainWindow = (HWND)callui(ui_get_hwnd).vptr;
```

Используя это окно как родительское, ida-x86emu не пытается встраиваться в рабочее пространство IDA. Его диалоговые окна создаются редактором ресурсов Windows, а взаимодействие обрабатывается прямыми вызовами API Windows.

Сочетание графического редактора окон и непосредственного обращения к системному API даёт самые широкие возможности, но увеличивает сложность: разработчику нужно обрабатывать сообщения Windows и знать низкоуровневые интерфейсные функции.

Создание интерфейса с Qt

Появившийся в IDA 6.0 интерфейс Qt позволил создавать сложные модули, работающие на всех платформах IDA. Даниэль Пистелли из Hex-Rays описал основные требования в публикации компании.

Для применения Qt сначала нужно правильно настроить среду разработки. IDA 6.1 поставлялась с библиотеками Qt 4.7.2,[6] которые Hex-Rays помещала в пространство имён C++ `QT`. Необходимо взять соответствующие исходники Qt. Версия `idaq` для Windows собиралась Visual Studio 2008, а версии для Linux и macOS - компилятором `g++`.

Ключевой параметр настройки исходников:

```
-qtnamespace QT
```

Он помещает код Qt в пространство имён `QT`.

Для сборки модуля Qt в Windows нужны библиотеки компоновки `.lib` для каждой используемой библиотеки. IDA включает ряд динамических библиотек Qt, но SDK содержит лишь ограниченный набор библиотек компоновки, в частности QtCore4 и QtGui, в `<SDKDIR>/lib/x86_win_qt`. Остальные придётся собрать самостоятельно из исходников Qt.

В Linux можно напрямую компоновать библиотеки из `<IDADIR>`, а в macOS - из `<IDADIR>/idaq.app/Contents/Frameworks`. Зависимость от библиотек, не входящих в поставку IDA, снижает переносимость модуля, если не распространять их вместе с ним.

Файл проекта `qmake` должен содержать директиву:

```
QT_NAMESPACE = QT
```

В SDK давно существуют безопасные строковые функции, например `qstrlen` и `qsnprintf`. Qt определяет одноимённые функции, поэтому возникает конфликт: варианты IDA находятся в глобальном пространстве имён, а варианты Qt - в `QT`. Явно вызвать функцию IDA можно так:

```
unsigned int len = ::qstrlen(myString);
```

Если создаваемому элементу нужен родитель, указатель на окно приложения верхнего уровня возвращает такой вызов:

```
QWidget *mainWindow = QApplication::activeWindow();
```

Это статический метод класса `QApplication`, возвращающий указатель на элемент единственного объекта `QApplication` приложения Qt.

Дополнительный пример находится в поставляемом с SDK модуле `qwindow`. Он создаёт пустой элемент через `create_tform`, получает уведомление о показе формы, извлекает её указатель `QWidget` и добавляет кнопку Qt. Рассматриваемые в главе 23 модули collabREate и ida-x86emu также используют Qt, чтобы работать на всех поддерживаемых IDA платформах.

Модули в виде сценариев

В IDA 5.6 появились модули загрузки в виде сценариев, а в 5.7 - модули расширения и процессорные модули такого типа. Они не обязательно мощнее скомпилированных, но снижают порог входа и ускоряют разработку, поскольку сложная сборка не требуется.

Сценарный модуль можно написать на IDC или Python. Обычно удобнее Python, поскольку он открывает значительную часть SDK IDA. Поэтому такой модуль способен быть не менее мощным, чем скомпилированный модуль C++.

Для Python нужно определить функцию `PLUGIN_ENTRY`, возвращающую экземпляр `plugin_t` из модуля `idaapi`. Поля этого класса соответствуют полям структуры C++ `plugin_t`.

```
from idaapi import *

class idabook_plugin_t(plugin_t):
    flags = 0
    wanted_name = "Модуль IdaBook на Python"
    wanted_hotkey = "Alt-8"
    comment = "Модуль IdaBook на Python"
    help = "Полезная справка"

    def init(self):
        msg("Вызвана инициализация модуля IdaBook.\n")
        return PLUGIN_OK

    def term(self):
        msg("Вызвано завершение модуля IdaBook.\n")
    def run(self, arg):
```

```
        warning("Вызван запуск модуля IdaBook с аргументом %d.\n" % arg)

def PLUGIN_ENTRY():
    return idabook_plugin_t()
```

Листинг 17-4. Минимальный модуль на Python

Для установки сценарий копируют в `<IDADIR>/plugins`.

В IDC нет базового класса модуля, поэтому класс должен самостоятельно определить все ожидаемые элементы под правильными именами:

```
#include <idc.idc>

class idabook_plugin_t {

    idabook_plugin_t() {
        this.flags = 0;
        this.wanted_name = "Модуль IdaBook на IDC";
        this.wanted_hotkey = "Alt-9";
        this.comment = "Модуль IdaBook на IDC";
        this.help = "Полезная справка";
    }

    init() {
        Message("Вызвана инициализация модуля IdaBook.\n");
        return PLUGIN_OK;
    }

    term() {
        Message("Вызвано завершение модуля IdaBook.\n");
    }

    run(arg) {
        Warning("Вызван запуск модуля IdaBook с аргументом %d.\n", arg);
    }
}

static PLUGIN_ENTRY() {
    return idabook_plugin_t();
}
```

Листинг 17-5. Минимальный модуль на IDC

Как и в примере Python, `PLUGIN_ENTRY` создаёт и возвращает экземпляр класса. Для установки файл `.idc` копируют в `<IDADIR>/plugins`.

Итоги

Модули расширения - следующий логичный шаг, когда сценариев недостаточно для развития возможностей IDA. Правда, с появлением модулей в виде сценариев обращаться к SDK приходится реже.

Если не требуется разбирать неизвестный IDA формат файла или машинный язык без готового процессорного модуля, это может оказаться единственным видом расширения IDA, который понадобится разработчику. Тем не менее следующие две главы продолжат знакомство с SDK и рассмотрят два других вида модулей: загрузчики и процессорные модули.

Сноски

[1] Здесь слово **Multithreaded** относится к самой библиотеке времени выполнения C++. IDA остаётся однопоточным приложением, использующим эту библиотеку. Выбор варианта DLL потребовал бы наличия `MSVCR80.DLL` на целевой системе. Статическая компоновка с вариантом без DLL снимает это ограничение и делает модуль переносимее.
[2] В SDK старше версии 6.1 использовался каталог `<SDKDIR>\lib\vc.w32`.
[3] В рассматриваемой версии нет программного способа расширить API IDAPython из скомпилированного модуля.
[4] Модальное окно необходимо закрыть, прежде чем продолжить работу с родительским приложением. Типичные примеры - окна открытия и сохранения файла. Немодальное окно разрешает взаимодействовать с приложением, пока остаётся открытым.
[5] Интерфейс нескольких документов MDI в Windows позволяет размещать несколько дочерних окон внутри одного окна-контейнера.
[6] IDA 6.0 использовала Qt 4.6.3. В Windows модуль с Qt следовало собирать Visual Studio, как и саму IDA.

Глава 18. Двоичные файлы и модули загрузки IDA

Однажды окружающие узнают, что вы стали местным знатоком IDA. Возможно, это покажется признанием, а возможно, с этого дня вас начнут постоянно отвлекать вопросами о назначении очередного файла. Рано или поздно один из таких вопросов либо привычка открывать в IDA всё подряд приведёт к окну, показанному на рисунке 18-1.

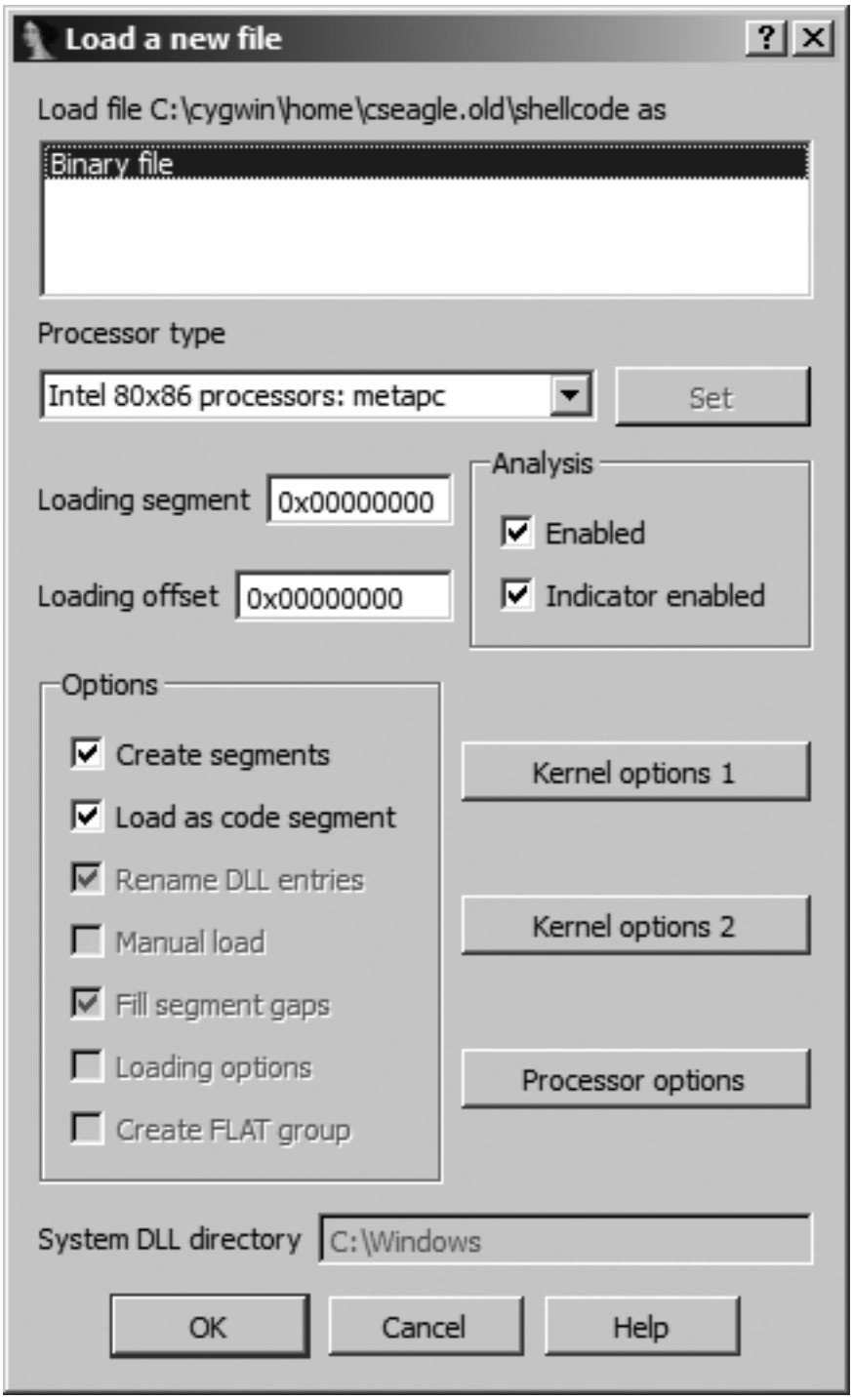


Рисунок 18-1. Загрузка двоичного файла

Это обычное окно загрузки IDA, но список распознанных типов содержит лишь пункт **Binary file**. Значит, ни один установленный модуль загрузки не распознал формат. Остаётся надеяться, что известен хотя бы машинный язык, и осмысленно выбрать тип процессора - других полезных

решений на этом этапе почти нет.

В этой главе сначала рассматривается ручной анализ неизвестных двоичных форматов, а затем на его основе разрабатывается собственный модуль загрузки IDA.

Анализ неизвестного файла

Исполняемый код хранится в бесчисленном множестве форматов. IDA поставляется с загрузчиками распространённых разновидностей, но охватить постоянно растущее число форматов невозможно.

Двоичный образ может быть исполняемым файлом операционной системы, образом ПЗУ встроенного устройства, микропрограммой из пакета обновления флеш-памяти или простым блоком машинного кода, например извлечённым из перехваченных сетевых пакетов. Формат определяется операционной системой, целевым процессором и устройством либо вообще отсутствует, как у кода эксплуатации уязвимости, внедрённого в данные прикладного уровня.

Если для содержащегося в файле кода существует процессорный модуль, задача аналитика - правильно разместить образ в базе IDA, а затем указать, какие области являются кодом, а какие данными.

Для большинства процессоров загрузка в двоичном режиме просто помещает всё содержимое в один сегмент с адреса ноль:

```
seg000:00000000                 db  4Dh ; M
seg000:00000001                 db  5Ah ; Z
seg000:00000002                 db  90h
seg000:00000003                 db    0
seg000:00000004                 db    3
seg000:00000005                 db    0
seg000:00000006                 db    0
seg000:00000007                 db    0
```

Листинг 18-1. Начальные строки файла PE, загруженного в двоичном режиме

Иногда процессорный модуль выполняет часть дизассемблирования сам. Так бывает у встроенных микроконтроллеров, для которых можно сделать определённые предположения о расположении памяти в образе ПЗУ.

Для работы с неизвестным файлом следует собрать все доступные материалы: руководство по процессору и операционной системе, документацию устройства, схему памяти, полученную отладкой или аппаратным анализом, например логическим анализатором.

В качестве примера предположим, что IDA не распознаёт формат PE Windows. Он хорошо известен и подробно документирован, поэтому произвольный файл PE сравнительно легко разобрать вручную.

Ручная загрузка файла PE Windows

Документация формата значительно облегчает отображение файла в базу IDA. Согласно спецификации PE, допустимый файл начинается с заголовка MS-DOS. Его первые два байта содержат сигнатуру `4Dh 5Ah`, то есть `MZ`, которая видна в листинге 18-1.

Далее требуется знать устройство заголовка MS-DOS. Четырёхбайтовое значение по смещению `0x3C` указывает на следующий необходимый заголовок - PE. Поля MS-DOS можно представить отдельными значениями нужного размера либо определить структуру `IMAGE_DOS_HEADER` и

применить её к данным. Второй подход даёт примерно такой результат:

```
seg000:00000000                dw 5A4Dh          ; e_magic
seg000:00000000                dw 90h            ; e_cblp
seg000:00000000                dw 3              ; e_cp
seg000:00000000                dw 0              ; e_crlc
seg000:00000000                dw 4              ; e_cparhdr
seg000:00000000                dw 0              ; e_minalloc
seg000:00000000                dw 0FFFFh          ; e_maxalloc
seg000:00000000                dw 0              ; e_ss
seg000:00000000                dw 0B8h            ; e_sp
seg000:00000000                dw 0              ; e_csum
seg000:00000000                dw 0              ; e_ip
seg000:00000000                dw 0              ; e_cs
seg000:00000000                dw 40h            ; e_lfarlc
seg000:00000000                dw 0              ; e_ovno
seg000:00000000                dw 4 dup(0)       ; e_res
seg000:00000000                dw 0              ; e_oemid
seg000:00000000                dw 0              ; e_oeminfo
seg000:00000000                dw 0Ah dup(0)     ; e_res2
seg000:00000000                dd 80h            ; e_lfanew
```

Поле `e_lfanew` равно `80h`, поэтому заголовок PE должен находиться по смещению `80h`, или 128 байт. Там обнаруживается сигнатура `50h 45h`, то есть `PE`, после чего по этому адресу можно применить созданную по спецификации структуру `IMAGE_NT_HEADERS`:

```
seg000:00000080        dd 4550h        ; Signature
seg000:00000080        dw 14Ch         ; FileHeader.Machine
seg000:00000080        dw 4            ; FileHeader.NumberOfSections
seg000:00000080        dd 47826AB4h    ; FileHeader.TimeDateStamp
seg000:00000080        dd 0E00h        ; FileHeader.PointerToSymbolTable
seg000:00000080        dd 0FBh         ; FileHeader.NumberOfSymbols
seg000:00000080        dw 0E0h         ; FileHeader.SizeOfOptionalHeader
seg000:00000080        dw 307h         ; FileHeader.Characteristics
seg000:00000080        dw 10Bh         ; OptionalHeader.Magic
seg000:00000080        db 2            ; OptionalHeader.MajorLinkerVersion
seg000:00000080        db 38h          ; OptionalHeader.MinorLinkerVersion
seg000:00000080        dd 600h         ; OptionalHeader.SizeOfCode
seg000:00000080        dd 400h         ; OptionalHeader.SizeOfInitializedData
seg000:00000080        dd 200h         ; OptionalHeader.SizeOfUninitializedData
seg000:00000080        dd 1000h        ; OptionalHeader.AddressOfEntryPoint
seg000:00000080        dd 1000h        ; OptionalHeader.BaseOfCode
seg000:00000080        dd 0            ; OptionalHeader.BaseOfData
seg000:00000080        dd 400000h      ; OptionalHeader.ImageBase
seg000:00000080        dd 1000h        ; OptionalHeader.SectionAlignment
seg000:00000080        dd 200h         ; OptionalHeader.FileAlignment
```

В отличие от разбора в главе 8, здесь заголовки необходимы для понимания всей базы, поскольку загрузчик PE не помогал.

Поле `Machine` определяет целевой процессор. Значение `14Ch` означает x86. При другом значении, например `1C0h` для ARM, пришлось бы закрыть базу и начать загрузку заново с правильным процессорным модулем: изменить тип процессора уже загруженной базы нельзя.

Поле `ImageBase` задаёт базовый виртуальный адрес образа. Командой **Edit > Segments > Rebase Program** можно назначить первый сегменту новый адрес, как на рисунке 18-2.

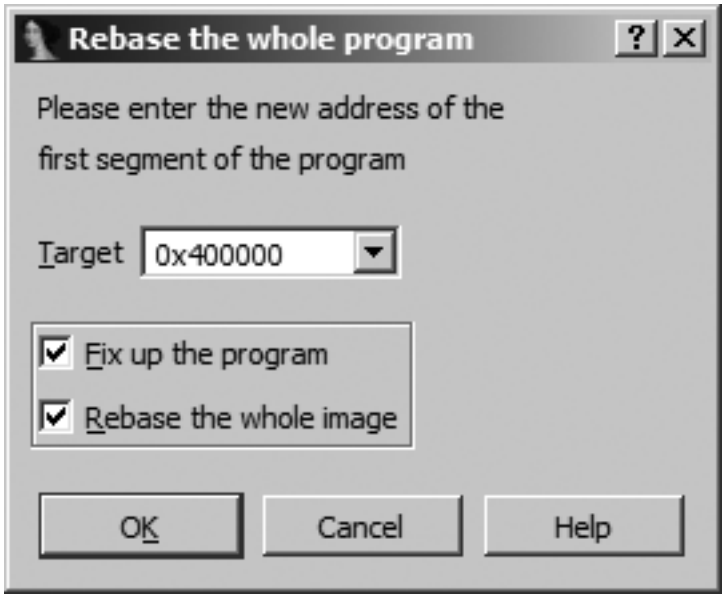


Рисунок 18-2. Новый базовый адрес программы

Сейчас существует только один сегмент, поскольку двоичная загрузка помещает в него весь файл. Два флажка управляют исправлением перемещаемых адресов и переносом всех сегментов. Сведений о перемещениях у IDA нет, а единственный сегмент и так означает перенос всего образа.

Поле `AddressOfEntryPoint` задаёт относительный виртуальный адрес точки входа. Такой адрес отсчитывается от базы образа, а точка входа указывает на первую исполняемую инструкцию. Значение `1000h` при базе `400000h` даёт виртуальный адрес `401000h`. Это первое указание на расположение кода, но сначала нужно правильно отобразить остальные данные по виртуальным адресам.

Формат PE описывает отображение содержимого файла в память с помощью разделов. Их заголовки позволяют построить основную схему виртуальной памяти. Поле `NumberOfSections` сообщает число разделов, здесь четыре. Сразу после `IMAGE_NT_HEADERS` расположен массив структур `IMAGE_SECTION_HEADER`; их можно определить в окне **Structures** и четыре раза применить к следующим байтам.

Поля `FileAlignment` и `SectionAlignment` задают выравнивание данных раздела в файле и памяти соответственно.[1] В примере разделы начинаются в файле со смещений, кратных `200h`, а в памяти - с адресов, кратных `1000h`. Меньшее файловое выравнивание экономит место, а большее выравнивание в памяти обычно соответствует размеру страницы виртуальной памяти операционной системы.

После оформления заголовков разделов данных достаточно для создания дополнительных сегментов. Применение `IMAGE_SECTION_HEADER` сразу после `IMAGE_NT_HEADERS` раскрывает первый заголовок:

```
seg000:00400178                db '.text',0,0,0  ; Name
seg000:00400178                dd 440h           ; VirtualSize
seg000:00400178                dd 1000h          ; VirtualAddress
seg000:00400178                dd 600h           ; SizeOfRawData
seg000:00400178                dd 400h           ; PointerToRawData
seg000:00400178                dd 0              ; PointerToRelocations
seg000:00400178                dd 0              ; PointerToLinenumbers
seg000:00400178                dw 0              ; NumberOfRelocations
seg000:00400178                dw 0              ; NumberOfLinenumbers
seg000:00400178                dd 60000020h      ; Characteristics
```

Поле `Name` показывает, что заголовок относится к разделу `.text`. Для размещения важны три поля. `PointerToRawData` со значением `400h` задаёт смещение содержимого в файле; оно кратно

файловому выравниванию 200h. Разделы PE упорядочены по возрастанию файловых смещений и виртуальных адресов. Поэтому первые 400h байт относятся к заголовкам. Формально они не образуют раздел, но логически связаны, и их удобно объединить в сегмент базы.

Сегмент вручную создаётся командой **Edit > Segments > Create Segment**.

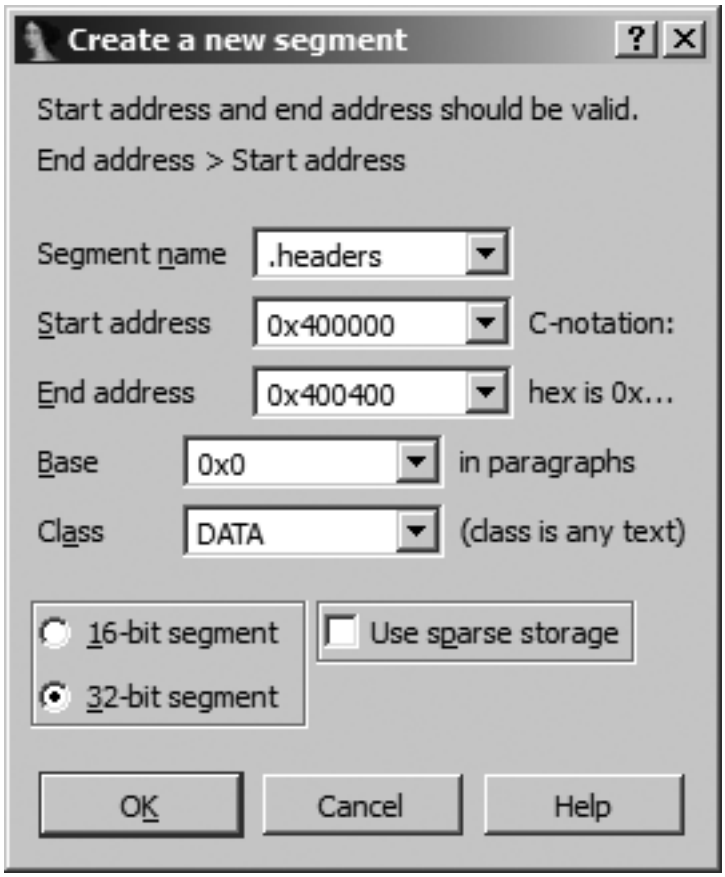


Рисунок 18-3. Создание сегмента

Имя может быть произвольным. Здесь выбрано .headers: оно описывает содержимое и вряд ли совпадёт с настоящим разделом файла. Начальный адрес включается в диапазон, конечный не включается. Их можно ввести вручную или заранее выделить нужные адреса.

База сегмента описана в segment.hpp. Для x86 IDA вычисляет виртуальный адрес байта, сдвигая базу на четыре бита влево и прибавляя смещение:

```
виртуальный адрес = (база << 4) + смещение
```

Если сегментация не применяется, база должна быть нулевой. Класс характеризует содержимое; IDA распознаёт, например, CODE, DATA и BSS. Эти классы также перечислены в segment.hpp.

У создания сегмента есть неприятное последствие: все ранее определённые данные в его пределах сбрасываются. После повторного применения структур заголовка вернёмся к .text. Поле VirtualAddress со значением 1000h задаёт относительный адрес загрузки, а SizeOfRawData, равное 600h, - объём данных в файле. Следовательно, байты файла со смещений 400h-9FFh должны оказаться по виртуальным адресам 401000h-4015FFh.

Все эти байты уже находятся в базе, но их необходимо переместить. После создания .headers конец этого сегмента может выглядеть так:

```
.headers:004003FF                 db    0
.headers:004003FF _headers        ends
.headers:004003FF
seg001:00400400 ; ========================================================
seg001:00400400
```

```
seg001:00400400 ; Тип сегмента: чистый код
seg001:00400400 seg001          segment byte public 'CODE' use32
seg001:00400400                 assume cs:seg001
seg001:00400400                 ;org 400400h
seg001:00400400                 assume es:_headers, ss:_headers, ds:_headers
seg001:00400400                 db  55h ; U
```

IDA разделила исходный `seg000`: диапазон заголовков стал `.headers`, а остальные байты перешли в `seg001`. Первые `600h` байт `seg001` составляют `.text`; остаётся перенести их и правильно задать размер.

Сначала командой **Edit > Segments > Move Current Segment** переместим `seg001` на виртуальный адрес `401000h`.

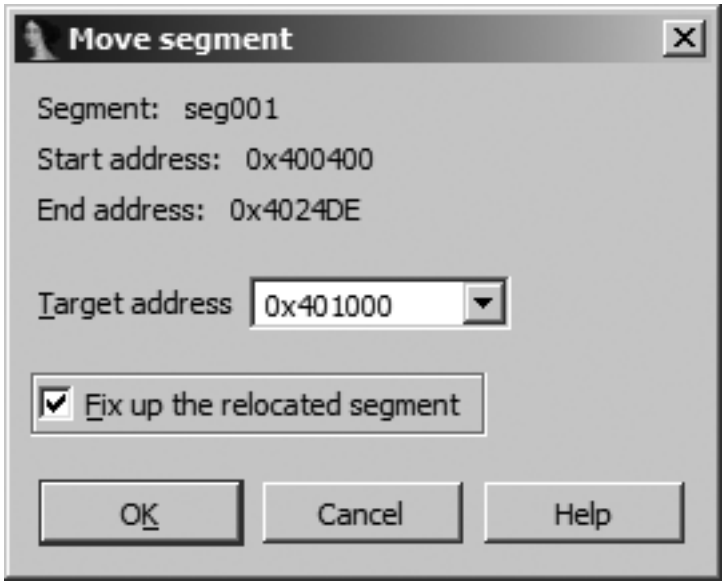


Рисунок 18-4. Перемещение сегмента

Затем командой **Edit > Segments > Create Segment** выделим из первых `600h` байт новый сегмент `.text`.

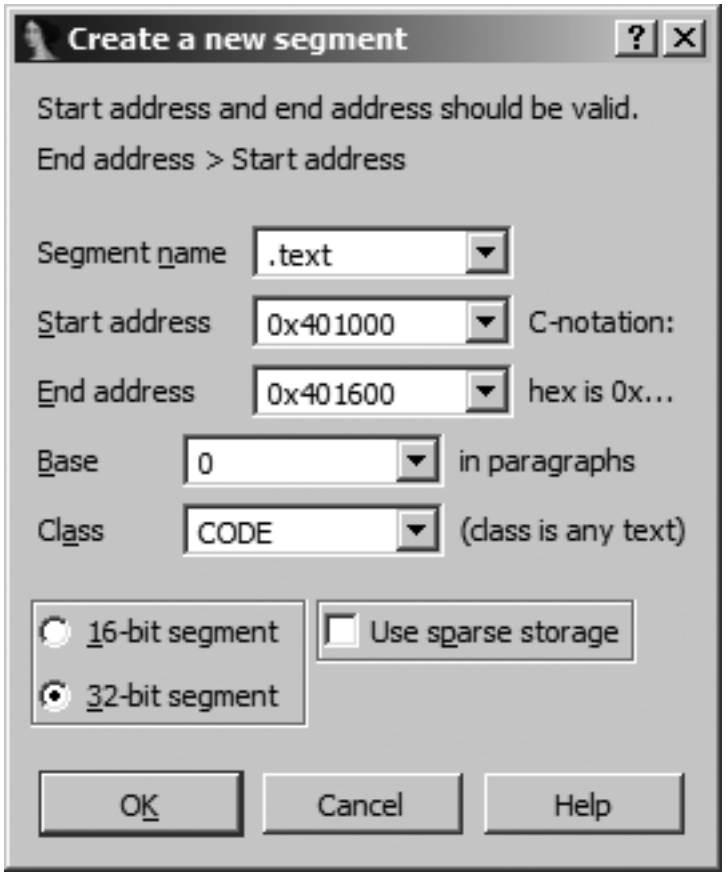


Рисунок 18-5. Ручное создание раздела `.text`

Конечный адрес не включается в диапазон. После создания `.text` остаток исходного файла образует следующий за ним `seg002`.

Второй заголовок раздела после применения `IMAGE_SECTION_HEADER` выглядит так:

```
.headers:004001A0                db '.rdata',0,0 ; Name
.headers:004001A0                dd 60h          ; VirtualSize
.headers:004001A0                dd 2000h        ; VirtualAddress
.headers:004001A0                dd 200h         ; SizeOfRawData
.headers:004001A0                dd 0A00h        ; PointerToRawData
.headers:004001A0                dd 0            ; PointerToRelocations
.headers:004001A0                dd 0            ; PointerToLinenumbers
.headers:004001A0                dw 0            ; NumberOfRelocations
.headers:004001A0                dw 0            ; NumberOfLinenumbers
.headers:004001A0                dd 40000040h    ; Characteristics
```

Раздел `.rdata` занимает `200h` байт с файлового смещения `0A00h` и отображается по относительному адресу `2000h`, то есть виртуальному `402000h`. После переноса `.text` напрямую сопоставлять `PointerToRawData` со смещением базы уже неудобно. Но содержимое `.rdata` следует сразу за `.text`, а значит, находится в первых `200h` байтах `seg002`.

Другой вариант - создавать разделы в обратном порядке, от последнего к `.text`. Тогда каждый остаётся по своему файловому смещению до переноса на виртуальный адрес.

Для `.rdata` сначала перемещают `seg002` на `402000h`, затем создают сегмент в диапазоне `402000h-402200h`.

Следующий раздел, `.bss`, обычно объединяет статически выделенные переменные, которые при запуске обнуляются. Переменные с ненулевыми начальными значениями чаще находятся в `.data`, если изменяемы, или `.rdata`, если постоянны. `.bss` не занимает места в файле: память для него выделяет загрузчик операционной системы.

```
.headers:004001C8                db '.bss',0,0,0 ; Name
```

```
.headers:004001C8                dd 40h          ; VirtualSize
.headers:004001C8                dd 3000h        ; VirtualAddress
.headers:004001C8                dd 0            ; SizeOfRawData
.headers:004001C8                dd 0            ; PointerToRawData
.headers:004001C8                dd 0            ; PointerToRelocations
.headers:004001C8                dd 0            ; PointerToLinenumbers
.headers:004001C8                dw 0            ; NumberOfRelocations
.headers:004001C8                dw 0            ; NumberOfLinenumbers
.headers:004001C8                dd 0C0000080h   ; Characteristics
```

Здесь `SizeOfRawData` равно нулю, а `VirtualSize` - `40h`, или 64 байтам. Для создания `.bss` нужно получить пустой диапазон с адреса `403000h`, а затем занять его сегментом. Проще всего сначала перенести оставшиеся разделы на правильные адреса. Итоговое окно сегментов может выглядеть так:

```
Имя      Начало   Конец    R W X D L Выравн. База Тип    Класс
.headers 00400000 00400400 ? ? ? . . byte    0000 public DATA
.text    00401000 00401600 ? ? ? . . byte    0000 public CODE
.rdata   00402000 00402200 ? ? ? . . byte    0000 public DATA
.bss     00403000 00403040 ? ? ? . . byte    0000 public BSS
.idata   00404000 00404200 ? ? ? . . byte    0000 public IMPORT
seg005   00404200 004058DE ? ? ? . L byte    0001 public CODE
```

Конечные адреса не соседствуют с началами следующих сегментов, поскольку использованы файловые размеры без учёта виртуальных размеров и выравнивания. Чтобы база точно отражала образ в памяти, конечные адреса можно увеличить до начала следующего сегмента.

Знаки вопроса означают неизвестные права доступа. В PE они кодируются битами поля `Characteristics`, но для созданных вручную сегментов назначить их можно только программно. Следующая команда IDC разрешает исполнение `.text`:

```
SetSegmentAttr(0x401000, SEGATTR_PERM, 1);
```

В IDC для прав нет символических констант. Числа совпадают с правами файлов Unix: чтение - 4, запись - 2, исполнение - 1. Несколько прав объединяются побитовой операцией ИЛИ.

Теперь процессорный модуль x86 наконец может приступить к работе. Вернёмся к найденной точке входа `401000h` и преобразуем байты в код. Чтобы адрес появился в окне **Exports**, его нужно обозначить программно:

```
AddEntryPoint(0x401000, 0x401000, 'start', 1)
```

IDA присвоит точке входа имя `start`, добавит её как экспортируемый символ, создаст код и начнёт рекурсивный спуск по связанным инструкциям. Подробнее функция `AddEntryPoint` описана во встроенной справке.

При двоичной загрузке IDA не анализирует содержимое автоматически: не определяет компилятор, библиотеки и импортируемые функции, не подключает библиотеки типов и сигнатуры. Чтобы получить обычное качество дизассемблирования, потребуется значительная ручная работа. Мы даже не рассмотрели все поля PE, которые можно было бы учесть.

Более того, для каждого нового файла неизвестного формата все действия пришлось бы повторять. Часть разбора заголовков и создания сегментов можно автоматизировать сценарием IDC. Именно для полной автоматизации этого процесса служат модули загрузки IDA.

Модули загрузки IDA

Модули загрузки создают начальную структуру новой базы. Они читают входной файл, создают сегменты в соответствии с его форматом и организуют базу до передачи управления процессорному модулю, который занимается дизассемблированием.

Позже IDA может вызвать специальные функции исходного загрузчика для переноса сегментов или создания исполняемого файла командой **File > Produce File > Create EXE File**.

Загрузчик применяется только при открытии нового файла, но не существующей базы. Как и модуль расширения, он может быть разделяемой библиотекой, собранной с SDK. Загрузчики стали первым видом расширений IDA, который разрешили писать как сценарии, начиная с IDA 5.6.

После выбора нового файла IDA динамически загружает каждый модуль из `<IDADIR>/loaders` и просит проверить файл. Все распознавшие формат загрузчики появляются в окне загрузки, а окончательный выбор делает пользователь.

Создание загрузчика IDA средствами SDK

Основной интерфейс загрузчика - глобальный экспортируемый объект `loader_t`, аналогичный `plugin_t` у модулей расширения. Структура объявлена в `loader.hpp`.

```
struct loader_t {
    ulong version; // версия API, обычно IDP_INTERFACE_VERSION
    ulong flags;   // флаги загрузчика

    // Проверка формата входного файла.
    int (idaapi *accept_file)(linput_t *li,
                              char fileformatname[MAX_FILE_FORMAT_NAME],
                              int n);

    // Загрузка файла в базу.
    void (idaapi *load_file)(linput_t *li, ushort neflags,
                             const char *fileformatname);

    // Создание выходного файла из базы; функция необязательна.
    int (idaapi *save_file)(FILE *fp, const char *fileformatname);

    // Обработка переноса сегмента, например исправление перемещений;
    // функция необязательна.
    int (idaapi *move_segm)(ea_t from, ea_t to, asize_t size,
                            const char *fileformatname);

    // Настройка пользовательских параметров по входному файлу.
    // Вызывается только при File > New, но не при File > Open;
    // функция необязательна.
    bool (idaapi *init_loader_options)(linput_t *li);
};
```

Поведение `loader_t`, как и `plugin_t`, задают функции автора. Каждый загрузчик экспортирует объект `loader_t` с именем `LDSC`, то есть описание загрузчика. Экспорт обеспечивает `loader.hpp`; разработчику остаётся объявить и инициализировать объект.

Несколько функций получают `linput_t` - внутреннюю независимую от компилятора оболочку SDK над стандартным типом C `FILE`. Операции ввода для неё объявлены в `diskio.hpp`.

Поля `LDSC` имеют следующее назначение.

`version`

: Версия интерфейса, аналогичная одноимённому полю `plugin_t` из главы 17.

`flags`

: Для загрузчиков определён лишь флаг `LDRF_RELOAD` из `loader.hpp`. Во многих случаях достаточно нуля.

`accept_file`

: Функция первичного распознавания. Она читает через `linput_t` достаточно данных, чтобы определить, способен ли загрузчик разобрать файл. При успехе имя формата копируется в `fileformatname`, а функция возвращает ненулевое значение; при неудаче - ноль. Побитовое объединение результата с `ACCEPT_FIRST` просит IDA поставить загрузчик первым в списке. Если так поступили несколько загрузчиков, первым окажется опрошенный последним.

`load_file`

: IDA вызывает эту функцию после выбора загрузчика пользователем. Файл читается через `linput_t`. Параметр `neflags` содержит побитовое ИЛИ флагов `NEF_XXX` из `loader.hpp`; некоторые отражают состояние флажков в окне загрузки. Функция разбирает файл и отображает часть или всё содержимое в новую базу. При неисправимой ошибке следует вызвать `loader_failure`.

`save_file`

: Необязательная функция, создающая файл по команде **File > Produce File > Create EXE File**. Название EXE условно: реализация может сформировать любой формат. Загрузчик умеет отображать файл в базу и иногда способен выполнить обратное преобразование, но исходных сведений может оказаться недостаточно. Например, штатный загрузчик PE не восстанавливает EXE из базы. Если сохранение невозможно, поле должно быть `NULL`.

`move_segm`

: Необязательный обработчик переноса сегмента базы, загруженной этим модулем. Он может учесть сведения о перемещениях из исходного файла. Если особая обработка не нужна, например перемещаемых адресов нет, указывают `NULL`.

`init_loader_options`

: Необязательная настройка параметров через мастер загрузки **File > New**. Она полезна только в родном графическом интерфейсе Windows (`idag`), где есть такие мастера, и вызывается после выбора загрузчика, до `load_file`. При открытии через **File > Open** она не вызывается никогда.

Сложные загрузчики, например штатный PE, используют здесь мастера на основе XML. Их шаблоны находятся в `<IDADIR>/cfg`, но отдельной документации по созданию собственных шаблонов нет.

Далее будут разработаны два загрузчика, показывающие распространённые операции.

Простой загрузчик Simpleton

Для демонстрации введём вымышленный формат Simpleton. Все значения имеют порядок байтов от младшего к старшему:

```
struct simpleton {
    uint32_t magic;      // сигнатура Simpleton: 0x1DAB00C
    uint32_t size;       // размер массива кода
    uint32_t base;       // базовый виртуальный адрес и точка входа
    uint8_t code[size];  // программный код
};
```

Формат состоит из сигнатуры, двух целых чисел, описывающих файл, и всего машинного кода. Исполнение начинается с первого байта массива `code`. Небольшой файл выглядит так:

```
0000000: 0cb0 da01 4900 0000 0040 0000 31c0 5050  ....I....@..1.PP
0000010: 89e7 6a10 5457 50b0 f350 cd91 5859 4151  ..j.TWP..P..XYAQ
0000020: 50cd 9166 817f 0213 8875 f16a 3e6a 025b  P..f.....u.j>j.[
0000030: 5853 6a09 516a 3ecd 914b 79f4 5068 6e2f  XSj.Qj>..Ky.Ph//
0000040: 7368 682f 2f62 6989 e350 5389 e150 5153  shh/bin..PS..PQS
0000050: b03b 50cd 91                             .;P..
```

Примеры загрузчиков находятся в `<SDKDIR>/ldr`. Собственные удобно создавать в отдельных соседних каталогах; здесь используется `<SDKDIR>/ldr/simpleton`. Начало исходного файла:

```
#include "../idaldr.h"
#define SIMPLETON_MAGIC 0x1DAB00C

struct simpleton {
    uint32_t magic; // сигнатура Simpleton: 0x1DAB00C
    uint32_t size;  // размер массива кода
    uint32_t base;  // базовый виртуальный адрес и точка входа
};
```

Вспомогательный заголовок `<SDKDIR>/ldr/idaldr.h` включает несколько часто нужных загрузчикам заголовков и определяет полезные макросы.

Объявим `LDSC`, указав функции поведения:

```
int idaapi accept_simpleton_file(linput_t *,
                                 char[MAX_FILE_FORMAT_NAME], int);
void idaapi load_simpleton_file(linput_t *, ushort, const char *);
int idaapi save_simpleton_file(FILE *, const char *);

loader_t LDSC = {
    IDP_INTERFACE_VERSION,
    0,                      // флаги загрузчика
    accept_simpleton_file,  // проверка формата Simpleton
    load_simpleton_file,    // загрузка файла в базу
    save_simpleton_file,    // простой формат легко сохранить
    NULL,                   // перенос сегментов не требует обработки
    NULL                    // File > New не требует обработки
};
```

Функции рассматриваются в порядке возможного вызова. Сначала распознавание:

```
int idaapi accept_simpleton_file(
    linput_t *li,
    char fileformatname[MAX_FILE_FORMAT_NAME],
    int n)
{
    uint32 magic;
    if (n || lread4bytes(li, &magic, false)) return 0;
    if (magic != SIMPLETON_MAGIC) return 0; // неверная сигнатура
    qsnprintf(fileformatname, MAX_FILE_FORMAT_NAME,
              "Исполняемый файл Simpleton");
    return 1; // формат распознан
}
```

Параметр `n` показывает, сколько раз `accept_file` вызвана в текущем процессе загрузки. Так один модуль может распознавать несколько родственных форматов: IDA увеличивает `n`, пока функция не вернёт ноль. Для каждого варианта нужно заполнить `fileformatname` и вернуть ненулевое значение.

Наш загрузчик учитывает только первый вызов с нулём. Функция `lread4bytes` из `diskio.hpp` читает четырёхбайтовую сигнатуру и при успехе возвращает ноль. Её третий логический аргумент выбирает порядок байтов: `false` - от младшего к старшему, `true` - от старшего к младшему. Это сокращает число отдельных перестановок байтов.

Особой настройки для **File > New** не требуется, поэтому следующей вызывается функция загрузки:

```
void idaapi load_simpleton_file(linput_t *li, ushort neflags, const char *) {
    simpleton hdr;
    // Чтение заголовка программы.
    lread(li, &hdr, sizeof(simpleton));
    // Загрузка содержимого файла в базу.
    file2base(li, sizeof(simpleton), hdr.base, hdr.base + hdr.size,
              FILEREG_PATCHABLE);
    // Создание сегмента вокруг кода.
    if (!add_segm(0, hdr.base, hdr.base + hdr.size,
```

```
                NAME_CODE, CLASS_CODE)) {
        loader_failure();
    }
    // Получение нового сегмента.
    segment_t *s = getseg(hdr.base);
    // Включение 32-разрядной адресации x86.
    set_segm_addressing(s, 1);
    // Однократное создание комментария с лицензией, MD5
    // и именем исходного файла.
    create_filename_cmt();
    // Точка входа сообщает процессорному модулю хотя бы один адрес кода
    // и служит корнем рекурсивного дизассемблирования.
    add_entry(hdr.base, hdr.base, "_start", true);
}
```

Функция выполняет шесть действий:

1. Читает заголовок через `lread` из `diskio.hpp`, похожую на операцию POSIX `read`.
2. Загружает код по нужным адресам базы функцией `file2base` из `loader.hpp`.
3. Создаёт сегмент вокруг новых байтов с помощью `add_segm` из `segment.hpp`.
4. Получает сегмент через `getseg` и включает 32-разрядную адресацию вызовом `set_segm_addressing`.
5. Создаёт заголовочный комментарий базы функцией `create_filename_cmt`.
6. Добавляет точку входа через `add_entry` из `entry.hpp`, давая процессорному модулю начало анализа.

Главная рабочая функция загрузчика `file2base` имеет прототип:

```
int ida_export file2base(linput_t *li, long pos,
                         ea_t ea1, ea_t ea2, int patchable);
```

Она читает байты `linput_t` с файловой позиции `pos` и помещает их в базу от `ea1` включительно до `ea2` исключительно. Читается `ea2 - ea1` байт. Если `patchable` равен `FILEREG_PATCHABLE`, IDA сохраняет соответствие файловых смещений адресам базы, что позволяет создавать файлы DIF, описанные в главе 14.

Не менее важна `add_entry`:

```
bool ida_export add_entry(uval_t ord, ea_t ea,
                          const char *name, bool makecode);
```

Параметр `ord` задаёт порядковый номер экспортируемой функции. Если номера нет, передают то же значение, что и в `ea`. Параметр `ea` содержит эффективный адрес, `name` - символическое имя, для начальной точки программы часто `_start`. `makecode` указывает, считать ли адрес кодом. Экспортируемые данные, например `LDSC` самого загрузчика, являются точками входа без кода.

Последняя реализуемая функция сохраняет Simpleton из содержимого базы:

```
int idaapi save_simpleton_file(FILE *fp, const char *fileformatname) {
    uint32 magic = SIMPLETON_MAGIC;
    if (fp == NULL) return 1; // особый запрос: сохранение поддерживается
    segment_t *s = getnseg(0); // единственный сегмент с номером ноль
    if (s) {
        uint32 sz = s->endEA - s->startEA; // размер сегмента
        qfwrite(fp, &magic, sizeof(uint32));
        qfwrite(fp, &sz, sizeof(uint32));
        qfwrite(fp, &s->startEA, sizeof(uint32));
        base2file(fp, sizeof(simpleton), s->startEA, s->endEA);
        return 1;
    }
    return 0;
}
```

Функция `save_file` получает поток `FILE`, в который записывает результат. Параметр `fileformatname` содержит имя, ранее заполненное `accept_file`. Сначала IDA вызывает функцию с `fp == NULL`, выясняя, поддерживается ли создание указанного формата. Следует вернуть 1 при

возможности сохранения и 0 в противном случае. Например, нужные сведения могут отсутствовать в базе.

Затем IDA показывает окно сохранения, создаёт настоящий поток `FILE` и вызывает функцию снова. Теперь в поток нужно записать допустимое представление выходного файла.

> **IDA и указатели FILE**
>
> При разработке для Windows важно учитывать замечание из `fpro.h`: основная библиотека IDA `ida_wll.dll` собрана средствами Borland, а указатели `FILE` Borland нельзя передавать между программными модулями. Это способно вызвать нарушение доступа и аварийное завершение IDA.
>
> Поэтому IDA предлагает оболочки `qfxxx`, например `qfprintf`, вместо стандартных функций C вроде `fprintf`. Их параметры не всегда совпадают с аналогами C, как у `qfwrite` и `fwrite`.
>
> Для применения стандартных функций C необходимо соблюдать три правила:
>
> - определить `USE_STANDARD_FILE_FUNCTIONS` до включения `fpro.h`;
> - не передавать указатели `FILE` от IDA функциям библиотеки C;
> - не передавать указатели `FILE` от библиотеки C функциям `qfxxx`.

Главная новая функция в `save_simpleton_file` - `base2file`, обратная к `file2base`: она записывает диапазон значений базы в заданную позицию потока.

Формат Simpleton почти бесполезен сам по себе, но наглядно показывает основное поведение загрузчиков IDA.

Сборка модуля загрузки IDA

Сборка и установка почти совпадают с процессом для модулей расширения из главы 17. Различаются лишь несколько деталей:

• расширения загрузчиков: `.ldw` и `.l64` в Windows, `.llx` и `.llx64` в Linux, `.lmc` и `.lmc64` в macOS;
• собранные файлы удобно помещать в `<SDKDIR>/bin/loaders`;
• для установки файл копируют в `<IDADIR>/loaders`.

Файл сборки из листинга 17-1 легко изменить: заменить `PLUGIN_EXT` на `LOADER_EXT` с нужными расширениями, все упоминания `idabook_plugin` - на `simpleton`, а `OUTDIR` направить в `$(IDA)/bin/loaders`.

Загрузчик pcap для IDA

Большинство сетевых пакетов не содержит машинного кода. Но при эксплуатации уязвимости в них может находиться двоичный код, требующий дизассемблирования. Чтобы показать нестандартные применения загрузчиков, создадим модуль для файла захвата пакетов pcap. Он не пытается заменить анализатор вроде Wireshark, но демонстрирует дополнительные возможности SDK.

Упрощённая грамматика pcap выглядит так:

```
pcap_file: pcap_file_header (pcap_packet)*
pcap_packet: pcap_packet_header pcap_content
pcap_content: (byte)+
```

pcap_file_header содержит 32-разрядную сигнатуру и другие поля, включая тип пакетов. Для простоты рассматриваются только кадры Ethernet DLT_EN10MB со скоростью 10 Мбит/с.

Загрузчик должен отделить заголовок файла от пакетов разными сегментами и распознать как можно больше заголовков внутри сегмента пакетов. Тогда пользователь сможет сосредоточиться на содержимом прикладного уровня. Рассмотрим только load_file, поскольку accept_file отличается от варианта Simpleton лишь проверяемой сигнатурой.

Для оформления заголовков структуры должны существовать в окне **Structures** уже во время загрузки. В библиотеке типов **GNU C++ Unix** есть заголовки pcap, Ethernet, IP, TCP и UDP, хотя в IDA до версии 5.3 определение IP-заголовка iphdr было ошибочным.

Функция load_pcap_file сначала вызывает вспомогательную add_types. Рассмотрим два варианта: импорт из готовой библиотеки и самостоятельное создание всех структур.

Первый вариант загружает библиотеку **GNU C++ Unix** и импортирует нужные типы:

```
void add_types() {
#ifdef ADDTIL_DEFAULT
    add_til2("gnuunx.til", ADDTIL_SILENT);
#else
    add_til("gnuunx.til");
#endif
    pcap_hdr_struct = til2idb(-1, "pcap_file_header");
    pkthdr_struct = til2idb(-1, "pcap_pkthdr");
    ether_struct = til2idb(-1, "ether_header");
    ip_struct = til2idb(-1, "iphdr");
    tcp_struct = til2idb(-1, "tcphdr");
    udp_struct = til2idb(-1, "udphdr");
}
```

Функции add_til из typinf.hpp загружают в базу готовый файл типов. Начиная с IDA 5.1 устаревшую add_til заменила add_til2. Это программный аналог загрузки .til через окно **Types**.

После загрузки til2idb переносит отдельный тип в текущую базу, подобно добавлению стандартной структуры в окно **Structures**. Она возвращает идентификатор, необходимый для преобразования байтов в структурированный тип. Идентификаторы типа tid_t сохранены в глобальных переменных для быстрого дальнейшего доступа.

У подхода два недостатка: ради шести типов загружается вся библиотека, а встроенное определение может оказаться ошибочным.

Второй вариант строит библиотеку на лету, разбирая объявления структур C:

```
void add_types() {
    til_t *t = new_til("pcap.til", "типы заголовков pcap");
    parse_decls(t, pcap_types, NULL, HTI_PAK1);
    sort_til(t);
    pcap_hdr_struct = import_type(t, -1, "pcap_file_header");
    pkthdr_struct = import_type(t, -1, "pcap_pkthdr");
    ether_struct = import_type(t, -1, "ether_header");
    ip_struct = import_type(t, -1, "iphdr");
    tcp_struct = import_type(t, -1, "tcphdr");
    udp_struct = import_type(t, -1, "udphdr");
    free_til(t);
}
```

new_til создаёт пустую временную библиотеку. parse_decls разбирает строку pcap_types с допустимыми объявлениями C. Её начало выглядит так:

```
char *pcap_types =
    "struct pcap_file_header {\n"
        "int magic;\n"
        "short version_major;\n"
        "short version_minor;\n"
        "int thiszone;\n"
        "int sigfigs;\n"
```

```
        "int snaplen;\n"
        "int linktype;\n"
    "};\n"
    /* ... остальные определения ... */;
```

Полная строка содержит все структуры загрузчика. Для упрощения разбора поля сведены к стандартным типам C.

Константа `HTI_PAK1` из `typeinf.hpp` требует упаковки структур по границе в один байт. После изменения библиотеку нужно отсортировать вызовом `sort_til`. `import_type`, подобно `til2idb`, переносит выбранный тип в базу и возвращает его идентификатор. В конце временная библиотека освобождается через `free_til`.

Так загрузчик полностью контролирует импортируемые типы и не добавляет ненужную библиотеку. Временную библиотеку можно сохранить на диск функцией `store_til`, предварительно вызвав `compact_til`, но для малого числа типов проще строить её при каждой загрузке.

Начало `load_pcap_file` добавляет типы и комментарий, загружает заголовок файла, создаёт вокруг него сегмент и применяет структуру:

```
void idaapi load_pcap_file(linput_t *li, ushort, const char *) {
    ssize_t len;
    pcap_pkthdr pkt;

    add_types();           // шаблоны структур в базу
    create_filename_cmt(); // основной комментарий файла
    // Загрузка заголовка pcap.
    file2base(li, 0, 0, sizeof(pcap_file_header), FILEREG_PATCHABLE);
    // Сегмент данных для байтов заголовка.
    if (!add_segm(0, 0, sizeof(pcap_file_header),
                  ".file_header", CLASS_DATA)) {
        loader_failure();
    }
    // Преобразование байтов в pcap_file_header.
    doStruct(0, sizeof(pcap_file_header), pcap_hdr_struct);
    // ... продолжение
}
```

`file2base` снова загружает данные с диска. Функция `doStruct` из `bytes.hpp` соответствует команде **Edit > Struct Var**: она получает адрес, размер и идентификатор типа, затем превращает указанное число байтов в структуру.

Далее функция читает все пакеты и создаёт вокруг них единый сегмент `.packets`:

```
// Продолжение load_pcap_file.
uint32 pos = sizeof(pcap_file_header); // текущая позиция файла
while ((len = qlread(li, &pkt, sizeof(pkt))) == sizeof(pkt)) {
    mem2base(&pkt, pos, pos + sizeof(pkt), pos);
    pos += sizeof(pkt); // переход к содержимому пакета
    // Чтение присутствующих в файле байтов пакета.
    file2base(li, pos, pos, pos + pkt.caplen, FILEREG_PATCHABLE);
    pos += pkt.caplen; // переход к следующему заголовку
}
// Сегмент пакетов начинается сразу после заголовка файла.
if (!add_segm(0, sizeof(pcap_file_header), pos,
              ".packets", CLASS_DATA)) {
    loader_failure();
}
segment_t *s = getseg(sizeof(pcap_file_header));
set_segm_addressing(s, 1); // 32-разрядная адресация
// ... продолжение
```

Новая функция `mem2base` переносит в базу данные, уже прочитанные в память.

Шаблоны следует применять только после создания сегмента, иначе создание сбросит всё оформление. Заключительная часть обходит пакеты и распознаёт вложенные заголовки.

```
// Заключительная часть load_pcap_file.
```

```
// Применение структур заголовков ко всем пакетам базы.
for (uint32 ea = s->startEA; ea < pos;) {
    uint32 pcap = ea; // начало пакета
    doStruct(pcap, sizeof(pcap_pkthdr), pkthdr_struct);

    uint32 eth = pcap + sizeof(pcap_pkthdr);
    doStruct(eth, sizeof(ether_header), ether_struct);

    // Проверка типа кадра Ethernet.
    uint16 etype = get_word(eth + 12);
    etype = (etype >> 8) | (etype << 8); // htons

    if (etype == ETHER_TYPE_IP) {
        uint32 ip = eth + sizeof(ether_header);
        doStruct(ip, sizeof(iphdr), ip_struct);

        // Проверка протокола IP и вычисление длины заголовка.
        uint8 proto = get_byte(ip + 9);
        uint32 iphl = (get_byte(ip) & 0xF) * 4;
        if (proto == IP_PROTO_TCP) {
            doStruct(ip + iphl, sizeof(tcphdr), tcp_struct);
        }
        else if (proto == IP_PROTO_UDP) {
            doStruct(ip + iphl, sizeof(udphdr), udp_struct);
        }
    }

    // Переход к следующему pcap_pkthdr.
    ea += get_long(pcap + 8) + sizeof(pcap_pkthdr);
}
```

Код перебирает пакеты, читает поля заголовков, определяет разновидность следующей структуры и её начальный адрес. Первые строки загруженного файла могут выглядеть так:

```
.file_header:0000 _file_header    segment byte public 'DATA' use16
.file_header:0000         assume cs:_file_header
.file_header:0000         pcap_file_header <0A1B2C3D4h, 2, 4, 0, 0, 0FFFFh, 1>
.file_header:0000 _file_header    ends
.file_header:0000
.packets:00000018 ; ===========================================================
.packets:00000018
.packets:00000018 ; Тип сегмента: чистые данные
.packets:00000018 _packets  segment byte public 'DATA' use32
.packets:00000018            assume cs:_packets
.packets:00000018            ;org 18h
.packets:00000018            pcap_pkthdr <<47DF275Fh, 1218Ah>, 19Ch, 19Ch>
.packets:00000028            db 0, 18h, 0E7h, 1, 32h, 0F5h; ether_dhost
.packets:00000028            db 0, 50h, 0BAh, 0B8h, 8Bh, 0BDh; ether_shost
.packets:00000028            dw 8                     ; ether_type
.packets:00000036            iphdr <45h, 0, 8E01h, 0EE4h, 40h, 80h, 6, 9E93h,
                                    200A8C0h, 6A00A8C0h>
.packets:0000004A            tcphdr <901Fh, 2505h, 0C201E522h, 6CE04CCBh, 50h,
                                     18h, 0E01Ah, 3D83h, 0>
.packets:0000005E            db  48h ; H
.packets:0000005F            db  54h ; T
.packets:00000060            db  54h ; T
.packets:00000061            db  50h ; P
.packets:00000062            db  2Fh ; /
.packets:00000063            db  31h ; 1
.packets:00000064            db  2Eh ; .
.packets:00000065            db  30h ; 0
```

Структуры позволяют сворачивать и разворачивать заголовки. В листинге легко заметить, что байт по адресу `0000005E` начинает ответ HTTP.

Базовая загрузка pcap создаёт основу для модулей расширения, которые собирают потоки TCP и извлекают данные. Можно также улучшить представление сетевых структур: показывать IP-адреса в привычном виде и преобразовывать сетевой порядок байтов других полей. Эти улучшения оставлены читателю.

Другие стратегии загрузки

Примеры SDK используют разные подходы. Особого внимания заслуживает загрузчик Java из `<SDKDIR>/ldr/javaldr`.

У некоторых форматов связь загрузчика с процессорным модулем слаба: достаточно сообщить точки входа, и дальнейшие сведения не нужны. Но иной процессорный модуль зависит от подробностей исходного файла и вынужден повторять уже выполненный разбор. Чтобы не дублировать работу, загрузчик и процессор можно связать теснее.

Загрузчик Java фактически передаёт все обычные задачи `load_file` процессорному модулю:

```
static void load_file(linput_t *li, ushort neflag, const char *) {
    if (ph.id != PLFM_JAVA) {
        set_processor_type("java", SETPROC_ALL | SETPROC_FATAL);
    }
    if (ph.notify(ph.loader, li, (bool)(neflag & NEF_LOPT))) {
        error("Внутренняя ошибка связи загрузчика с процессорным модулем");
    }
}
```

Загрузчик лишь проверяет выбор процессора Java и отправляет ему уведомление `ph.loader` из `idp.hpp` о начале загрузки. Процессор берёт работу на себя и одновременно формирует внутреннее состояние, которое позднее применит при дизассемблировании.

Такой подход оправдан, если разрабатываются оба модуля и процессору полезны сведения, обычно получаемые загрузчиком: сегментация, поля заголовков, отладочная информация и прочее.

Другой способ передачи состояния - сетевые узлы базы. Загрузчик записывает туда сведения, а процессор извлекает их при анализе. Частое чтение базы может быть медленнее прямого использования типов C++.

Загрузчик в виде сценария

Начиная с IDA 5.6 загрузчики можно писать на Python или IDC. Один из ранних примеров Hex-Rays загружал особую разновидность вредоносного файла `.pdf` со встроенным машинным кодом. Он не подходил для всех файлов PDF, но хорошо показывал поддержку неизвестных форматов.

Сценарный загрузчик должен содержать как минимум `accept_file` и `load_file`, назначение которых совпадает с вариантами SDK. Загрузчик Simpleton на IDC выглядит так:

```
#include <idc.idc>

#define SIMPLETON_MAGIC 0x1DAB00C

// Проверка формата входного файла.
// li - объект loader_input_t, n - номер вызова.
// Возвращает 0 при нераспознанном файле либо имя формата.
static accept_file(li, n) {
    auto magic;
    if (n) return 0;
    li.readbytes(&magic, 4, 0);
    if (magic != SIMPLETON_MAGIC) {
        return 0;
    }
    return "Загрузчик Simpleton на IDC";
}
// Загрузка файла. neflags описан в loader.hpp,
```

```
// format содержит выбранный пользователем формат.
// Возвращает 1 при успехе и 0 при неудаче.
static load_file(li, neflags, format) {
    auto magic, size, base;
    li.seek(0, 0);
    li.readbytes(&magic, 4, 0);
    li.readbytes(&size, 4, 0);
    li.readbytes(&base, 4, 0);
    loadfile(li, 12, base, size); // копирование байтов в базу
    AddSeg(base, base + size, 0, 1, saRelPara, scPub);
    AddEntryPoint(base, base, "_start", 1);
    return 1;
}
```

Различия с C++ сводятся главным образом к функциям IDC. Сценарий устанавливается копированием в `<IDADIR>/loaders`.

Python даёт более полный доступ к SDK и позволяет написать более развитый загрузчик. Тот же пример:

```
# Проверка формата входного файла.
# li - loader_input_t, n - номер вызова.
# Возвращает 0 либо имя распознанного формата.
def accept_file(li, n):
    if n:
        return 0
    li.seek(0)
    magic = struct.unpack("<I", li.read(4))[0]
    if magic != 0x1DAB00C:
        return 0
    return "Загрузчик Simpleton на Python"

# Загрузка файла; возвращает 1 при успехе и 0 при неудаче.
def load_file(li, neflags, format):
    li.seek(0)
    (magic, size, base) = struct.unpack("<III", li.read(12))
    li.file2base(12, base, base + size, 1)
    add_segm(0, base, base + size, ".text", "CODE")
    add_entry(base, base, "_start", 1)
    return 1
```

Главное достоинство загрузчиков и модулей расширения в виде сценариев - быстрое создание прототипа, который позднее при необходимости переносится на SDK.

Итоги

После понимания места загрузчиков в модульной архитектуре IDA создавать их не сложнее, чем модули расширения. Они используют собственную часть SDK, сосредоточенную прежде всего в `loader.hpp`, `segment.hpp`, `entry.hpp` и `diskio.hpp`.

Загрузчик работает до того, как процессорный модуль начинает анализ нового кода. Поэтому он не должен заниматься дизассемблированием, функциями или инструкциями.

В следующей главе завершается обзор модулей IDA и рассматриваются процессорные модули, которые в наибольшей степени отвечают за представление дизассемблированного файла.

Сноски

[1] Выравнивание определяет допустимый начальный адрес или смещение блока: он должен быть кратен заданной величине. Например, при выравнивании `200h`, то есть 512 байт, адрес делится на `200h` без остатка.

Глава 19. Процессорные модули IDA

Последний вид модулей IDA, создаваемых с помощью SDK, - процессорные модули. Это самые сложные компоненты модульной системы. Они отвечают за все операции дизассемблирования: не только преобразуют коды машинных команд в инструкции ассемблера, но и создают функции, порождают перекрёстные ссылки и отслеживают указатель стека.

Начиная с IDA 5.7 процессорные модули, как и расширения с загрузчиками, можно писать на языках сценариев.

Очевидная причина создать такой модуль - необходимость разобрать двоичный файл для неподдерживаемого процессора, например микропрограмму встроенного микроконтроллера или исполняемый образ портативного устройства.

Менее очевидный случай - набор инструкций особой виртуальной машины внутри запутанной программы. Обычный модуль x86 поможет понять устройство самой виртуальной машины, но не расшифрует исполняемый ею байт-код.

Сторонних процессорных модулей опубликовано мало. Возможных применений модулей расширения бесконечно много, форматов файлов меньше, а наборов команд ещё меньше. Кроме того, разработчики нередко создают собственные процессорные модули, но не распространяют их.

Эта глава должна сделать устройство процессорных модулей понятнее. Сквозным примером станет дизассемблер байт-кода Python. Компоненты слишком объёмны для полного воспроизведения каждого файла, поэтому приводятся основные фрагменты.

Без загрузчика Python полностью автоматическое дизассемблирование `.pyc` невозможно. Придётся открыть файл в двоичном режиме, выбрать процессорный модуль Python, найти вероятное начало функции и преобразовать байты в инструкции командой **Edit > Code**.

Байт-код Python

Python - объектно-ориентированный интерпретируемый язык программирования, часто применяемый для сценариев. Исходные файлы обычно имеют расширение `.py`. При запуске интерпретатор компилирует исходный код во внутреннее представление - байт-код Python, который исполняется виртуальной машиной.

Это похоже на преобразование Java в байт-код и его выполнение виртуальной машиной Java. Главное различие в том, что Java обычно компилируется пользователем явно, а Python преобразует исходный текст при каждом запуске автоматически.

Чтобы не повторять компиляцию, интерпретатор может сохранить байт-код в `.pyc` и при следующем запуске загрузить его напрямую. Пользователь обычно не создаёт такие файлы вручную: интерпретатор делает это для исходного модуля, импортированного другим модулем. Часто используемый модуль тогда запускается быстрее. Файл `.pyc` примерно соответствует файлу `.class` Java.

Поскольку при наличии байт-кода исходный текст не нужен, части проекта можно распространять только как `.pyc`. Тогда для понимания поведения приходится исследовать байт-код так же, как любую двоичную программу. Именно этому помогает учебный процессорный модуль Python.

Интерпретатор Python

Интерпретатор реализует стековую виртуальную машину для байт-кода Python. У неё нет обычных регистров, кроме указателей команд и стека. Большинство инструкций читает, записывает или проверяет стек.

Например, `BINARY_ADD` снимает два верхних значения, складывает их и помещает единственный результат обратно на вершину.

Набор команд сравнительно прост: каждая инструкция содержит однобайтовый код операции и либо ноль, либо два байта операнда. Предварительное знание байт-кода для этой главы не требуется; нужные особенности будут объяснены по ходу. Главная цель - понять процессорные модули IDA, а Python лишь даёт удобный пример.

Создание процессорного модуля средствами SDK

Документации по процессорным модулям мало. Основные источники - заголовки SDK, исходный код примеров и раздел **Description of processor modules** в `readme.txt`.

Файл справки упоминает определённые имена исходников так, будто они обязательны, но это не так. Такие имена лишь приняты в примерах и указаны в их сценариях сборки. Допускаются любые имена при соответствующем изменении сборки.

Разделение по файлам подчёркивает три логических компонента процессорного модуля:

1. анализатор инструкций;
2. эмулятор их поведения;
3. генератор вывода.

Примеры находятся в `<SDKDIR>/module`. Одним из сравнительно простых считается процессор Z8. Сложность других зависит от набора команд и того, берут ли они на себя часть загрузки.

Для начала разработчик IDA рекомендует скопировать существующий модуль и изменить его. Следует выбирать пример, наиболее близкий по логическому устройству, а не обязательно по архитектуре процессора.

Структура processor_t

Процессорный модуль экспортирует один объект - структуру `processor_t` с обязательным именем `LPH`. Заголовок `<SDKDIR>/module/idaidp.hpp` экспортирует её автоматически и подключает другие часто используемые файлы SDK.

Сложность хорошо видна по размерам описания: `processor_t` содержит 56 полей, из них 26 - указатели на функции. Ещё одно поле указывает на массив структур `asm_t`, каждая из которых требует инициализировать 59 полей. Большой объём статических данных легко приводит к ошибкам и объясняет совет начинать с готового модуля.

Перечислять все поля здесь не будем; полное описание находится в `idp.hpp`. Далее рассматриваются важнейшие из них, не в порядке объявления.

Основная инициализация LPH

До реализации поведения нужно подготовить статические данные. Сначала составляется список всех распознаваемых мнемоник целевого процессора. Обычно это массив структур `instruc_t` из `idp.hpp` в файле `ins.cpp`:

```
struct instruc_t {
    const char *name; // мнемоника инструкции
    ulong feature;    // побитовое ИЛИ флагов CF_xxx из idp.hpp
};
```

Структура одновременно служит таблицей мнемоник и описывает основные свойства инструкции. Поле `feature` сообщает, читает или изменяет ли команда операнды и как продолжается исполнение: последовательно, переходом или вызовом. Префикс `CF` означает каноническое свойство. Эти флаги определяют поток управления и перекрёстные ссылки.

Особенно важны следующие значения:

`CF_STOP`

: Инструкция не передаёт управление следующей, например безусловный переход или возврат из функции.

`CF_CHGn`

: Инструкция изменяет операнд *n*, где *n* находится в диапазоне от 1 до 6.

`CF_USEn`

: Инструкция использует, то есть читает либо упоминает, операнд *n*, но не изменяет его.

`CF_CALL`

: Инструкция вызывает функцию.

Порядок инструкций произволен. Их не обязательно сортировать по двоичным кодам, а взаимно однозначное соответствие записей допустимым кодам не требуется.

В нашем простом примере индекс записи совпадает с кодом операции Python. Для отсутствующих кодов приходится оставлять заполнители:

```
instruc_t Instructions[] = {
    {"STOP_CODE", CF_STOP},   /* 0 */
    {"POP_TOP", 0},           /* 1 */
    {"ROT_TWO", 0},           /* 2 */
    {"ROT_THREE", 0},         /* 3 */
    {"DUP_TOP", 0},           /* 4 */
    {"ROT_FOUR", 0},          /* 5 */
    {NULL, 0},                 /* 6: заполнение */
    /* ... */
    {"CALL_FUNCTION_VAR_KW", CF_CALL}, /* 142 */
    {"SETUP_WITH", 0},                 /* 143 */
    {"EXTENDED_ARG", 0},               /* 145 */
    {"SET_ADD", 0},                     /* 146 */
    {"MAP_ADD", 0}                      /* 147 */
};
```

Связанный набор перечислимых констант обычно определяется в `ins.hpp`:

```
enum python_opcodes {
    STOP_CODE = 0,
    POP_TOP = 1,    // снять верхнее значение стека
    ROT_TWO = 2,    // поменять местами два верхних значения
    ROT_THREE = 3,  // переместить верхнее под второе и третье
    DUP_TOP = 4,    // продублировать верхнее значение
    ROT_FOUR = 5,   // переместить верхнее под следующие три
    NOP = 9,        // нет действия
    /* ... */
    CALL_FUNCTION_VAR_KW = 142,
    SETUP_WITH = 143,
    EXTENDED_ARG = 145,
```

```
    SET_ADD = 146,
    MAP_ADD = 147,
    PYTHON_LAST = 148
};
```

Явные значения улучшают читаемость и учитывают пропуски, возникшие из-за использования настоящих кодов операций как индексов. Дополнительная константа `PYTHON_LAST` обозначает конец списка.

Этого достаточно для инициализации трёх полей глобальной структуры `LPH`:

```
int instruc_start;   // целый код первой инструкции
int instruc_end;     // целый код после последней инструкции
instruc_t *instruc;  // массив инструкций
```

Им присваиваются соответственно `STOP_CODE`, `PYTHON_LAST` и `Instructions`. Так модуль быстро находит мнемонику любой инструкции.

Большинству процессоров также нужен список имён регистров и соответствующие перечислимые константы. Для сокращённого набора x86 это могло бы выглядеть так:

```
static char *RegNames[] = {
    "eax", "ebx", "ecx", "edx", "edi", "esi", "ebp", "esp",
    "ax", "bx", "cx", "dx", "di", "si", "bp", "sp",
    "al", "ah", "bl", "bh", "cl", "ch", "dl", "dh",
    "cs", "ds", "es", "fs", "gs"
};
```

Массив `RegNames` часто объявляют в `reg.cpp`. Там же примеры определяют `LPH`, что позволяет оставить массив статическим. Перечисление обычно помещают в заголовок с именем процессора, например `x86.hpp`:

```
enum x86_regs {
    r_eax, r_ebx, r_ecx, r_edx, r_edi, r_esi, r_ebp, r_esp,
    r_ax, r_bx, r_cx, r_dx, r_di, r_si, r_bp, r_sp,
    r_al, r_ah, r_bl, r_bh, r_cl, r_ch, r_dl, r_dh,
    r_cs, r_ds, r_es, r_fs, r_gs
};
```

Порядок массива и констант должен строго совпадать. Вместе они позволяют находить имена регистров при форматировании операндов. Ещё два поля `LPH`:

```
int regsNum;      // общее число регистров
char **regNames;  // массив имён регистров
```

Обычно им присваивают `qnumber(RegNames)` и `RegNames`. Макрос `qnumber` из `pro.h` вычисляет число элементов статического массива.

Процессорный модуль IDA обязан предоставить сведения о сегментных регистрах, даже если архитектура их не использует:

```
// Если настоящих сегментных регистров нет, применяются виртуальные CS и DS.
int regFirstSreg; // номер первого сегментного регистра
int regLastSreg;  // номер последнего сегментного регистра
int segreg_size;  // размер сегментного регистра в байтах

// Для процессора без сегментных регистров следует определить
// два виртуальных регистра, например rVcs и rVds.
int regCodeSreg;  // номер CS
int regDataSreg;  // номер DS
```

Для x86 эти пять полей получили бы значения:

```
r_cs, r_gs, 2, r_cs, r_ds
```

У стековой машины Python настоящих регистров нет, но ожидания IDA всё равно приходится удовлетворить. Создадим фиктивные регистры кода и данных, а в поведении модуля просто не будем их использовать:

```
// reg.cpp
static char *RegNames[] = { "cs", "ds" };

// python.hpp
enum py_registers { rVcs, rVds };
```

Соответствующая последовательность для LPH:

```
rVcs, rVds, 0, rVcs, rVds
```

Первые пять полей processor_t задают общие свойства:

```
int version; // IDP_INTERFACE_VERSION
int id;      // значение PLFM_xxx либо собственное больше 0x8000
ulong flag;  // побитовое ИЛИ свойств PR_xxx
int cnbits;  // число битов в байте сегмента кода, обычно 8
int dnbits;  // число битов в байте сегмента данных, обычно 8
```

Поле version уже знакомо по модулям расширения и загрузчикам. Собственный id процессора должен превышать 0x8000. В flag объединяются свойства PR_xxx из idp.hpp. Для Python достаточно PR_RNAMESOK, разрешающего применять имена регистров как имена адресов, и PRN_DEC, выбирающего десятичный формат чисел. Оба размера байта равны 8.

Анализатор

Теперь данных LPH достаточно для первого исполняемого компонента - анализатора. В примерах это обычно функция ana в файле ana.cpp, хотя имена произвольны:

```
int idaapi ana(void); // разобрать одну инструкцию и вернуть её длину
```

Поле u_ana объекта LPH получает указатель на эту функцию. Анализатор разбирает одну инструкцию, заполняет глобальную переменную cmd и возвращает длину. Базу данных он изменять не должен.

cmd - глобальный экземпляр insn_t из ua.hpp, описывающий одну инструкцию:

```
class insn_t {
public:
    ea_t cs; // базовый параграф текущего сегмента, задаёт ядро
    ea_t ip; // виртуальный адрес инструкции в сегменте, задаёт ядро
    ea_t ea; // линейный адрес инструкции, задаёт ядро
    uint16 itype; // значение перечисления инструкции, задаёт ana
    uint16 size;  // размер в байтах, задаёт ana
    union {       // поле по усмотрению процессора
        uint16 auxpref;
        struct {
            uchar low;
            uchar high;
        } auxpref_chars;
    };
    char segpref;      // поле по усмотрению процессора
    char insnpref;     // поле по усмотрению процессора
    op_t Operands[6];  // сведения об операндах, задаёт ana
    char flags;        // флаги инструкции, задаёт ana
};
```

До вызова анализатора ядро заполняет cs, ip и ea. Анализатор обязан задать прежде всего itype, size и Operands. Поле itype получает одно из ранее объявленных значений типа инструкции. size содержит полный размер и одновременно служит возвращаемым значением. Если инструкция не разбирается, возвращается ноль. Для каждого из не более чем шести операндов заполняется отдельный элемент массива.

Анализатор часто реализуется оператором switch. Сначала он запрашивает один или несколько байтов потока инструкций и выбирает по ним ветвь. SDK предоставляет специальные функции:

```
uchar ua_next_byte(void);       // прочитать один байт
ushort ua_next_word(void);      // прочитать два байта
ulong ua_next_long(void);       // прочитать четыре байта
ulonglong ua_next_qword(void);  // прочитать восемь байтов
```

Начальная позиция равна cmd.ip. Каждый вызов сдвигает её и автоматически увеличивает cmd.size на 1, 2, 4 или 8. Полученные байты нужно разобрать настолько, чтобы определить itype, число и тип операндов, а также полный размер команды. При необходимости читаются дополнительные байты.

На высоком уровне анализатор повторяет выборку и декодирование реального процессора. Фиксированная длина, характерная для многих архитектур RISC, упрощает разбор, а переменная длина, как у x86, усложняет.

Каждый операнд представлен классом op_t из ua.hpp:

```
class op_t {
public:
    char n;         // номер операнда, задаёт ядро; изменять нельзя
    optype_t type;  // тип операнда, задаёт ana
    char offb;      // смещение операнда от начала инструкции
    char offo;      // смещение второй части операнда
    uchar flags;    // флаги из ua.hpp
    char dtyp;      // тип данных операнда

    union {
        uint16 reg;    // номер регистра для o_reg
        uint16 phrase; // номер регистрового выражения для o_phrase и o_displ
    };

    union {
        uval_t value; // значение o_imm или внешнее смещение o_displ
        struct { uint16 low, high; } value_shorts;
    };

    union {
        ea_t addr; // виртуальный адрес для o_mem, o_displ, o_far и o_near
        struct { uint16 low, high; } addr_shorts;
    };

    union { // поля по усмотрению процессора
        ea_t specval;
        struct { uint16 low, high; } specval_shorts;
    };
    char specflag1, specflag2, specflag3, specflag4;
};
```

Сначала полю type присваивается одна из констант optype_t из ua.hpp. Тип приблизительно описывает способ адресации, то есть источник или назначение данных. Например, o_reg обозначает содержимое регистра, o_mem - известный при сборке адрес памяти, а o_imm - непосредственное значение внутри инструкции.

Поле dtyp задаёт размер данных одной из констант dt_xxx: dt_byte для 8 бит, dt_word для 16 и dt_dword для 32. Несколько примеров x86:

```
mov   eax, 0x31337         ; o_reg(dt_dword), o_imm(dt_dword)
push  word ptr [ebp - 12]  ; o_displ(dt_word)
mov   [0x08049130], bl     ; o_mem(dt_byte), o_reg(dt_byte)
movzx eax, ax              ; o_reg(dt_dword), o_reg(dt_word)
ret                        ; o_void(dt_void)
```

Используемое поле объединения зависит от type. Для o_imm значение сохраняется в value, для o_reg номер перечислимой константы регистра - в reg. Полная схема приведена в ua.hpp.

Ни одно поле op_t не сообщает, является ли операнд источником или назначением. Это не задача анализатора: позже назначение определяют канонические флаги из массива инструкций.

Поля, помеченные как зависящие от процессора, можно использовать свободно. В них удобно хранить сведения, не помещающиеся в стандартные поля, и передавать результаты анализа последующим этапам без повторной работы.

Теперь можно написать минимальный анализатор байт-кода Python. Код операции занимает один байт; значения меньше 90 не имеют операнда.

Коды от 90 и выше содержат двухбайтовый операнд:

```
#define HAVE_ARGUMENT 90

int idaapi py_ana(void) {
    // В этом модуле код операции совпадает с itype; вызов увеличивает cmd.size.
    cmd.itype = ua_next_byte();
    if (cmd.itype >= PYTHON_LAST) return 0;             // неверная команда
    if (Instructions[cmd.itype].name == NULL) return 0; // неверная команда

    if (cmd.itype < HAVE_ARGUMENT) {
        cmd.Op1.type = o_void; // Op1 означает Operands[0]
        cmd.Op1.dtyp = dt_void;
    }
    else {
        // Команда обязательно содержит два байта операнда.
        if (flags[cmd.itype] & (HAS_JREL | HAS_JABS)) {
            cmd.Op1.type = o_near; // ссылка на адрес кода
        }
        else {
            cmd.Op1.type = o_mem;  // условная ссылка на память
        }
        cmd.Op1.offb = 1;          // операнд начинается через байт
        cmd.Op1.dtyp = dt_dword;   // у Python нет размеров, выбираем условный

        cmd.Op1.value = ua_next_word(); // чтение и увеличение cmd.size
        cmd.auxpref = flags[cmd.itype];  // флаги для следующих этапов

        if (flags[cmd.itype] & HAS_JREL) {
            cmd.Op1.addr = cmd.ea + cmd.size + cmd.Op1.value;
        }
        else if (flags[cmd.itype] & HAS_JABS) {
            cmd.Op1.addr = cmd.Op1.value;
        }
        else if (flags[cmd.itype] & HAS_CALL) {
            // В Python цель вызова лежит в стеке, а операнд задаёт
            // число позиционных и именованных аргументов.
            cmd.Op1.specflag1 = cmd.Op1.value & 0xFF;
            cmd.Op1.specflag2 = (cmd.Op1.value >> 8) & 0xFF;
        }
    }
    return cmd.size;
}
```

Для каждой инструкции введён дополнительный элемент массива flags, дополняющий и иногда повторяющий канонические свойства. Флаги HAS_JREL, HAS_JABS и HAS_CALL обозначают относительное смещение перехода, абсолютную цель перехода и описание стека вызова.

Работа анализатора сводится к четырём шагам:

1. Прочитать код операции и проверить его допустимость.
2. Для команды без операндов задать cmd.Op1 как o_void.
3. Для команды с операндом определить его тип и сохранить дополнительные сведения в полях процессора.
4. Вернуть длину инструкции.

В общем случае анализатор должен:

1. Прочитать достаточно байтов, чтобы проверить инструкцию, сопоставить её перечислимой константе и записать результат в `cmd.itype`. Часто для этого применяется большой `switch`.
2. Прочитать остальные байты, определить число операндов, способы адресации, регистры и непосредственные значения, затем заполнить `cmd.Operands`. Декодирование операндов можно вынести в отдельную функцию.
3. Вернуть общую длину команды вместе с операндами.

После разбора IDA уже может вывести ассемблерное представление. Но для перекрёстных ссылок, рекурсивного спуска и наблюдения за указателем стека нужны сведения о поведении команды. Их предоставляет эмулятор.

Эмулятор

Анализатор описывает строение одной инструкции, а эмулятор - её поведение. В примерах это обычно функция `emu` в `emu.cpp`:

```
int idaapi emu(void); // эмулировать одну инструкцию
```

Согласно `idp.hpp`, функция должна вернуть длину, хотя большинство примеров возвращает 1. Указатель на неё записывается в `u_emu` структуры `LPH`. К моменту вызова анализатор уже заполнил `cmd`.

Основная задача эмулятора - создавать ссылки на код и данные из поведения инструкции. Здесь же отслеживается указатель стека и создаются локальные переменные по обращениям к кадру. В отличие от анализатора эмулятор может изменять базу.

Решение о ссылках обычно принимается по каноническим свойствам инструкции и типам операндов. Простейший вариант для двух операндов:

```
void TouchArg(op_t &op, int isRead); // реализует автор процессора

int idaapi emu() {
    ulong feature = cmd.get_canon_feature();

    if (feature & CF_USE1) TouchArg(cmd.Op1, 1);
    if (feature & CF_USE2) TouchArg(cmd.Op2, 1);

    if (feature & CF_CHG1) TouchArg(cmd.Op1, 0);
    if (feature & CF_CHG2) TouchArg(cmd.Op2, 0);

    if ((feature & CF_STOP) == 0) {
        // Ссылка потока на следующую последовательную инструкцию.
        ua_add_cref(0, cmd.ea + cmd.size, fl_F);
    }
    return 1;
}
```

Для каждого операнда свойства показывают, нужна ли ссылка. В примере `TouchArg` определяет её разновидность и создаёт. В эмуляторе следует применять функции из `ua.hpp`, а не из `xref.hpp`.

Полезны такие правила:

• для `o_imm`, читаемого операнда и значения-указателя создаётся ссылка-смещение; указатель проверяется, например, `isOff(uFlag, op.n)`, а ссылка добавляется `ua_add_off_drefs(op, dr_O)`;
• для `o_displ` со значением-указателем создаётся ссылка-смещение чтения или записи: `ua_add_off_drefs(op, isRead ? dr_R : dr_W)`;
• для `o_mem` создаётся ссылка на данные: `ua_add_dref(op.offb, op.addr, isRead ? dr_R : dr_W)`;

• для o_near создаётся ссылка на код вызова либо перехода: ua_add_cref(op.offb, op.addr, feature & CF_CALL ? fl_CN : fl_JN).

Эмулятор также сообщает об изменениях указателя стека:

```
bool add_auto_stkpnt2(func_t *pfn, ea_t ea, sval_t delta);
```

pfn указывает на содержащую адрес функцию; при NULL IDA определяет её сама. ea задаёт адрес конца изменяющей стек инструкции, обычно cmd.ea + cmd.size. delta - число байтов изменения: отрицательное при росте стека, например после push, и положительное при уменьшении, например после pop.

```
if (cmd.itype == X86_push) {
    add_auto_stkpnt2(NULL, cmd.ea + cmd.size, -4);
}
```

Нужно учитывать все способы изменения, а не только push и pop. Например, выделение локальных переменных вычитанием константы из esp:

```
// Обработка команды вида sub esp, 48h.
if (cmd.itype == X86_sub && cmd.Op1.type == o_reg
    && cmd.Op1.reg == r_esp && cmd.Op2.type == o_imm) {
    add_auto_stkpnt2(NULL, cmd.ea + cmd.size, -cmd.Op2.value);
}
```

Архитектуры слишком различаются, чтобы существовал универсальный рецепт эмулятора. Приходится изучать готовые исходники и проверять поведение опытным путём.

Эмулятор учебного процессора Python очень краток:

```
int idaapi py_emu(void) {
    // Разрешить можно только адреса относительных переходов.
    if (cmd.auxpref & HAS_JREL) {
        ua_add_cref(cmd.Op1.offb, cmd.Op1.addr, fl_JN);
    }
    // Последовательный поток существует, пока не задан CF_STOP.
    if ((cmd.get_canon_feature() & CF_STOP) == 0) {
        ua_add_cref(0, cmd.ea + cmd.size, fl_F);
    }
    return 1;
}
```

Архитектура Python сильно ограничивает возможные ссылки. У элементов данных нет адресов памяти: они находятся в таблицах и выбираются по индексам либо лежат в стеке. Абсолютные адреса инструкций определяются только разбором метаданных .pyc. Без этих метаданных неизвестна и структура таблиц, хотя индексы из операндов читаются напрямую.

Поэтому модуль вычисляет лишь цели относительных переходов и следующую инструкцию. Для лучшего результата ему нужно знать устройство файла; к этому ограничению вернёмся в разделе об архитектуре процессорного модуля.

Отслеживание стека Python также опущено: IDA считает такие изменения значимыми лишь внутри функции, а границы функций пока не распознаются. Если добавить эту возможность, придётся учесть, что почти каждая инструкция стековой машины меняет стек. Удобно создать массив величин изменения для всех команд и передавать соответствующее значение в add_auto_stkpnt2.

После добавления ссылок и остальных изменений остаётся сформировать текст дизассемблирования.

Генератор вывода

Он выводит в IDA одну разобранную инструкцию из cmd. Обычно это функция out в out.cpp:

```
void idaapi out(void); // вывести одну инструкцию
```

Указатель записывается в u_out структуры LPH. К этому моменту cmd уже заполнена, а изменять базу функция не должна.

Дополнительно требуется функция форматирования одного операнда, обычно outop, на которую указывает u_outop. Вызывать её напрямую из out не следует; для каждого операнда применяется out_one_operand. Данные выводит отдельная функция, обычно cpu_data, указанная в d_out; в нашем модуле она называется python_data.

Строка дизассемблирования состоит из префикса, метки, мнемоники, операндов и возможного комментария. Ядро рисует часть компонентов, включая префиксы, комментарии и ссылки, а остальное формирует процессорный модуль. Полезные функции находятся в разделе вывода вспомогательных функций IDP файла ua.hpp.

Части строки можно окрашивать специальными метками в выходном буфере. Дополнительные средства находятся в lines.hpp.

IDA использует не прямой консольный вывод, а буферную схему. Для каждой строки выполняются четыре действия:

1. Вызвать init_output_buffer(char *buf, size_t bufsize) из ua.hpp.
2. Добавить содержимое функциями out_xxx и OutXxx; большинство автоматически пишет в текущий буфер.
3. Завершить его вызовом term_output_buffer().
4. Передать строку ядру через MakeLine или printf_line из lines.hpp.

Инициализация, завершение и MakeLine обычно вызываются только из out. Функция outop пользуется уже подготовленным буфером. Можно сформировать буфер полностью самостоятельно и сразу вызвать MakeLine, но тогда теряются удобства ua.hpp, многие из которых также автоматически используют cmd.

Основные вспомогательные функции:

OutMnem(int width, char *suffix)

: Выводит мнемонику для cmd.itype в поле шириной не меньше width, добавляет suffix и хотя бы один пробел. По умолчанию ширина равна 8, а суффикс - NULL. Суффикс пригоден для размера операнда, например в movsb, movsw и movsd.

out_one_operand(int n)

: Вызывает outop процессора для cmd.Operands[n].

out_snprintf(const char *format, ...)

: Добавляет форматированный текст в текущий буфер.

OutValue(op_t &op, int outflags)

: Выводит постоянное поле операнда - op.value или op.addr в зависимости от outflags. Значение флагов описано в ua.hpp; по умолчанию они равны нулю. Обычно вызывается из outop.

out_symbol(char c)

: Выводит знак цветом пунктуации COLOR_SYMBOL. Применяется для запятых, скобок и других синтаксических частей операнда.

out_line(char *str, color_t color)

: Добавляет строку указанного цвета в текущий буфер. Несмотря на имя, целую строку вывода не завершает.

`OutLine(char *str)`

: То же без указания цвета.

`out_register(char *str)`

: Выводит текст цветом регистра `COLOR_REG`.

`out_tagon(color_t tag)` и `out_tagoff(color_t tag)`

: Включают и выключают заданный цвет для последующего содержимого.

В `ua.hpp` нет простой функции вывода имени регистра по номеру. Её легко добавить:

```
void OutReg(int regnum) {
    out_register(ph.regNames[regnum]);
}
```

IDA вызывает `out` по мере необходимости: когда адрес появляется на экране или строка переформатируется. Функция должна вывести столько строк, сколько требуется для представления текущей инструкции. Обычно достаточно одного `MakeLine`; если строк несколько, нельзя вставлять переводы строки в один буфер - нужно вызвать `MakeLine` отдельно для каждой.

```
bool MakeLine(const char *contents, int indent = -1);
```

Отступ -1 выбирает текущее значение `inf.indent` из раздела **Disassembly** окна **Options > General**. Для многострочной инструкции он также помечает главную строку. Подробности есть в комментариях к `printf_line` в `lines.hpp`.

Комментарии, как имена и ссылки, рисует ядро. Но глобальная переменная `gl_comm` из `lines.hpp` определяет, разрешён ли комментарий у текущей строки. По умолчанию она равна нулю, поэтому её необходимо установить в 1. При нескольких строках значение позволяет выбрать строку для пользовательского комментария.

Функция вывода процессора Python:

```
void py_out(void) {
    char str[MAXSTR]; // MAXSTR определён IDA в pro.h
    init_output_buffer(str, sizeof(str));
    OutMnem(12);
    if (cmd.Op1.type != o_void) {
        out_one_operand(0);
    }
    term_output_buffer();
    gl_comm = 1;
    MakeLine(str);
}
```

Если бы команда имела два операнда, между двумя вызовами `out_one_operand` можно было бы вывести запятую через `out_symbol`. Обычно `outop` сложнее `out`, поскольку внутреннее устройство операнда богаче общей структуры инструкции. Типичная реализация выбирает формат оператором `switch` по полю `type`.

В примере Python недостаточно сведений, чтобы преобразовать большинство целых операндов в более понятные значения. Особая обработка есть только для сравнений и относительных переходов:

```
char *compare_ops[] = {
    "<", "<=", "==", "!=", ">", ">=",
    "in", "not in", "is", "is not", "exception match"
};

bool idaapi py_outop(op_t& x) {
    if (cmd.itype == COMPARE_OP) {
        // Операнд сравнения обозначает его вид; выводим знак, а не число.
        if (x.value < qnumber(compare_ops)) {
            OutLine(compare_ops[x.value]);
```

```
        }
        else {
            OutLine("НЕВЕРНЫЙ ОПЕРАНД");
        }
    }
    else if (cmd.auxpref & HAS_JREL) {
        // HAS_JREL отличает относительный переход от абсолютного
        // и в нашем модуле подразумевает o_near.
        out_name_expr(x, x.addr, x.addr);
    }
    else {
        OutValue(x);
    }
    return true;
}
```

Кроме инструкций листинг содержит байты данных. За них отвечает поле `d_out` объекта `LPH`. Ядро вызывает эту функцию для любого байта вне инструкции: как с неизвестным типом, так и оформленного пользователем или эмулятором.

```
void idaapi d_out(ea_t ea); // вывести данные по адресу
```

Функция проверяет флаги адреса и формирует представление в синтаксисе выбранного ассемблера. Она обязательна для каждого процессорного модуля. SDK предлагает простейшую `intel_data`, но она редко удовлетворяет всем требованиям.

В Python статические данные почти не форматируются, поскольку их расположение неизвестно. Для примера используется такая реализация:

```
void idaapi python_data(ea_t ea) {
    char obuf[256];
    init_output_buffer(obuf, sizeof(obuf));
    flags_t flags = get_flags_novalue(ea);
    if (isWord(flags)) {
        out_snprintf("%s %xh", ash.a_word ? ash.a_word : "", get_word(ea));
    }
    else if (isDwrd(flags)) {
        out_snprintf("%s %xh", ash.a_dword ? ash.a_dword : "", get_long(ea));
    }
    else {
        int val = get_byte(ea);
        char ch = ' ';
        if (val >= 0x20 && val <= 0x7E) {
            ch = val;
        }
        out_snprintf("%s %02xh   ; %c",
                     ash.a_byte ? ash.a_byte : "", val, ch);
    }
    term_output_buffer();
    gl_comm = 1;
    MakeLine(obuf);
}
```

Средства чтения и проверки флагов адреса находятся в `bytes.hpp`. Здесь определяется слово либо двойное слово, после чего выбирается соответствующая директива текущего ассемблера. Глобальная `ash` - структура `asm_t`, описывающая используемый синтаксис. Для сложных данных, например массивов, логики потребовалось бы больше.

Уведомления процессора

В главе 17 модули расширения подписывались на уведомления через `hook_to_notification_point`. Процессорные модули тоже получают события, но иначе.

Поле `notify` структуры `LPH` должно указывать на функцию:

```
int idaapi notify(idp_notify msgid, ...);
```

Она принимает код события и зависящий от него набор аргументов. Все коды и параметры перечислены в `idp.hpp`. Есть простые события загрузки `init` и выгрузки `term`, а также уведомления о создании кода или данных, добавлении и удалении функций и сегментов.

Правильно работающий модуль должен сначала вызвать `invoke_callbacks`. Если результат равен нулю, событие обрабатывается самим процессором; иначе код сразу возвращается вызывающей стороне. Так ядро передаёт уведомление всем зарегистрированным обработчикам.

```
static int idaapi notify(processor_t::idp_notify msgid, ...) {
    va_list va;
    va_start(va, msgid);
    int result = invoke_callbacks(HT_IDP, msgid, va);
    if (result == 0) {
        result = 1; // успех по умолчанию
        switch (msgid) {
            case processor_t::init:
                inf.mf = 0; // порядок байтов от младшего к старшему
                break;
            case processor_t::make_data: {
                ea_t ea = va_arg(va, ea_t);
                flags_t flags = va_arg(va, flags_t);
                tid_t tid = va_arg(va, tid_t);
                asize_t len = va_arg(va, asize_t);
                if (len > 4) {
                    // d_out поддерживает только байт, слово и двойное слово.
                    result = 0;
                }
                break;
            }
        }
    }
    va_end(va);
    return result;
}
```

Эта функция обрабатывает лишь `init` и `make_data`. Первое событие явно выбирает порядок байтов от младшего к старшему. Флаг `inf.mf` означает размещение старшего байта первым.

Событие `make_data` приходит при попытке преобразовать байты в данные. Поскольку `d_out` поддерживает только байт, слово и двойное слово, размер больше четырёх запрещается.

Прочие поля processor_t

В `LPH` остаётся множество указателей на функции. Определение `processor_t` в `idp.hpp` иногда прямо разрешает `NULL`; такие функции ядро не вызывает. Для остальных безопаснее предоставить хотя бы пустую заглушку, если требуемое поведение пока неясно.

В процессоре Python поля настроены так:

Поле	Значение
`header`	Пустая функция
`footer`	Пустая функция
`segstart`	Пустая функция
`segend`	Пустая функция

Поле	Значение
is_far_jump	NULL
translate	NULL
realcvt	ieee_realcvt из ieee.h
is_switch	NULL
extract_address	Функция, возвращающая BADADDR - 1
is_sp_based	NULL
create_func_frame	NULL
get_frame_retsize	NULL
u_outspec	NULL
set_idp_options	NULL

Ещё три поля данных особенно важны:

shnames

: Завершённый NULL массив коротких, менее девяти знаков, имён процессора, например python.

lnames

: Завершённый NULL массив длинных имён, например Байт-код Python 2.4. Число элементов должно совпадать с shnames.

asms

: Завершённый NULL массив указателей на структуры целевых ассемблеров asm_t.

Массивы имён перечисляют все процессоры, поддерживаемые модулем. Пользователь выбирает вариант на вкладке **Analysis** окна **Options > General**.

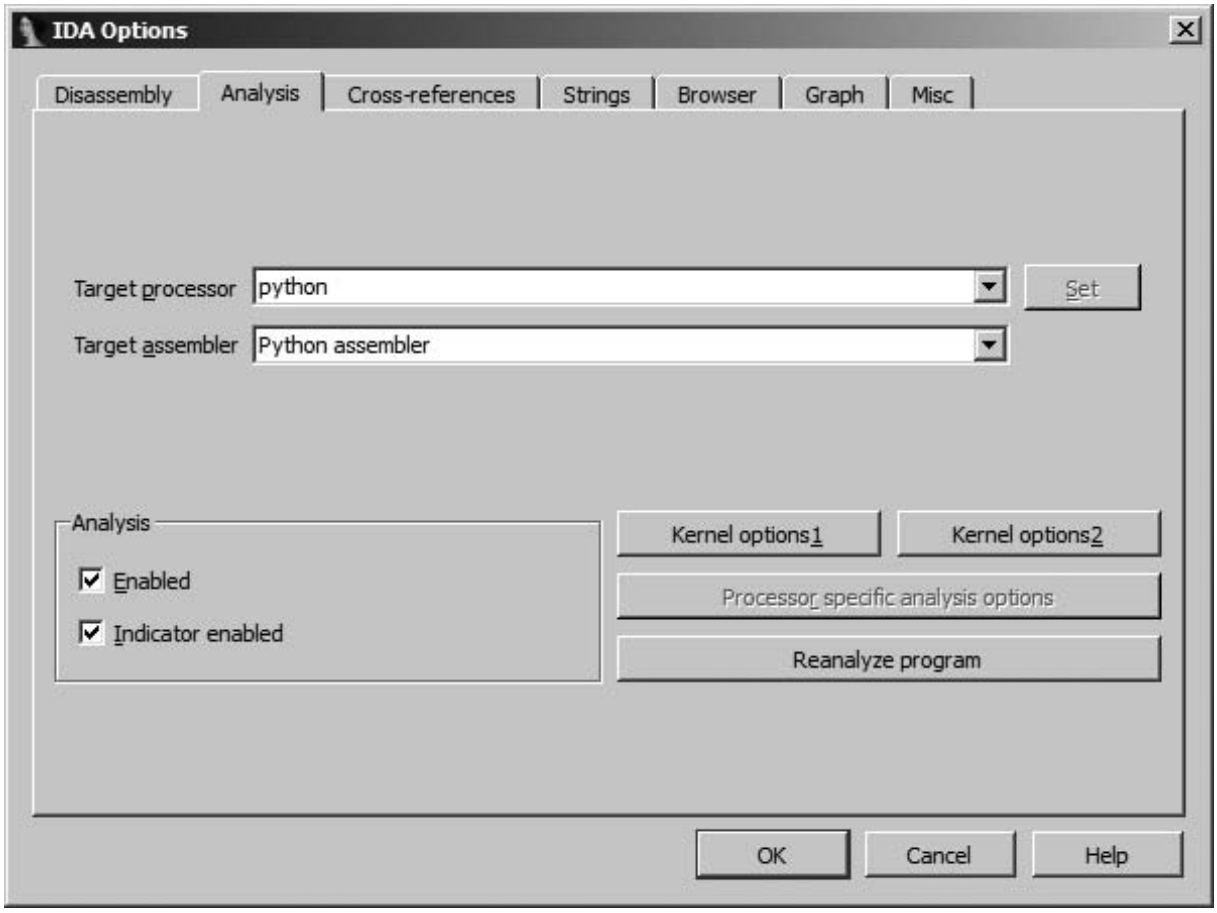


Рисунок 19-1. Выбор другого процессора и ассемблера

Модуль с несколькими процессорами должен обрабатывать уведомление `processor_t.newprc`.

Структура `asm_t` описывает синтаксис ассемблера: запись шестнадцатеричных чисел, ограничители строк и знаков, а также директивы. Поле `asms` позволяет одному процессору поддерживать несколько стилей. При смене ассемблера приходит уведомление `processor_t.newasm`.

Завершённый учебный модуль создаёт примерно такой листинг:

```
ROM:00156                LOAD_CONST 12
ROM:00159                COMPARE_OP ==
ROM:00162                JUMP_IF_FALSE loc_182
ROM:00165                POP_TOP
ROM:00166                LOAD_NAME 4
ROM:00169                LOAD_ATTR 10
ROM:00172                LOAD_NAME 5
ROM:00175                CALL_FUNCTION 1
ROM:00178                POP_TOP
ROM:00179                JUMP_FORWARD loc_183
ROM:00182 # ------------------------------------------------------
ROM:00182 loc_182:                          # CODE XREF: ROM:00162j
ROM:00182                POP_TOP
ROM:00183
ROM:00183 loc_183:                          # CODE XREF: ROM:00179j
ROM:00183                LOAD_CONST 0
ROM:00186                RETURN_VALUE
```

Более содержательное представление потребовало бы глубокого разбора формата `.pyc`, который в упрощённом примере не предполагается.

Сборка процессорных модулей

Процесс похож на сборку расширений и загрузчиков, но есть одно обязательное отличие, без которого IDA не сможет использовать модуль.

Основные особенности:

1. Расширения файлов: `.w32` и `.w64` в Windows, `.ilx` и `.ilx64` в Linux, `.imc` и `.imc64` в macOS.
2. Примеры SDK помещают результат в `<SDKDIR>/bin/procs`.
3. Для установки файл копируют в `<IDADIR>/procs`.
4. Модулю Windows требуется особая заглушка MS-DOS из SDK.[1]
5. После сборки Windows необходимо вставить строку описания в определённое место файла. Она показывается в списке процессоров окна загрузки.

Заглушка находится в `<SDKDIR>/module/stub`. Компоновщику нужно велеть использовать её вместо стандартной. Средства Borland, применявшиеся Hex-Rays, умеют задавать её через файл определения модуля `.def`; SDK включает `<SDKDIR>/module/idp.def`.

Компоновщики GNU и Microsoft понимают файлы `.def` с немного разным синтаксисом, но не позволяют выбрать заглушку MS-DOS. Даже после включения правильной заглушки остаётся вставить описание утилитой `<SDKDIR>/bin/mkidp.exe`:

```
mkidp модуль описание
```

Описание имеет форму:

```
Длинное имя модуля:короткое имя
```

Для процессора Python:

```
./mkidp procs/python.w32 "Байт-код Python:python"
```

`mkidp` пытается записать строку по смещению 128 байт между заглушкой MS-DOS и заголовком PE. Если места недостаточно, появляется ошибка:

```
mkidp: слишком длинное описание процессора
```

Модуль, созданный компоновщиком Microsoft, обычно оставляет достаточно места, а результат GNU - нет. Для независимости от инструментов авторы разработали утилиту `fix_proc` с тем же синтаксисом. Она заменяет существующую заглушку вариантом SDK, переносит заголовки PE, освобождая место, и вставляет описание. В сборке она заменяет `mkidp`.

Строго говоря, сама заглушка SDK не обязательна: IDA требуется лишь строка описания по смещению 128. `fix_proc` заменяет заглушку, чтобы гарантированно избежать конфликта за это место.

Инструменты	Используется `.def`	Заглушка сразу	После `mkidp`: заглушка	После `mkidp`: описание	После `fix_proc`: заглушка	После `fix_proc`: описание
Borland	Да	Да	Да	Да	Да	Да
Microsoft	Нет	Нет	Нет	Да	Да	Да
GNU	Нет	Нет	Нет	Нет	Да	Да

Таблица 19-1. Последующая обработка процессорных модулей по компилятору

Только модуль с допустимым описанием появляется в окне загрузки; без него выбрать процессор невозможно.

Файл сборки из главы 17 требует больше изменений, чем для загрузчика. Ниже начинается вариант для процессора Python:

```
# Каталог SDK
IDA_SDK=../../

PLATFORM=$(shell uname | cut -f 1 -d _)

ifneq "$(PLATFORM)" "MINGW32"
IDA=$(HOME)/ida
endif

# Имя собираемого процессора
PROC=python

# Обязательное описание: <длинное имя>:<короткое имя>
DESCRIPTION=Байт-код Python:python

ifeq "$(PLATFORM)" "MINGW32"
PLATFORM_CFLAGS=-D__NT__ -D__IDP__ -DWIN32 -Os -fno-rtti
PLATFORM_LDFLAGS=-shared -s
LIBDIR=$(shell find ../../ -type d | grep -E "(lib|lib/)gcc.w32")
ifeq ($(strip $(LIBDIR)),)
LIBDIR=../../lib/x86_win_gcc_32
endif
IDALIB=$(LIBDIR)/ida.a
PROC_EXT=.w32

else ifeq "$(PLATFORM)" "Linux"
PLATFORM_CFLAGS=-D__LINUX__
PLATFORM_LDFLAGS=-shared -s
IDALIB=-lida
IDADIR=-L$(IDA)
PROC_EXT=.ilx

else ifeq "$(PLATFORM)" "Darwin"
PLATFORM_CFLAGS=-D__MAC__
PLATFORM_LDFLAGS=-dynamiclib
IDALIB=-lida
IDADIR=-L$(IDA)/idaq.app/Contents/MacOs
PROC_EXT=.imc
endif

# Параметры компилятора для платформы
CFLAGS=-Wextra $(PLATFORM_CFLAGS)

# Параметры компоновщика для платформы
LDFLAGS=$(PLATFORM_LDFLAGS)

# Дополнительные библиотеки
EXTRALIBS=

# Каталог назначения
OUTDIR=$(IDA_SDK)bin/procs/

# Средство последующей вставки описания процессора
MKIDP=$(IDA_SDK)bin/fix_proc
#MKIDP=$(IDA)bin/mkidp

OBJS=ana.o emu.o ins.o out.o reg.o
BINARY=$(OUTDIR)$(PROC)$(PROC_EXT)

all: $(OUTDIR) $(BINARY)

clean:
 -@rm *.o
 -@rm $(BINARY)

$(OUTDIR):
 -@mkdir -p $(OUTDIR)
```

```
CC=g++
INC=-I$(IDA_SDK)include/

%.o: %.cpp
 $(CC) -c $(CFLAGS) $(INC) $< -o $@

LD=g++

ifeq "$(PLATFORM)" "MINGW32"
# Процессор Windows требует последующей обработки.
$(BINARY): $(OBJS)
 $(LD) $(LDFLAGS) -o $@ $(OBJS) $(IDALIB) $(EXTRALIBS)
 $(MKIDP) $(BINARY) "$(DESCRIPTION)"
else
$(BINARY): $(OBJS)
 $(LD) $(LDFLAGS) -o $@ $(OBJS) $(IDALIB) $(EXTRALIBS)
endif

# Замените python именем процессора и добавьте зависимости.
python.o: python.cpp
ana.o: ana.cpp
emu.o: emu.cpp
ins.o: ins.cpp
out.o: out.cpp
reg.o: reg.cpp
```

Листинг 19-1. Файл сборки процессорного модуля Python

Главные изменения - строка `DESCRIPTION`, выбор утилиты вставки описания и отдельный шаг последующей обработки для Windows.

Настройка существующих процессоров

Иногда штатный процессор делает почти всё необходимое. При наличии исходного кода его можно изменить напрямую, но IDA позволяет настраивать даже закрытый модуль через расширение. Подписавшись на события процессора, расширение перехватывает анализатор, эмулятор или генератор вывода.

Так можно:

• распознавать дополнительные инструкции;
• исправить ошибочное поведение существующего модуля;
• изменить формат вывода под конкретные требования.

Для перехвата используются коды из `processor_t`, описанные в `idp.hpp`:

`custom_ana`

: Работает как `u_ana`; новая инструкция должна иметь `cmd.itype` не меньше `0x8000`.

`custom_emu`

: Эмулирует собственные типы. Существующий эмулятор вызывается через `(*ph.u_emu)()`.

`custom_out`

: Выводит собственные инструкции или меняет представление штатных. Исходный `out` вызывается через `(*ph.u_out)()`.

`custom_outop`

: Выводит один особый операнд. Исходная функция вызывается как `(*ph.u_outop)(op)`.

`custom_mnem`

: Создаёт мнемонику собственной инструкции.

Следующий модуль заменяет инструкцию `leave` процессора x86 на `cya` и меняет местами два операнда, приближая запись к синтаксису AT&T:

```
int idaapi init(void) {
    if (ph.id != PLFM_386) return PLUGIN_SKIP;
    hook_to_notification_point(HT_IDP, hook, NULL);
    return PLUGIN_KEEP;
}

int idaapi hook(void *user_data, int notification_code, va_list va) {
    switch (notification_code) {
        case processor_t::custom_out: {
            if (cmd.itype == NN_leave) {
                MakeLine(SCOLOR_ON SCOLOR_INSN "cya" SCOLOR_OFF);
                return 2;
            }
            else if (cmd.Op2.type != o_void) {
                // Временная перестановка двух операндов.
                op_t op1 = cmd.Op1;
                op_t op2 = cmd.Op2;
                cmd.Op1 = op2;
                cmd.Op2 = op1;
                (*ph.u_out)();
                cmd.Op1 = op1;
                cmd.Op2 = op2;
                return 2;
            }
        }
    }
    return 0;
}

plugin_t PLUGIN = {
    IDP_INTERFACE_VERSION,
    PLUGIN_PROC | PLUGIN_HIDE | PLUGIN_MOD,
    init,         // инициализация
    term,         // завершение; допускается NULL
    run,          // запуск
    comment,      // длинное описание
    help,         // многострочная справка
    wanted_name,  // короткое имя
    wanted_hotkey // клавиша запуска
};
```

`init` проверяет процессор x86 и подписывается на события. Обработчик `custom_out` перехватывает `leave` и выводит другую строку. Для двух операндов их значения временно меняются местами, затем вызывается штатная `u_out`, после чего исходный порядок восстанавливается.

Флаги требуют загрузить расширение вместе с процессором, скрывают его из меню **Edit > Plugins** и отмечают изменение базы. Результат:

```
.text:00401350            push    ebp
.text:00401351            mov     400000h, edx
.text:00401356            mov     esp, ebp
.text:00401358            mov     offset unk_402060, eax
.text:0040135D            sub     0Ch, esp
.text:00401360            mov     edx, [esp+8]
.text:00401364            mov     eax, [esp+4]
.text:00401368            mov     offset unk_402060, [esp]
.text:0040136F            call    sub_401320
.text:00401374            cya
.text:00401375            retn
```

Константы теперь стоят первыми в четырёх инструкциях, а вместо `leave` показана `cya`.

Архитектура процессорного модуля

При проектировании нужно решить, будет ли процессор тесно связан с одним загрузчиком или останется независимым.

Например, процессор x86 ничего не предполагает о формате файла и одинаково работает с PE, ELF и Mach-O. Хороший загрузчик тоже может не зависеть от процессора: PE поддерживает x86 и ARM, ELF - x86, MIPS и SPARC, Mach-O - PPC и x86.

Для настоящих процессоров независимость естественна. С виртуальными машинами сложнее: сама машина обычно одновременно играет роль загрузчика и процессора, поэтому формат файла тесно связан с байт-кодом. В примере Python отсутствие знаний о `.pyc` неоднократно мешало создавать понятный вывод.

Можно написать загрузчик Python, который особым образом строит базу и оставляет нужные процессору сведения в известных местах. Значительный объём состояния между модулями удобно передавать через сетевые узлы базы.

Другой вариант - загрузчик лишь распознаёт `.pyc`, а всю загрузку поручает процессору. Тогда тот сам получает и сохраняет все необходимые сведения. Так устроены загрузчик и процессор Java в SDK.

Для передачи работы загрузчик сначала распознаёт файл с типом `f_LOADER` из `ida.hpp`. После выбора он при необходимости задаёт процессор через `set_processor_type` из `idp.hpp`, затем отправляет уведомление `loader`:

```
void idaapi load_file(linput_t *li, ushort neflag, const char *) {
    if (ph.id != PLFM_PYTHON) {
        set_processor_type("python", SETPROC_ALL | SETPROC_FATAL);
    }
    // Процессор Python выполняет загрузку сам.
    if (ph.notify(processor_t::loader, li, neflag)) {
        error("Ошибка совместной работы процессора и загрузчика Python");
    }
}
```

Получив событие, процессор отображает файл в базу и сохраняет всё необходимое для этапов `ana`, `emu` и `out`.

Процессорный модуль в виде сценария

Начиная с IDA 5.7 процессор можно реализовать на языке сценариев. Это как минимум устраняет этап сборки. Процессор байт-кода EFI в IDA написан на Python и находится в `<IDADIR>/procs/ebc.py`.

Лучшей отправной точкой служит шаблон `<SDKDIR>/module/script/proctemplate.py`, перечисляющий обязательные поля. Полезны и поставляемые с IDA процессоры Python: их структура понятнее многофайловых примеров C++, требующих настроенной сборки.

На высоком уровне нужны две вещи:

1. Подкласс `idaapi.processor_t`, реализующий обязательные функции `emu`, `ana`, `out` и `outop`.
2. Отдельная функция `PROCESSOR_ENTRY`, возвращающая экземпляр класса.

```
from idaapi import *

class demo_processor_t(idaapi.processor_t):
    # Здесь инициализируются обязательные поля, включая id и assembler.
    # assembler - словарь всех полей asm_t.
```

```
    # Также нужен список instruc; каждый его элемент является словарём
    # из двух значений с ключами name и feature.

    def ana(self):
        # Поведение анализатора.
        pass

    def emu(self):
        # Поведение эмулятора.
        pass

    def out(self):
        # Поведение генератора вывода.
        pass

    def outop(self):
        # Поведение вывода операнда.
        pass

def PROCESSOR_ENTRY():
    return demo_processor_t()
```

Настоящий модуль содержит намного больше полей и функций, практически повторяя требования C++. Для установки сценарий копируют в `<IDADIR>/procs`.

Итоги

Процессорные модули - самые сложные расширения IDA. Их изучение и разработка требуют времени, хотя сценарии заметно упрощают путь. В узкой области обратной разработки или при исследовании новых архитектур необходимость в собственном процессоре почти неизбежна. Терпение, изучение примеров и многократная проверка здесь особенно важны, зато готовый модуль можно повторно применять к каждому новому двоичному файлу.

Этой главой завершается обзор расширяемости IDA. Далее рассматривается практическое применение программы и её расширений в реальных задачах анализа.

Сноски

[1] Заглушка содержит заголовок MS-DOS и код, сообщающий, что программу Windows нельзя выполнить в режиме MS-DOS.

Глава 20. Особенности компиляторов

К этому моменту у читателя уже должны быть основные навыки эффективной работы с IDA и её настройки. Теперь пора учиться справляться с неожиданностями самих двоичных файлов.

В зависимости от задач один вид ассемблерного кода может стать привычным, а другой - полностью озадачить. Постоянное исследование программ GCC для Linux хорошо знакомит с их характерным стилем, но отладочная сборка Microsoft Visual C++ способна выглядеть совершенно иначе.

Особенно разнообразен материал у исследователей вредоносных программ: за один день встречаются Visual Basic, Delphi, Visual C/C++ и блоки машинного кода внутри документов, даже если пока не учитывать намеренное запутывание.

В этой главе кратко рассматриваются различия компиляторов, видимые в IDA. Нас интересуют не причины, а проявление этих различий в дизассемблированном тексте и способы его правильного толкования. Компилятор и параметры сборки также помогают составить профиль автора программы.

Хотя языков и компиляторов много, примеры в основном написаны на C, для которого существует множество средств и платформ.

Таблицы переходов и оператор switch

Оператор C `switch` часто оптимизируется. Компилятор стремится как можно быстрее сопоставить проверяемое значение с меткой `case`; способ зависит от распределения меток.

Если числа сильно разнесены, большинство компиляторов создаёт двоичный поиск:[1]

```
switch (value) {
    case 1:
        // Код для value == 1.
        break;
    case 211:
        // Код для value == 211.
        break;
    case 295:
        // Код для value == 295.
        break;
    case 462:
        // Код для value == 462.
        break;
    case 1093:
        // Код для value == 1093.
        break;
    case 1839:
        // Код для value == 1839.
        break;
}
```

Для тесно сгруппированных, особенно последовательных меток обычно используется поиск по таблице,[2] связывающей значение с адресом обработчика:

```
switch (value) {
    case 1:
        // Код для value == 1.
        break;
    case 2:
        // Код для value == 2.
        break;
```

```
    case 3:
        // Код для value == 3.
        break;
    case 4:
        // Код для value == 4.
        break;
    case 5:
        // Код для value == 5.
        break;
    case 6:
        // Код для value == 6.
        break;
}
```

Ниже показана сборка оператора с последовательными вариантами от 1 до 12 компилятором Borland:

```
.text:00401155         mov     edx, [ebp+arg_0]
.text:00401158         cmp     edx, 0Ch             ; switch: 13 вариантов
.text:0040115B         ja      loc_4011F1           ; значение по умолчанию
.text:0040115B                                      ; таблица: вариант 0
.text:00401161         jmp     ds:off_401168[edx*4] ; табличный переход
.text:00401161 ; ---------------------------------------------------------
.text:00401168 off_401168 dd offset loc_4011F1      ; ссылка из sub_401150+11
.text:00401168         dd offset loc_40119C         ; таблица switch
.text:00401168         dd offset loc_4011A1
.text:00401168         dd offset loc_4011A6
.text:00401168         dd offset loc_4011AB
.text:00401168         dd offset loc_4011B3
.text:00401168         dd offset loc_4011BB
.text:00401168         dd offset loc_4011C3
.text:00401168         dd offset loc_4011CB
.text:00401168         dd offset loc_4011D3
.text:00401168         dd offset loc_4011DB
.text:00401168         dd offset loc_4011E3
.text:00401168         dd offset loc_4011EB
.text:0040119C ; ---------------------------------------------------------
.text:0040119C loc_40119C:
.text:0040119C         mov     eax, [ebp+arg_4]     ; вариант 1
```

IDA хорошо понимает вывод Borland и сама добавляет пояснения: распознаёт проверку, таблицу переходов и обработчики отдельных значений.

Таблица содержит 13 элементов, хотя исходник явно задаёт лишь варианты 1-12. Компилятор добавил индекс 0 вместо отдельной обработки; он ведёт туда же, куда любое значение вне допустимого диапазона.

Сравнение с 12 и последующая инструкция `ja` на первый взгляд могут пропускать отрицательные числа. Но `ja` трактует сравнение как беззнаковое: -1, то есть `0xFFFFFFFF`, становится 4294967295 и исключается как число больше 12. Отрицательный индекс к таблице не попадёт.

Microsoft Visual C++ создаёт другой листинг:

```
.text:004013D5            mov     ecx, [ebp+var_8]
.text:004013D8            sub     ecx, 1
.text:004013DB            mov     [ebp+var_8], ecx
.text:004013DE            cmp     [ebp+var_8], 0Bh ; switch: 12 вариантов
.text:004013E2            ja      loc_40146E       ; значение по умолчанию
.text:004013E8            mov     edx, [ebp+var_8]
.text:004013EB            jmp     ds:off_401478[edx*4]
.text:004013F2 loc_4013F2:
.text:004013F2            mov     eax, [ebp+arg_4] ; вариант 0 после сдвига
; ... остаток функции опущен ...
.text:00401477            retn
.text:00401477 sub_4013B0 endp
.text:00401478 off_401478 dd offset loc_4013F2
.text:00401478            dd offset loc_4013FA
.text:00401478            dd offset loc_401402
.text:00401478            dd offset loc_40140A
```

```
.text:00401478            dd offset loc_401415
.text:00401478            dd offset loc_401420
.text:00401478            dd offset loc_40142B
.text:00401478            dd offset loc_401436
.text:00401478            dd offset loc_401441
.text:00401478            dd offset loc_40144C
.text:00401478            dd offset loc_401458
.text:00401478            dd offset loc_401464
```

Здесь таблица вынесена сразу за функцией, а не встроена внутрь неё. Это чётче разделяет код и данные, почти не меняя поведения. IDA всё равно распознаёт число вариантов и соответствующие блоки.

Переменная `var_8` уменьшается на единицу, переводя допустимый диапазон 1-12 в 0-11. Теперь её можно использовать как индекс без фиктивного элемента для исходного нуля. Поэтому нулевой элемент таблицы фактически ведёт к `case 1`.

Тот же исходник после GCC:

```
.text:004011FA          cmp     [ebp+arg_0], 0Ch ; switch: 13 вариантов
.text:004011FE          ja      loc_40129D       ; вариант 0, по умолчанию
.text:00401204          mov     eax, [ebp+arg_0]
.text:00401207          shl     eax, 2
.text:0040120A          mov     eax, ds:off_402010[eax]
.text:00401210          jmp     eax              ; табличный переход
.text:00401212 loc_401212:
.text:00401212          mov     eax, [ebp+arg_4] ; вариант 1
; ... остаток раздела .text опущен ...
.rdata:00402010 off_402010 dd offset loc_40129D
.rdata:00402010            dd offset loc_401212
.rdata:00402010            dd offset loc_40121D
.rdata:00402010            dd offset loc_401225
.rdata:00402010            dd offset loc_40122D
.rdata:00402010            dd offset loc_40123C
.rdata:00402010            dd offset loc_40124B
.rdata:00402010            dd offset loc_40125A
.rdata:00402010            dd offset loc_401265
.rdata:00402010            dd offset loc_401270
.rdata:00402010            dd offset loc_40127B
.rdata:00402010            dd offset loc_401287
.rdata:00402010            dd offset loc_401293
```

Как у Borland, сравнение выполняется с 12, таблица содержит 13 элементов, а индекс 0 ведёт к обработчику по умолчанию. Индекс 1 указывает на `case 1`. Отличаются способ косвенного перехода и размещение таблицы в разделе только для чтения `.rdata`, отдельно от кода.

Единственного правильного способа перевести исходник в ассемблер нет. Знакомство с одним компилятором не гарантирует узнавания конструкций другого или даже иной версии того же семейства. Не следует считать, что перед вами не `switch`, лишь потому что IDA не добавила соответствующий комментарий: одни шаблоны ей знакомы лучше других.

Поэтому автоматический анализ необходимо дополнять знанием ассемблера, особенностей компиляторов и самостоятельным исследованием.

Реализации RTTI

В главе 8 рассматривалась идентификация типов времени выполнения C++ - RTTI. Единого стандарта её двоичной реализации нет, поэтому способность IDA автоматически распознавать такие структуры зависит от компилятора. Особенно хорошо поддерживаются двоичные файлы Borland. Для Microsoft существуют сторонние сценарии IDC и модуль Class Informer, который будет упомянут в главе 23.

Чтобы понять реализацию конкретного компилятора, можно написать небольшую программу с классами и виртуальными функциями, собрать её и найти в IDA строки с именами классов.

Независимо от компилятора структура RTTI содержит указатель на строку имени представляемого класса. По ссылкам на данные можно найти указатель на такую строку, а значит, вероятную структуру RTTI. Затем следует идти по обратным ссылкам на данные, пока не встретится таблица указателей на функции - виртуальная таблица соответствующего класса.

Поиск main

При наличии исходного кода анализ C/C++ естественно начинать с `main`, где логически начинается программа. Для двоичного файла стратегия тоже полезна, но компилятор, компоновщик и библиотеки добавляют выполняемый до неё код. Поэтому точка входа файла обычно не совпадает с функцией автора.

Само требование `main` - соглашение C/C++, а не универсальное правило. Графическая программа Windows может использовать `WinMain`, другие языки выбирают иные имена. Здесь любую главную пользовательскую функцию будем условно называть `main`.

Как описано в главе 12, IDA применяет специальные стартовые сигнатуры. Если начальная последовательность совпадает с известной, программа понимает поведение служебной функции и находит `main`. Но код запуска тесно связан с компилятором и платформой, а все возможные варианты охватить нельзя.

Сигнатуры сгруппированы по типу файла: для PE они находятся в `pe.sig`, для MS-DOS - в `exe.sig`. Наличие файла не гарантирует распознавание. Для многих форматов, включая ELF и Mach-O, стартовых сигнатур вообще нет; `main` в ELF обнаружится лишь при сохранённом имени либо другими средствами.

Иногда главную функцию придётся искать самостоятельно. Особенно это вероятно после запутывания стартовой процедуры: даже после распутывания необходимо найти и исходный запуск, и `main`.

В традиционной программе C/C++ стартовый код подготавливает три аргумента:

- `argc` - число аргументов командной строки;
- `argv` - массив указателей на их строки;
- `envp` - массив указателей на строки переменных окружения.

Следующий фрагмент взят из динамически скомпонованного файла FreeBSD 8.0 без символов, собранного GCC:

```
.text:08048365          mov     dword ptr [esp], offset _term_proc
.text:0804836C          call    _atexit
.text:08048371          call    _init_proc
.text:08048376          lea     eax, [ebp+arg_0]
.text:08048379          mov     [esp+8], esi
.text:0804837D          mov     [esp+4], eax
.text:08048381          mov     [esp], ebx
.text:08048384          call    sub_8048400
.text:08048389          mov     [esp], eax      ; код завершения
.text:0804838C          call    _exit
```

Вызов `sub_8048400` ведёт к `main`. Перед ним находятся служебные `_atexit` и `_init_proc`, после возврата - `_exit`, обеспечивающая корректное завершение. В неё передаётся результат `main` из `EAX`, поэтому возвращаемое значение становится кодом завершения процесса.

В статически скомпонованном файле без символов структура осталась бы такой же, но имена библиотечных функций пропали бы. `main` всё равно выделялась бы как единственная функция с тремя аргументами. Раннее применение FLIRT восстановило бы многие имена и дополнительно упростило поиск.

На другой платформе тот же GCC создаёт совершенно иной стиль. Динамически скомпонованный файл Linux без символов начинается так:

```
.text:080482B0 start           proc near
.text:080482B0                 xor     ebp, ebp
.text:080482B2                 pop     esi
.text:080482B3                 mov     ecx, esp
.text:080482B5                 and     esp, 0FFFFFFF0h
.text:080482B8                 push    eax
.text:080482B9                 push    esp
.text:080482BA                 push    edx
.text:080482BB                 push    offset sub_80483C0
.text:080482C0                 push    offset sub_80483D0
.text:080482C5                 push    ecx
.text:080482C6                 push    esi
.text:080482C7                 push    offset loc_8048384
.text:080482CC                 call    ___libc_start_main
.text:080482D1                 hlt
.text:080482D1 start           endp
```

Единственный вызов `___libc_start_main` выполняет инициализацию, вызывает `main`, а затем завершает процесс. Адрес главной функции передаётся как первый аргумент, то есть последним помещённым в стек: `loc_8048384`.

IDA не обозначила этот адрес как функцию по двум причинам. Во-первых, на него нет прямого `call`. Во-вторых, его пролог не совпадает с эвристикой IDA, хотя два других указателя были распознаны по своим прологам:

```
.text:08048384 loc_8048384:                ; ссылка из start+17
.text:08048384          lea     ecx, [esp+4]       ; адрес arg_0 в ecx
.text:08048388          and     esp, 0FFFFFFF0h    ; выравнивание esp на 16 байт
.text:0804838B          push    dword ptr [ecx-4]  ; копия адреса возврата
.text:0804838E          push    ebp                 ; сохранение ebp вызывающей функции
.text:0804838F          mov     ebp, esp            ; указатель текущего кадра
.text:08048391          push    ecx                 ; сохранение ecx
.text:08048392          sub     esp, 24h            ; локальные переменные
```

Здесь есть обычные сохранение `EBP`, установка кадра и выделение локальных данных, но они выполняются не первыми. Функцию легко создать вручную командой **Edit > Functions > Create Function**. Затем следует проверить, распознала ли IDA кадр на основе BP. При необходимости свойства меняются сочетанием `Alt-P`: принудительно включается кадр BP и корректируются размеры сохранённых регистров и локальных переменных.

В статически скомпонованном Linux-файле без символов имя `___libc_start_main` исчезло бы, однако структура не изменилась бы: стартовая процедура делает один вызов, а её первый аргумент содержит адрес `main`.

В Windows компиляторов и стартовых процедур больше. Но знания о GCC на других платформах иногда помогают. Вот начало файла GCC/Cygwin:

```
.text:00401000 start     proc near
.text:00401000 var_28    = dword ptr -28h
.text:00401000 var_24    = dword ptr -24h
.text:00401000 var_20    = dword ptr -20h
.text:00401000 var_2     = word ptr -2
.text:00401000           push    ebp
.text:00401001           mov     ebp, esp
.text:00401003           sub     esp, 28h
.text:00401006           and     esp, 0FFFFFFF0h
.text:00401009           fnstcw  [ebp+var_2]
.text:0040100C           movzx   eax, [ebp+var_2]
```

```
.text:00401010          and     ax, 0F0C0h
.text:00401014          mov     [ebp+var_2], ax
.text:00401018          movzx   eax, [ebp+var_2]
.text:0040101C          or      ax, 33Fh
.text:00401020          mov     [ebp+var_2], ax
.text:00401024          fldcw   [ebp+var_2]
.text:00401027          mov     [esp+28h+var_28], offset sub_4010B0
.text:0040102E          call    sub_401120
```

Код мало похож на Linux, но тоже вызывает единственную функцию и передаёт ей указатель на функцию. `sub_401120` играет роль `___libc_start_main`, а `sub_4010B0` является `main`.

GCC/MinGW применяет ещё один вариант:

```
.text:00401280 start          proc near
.text:00401280 var_8          = dword ptr -8
.text:00401280                push    ebp
.text:00401281                mov     ebp, esp
.text:00401283                sub     esp, 8
.text:00401286                mov     [esp+8+var_8], 1
.text:0040128D                call    ds:__set_app_type
.text:00401293                call    sub_401150
.text:00401293 start          endp
```

Здесь `main` снова не распознаётся. Единственная небиблиотечная `sub_401150` не получает видимых аргументов, поэтому поиск продолжается внутри неё:

```
.text:0040122A                call    __p__environ
.text:0040122F                mov     eax, [eax]
.text:00401231                mov     [esp+8], eax
.text:00401235                mov     eax, ds:dword_404000
.text:0040123A                mov     [esp+4], eax
.text:0040123E                mov     eax, ds:dword_404004
.text:00401243                mov     [esp], eax
.text:00401246                call    sub_401395
.text:0040124B                mov     ebx, eax
.text:0040124D                call    _cexit
.text:00401252                mov     [esp], ebx
.text:00401255                call    ExitProcess
```

`sub_401395` - единственная небиблиотечная функция с тремя аргументами. Третий связан с результатом `__p__environ`, что соответствует ожидаемому `envp`. Ранее также вызывается `getmainargs`, подготавливающая `argc` и `argv`, - ещё один признак близкого вызова `main`.

Старт Visual C/C++ короток:

```
.text:0040134B start          proc near
.text:0040134B                call    ___security_init_cookie
.text:00401350                jmp     ___tmainCRTStartup
.text:00401350 start          endp
```

Имена распознаны стартовыми сигнатурами, а не импортом из динамической библиотеки. Та же сигнатура позволяет найти вызов главной функции:

```
.text:004012D8                mov     eax, envp
.text:004012DD                mov     dword_40ACF4, eax
.text:004012E2                push    eax             ; envp
.text:004012E3                push    argv            ; argv
.text:004012E9                push    argc            ; argc
.text:004012EF                call    _main
.text:004012F4                add     esp, 0Ch
.text:004012F7                mov     [ebp+var_1C], eax
.text:004012FA                cmp     [ebp+var_20], 0
.text:004012FE                jnz     short $LN35
.text:00401300                push    eax             ; uExitCode
.text:00401301                call    $LN27
.text:00401306 $LN35:
.text:00401306                call    __cexit
.text:0040130B                jmp     short loc_40133B
```

Во всём теле tmainCRTStartup только _main вызывается с тремя аргументами. Перед ней находится GetCommandLine, дополнительно указывающая на подготовку главной функции.

Имя _main полностью создано IDA по сигнатуре: строки main в файле не было. Значит, при совпадении стартовой сигнатуры главная функция может получить имя даже в файле без символов.

Последний пример - бесплатный консольный компилятор Borland. Конец его стартовой функции:

```
.text:00401041                push    offset off_4090B8
.text:00401046                push    0               ; lpModuleName
.text:00401048                call    GetModuleHandleA
.text:0040104D                mov     dword_409117, eax
.text:00401052                push    0               ; фиктивный адрес возврата
.text:00401054                jmp     __startup
```

Помещённый в стек указатель ведёт к структуре, которая, в свою очередь, содержит адрес main. Внутри __startup вызов подготавливается так:

```
.text:00406997                mov     edx, dword_40BBFC
.text:0040699D                push    edx
.text:0040699E                mov     ecx, dword_40BBF8
.text:004069A4                push    ecx
.text:004069A5                mov     eax, dword_40BBF4
.text:004069AA                push    eax
.text:004069AB                call    dword ptr [esi+18h]
.text:004069AE                add     esp, 0Ch
.text:004069B1                push    eax             ; состояние завершения
.text:004069B2                call    _exit
```

Как и раньше, единственный вызов с тремя аргументами является main, а его результат сразу передаётся _exit. В __startup также вызываются GetEnvironmentStrings и GetCommandLine, часто предшествующие главной функции.

Поиск относится не только к C. В программе Visual Basic 6.0 начало выглядит так:

```
.text:004018A4 start:
.text:004018A4             push    offset dword_401994
.text:004018A9             call    ThunRTMain
```

Библиотечная ThunRTMain, подобно libc_start_main, выполняет инициализацию и передаёт управление главной функции. Единственный аргумент указывает на структуру со служебными сведениями и её адресом:

```
.text:00401994 dword_401994 dd 21354256h, 2A1FF0h, 3 dup(0)
.text:004019A8               dd 7Eh, 2 dup(0)
.text:004019B4               dd 0A0000h, 409h, 0
.text:004019C0               dd offset sub_4045D0
.text:004019C4               dd offset dword_401A1C
.text:004019C8               dd 30F012h, 0FFFFFF00h, 8, 2 dup(1), 0E9h,
.text:004019C8               dd 401944h, 4018ECh, 4018B0h, 78h, 7Dh, 82h,
.text:004019C8               dd 83h, 4 dup(0)
```

Единственный элемент, похожий на адрес кода, - sub_4045D0; это и есть главная функция.

В конечном счёте поиск main требует понимания сборки исполняемого файла. При трудностях полезно теми же инструментами собрать простую контрольную программу, поместив в main легко узнаваемую строку. Изучение такого образца раскрывает структуру файлов конкретной цепочки сборки и помогает анализировать более сложные результаты.

Отладочные и выпускные сборки

Проект Visual Studio обычно создаёт отладочный и выпускной варианты. Их различия видны в параметрах сборки. Выпуск обычно оптимизирован,[3] а отладочный - нет. Последний включает дополнительные символы и отладочные версии библиотек времени выполнения.

Символы позволяют отладчику связывать ассемблерные инструкции с исходными строками и восстанавливать имена локальных переменных, обычно теряемые при компиляции. Отладочные библиотеки Microsoft тоже содержат символы, собираются без оптимизации и выполняют дополнительные проверки параметров.

В IDA отладочная сборка Visual Studio заметно отличается от выпускной из-за особых параметров компилятора и компоновщика. Например, основные проверки времени выполнения `/RTCx` добавляют код и способны нарушить сопоставление стартовых сигнатур, поэтому IDA часто не находит `main` автоматически.

Первое заметное отличие - почти все функции достигаются через функции-переходники, содержащие один переход:

```
.text:00411050 sub_411050      proc near
.text:00411050                 jmp     sub_412AE0
.text:00411050 sub_411050      endp
; ...
.text:0041110E start           proc near
.text:0041110E                 jmp     start_0
.text:0041110E start           endp
; ...
.text:00411920 start_0         proc near
.text:00411920                 push    ebp
.text:00411921                 mov     ebp, esp
.text:00411923                 call    sub_411050
.text:00411928                 call    sub_411940
.text:0041192D                 pop     ebp
.text:0041192E                 retn
.text:0041192E start_0         endp
```

Точка входа лишь переходит к настоящей стартовой функции. Та вызывает `sub_411050`, которая снова только переходит к реализации. Обилие таких переходников мешает сигнатурам IDA, но стратегии предыдущего раздела всё равно позволяют найти `main`.

Проверки времени выполнения расширяют и пролог каждой функции:

```
.text:00411500                 push    ebp
.text:00411501                 mov     ebp, esp
.text:00411503                 sub     esp, 0F0h
.text:00411509                 push    ebx
.text:0041150A                 push    esi
.text:0041150B                 push    edi
.text:0041150C                 lea     edi, [ebp+var_F0]
.text:00411512                 mov     ecx, 3Ch
.text:00411517                 mov     eax, 0CCCCCCCCh
.text:0041151C                 rep stosd
.text:0041151E                 mov     [ebp+var_8], 0
.text:00411525                 mov     [ebp+var_14], 1
.text:0041152C                 mov     [ebp+var_20], 2
.text:00411533                 mov     [ebp+var_2C], 3
```

Четырём локальным переменным достаточно 16 байт, но функция выделяет 240 и заполняет их значением `0xCC`. Четыре инструкции от `lea` до `rep stosd` эквивалентны:

```
memset(&var_F0, 0xCC, 240);
```

Байт `0xCC` является кодом x86 `int 3`, программного прерывания отладчика. Заполнение кадра помогает обнаружить ошибочную попытку исполнить стек.

Локальные переменные располагаются с промежутками, заполненными `0xCC`. Выход за границу одной переменной тогда повреждает заметный заполнитель до соседней, что облегчает

обнаружение переполнения. В нормальной работе байты вне объявленных переменных не меняются.

Выпускной вариант того же кода значительно короче:

```
.text:004018D0                push    ebp
.text:004018D1                mov     ebp, esp
.text:004018D3                sub     esp, 10h
.text:004018D6                mov     [ebp+var_4], 0
.text:004018DD                mov     [ebp+var_C], 1
.text:004018E4                mov     [ebp+var_8], 2
.text:004018EB                mov     [ebp+var_10], 3
```

Выделяется ровно 16 байт, переменные соседствуют, а заполнитель `0xCC` отсутствует.

Нестандартные соглашения о вызовах

В главе 6 рассматривались распространённые соглашения C и C++. Единое соглашение необходимо на границе независимо собранных модулей, но внутри одного файла функции могут использовать собственные правила. Это часто встречается в сильно оптимизированном внутреннем коде.

Первые строки такой функции:

```
.text:000158AC sub_158AC      proc near
.text:000158AC arg_0          = dword ptr 4
.text:000158AC                push    [esp+arg_0]
.text:000158B0                mov     edx, [eax+118h]
.text:000158B6                push    eax
.text:000158B7                movzx   ecx, cl
.text:000158BA                mov     cl, [edx+ecx+0A0h]
```

IDA видит один аргумент в стеке. Но `EAX` и `CL` используются до инициализации, значит, их должен задать вызывающий код. На деле функция принимает три аргумента, и при вызове все должны оказаться в правильных местах.

Пользовательское соглашение задаётся типом функции через **Edit > Functions > Set function type** и ключевое слово IDA `__usercall`.

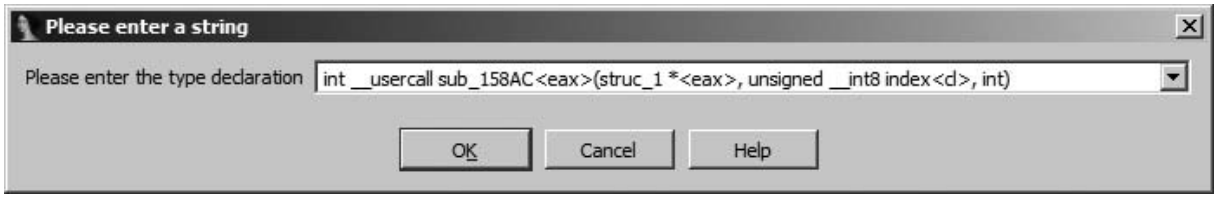


Рисунок 20-1. Обозначение функции как `__usercall`

Объявление для примера:

```
int __usercall sub_158AC<eax>(struc_1 *<eax>, unsigned __int8 index<cl>, int)
```

`__usercall` заменяет стандартное `__cdecl` или `__stdcall`. Регистр результата добавляется к имени функции: `sub_158AC<eax>`. Если результата нет, его можно опустить. Для каждого регистрового параметра регистр дописывается к типу.

После установки типа IDA распространяет сведения на места вызова:

```
.text:00014B9F      lea     eax, [ebp+var_218] ; struc_1 *
.text:00014BA5      mov     cl, 1              ; index
.text:00014BA7      push    edx                ; int
.text:00014BA8      call    sub_158AC
```

Теперь видно, что EAX содержит первый аргумент, CL - второй, а третий помещается в стек.

Даже в одном файле соглашения могут различаться. Другая функция:

```
.text:0001669E sub_1669E       proc near
.text:0001669E arg_0           = byte ptr 4
.text:0001669E                 mov     eax, [esi+18h]
.text:000166A1                 add     eax, 684h
.text:000166A6                 cmp     [esp+arg_0], 0
```

IDA показывает один стековый аргумент, но ESI также используется до инициализации и потому содержит параметр. Выбор регистра способен меняться от функции к функции даже внутри одной программы.

Необходимо всегда выяснять происхождение каждого регистра. Если функция читает его до собственной записи, вероятнее всего, через него передан параметр. Распространённые правила компиляторов перечислены в главе 6.

Итоги

Особенностей компиляторов слишком много для одной главы или книги. Они различаются алгоритмами реализации высокоуровневых конструкций и оптимизациями. Более того, один компилятор способен создать радикально разные файлы из одинакового исходника при разных параметрах.

Умение справляться с такими вариантами в основном приходит с опытом. Поиск помощи по конкретному фрагменту ассемблера тоже труден: запрос сложно сделать достаточно точным. В таких случаях полезнее всего специализированный форум по обратной разработке, где можно показать код и обратиться к опыту исследователей, уже встречавших похожие конструкции.

Сноски

[1] При *N* вариантах требуется не более log2(N) сравнений.
[2] Табличный поиск находит цель за одну операцию, то есть за постоянное время O(1).
[3] Оптимизация удаляет избыточность и выбирает более быстрые либо компактные последовательности. Такой код может быть сложнее анализировать, поэтому во время разработки и отладки её часто отключают.

Глава 21. Анализ запутанного кода

Даже в идеальных условиях понять дизассемблированный листинг трудно. Именно поэтому предыдущие двадцать глав были посвящены качественному разбору средствами IDA Pro.

IDA снизила порог входа в анализ двоичных файлов, а общий уровень обратной разработки заметно вырос. Авторы, не желающие раскрывать свои программы, ответили новыми средствами защиты. Так возникло своеобразное соревнование между исследователями и разработчиками секретного кода. В этой главе рассматривается роль IDA в таком противостоянии, методы защиты и способы их преодоления.

Запутывание намеренно делает объект неясным и трудным для понимания. Противодействие обратной разработке - более широкое понятие, включающее запутывание и другие способы затруднить анализ. Здесь речь идёт о двоичных исполняемых файлах, а не об исходниках или аппаратных кристаллах.

Полезно разделить техники на категории по проявлениям в IDA. Единственной правильной классификации нет: группы пересекаются, а новые методы появляются постоянно.

Противодействие статическому анализу

Такие методы мешают понять программу без её запуска. Они непосредственно нацелены на дизассемблеры вроде IDA и потому особенно важны при статической обратной разработке.

Рассинхронизация дизассемблирования

Один из старых приёмов творчески смешивает инструкции и данные, чтобы дизассемблер потерял правильные начальные адреса команд. В результате листинг становится неверным или вовсе не строится.

Фрагмент средства Shiva разобран IDA так:

```
LOAD:0A04B0D1                 call    near ptr loc_A04B0D6+1
LOAD:0A04B0D6
LOAD:0A04B0D6 loc_A04B0D6:
LOAD:0A04B0D6                 mov     dword ptr [eax-73h], 0FFEB0A40h
LOAD:0A04B0D6 start           endp
LOAD:0A04B0DD loc_A04B0DD:
LOAD:0A04B0DD                 loopne  loc_A04B06F
LOAD:0A04B0DF                 mov     dword ptr [eax+56h], 5CDAB950h
LOAD:0A04B0E6                 iret
LOAD:0A04B0E6 ; ---------------------------------------------------------
LOAD:0A04B0E7                 db 47h
LOAD:0A04B0E8                 db 31h, 0FFh, 66h
LOAD:0A04B0EB ; ---------------------------------------------------------
LOAD:0A04B0EB loc_A04B0EB:
LOAD:0A04B0EB                 mov     edi, 0C7810D98h
```

`call` передаёт управление в середину уже распознанной инструкции. Поскольку вызов считается возвращающим управление, IDA разбирает команду с адреса `0A04B0D6`, но неверно. Настоящая цель `0A04B0D7` не оформляется, потому что её байты уже входят в пятибайтовую команду.

После такого места весь последующий листинг подозрителен. Признаки ошибки - неожиданная привилегированная `iret`, обычно используемая обработчиком прерывания ядра, и бессвязные байты данных. Приём обманывает и рекурсивный спуск, и линейный просмотр почти любого

дизассемблера.

В IDA нужно снять определение с команды, содержащей целевые байты, и создать новую по настоящему адресу. Команда **Edit > Undefine** с клавишей `U`, а затем **Edit > Code** с клавишей `C` на `0A04B0D7` дают:

```
LOAD:0A04B0D1                 call    loc_A04B0D7
LOAD:0A04B0D1 ; ---------------------------------------------------------
LOAD:0A04B0D6                 db 0C7h
LOAD:0A04B0D7 ; ---------------------------------------------------------
LOAD:0A04B0D7 loc_A04B0D7:
LOAD:0A04B0D7                 pop     eax
LOAD:0A04B0D8                 lea     eax, [eax+0Ah]
LOAD:0A04B0DB loc_A04B0DB:
LOAD:0A04B0DB                 jmp     short near ptr loc_A04B0DB+1
LOAD:0A04B0DB start           endp
LOAD:0A04B0DD                 db 0E0h
```

Байт по `0A04B0D6` не исполняется. Команда `pop` снимает фиктивный адрес возврата со стека, после чего выполнение продолжается. Но уже по `0A04B0DB` двухбайтовый переход снова направлен в середину самого себя. Нужно повторно снять определение и создать инструкцию с `0A04B0DC`:

```
LOAD:0A04B0D7                 pop     eax
LOAD:0A04B0D8                 lea     eax, [eax+0Ah]
LOAD:0A04B0D8 ; ---------------------------------------------------------
LOAD:0A04B0DB                 db 0EBh
LOAD:0A04B0DC ; ---------------------------------------------------------
LOAD:0A04B0DC                 jmp     eax
LOAD:0A04B0DC start           endp
```

Теперь обнаружен ещё один переход, но его цель вычисляется во время исполнения и находится в `EAX`. Предыдущий `pop` загрузил туда адрес возврата `0A04B0D6`, а `lea` прибавила 10. Следовательно, переход ведёт на `0A04B0E0`, откуда и следует продолжить разбор.

Другой вариант использует флаги процессора, превращая условный переход в фактически безусловный:

```
.text:00401000                 xor     eax, eax
.text:00401002                 jz      short near ptr loc_401009+1
.text:00401004                 mov     ebx, [eax]
.text:00401006                 mov     [ecx-4], ebx
.text:00401009 loc_401009:
.text:00401009                 call    near ptr 0ADFEFFC6h
.text:0040100E                 ficom   word ptr [eax+59h]
```

`xor` обнуляет `EAX` и устанавливает флаг нуля Z. Автор знает, что следующий `jz` обязательно сработает, поэтому получает безусловный переход. Инструкции между ним и целью никогда не исполняются и только сбивают аналитика. Кроме того, цель снова расположена внутри команды `call`.

После правильного переопределения получается:

```
.text:00401000                 xor     eax, eax
.text:00401002                 jz      short loc_40100A
.text:00401004                 mov     ebx, [eax]
.text:00401006                 mov     [ecx-4], ebx
.text:00401006 ; ---------------------------------------------------------
.text:00401009                 db 0E8h
.text:0040100A ; ---------------------------------------------------------
.text:0040100A loc_40100A:
.text:0040100A                 mov     eax, 0DEADBEEFh
.text:0040100F                 push    eax
.text:00401010                 pop     ecx
```

Открылись настоящая цель и лишний байт, вызвавший рассинхронизацию. Значение флагов можно подготавливать гораздо более длинной цепочкой, и сложность анализа растёт с числом предшествующих операций, способных эти флаги менять.

Динамически вычисляемые целевые адреса

Здесь слово «динамически» не означает противодействие динамическому анализу. Оно лишь указывает, что адрес дальнейшего исполнения вычисляется во время работы. Цель - скрыть настоящий поток управления от статического анализа.

Предыдущий пример сначала поместил адрес возврата в стек, затем извлёк его в регистр, прибавил константу и выполнил косвенный переход. Подобных последовательностей можно придумать бесконечно много.

Завершение стартового кода Shiva использует другой расчёт:

```
LOAD:0A04B3BE                mov     ecx, 7F131760h  ; ecx = 7F131760
LOAD:0A04B3C3                xor     edi, edi        ; edi = 00000000
LOAD:0A04B3C5                mov     di, 1156h       ; edi = 00001156
LOAD:0A04B3C9                add     edi, 133AC000h  ; edi = 133AD156
LOAD:0A04B3CF                xor     ecx, edi        ; ecx = 6C29C636
LOAD:0A04B3D1                sub     ecx, 622545CEh  ; ecx = 0A048068
LOAD:0A04B3D7                mov     edi, ecx        ; edi = 0A048068
LOAD:0A04B3D9                pop     eax
LOAD:0A04B3DA                pop     esi
LOAD:0A04B3DB                pop     ebx
LOAD:0A04B3DC                pop     edx
LOAD:0A04B3DD                pop     ecx
LOAD:0A04B3DE                xchg    edi, [esp]      ; вершина = 0A048068
LOAD:0A04B3E1                retn                    ; возврат на 0A048068
```

Вычисленное значение помещается на вершину стека, поэтому `retn` передаёт управление на `0A048068`. При статическом анализе такую цепочку приходится фактически исполнять вручную.

Более сложные варианты используют несколько потоков или дочерних процессов. Один вычисляет адреса, другой получает их через межпроцессное взаимодействие либо средства синхронизации. Аналитику приходится понимать поведение всех участников и точный обмен сведениями.

Например, один поток ждёт семафор,[1] а другой вычисляет значения или изменяет код, затем сигнализирует о завершении. Только после этого первый продолжает работу с подготовленными данными.

Вредоносные программы Windows часто настраивают обработчик исключений, намеренно вызывают исключение и меняют состояние регистров внутри обработчика. Средство tElock начинает такой приём следующей последовательностью:

```
.shrink:0041D07A        call    $+5
.shrink:0041D07F        pop     ebp
.shrink:0041D080        lea     eax, [ebp+46h]  ; eax = 0041D07F + 46h
.shrink:0041D081        inc     ebp
.shrink:0041D083        push    eax
.shrink:0041D084        xor     eax, eax
.shrink:0041D086        push    dword ptr fs:[eax]
.shrink:0041D089        mov     fs:[eax], esp
```

Продолжение намеренно вызывает два исключения:

```
.shrink:0041D08C        int     3               ; ловушка отладчика
.shrink:0041D08D        nop
.shrink:0041D08E        mov     eax, eax
.shrink:0041D090        stc
.shrink:0041D091        nop
.shrink:0041D092        lea     eax, ds:1234h[ebx*2]
.shrink:0041D099        clc
.shrink:0041D09A        nop
.shrink:0041D09B        shr     ebx, 5
.shrink:0041D09E        cld
.shrink:0041D09F        nop
```

```
.shrink:0041D0A0        rol     eax, 7
.shrink:0041D0A3        nop
.shrink:0041D0A4        nop
.shrink:0041D0A5        xor     ebx, ebx
.shrink:0041D0A7        div     ebx             ; деление на ноль
.shrink:0041D0A9        pop     dword ptr fs:0
```

`call $+5` помещает на стек адрес следующей инструкции `0041D07F`, который сразу извлекается в `EBP`. Затем `EAX` получает `EBP + 46h`, то есть `0041D0C5`, и этот адрес помещается в стек как обработчик исключения. Следующие команды включают его в цепочку, на которую указывает `fs:[0]`.[2]

Команда `int 3` намеренно создаёт программную ловушку, обычно используемую отладчиком как точку останова. Подключённый отладчик получает первое право обработать событие, но программа рассчитывает принять его сама. Поэтому исследователь должен передать исключение приложению; иначе оно может работать неверно или завершиться аварийно.

Не зная обработчика, невозможно понять дальнейший поток. При простом продолжении после `int 3` вскоре возникло бы деление на ноль.

Обработчик начинается по `0041D0C5`:

```
.shrink:0041D0C5 sub_41D0C5      proc near
.shrink:0041D0C5 pEXCEPTION_RECORD = dword ptr 4
.shrink:0041D0C5 arg_4              = dword ptr 8
.shrink:0041D0C5 pCONTEXT           = dword ptr 0Ch
.shrink:0041D0C5        mov     eax, [esp+pEXCEPTION_RECORD]
.shrink:0041D0C9        mov     ecx, [esp+pCONTEXT] ; адрес CONTEXT
.shrink:0041D0CD        inc     [ecx+CONTEXT._Eip]  ; изменение EIP
.shrink:0041D0D3        mov     eax, [eax]          ; тип исключения
.shrink:0041D0D5        cmp     eax, EXCEPTION_INT_DIVIDE_BY_ZERO
.shrink:0041D0DA        jnz     short loc_41D100
.shrink:0041D0DC        inc     [ecx+CONTEXT._Eip]  ; повторное изменение EIP
.shrink:0041D0E2        xor     eax, eax            ; очистка отладочных регистров
.shrink:0041D0E4        and     [ecx+CONTEXT.Dr0], eax
.shrink:0041D0E7        and     [ecx+CONTEXT.Dr1], eax
.shrink:0041D0EA        and     [ecx+CONTEXT.Dr2], eax
.shrink:0041D0ED        and     [ecx+CONTEXT.Dr3], eax
.shrink:0041D0F0        and     [ecx+CONTEXT.Dr6], 0FFFF0FF0h
.shrink:0041D0F7        and     [ecx+CONTEXT.Dr7], 0DC00h
.shrink:0041D0FE        jmp     short locret_41D160
```

Третий аргумент указывает на структуру Windows `CONTEXT` из `winnt.h`, содержащую значения всех регистров в момент исключения. Обработчик может их изучить и изменить. Если проблема исправлена, операционная система восстанавливает регистры из изменённой структуры и продолжает поток, словно исключения не было.

Здесь обработчик увеличивает сохранённый `EIP`, пропуская вызвавшую событие инструкцию, затем читает тип из `EXCEPTION_RECORD`. Для деления на ноль он ещё раз меняет `EIP` и очищает аппаратные отладочные регистры x86.[3]

tElock стирает значения после предыдущего этапа, на котором устанавливались четыре аппаратные точки останова в дополнение к `int 3`. Это не только скрывает поток управления, но и нарушает работу OllyDbg либо встроенного отладчика IDA.

Запутывание кодов операций

Предыдущие методы затрудняли понимание потока, но не скрывали правильный вид самих инструкций. Рассинхронизацию можно было победить переоформлением листинга.

Более действенный способ - кодировать или шифровать инструкции при создании файла. В таком виде процессор не может их исполнять, поэтому перед выборкой они обязательно

восстанавливаются. Часть программы остаётся открытой как стартовая процедура, которая распутывает остальные код и данные.

Общая схема процесса:

```
flowchart LR
    A["Исходный файл<br/>заголовок, код и данные"] --> B["Средство запутывания"]
    B --> C["Защищённый файл<br/>изменённая точка входа"]
    C --> D["Распутывающая заглушка"]
    D --> E["Восстановленные код и данные"]
    E --> F["Переход к исходной точке входа"]
```

Рисунок 21-1. Общий процесс запутывания

Обычная программа создаётся стандартными языками и средствами, затем готовый исполняемый файл передаётся защитной утилите. Та преобразует код и данные, добавляет распутывающую заглушку и меняет заголовок так, чтобы новая точка входа вела на неё. После восстановления управление обычно передаётся исходной точке входа, и программа продолжает работу как прежде.

Конкретные реализации сильно различаются. Защитные программы предлагают сжатие, противодействие дизассемблированию и отладке. Для PE существуют UPX, ASPack, ASProtect и tElock; для ELF - Burneye и Shiva. Более развитые средства, например WinLicense, встраивают защиту во весь процесс сборки от исходников до последующей обработки.

Современное направление помещает исходную программу внутрь исполнительной среды виртуальной машины. Настоящий машинный код иногда вообще не исполняется напрямую, а интерпретируется как байт-код. Сложные средства создают новый вариант виртуальной машины при каждом запуске, затрудняя универсальное распутывание. Пример - VMProtect, применявшийся для троянской программы Clampi.

Для многих защит появились средства восстановления исходного или близкого к нему файла. QuickUnpack и похожие автоматические распаковщики действуют как отладчик: дают программе выполнить стадию самораспаковки, а затем сохраняют образ процесса из памяти.

Такие средства действительно запускают потенциально вредоносный код, надеясь перехватить его после распаковки, но до опасных действий. Поэтому их необходимо использовать только в изолированной среде.

Чисто статический анализ защищённого файла намного сложнее. Без исполнения заглушки требуется самостоятельно распаковать или расшифровать данные до дизассемблирования. На рисунке 21-2 показана полоса навигации файла после UPX. Единственная область, распознанная как код, - тонкая полоса распаковывающей заглушки.



Рисунок 21-2. Полоса навигации IDA для файла, упакованного UPX

Слева находится пустая область, а между ней и заглушкой - выглядящие случайными сжатые данные. Заглушка разворачивает их в пустой диапазон и передаёт управление восстановленному коду.

Необычный вид полосы уже указывает на защиту. Другие признаки:

• в полосе навигации отмечено очень мало кода;
• в окне **Functions** почти нет функций, иногда только `start`;
• в окне **Imports** мало импортов;

• в окне **Strings** мало читаемых строк, часто только имена немногочисленных библиотек и импортов;
• один или несколько разделов одновременно доступны для записи и исполнения;
• применяются нестандартные имена разделов вроде `UPX0` или `.shrink`.

> **Изолированная среда**
>
> Её задача - позволить запустить программу и наблюдать поведение, не давая ему повредить важные части исследовательской системы. Часто применяется виртуализация, например VMware, либо отдельный компьютер, который после опыта возвращается к известному чистому состоянию.
>
> Такая среда обычно насыщена средствами наблюдения. Они собирают сведения о файловой системе, реестре Windows и сетевой активности программы.

Сведения полосы навигации следует сопоставлять со свойствами сегментов:

```
Имя   Начало   Конец    R W X D L Выравн. База Тип    Класс
UPX0   00401000 00407000 R W X . L para    0001 public CODE
UPX1   00407000 00409000 R W X . L para    0002 public CODE
UPX2   00409000 0040908C R W . . L para    0003 public DATA
.idata 0040908C 004090C0 R W . . L para    0003 public XTRN
UPX2   004090C0 0040A000 R W . . L para    0003 public DATA
```

Весь диапазон `UPX0` и `UPX1`, от `00401000` до `00409000`, помечен исполняемым. Поэтому он должен был бы целиком отображаться как код. Но этого нет, а `UPX0` при просмотре пуст. Заголовок подтверждает подозрение:

```
UPX0:00401000 ; Раздел 1, виртуальный адрес: 00001000
UPX0:00401000 ; Виртуальный размер:           00006000 (24576)
UPX0:00401000 ; Размер раздела в файле:       00000000 (0)
UPX0:00401000 ; Смещение исходных данных:     00000200
UPX0:00401000 ; Флаги E0000080: BSS, исполнение, чтение, запись
```

Статическая распаковка средствами IDA будет рассмотрена далее.

Сокрытие импортируемых функций

Чтобы не раскрывать возможные действия, защищённая программа скрывает используемые библиотеки и функции. Тогда `dumpbin`, `ldd` и `objdump` не показывают реальные зависимости.

У tElock окно импорта содержит всего две записи:

```
Адрес     Номер  Имя               Библиотека
0041EC2E         GetModuleHandleA  kernel32
0041EC36         MessageBoxA       user32
```

По такому списку почти невозможно судить о поведении программы.

Чтобы выполнить полезную работу, программа сама загружает остальные библиотеки и находит нужные функции. Обычно это делает распутывающая заглушка до передачи управления основному коду, в итоге вручную восстанавливая таблицу импорта так, как это сделал бы загрузчик операционной системы.

В Windows простой путь - вызвать `LoadLibrary` по имени библиотеки, затем искать адреса через `GetProcAddress`. Но сначала нужно каким-либо способом получить адреса самих этих функций. В tElock они не импортированы, тогда как UPX оставляет их в списке:

```
Адрес     Номер  Имя                  Библиотека
0040908C         LoadLibraryA         KERNEL32
00409090         GetProcAddress       KERNEL32
00409094         ExitProcess          KERNEL32
```

```
0040909C        RegCloseKey             ADVAPI32
004090A4        atoi                    CRTDLL
004090AC        ExitWindowsEx           USER32
004090B4        InternetOpenA           WININET
004090BC        recv                    wsock32
```

Код UPX, восстанавливающий таблицу:

```
UPX1:0040886C loc_40886C:
UPX1:0040886C        mov     eax, [edi]
UPX1:0040886E        or      eax, eax
UPX1:00408870        jz      short loc_4088AE
UPX1:00408872        mov     ebx, [edi+4]
UPX1:00408875        lea     eax, [eax+esi+8000h]
UPX1:0040887C        add     ebx, esi
UPX1:0040887E        push    eax
UPX1:0040887F        add     edi, 8
UPX1:00408882        call    dword ptr [esi+808Ch] ; LoadLibraryA
UPX1:00408888        xchg    eax, ebp
UPX1:00408889 loc_408889:
UPX1:00408889        mov     al, [edi]
UPX1:0040888B        inc     edi
UPX1:0040888C        or      al, al
UPX1:0040888E        jz      short loc_40886C
UPX1:00408890        mov     ecx, edi
UPX1:00408892        push    edi
UPX1:00408893        dec     eax
UPX1:00408894        repne scasb
UPX1:00408896        push    ebp
UPX1:00408897        call    dword ptr [esi+8090h] ; GetProcAddress
UPX1:0040889D        or      eax, eax
UPX1:0040889F        jz      short loc_4088A8
UPX1:004088A1        mov     [ebx], eax            ; запись в таблицу импорта
UPX1:004088A3        add     ebx, 4
UPX1:004088A6        jmp     short loc_408889
```

Листинг 21-1. Восстановление таблицы импорта в UPX

Внешний цикл вызывает `LoadLibraryA`, внутренний - `GetProcAddress`, а каждый найденный адрес сохраняется в таблице. Это последняя часть заглушки: обе функции получают строки имён, которые сами хранятся в сжатой области и до распаковки не видны средствам поиска строк.

У tElock импортированы лишь две другие функции. Однако `kernel32.dll` присутствует в памяти каждого процесса Windows. Найдя её, программа может разобрать таблицу экспорта и получить любые функции, включая `LoadLibraryA` и `GetProcAddress`, а затем загрузить всё остальное.

Такой самостоятельный поиск скрывает обычную загрузку и связывание: без внимательного прослеживания инструкций легко пропустить библиотеку или разрешённый адрес. tElock ищет `LoadLibraryA` так:

```
.shrink:0041D1E4                cmp     dword ptr [eax], 64616F4Ch
.shrink:0041D1EA                jnz     short loc_41D226
.shrink:0041D1EC                cmp     dword ptr [eax+4], 7262694Ch
.shrink:0041D1F3                jnz     short loc_41D226
.shrink:0041D1F5                cmp     dword ptr [eax+8], 41797261h
.shrink:0041D1FC                jnz     short loc_41D226
```

Если представить константы как знаки, смысл становится заметнее:

```
.shrink:0041D1E4                cmp     dword ptr [eax], 'daoL'
.shrink:0041D1EA                jnz     short loc_41D226
.shrink:0041D1EC                cmp     dword ptr [eax+4], 'rbiL'
.shrink:0041D1F3                jnz     short loc_41D226
.shrink:0041D1F5                cmp     dword ptr [eax+8], 'Ayra'
.shrink:0041D1FC                jnz     short loc_41D226
```

Каждая константа содержит четыре знака ASCII. С учётом порядка байтов x86 их нужно читать наоборот; вместе они образуют `LoadLibraryA`. Если все три сравнения успешны, найдена запись

экспорта и через несколько команд будет получен адрес.

Четырёхбайтовые константы внутри инструкций не похожи на завершённую нулём строку, поэтому IDA не включает их в список строк. Это делает поиск устойчивым к простому строковому анализу.

UPX и tElock всё же содержат знаки имён. Более скрытный способ заранее вычисляет уникальную свёртку каждого требуемого имени. Затем программа перебирает экспортируемые имена библиотеки, вычисляет свёртку каждого и сравнивает с искомой. При совпадении адрес берётся из таблицы экспорта.

Для статического анализа нужно понять алгоритм, применить его ко всем именам библиотеки и построить таблицу. Её часть для `kernel32.dll` могла бы выглядеть так:

```
GetProcAddress          : 8A0FB5E2
GetProcessAffinityMask : B9756EFE
GetProcessHandleCount  : B50EB87C
GetProcessHeap         : C246DA44
GetProcessHeaps        : A18AAB23
GetProcessId           : BE05ED07
```

Значения зависят от конкретного алгоритма и могут различаться между файлами. В этой таблице `8A0FB5E2` однозначно указывает на `GetProcAddress`. Такой подход изначально применялся в полезной нагрузке для уязвимостей Windows, а затем перешёл в средства запутывания, например WinLicense.

Иногда IDA сама предупреждает о повреждённой или необычной таблице импорта.

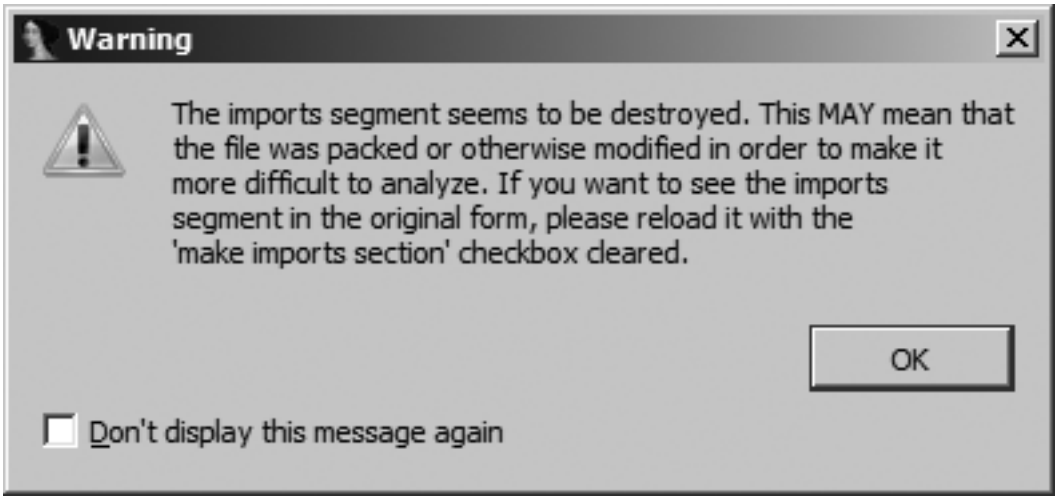


Рисунок 21-3. Предупреждение об испорченном сегменте импорта

Это один из самых ранних признаков защиты и сигнал о предстоящем сложном анализе.

Целенаправленные атаки на средства анализа

Средства обратной разработки являются специализированными синтаксическими анализаторами входных данных и подвержены обычным программным уязвимостям. Ошибка обработки данных иногда допускает эксплуатацию.

Специально подготовленный файл может оставаться достаточно корректным для запуска, но одновременно использовать уязвимость IDA или другого анализатора. Минимальный результат - постоянное аварийное завершение до создания базы; худший - захват компьютера и сети исследователя.

Начальный анализ подозрительных файлов разумно выполнять в изолированной среде. Например, отдельная копия IDA создаёт первичную базу, после чего другим исследователям передаётся уже она, а не исходный опасный файл.

Противодействие динамическому анализу

Статические защитные приёмы не должны мешать самой программе выполняться, иначе она теряет смысл. Они затрудняют понимание, но не запуск.

Динамический анализ наблюдает программу в движении, тогда как статический изучает её в покое. Далее кратко перечислены распространённые способы мешать наблюдению во время исполнения. Большинство почти не влияет на статический разбор; взаимодействие со встроенным отладчиком IDA подробно рассматривается с главы 24.

Обнаружение виртуализации

Изолированную среду часто строят на VMware или другой виртуализации. Контрольные точки и откат позволяют быстро возвращаться к чистому состоянию. Недостаток, особенно на 32-разрядной x86, - программа довольно легко замечает виртуальную машину. Считая виртуализацию признаком наблюдения, вредоносный код может сразу завершиться.

Применялись следующие методы.

Поиск программ виртуализации. Пользователь часто ставит вспомогательные средства связи и ускорения, например VMware Tools. Они оставляют легко читаемые следы, включая записи реестра Windows. Для запуска вредоносной программы такие дополнения обычно не нужны, поэтому в исследовательской машине их лучше не устанавливать.

Поиск виртуального оборудования. Гипервизор предоставляет особые устройства. Например, VMware имеет собственные организационно уникальные префиксы сетевых адресов. Такой префикс MAC убедительно указывает на виртуализацию, хотя адрес виртуального адаптера обычно можно изменить в настройках хозяина.

Особое поведение виртуальной машины. Некоторые платформы имеют скрытые каналы связи между гостем и хозяином. Для VMware достаточно такой последовательности:

```
mov  eax, 0x564D5868    ; 'VMXh'
mov  ecx, 10
xor  ebx, ebx
mov  dx,  0x5658        ; 'VX'
in   eax, dx
```

Внутри виртуальной машины `EBX` получит `0x564D5868`. На настоящем оборудовании возникнет исключение либо регистр не изменится - это зависит от операционной системы. Пользовательским программам x86 обычно запрещена `in`, но VMware использует эту последовательность для проверки канала связи гостя с хозяином. Через тот же канал VMware Tools, например, передаёт содержимое буфера обмена.

Поиск особенностей поведения процессора. Совершенная виртуализация труднодостижима: программа в идеале не должна замечать разницы, но на практике она есть. Метод redpill Йоанны Рутковской основан на различии поведения инструкции x86 `sidt` на настоящем оборудовании и в виртуальной машине.

Обнаружение средств наблюдения

Перед запуском в изолированной среде устанавливают средства сбора поведения. Распространённые примеры - Process Monitor группы Sysinternals и Wireshark. Первый наблюдает обращения процессов Windows к реестру и файловой системе, второй перехватывает и разбирает сетевые пакеты.

Вредоносная программа может искать такие средства. Простейшие варианты просматривают список процессов и заголовки окон по известным строкам. Более глубокие проверки выявляют характерные элементы графического интерфейса.

Например, WinLicense определяет старую программу Filemon по зарегистрированному классу окна:

```
if (FindWindow("FilemonClass", NULL)) {
    // Завершиться, поскольку работает Filemon.
}
```

`FindWindow` ищет главное окно не по заголовку, а по имени класса `FilemonClass`. При успехе программа считает Filemon запущенной и прекращает работу.

Обнаружение отладчиков

Отладчик не просто наблюдает, а полностью управляет исполнением. Защищённую программу часто запускают лишь до завершения распаковки или расшифрования, затем извлекают восстановленный образ из памяти и продолжают обычный статический анализ.

Авторы защиты стараются этому помешать. Обнаружив отладчик, программа часто завершает работу до раскрытия поведения.

Методы варьируются от системного вызова Windows `IsDebuggerPresent` до низкоуровневого поиска следов в памяти и процессоре, например установленного флага пошагового исполнения. Иногда определяется конкретный отладчик: ядровой SoftIce выдаёт устройство `\\.\NTICE`, используемое для связи.

Если известно, что искать, такие проверки не особенно хитры и видны при статическом анализе, пока одновременно не применено его запутывание.

Предотвращение отладки

Даже незаметному отладчику можно мешать ложными точками останова, очисткой аппаратных точек, затруднением выбора адресов или запретом присоединения к процессу.

Один способ - намеренно создавать исключения. Отладчик перехватывает событие, а исследователь должен понять его причину и решить, передавать ли программе. Для `int 3` трудно отличить ловушку самого кода от настоящей программной точки останова. Именно такая путаница и нужна автору защиты. Внимательный разбор обычно восстанавливает поток, но требует больше труда.

Кодирование одновременно мешает статическому анализу и установке точек. До расшифрования невозможно разместить программную точку останова: изменение зашифрованных байтов нарушит восстановление и приведёт к аварии при достижении адреса.

Некоторые заглушки вычисляют контрольную сумму областей памяти. Вставленная программная точка меняет байт, сумма не совпадает, и программа прекращает работу.

Средство Shiva для ELF применяет взаимный `ptrace`, не позволяя отладчику присоединиться.

> **Трассировка процессов**
>
> Интерфейс `ptrace` во многих системах Unix позволяет одному процессу наблюдать и управлять другим. Его использует, например, отладчик GNU GDB. Родитель сначала присоединяется к дочернему процессу. При получении дочерним сигнала он останавливается, родитель узнаёт об этом через POSIX `wait`, изучает или меняет состояние и разрешает продолжить.
>
> Одновременно к дочернему процессу может быть присоединён только один наблюдатель, пока прежний не отсоединится.

Shiva в начале создаёт копию через `fork`. Исходный процесс сразу присоединяется через `ptrace` к дочернему, а дочерний - к родителю. Неудача любого присоединения означает присутствие другого отладчика, и оба завершаются. При успехе сторонний отладчик уже не сможет занять место наблюдателя.

Кроме того, два процесса могут менять состояние друг друга, что ещё сильнее затрудняет статическое восстановление точного потока управления.

Статическое распутывание двоичных файлов в IDA

Универсального способа победить все защиты нет. Выбор зависит от навыков и доступных средств. Пользователю отладчика нужны приёмы обхода обнаружения и запрета присоединения. Пользователю дизассемблера - способы получить верный листинг и имитировать самомодифицирующийся код.

Далее рассматриваются два метода обработки самомодифицирующегося кода без исполнения файла. Статика может быть единственным вариантом, если опасный код нельзя запускать или отсутствует нужное оборудование и изолированная среда.

Распутывание сценарием

IDA разбирает программы множества процессоров, поэтому целевая платформа часто не совпадает с машиной исследователя. Например, версия IDA для Windows может изучать Linux x86, MIPS или ARM. Подходящего отладчика тогда может не быть.

Если части программы закодированы, остаётся написать сценарий IDA, имитирующий распутывающий этап и обновляющий инструкции с данными. Задача выглядит большой, но защитная заглушка часто использует лишь малую долю набора команд, поэтому изучать всю архитектуру не требуется.

Применим алгоритм из главы 15 к файлу ELF, зашифрованному Burneye. Исполнение начинается так:

```
LOAD:05371035 start           proc near
LOAD:05371035                 push    off_5371008
LOAD:0537103B                 pushf
LOAD:0537103C                 pusha
LOAD:0537103D                 mov     ecx, dword_5371000
LOAD:05371043                 jmp     loc_5371082
; ...
LOAD:05371082 loc_5371082:
LOAD:05371082                 call    sub_5371048
LOAD:05371087                 sal     byte ptr [ebx-2Bh], 1
LOAD:0537108A                 pushf
```

```
LOAD:0537108B                 xchg    al, [edx-11h]
LOAD:0537108E                 pop     ss
LOAD:0537108F                 xchg    eax, esp
LOAD:05371090                 cwde
LOAD:05371091                 aad     8Eh
LOAD:05371093                 push    ecx
LOAD:05371094                 out     dx, eax
LOAD:05371095                 add     [edx-57E411A0h], bh
LOAD:0537109B                 push    ss
LOAD:0537109C                 rcr     dword ptr [esi+0Ch], cl
LOAD:0537109F                 push    cs
LOAD:053710A0                 sub     al, 70h
LOAD:053710A2                 cmp     ch, [eax+6Eh]
LOAD:053710A5                 cmp     dword ptr ds:0CBD35372h, 9C38A8BCh
LOAD:053710AF                 and     al, 0F4h
LOAD:053710B1                 db      67h
```

Листинг 21-2. Стартовая последовательность Burneye и запутанный код

Сначала в стек помещается содержимое `05371008h`, затем флаги и все регистры. Пока назначение неясно, просто запомним это. `ECX` загружается из `5371000h`, поэтому в IDC нужна переменная:

```
auto ecx;
ecx = Dword(0x5371000); // команда 0537103D
```

После безусловного перехода вызывается `sub_5371048`, что помещает в стек адрес возврата `05371087h`. Следующие команды становятся всё менее правдоподобными: `out` необычна для пользовательского кода, а по `053710B1h` IDA вообще не находит инструкцию. Вместе с двумя функциями во всём окне это явные признаки защиты.

Продолжим анализ внутри `sub_5371048`:

```
LOAD:05371048 sub_5371048     proc near
LOAD:05371048                 pop     esi
LOAD:05371049                 mov     edi, esi
LOAD:0537104B                 mov     ebx, dword_5371004
LOAD:05371051                 or      ebx, ebx
LOAD:05371053                 jz      loc_537107F
LOAD:05371059                 xor     edx, edx
LOAD:0537105B loc_537105B:
LOAD:0537105B                 mov     eax, 8
LOAD:05371060 loc_5371060:
LOAD:05371060                 shrd    edx, ebx, 1
LOAD:05371064                 shr     ebx, 1
LOAD:05371066                 jnb     loc_5371072
LOAD:0537106C                 xor     ebx, 0C0000057h
LOAD:05371072 loc_5371072:
LOAD:05371072                 dec     eax
LOAD:05371073                 jnz     short loc_5371060
LOAD:05371075                 shr     edx, 18h
LOAD:05371078                 lodsb
```

Продолжение и завершение функции:

```
LOAD:05371079                 xor     al, dl
LOAD:0537107B                 stosb
LOAD:0537107C                 dec     ecx
LOAD:0537107D                 jnz     short loc_537105B
LOAD:0537107F loc_537107F:
LOAD:0537107F                 popa
LOAD:05371080                 popf
LOAD:05371081                 retn
```

Листинг 21-3. Основная функция расшифрования Burneye

Это необычная функция: первой командой она снимает собственный адрес возврата `05371087h` в `ESI`. После инициализации `EDI`, `EBX` и `EDX` сценарий растёт так:

```
auto ecx, esi, edi, ebx, edx;
ecx = Dword(0x5371000); // 0537103D
```

```
esi = 0x05371087;       // 05371048
edi = esi;              // 05371049
ebx = Dword(0x5371004); // 0537104B
edx = 0;                // 05371059
```

Далее EBX проверяется перед внешним и внутренним циклами. Полный сценарий:

```
auto ecx, esi, edi, ebx, edx, eax, cf;
ecx = Dword(0x5371000); // 0537103D
esi = 0x05371087;       // 05371048
edi = esi;              // 05371049
ebx = Dword(0x5371004); // 0537104B

if (ebx != 0) {         // 05371051 и 05371053
    edx = 0;            // 05371059
    do {
        eax = 8;        // 0537105B
        do {
            // В IDC нет аналога shrd, поэтому действие составляется вручную.
            edx = (edx >> 1) & 0x7FFFFFFF;
            cf = ebx & 1; // младший бит EBX и флаг переноса
            if (cf == 1) {
                edx = edx | 0x80000000;
            }
            ebx = (ebx >> 1) & 0x7FFFFFFF;
            if (cf == 1) {
                ebx = ebx ^ 0xC0000057;
            }
            eax--;
        } while (eax != 0);

        edx = (edx >> 24) & 0xFF;
        eax = Byte(esi++);
        eax = eax ^ edx;
        PatchByte(edi++, eax);
        ecx--;
    } while (ecx != 0);
}
```

У сдвигов есть тонкость. Оператор >> в IDC выполняет знаковый сдвиг, повторяя старший бит, а shr и shrd x86 - беззнаковый. Поэтому появившиеся слева биты очищаются масками 0x7FFFFFFF и 0xFF.

Инструкции lodsb и stosb работают только с младшими восемью битами EAX, то есть AL, сохраняя остальные 24. Полная имитация lodsb выглядела бы так:

```
eax = (eax & 0xFFFFFF00) | (Byte(esi++) & 0xFF);
```

Но в начале внешнего цикла весь EAX устанавливается в 8, то есть старшие биты уже нулевые. Поэтому упрощённого присваивания достаточно. PatchByte и так читает лишь младшие восемь бит.

После выполнения сценария база отражает изменения, которые в настоящей программе возникли бы только при запуске Linux. В окне **Strings** должно появиться больше читаемых строк. Чтобы IDA повторно просканировала базу, окно нужно закрыть и открыть либо выбрать **Setup** в контекстном меню и нажать **OK**.

Остаётся определить адрес возврата и заново оформить восстановленные байты как код или данные. Конец функции:

```
LOAD:0537107F                popa
LOAD:05371080                popf
LOAD:05371081                retn
```

В начале собственный адрес возврата был снят. Остальные значения подготовила вызывающая процедура:

```
LOAD:05371035 start          proc near
```

```
LOAD:05371035                 push    off_5371008
LOAD:0537103B                 pushf
LOAD:0537103C                 pusha
```

`popa` и `popf` восстанавливают регистры и флаги, поэтому в стеке остаётся только `off_5371008`. Именно туда вернётся расшифровщик, и оттуда следует продолжить анализ.

У Burneye начальный алгоритм сравнительно прост. Заглушки ASPack и tElock потребовали бы гораздо больше кода IDC.

Преимущества сценария:

• опасный файл ни разу не исполняется;
• можно получить рабочий результат, не понимая высокоуровневый алгоритм целиком.

Второй пункт кажется странным, но достаточно точно воспроизвести каждую инструкцию и их порядок. Сценарий повторит общее поведение, даже если математический смысл всей схемы неясен.

Недостаток - хрупкость. Новая версия упаковщика или другие параметры командной строки могут изменить алгоритм и потребовать доработки. Даже универсальный сценарий для UPX приходится постоянно приспосабливать.

Единого всеобъемлющего сценария не существует. Для каждого нового упаковщика нужен свой, а небольшое изменение ломает старый вариант, как это бывает у сигнатурных систем обнаружения вторжений и антивирусов.

Распутывание с помощью эмуляции

Сценарии фактически эмулируют набор команд процессора. Настоящий эмулятор мог бы взять эту работу на себя и сильно сократить время. Он занимает промежуточное место между сценарием и отладчиком: эффективнее первого и гибче второго. Например, MIPS можно исполнять на x86, а инструкции Linux ELF - внутри Windows.

Минимальному эмулятору нужны поток байтов команд, память для стека и состояние регистров. Более развитые варианты предоставляют устройства и службы операционной системы.

В рассматриваемой версии IDA нет встроенной эмуляции, но архитектура расширений позволяет её добавить. Один подход считает базу IDA виртуальной памятью с уже отображённым загрузчиком файлом. Модуль хранит регистры и реализует стек, например отдельным сегментом базы.

Эмулятор читает байты по текущему указателю команд, декодирует их и обновляет состояние: регистры, стек, данные или код базы. Управление похоже на отладчик - пошаговое исполнение, просмотр памяти, изменение регистров и точки останова. Память видна в обычных окнах дизассемблирования и шестнадцатеричного представления, а регистрам требуется отдельное окно.

Для распутывания эмуляция начинается в точке входа и проходит заглушку. Поскольку база служит памятью, самомодификация немедленно изменяет её. По завершении база становится восстановленной версией программы, как при отладке, но вредоносный код никогда реально не исполняется.

Модуль ida-x86emu, или x86emu, эмулирует набор команд x86. Он открыт, совместим с SDK начиная с 4.9 и поставляется в исходном и двоичном виде. Есть варианты для родного интерфейса Windows и Qt; сборка поддерживает MinGW либо Visual Studio 2008. Qt-вариант работает также в Linux и macOS.

Кроме подходящего SDK зависимостей нет. Файл `x86emu.plw` или `x86emu_qt.plw` копируется в `<IDADIR>/plugins`. Настройка не требуется, по умолчанию запуск выполняется `Alt-F8`. Модуль доступен только для x86, но поддерживает любой формат файла, включая PE, ELF и Mach-O.

Свойство	Значение
Имя	ida-x86emu
Автор	Крис Игл
Распространение	Исходники для SDK 6.1 и двоичные файлы для IDA от 5.0, включая бесплатную версию; исходники совместимы начиная с SDK 4.9
Цена	Бесплатно
Назначение	Встроенный эмулятор инструкций x86 для IDA

Инициализация x86emu

При запуске появляется окно управления с регистрами и кнопками пошагового исполнения и изменения данных.

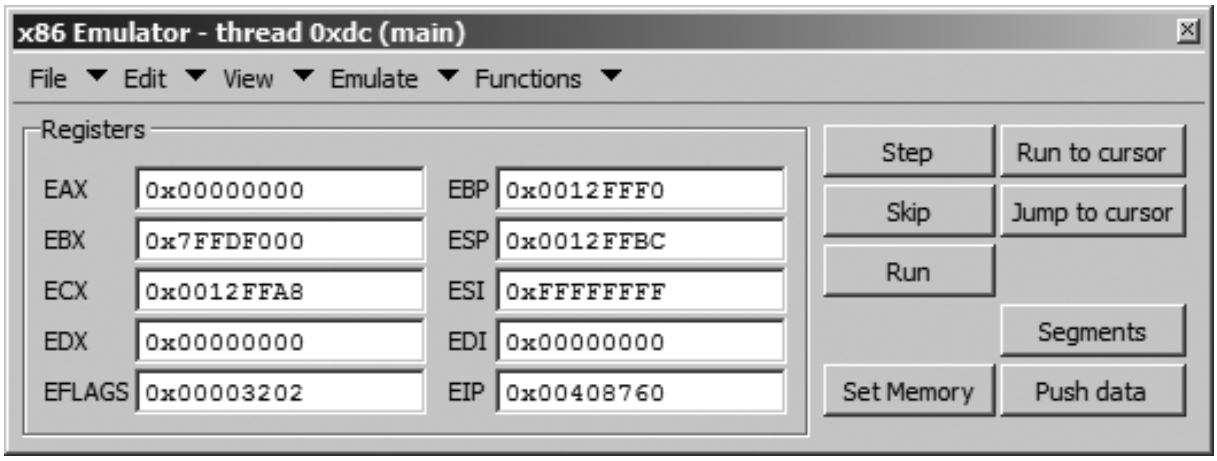


Рисунок 21-4. Окно управления эмулятором x86emu

Для всех форматов модуль создаёт сегменты `.stack` и `.heap`, обеспечивая память времени выполнения. При первом запуске текущая позиция курсора становится начальным `EIP`.

Для PE Windows дополнительно выполняются действия:

1. Создаётся `.headers`; исходный файл перечитывается, а заголовки MS-DOS и PE загружаются в базу.
2. Выделяется память для TEB и PEB, структуры заполняются правдоподобными значениями, чтобы программа считала себя работающей в Windows.
3. Сегментные регистры получают разумные значения, создаётся фиктивная таблица дескрипторов прерываний для минимальной обработки исключений.
4. Ищутся библиотеки DLL из каталога импорта. Для найденных создаются сегменты с заголовками и таблицами экспорта. Таблица импорта программы заполняется полученными адресами, но код DLL в базу не загружается.

Состояние регистров сохраняется в сетевом узле при сохранении или закрытии базы. Стек и куча уже находятся в сегментах базы. При следующем запуске всё состояние восстанавливается.

Основная работа с x86emu

Окно управления напоминает простой отладчик. Значение регистра меняется вводом нового числа в его поле.

Кнопка **Step** эмулирует одну инструкцию: читает байты по `EIP`, выполняет действие и обновляет регистры. Перед каждым шагом байты по `EIP` принудительно оформляются как код, что помогает побеждать рассинхронизацию. Окно дизассемблирования переходит к новой инструкции.

Кнопка **Run To Cursor** исполняет последовательность от текущего `EIP` до точки останова либо адреса курсора. Поддерживаются точки встроенного отладчика IDA, добавляемые командой **Add breakpoint**, и собственные через **Emulate > Set Breakpoint**.

> **Точки останова x86emu**
>
> Эмулятор не использует аппаратные отладочные регистры и не вставляет `int 3`. Он хранит внутренний список и перед каждой командой сравнивает с ним `EIP`. Это не медленнее самой эмуляции, зато программа не может обнаружить или изменить такие точки.

Во время длинного прохода листинг не переоформляется на каждом шаге, только в начале и конце. Иначе работа была бы неприемлемо медленной.

Команду до курсора нужно применять осторожно: управление IDA не вернётся, пока `EIP` не достигнет цели или точки останова. Если этого не случится, придётся принудительно завершить IDA с риском потерять работу.

Кнопка **Skip** сдвигает указатель на одну инструкцию без её выполнения. Так можно обойти условный переход независимо от флагов или пропустить вызов импортированной функции, код которой недоступен.

После пропущенного вызова состояние нужно исправить вручную: например, задать результат в `EAX` или заполнить переданный буфер. Для соглашения `stdcall` следует также увеличить `ESP` на число байтов аргументов, которые функция сняла бы при возврате.

Jump To Cursor записывает адрес курсора в `EIP`. Это позволяет пропустить целый блок, принудительно последовать по условному переходу или сразу исследовать отдельную функцию. Но пропуск `push` или изменения указателя стека нарушит его устройство и может вызвать неожиданное поведение.

Run похожа на запуск до курсора, но продолжается до точки останова и потому опаснее. Её следует применять только при уверенности, что одна из точек будет достигнута.

Кнопка **Segments** открывает настройку сегментных регистров и базовых адресов.

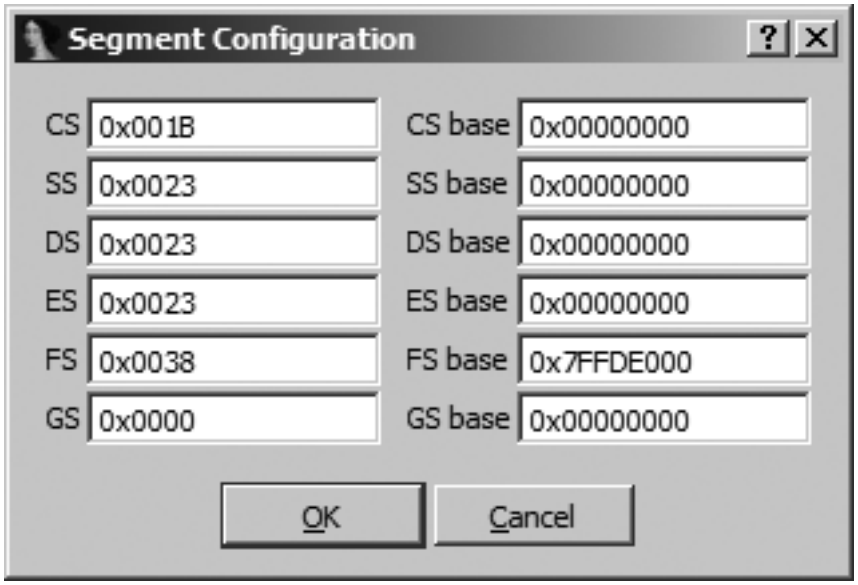


Рисунок 21-5. Настройка сегментных регистров x86emu

Расчёт адресов учитывает заданные базы, но полная таблица глобальных дескрипторов GDT не эмулируется.

Кнопка **Set Memory** открывает простое изменение памяти поверх функций SDK семейства `PatchXXX`.

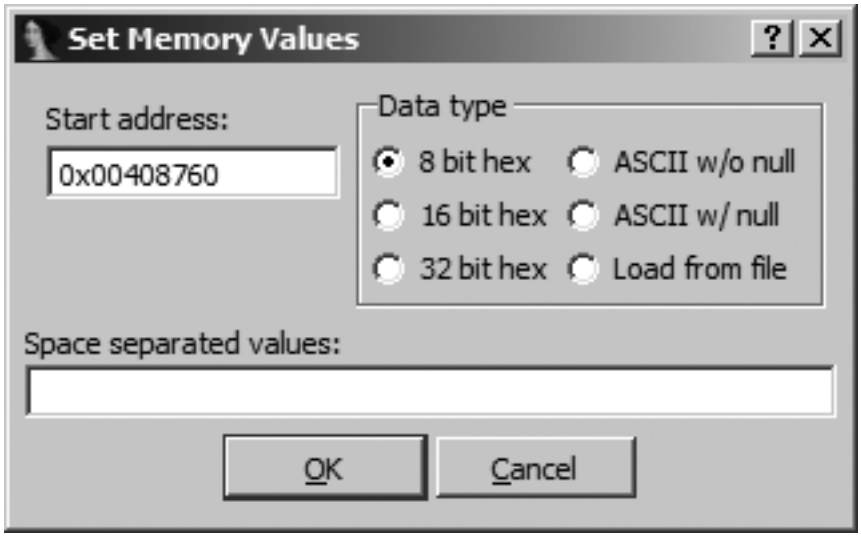


Рисунок 21-6. Изменение памяти в x86emu

Переключатели выбирают тип, значение вводится в поле. При **Load from file** показывается обычное окно выбора, а содержимое файла переносится в базу с указанного адреса.

Push Data помещает значения на вершину эмулируемого стека.

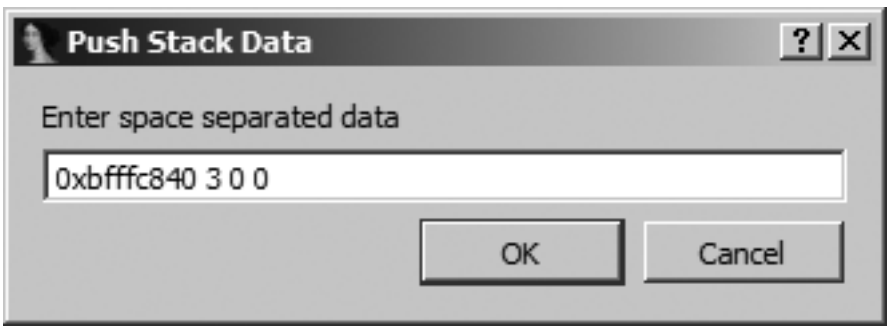


Рисунок 21-7. Данные стека x86emu

Принимаются только числа. Они помещаются как четырёхбайтовые значения справа налево, словно параметры функции, а ESP изменяется соответствующим образом. Это удобно перед прямым переходом в исследуемую функцию, для которой не хочется искать настоящий путь вызова.

Распутывание с x86emu

Вернёмся к Burneye, для которого ранее был написан целый сценарий IDC. Не зная алгоритма, можно действовать так:

1. Открыть защищённый файл; курсор окажется в `start`. Запустить эмулятор сочетанием `Alt-F8`.
2. Выполнить шесть шагов до `sub_5371048` из листинга 21-3, наблюдая команды.
3. Функция выглядит упорядоченной. Можно изучить её либо поставить курсор на `retn` по `05371081h` и выбрать **Run To Cursor**.
4. Распутывание завершено. Ещё два шага выполняют возврат в восстановленный код и заставляют IDA оформить байты как инструкции.

Результат:

```
LOAD:05371082 loc_5371082:
LOAD:05371082                 call    sub_5371048
LOAD:05371082 ; ---------------------------------------------------------
LOAD:05371087                 db    0
LOAD:05371088                 db    0
LOAD:05371089                 db    0
LOAD:0537108A                 db    0
LOAD:0537108B                 db    0
LOAD:0537108C                 db    0
LOAD:0537108D                 db    0
LOAD:0537108E                 db    0
LOAD:0537108F                 db    0
LOAD:05371090 ; ---------------------------------------------------------
LOAD:05371090 loc_5371090:
LOAD:05371090                 pushf
LOAD:05371091                 pop     ebx
LOAD:05371092                 mov     esi, esp
LOAD:05371094                 call    sub_5371117
LOAD:05371099                 mov     ebp, edx
LOAD:0537109B                 cmp     ecx, 20h
LOAD:0537109E                 jl      loc_53710AB
LOAD:053710A4                 xor     eax, eax
LOAD:053710A6                 jmp     loc_53710B5
```

Инструкции изменились, а основное выполнение после расшифрования начинается с `pushf` по `loc_5371090`.

Эмуляция заметно проще отдельного сценария. Разработка универсального эмулятора окупается гибкостью, тогда как сценарий Burneye почти бесполезен в другом случае.

Файл был Linux ELF, но x86emu важен набор команд, а не операционная система или формат. Точно так же он работает с PE Windows, например UPX. Большинство защищённых вредоносных программ рассчитано на Windows, поэтому модуль имеет дополнительные возможности для PE.

Чтобы распаковать UPX, запустите эмулятор в `start`, перенесите курсор на начало циклов восстановления импорта `0040886Ch` из листинга 21-1 и выполните **Run To Cursor**. К этому моменту содержимое распаковано, а окно **Strings** показывает имена библиотек и функций.

При пошаговом прохождении встречается косвенный вызов:

```
UPX1:00408882                 call    dword ptr [esi+808Ch]
```

Он потенциально опасен, потому что цель не видна сразу. Вызов ведёт либо во внутренний раздел кода, либо в разделяемую библиотеку. Эмулятор проверяет, принадлежит ли адрес файлу или экспортам загруженной библиотеки.

Для PE x86emu заранее загружает каталоги экспорта. Если цель лежит вне файла, он просматривает их и определяет библиотечную функцию. Встроенно эмулируются:

CheckRemoteDebuggerPresent	GetTickCount	LocalFree	VirtualAlloc
CreateThread	GetVersion	NtQuerySystemInformation	VirtualFree
GetCurrentThreadId	HeapAlloc	NtQueryInformationProcess	calloc
GetCurrentProcess	HeapCreate	NtSetInformationThread	free
GetCurrentProcessId	HeapDestroy	RtlAllocateHeap	lstrcat
GetModuleHandleA	HeapFree	TlsAlloc	lstrcpy
GetProcAddress	IsDebuggerPresent	TlsFree	lstrlen
GetProcessHeap	LoadLibraryA	TlsGetValue	malloc
GetThreadContext	LocalAlloc	TlsSetValue	realloc

Таблица 21-1. Функции, эмулируемые x86emu

Модуль читает параметры из стека и либо повторяет настоящее действие, либо выполняет минимум и возвращает правдоподобный результат. Для `stdcall` он правильно снимает аргументы.

Функции кучи работают с внутренним сегментом `.heap`; например, `HeapAlloc` возвращает допустимый адрес для записи. `VirtualAlloc` создаёт новый сегмент базы. `IsDebuggerPresent` всегда возвращает ложь.

При `LoadLibraryA` из стека извлекается имя, локальная библиотека открывается для загрузки её экспорта, а вызывающей стороне возвращается подходящий дескриптор. `GetProcAddress` читает библиотеку и имя из аргументов, находит экспорт и возвращает его адрес. Оба вызова отмечаются в окне вывода IDA.

Если внутренней реализации нет, появляется окно с распознанным именем функции и аргументами.

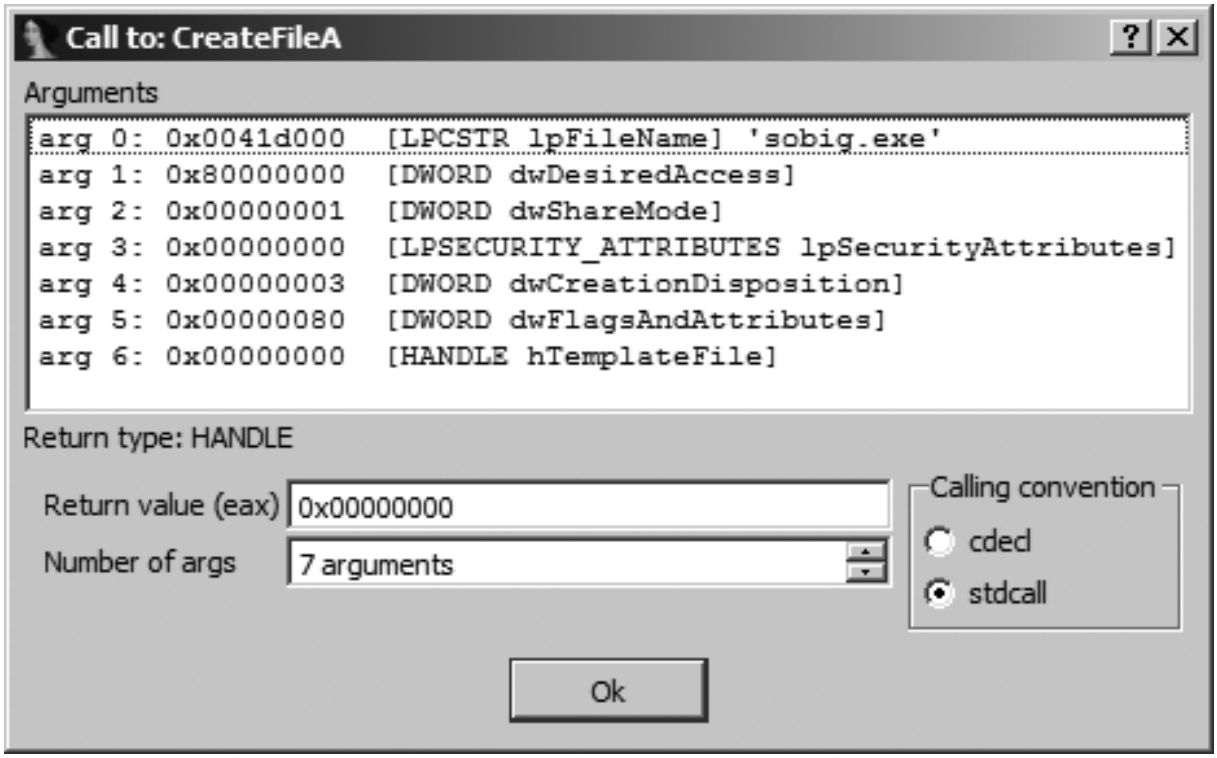


Рисунок 21-8. Вызов функции без встроенной эмуляции

Эмулятор запрашивает библиотеку типов IDA, чтобы узнать число, типы и официальные имена параметров, затем извлекает значения из стека. Пользователь задаёт возвращаемое значение и соглашение о вызовах. Для `stdcall` необходимо указать число аргументов, снимаемых со стека, чтобы сохранить его целостность.

После завершения циклов UPX окно вывода содержит журнал загрузки и разрешения адресов:

```
x86emu: загружена библиотека KERNEL32.DLL (7C800000)
x86emu: найден ExitProcess (0x7C81CDDA)
x86emu: найден ExitThread (0x7C80C058)
x86emu: найден GetCurrentProcess (0x7C80DDF5)
x86emu: найден GetCurrentThread (0x7C8098EB)
x86emu: найден GetFileSize (0x7C810A77)
x86emu: найден GetModuleHandleA (0x7C80B6A1)
x86emu: найден CloseHandle (0x7C809B47)
```

Так сохраняется список загруженных библиотек и найденных функций. Полученные адреса обычно записываются в массив - будущую таблицу импорта - и затем вызываются косвенно без явного связывания при сборке.

У восстановленной программы отсутствуют имена символов обычной таблицы импорта. При целой таблице загрузчик PE IDA именует каждую ячейку по функции, чей адрес там появится. В UPX запись выполняется так:

```
UPX1:00408897        call    dword ptr [esi+8090h] ; GetProcAddress
UPX1:0040889D        or      eax, eax
UPX1:0040889F        jz      short loc_4088A8
UPX1:004088A1        mov     [ebx], eax            ; сохранение в импорте
UPX1:004088A3        add     ebx, 4
```

Адрес `004088A1h` можно назначить точкой сохранения импорта через **Emulate > Windows > Set Import Address Save Point** до её эмуляции. При каждом выполнении x86emu определит функцию по записываемому адресу и присвоит ячейке её имя.

Без обработки таблица выглядела бы так:

```
UPX0:00406270                dd 7C81CDDAh
```

```
UPX0:00406274                 dd 7C80C058h
UPX0:00406278                 dd 7C80DDF5h
UPX0:0040627C                 dd 7C8098EBh
```

После автоматического именования:

```
UPX0:00406270 ; void __stdcall ExitProcess(UINT uExitCode)
UPX0:00406270 ExitProcess       dd 7C81CDDAh
UPX0:00406274 ; void __stdcall ExitThread(DWORD dwExitCode)
UPX0:00406274 ExitThread        dd 7C80C058h
UPX0:00406278 ; HANDLE __stdcall GetCurrentProcess()
UPX0:00406278 GetCurrentProcess dd 7C80DDF5h
UPX0:0040627C ; HANDLE __stdcall GetCurrentThread()
UPX0:0040627C GetCurrentThread  dd 7C8098EBh
```

Теперь IDA правильно подписывает библиотечные вызовы и добавляет параметры из библиотек типов, заметно улучшая листинг.

Дополнительные возможности x86emu

File > Dump. Сохраняет указанный диапазон адресов базы в файл. По умолчанию он начинается с курсора и заканчивается наибольшим виртуальным адресом.

File > Dump Embedded PE. Ищет допустимый PE по курсору, разбирает заголовки, определяет размер и сохраняет байты. Это полезно для вредоносных программ со встроенными исполняемыми файлами, которые позднее устанавливаются в систему.

View > Enumerate Heap. Выводит выделенные блоки кучи:

```
x86emu: состояние кучи
   0x5378000-0x53781ff (0x200 байт)
   0x5378204-0x5378217 (0x14 байт)
   0x537821c-0x5378347 (0x12c байт)
```

Emulate > Switch Thread. В PE модуль перехватывает `CreateThread` и создаёт ресурсы нового потока. Собственного планировщика нет, поэтому переключение выполняется вручную этой командой.

Functions > Allocate Heap Block. Резервирует блок заданного размера во внутренней куче и сообщает его адрес. Это даёт временную рабочую память.

Functions > Allocate Stack Block. Аналогично резервирует область в эмулируемом стеке.

x86emu и противодействие отладке

Хотя x86emu не является отладчиком, он имитирует среду выполнения и не должен попадаться активным проверкам.

Один приём измеряет время инструкцией `rdtsc`. Она читает 64-разрядный счётчик тактов TSC с момента сброса процессора. Настоящий отладчик не останавливает счётчик, поэтому большая разница между двумя чтениями выдаёт паузу.

x86emu ведёт собственный TSC и увеличивает его на каждой эмулированной инструкции. Реальное время между шагами не важно; разница зависит только от числа команд и остаётся достаточно малой, чтобы программа не заметила отладчик.

Намеренные исключения требуют имитации структурной обработки Windows. Для PE эмулятор создаёт `CONTEXT`, находит текущий обработчик по цепочке `fs:[0]` и передаёт ему управление. После возврата состояние процессора восстанавливается из структуры, которую обработчик мог

изменить.

x86emu также имитирует аппаратные отладочные регистры, но не использует их для собственных точек. Поэтому очистка или изменение регистров обработчиком исключения не мешает эмуляции.

Запутывание на основе виртуальной машины

Самые сложные средства переводят исходную программу в особый байт-код и выполняют его собственной виртуальной машиной. Из родного машинного кода исследователь видит лишь интерпретатор, полное понимание которого ещё не раскрывает назначение программы: оно скрыто во встроенном байт-коде.

Нужно найти все области байт-кода и восстановить набор команд виртуальной машины. Это похоже на ситуацию, когда ничего не известно о Java, но даны её виртуальная машина и файл `.class`: сначала пришлось бы понять формат и интерпретатор, а затем уже содержимое файла.

Коммерческий пример - VMProtect. Более учебный - задача HyperUnpackMe2. Рольф Роллес полностью исследовал виртуальную машину и написал процессорный модуль, дизассемблирующий встроенный байт-код.

Ограничение отдельного процессора в том, что одна база показывает либо x86, либо виртуальный код. Приходится создавать две базы. Альтернатива - расширить существующий процессор через события настройки из главы 19 и добавить к x86 инструкции виртуальной машины. Тогда оба вида видны рядом:

```
TheHyper:01013B2F           h_pop.l       R9
TheHyper:01013B32           h_pop.l       R7
TheHyper:01013B35           h_pop.l       R5
TheHyper:01013B38           h_mov.l       SP, R2
TheHyper:01013B3C           h_sub.l       SP, 0Ch
TheHyper:01013B44           h_pop.l       R2
TheHyper:01013B47           h_pop.l       R1
TheHyper:01013B4A           h_retn        0Ch
TheHyper:01013B4A sub_1013919 endp
TheHyper:01013B4D           dd 24242424h
TheHyper:01013B51           dd 0A9A4285Dh ; метка
TheHyper:01013B55 sub_1013B55 proc near
TheHyper:01013B55 var_8     = dword ptr -8
TheHyper:01013B55 var_4     = dword ptr -4
TheHyper:01013B55 arg_0     = dword ptr 8
TheHyper:01013B55 arg_4     = dword ptr 0Ch
TheHyper:01013B55           push    ebp
TheHyper:01013B56           mov     ebp, esp
TheHyper:01013B58           sub     esp, 8
TheHyper:01013B5B           mov     eax, [ebp+arg_0]
TheHyper:01013B5E           mov     [esp+8+var_8], eax
TheHyper:01013B61           mov     [esp+8+var_4], 0
TheHyper:01013B69           push    4
TheHyper:01013B6B           push    1000h
```

В начале показан байт-код HyperUnpackMe2, ниже - обычный x86.

IDA 5.7 добавила пользовательские типы и форматы данных. Новый формат регистрируется сценарием или модулем: задаются имя пункта меню и функция представления. После выбора IDA вызывает её при каждом показе элемента.

Пользовательский тип пригоден, когда встроенные типы не выражают встретившиеся данные. В примере Hex-Rays им отмечается байт-код виртуальной машины, а пользовательский формат показывает каждое значение как инструкцию.

Недостаток - каждую виртуальную инструкцию нужно найти и вручную изменить её тип. Расширение процессорного модуля лучше: обозначение одной инструкции запускает рекурсивное обнаружение всех достижимых команд через реализацию `custom_emu`.

Итоги

Для современных вредоносных программ запутывание стало правилом, а не исключением. Изучение внутреннего поведения почти всегда требует распутывания.

Можно использовать отладчик и динамический запуск либо отказаться от исполнения опасного кода в пользу сценариев и эмуляции. В любом случае цель одна - получить восстановленный двоичный файл, который полностью дизассемблируется и анализируется, обычно средствами IDA.

Поэтому разумно применять IDA от начала до конца. Эта глава показала, что её возможности далеко выходят за рамки простого построения листинга. В главе 25 мы вернёмся к защищённому коду и используем для распутывания встроенный отладчик.

Сноски

[1] Семафор можно представить как единственный жетон для входа в помещение. Пока жетон у одного участника, остальные ждут. После завершения работы жетон передаётся следующему. Так обеспечивается взаимное исключение при доступе к коду или данным.
[2] В Windows регистр `FS` указывает на блок окружения текущего потока TEB. По нулевому смещению находится начало связного списка обработчиков исключений, вызываемых по очереди.
[3] Регистры `Dr0-Dr7` управляют аппаратными точками останова: `Dr0-Dr3` хранят адреса, а `Dr6` и `Dr7` включают и отключают условия.

Глава 22. Анализ уязвимостей

Сразу уточним: IDA не является автоматическим средством поиска уязвимостей. Открытие файла не раскрывает все тайны, комментарии не объясняют полностью вредоносную программу, опасные места не подсвечиваются красным, а щелчок правой кнопкой не создаёт готовую эксплуатацию.

Без опытного исследователя, полезных сценариев и модулей это прежде всего дизассемблер и отладчик. В статическом анализе IDA лишь помогает искать слабые места; успех зависит от навыков и их применения.

По опыту авторов, для обнаружения совершенно новых уязвимостей есть более подходящие средства, особенно фаззинг. Но после обнаружения ошибки IDA вместе с отладчиком становится одним из лучших помощников при разработке эксплуатации.

Есть и другая роль - поиск уже существующих, но не раскрытых уязвимостей. Поставщик закрытой программы нередко выпускает исправление без точного описания. Сравнение новой и старой версий выделяет изменённые участки. Если изменение сделано не случайно, оно указывает на прежний уязвимый код. Исследователь может изучить его и создать эксплуатацию для необновлённых систем. Ежемесячный выпуск исправлений Microsoft давно породил регулярный такой анализ.

Тема заслуживает отдельных книг. Здесь предполагается знакомство с основами, например переполнением буфера, и рассматривается применение IDA для поиска, анализа и последующей разработки эксплуатации.

Поиск новых уязвимостей с IDA

Подходов много. При наличии исходников автоматические средства аудита отмечают вероятные проблемы, но обычно находят лишь простые случаи, а глубокие ошибки требуют ручной проверки.

Автоматический анализ двоичных файлов даёт похожие отчёты без доступа к исходному коду и потому применим к закрытым программам. Однако ни одна система не гарантирует нахождение всех уязвимостей.

При статическом аудите исходников или двоичного файла обычно проверяют:

• опасные функции вроде `strcpy` и `sprintf`;
• использование буферов от `malloc` и `VirtualAlloc`;
• обработку пользовательского ввода из `recv`, `read`, `fgets` и подобных источников.

Найти вызовы несложно. Для `strcpy`:

1. Найти саму функцию.
2. Поставить курсор на метку и открыть **View > Open Subviews > Cross References**.
3. Посетить каждую ссылку и изучить параметры на возможность переполнения.

Третий шаг может потребовать значительного анализа кода и потока данных. Даже первый не всегда тривиален. В PE Windows или статически скомпонованном ELF обычно достаточно команды **Jump > Jump to Address** с клавишей `G` и имени `strcpy`.

В динамически скомпонованном ELF переход часто ведёт в раздел `extern`:

```
extern:804DECC          extrn strcpy:near ; ссылка из _strcpy
extern:804DECC                            ; ссылка из .got:off_804D5E4
```

Имя визуально сдвинуто, а ссылка на код приходит от функции, показанной как `_strcpy`; фактическое имя `.strcpy`. IDA по умолчанию не разрешает точку в идентификаторе и заменяет её подчёркиванием.

Двойной щелчок по ссылке на код открывает запись таблицы связывания процедур `.plt`:

```
.plt:08049E90 _strcpy    proc near
.plt:08049E90            jmp     ds:off_804D5E4
.plt:08049E90 _strcpy    endp
```

Ссылка на данные приводит к соответствующей записи глобальной таблицы смещений `.got`:

```
.got:0804D5E4 off_804D5E4 dd offset strcpy
```

Отсюда ещё одна ссылка ведёт в `.plt`. На практике путь по ссылкам данных из `extern` в `.got`, а затем `.plt` наиболее надёжен. Динамический ELF вызывает функции косвенно через таблицу связывания процедур. Достигнув её, можно открыть ссылки на `_strcpy` и проверить все места.

Для длинного списка функций ручной обход утомителен. Сценарий может автоматически находить вызовы и добавлять комментарии, после чего между местами аудита удобно переходить поиском.

Основа - функция, которая по имени возвращает адрес, пригодный для перебора ссылок:

```
static getFuncAddr(fname) {
    auto func = LocByName(fname);
    if (func != BADADDR) {
        auto seg = SegName(func);
        if (seg == "extern") {
            // Первая и единственная ссылка на данные должна прийти из .got.
            func = DfirstB(func);
            if (func != BADADDR) {
                seg = SegName(func);
                if (seg != ".got") return BADADDR;
                // Следующая ссылка на данные должна прийти из .plt.
                func = DfirstB(func);
                if (func != BADADDR) {
                    seg = SegName(func);
                    if (seg != ".plt") return BADADDR;
                }
            }
        }
        else if (seg != ".text") {
            // Для другого раздела автоматического алгоритма нет.
            func = BADADDR;
        }
    }
    return func;
}
```

Листинг 22-1. Поиск вызываемого адреса функции

Затем все ссылки помечаются комментариями:

```
static flagCalls(fname) {
    auto func, xref;
    func = getFuncAddr(fname);
    if (func != BADADDR) {
        // Прямые и дальние вызовы.
        for (xref = RfirstB(func); xref != BADADDR;
             xref = RnextB(func, xref)) {
            if (XrefType() == fl_CN || XrefType() == fl_CF) {
                MakeComm(xref, "*** ПРОВЕРИТЬ ЗДЕСЬ ***");
            }
        }
        // Места, где берётся адрес функции.
        for (xref = DfirstB(func); xref != BADADDR;
             xref = DnextB(func, xref)) {
            if (XrefType() == dr_O) {
                MakeComm(xref, "*** ПРОВЕРИТЬ ЗДЕСЬ ***");
            }
```

```
        }
    }
}
```

Листинг 22-2. Пометка вызовов указанной функции

После нахождения адреса первый цикл отмечает вызовы, второй - места взятия адреса. Последнее необходимо для косвенного вызова:

```
.text:000194EA                mov     esi, ds:strcpy
.text:000194F0                push    offset loc_40A006
.text:000194F5                add     edi, 160h
.text:000194FB                push    edi
.text:000194FC                call    esi
```

Компилятор сохранил адрес `strcpy` в `ESI`, чтобы последующие вызовы были короче и не требовали разрешения цели. Такие два байта особенно выгодны при нескольких вызовах из одной функции.

Сценарий может увидеть только ссылку на данные, потому что `call esi` напрямую не упоминает `strcpy`. Но IDA умеет ограниченно отслеживать поток данных и часто добавляет пояснение и ссылку на код:

```
.text:000194EA                mov     esi, ds:strcpy
.text:000194F0                push    offset loc_40A006
.text:000194F5                add     edi, 160h
.text:000194FB                push    edi
.text:000194FC                call    esi ; strcpy
```

Тогда `flagCalls` найдёт и сам вызов.

Главная функция перечисляет интересующие цели:

```
static main() {
    flagCalls("strcpy");
    flagCalls("strcat");
    flagCalls("sprintf");
    flagCalls("gets");
}
```

После запуска между отмеченными местами переходят поиском строки `*** ПРОВЕРИТЬ ЗДЕСЬ ***`. Но наличие `strcpy` само по себе ещё не означает эксплуатацию. Нужно выяснить передаваемые параметры и понять, может ли злоумышленник ими управлять. Здесь начинается анализ потока данных.

Он намного сложнее простого поиска вызовов. Средство должно понимать набор команд, происхождение значений регистров, их преобразования и переносы. Также требуется определить размеры исходных и целевых буферов, а значит, устройство кадров стека и глобальных переменных, а также размер динамически выделенных блоков. Всё это выполняется без запуска программы.

Интересный пример творческих сценариев - BugScam Хальвара Флейка. Он находит опасные вызовы и выполняет простейший анализ потока данных, создавая отчёт HTML о вероятных проблемах. Для `sprintf` запись могла выглядеть так:

Адрес	Опасность	Описание
`8048c03`	5	Наибольший размер результата, 1053 байта, превышает целевой буфер в 1036 байт; возможно переполнение

BugScam определил размеры входных и выходных буферов и вместе со спецификаторами строки формата вычислил наибольший вывод.

Подобный сценарий требует глубокого знания классов уязвимостей и общего алгоритма для множества файлов. Но даже без такого опыта можно автоматизировать простые вопросы.

Например, найдём функции с локальными буферами стека, которые могут быть подвержены переполнению. Вместо ручного просмотра сценарий исследует кадр каждой функции и ищет переменные, занимающие много места:

```
def findStackBuffers(func_addr, minsize):
    prev_idx = -1
    frame = GetFrame(func_addr)
    if frame == -1:
        return  # неверная функция

    idx = 0
    prev = None
    while idx < GetStrucSize(frame):
        member = GetMemberName(frame, idx)
        if member is not None:
            if prev_idx != -1:
                # Расстояние от предыдущего поля до текущего.
                delta = idx - prev_idx
                if delta >= minsize:
                    Message("%s: возможный буфер %s: %d байт\n" %
                            (GetFunctionName(func_addr), prev, delta))
            prev_idx = idx
            prev = member
            idx = idx + GetMemberSize(frame, idx)
        else:
            idx = idx + 1
```

Листинг 22-3. Поиск буферов, выделенных в стеке

Функция вызывает `GetMemberName` для допустимых смещений. Размер переменной считается как разность начальных смещений двух соседних полей. Если он не меньше `minsize`, переменная отмечается возможным буфером. При отсутствии элемента индекс увеличивается на байт, иначе - на заявленный размер элемента.

Почему не использовать только `GetMemberSize`? Она верна лишь при правильном размере, заданном IDA или пользователем. Рассмотрим кадр:

```
.text:08048B38 sub_8048B38     proc near
.text:08048B38 var_818         = byte ptr -818h
.text:08048B38 var_418         = byte ptr -418h
.text:08048B38 var_C           = dword ptr -0Ch
.text:08048B38 arg_0           = dword ptr 8
```

Между `var_818` и `var_418` лежит 1024 байта, а между `var_418` и `var_C` - 1036. Но развёрнутое представление может показать:

```
-00000818 var_818         db ?
-00000817                 db ? ; не определено
-00000816                 db ? ; не определено
...
-0000041A                 db ? ; не определено
-00000419                 db ? ; не определено
-00000418 var_418         db 1036 dup(?)
-0000000C var_C           dd ?
```

`var_418` оформлена массивом, а `var_818` выглядит однобайтовой с 1023 неопределёнными байтами. Поэтому `GetMemberSize` вернёт 1 и 1036, хотя логический первый буфер занимает 1024. Расстояние между начальными адресами устойчиво к такому оформлению:

```
sub_8048B38: возможный буфер var_818: 1024 байт
sub_8048B38: возможный буфер var_418: 1036 байт
```

Главная функция может перебрать все функции базы и вызвать `findStackBuffers` для каждой. Это быстро выделит локальные буферы, но возможность их переполнить всё равно проверяется вручную. Именно трудоёмкость статического анализа объясняет популярность фаззинга.

Поиск уязвимости после выхода исправления

Вокруг раскрытия слабых мест постоянно спорят. Условно есть обнаруживший уязвимость исследователь и сопровождающий программу поставщик. События процесса можно описать так, хотя термины не стандартизованы.

`Обнаружение`

: Момент первоначального нахождения; здесь также считаем, что тогда создаётся первая эксплуатация.

`Уведомление`

: Поставщик впервые узнаёт о проблеме. Если он нашёл её сам, совпадает с обнаружением.

`Раскрытие`

: Сведения становятся общедоступными. Степень подробности различается, эксплуатация может быть опубликована или лишь упомянута. Иногда раскрытие одновременно уведомляет поставщика.

`Снижение риска`

: Публикуются временные действия, защищающие пользователя до выпуска исправления.

`Доступность исправления`

: Поставщик или третья сторона выпускает исправленную версию.

`Установка исправления`

: Пользователь действительно обновляет программу и, предположительно, становится защищён от известных атак через эту ошибку.

Часто раскрытие совпадает с выпуском исправления и сопровождается бюллетенем. Он описывает характер и опасность, но обычно не даёт достаточно деталей для рабочей эксплуатации.

Причины исследовать ошибку бывают разными. Злоумышленник стремится атаковать ещё необновлённые системы. Защитник хочет обнаруживать такие машины в сети либо узнавать попытки эксплуатации в реальном времени. Для обоих необходимо точное понимание.

Бюллетень может не назвать файл, функцию, адрес и конкретное изменение. Но исправленные файлы содержат всё это в виде отличий, внесённых для устранения ошибки. Сравнение новой версии со старой выделяет необходимые участки.

Для исходников достаточно текстового `diff`. Двоичные версии сравнивать намного сложнее: различия возникают из-за оптимизации, новой версии компилятора, перестройки исходников, несвязанных функций и самого исправления. Нужно отделить изменение поведения от косметики, например выбора другого регистра для той же операции.

Существуют специализированные средства: коммерческий BinDiff, бесплатные Binary Diffing Suite, Turbodiff и PatchDiff2. Все используют IDA. BinDiff и BDS извлекают сведения сценариями и модулями, сохраняют их во внешней базе и показывают различия графами. Turbodiff и PatchDiff2 полностью работают внутри IDA.

Их цель - быстро выделить исправленный код и понять прежнюю уязвимость. Рассмотрим открытый PatchDiff2, поставлявшийся в 32- и 64-разрядных вариантах Windows. Для установки

двоичный файл копируют в `<IDADIR>/plugins`.

Сначала создаются две отдельные базы: для исходной и исправленной программ.

Свойство	Значение
Имя	PatchDiff2
Автор	Николя Пувезль
Распространение	Исходники и двоичные файлы для IDA 5.7
Цена	Бесплатно
Назначение	Построение и показ различий двоичных файлов

Обычно открывают исходную базу и запускают PatchDiff2 через **Edit > Plugins** или `Ctrl-8`. Она называется `IDB1`. Затем выбирается исправленная `IDB2`.

Модуль вычисляет для каждой функции признаки, сигнатуры, свёртки и контрольные суммы CRC, затем открывает три списка:

• **Identical Functions** - одинаковые функции, обычно неинтересные для исправления;
• **Unmatched Functions** - функции без надёжной пары: добавленные, удалённые или слишком похожие на другие;
• **Matched Functions** - сопоставленные, но не обязательно одинаковые функции.

В несопоставленном списке пары часто удаётся назначить вручную. Полезно отсортировать по столбцу `sig`, чтобы функции с похожим числом сигнатур оказались рядом:

```
Файл Функция      Адрес     Sig      Hash     CRC
1    sub_7CB25FE9 7CB25FE9 000000F0 F4E7267B 411C3DCC
1    sub_7CB6814C 7CB6814C 000000F0 F4E7267B 411C3DCC
2    sub_7CB6819A 7CB6819A 000000F0 F4E7267B 411C3DCC
2    sub_7CB2706A 7CB2706A 000000F0 F4E7267B 411C3DCC
```

Две функции первого файла явно связаны с двумя функциями второго, но автоматическое попарное соответствие неоднозначно. Такое часто бывает у двоичных файлов со стандартной библиотекой шаблонов C++ STL.

Команда **Set Match** контекстного меню позволяет выбрать функцию одного файла и назначить ей пару из другого.

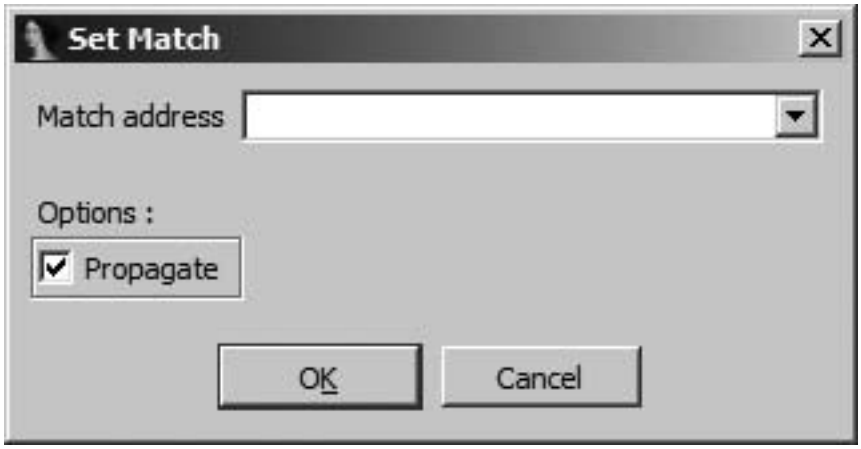


Рисунок 22-1. Ручное сопоставление функций в PatchDiff2

Сначала выбирают функцию командой **Set Match**, затем вводят адрес пары из другой базы. Флажок **Propagate** просит PatchDiff2 распространить новое соответствие и автоматически сопоставить дополнительные функции.

Список **Matched Functions** содержит достаточно похожие, но не идентичные пары. Команда **Display Graphs** из контекстного меню показывает их графы потока рядом.

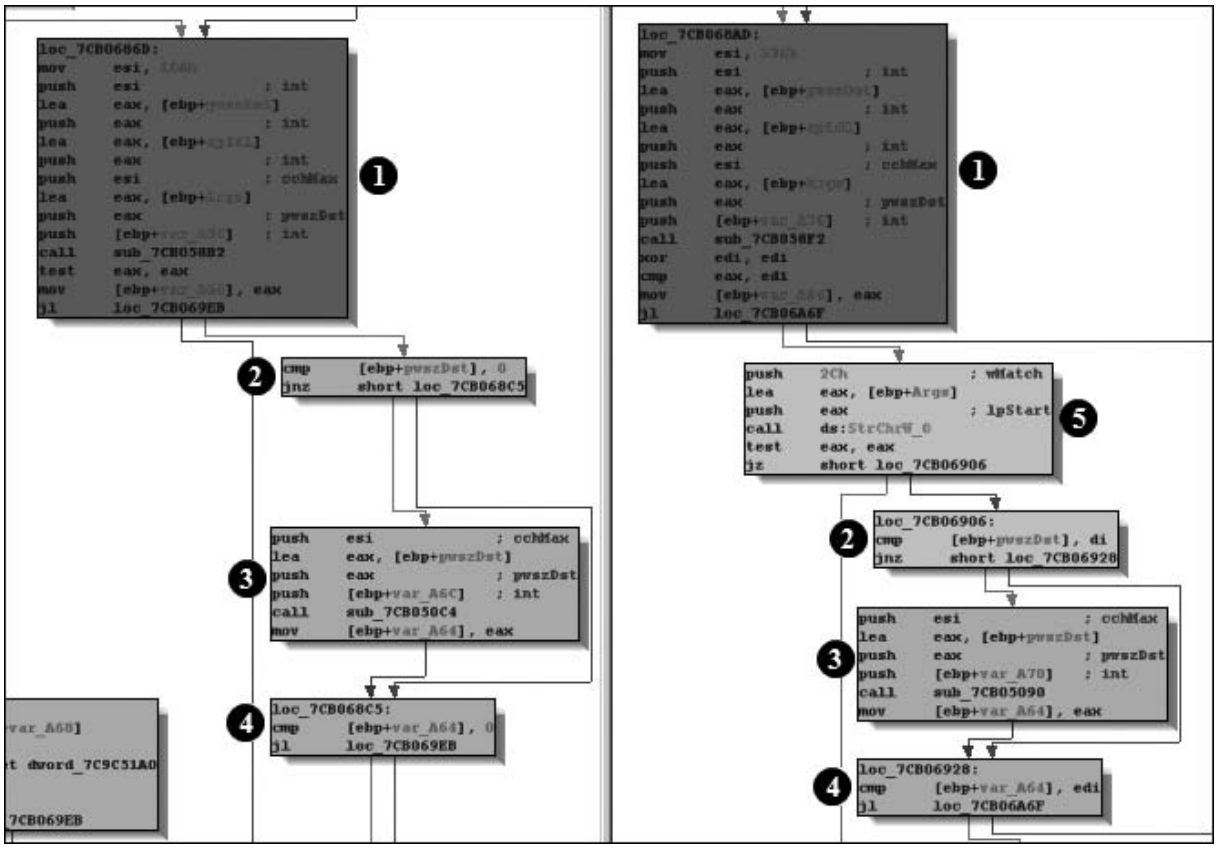


Рисунок 22-2. Графическое сравнение функций в PatchDiff2

На рисунке блоки 1-4 есть в обеих версиях, а блок 5 добавлен исправлением. Поэтому сопоставленные функции обычно интереснее всего: именно в них вероятны новые проверки и исправления неправильного либо эксплуатируемого поведения. Если здесь ничего важного нет, остаётся изучать несопоставленные функции.

IDA в разработке эксплуатации

После нахождения вероятно эксплуатируемой ошибки IDA экономит много проб и ошибок:

• графы показывают пути управления к уязвимой функции; в больших программах параметры графа нужно ограничивать;
• подробные кадры стека показывают, какие части переполняющего буфера затирают какие значения, и помогают с буферами строк формата;
• поиск быстро находит отдельную инструкцию вроде `jmp esp` или последовательность `pop/pop/ret` и сообщает точный виртуальный адрес;
• отображение файла как загруженного в память облегчает поиск адресов глобальных буферов и целей записи, например записей GOT.

Разбор кадра стека

Современные системы всё чаще запрещают исполнение стека, но многие компьютеры ещё допускают классическое переполнение с машинным кодом в стеке. Даже при защите можно повредить стековые указатели и использовать их дальше.

В любом случае важно знать, что именно будет перезаписано и сколько байтов нужно до конкретной переменной или сохранённого адреса возврата. В IDA расстояние вычисляется вычитанием смещений.

Пример уязвимого кадра:

```
-0000009C result          dd ?
-00000098 buffer_132      db 132 dup(?)  ; этот буфер переполняется
-00000014 p_buf           dd ?           ; указатель внутрь buffer_132
-00000010 num_bytes       dd ?           ; байтов за один цикл
-0000000C total_read      dd ?           ; всего прочитано
-00000008                 db ?           ; не определено
-00000007                 db ?           ; не определено
-00000006                 db ?           ; не определено
-00000005                 db ?           ; не определено
-00000004                 db ?           ; не определено
-00000003                 db ?           ; не определено
-00000002                 db ?           ; не определено
-00000001                 db ?           ; не определено
+00000000 s               db 4 dup(?)
+00000004 r               db 4 dup(?)     ; сохранённый адрес возврата
+00000008 filedes         dd ?            ; дескриптор сокета
```

От начала `buffer_132` до адреса возврата 156 байт: `4 - (-98h)`. Через 132 байта начинается перезапись `p_buf`, что уже может нарушить работу.

Нужно понимать последствия для всех следующих переменных, чтобы программа не завершилась до срабатывания эксплуатации. Например, если после переполнения используется `filedes`, испорченный дескриптор вызовет ошибку. Поэтому перезаписываемым полям иногда подбирают правдоподобные значения, позволяющие продолжить выполнение.

Сценарий из листинга 22-3 можно изменить так, чтобы перечислить все элементы, вычислить видимый размер и расстояние до адреса возврата:

```
func = ScreenEA()  # функция в позиции курсора
frame = GetFrame(func)
if frame != -1:
    Message("Перечень стека функции %s\n" % GetFunctionName(func))
    eip_loc = GetFrameLvarSize(func) + GetFrameRegsSize(func)

    prev_idx = -1
    idx = 0
    prev = None
    while idx < GetStrucSize(frame):
        member = GetMemberName(frame, idx)
        if member is not None:
            if prev_idx != -1:
                delta = idx - prev_idx
                Message("%15s: %4d байт (%4d до eip)\n" %
                        (prev, delta, eip_loc - prev_idx))
            prev_idx = idx
            prev = member
            idx = idx + GetMemberSize(frame, idx)
        else:
            idx = idx + 1

    if prev_idx != -1:
        delta = GetStrucSize(frame) - prev_idx
        Message("%15s: %4d байт (%4d до eip)\n" %
                (prev, delta, eip_loc - prev_idx))
```

Листинг 22-4. Перечисление одного кадра стека на Python

`GetFrameLvarSize` и `GetFrameRegsSize`, доступные также в IDC, возвращают размеры областей локальных переменных и сохранённых регистров. Сразу после них лежит адрес возврата, поэтому его смещение равно сумме.

Для примера вывод будет таким:

```
Перечень стека функции handleSocket
         result:    4 байт (160 до eip)
     buffer_132:  132 байт (156 до eip)
          p_buf:    4 байт ( 24 до eip)
       num_bytes:    4 байт ( 20 до eip)
     total_read:   12 байт ( 16 до eip)
              s:    4 байт (  4 до eip)
              r:    4 байт (  0 до eip)
        filedes:    4 байт ( -4 до eip)
```

Так получается краткая карта кадра с полезными для разработки расстояниями.

Кадр помогает и при ошибках строк формата. Здесь `fprintf` получает пользовательский буфер как строку формата:

```
.text:080488CA        lea     eax, [ebp+format]
.text:080488D0        mov     [esp+4], eax    ; format
.text:080488D4        mov     eax, [ebp+stream]
.text:080488DA        mov     [esp], eax      ; stream
.text:080488DD        call    _fprintf
```

Передаются только поток и адрес пользовательского буфера. Компилятор заранее выделил место аргументов в прологе. Кадр:

```
-00000128                db ? ; не определено
-00000127                db ? ; не определено
-00000126                db ? ; не определено
-00000125                db ? ; не определено
-00000124                db ? ; не определено
-00000123                db ? ; не определено
-00000122                db ? ; не определено
-00000121                db ? ; не определено
-00000120                db ? ; не определено
-0000011F                db ? ; не определено
-0000011E                db ? ; не определено
-0000011D                db ? ; не определено
-0000011C                db ? ; не определено
-0000011B                db ? ; не определено
-0000011A                db ? ; не определено
-00000119                db ? ; не определено
-00000118 s1             dd ? ; смещение
-00000114 stream         dd ? ; смещение
-00000110 format         db 264 dup(?)
```

Листинг 22-5. Кадр стека для строки формата

Шестнадцать неопределённых байтов от `128h` до `119h` зарезервированы под аргументы вызываемых функций. `stream` размещается на вершине, указатель `format` сразу под ним.

Между указателем строки формата и началом самого буфера 16 байт. Пусть ввод равен:

```
%x %x %x %x %x
```

`fprintf` ожидает пять аргументов после строки. Первые четыре занимают эти 16 байт, а пятый перекрывает первые четыре байта пользовательского буфера.

В строке формата аргумент можно выбрать явным индексом:

```
%5$x
```

Тогда первые четыре байта буфера читаются как целое и выводятся в шестнадцатеричном виде. Шестой, седьмой и последующие аргументы перекрывают следующие четырёхбайтовые блоки.

Эксплуатация строки формата требует точных индексов. IDA быстро вычисляет эти смещения. Вместе с адресами GOT, таблицы деструкторов `.dtor` и других разделов это позволяет построить правильную строку без многократных проб в отладчике.

Поиск последовательностей инструкций

Для надёжной передачи управления полезно не зависеть от точного адреса машинного кода в куче или стеке. Если в момент захвата указателя команд `ESI` уже указывает на нужный буфер, адрес `jmp esi` или `call esi` передаст управление прямо туда. Для стека часто пригодна `jmp esp`: после возврата из уязвимой функции `ESP` указывает сразу за перезаписанным адресом возврата.

Сочетание регистра, указывающего на внедрённый код, и инструкции перехода либо вызова по этому регистру называется трамплином.

Поиск таких последовательностей давно автоматизируется отдельными средствами, но IDA справляется не хуже. Допустим, нужна `jmp esp` в x86. Текстовый поиск ненадёжен: требуется точное число пробелов, а компилятор редко порождает такую инструкцию явно. На деле важна последовательность байтов `FF E4` в любом месте.

Она может находиться даже внутри другой инструкции:

```
.text:080486CD B8 FF FF E4 34    mov eax, 34E4FFFFh
```

Адрес `080486CFh` начинает нужную `jmp esp`. Поэтому следует использовать **Search > Sequence of Bytes**.

Для точного двоичного совпадения включается чувствительность к регистру знаков. Иначе `50 C3` (`push eax; ret`) может совпасть с `70 C3`, поскольку байты `50h` и `70h` соответствуют латинским буквам разного регистра, хотя вторая последовательность означает условный переход с другим смещением.

Поиск легко выполнить сценарием:

```
ea = FindBinary(MinEA(), SEARCH_DOWN | SEARCH_CASE, "FF E4");
```

Он идёт от наименьшего виртуального адреса вверх по адресам и возвращает начало первого совпадения либо `BADADDR`, то есть -1.

Сценарий для нескольких вариантов может вывести:

```
Поиск...
Найдена jmp edx (FF E2) по адресу 0x80816e6
Найдена call edx (FF D2) по адресу 0x8048138
Всего совпадений: 2
```

Такие сценарии экономят время и не дают забыть один из вариантов.

Поиск полезных виртуальных адресов

Статический буфер в `.data` или `.bss` удобнее кучи и стека: его фиксированный адрес известен, поэтому не нужен трамплин или длинная полоса nop.

Полоса NOP

> Это длинная последовательность nop, расширяющая допустимую цель при немного плавающем адресе машинного кода. Вместо первой полезной инструкции переход направляют в середину полосы. Если вся нагрузка сдвинется, исполнение всё равно попадёт на один из nop и дойдёт до кода. Например, полоса из 500 байт допускает ошибку примерно до 250 байт в любую сторону.

Некоторые ошибки дают возможность записать выбранные четыре байта по выбранному адресу. Этого часто достаточно, чтобы заменить указатель на функцию адресом внедрённого кода.

Динамический ELF хранит адреса библиотечных функций в глобальной таблице смещений GOT. Перезаписав её запись, можно перенаправить будущий библиотечный вызов. Обычно машинный код заранее размещают по известному адресу, затем меняют запись следующей вызываемой функции.

Адрес записи легко найти в разделе `.got`, но сценарий делает это быстрее. Курсор ставится на вызов библиотечной функции:

```
ea = ScreenEA()
dref = ea
for xref in XrefsFrom(ea, 0):
    if xref.type == fl_CN and SegName(xref.to) == ".plt":
        for dref in DataRefsFrom(xref.to):
            Message("Запись GOT для %s находится по 0x%08x\n" %
                    (GetFunctionName(xref.to), dref))
            break
if ea == dref:
    Message("Похоже, это не вызов библиотечной функции\n")
```

Например:

```
.text:080513A8                call    _memset
```

Первая ссылка проверяется как вызов в `.plt`. Запись PLT читает адрес из GOT и передаёт туда управление. Следующая ссылка на данные даёт адрес нужной ячейки. Результат:

```
Запись GOT для .memset находится по адресу 0x080618d8
```

Если цель - перехватить `memset`, содержимое `0x080618d8` заменяется адресом внедрённого кода.

Анализ машинного кода полезной нагрузки

До сих пор IDA применялась преимущественно для наступательного исследования. Защитная задача - понять перехваченную машинную нагрузку. Для этого её тоже нужно дизассемблировать.

Сырой двоичный файл открывается в IDA без формата, затем байты преобразуются в код. Другой источник - захват сетевых пакетов. Wireshark умеет собрать поток TCP командой **Follow TCP Stream** и сохранить исходное содержимое в файл для двоичной загрузки IDA.

Сетевой сеанс часто смешивает протокольные данные с машинным кодом. Начало полезной нагрузки приходится находить вручную. Иногда её выдаёт длинная последовательность `0x90`, но возможна полиморфная полоса из случайных однобайтовых команд без побочного эффекта, которую труднее обнаружить сигнатурной системе.

Пример собранного клиентского потока атаки на соревновании DEFCON 18:

```
00000000   AD 02 0E 08 01 00 00 00 47 43 4E 93 43 4B 91 90  ........GCN.CK..
00000010   92 47 4E 46 96 46 41 4A 43 4F 99 41 40 49 48 43  .GNF.FAJCO.A@IHC
00000020   4A 4E 4B 43 42 49 93 4B 4A 41 47 46 46 46 43 90  JNKCBI.KJAGFFFC.
00000030   4E 46 97 4A 43 90 42 91 46 90 4E 97 42 48 41 48  NF.JC.B.F.N.BHAH
00000040   97 93 48 97 93 42 40 4B 99 4A 6A 02 58 CD 80 09  ..H..B@K.Jj.X...
00000050   D2 75 06 6A 01 58 50 CD 80 33 C0 B4 10 2B E0 31  .u.j.XP..3...+.1
```

```
00000060   D2 52 89 E6 52 52 B2 80 52 B2 04 52 56 52 52 66  .R..RR..R..RVRRf
00000070   FF 46 E8 6A 1D 58 CD 80 81 3E 48 41 43 4B 75 EF  .F.j.X...>HACKu.
00000080   5A 5F 6A 02 59 6A 5A 58 99 51 57 51 CD 80 49 79  Z_j.YjZX.QWQ..Iy
00000090   F4 52 68 2F 2F 73 68 68 2F 62 69 6E 89 E3 50 54  .Rh//shh/bin..PT
000000A0   53 53 B0 3B CD 80 41 41 49 47 41 93 97 97 4B 48  SS.;..AAIGA...KH
```

На основе остальных сведений соединения двоичная часть считается исполняемой. После загрузки и ручного определения границы IDA показывает:

```
seg000:00000000           db 0ADh
seg000:00000001           db    2
seg000:00000002           db  0Eh
seg000:00000003           db    8
seg000:00000004           db    1
seg000:00000005           db    0
seg000:00000006           db    0
seg000:00000007           db    0
seg000:00000008 ; ---------------------------------------------------------
seg000:00000008           inc     edi
seg000:00000009           inc     ebx
seg000:0000000A           dec     esi
; ... полоса безвредных команд и инициализация опущены ...
seg000:0000006D           push    edx
seg000:0000006E           push    edx
seg000:0000006F loc_6F:
seg000:0000006F           inc     word ptr [esi-18h]
seg000:00000073           push    1Dh
seg000:00000075           pop     eax
seg000:00000076           int     80h     ; IDA ошибочно: Linux sys_pause
seg000:00000078           cmp     dword ptr [esi], 4B434148h
seg000:0000007E           jnz     short loc_6F
seg000:00000080           pop     edx
seg000:00000081           pop     edi
seg000:00000082           push    2
seg000:00000084           pop     ecx
seg000:00000085 loc_85:
seg000:00000085           push    5Ah
seg000:00000087           pop     eax
seg000:00000088           cdq
seg000:00000089           push    ecx
seg000:0000008A           push    edi
seg000:0000008B           push    ecx
seg000:0000008C           int     80h     ; IDA ошибочно: Linux old_mmap
seg000:0000008E           dec     ecx
seg000:0000008F           jns     short loc_85
seg000:00000091           push    edx
seg000:00000092           push    'hs//'
seg000:00000097           push    'nib/'
; ... далее вызывается execve для запуска оболочки ...
```

Первые восемь байтов являются протокольными данными и не дизассемблируются. IDA также неверно определила системные вызовы: нагрузка предназначалась FreeBSD, а программа умеет подписывать только номера Linux. В FreeBSD вызов 29 (`1Dh`) - `recvfrom`, а 90 (`5Ah`) - `dup2`, а не `pause` и `old_mmap`.

У сырой нагрузки нет заголовков, помогающих IDA, поэтому границы и платформа требуют ручного внимания. Кроме того, кодировщики полезной нагрузки скрывают её от систем обнаружения вторжений примерно так же, как упаковщики обычных файлов, ещё сильнее усложняя дизассемблирование.

Итоги

IDA не является волшебным средством, автоматически извлекающим уязвимости из двоичного файла. Если анализ выполняется преимущественно в ней, следует максимально автоматизировать

повторяемые алгоритмы сценариями и модулями.

Наконец, одной теории недостаточно. Навык анализа уязвимостей и разработки эксплуатации появляется только с практикой на специально подготовленных задачах и учебных полигонах.

Глава 23. Практические модули расширения IDA

За годы применения IDA пользователи создали множество расширений под свои задачи. Единого каталога для всех открытых модулей нет. Их традиционно искали на странице загрузок Hex-Rays, в архиве OpenRCE, на форумах обратной разработки и через обычный поиск.

Установка стороннего кода бывает непростой. Расширение распространяется исходниками, двоичным файлом или обоими способами. Сборка может зависеть от файла `makefile`, не подходящего к вашей среде. Готовый файл способен быть собран несовместимой версией SDK. Наконец, могут понадобиться внешние библиотеки для сборки или запуска.

В этой главе рассматриваются несколько популярных расширений: назначение, получение, сборка, установка и применение.

Hex-Rays

Одно из самых известных расширений - коммерческий декомпилятор Hex-Rays от создателей IDA. Рассматриваемая версия создаёт «похожий на C псевдокод» функций ARM и 32-разрядной x86 и поставляется только в двоичном виде. Установка сводится к копированию файла в `<IDADIR>/plugins`.

Декомпиляция текущей функции запускается командой **View > Open Subviews > Pseudocode** либо клавишей `F5`. **File > Produce File > Create C File** или `Ctrl-F5` обрабатывает все функции базы и сохраняет результат.

Для одной функции открывается новая вкладка. Пример из файла соревнования DEFCON 15:

```
signed int __cdecl sub_80489B4(int fd)
{
    int v1;        // eax@1
    signed int v2; // edx@1
    char buf;      // [sp+4h] [bp-208h]@2
    char s;        // [sp+104h] [bp-108h]@2

    v1 = sub_8048B44(fd,
        (int)"Ханс Брикс? Какая встреча! Рад снова вас видеть! ", 0);
    v2 = -1;
    if (v1 != -1)
    {
        recv(fd, &buf, 0x100u, 0);
        snprintf(&s, 0x12Cu, "Ханс Брикс говорит: \"%s\"\n", &buf);
        sub_8048B44(fd, (int)&s, 0);
        v2 = 0;
    }
    return v2;
}
```

Листинг 23-1. Пример вывода Hex-Rays

Для аргументов используются условные имена `a1`, `a2`, а для локальных переменных - `v1`, `v2`, но разница между ними сохраняется. Имена, уже заданные в дизассемблированном тексте, переносятся в псевдокод.

Свойство	Значение
Имя	Hex-Rays Decompiler

Свойство	Значение
Автор	Ильфак Гильфанов, Hex-Rays
Распространение	Только двоичный файл
Цена на момент издания	2239 долларов США
Назначение	Псевдокод C из функций ARM и 32-разрядной x86

Декомпилятор выводит типы по тем же признакам, что IDA, и часто вставляет приведения для согласования операций. Их можно скрыть командой **Hide Casts**.

Окно псевдокода работает почти как редактор и средство навигации исходников. Двойной щелчок по имени функции декомпилирует её в той же вкладке. Контекстное меню позволяет менять имена и типы переменных и функций.

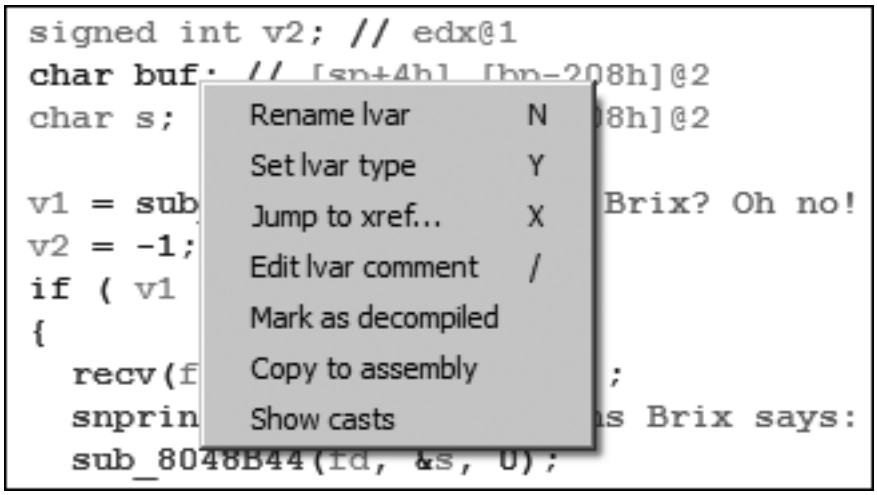


Рисунок 23-1. Редактирование в декомпиляторе Hex-Rays

Изменения имён и типов возвращаются в окна дизассемблирования. После повторного **Rename**, **Set Type** и скрытия приведений листинг 23-1 становится понятнее:

```
signed int __cdecl sub_80489B4(int fd)
{
    int length;
    signed int error;
    char buf[256];
    char s[264];

    length = write_string(fd,
        "Ханс Брикс? Какая встреча! Рад снова вас видеть! ", 0);
    error = -1;
    if (length != -1)
    {
        recv(fd, buf, 256u, 0);
        snprintf(s, 300u, "Ханс Брикс говорит: \"%s\"\n", buf);
        write_string(fd, s, 0);
        error = 0;
    }
    return error;
}
```

При компиляции теряются внутренние символы, а оптимизация устраняет избыточность и перестраивает код. Поэтому в псевдокоде бывают многочисленные приведения и больше `goto`, чем

написал бы человек: поток компилятора не всегда удаётся красиво вернуть к исходным конструкциям C. При этом Hex-Rays распознаёт сложные конструкции вроде `switch` и множество типичных последовательностей разных компиляторов.

Не следует слепо доверять декомпиляции. C короче и привычнее ассемблера, но восстановление не идеально. Все важные выводы из псевдокода нужно сверять с настоящими инструкциями. Для программы C++ результат всё равно будет кодом C без специфических возможностей C++.

IDAPython

IDAPython начинался как стороннее открытое расширение Гергели Эрдеи и подробно рассматривался в главе 15. С IDA 5.4 он входит во все поставки, но исходный проект можно скачать и изменять.

Инструкции сборки находятся в `BUILDING.txt`, установка описана на сайте проекта. Для самостоятельной сборки требовался рабочий 32-разрядный Python. На момент издания версия 3.x не поддерживалась.

Свойство	Значение
Имя	IDAPython
Автор	Гергели Эрдеи
Распространение	Исходники и двоичный файл; готовый вариант входит в IDA
Цена	Бесплатно
Назначение	Среда сценариев Python для IDA Pro

Сценарий `build.py` использует SWIG для создания связующего кода между Python и библиотеками C++ IDA. Заголовки SDK начиная с 5.4 содержат макросы совместимости. Нужен также компилятор C++: Microsoft Visual C++ в Windows и g++ в Linux и macOS.

collabREate

collabREate позволяет нескольким исследователям одновременно работать с одним двоичным файлом. Он объединяет клиент синхронизации внутри IDA, сервер и базу SQL, предоставляя больше возможностей, чем простая передача изменений.

Свойство	Значение
Имя	collabREate
Авторы	Крис Игл и Тим Видас
Распространение	Исходники C++ и двоичные файлы, включая бесплатную IDA

Свойство	Значение
Цена	Бесплатно
Назначение	Совместная синхронизация удалённых сеансов IDA

Архитектура развивает идеи IDA Sync. Клиент обрабатывает обновления базы и связывается с удалённым сервером. Поскольку IDA однопоточна, сеть должна работать асинхронно и без блокировки. До IDA 6.0 применялись асинхронные сокеты Windows, затем классы сокетов Qt сделали модуль многоплатформенным.

collabREate подписывается на события процессов и IDB, перехватывая изменения базы и отправляя их серверу. Число событий IDA росло от версии к версии, поэтому модуль использует максимум доступных в конкретной сборке.

Благодаря протоколу даже пользователи несовместимых версий IDA, например 5.2 и 6.0, могут синхронизироваться, хотя напрямую обменяться `.idb` не способны.

Участник отдельно выбирает публикацию своих изменений и подписку на чужие. Опытный исследователь может передавать свои правки, не принимая чужих. Фильтруются и виды действий: изменение байтов и имён, добавление или удаление комментариев. Один пользователь публикует только комментарии, другой получает лишь новые имена и исправленные байты.

Глубокая интеграция с SDK важна: уведомления связаны с действиями над базой, а не только с ручными действиями пользователя. Их могут порождать также сценарии и другие модули.

Вызовы функций API тоже создают уведомления. Если сценарий исправляет байты базы, переименовывает адреса или переменные либо добавляет комментарии, эти действия отправляются серверу collabREate и передаются остальным участникам проекта.

Сервер collabREate написан на Java и обращается через JDBC[1] к базе SQL. Он управляет пользователями и проектами. Учётные записи настраиваются из командной строки сервера, а проекты создают подключившиеся пользователи. После проверки учётных данных клиент передаёт серверу хеш MD5 исследуемого файла: так подтверждается, что участники действительно работают с одинаковыми входными данными.

При первом подключении пользователь выбирает виды обновлений, на которые хочет подписаться. Затем сервер пересылает все подходящие изменения, накопившиеся после предыдущего сеанса.

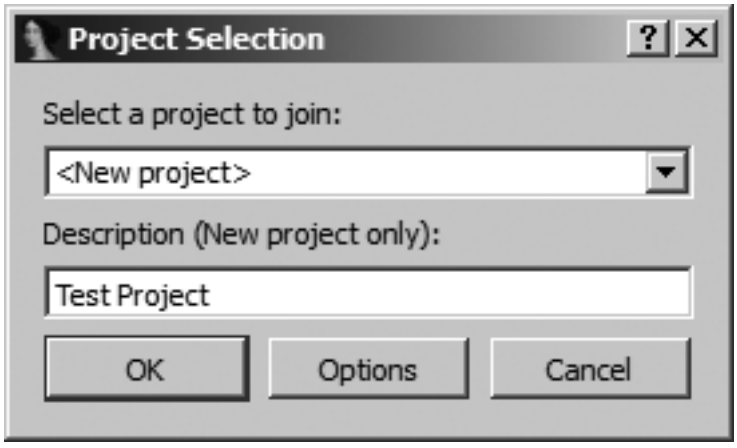


Рисунок 23-2. Выбор проекта collabREate

В раскрывающемся списке показаны проекты, совместимые с текущей базой. Всегда можно создать новый проект и добавить описание, которое увидят остальные участники.

Существующий проект разрешается разветвить. Это позволяет независимо внести и отслеживать множество изменений, не навязывая их другим пользователям. Поскольку одному двоичному файлу могут соответствовать несколько проектов, клиент и сервер дополнительно проверяют, к какой именно ветви относится текущая база.

Полного отката сервер не предоставляет, но поддерживает подобие точки сохранения. В любой момент можно создать снимок. Чтобы вернуться к этому состоянию, пользователь заново открывает двоичный файл в новой базе `.idb` и создаёт ветвь от снимка. Команды ветвления и создания снимка доступны через то же сочетание клавиш, которым модуль запускается впервые.

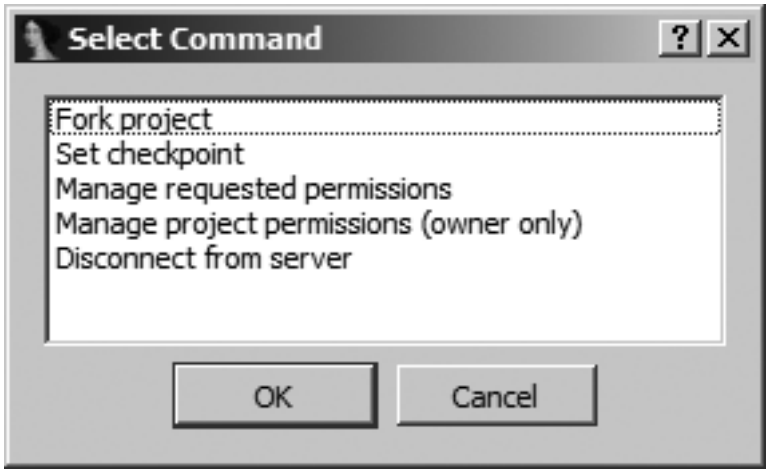


Рисунок 23-3. Выбор команды collabREate

Сервер также ограничивает разрешённые виды обновлений. Одной учётной записи можно позволить только подписку, другой - публикацию комментариев, третьей - публикацию любых изменений.

ida-x86emu

При обратной разработке часто приходится вручную прослеживать инструкции, чтобы понять поведение функции. Для этого нужны знание набора команд и справочник по редко встречающимся инструкциям. Эмулятор помогает учитывать изменения регистров и состояния процессора на длинном участке кода. Одним из таких средств является ida-x86emu, подробно рассмотренный в главе 21.

Свойство	Значение
Имя	ida-x86emu
Автор	Крис Игл
Распространение	Исходники для SDK 6.1 и двоичные файлы для всех версий IDA начиная с 5.0, включая бесплатную; исходники совместимы вплоть до SDK 4.9
Цена	Бесплатно
Назначение	Встроенный эмулятор инструкций x86 для IDA

Модуль распространяется в исходном и двоичном виде и совместим с SDK IDA 4.6 и новее. В комплект входят сценарии и проекты для сборки с MinGW или Microsoft Visual Studio в Windows и с g++ в других системах. Также поставляется готовая сборка для бесплатной IDA.

ida-x86emu работает со всеми версиями IDA на Qt. До IDA 6.0 он поддерживал только графический вариант IDA для Windows.

Модуль разрабатывался в том числе для самомодифицирующегося кода. Он читает байты инструкций из текущей базы IDA, декодирует их и выполняет соответствующие операции. Результатом может быть изменение внутренних регистров эмулятора или запись в базу, если код меняет сам себя. Имитируемые стек и куча размещаются в новых сегментах IDA. Подробности использования приведены в главе 21.

Class Informer

Как говорилось в главе 8, программы C++ могут содержать сведения, помогающие восстановить имена и иерархии классов. Эти данные предназначены для определения типов во время выполнения, RTTI.

Class Informer, созданный Сирмабасом, облегчает разбор программ C++, собранных Microsoft Visual Studio. Он автоматизирует значительную часть методики Игоря Скочинского по исследованию Microsoft Visual C++: находит таблицы виртуальных функций и данные RTTI, затем извлекает имена классов и сведения о наследовании.

Свойство	Значение
Имя	Class Informer
Автор	Сирмабас
Распространение	Только двоичный файл
Цена	Бесплатно
Назначение	Распознавание классов Microsoft Visual C++

После запуска появляется окно параметров. В нём задаются области двоичного файла для поиска таблиц виртуальных функций и подробность отчёта.

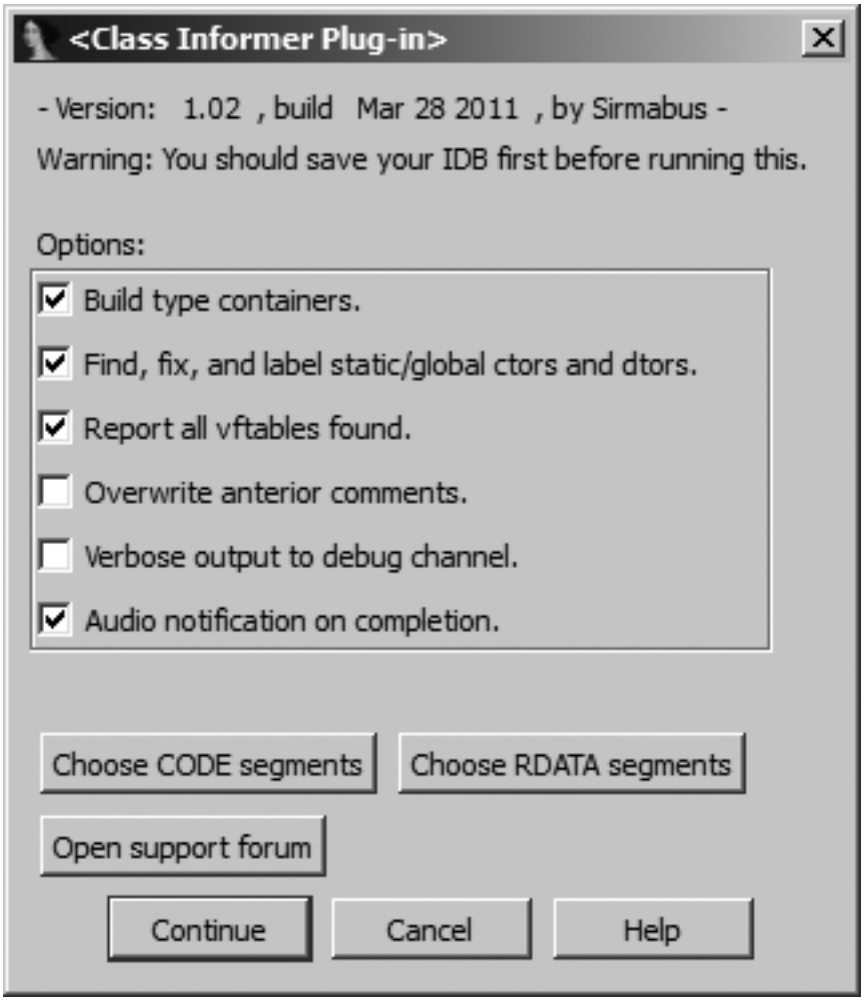


Рисунок 23-4. Параметры Class Informer

После нажатия **Continue** начинается поиск. Его продолжительность зависит от размера файла и числа найденных таблиц. По окончании модуль открывает в IDA новую вкладку со сводкой, похожей на следующую:

```
Vftable   Method count   Class & structure info
0041A298  0003           ChildClass;                          [MI]
0041A2A8  0003           ChildClass: SuperClass1, SuperClass2; [MI]
0041A2B8  0003           SuperClass1;                         [SI]
0041A2C8  0003           SuperClass2;                         [SI]
0041A2D8  0004           BaseClass;                           [SI]
0041A2EC  0005           SubClass: BaseClass;                 [SI]
```

Для каждой таблицы показаны её адрес, число методов - то есть указателей функций - и сведения о классе из RTTI. Восстанавливаются имя класса, имена родительских классов и вид наследования: от одного базового класса (`[SI]`) или нескольких (`[MI]`).

Кроме того, Class Informer применяет шаблоны структур ко всем связанным данным RTTI и присваивает структурам и таблицам имена по правилам искажения имён Microsoft. При исследовании сколько-нибудь сложного кода Visual C++ это экономит много времени.

MyNav

Набор сценариев Python под названием MyNav, созданный Джоксеаном Коретом, формально не является обычным двоичным модулем, но служит полезным расширением IDA. В 2010 году он

занял первое место в конкурсе Hex-Rays по созданию расширений.

Сценарий `mynav.py` следует запускать после загрузки двоичного файла и завершения первоначального автоматического анализа. Он добавляет 20 команд в меню **Edit > Plugins**.

Свойство	Значение
Имя	MyNav
Автор	Джоксеан Корет
Распространение	Исходники Python
Цена	Бесплатно
Назначение	Трассировка отладчика и анализ покрытия кода

MyNav предоставляет графический навигатор на уровне функций, созданный под влиянием BinNavi компании Zynamics. В отличие от обычного графа базовых блоков, он показывает связи между целыми функциями. Дополнительные средства позволяют, например, вывести пути между любыми двумя функциями. Несколько возможностей расширяют встроенный отладчик IDA.

MyNav записывает сведения об отладочных сеансах и позволяет использовать один сеанс как фильтр для следующего. После каждого запуска строится граф только тех функций, которые действительно выполнялись. Так можно быстро сузить множество функций, отвечающих за определённое действие.

Например, сначала программу запускают, не устанавливая сетевое соединение, а во втором сеансе соединение создают. Если исключить все функции, выполненные в первый раз, итоговый граф оставит главным образом функции, связанные с установлением соединения. Такой приём особенно полезен для очень крупных двоичных файлов.

IdaPdf

Вредоносные документы встречаются всё чаще. В частности, файлы PDF могут использовать уязвимости программ просмотра. Для их исследования необходимо понимать внутреннее устройство формата и находить встроенный код, который выполняется при успешной эксплуатации.

Большинство отдельных средств анализа PDF рассчитано на командную строку и извлекает данные, которые затем приходится загружать в IDA. IdaPdf объединяет загрузчик и модуль анализа непосредственно в IDA.

Свойство	Значение
Имя	IdaPdf
Автор	Крис Игл
Распространение	Исходники C++

Свойство	Значение
Цена	Бесплатно
Назначение	Загрузчик и модуль для разбора файлов PDF и навигации по ним

Загрузчик распознаёт PDF, создаёт новую базу IDA и разделяет документ на отдельные составляющие. Во время загрузки он старается извлечь и обработать все потоковые объекты PDF. После завершения загрузчики выгружаются из памяти, поэтому дальнейший анализ выполняет отдельный модуль IdaPdf.

Обнаружив загруженный PDF, модуль перечисляет все его объекты и открывает вкладку со сводной таблицей. Пример её содержимого:

```
Num  Location  Type        Data Offs  Data size  Filters        Filtered stream  Filtered size  Ascii
17   000e20fe  Stream      000e2107   313        /FlateDecode   000f4080         210            No
35   00000010  Dictionary  00000019   66                                                        Yes
36   000002a3  Dictionary  000002ac   122                                                       Yes
37   0000032e  Stream      00000337   470        [/FlateDecode] 000f4170         1367           Yes
```

Таблица содержит номера и адреса объектов, положение и размер их данных, применяемые к потокам фильтры и адрес извлечённых данных после фильтрации. Контекстное меню позволяет перейти к исходному объекту или обработанному потоку и сохранить их содержимое. Столбец `Ascii` сообщает, считает ли модуль исходный или обработанный вариант объекта состоящим только из данных ASCII.

После запуска IdaPdf добавляет в меню **Edit > Other** ещё две команды. Пользователь выделяет блок данных и просит декодировать его из Base64 либо обработать функцией `unescape` JavaScript. Результат копируется в новый сегмент IDA. Нередко именно там обнаруживается вредоносная полезная нагрузка, которую затем можно сразу дизассемблировать.

Итоги

Если вам не хватает какой-либо возможности IDA, стоит проверить, не столкнулся ли кто-нибудь с той же задачей и не реализовал ли уже решение. Именно так возникло множество расширений. Большинство общедоступных модулей невелики и решают одну конкретную задачу. А открытые исходники полезны не только как готовый инструмент, но и как примеры интересного применения SDK IDA.

Сноски

[1] JDBC - интерфейс Java для подключения к базам данных.

Глава 24. Отладчик IDA

IDA прежде всего известна как один из лучших дизассемблеров для статического анализа. Однако современные способы противодействия статическому исследованию часто вынуждают сочетать его с динамическим анализом. Hex-Rays сделала IDA универсальным средством обратной разработки, добавив отладчик в версии 4.5 и последовательно развивая его.

IDA умеет локально и удалённо отлаживать программы для нескольких платформ и процессоров. Она также может служить внешним интерфейсом Microsoft WinDbg, в том числе при отладке ядра Windows. В этой и следующих главах рассматриваются основные возможности отладчика, анализ запутанного кода и удалённая отладка программ Windows, Linux и macOS.

Запуск отладчика

Обычно отладчик применяют для исследования образа памяти аварийно завершившегося процесса либо для управляемого выполнения программы. Рабочий сеанс начинается с выбора процесса: можно присоединиться к уже выполняющемуся или запустить новый под управлением отладчика.

При присоединении начальный запуск и инициализация уже завершены, поэтому наблюдать их невозможно. Если база IDA не открыта, команда **Debugger > Attach** предлагает подходящие отладчики, набор которых зависит от платформы.

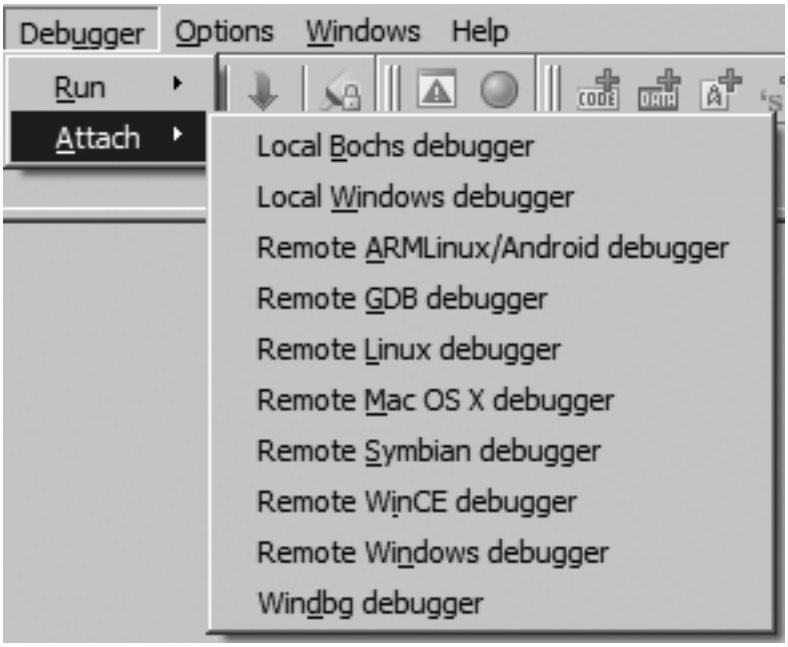


Рисунок 24-1. Присоединение к произвольному процессу

После выбора локального отладчика IDA показывает доступные процессы.

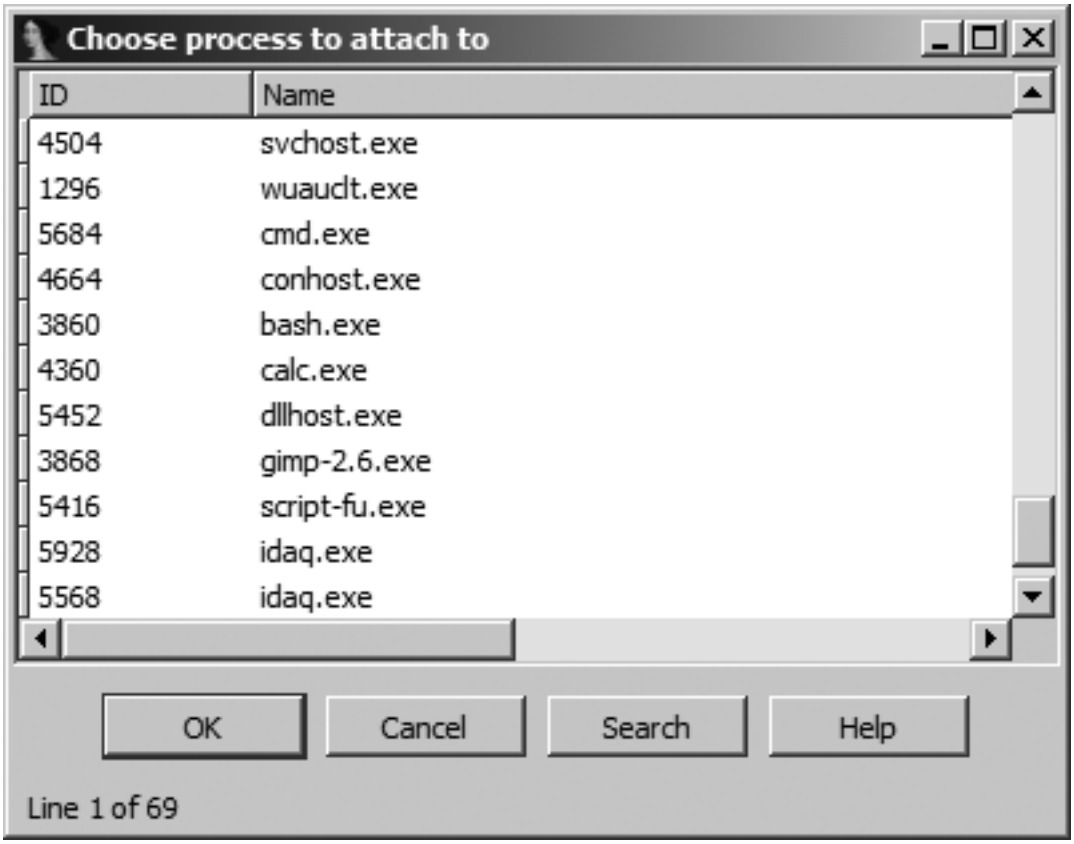


Рисунок 24-2. Выбор процесса для присоединения

Отладчик приостанавливает выбранный процесс и создаёт временную базу из снимка памяти. В неё входят все загруженные библиотеки, поэтому она обычно крупнее и загромождённее привычной базы. Загрузчик IDA не обрабатывал исходный исполняемый файл, автоматический анализ не выполнялся, и поначалу дизассемблируются только текущая инструкция и достижимый из неё код. До возобновления выполнения можно расставить точки останова.

Больше сведений удаётся получить, если сначала открыть исполняемый файл в IDA, а затем присоединиться к процессу. При открытой базе меню отладчика выглядит иначе.

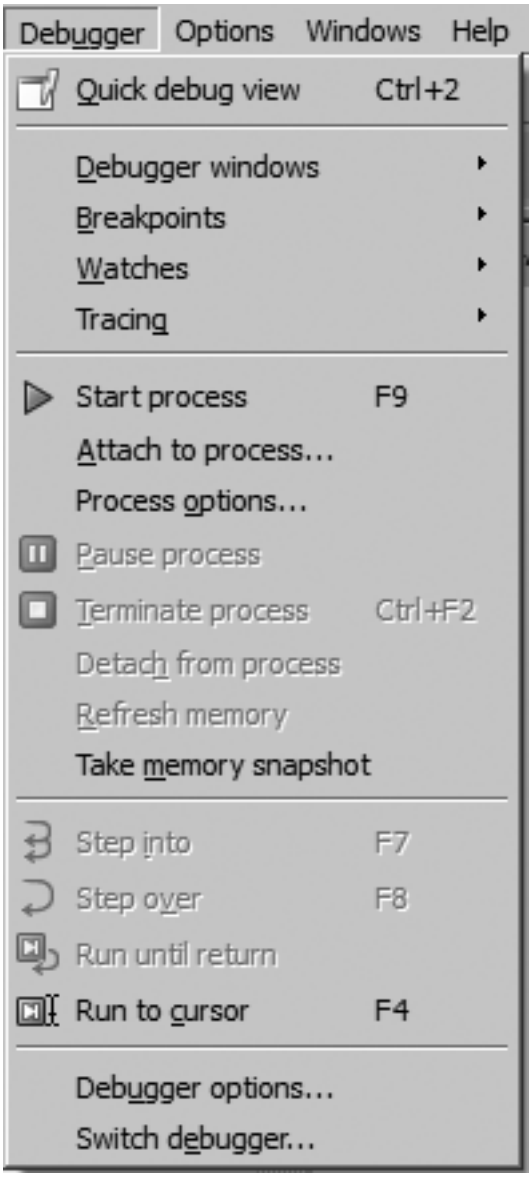


Рисунок 24-3. Меню отладчика при открытой базе

Если нужных команд нет, для текущего типа файла ещё не выбран отладчик. Команда **Debugger > Select Debugger** открывает список подходящих вариантов.



Рисунок 24-4. Выбор отладчика

Выбранный вариант можно сделать стандартным для этого типа файлов. В дальнейшем его разрешается сменить командой **Debugger > Switch Debugger**.

При выполнении **Debugger > Attach to Process** для исполняемого файла IDA сначала показывает процессы с совпадающим именем. Если таких нет, выводится полный список. Совпадение имени не доказывает, что процесс запущен именно из файла, открытого в базе.

Для библиотеки DLL в Windows список ограничивается процессами, куда она загружена. Например, при анализе `wininet.dll` будут показаны только процессы с этой библиотекой. В Linux и macOS такой фильтрации нет: видны все процессы, к которым разрешено присоединение.

Вместо присоединения можно запустить новый процесс. Без открытой базы используется **Debugger > Run**, с открытой - **Debugger > Start Process** или **Debugger > Run to Cursor**. Первая команда выполняет программу до заранее установленной точки останова либо до ручной приостановки. Вторая временно ставит точку в позиции курсора. Если выполнение туда не попадёт и не встретит другую точку, оно продолжится до паузы или завершения.

Только запуск под отладчиком позволяет наблюдать все действия программы с самого начала. Это особенно важно для запутанных программ: процесс удобно остановить сразу после распаковки или снятия маскировки, но до обычной работы. Ещё одно преимущество открытой базы - предварительный автоматический анализ, заметно улучшающий качество дизассемблирования.

Локально IDA выполняет лишь программы, совместимые с текущей системой и процессором: встроенного слоя эмуляции нет. Для удалённой отладки поставляются серверы Windows, Windows CE, macOS, Linux и Android для разных архитектур. IDA также взаимодействует с `gdbserver`, удалённой заглушкой GDB и средством App TRK для Symbian. В этой редакции внешний интерфейс отладчика поддерживал процессы x86, x64, MIPS, ARM и PowerPC. Удалённая работа рассматривается в главе 26.

Для запуска нужен не только файл базы IDA, но и исходный исполняемый файл на целевой машине. Он получает все права пользователя, запустившего IDA. При исследовании вредоносной

программы это способно заразить систему. Перед запуском IDA предупреждает, что код будет выполнен; отрицательный ответ отключает меню отладчика до закрытия базы.

Вредоносные образцы следует отлаживать только в изолированной среде. В отличие от настоящего отладчика, эмулятор x86 из главы 21 не требует исходного файла и не выполняет его инструкции непосредственно процессором исследовательской машины.

Основные окна отладчика

Когда процесс приостановлен, IDA переходит из обычного режима дизассемблирования в режим отладки и открывает стандартный набор окон.

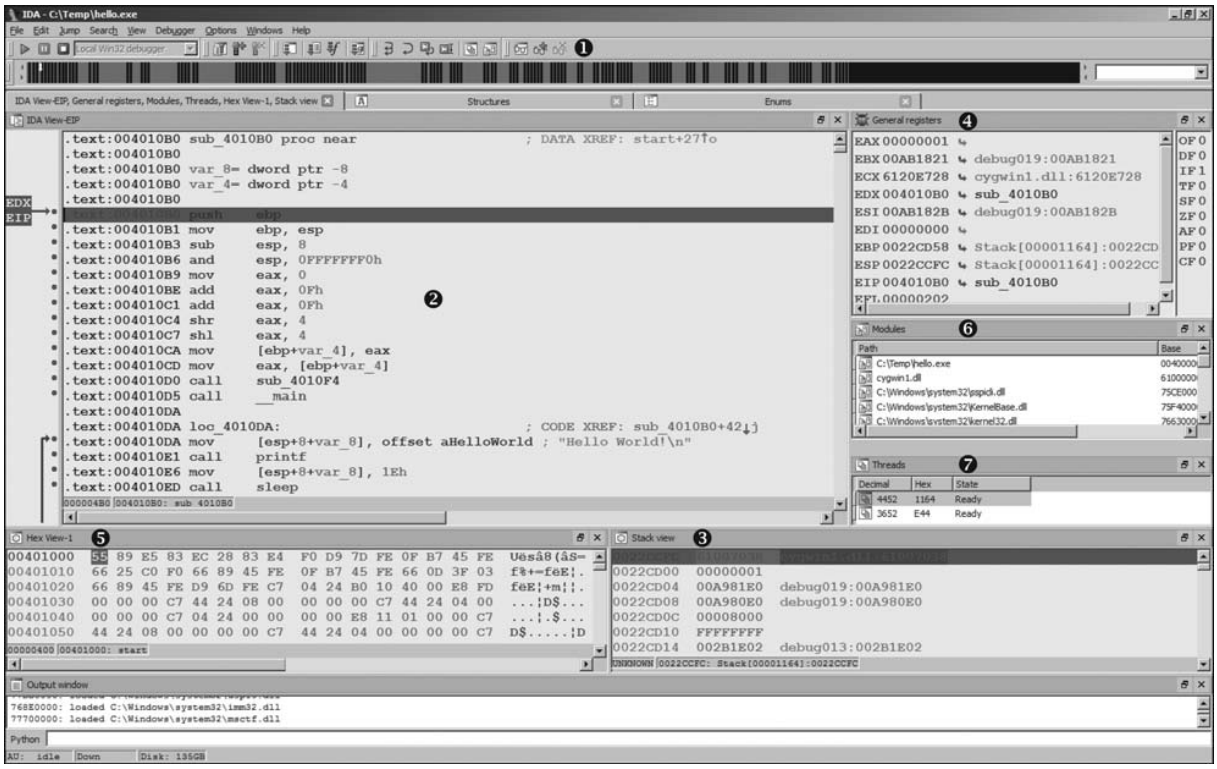


Рисунок 24-5. Основное представление отладчика IDA

Панель отладчика заменяет панель дизассемблирования и содержит кнопки управления процессом и точками останова. Размер шрифта меняется через **Options > Font**, а удачную раскладку окон можно сохранить командой **Windows > Save Desktop**.

Окно `IDA View-EIP` синхронизировано с указателем инструкций. Если регистр указывает на адрес в видимой области, его имя появляется на левом поле. Точки останова по умолчанию выделены красным, следующая инструкция - синим. Используется тот же механизм дизассемблирования, что и при статическом анализе. Поэтому код основного файла, заранее проанализированный IDA, отображается особенно хорошо; для загруженных библиотек сведений меньше.

Окно стека показывает содержимое стека времени выполнения. Регистры, указывающие в стек, отмечаются в окне общих регистров. IDA старается пояснить каждый элемент: адрес возврата связывает с местом вызова, а указатель данных - с соответствующим объектом.

Остальные стандартные окна показывают шестнадцатеричный дамп памяти, загруженные модули и потоки. Двойной щелчок по потоку переносит дизассемблер к его текущей инструкции и обновляет значения регистров.

Окно общих регистров содержит значения регистров процессора и флаги. Отдельные окна сегментных, вещественных и MMX-регистров открываются из меню **Debugger**.

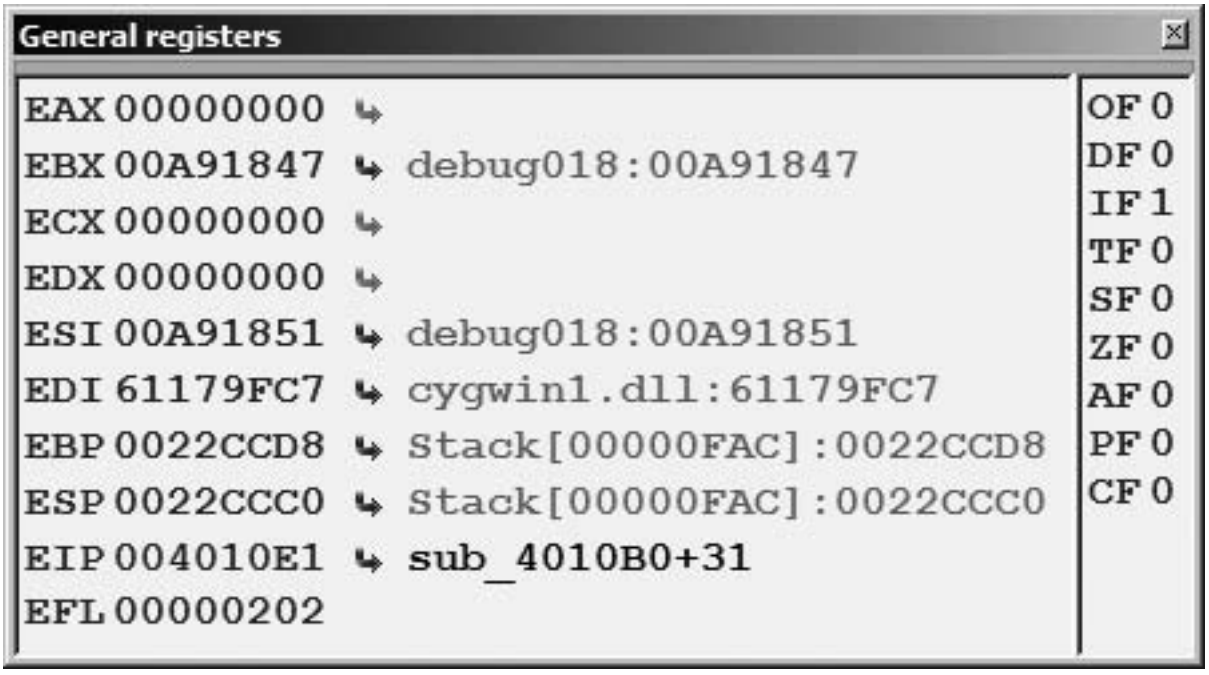


Рисунок 24-6. Окно общих регистров

Справа от каждого значения IDA выводит его смысл, если может его определить. Через контекстную команду **Modify** значение или флаг меняют вручную; доступны быстрое обнуление, переключение, увеличение и уменьшение. Переключение особенно удобно для флагов процессора.

Команда **Open Register Window** открывает дизассемблирование по адресу из регистра. Так же можно вернуть случайно закрытые окна `IDA View-EIP` и `IDA View-ESP`. Если значение похоже на допустимый адрес памяти, рядом активируется стрелка перехода.

Окно модулей перечисляет исполняемые файлы и библиотеки в адресном пространстве процесса. Двойной щелчок открывает экспортируемые символы выбранного модуля, что упрощает установку точек останова на библиотечные функции.

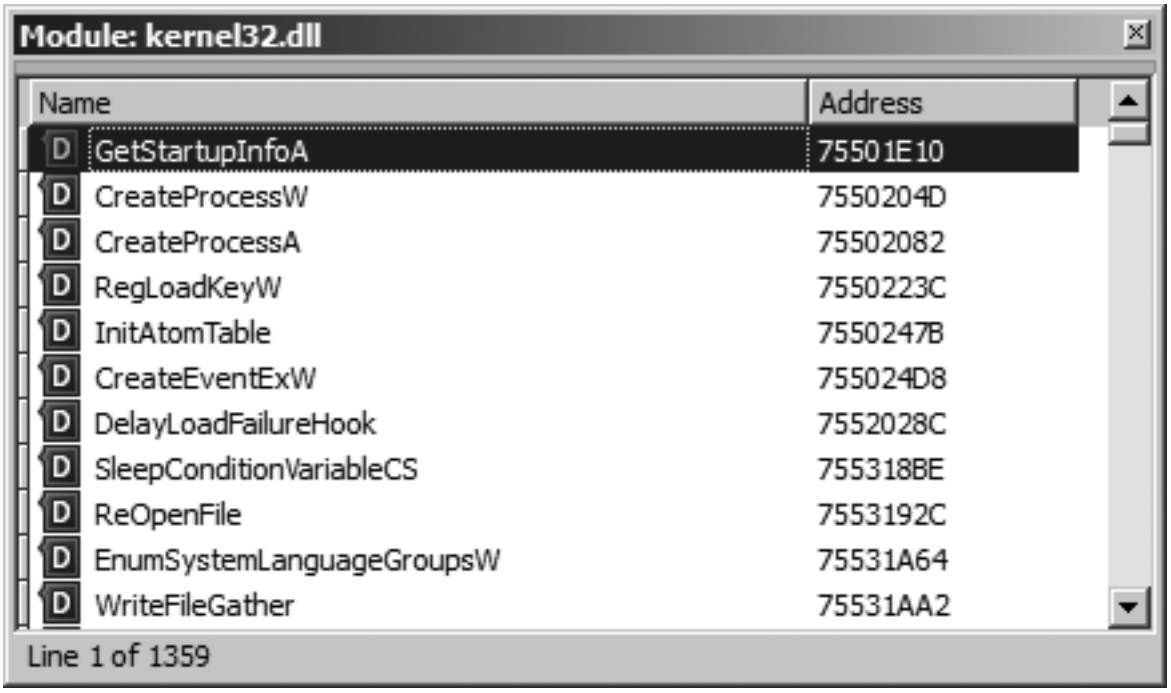


Рисунок 24-7. Содержимое модуля в окне Modules

Другие окна отладчика открываются из его меню. Обычные представления IDA, включая функции и сегменты, остаются доступны через **Views > Open Subviews**.

Управление процессом

Главная задача отладчика - точно управлять поведением процесса и при необходимости изменять его. Команды пошагового выполнения, продолжения и запуска обычно сочетаются с точками останова, которые прерывают программу на заданной инструкции или при выполнении условия.



Рисунок 24-8. Средства управления процессом

Основные команды работают так:

• **Continue** возобновляет процесс до точки останова, ручной паузы или завершения.
• **Pause** приостанавливает выполняющийся процесс.
• **Terminate** принудительно завершает процесс.
• **Step Into** выполняет одну инструкцию; если это вызов, останавливается на первой инструкции вызываемой функции.
• **Step Over** тоже выполняет одну инструкцию, но воспринимает вызов как единое целое и останавливается после возврата. Внутри вызова всё же может сработать точка останова.
• **Run Until Return** продолжает текущую функцию до возврата или точки останова. Команда полезна, когда исследование функции уже неинтересно либо вход в неё был случайным.
• **Run to Cursor** продолжает выполнение до позиции курсора или точки останова. Если выбранный адрес не достигается, автоматической остановки не будет.

Команды доступны с панели, по горячим клавишам и из меню **Debugger**. После каждой остановки все окна обновляют регистры, флаги и память.

Точки останова

Точка останова прерывает выполнение в заданном месте независимо от положения курсора. Их может быть много. В IDA точку ставят клавишей F2 или командой **Add Breakpoint** в контекстном меню; строка по умолчанию окрашивается красным. Повторное нажатие F2 удаляет точку. Полный список открывается через **Debugger > Breakpoints > Breakpoint List**.

По умолчанию используются программные точки останова: байт кода заменяется специальной инструкцией. Для x86 это `int 3` с кодом `0xCC`. Операционная система передаёт управление отладчику при её выполнении. Запутанный код может обнаруживать или намеренно использовать такое изменение, как обсуждалось в главе 21.

Аппаратные точки не меняют код. Процессор сравнивает выполнение и обращения к памяти со значениями специальных регистров. В x86 регистры `DR0`-`DR3` задают не более четырёх точек, а `DR4`-`DR7` содержат дополнительные параметры.

После создания точку можно отредактировать и сделать условной или аппаратной.

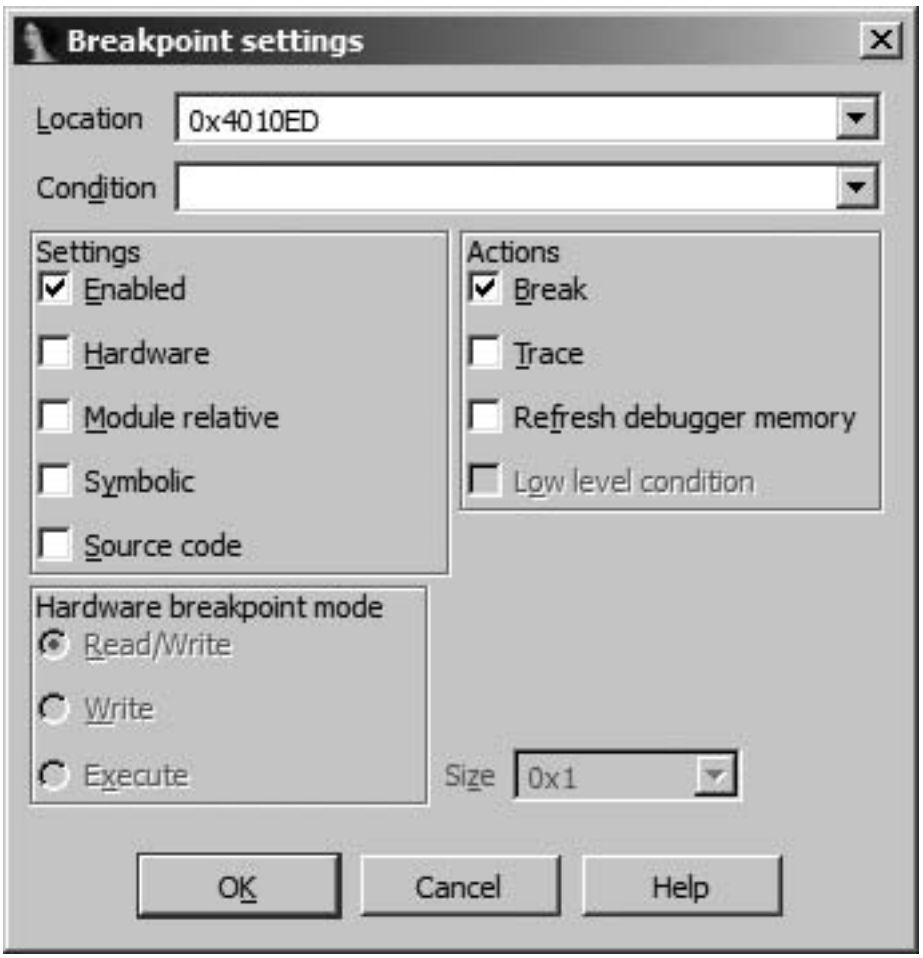


Рисунок 24-9. Параметры точки останова

Поле **Location** задаёт адрес, а флажок **Enabled** включает точку. Отключённая точка игнорируется вместе со своим условием. Флажок **Hardware** выбирает аппаратную реализацию.

> **Предупреждение.** IDA 6.1 позволяла отметить аппаратными более четырёх точек x86, хотя процессор обслуживал только четыре. Остальные молча игнорировались.

Для аппаратной точки выбирают срабатывание при выполнении, записи либо чтении и записи. Последние два режима позволяют узнать, когда изменяется или читается конкретный участок

данных, независимо от выполняемой инструкции.

Размер точки выполнения всегда равен одному байту. Для наблюдения за данными допустимы 1, 2 или 4 байта. Двухбайтовый адрес должен быть кратен двум, четырёхбайтовый - четырём. Адрес вместе с размером образует контролируемый диапазон. Например, четырёхбайтовая точка записи по адресу `0804C834h` сработает при однобайтовой записи в `0804C837h`, двухбайтовой в `0804C836h` и четырёхбайтовой в `0804C832h`, поскольку каждое обращение затрагивает диапазон `0804C834h-0804C837h`.

Условие задаётся выражением IDC, а не Python. Условные точки реализует отладчик, поэтому они могут быть и программными, и аппаратными. Ненулевой результат означает истину и приводит к остановке; нулевой - ложь, после которой выполнение автоматически продолжается.

Во время отладки IDC предоставляет переменные регистров, например `EAX`, `EBX`, `ECX`, `EDX`, `ESI`, `EDI`, `EBP`, `ESP`, `EIP`, `EFL`, а также их 16- и 8-разрядные части. Для отдельных флагов прямых переменных нет; значение получают функцией `GetRegValue`.

```
EAX == 100             // остановиться, если EAX равен 100
ESI > EDI              // остановиться, если ESI больше EDI
Dword(EBP-20) == 10    // прочитать var_20 текущего кадра и сравнить с 10
GetRegValue("ZF")      // остановиться, если установлен флаг нуля
EAX = 1                // присвоить EAX единицу; результат также истинен
EIP = 0x0804186C       // изменить EIP, например чтобы обойти участок кода
```

Условие может вызывать возвращающую значение функцию IDC. Присваивания позволяют менять регистры при достижении определённых мест - например, подменять результат функции.

В группе **Actions** флажок **Break** определяет, нужно ли действительно приостанавливать процесс. Точка без остановки полезна, когда требуется при каждом достижении адреса менять память или регистр. Флажок **Trace** записывает событие трассировки.

Трассировка

Трассировка записывает события выполняющегося процесса в кольцевой буфер заданного размера и, при желании, в файл. Доступны трассировка инструкций и функций.

При выборе **Debugger > Tracing > Instruction Tracing** IDA сохраняет адрес, инструкцию и новые значения изменённых ею регистров, кроме указателя инструкций. Это сильно замедляет процесс, поскольку отладчику приходится выполнять его по одной инструкции. Трассировка функций записывает только вызовы и, по выбору, возвраты; значения регистров в такие записи не входят.

Отдельно можно трассировать запись, чтение с записью и выполнение по конкретному адресу. Это точки останова с включённой трассировкой и выключенной остановкой. Наблюдение за данными использует аппаратные точки, поэтому одновременно доступно не более четырёх таких событий. Трассировка выполнения по умолчанию программная и этим пределом не ограничена.

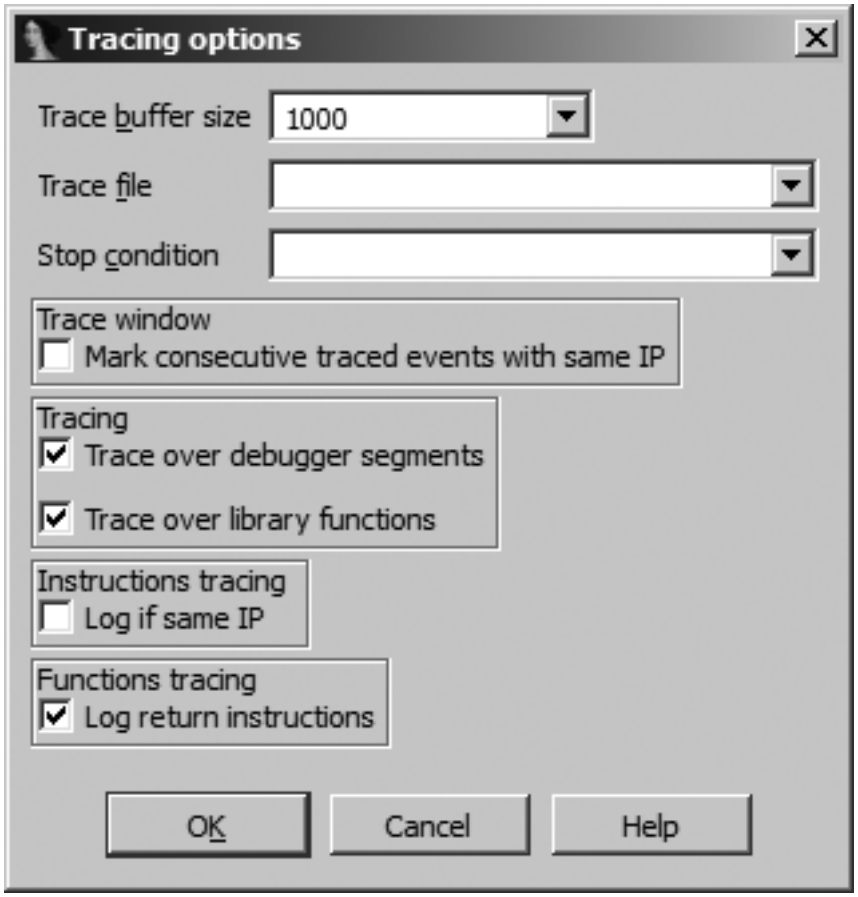


Рисунок 24-10. Параметры трассировки

Параметры окна относятся только к трассировке инструкций и функций. Размер буфера ограничивает число показанных последних событий. Если указан полный путь к файлу, все записи дописываются туда. Условие остановки в виде выражения IDC проверяется перед каждой инструкцией и при истинном результате приостанавливает процесс независимо от адреса.

Параметр **Mark consecutive traced events with same IP** отмечает знаком равенства соседние события с одинаковым указателем инструкций. Такое бывает, например, у строковых инструкций x86 с префиксом `rep`. Чтобы записывать каждое повторение, нужно также включить **Log if same IP**; иначе вся повторяемая инструкция появляется один раз.

Обычная запись выглядит так:

```
Thread   Address              Instruction  Result
00000150 .text:sub_401320+17  rep movsb    ECX=00000000 ESI=0022FE2C EDI=0022FCF4
00000150 .text:sub_401320+19  pop esi      ESI=00000000 ESP=0022FCE4
```

С включённой записью одинакового указателя каждая итерация сохраняется отдельно:

```
Thread   Address              Instruction  Result
000012AC .text:sub_401320+17  rep movsb    ECX=0000000B ESI=0022FE21 EDI=0022FCE9
000012AC .text:sub_401320+17  rep movsb    ECX=0000000A ESI=0022FE22 EDI=0022FCEA
000012AC .text:sub_401320+17  rep movsb    ECX=00000009 ESI=0022FE23 EDI=0022FCEB
000012AC .text:sub_401320+17  rep movsb    ECX=00000008 ESI=0022FE24 EDI=0022FCEC
```

Если одновременно включить отметку одинакового указателя, повторные строки сокращаются до знаков равенства, подчёркивая неизменность адреса инструкции.

Параметр **Trace over debugger segments** временно выключает трассировку после перехода из сегментов исходного файла, чаще всего при входе в разделяемую библиотеку. Параметр **Trace over library functions** пропускает функции, которые IDA распознала как библиотечные, например по сигнатуре FLIRT. Встроенная в двоичный файл библиотечная функция здесь отличается от

функции внешней DLL. Оба параметра включены изначально: это ускоряет работу и не позволяет библиотечному коду быстро вытеснить полезные события из буфера.

Стек вызовов

Стек вызовов показывает последовательность функций, приведшую к текущему месту. Он открывается командой **Debugger > Stack Trace**.

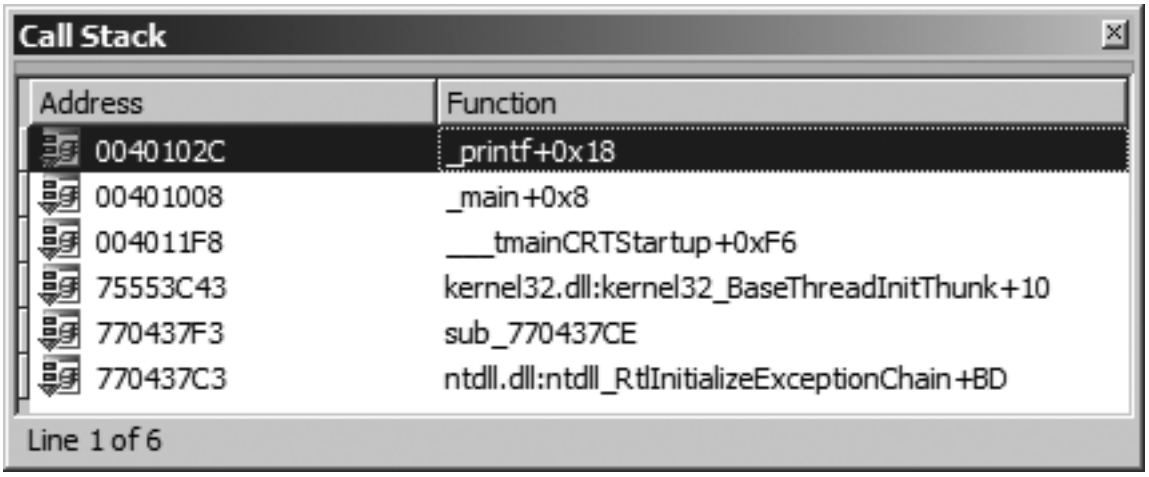


Рисунок 24-11. Пример стека вызовов

Верхняя строка соответствует текущей функции, следующая - вызвавшей её функции и адресу вызова, а остальные описывают более ранние вызовы. Отладчик проходит кадры стека, обычно начиная с регистра указателя кадра EBP, извлекает сохранённый указатель предыдущего кадра и адрес возврата. Отладчик IDA не может пройти кадр x86, в котором EBP не используется как указатель. На уровне функций такой список отвечает на вопрос: какая цепочка вызовов привела сюда?

Наблюдаемые значения

Во время отладки бывает нужно постоянно видеть одну или несколько переменных. Список наблюдения показывает выбранные области памяти при каждой остановке, избавляя от повторной навигации. В отличие от точки останова, наблюдение само по себе не приостанавливает выполнение.

Чаще всего наблюдают стек, кучу и разделы данных. Для добавления используется команда **Add Watch** в контекстном меню. Адрес глобальной переменной известен заранее, а локальная существует только во время вызова содержащей её функции. После входа в функцию IDA умеет вычислить адрес локальной переменной.

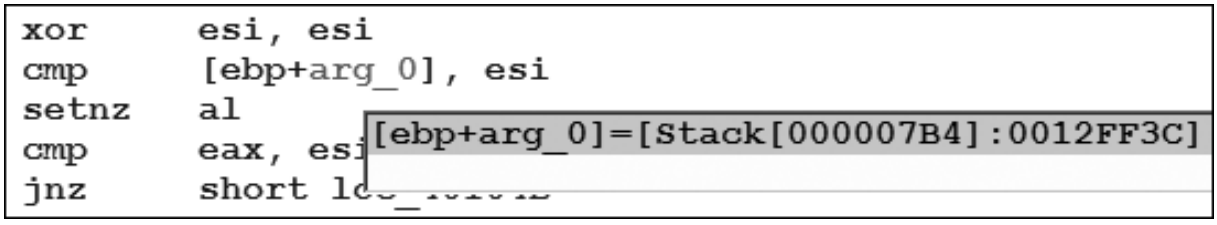


Рисунок 24-12. Определение адреса локальной переменной

Двойной щелчок по локальной переменной активной функции переносит основное окно к её адресу. Затем адрес можно вручную добавить в список. Если предварительно присвоить области имя и применить команду к имени, IDA заполнит адрес сама. Список открывается через **Debugger > Watches > Watch List**, выбранная запись удаляется клавишей Delete.

Автоматизация отладки

Сценарии и модули полезны не только при статическом анализе. Они могут исследовать данные времени выполнения, реализовывать сложные условия точек останова и противодействовать способам обнаружения отладчика.

Сценарии управления отладчиком

В режиме отладки сохраняются все способы запуска сценариев из главы 15: меню **File**, горячие клавиши и командная строка. Пользовательские функции IDC разрешается вызывать из условий точек останова и условий завершения трассировки.

Для памяти предназначены `DbgByte`, `PatchDbgByte`, `DbgWord`, `PatchDbgWord`, `DbgDword` и `PatchDbgDword`, соответствующие обычным функциям чтения и исправления базы. Основные функции регистров и точек останова:

• `GetRegValue(reg)` возвращает значение указанного регистра. В выражениях IDC регистр также доступен как переменная.
• `SetRegValue(val, name)` меняет регистр; в IDC допустимо и обычное присваивание.
• `AddBpt(addr)` создаёт программную точку останова.
• `AddBptEx(addr, size, type)` создаёт точку заданного размера и вида; вид выбирается из констант `BPT_xxx`.
• `DelBpt(addr)` удаляет точку.
• `GetBptQty()` возвращает количество точек, а `GetBptEA(n)` - адрес точки с данным номером.
• `GetBptAttr(addr, attr)` получает числовой или строковый атрибут `BPTATTR_xxx`.
• `SetBptAttr(addr, attr, value)` меняет атрибут, кроме выражения условия.
• `SetBptCnd(addr, cond)` задаёт условие IDC.
• `CheckBpt(addr)` сообщает, отсутствует ли точка, отключена ли она, включена или активна вместе с отладчиком.

Следующий сценарий ставит в позиции курсора точку с пользовательским обработчиком:

```
#include <idc.idc>

/* Вернуть 1 для остановки или 0 для продолжения. */
static my_breakpoint_condition() {
    return AskYN(1,
        "Условие my_breakpoint_condition сработало. Остановиться?") == 1;
}

/* Зарегистрировать функцию как условие точки останова. */
static main() {
    auto addr;
    addr = ScreenEA();
    AddBpt(addr);
    SetBptCnd(addr, "my_breakpoint_condition()");
}
```

При каждом достижении адреса диалог спрашивает, следует ли остановиться. Возвращаемое функцией значение определяет решение отладчика. Внутренняя логика обработчика может быть

сколь угодно сложной.

SDK уведомляет модули о событиях отладчика через обратные вызовы. Сценариям такой событийный подход недоступен, поэтому для них Hex-Rays добавила синхронные функции: сценарий начинает действие и ждёт соответствующего события. Пока ожидание не закончится, остальные операции IDA заблокированы.

Основные функции синхронного управления:

• `GetDebuggerEvent(wait_evt, timeout)` ждёт событие, выбранное флагами `WFNE_xxx`; значение `-1` означает неограниченное ожидание.
• `RunTo(addr)` выполняет процесс до адреса или точки останова.
• `StepInto()` выполняет одну инструкцию с входом в вызов.
• `StepOver()` выполняет одну инструкцию без входа в вызов, если раньше не сработает точка.
• `StepUntilRet()` выполняет текущую функцию до возврата либо точки.
• `EnableTracing(level, enable)` включает или выключает вид трассировки `TRACE_xxx`.
• Семейство `GetEventXXX()` получает сведения о текущем событии; допустимые функции зависят от вида события.

После каждой функции, запускающей процесс, необходимо вызвать `GetDebuggerEvent`, иначе следующее действие может не выполниться. Этот фрагмент сделает только первый шаг:

```
StepOver();
StepOver();    // этот и последующие вызовы завершатся неудачей
StepOver();
```

Правильный вариант забирает событие после каждого действия:

```
StepOver();
GetDebuggerEvent(WFNE_SUSP, -1);
StepOver();
GetDebuggerEvent(WFNE_SUSP, -1);
StepOver();
GetDebuggerEvent(WFNE_SUSP, -1);
```

Флаг `WFNE_SUSP` означает ожидание события, которое приостанавливает процесс, например исключения или точки останова. Отдельной функции простого продолжения нет, но флаг `WFNE_CONT` сначала возобновляет процесс, а затем позволяет дождаться остановки:

```
GetDebuggerEvent(WFNE_SUSP | WFNE_CONT, -1);
```

Дополнительные функции умеют запускать отладчик и присоединяться к процессам. Их полный перечень приведён в справке IDA.

Следующий сценарий подсчитывает, сколько раз выполнялась инструкция по каждому адресу:

```
static main() {
    auto ca, code, addr, count, idx;
    ca = GetArrayId("stats");
    if (ca != -1) {
        DeleteArray(ca);
    }
    ca = CreateArray("stats");
    EnableTracing(TRACE_STEP, 1);
    for (code = GetDebuggerEvent(WFNE_ANY | WFNE_CONT, -1);
         code > 0;
         code = GetDebuggerEvent(WFNE_ANY | WFNE_CONT, -1)) {
        addr = GetEventEa();
        count = GetArrayElement(AR_LONG, ca, addr) + 1;
        SetArrayLong(ca, addr, count);
    }
    EnableTracing(TRACE_STEP, 0);
    for (idx = GetFirstIndex(AR_LONG, ca);
         idx != BADADDR;
         idx = GetNextIndex(AR_LONG, ca, idx)) {
        count = GetArrayElement(AR_LONG, ca, idx);
```

```
        Message("%x: %d\n", idx, count);
    }
    DeleteArray(ca);
}
```

Сначала старый глобальный массив `stats`, если он существует, удаляется и создаётся заново. Затем включается пошаговая трассировка. Для каждого события адрес инструкции становится индексом разреженного массива, а значение по этому индексу увеличивается. После завершения процесса второй цикл печатает все заполненные элементы и удаляет массив.

```
401028: 1
40102b: 1
40102e: 2
401031: 2
401034: 2
401036: 1
40103b: 1
```

Небольшое изменение позволяет считать не адреса, а виды выполненных инструкций:

```
for (code = GetDebuggerEvent(WFNE_ANY | WFNE_CONT, -1);
     code > 0;
     code = GetDebuggerEvent(WFNE_ANY | WFNE_CONT, -1)) {
    addr = GetEventEa();
    mnem = GetMnem(addr);
    count = GetHashLong(ht, mnem) + 1;
    SetHashLong(ht, mnem, count);
}
```

Мнемоника служит строковым ключом хеш-таблицы глобального массива. Получается статистика наподобие этой:

```
add:   18
and:   2
call:  46
cmp:   16
dec:   1
imul:  2
jge:   2
jmp:   5
jnz:   7
js:    1
jz:    5
lea:   4
mov:   56
pop:   25
push:  59
retn:  19
sar:   2
setnz: 3
test:  3
xor:   7
```

В главе 25 управление отладчиком из сценария применяется для снятия запутывания двоичных файлов.

Автоматизация с помощью модулей IDA

API в SDK охватывает все возможности IDC и дополнительно предоставляет полноценное асинхронное управление отладчиком. Объявления находятся в `<SDKDIR>/dbg.hpp`.

Для асинхронной работы модуль подписывается на уведомления `HT_DBG`. Их коды перечислены в `dbg_notification_t` из `dbg.hpp`. Команды часто существуют парами: синхронная `COMMAND()` и асинхронная `request_COMMAND()`. Асинхронные варианты ставят действия в очередь; после заполнения очереди нужно вызвать `run_requests`. Уведомления приходят обработчикам,

зарегистрированным через hook_to_notification_point.

Асинхронный вариант счётчика адресов сначала подключает и отключает уведомления в функциях инициализации и завершения:

```
// Узел для накопления статистики.
netnode stats("$ stats", 0, true);

int idaapi init(void) {
    hook_to_notification_point(HT_DBG, dbg_hook, NULL);
    return PLUGIN_KEEP;
}

void idaapi term(void) {
    unhook_from_notification_point(HT_DBG, dbg_hook, NULL);
}
```

При запуске модуль очищает статистику, ставит в очередь включение пошаговой трассировки и выполнение до возврата из текущей функции, затем запускает очередь:

```
void idaapi run(int arg) {
    stats.altdel();
    request_enable_step_trace();
    request_step_until_ret();
    run_requests();
}
```

Остаётся обработать два ожидаемых уведомления:

```
int idaapi dbg_hook(void *user_data, int notification_code, va_list va) {
    switch (notification_code) {
        case dbg_trace:
            va_arg(va, thid_t);
            ea_t ea = va_arg(va, ea_t);
            stats.altset(ea, stats.altval(ea) + 1);
            return 0;

        case dbg_step_until_ret:
            for (nodeidx_t i = stats.alt1st();
                 i != BADNODE;
                 i = stats.altnxt(i)) {
                msg("%x: %d\n", i, stats.altval(i));
            }
            stats.kill();
            request_disable_step_trace();
            run_requests();
            break;
    }
}
```

dbg_trace приходит после каждой инструкции. Обработчик извлекает адрес из списка аргументов и увеличивает соответствующий элемент netnode. После возврата из исходной функции приходит dbg_step_until_ret: обработчик печатает накопленные значения, удаляет узел и ставит в очередь отключение трассировки. Для обработки последнего запроса снова вызывается run_requests.

Во время обработки асинхронного уведомления нельзя вызывать синхронную версию команды отладчика.

Синхронная работа через SDK похожа на управление из сценария, но имена функций часто различаются, поэтому приходится обращаться к dbg.hpp. Например, аналог GetDebuggerEvent называется wait_for_next_event. Переменные регистров в C++ автоматически не объявляются: значения читаются через get_reg_val и меняются через set_reg_val.

Итоги

Отладчик IDA тесно объединён с дизассемблером. Его интерфейс требует некоторого привыкания, но содержит все основные средства управления процессом. Особенно сильны сценарии, расширения, привычные представления дизассемблирования и предварительный анализ. В результате одна среда подходит для статического, динамического и совместного анализа.

Глава 25. Интеграция дизассемблера и отладчика

Объединение дизассемблера и отладчика в IDA даёт мощное средство изменения двоичных файлов и совместного применения статических и динамических методов. Для уверенной работы нужно понимать возможности и ограничения обеих сторон.

В этой главе рассматривается взаимодействие статической базы с выполняющимся процессом и применение отладчика для обхода некоторых способов противодействия отладке и дизассемблированию. При исследовании вредоносной программы цель обычно состоит не в её запуске как таковом, а в получении достаточно качественного дизассемблированного текста. Отладчик здесь лишь помогает снять запутывание, после чего предпочтительнее вернуться к статическому анализу.

Основной порядок снятия запутывания

Запутанная программа должна восстановить исполняемый код прежде, чем выполнить основную задачу. Упрощённый порядок динамического снятия защиты выглядит так:

1. Открыть программу в отладчике.
2. Найти конец процедуры восстановления и поставить там точку останова.
3. Запустить процесс и дождаться точки.
4. Сохранить текущее состояние памяти в файл.
5. Завершить процесс до выполнения вредоносных действий.
6. Статически исследовать сохранённый образ.

Большинство современных отладчиков поддерживает эти действия. Сложнее всего обычно определить конец восстановления. Для этого могут потребоваться дизассемблер и длительное пошаговое выполнение. Иногда ориентиром служит не конкретная инструкция, а поведение - например, большой скачок указателя инструкций. У упакованных UPX программ переход ниже исходного адреса запуска часто означает, что распаковка завершена. Адрес, с которого незапутанная программа начала бы работу, называют исходной точкой входа, OEP.

Виртуализирующие средства защиты могут преобразовать исходные инструкции в собственный байт-код, выполняемый специально созданной виртуальной машиной. В таком случае оригинальных инструкций x86 или другой архитектуры в файле вообще нет, поэтому восстановить исходный двоичный образ и OEP обычным способом невозможно.

Запуск вредоносной программы опасен: неверная точка останова может быть обойдена, после чего вредоносный код выполнится раньше, чем исследователь успеет отреагировать. Работать следует в изолированной среде, которую можно полностью восстановить.

Отладчик обычно сохраняет отдельные диапазоны памяти, но не обязательно умеет собрать весь образ процесса. Специализированные расширения, например OllyDump для OllyDbg, добавляют такую возможность. Дамп отражает работающий процесс, а не исходное состояние файла на диске. Для анализа обычно достаточно корректно структурированного образа, который загружается в дизассемблер; полностью рабочий исполняемый файл не всегда нужен.

Особенно трудно восстановить таблицу импортов. Упаковщик часто скрывает её, а при распаковке сам связывает процесс с нужными библиотеками. В памяти остаётся таблица разрешённых адресов функций. Для создания запускаемого файла заголовки дампа пришлось бы направить на заново построенную таблицу со всеми зависимостями. Такие средства, как ImpREC, автоматизируют работу. Но для исследования важнее знать, какие функции разрешены и где

сохранены их адреса, чем получить идеальный исполняемый файл.

База IDA и отладчик

Отладчику нужен образ процесса: он либо присоединяется к существующему процессу, либо создаёт новый из исполняемого файла. База IDA таким образом не является полноценным образом процесса, и обычно восстановить исполняемый файл только из неё нельзя.

При запуске отладки IDA передаёт отладчику имя исходного файла, чтобы создать процесс, а также оформление дизассемблирования, имена символов, типы данных и комментарии. Исправления байтов, внесённые только в базу, в новый процесс не переносятся. Нельзя изменить инструкцию в базе и ожидать, что при запуске отладчика выполнится изменённый вариант.

Обратное тоже верно. После завершения сеанса в базе по умолчанию сохраняются лишь логические изменения вроде новых имён. Изменения памяти и самомодифицированный код автоматически назад не копируются. Для переноса распакованного содержимого служит **Debugger > Take Memory Snapshot**.

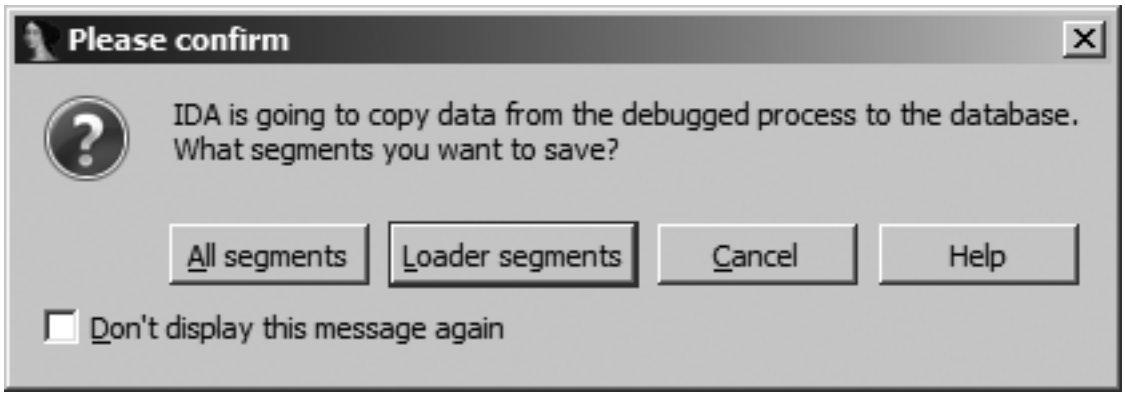


Рисунок 25-1. Подтверждение снимка памяти

По умолчанию из процесса копируются сегменты загрузчика - те, которые исходный загрузчик IDA создал при формировании базы. В запутанной программе именно они часто содержали недоступные анализу зашифрованные данные, а теперь хранят восстановленный код.

Вариант **All segments** копирует также сегменты, созданные отладчиком: загруженные библиотеки, стек, кучу и другие области процесса. Если отладчик присоединился без открытой базы, сегментов загрузчика нет. Тогда можно сохранить всё либо вручную отметить нужные области.

Сегменты открываются через **View > Open Subviews > Segments**. Буква L в соответствующем столбце обозначает сегмент загрузчика. Команда **Edit Segment** открывает атрибуты.

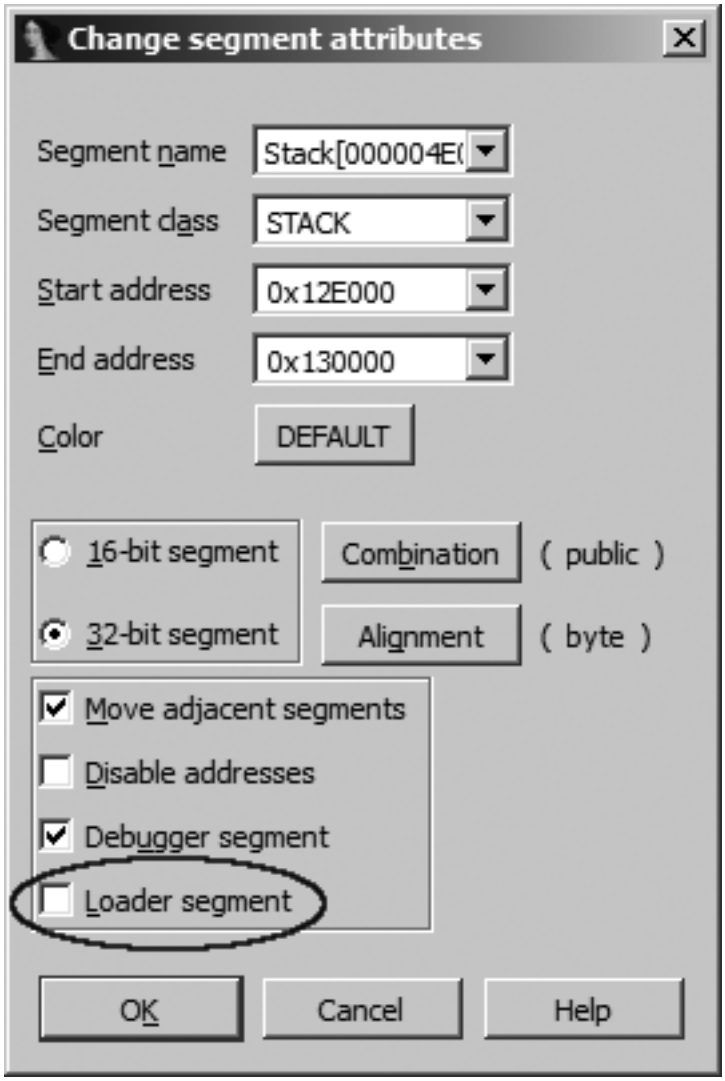


Рисунок 25-2. Флажок сегмента загрузчика

Флажок **Loader segment** позволяет включить область в обычный снимок. Это особенно важно, когда распаковщик помещает исходный код в выделенный блок кучи или отображённой памяти: без такой отметки восстановленный код не попадёт в базу.

Отладка запутанного кода

С практической точки зрения это скорее управляемое выполнение: исследователь наблюдает за распаковкой и делает снимок в нужный момент. Успех зависит от сложности шифрования или сжатия и от способов противодействия анализу.

При работе с вредоносной программой полезно соблюдать следующие правила:

1. Защищать сеть и основную систему, всегда использовать изолированную среду.
2. При первом исследовании по возможности выполнять по одной инструкции. Это медленно, но не даёт программе уйти из-под контроля.
3. Перед любой командой, выполняющей несколько инструкций, проверять ожидаемый путь; иначе процесс может перейти к вредоносной части.
4. По возможности применять аппаратные точки. Алгоритм распаковки способен затереть программную точку или обнаружить её по контрольной сумме.

5. Сначала позволять отладчику перехватывать все исключения, а затем осознанно решать, какие из них передавать программе.
6. Быть готовым часто перезапускать сеанс. Одна ошибка может позволить процессу обнаружить отладчик. Следует записывать адреса, до которых безопасно продолжать выполнение.

В первый раз важнее осторожно получить распакованный вариант, чем ускорить процедуру. Оптимизировать путь и определять наиболее дальнюю безопасную точку останова лучше уже после первого успешного результата.

Запуск процесса

При первом запуске вредоносного файла управление желательно получить как можно раньше. Очевидное место - точка входа процесса, обычно обозначенная `start`. Но формат PE поддерживает функции обратного вызова TLS, которые инициализируют данные потоков и выполняются раньше `start`.

Создатели вредоносных программ используют TLS, рассчитывая, что исследователь не заметит ранний код. IDA разбирает заголовки PE и добавляет такие функции в список точек входа окна экспортов.



Рисунок 25-3. Функция TLS в окне экспортов

Чтобы перехватить процесс вовремя, нужно распознать все функции TLS и поставить точки в начале каждой. Дополнительно IDA умеет автоматически останавливаться при событиях, выбранных в **Debugger > Debugger Options**.

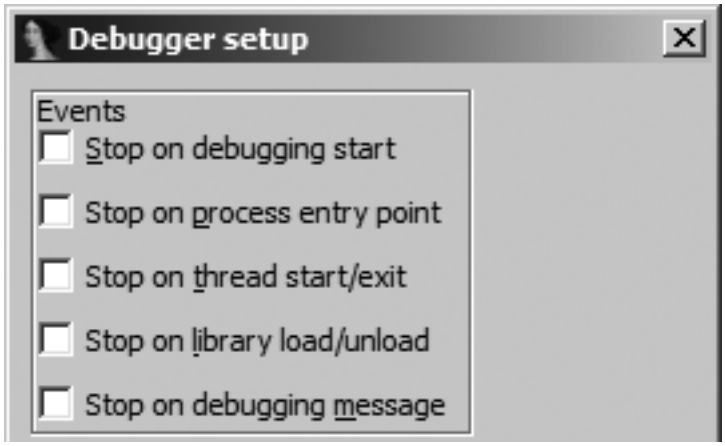


Рисунок 25-4. События остановки отладчика

• **Stop on debugging start** даёт самое раннее управление после создания процесса. Например, в Windows 7 остановка происходит в начале `RtlUserThreadStart` из `ntdll.dll`, до кода программы и функций TLS.
• **Stop on process entry point** останавливает в основной точке входа, обычно `start`. Функции TLS к этому времени уже выполнены.

• **Stop on thread start/exit** останавливает при создании и завершении потока; в Windows текущий адрес обычно находится в `kernel32.dll`.
• **Stop on library load/unload** прерывает выполнение при загрузке и выгрузке библиотеки, также обычно внутри `kernel32.dll`.
• **Stop on debugging message** реагирует на отладочную печать; в Windows это вызов `OutputDebugString` с остановкой внутри `kernel32.dll`.

Нужно понимать, где именно произойдёт каждая остановка, чтобы процесс не выполнил больше ожидаемого.

Простые циклы расшифровки и распаковки

Простым здесь считается цикл без вложенных способов запутывания, у которого известны все выходы. После одного-двух пошаговых проходов можно поставить точки на каждом выходе и продолжить выполнение. Следует убедиться, что цикл не перепишет байты программной точки; при сомнениях нужна аппаратная точка.

Для полной автоматизации требуется условие завершения распаковки: большой скачок указателя инструкций или появление характерной команды. У UPX распаковщик Windows может выглядеть так:

```
UPX1:00410370 start proc near
UPX1:00410370 pusha
UPX1:00410371 mov     esi, offset off_40A000
UPX1:00410376 lea     edi, [esi-9000h]
UPX1:0041037C push    edi
...
UPX1:004104EC pop     eax
UPX1:004104ED popa
UPX1:004104EE lea     eax, [esp-80h]
UPX1:004104F2 loc_4104F2:
UPX1:004104F2 push    0
UPX1:004104F4 cmp     esp, eax
UPX1:004104F6 jnz     short loc_4104F2
UPX1:004104F8 sub     esp, 0FFFFFF80h
UPX1:004104FB jmp     loc_40134C
```

В точке входа `pusha` сохраняет все регистры. Ближе к концу `popa` восстанавливает их, после чего переход передаёт управление распакованной программе. Вместо медленной пошаговой трассировки сценарий может заранее найти `popa` и выполнить процесс до неё:

```
#include <idc.idc>

#define POPA 0x53

static main() {
    auto addr, seg;
    addr = BeginEA();
    seg = SegName(addr);
    while (addr != BADADDR && SegName(addr) == seg) {
        if (Byte(addr) == POPA) {
            RunTo(addr);
            GetDebuggerEvent(WFNE_SUSP, -1);
            Warning("Программа распакована!");
            TakeMemorySnapshot(1);
            return;
        }
        addr = FindCode(addr, SEARCH_NEXT | SEARCH_DOWN);
    }
    Warning("Инструкция popa не найдена!");
}
```

Листинг 25-1. Простой сценарий распаковки UPX

Сценарий запускается из базы до отладчика; подходящий отладчик должен быть выбран заранее. Он предполагает, что распаковщик находится в конце одного сегмента и что UPX не нарушает синхронизацию дизассемблирования. Поэтому это пример приёма, а не универсальное решение.

UPX популярен и бесплатен, но умеет самостоятельно возвращать упакованный файл к исходному виду. Поэтому защитные средства иногда намеренно портят признаки, проверяемые распаковщиком UPX, не нарушая работу самой программы. Простой способ - переименовать стандартные разделы `UPX0`, `UPX1` и `UPX2`. Значит, жёстко задавать эти имена в сценариях не стоит.

Более общий признак: распаковщик часто добавлен в конец файла, а после работы переходит к исходной точке входа, расположенной значительно раньше или в другом сегменте. Следующий сценарий Python выполняет процесс, пока адрес события не станет меньше начальной точки входа:

```
start = BeginEA()
RunTo(start)
GetDebuggerEvent(WFNE_SUSP, -1)
EnableTracing(TRACE_STEP, 1)
code = GetDebuggerEvent(WFNE_ANY | WFNE_CONT, -1)
while code > 0:
    if GetEventEa() < start:
        break
    code = GetDebuggerEvent(WFNE_ANY | WFNE_CONT, -1)
PauseProcess()
GetDebuggerEvent(WFNE_SUSP, -1)
EnableTracing(TRACE_STEP, 0)
MakeCode(GetEventEa())
TakeMemorySnapshot(1)
```

Листинг 25-2. Упрощённый поиск исходной точки входа

Предполагается, что весь код ниже точки входа был скрыт и что до перехода туда ничего вредоносного не происходит. Сценарий запускает отладчик, останавливается на входе, включает пошаговую трассировку и проверяет адрес каждого события. При переходе назад он ставит процесс на паузу, отключает трассировку, оформляет текущие байты как код и делает снимок.

При выполнении запутанного кода IDA может предупредить, что указатель инструкций попал в данные или в середину ранее распознанной инструкции.

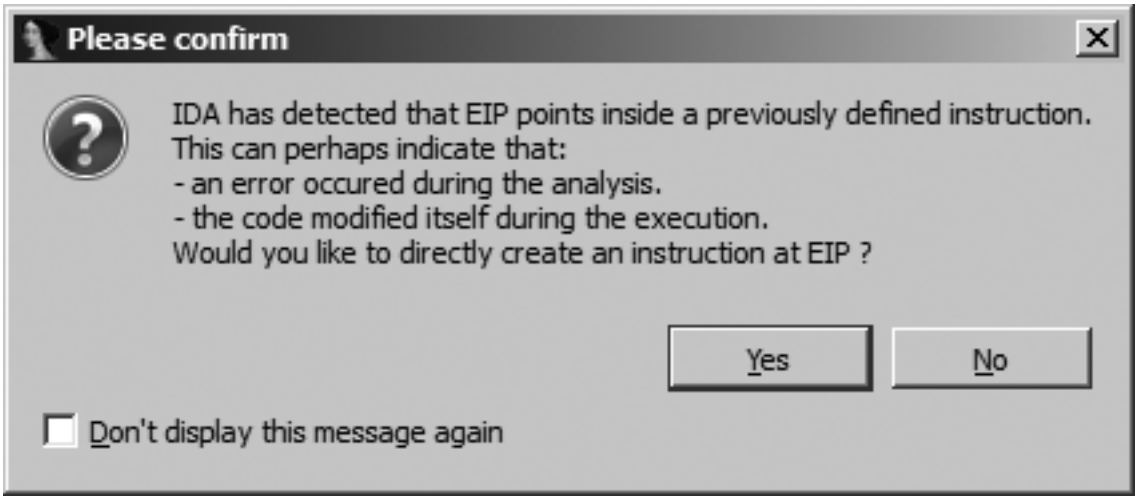


Рисунок 25-5. Предупреждение отладчика об указателе инструкций

Так бывает при нарушении синхронизации дизассемблера либо после перехода в область, которая раньше содержала зашифрованные данные. Поскольку процессор собирается выполнить эти байты, обычно следует согласиться и создать инструкцию по текущему адресу.

Пошаговая трассировка заметно медленнее прямого перехода к `popa`, зато условие завершения не привязано к одному адресу, а границы инструкций определяются фактическим указателем

процессора и не зависят от ошибок статического дизассемблирования.

Восстановление импортов

После снятия запутывания таблица импортов помогает понять поведение программы, даже если сохранённый образ никогда не будет запущен.

При создании базы IDA обычно разбирает импорты. Но в защищённом файле видна лишь таблица распаковщика с минимальным набором функций. Более сложная защита может вообще оставить её пустой и самостоятельно загрузить библиотеки и разрешить адреса.

Импорты исходной программы восстанавливаются в памяти во время распаковки. В Windows для этого обычно вызываются `LoadLibrary` и многократно `GetProcAddress`. Более сложные реализации заменяют `GetProcAddress` собственным поиском, используют хеши вместо строк и иногда даже самостоятельно реализуют загрузку вместо `LoadLibrary`, чтобы не сработали очевидные точки останова.

После восстановления в памяти часто остаётся безымянная таблица адресов:

```
UPX1:0040A000 dword_40A000 dd 7C812F1Dh
UPX1:0040A004 dword_40A004 dd 7C91043Dh
UPX1:0040A008              dd 7C812ADEh
UPX1:0040A00C dword_40A00C dd 7C9105D4h
UPX1:0040A010              dd 7C80ABC1h
UPX1:0040A014 dword_40A014 dd 7C901005h
```

Это соседние четырёхбайтовые значения, на которые ссылается код. Они являются адресами библиотечных функций в исследуемом процессе, но в статической базе сами по себе ничего не объясняют:

```
UPX0:00403C5B call    ds:dword_40A004
UPX0:00403C61 test    eax, eax
UPX0:00403C63 jnz     short loc_403C7B
UPX0:00403C65 call    sub_40230F
UPX0:00403C6A mov     esi, eax
UPX0:00403C6C call    ds:dword_40A058
```

Лучше решить проблему до снимка памяти. Пока отладчик активен, он знает, в каких модулях находятся адреса, и показывает символические имена:

```
UPX1:0040A000 off_40A000 dd offset kernel32_GetCommandLineA
UPX1:0040A004 off_40A004 dd offset ntdll_RtlFreeHeap
UPX1:0040A008 off_40A008 dd offset kernel32_GetVersionExA
UPX1:0040A00C            dd offset ntdll_RtlAllocateHeap
UPX1:0040A010 off_40A010 dd offset kernel32_GetProcessHeap
UPX1:0040A014            dd offset ntdll_RtlEnterCriticalSection
```

Отладчик добавляет к экспортируемой функции имя библиотеки и знак подчёркивания: `GetCommandLineA` из `kernel32.dll` становится `kernel32_GetCommandLineA`. Это предотвращает совпадение символов из разных библиотек.

Нужно назвать каждую ячейку таблицы по её функции, тогда IDA автоматически применит прототипы из библиотек типов. Поставляемый с IDA сценарий `<IDADIR>/idc/renimp.idc` удаляет префикс библиотеки и присваивает оставшуюся часть ячейке. Он должен выполняться при активном отладчике и по выделенному диапазону восстановленной таблицы.

Границы таблицы можно найти, наблюдая за `GetProcAddress` и местом сохранения результата. У UPX это выглядит так:

```
UPX1:00408897 call    dword ptr [esi+8090h]
UPX1:0040889D or      eax, eax
UPX1:0040889F jz      short loc_4088A8
```

```
UPX1:004088A1 mov     [ebx], eax
UPX1:004088A3 add     ebx, 4
```

Листинг 25-3. Получение и сохранение адресов импортируемых функций в UPX

`GetProcAddress` возвращает адрес в `EAX`, инструкция `mov` помещает его по адресу `EBX`, затем указатель продвигается на четыре байта. Значение `EBX` раскрывает местоположение таблицы. После обработки `renimp.idc` она получает имена и, где возможно, прототипы:

```
UPX1:0040A000 ; LPSTR __stdcall GetCommandLineA()
UPX1:0040A000 GetCommandLineA dd offset kernel32_GetCommandLineA
UPX1:0040A004 RtlFreeHeap dd offset ntdll_RtlFreeHeap
UPX1:0040A008 ; BOOL __stdcall GetVersionExA(LPOSVERSIONINFOA lpVersionInformation)
UPX1:0040A008 GetVersionExA dd offset kernel32_GetVersionExA
UPX1:0040A00C RtlAllocateHeap dd offset ntdll_RtlAllocateHeap
UPX1:0040A010 ; HANDLE __stdcall GetProcessHeap()
UPX1:0040A010 GetProcessHeap dd offset kernel32_GetProcessHeap
```

Префикс библиотеки нужно удалить, иначе имя не совпадёт с библиотекой типов. У `renimp.idc` есть слабое место: и библиотека, и функция могут содержать знаки подчёркивания. Для известной `ws2_32` предусмотрена особая обработка, но другие подобные имена могут разбираться неверно.

Если все адреса сохраняет одна инструкция, таблицу можно именовать прямо во время выполнения. Условная точка на `004088A1` вызывает пользовательскую функцию IDC:

```
static createImportLabel() {
    auto n = Name(EAX);
    auto i = strstr(n, "_");
    while (i != -1) {
        n = n[i+1:];
        i = strstr(n, "_");
    }
    MakeUnkn(EBX, DOUNK_EXPAND);
    MakeDword(EBX);
    if (MakeNameEx(EBX, n, SN_NOWARN) == 0) {
        MakeNameEx(EBX, n + "_", SN_NOWARN);
    }
    return 0;
}
```

Функция получает имя по адресу `EAX`, оставляет часть после последнего подчёркивания, оформляет область по `EBX` как четырёхбайтовое значение и присваивает имя. Нулевой результат говорит отладчику не останавливаться.

Чтобы IDA знала функцию условия, её помещают в файл IDC вместе с `main`, который создаёт точку в позиции курсора:

```
static main() {
    AddBpt(ScreenEA());
    SetBptCnd(ScreenEA(), "createImportLabel()");
}
```

После запуска через **File > Script File** обработчик вызывается при каждом сохранении адреса. К завершению распаковки таблица уже размечена, искать и выделять её не требуется.

Другой способ - по адресу функции найти в памяти заголовок модуля, разобрать таблицу экспортов и выполнить обратный поиск имени. Сценарии `RebuildImports.idc` и `RebuildImports.py` с сайта книги используют этот подход и не путаются из-за подчёркиваний в имени библиотеки.

Правильные имена ячеек сразу распространяются на все вызовы:

```
UPX0:00403C5B call    ds:RtlFreeHeap
UPX0:00403C61 test    eax, eax
UPX0:00403C63 jnz     short loc_403C7B
UPX0:00403C65 call    sub_40230F
UPX0:00403C6A mov     esi, eax
UPX0:00403C6C call    ds:RtlGetLastWin32Error
```

Изменения оформления, сделанные в отладчике, автоматически попадают в базу. Снимок нужен только для переноса содержимого памяти - кода и данных.

Сокрытие отладчика

Защита часто проверяет наличие отладчика и прекращает работу при обнаружении. Несколько таких проверок Windows можно обойти сценарием, который запускает сеанс и заранее настраивает точки. Поскольку условия точек задаются только на IDC, примеры этого раздела написаны на нём.

Сначала сценарий ищет функцию TLS, при наличии ставит на неё точку, запускает процесс до основной точки входа и ждёт остановки:

```
auto n;
for (n = 0; n < GetEntryPointQty(); n++) {
    auto ord = GetEntryOrdinal(n);
    if (GetEntryName(ord) == "TlsCallback_0") {
        AddBpt(GetEntryPoint(ord));
        break;
    }
}
RunTo(BeginEA());
GetDebuggerEvent(WFNE_SUSP, -1);
```

После этого процесс и все его библиотеки уже отображены в память.

Первая проверка использует однобайтовое поле `IsDebugged` блока окружения процесса, PEB. При отладке оно равно единице, иначе нулю. Поле находится со смещением 2 и проверяется также функцией `IsDebuggerPresent`.

В основной точке входа Windows регистр `EBX` содержит PEB. Но при остановке в TLS нужен общий способ: найти блок сведений текущего потока, TIB или TEB, и пройти по хранящемуся там указателю PEB.

```
auto seg;
auto peb = 0;
auto tid = GetCurrentThreadId();
auto tib = sprintf("TIB[%08X]", tid);
for (seg = FirstSeg(); seg != BADADDR; seg = NextSeg(seg)) {
    if (SegName(seg) == tib) {
        peb = Dword(seg + 0x30);
        break;
    }
}
if (peb != 0) {
    PatchDbgByte(peb + 2, 0);
}
```

`PatchDbgByte` появилась в IDA 5.5. В более ранних версиях подходит `PatchByte`, но она заодно исправит байт в базе, если адрес входит в неё.

Другая проверка изучает биты поля `NtGlobalFlags` в PEB, которые устанавливаются при отладке из-за особенностей кучи. Их можно сбросить так:

```
globalFlags = Dword(peb + 0x68) & ~0x70;
PatchDword(peb + 0x68, globalFlags);
```

Некоторые системные функции возвращают отлаживаемому процессу иные сведения. Через `NtQueryInformationProcess` из `ntdll.dll` программа может запросить `ProcessDebugPort`: ненулевой результат выдаёт пользовательский отладчик. Для обхода нужно найти функцию, поставить точку и назначить обработчик:

```
func = LocByName("ntdll_NtQueryInformationProcess");
AddBpt(func);
```

```
SetBptCnd(func, "bpt_NtQueryInformationProcess()");
```

Сокращённый прототип функции:

```
NTSTATUS WINAPI NtQueryInformationProcess(
    HANDLE ProcessHandle,
    PROCESSINFOCLASS ProcessInformationClass,
    PVOID ProcessInformation,
    ULONG ProcessInformationLength,
    PULONG ReturnLength
);
```

Идентификатор запроса передаётся во втором аргументе, буфер результата - в третьем. `ProcessDebugPort` имеет значение 7. Обработчик подменяет результат нулём и имитирует успешный возврат, не выполняя тело функции:

```
#define ProcessDebugPort 7

static bpt_NtQueryInformationProcess() {
    auto p_ret;
    if (Dword(ESP + 8) == ProcessDebugPort) {
        p_ret = Dword(ESP + 12);
        if (p_ret) {
            PatchDword(p_ret, 0);
        }
        EIP = Dword(ESP);
        ESP = ESP + 24;
        EAX = 0;
    }
    return 0;
}
```

При входе `ESP` указывает на адрес возврата, за которым лежат пять аргументов. После проверки второго аргумента обработчик записывает ноль по указателю результата. Затем он загружает в `EIP` адрес возврата, удаляет из стека адрес и аргументы соглашения `stdcall`, а в `EAX` помещает код успеха. Нулевое значение самого условия запрещает остановку.

Функция `NtSetInformationThread` позволяет передать значение `ThreadHideFromDebugger` и отсоединить поток от отладчика:

```
NTSTATUS NtSetInformationThread(
    HANDLE ThreadHandle,
    THREADINFOCLASS ThreadInformationClass,
    PVOID ThreadInformation,
    ULONG ThreadInformationLength
);
```

Точку и условие настраивают аналогично:

```
func = LocByName("ntdll_NtSetInformationThread");
AddBpt(func);
SetBptCnd(func, "bpt_NtSetInformationThread()");
```

Обработчик пропускает тело только для опасного класса сведений:

```
#define ThreadHideFromDebugger 0x11

static bpt_NtSetInformationThread() {
    if (Dword(ESP + 8) == ThreadHideFromDebugger) {
        EAX = 0;
        EIP = Dword(ESP);
        ESP = ESP + 20;
    }
    return 0;
}
```

Здесь успешный результат помещается в `EAX`, адрес возврата - в `EIP`, а `ESP` увеличивается на 20 байт: четыре аргумента и адрес возврата.

Ещё одна проверка использует `OutputDebugStringA` из `kernel32.dll`:

```
void WINAPI OutputDebugStringA(LPCTSTR lpOutputString);
```

Официально функция ничего не возвращает, но наблюдаемое значение `EAX` различалось: единица без отладчика и адрес строки при отладке. Хотя такое поведение не гарантировано, защита могла на него рассчитывать. Условие точки сразу подставляет единицу и обходит функцию:

```
func = LocByName("kernel32_OutputDebugStringA");
AddBpt(func);
SetBptCnd(func,
    "!((EAX = 1) && (EIP = Dword(ESP)) && (ESP = ESP + 8))");
```

Логическое отрицание гарантирует ложный результат условия, поэтому точка никогда не приостанавливает процесс. Сценарий `HideDebugger.idc` с сайта книги объединяет все описанные меры.

IDAStealth

Число и сложность проверок отладчика слишком велики для одного простого сценария. Расширение IDAStealth Яна Ньюгера, победившее в конкурсе Hex-Rays 2009 года, реализует более полный набор способов сокрытия.

Свойство	Значение
Имя	IDAStealth
Автор	Ян Ньюгер
Распространение	Исходники C++ и двоичные файлы
Цена	Бесплатно
Назначение	Сокрытие отладчика в Windows

Двоичная поставка состоит из модуля и вспомогательной библиотеки; оба файла устанавливаются в `<IDADIR>/plugins`. При первом запуске открывается окно настройки.

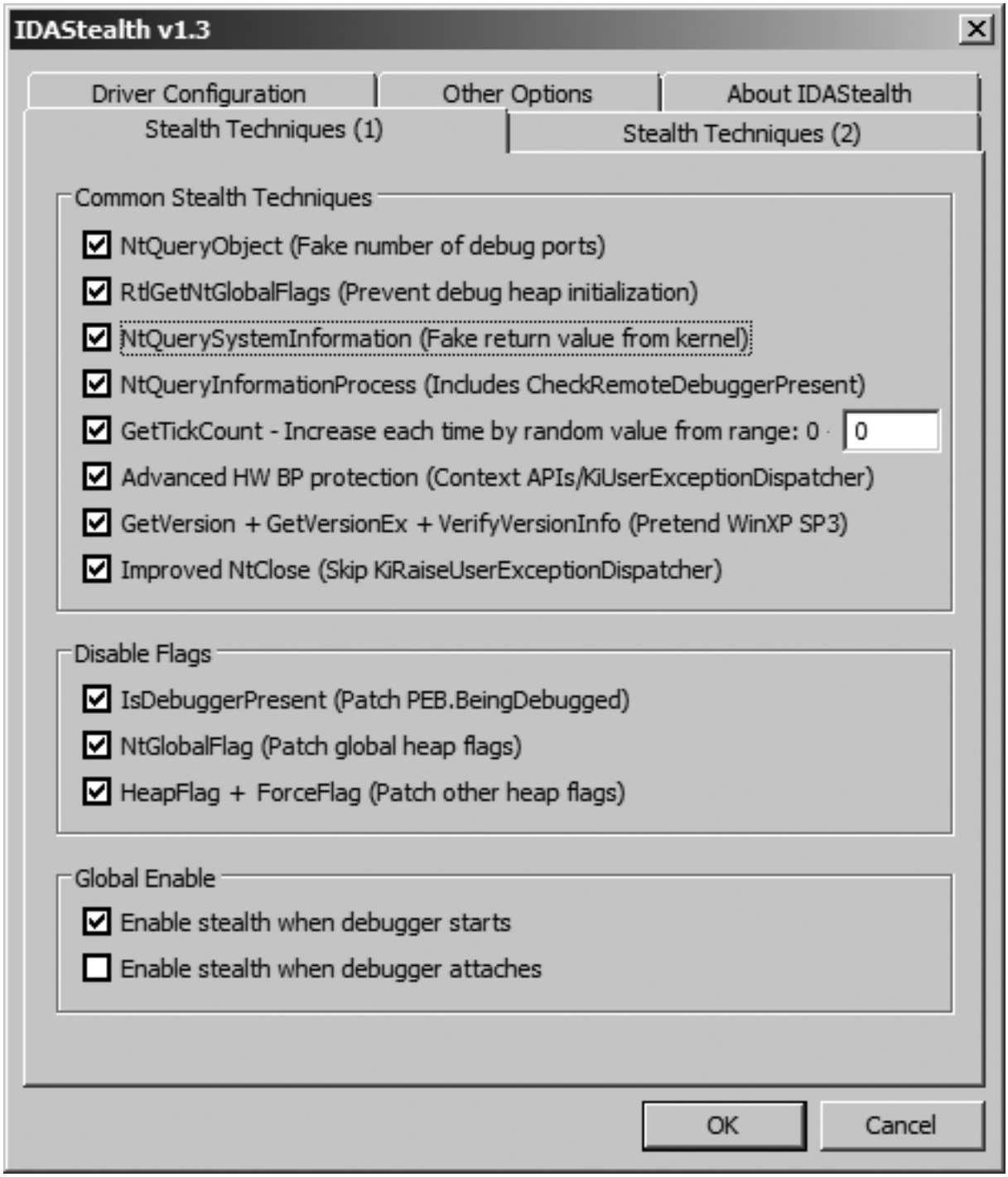


Рисунок 25-6. Настройка IDAStealth

На нескольких вкладках выбираются способы противодействия проверкам. После включения IDAStealth обходит почти все известные на момент издания способы обнаружения отладчика, включая рассмотренные выше.

Обработка исключений

Некоторые программы рассчитывают самостоятельно обрабатывать исключения. Запутанный код может намеренно порождать их для сокрытия потока управления и обнаружения отладчика. Но обычный отладчик стремится перехватывать исключения, поскольку они часто указывают на

ошибку.

Если программа ожидает исключение, отладчик должен либо не перехватывать его, либо по решению исследователя передать процессу. IDA умеет передавать отдельные события вручную и автоматически пропускать выбранные виды.

Общая настройка открывается через **Debugger > Debugger Options**.

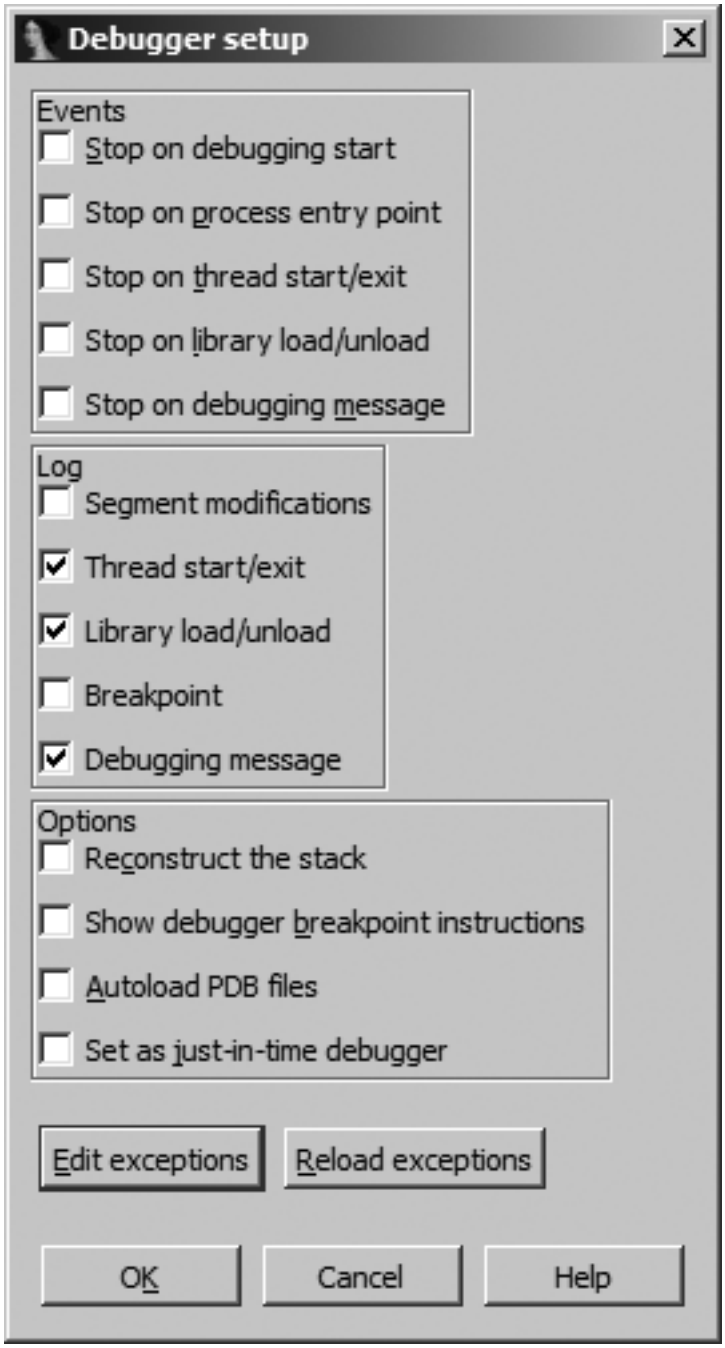


Рисунок 25-7. Окно настройки отладчика

Здесь задаются события остановки, записи в журнал и обработка исключений. Кнопка **Edit exceptions** открывает отдельный список.

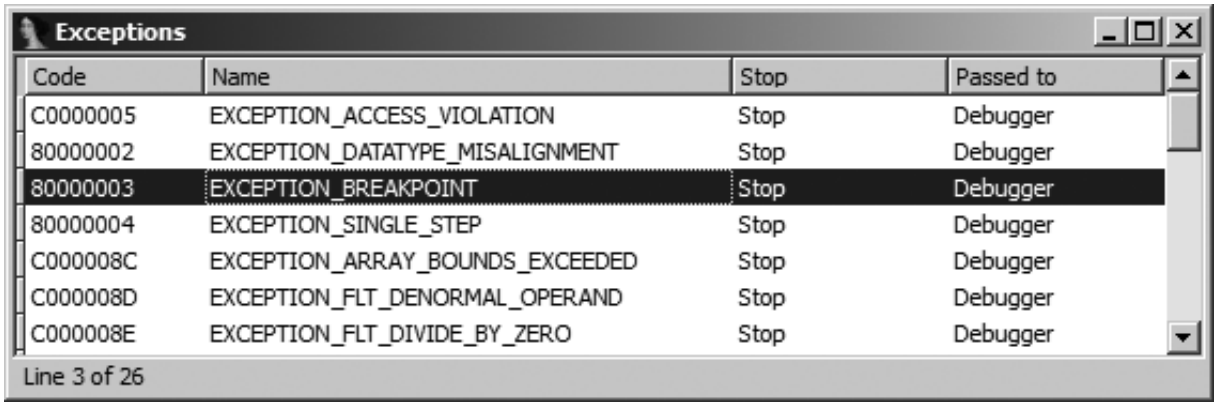


Рисунок 25-8. Настройка исключений

Для каждого известного исключения указаны системный код, имя, необходимость остановки и получатель - отладчик или приложение. Общий список и стандартные правила хранятся в `<IDADIR>/cfg/exceptions.cfg`; там же задаются сообщения. Значения `stop` и `nostop` определяют приостановку процесса.

На время работы с конкретной базой правило можно изменить через команду **Edit** в контекстном меню исключения.

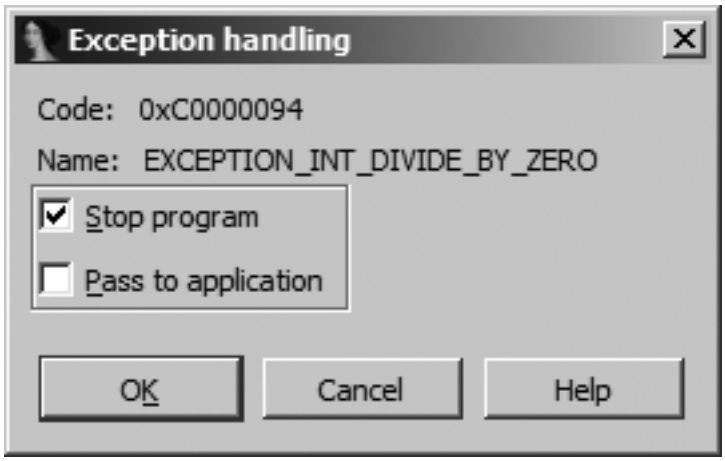


Рисунок 25-9. Изменение правила исключения

Первый флажок задаёт остановку процесса. Если разрешить продолжение, но оставить исключение отладчику, процесс может попасть в бесконечный цикл повторного возникновения той же ошибки.

Второй флажок передаёт исключение приложению, чтобы сработали его обработчики. Это необходимо для программ, чей нормальный поток зависит от исключений, например защищённых tElock, рассмотренным в главе 21.

Если не настроены одновременно автоматическое продолжение и передача данного вида приложению, IDA остановится и сообщит о событии. При попытке продолжить появится дополнительный выбор.

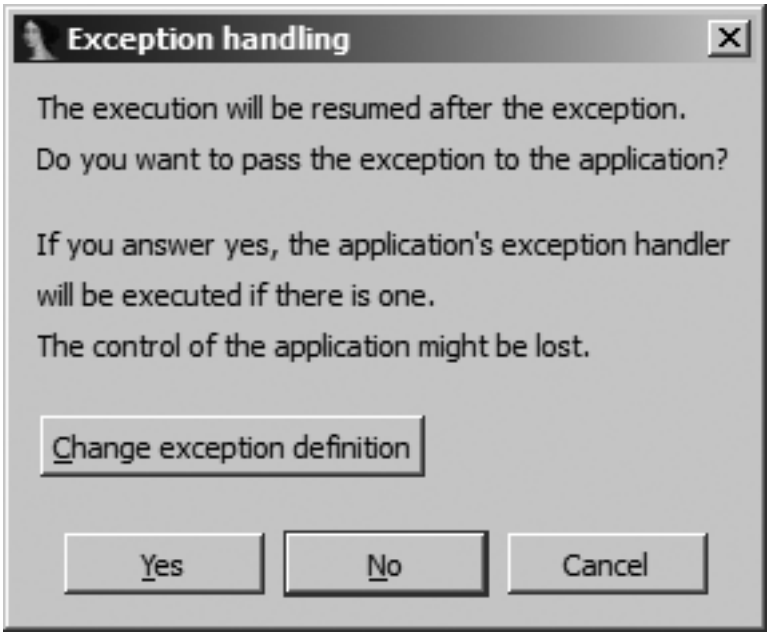


Рисунок 25-10. Выбор получателя исключения

Можно изменить постоянное правило, передать событие приложению кнопкой **Yes** или поглотить его в IDA кнопкой **No**. Последний вариант обычно снова приведёт к ошибке, если причина исключения не исправлена.

Особый случай возникает при пошаговом выполнении инструкции, которая заведомо породит исключение: `int 3`, `icebp` или `popf`, устанавливающей флаг трассировки.

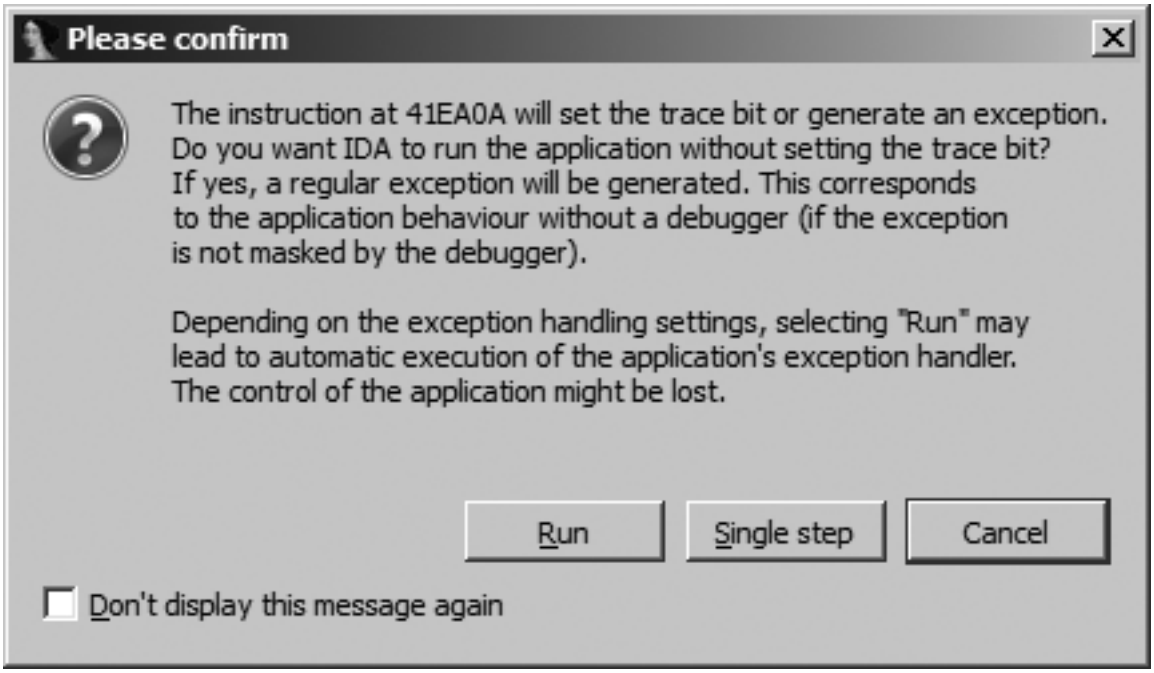


Рисунок 25-11. Подтверждение ожидаемого исключения

Обычно следует выбрать **Run**, чтобы приложение увидело поведение без отладчика. После возникновения события IDA снова предложит решить, передавать ли его обработчику программы.

Поиск обработчиков SEH

Чтобы понять дальнейший путь, нужно найти обработчики исключений. В Windows указатель начала цепочки SEH находится в первом двойном слове блока окружения потока, TEB. Каждый элемент связного списка содержит указатель следующего элемента и адрес функции-обработчика. Исключение проходит по цепочке, пока обработчик не примет его. Если никто не принимает, система завершает обычный процесс либо уведомляет отладчик.

IDA отображает TEB в сегменте `TIB[NNNNNNNN]`, где восемь шестнадцатеричных цифр представляют номер потока:

```
TIB[000009E0]:7FFDF000 TIB_000009E0_ segment byte public 'DATA' use32
TIB[000009E0]:7FFDF000 assume cs:TIB_000009E0_
TIB[000009E0]:7FFDF000 ; org 7FFDF000h
TIB[000009E0]:7FFDF000 dd offset dword_22FFE0
```

Здесь первый элемент цепочки находится по адресу `22FFE0h`. Значение `FFFFFFFFh` означало бы пустую цепочку или её конец. Сам элемент состоит из двух двойных слов:

```
Stack[000009E0]:0022FFE0 dword_22FFE0 dd 0FFFFFFFFh
Stack[000009E0]:0022FFE4              dd offset loc_7C839AA8
```

Первое значение сообщает, что элемент последний, второе указывает обработчик `7C839AA8h`. Для наблюдения за исключениями можно поставить точку на этот адрес.

Встроенного окна цепочки SEH у отладчика IDA того времени не было. Расширение SEH Chain с сайта книги добавляет его.

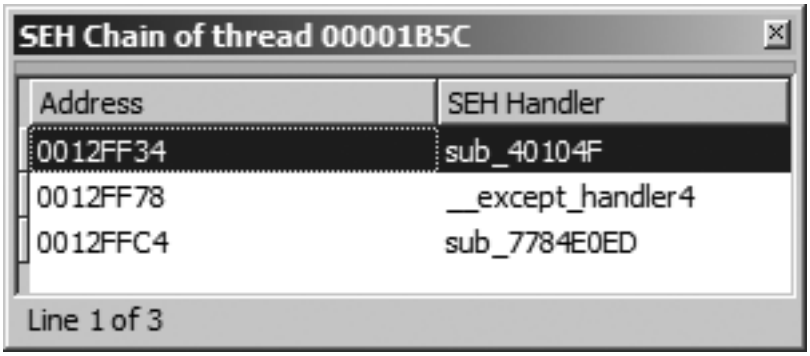


Рисунок 25-12. Цепочка SEH текущего потока

Модуль использует функцию SDK `choose2` и показывает немодальное окно с адресом каждой записи и соответствующего обработчика. Двойной щелчок переносит активное окно дизассемблирования к функции. Задача модуля - только упростить поиск обработчиков.

Возврат из обработчика

Операционная система передаёт обработчику все значения регистров на момент исключения. Он может изменить их, исправив состояние процесса, и разрешить продолжение. Защита использует эту возможность, чтобы подменить сохранённый указатель инструкций и после возврата направить выполнение в неожиданное место.

Обработчики SEH в Windows возвращают управление через `NtContinue`, также известную как `ZwContinue`, из `ntdll.dll`. Один из аргументов указывает на структуру `CONTEXT` со всеми сохранёнными регистрами. Прочитав из неё указатель инструкций, можно поставить точку на фактический адрес продолжения и не проходить пошагово системный код.

Порядок действий:

1. Найти `NtContinue` и поставить на первую инструкцию точку без остановки.
2. Назначить условный обработчик.

3. При срабатывании получить из стека указатель CONTEXT.
4. Прочитать сохранённый указатель инструкций.
5. Поставить точку на этот адрес и продолжить процесс.

Настройку можно автоматизировать:

```
static main() {
    auto func;
    RunTo(BeginEA());
    GetDebuggerEvent(WFNE_SUSP, -1);
    func = LocByName("ntdll_NtContinue");
    AddBpt(func);
    SetBptCnd(func, "bpt_NtContinue()");
}
```

Условная функция читает адрес будущего продолжения и создаёт там точку:

```
static bpt_NtContinue() {
    auto p_ctx = Dword(ESP + 4);
    auto next_eip = Dword(p_ctx + 0xB8);
    AddBpt(next_eip);
    SetBptCnd(next_eip,
        "Warning(\"Достигнут возврат из исключения\") || 1");
    return 0;
}
```

Указатель CONTEXT является первым аргументом, а сохранённый EIP находится по смещению 0xB8. Вторая точка всегда останавливает процесс и выводит пояснение, потому что пользователь не ставил её вручную. Обработчик можно расширить, например восстановить регистры отладки, если защита изменила их для уничтожения аппаратных точек.

Итоги

Отладчик полезен не только для поиска ошибок, но и для обратной разработки. Вредоносный и запутанный код удобно сначала контролируемо выполнить, а затем перенести восстановленные байты и разметку в статическую базу. Интеграция экономит время и устраняет обмен данными между разными средствами. Глава 26 завершает книгу удалённой отладкой и особенностями Linux и macOS.

Глава 26. Дополнительные возможности отладчика

Предыдущие главы охватили основные функции отладчика, автоматизацию и снятие запутывания. В заключительной главе рассматриваются удалённая отладка, применение эмулятора x86 Bochs и возможность Appcall, которая позволяет сценариям IDA вызывать функции исследуемого процесса и его библиотек.

Удалённая отладка в IDA

Все поставки IDA содержат серверы удалённой отладки. Кроме того, клиент умеет работать с удалённым GDB через `gdbserver` или встроенную заглушку. Главное преимущество - единый графический интерфейс IDA независимо от целевой системы. После настройки соединения удалённый сеанс почти не отличается от локального.

Сервер отладки Hex-Rays

На целевой машине запускают сервер, соответствующий системе и архитектуре:

Файл	Назначение
`win32_remote.exe`	32-разрядные приложения Windows
`win64_remotex64.exe`	64-разрядные приложения Windows; требовалась IDA Advanced
`wince_remote_arm.dll`	устройства Windows CE с ARM; передаётся через ActiveSync
`mac_server`	32-разрядные приложения macOS
`mac_serverx64`	64-разрядные приложения macOS; требовалась IDA Advanced
`linux_server`	32-разрядные приложения Linux
`linux_serverx64`	64-разрядные приложения Linux; требовалась IDA Advanced
`armlinux_server`	приложения ARM в Linux
`android_server`	приложения Android

Полную IDA на целевой машине устанавливать не нужно. Например, клиент Windows может исследовать программу Linux, если на целевую систему скопированы лишь сама программа, `linux_server` и необходимые ему разделяемые библиотеки. Зависимости проверяются `ldd`, а в macOS - `otool -L`.

Серверы принимают три параметра командной строки:

• `-p<порт>` меняет порт TCP, по умолчанию 23946; пробел после `-p` не ставится.
• `-P<пароль>` задаёт пароль клиента; пробел после `-P` также не ставится.
• `-v` включает подробные сообщения.

Выбрать конкретный локальный адрес прослушивания нельзя. Ограничивать входящие соединения нужно межсетевым экраном целевой системы. Один экземпляр сервера обслуживает только один активный сеанс; для нескольких одновременных подключений запускают несколько экземпляров на разных портах.

На клиенте перед запуском или присоединением выполняют **Debugger > Process Options** и задают сервер.

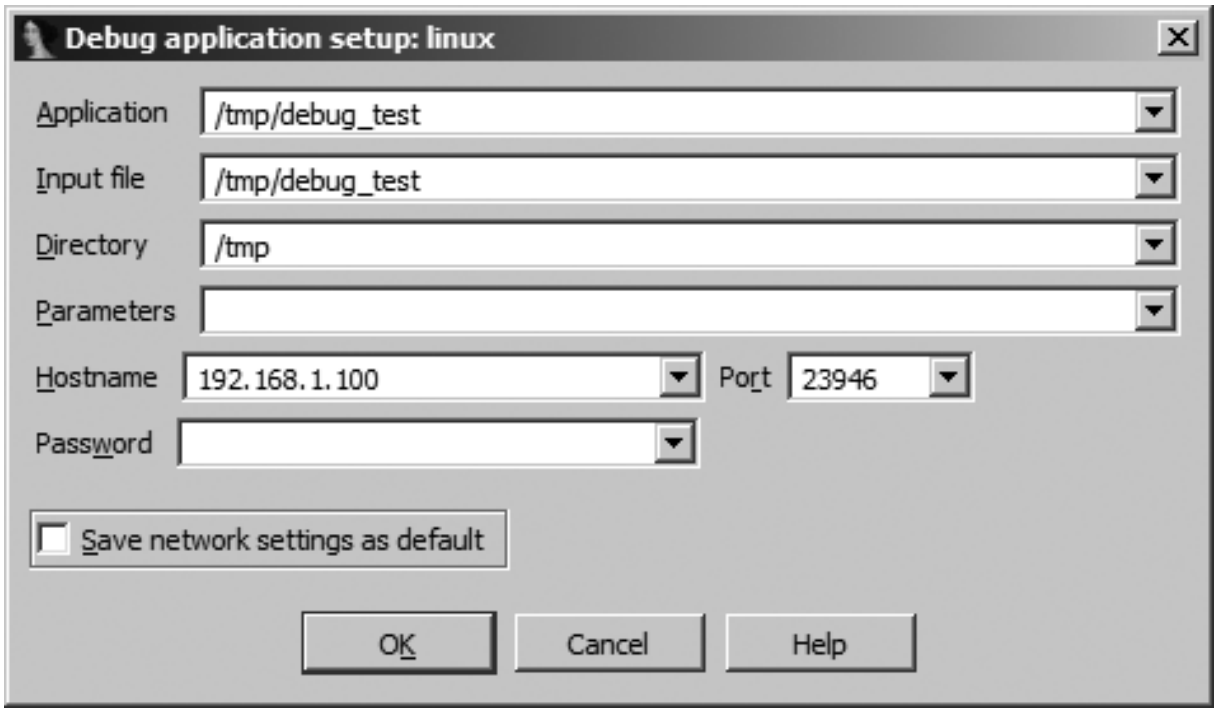


Рисунок 26-1. Параметры процесса отладчика

Поля означают следующее:

• **Application** - полный путь к запускаемой программе. Для удалённого сеанса это путь на сервере. Без полного пути поиск идёт в текущем рабочем каталоге сервера.
• **Input file** - путь к файлу, из которого создана база IDA. В удалённом сеансе он тоже относится к серверу.
• **Directory** - рабочий каталог запуска на локальной или удалённой машине.
• **Parameters** - аргументы командной строки. Метасимволы оболочки <, > и | передаются буквально, поэтому перенаправить ввод или вывод нельзя. Вывод удалённой программы появляется в консоли сервера.
• **Hostname** - имя или адрес удалённого сервера; для локальной отладки оставляется пустым.
• **Port** - порт TCP сервера.
• **Password** - пароль сервера. В старой версии он не скрывался на экране и передавался открытым текстом, поэтому мог быть перехвачен.

Поля запускаемой программы и входного файла обычно совпадают, но не при исследовании библиотеки. Библиотеку нельзя выполнить самостоятельно: **Input file** указывает на неё, а **Application** - на программу, которая её загружает.

Подключение к `gdbserver` настраивается почти так же, но пароль не требуется, а кнопка специальных параметров открывает настройки GDB.

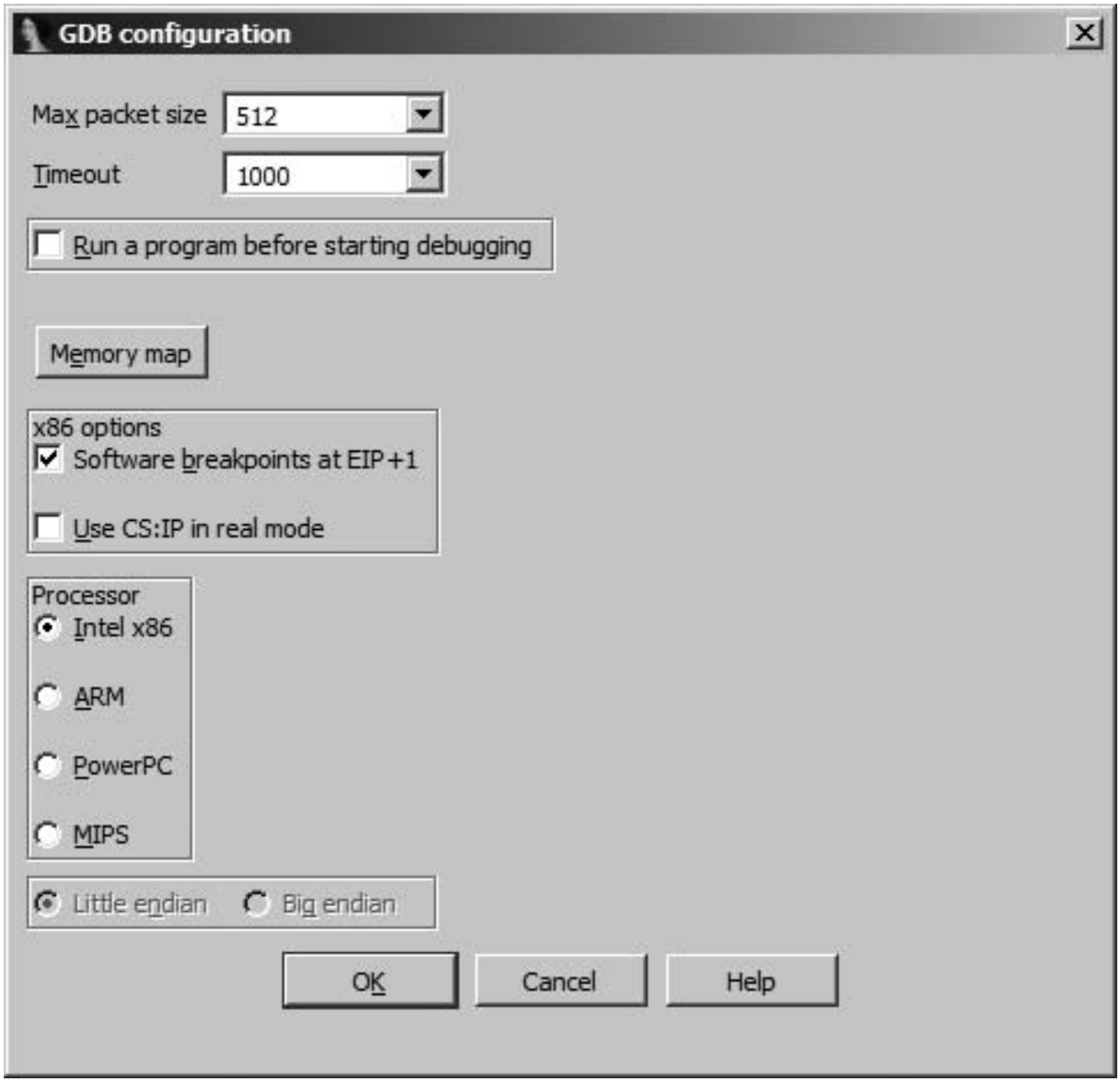


Рисунок 26-2. Настройка GDB

IDA не может сама определить архитектуру удалённой машины. Пользователь обязан выбрать процессор, по умолчанию x86, и при необходимости порядок байтов. В этой версии поддерживались x86, ARM, PowerPC и MIPS.

Присоединение к удалённому процессу

Если база не открыта, выбирают **Debugger > Attach**, затем удалённый отладчик и параметры соединения.

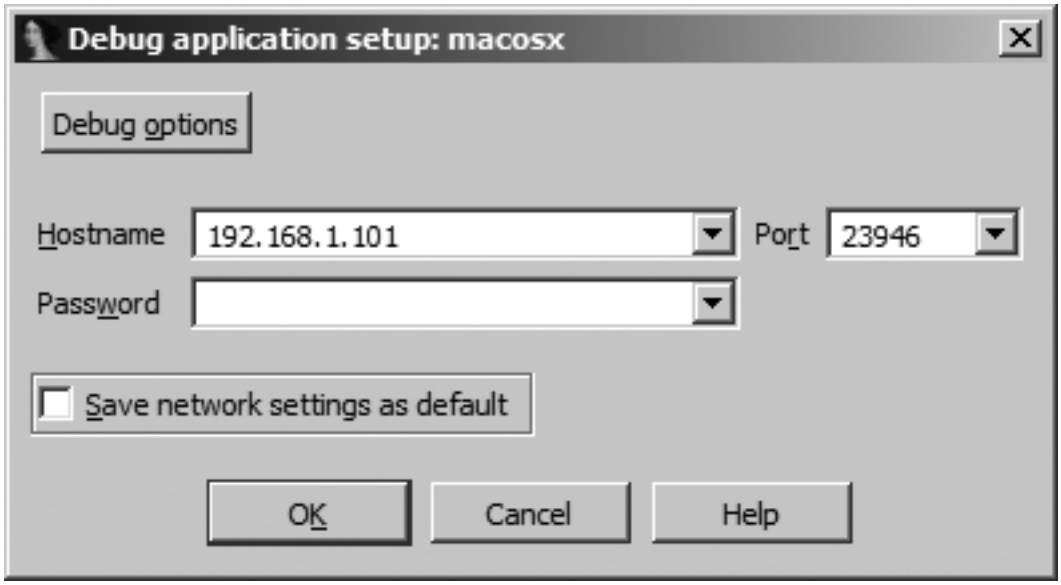


Рисунок 26-3. Параметры удалённого отладчика

После подключения IDA получает от сервера список процессов и предлагает выбрать один. При открытой базе сначала выбирают или переключают отладчик, затем вводят узел и пароль в параметрах процесса и выполняют **Debugger > Attach to Process**.

Исключения при удалённой отладке

Стандартные правила обработки исключений берутся из `exceptions.cfg` на клиентской машине, где работает IDA. Поэтому файл можно изменить и перечитать через окно настройки отладчика без доступа к серверу.

Сценарии и расширения

Сценарии и модули выполняются внутри IDA на клиенте. Она сама передаёт серверу установку точек, чтение состояния, изменение памяти и продолжение процесса, поэтому для автоматизации удалённый сеанс выглядит как локальный.

Однако сценарий должен учитывать архитектуру и систему целевого процесса, а не клиента. Например, сценарий сокрытия отладчика Windows бесполезен для удалённой программы Linux, даже если сама IDA работает в Windows.

Отладка с Bochs

Bochs - открытый эмулятор компьютера x86 с устройствами ввода-вывода и собственной BIOS. Он служит альтернативой виртуальным машинам вроде VMware Workstation. Элиас Бахаалани из Hex-Rays интегрировал Bochs с IDA, создав основанную на эмуляции замену обычной отладке.

Версия IDA для Windows устанавливает совместимый Bochs автоматически. В других системах требовалась версия 2.4.2 или новее. При открытом файле x86 появляется вариант **Local Bochs debugger**. Поскольку программа эмулируется, а не выполняется процессором напрямую, приложения Windows можно локально исследовать даже не в Windows.

Bochs работает в трёх существенно различающихся режимах: образ диска, IDB и PE.

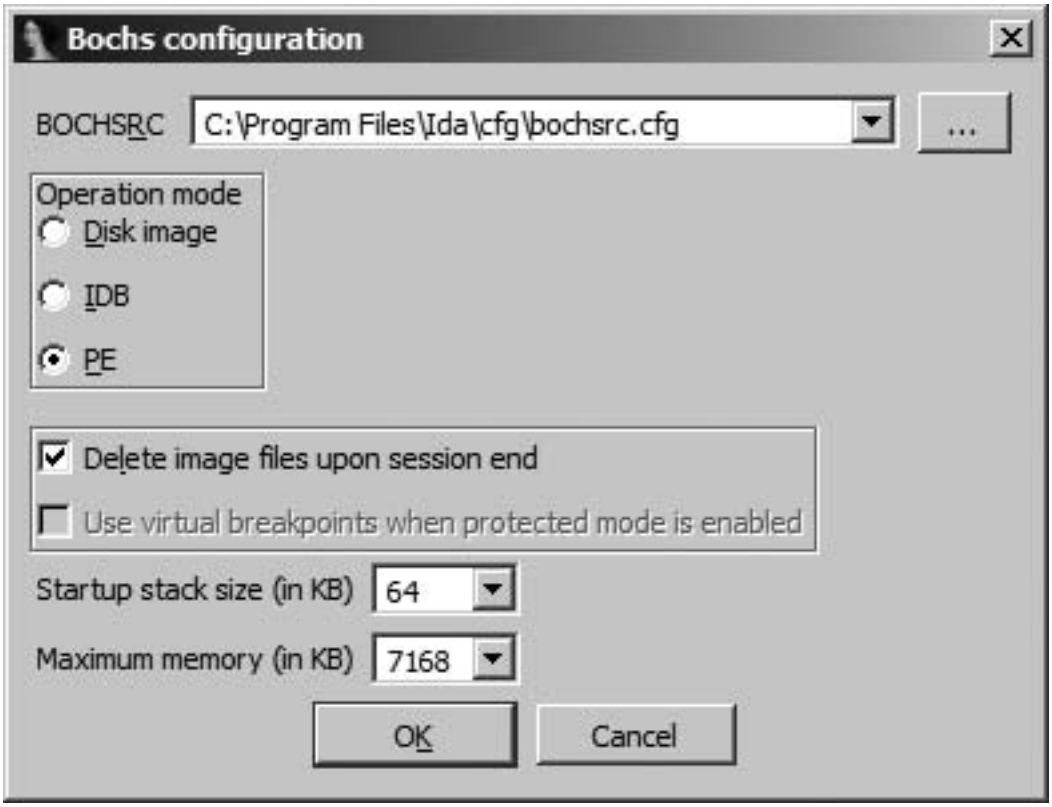


Рисунок 26-4. Параметры отладчика Bochs

Режим IDB

Это самый простой режим. Bochs знает только код и данные из текущей базы: IDA отображает её диапазоны в память эмулятора и копирует байты. Размер стека задаётся в настройках, а место выбирает IDA.

Выполнение начинается с символа `ENTRY`, если он существует. Иначе используется начало выделенного диапазона, а если выделения нет - позиция курсора.

Операционной системы, разделяемых библиотек и стандартных структур процесса в этом режиме нет. Можно выполнять PE, ELF, Mach-O или сырой машинный код, например полезную нагрузку эксплуатации, пока он не обращается за пределы базы. Режим особенно удобен для запуска одной функции без подготовки полного процесса или образа диска.

Режим PE

Этот режим имитирует процесс Windows. Управляющий модуль Bochs действует подобно загрузчику: создаёт PEB, TEB и правдоподобный стек.

Модуль также отображает в адресное пространство распространённые библиотеки Windows, не выполняя их код. Набор задаётся в `<IDADIR>/plugins/bochs/startup.idc`. Библиотеку можно загрузить как есть либо заменить заглушками. Во втором случае все её экспорты перенаправляются в перехватчики Bochs.

Для `foolib.dll` модуль ищет сценарий `api_foolib.idc` или `api_foolib.py` в `<IDADIR>/plugins/bochs`. Поставляемый `api_kernel32.idc` показывает устройство такого файла

и примеры поведения функций.

Перехват необходим, потому что настоящей операционной системы под библиотеками нет. Например, реальная `VirtualAlloc` без ядра завершилась бы неудачей, а сценарная замена может выделить эмулируемую память и вернуть ожидаемый результат. Задача заглушки - убедительно воспроизвести ответы настоящей библиотеки и системы.

В системе не на Windows требуемые DLL копируют с машины Windows, а в `startup.idc` указывают каталог. Пример правила:

```
// Дополнительный путь к DLL.
// Для включения следующих строк добавьте три косые черты.
/// path /home/idauser/xp_dlls/=c:\winnt\system32\
```

При запуске в режиме PE IDA не показывает предупреждение о выполнении вредоносного процесса. Создаётся только процесс Bochs, а исследуемый файл остаётся данными для эмуляции; собственный процесс из него не запускается.

Режим образа диска

Bochs способен эмулировать весь компьютер. Средство `bximage` создаёт образ жёсткого диска, на него через Bochs устанавливается операционная система, а затем гостевая система выполняется целиком.

Для исследования одного процесса этот режим сложен: изолировать его внутри полной системы можно лишь при хорошем знании управления процессами и памятью. Зато удобно разбирать BIOS и загрузочный код до передачи управления операционной системе.

Исполняемого файла для обычной загрузки в IDA здесь нет. Вместо него специальный загрузчик распознаёт файл настройки `bochsrc`. Стандартный вариант находится в `<IDADIR>/cfg/bochsrc.cfg` и задаёт системную BIOS, видеопамять ROM и образы дисков.

Загрузчик IDA читает только первый сектор первого образа, указанного в настройке, и выбирает отладчик Bochs для новой базы. Этого достаточно, чтобы начать исследование главной загрузочной записи.

Appcall

Возможность Appcall позволяет сценарию IDC или IDAPython вызвать любую функцию активного процесса. Так можно отобразить дополнительную память через `VirtualAlloc`, внедрить библиотеку через `LoadLibrary`, попросить собственную функцию программы декодировать данные или вычислить хеш.

Функция должна быть загружена в адресное пространство, а IDA должна знать её прототип, чтобы правильно передать аргументы и получить результат. Вызов выполняется в текущем потоке отладчика. IDA предварительно сохраняет все его регистры, а после завершения восстанавливает состояние, словно вызова не было.

Например, прототип `VirtualAlloc` выглядит так:

```
LPVOID WINAPI VirtualAlloc(
    LPVOID lpAddress,
    SIZE_T dwSize,
    DWORD flAllocationType,
    DWORD flProtect
);
```

Для выделения 4096 байт с разрешением чтения и записи в C использовался бы вызов:

```
VirtualAlloc(NULL, 4096,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_READWRITE);
```

После подстановки констант:

```
VirtualAlloc(0, 4096, 0x3000, 4);
```

В отладчике к имени библиотечной функции добавляется модуль, поэтому символ называется `kernel32_VirtualAlloc`. Библиотека типов не знает такого составного имени и не показывает прототип. Через **Set Function Type** достаточно задать совместимую, пусть и не идеально подробную сигнатуру:

```
int __stdcall kernel32_VirtualAlloc(int, int, int, int);
```

В IDC функция после этого вызывается как обычная функция сценария:

```
IDC>Message("%x\n", kernel32_VirtualAlloc(0, 4096, 0x3000, 4));
3c0000
```

Результат сообщает адрес нового блока размером 4096 байт. Чтобы он появился в окнах IDA, нужно выполнить **Debugger > Refresh memory** либо дождаться очередного автоматического обновления.

В Python используется объект `Appcall` из модуля `idaapi`, но требования к имени и типу сохраняются:

```
Python>Message("%x\n" % Appcall.kernel32_VirtualAlloc(0, 4096, 0x3000, 4))
3d0000
```

Итоги

Отладчик IDA постоянно развивается силами Hex-Rays и сообщества. Он поддерживает локальную и удалённую работу и служит единым внешним интерфейсом для GDB, WinDbg и других средств на разных платформах. Сценарии и двоичные расширения позволяют дополнять его практически без жёсткого предела.

Этой главой завершается книга: дизассемблер и отладчик IDA вместе образуют единую среду для статического и динамического исследования двоичных программ.